

Kubernetes / Cloud Native

Kubernetes、KEP 设计追踪、GitOps、controller/operator、GPU
资源栈、安全和可观测。

164 篇 · 2026-10-04

目录

1. Kubernetes KEP Feature Coverage
2. Kubernetes KEP Implementation Matrix
3. Kubernetes KEPs Design Tracking
4. Kubernetes
5. Agent Runtime / Sandbox 细分选型地图
6. Agent Runtime / Sandbox 项目地图
7. Kubernetes Core / Controller 项目地图
8. Kubernetes DRA Design Deep Dive
9. Kubernetes GPU / Device Stack 项目地图
10. Kubernetes HPA Autoscaling Design
11. Kubernetes In-Place Pod Resize Design
12. Kubernetes Node Runtime Observability Security Design
13. Kubernetes Scheduler Core Design
14. Kubernetes v1.37 Scheduling / Node / DRA Progress
15. Kubernetes Workload and Gang Scheduling Design
16. llm-d / Kubernetes SIGs 候选项目地图
17. OpenKruise 项目候选地图
18. Agent Runtime Substrate
19. Agent Substrate 架构与设计思路分析
20. AgentCube
21. AgentCube 架构与设计思路分析
22. agentgateway
23. agentgateway 架构与设计思路分析
24. AI 运维 (AIOps)
25. AI 驱动的漏洞发现变革
26. AIBrix
27. AIBrix 架构与设计思路分析
28. apiserver-network-proxy

- 29. apiserver-network-proxy 架构与设计思路分析
- 30. Argo CD
- 31. Argo CD 概览
- 32. Cluster API
- 33. Cluster API 架构与设计思路分析
- 34. Container Device Interface
- 35. controller-runtime
- 36. controller-runtime 架构与设计思路分析
- 37. controller-tools
- 38. controller-tools 架构与设计思路分析
- 39. ControllorMesh
- 40. CRI Tools
- 41. CRI Tools 架构与设计思路分析
- 42. Descheduler
- 43. Descheduler 架构与设计思路分析
- 44. DRA Driver for NVIDIA GPUs
- 45. DRA Driver for NVIDIA GPUs 架构与设计思路分析
- 46. Dynamo
- 47. Envoy AI Gateway
- 48. external-dns
- 49. external-dns 架构与设计思路分析
- 50. Gateway API Inference Extension
- 51. Gateway API Inference Extension 架构与设计思路分析
- 52. GitOps
- 53. GPU Sharing
- 54. GPUStack
- 55. HAMi
- 56. HAMi 架构与设计思路分析
- 57. Headlamp
- 58. Headlamp 架构与设计思路分析

- 59. Higress
- 60. Higress 架构与设计思路分析
- 61. inference-perf
- 62. inference-perf 架构与设计思路分析
- 63. ingress2gateway
- 64. ingress2gateway 架构与设计思路分析
- 65. JobSet
- 66. JobSet 架构与设计思路分析
- 67. K3s + k0rdent GitOps 部署 On-Prem 集群
- 68. k8m
- 69. k8m 架构与设计思路分析
- 70. K8s Core & Controllers Star 项目清单整理
- 71. K8s GPU & Device Plugins Star 项目清单整理
- 72. kagent
- 73. kagent 架构与设计思路分析
- 74. Karpenter
- 75. Karpenter 架构与设计思路分析
- 76. kgateway
- 77. kgateway 架构与设计思路分析
- 78. kind
- 79. kind 架构与设计思路分析
- 80. KRO
- 81. KRO 架构与设计思路分析
- 82. Kruise Dashboard
- 83. Kruise Game
- 84. Kruise State Metrics
- 85. Kruise Tools
- 86. KServe
- 87. KServe 架构与设计思路分析
- 88. Kthena

- 89. kube-agentic-networking
- 90. kube-agentic-networking 架构与设计思路分析
- 91. kube-scheduler-simulator
- 92. kube-scheduler-simulator 架构与设计思路分析
- 93. KubeAI
- 94. KubeAI 架构与设计思路分析
- 95. Kubebuilder
- 96. Kubebuilder 架构与设计思路分析
- 97. kubectl-ai
- 98. kubectl-ai 架构与设计思路分析
- 99. Kubernetes Device Plugin
- 100. Kubernetes Dynamic Resource Allocation
- 101. Kubernetes Gateway API
- 102. Kubernetes KEP Design Tracking
- 103. Kubernetes Serving Stack 对比: llm-d、AIBrix、KServe、KubeAI、OME、GPUStack、RBG、Kthena
- 104. Kubernetes v1.36 Sneak Peek
- 105. Kubernetes Workload Automation
- 106. Kubespray
- 107. Kubespray 架构与设计思路分析
- 108. kubewall
- 109. kubewall 架构与设计思路分析
- 110. Kueue
- 111. Kueue 架构与设计思路分析
- 112. Kustomize
- 113. Kustomize 架构与设计思路分析
- 114. KWOK
- 115. KWOK 架构与设计思路分析
- 116. LeaderWorkerSet
- 117. LeaderWorkerSet 架构与设计思路分析
- 118. LLM 网关与路由层对比

- 119. llm-d
- 120. llm-d Planner
- 121. llm-d Router 架构与设计思路分析
- 122. llm-d Workload Variant Autoscaler
- 123. llm-d Workload Variant Autoscaler 架构与设计思路分析
- 124. llm-d 架构与设计思路分析
- 125. llm-d 核心项目群架构再调研
- 126. Local Static Provisioner
- 127. Local Static Provisioner 架构与设计思路分析
- 128. MCP Lifecycle Operator
- 129. MCP Lifecycle Operator 架构与设计思路分析
- 130. metrics-server
- 131. metrics-server 架构与设计思路分析
- 132. Model Serving Operator
- 133. NFS Subdir External Provisioner
- 134. NFS Subdir External Provisioner 架构与设计思路分析
- 135. Node Feature Discovery
- 136. Node Feature Discovery 架构与设计思路分析
- 137. NVIDIA Dynamo 架构与业务问题（文档重构版）
- 138. NVIDIA GPU Operator
- 139. NVIDIA GPU Operator 架构与设计思路分析
- 140. NVIDIA k8s-device-plugin
- 141. NVIDIA k8s-device-plugin 架构与设计思路分析
- 142. OME
- 143. OME 架构与设计思路分析
- 144. OpenKruise Agents
- 145. OpenKruise Agents 架构与设计思路分析
- 146. OpenKruise Kruise
- 147. OpenKruise Rollouts
- 148. OpenKruise 组织项目当前状态核验

- 149. prometheus-adapter
- 150. prometheus-adapter 架构与设计思路分析
- 151. RBG (RoleBasedGroup)
- 152. scheduler-plugins
- 153. scheduler-plugins 架构与设计思路分析
- 154. Secrets Store CSI Driver
- 155. Secrets Store CSI Driver 架构与设计思路分析
- 156. Security Profiles Operator
- 157. Security Profiles Operator 架构与设计思路分析
- 158. sig-storage-lib-external-provisioner
- 159. sig-storage-lib-external-provisioner 架构与设计思路分析
- 160. SkyPilot
- 161. SkyPilot 架构与设计思路分析
- 162. substrate
- 163. 云原生安全
- 164. 用 HolmesGPT 自动诊断 Kubernetes 告警

Kubernetes KEP Feature Coverage

本页回答“到底整理了哪些 feature”。这里的 feature 不是逐个 feature gate，而是可长期追踪的合并设计组：把同一设计问题下的多个 KEP 合并讲，避免后续按 KEP 编号碎片化。

覆盖原则

- 重要 feature 都要入组：P0/P1 必须进入一个合并设计组；P2 只作为历史或边界补充。
- 相关设计合并讲：例如 DRA 不拆成 node/scheduling 两套，而是合并为一个跨 SIG 设计组。
- 总表追踪，矩阵看状态，详解讲设计：[kubernetes-keps-design-tracking](#) 保留完整 KEP 表；[kubernetes-keps-implementation-matrix](#) 追踪 Alpha/Beta/GA、是否实现和 feature gates；本页说明 feature 覆盖；详解页解释为什么这样设计。
- 优先覆盖影响生产系统的问题：调度正确性、容量推理、autoscaling 可靠性、kubelet/runtime 边界、设备资源、可观测、安全和升级回滚。

当前合并设计组

设计组	覆盖 SIG	代表 KEP	详解页	覆盖状态
Scheduler core / queue / placement	scheduling	624, 785, 1451, 3521, 4247, 6132, 5598, 895, 1258, 3022, 3094, 4832, 5229	kubernetes-scheduler-core-design	已详解
Workload / gang / workload-aware scheduling	scheduling	4671, 5710, 6012, 6089, 5732, 5832, 583	kubernetes-workload-gang-scheduling-design	已详解
DRA / device resource model	node + scheduling + autoscaling	4381, 3063, 5007, 5075, 5517, 5729, 5941, 5963, 6080, 5055, 4815, 4816, 5030	kubernetes-dra-design-deep-dive	已详解
HPA / metrics / scale from zero	autoscaling	4951, 853, 2702, 1610, 117, 5325, 2021, 5679, 5030	kubernetes-hpa-autoscaling-design	已详解
In-place Pod resize / resource mutation	node + scheduling	1287, 5419, 5526, 5554, 6122, 5836	kubernetes-in-place-pod-resize-design	已详解
Node runtime / resource managers / observability / security	node	2040, 2221, 585, 3570, 1769, 693, 753, 127, 2033, 2371, 4205, 5394, 4680, 5328	kubernetes-node-runtime-observability-security-design	已详解

实现状态、Alpha/Beta/GA、feature gates 和关键实现路径统一见 [kubernetes-keps-implementation-matrix](#)。这里的“已详解”只表示已经有设计讲解页，不等于所有 KEP 都已经 GA。

2026-07-15 Upstream Refresh

最新 kubernetes/enhancements 复核见 [k8s-v1.37-scheduling-node-dra-progress](#)。本轮没有新增合并设计组，但 v1.37 的重心更清楚：

方向	近期状态	影响
Workload-aware scheduling	4671、5732 进入 beta；6089、6012 进入 alpha	Workload / PodGroup 不再只是历史 coscheduling 替代，而是在 API、拓扑、controller building blocks 上成组推进。
Scheduler queue / batching	6132 beta，5598 继续扩展 rescoring	gang、DRA、resize 等等待型调度场景需要更精准的事件唤醒和批量处理。
DRA 设备资源模型	5729、5075 beta；5941、5963、5517 alpha；4817、6072 stable	DRA 从基础 claim/slice 走向 workload 级、capacity 型、NUMA/topology-aware 的设备资源语义。
Node resource management	5526、2570、2033 beta；6122、5823、4563 alpha	Pod scope 资源管理、Memory QoS、rootless kubelet 和 checkpoint/eviction API 补齐节点执行面。
Autoscaling 联动	4951 stable，2021 / 5030 beta	autoscaler 必须理解调度约束、指标失败和从 0 恢复，不再只是副本数算法。

SIG Scheduling Feature 覆盖

Feature	合并到	重要性
Scheduler Framework	kubernetes-scheduler-core-design	所有后续调度 KEP 的 plugin 扩展底座。
Scheduler ComponentConfig / profiles	kubernetes-scheduler-core-design	让调度策略可配置、可多 profile 共存。
Pod scheduling readiness / queueing hints / prequeueing hints	kubernetes-scheduler-core-design	减少无效调度和 pending pod 抖动，是 DRA、gang、resize 场景的性能基础。
Opportunistic batching / async API calls	kubernetes-scheduler-core-design	面向大规模集群和高吞吐调度。

Feature	合并到	重要性
Pod topology spread / affinity extension	kubernetes-scheduler-core-design	普通 workload placement 的稳定基础。
Async preemption / nominated node expectation	kubernetes-scheduler-core-design	降低抢占阻塞和状态不一致。
Workload / PodGroup / gang scheduling	kubernetes-workload-gang-scheduling-design	AI/HPC/batch 的核心调度语义。
Workload-aware preemption	kubernetes-workload-gang-scheduling-design	让抢占从单 Pod 扩展到 workload 级。
Topology-aware workload scheduling	kubernetes-workload-gang-scheduling-design	让一组 Pod 作为整体选择 topology placement。
DRA-aware scheduling	kubernetes-dra-design-deep-dive	让设备资源进入 scheduler 可推理路径。
Scheduler preemption for in-place resize	kubernetes-in-place-pod-resize-design	让已运行 Pod 的 resize 也能触发调度层资源重排。

SIG Autoscaling Feature 覆盖

Feature	合并到	重要性
HPA v2 / behavior policies	kubernetes-hpa-autoscaling-design	HPA API 稳定底座。
Configurable HPA tolerance	kubernetes-hpa-autoscaling-design	把全局 10% tolerance 下放到 workload 和方向级别。
Container resource autoscaling	kubernetes-hpa-autoscaling-design	避免 sidecar 稀释主容器指标。
Metrics specificity / label selector	kubernetes-hpa-autoscaling-design	让 custom/external metrics 精确匹配目标。
Pod selection accuracy	kubernetes-hpa-autoscaling-design	防止 HPA 把同 label 但非同 owner 的 Pod 算进分母。
Scale from zero	kubernetes-hpa-autoscaling-design	让高成本 workload 能基于 object/external metrics 缩到 0 再拉起。
External metric fallback	kubernetes-hpa-autoscaling-design	降低外部指标系统故障对扩缩容的影响。

Feature	合并到	重要性
Volume attach limit autoscaler	kubernetes-hpa-autoscaling-design , kubernetes-dra-design-deep-dive	代表 autoscaler 必须理解 scheduler 约束，而不只是加节点。

SIG Node Feature 覆盖

Feature	合并到	重要性
kubelet / CRI / dockershim removal / RuntimeClass	kubernetes-node-runtime-observability-security-design	kubelet 与 runtime 解耦的基础。
CPU / Memory / Topology Manager	kubernetes-node-runtime-observability-security-design	性能型 workload 的资源隔离和 NUMA 对齐基础。
Pod-level resources	kubernetes-in-place-pod-resize-design , kubernetes-node-runtime-observability-security-design	从 container resource 走向 pod scope resource 管理。
Device Plugin / CDI / PodResources API	kubernetes-dra-design-deep-dive , kubernetes-node-runtime-observability-security-design	连接传统设备插件和下一代 DRA。
DRA structured parameters / ResourceClaim status	kubernetes-dra-design-deep-dive	设备资源标准化主线。
In-place resource resize	kubernetes-in-place-pod-resize-design	Pod resource 从 immutable spec 变成 desired/actual 状态机。
Sidecar / restart policy / graceful shutdown	kubernetes-node-runtime-observability-security-design	Pod lifecycle 语义补全，影响 Job、服务网格和 agent workload。
User namespaces / rootless kubelet / seccomp / cgroup v2	kubernetes-node-runtime-observability-security-design	节点隔离和最小权限方向。
CRI stats / PSI / resource health / node declared features	kubernetes-node-runtime-observability-security-design	可观测和自动化调度/修复的输入信号。

还没有单独展开的补充项

这些不是漏掉，而是归入补充或后续低优先级：

类别	KEP / feature	当前处理
历史 extender / coscheduling	1819, 583	作为 scheduler framework 和 Workload/ Gang 的历史对照。
ResourceQuota / node labels quota	986, 2372	放在 P2，除非后续要专门整理 quota/ capacity governance。
probe/log 细节	gRPC probe、CRI log rotation、 stdout/stderr split	放在 Node lifecycle 补充，不作为第一批 详解。
storage/resource 小项	hugepages、ephemeral storage、 memory-backed volumes	暂归 P2；如后续做 storage/resource 管 理专题再拉出。
node config	kubelet drop-in config、dynamic kubelet config removed	放在 Node runtime 页的边界说明。

后续维护动作

1. 新增 KEP 先落到本页的合并设计组，再进入 [kubernetes-keps-design-tracking](https://github.com/kubernetes/keps-design-tracking) 的 SIG 表。
2. 如果一个 KEP 同时影响多个设计组，例如 DRA + autoscaler + scheduler，要在两个详解页互链，而不是复制两份解释。
3. 如果某组超过 20 个活跃 KEP，再拆二级页；否则保持合并讲解。
4. 下一轮应补 prod-readiness 数据，把每个设计组的 feature gates、metrics、rollback、upgrade/downgrade 汇总到本页。

Kubernetes KEP Implementation Matrix

本页补齐“每个重要 KEP 到底有没有实现、现在是 alpha / beta / GA、关键 feature gate 是什么、核心实现或设计是什么”。它和 [kubernetes-keps-feature-coverage](#) 的关系是：覆盖矩阵按 feature group 合并阅读，本页按 KEP 逐项追踪实现状态。

状态判读

字段	含义
implemented + stable	KEP metadata 已标实现，并且成熟度是 stable；可视为已实现 / GA。
implementable + stable	KEP 已可实现，目标成熟度为 stable；通常表示 GA 设计线已完成或进入 GA 目标，但 metadata 还未改成 implemented。
implementable + beta	设计可实现，处于 beta；默认可试用但仍要关注 feature gate、升级和指标。
implementable + alpha	设计可实现但仍是 alpha；生产采用需要明确开关、回滚和兼容性风险。
provisional	设计仍在早期，不能按落地能力依赖。
withdrawn / removed	设计已撤回或移除，只作为历史对照。

P0 Landmark KEPs

这些是当前最重要、最应该逐个追踪的 KEP。著名度不是社区 star，而是这里的阅读优先级：是否改变核心 API、是否跨 SIG、是否影响调度/弹性/节点生产路径。

2026-07-15 已用 GitHub API 复核 upstream kubernetes/enhancements master 中活跃 KEP 的 status、stage、latest-milestone 和 Alpha/Beta/GA milestones。v1.37 汇总页见 [k8s-v1.37-scheduling-node-dra-progress](#)。本页仍以 KEP 元数据为准；是否真正可生产采用还要继续核验 kubernetes/kubernetes 实现 PR、feature gate 默认值和 release notes。

KEP	SIG	著名度	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
4671-gang-scheduling	scheduling	Landmark	implementable / beta	v1.35 / v1.37 / v1.38	GenericWorkload	引入 Workload / PodGroup, scheduler 以组为单位做准入、permit、binding 和失败回退。
5710-workload-aware-preemption	scheduling	Landmark	implementable / beta	v1.36 / v1.37 / v1.39	GenericWorkload	抢占从 Pod 级扩展到 PodGroup / workload 级, 避免抢占后仍不能整体运行。
6012-composite-podgroup-api	scheduling	Important	implementable / alpha	v1.37 / v1.38 / v1.40	CompositePodGroup	表达多组件 workload 的组合 PodGroup, 是 JobSet/LWS/训练任务接入 gang 语义的 API 补充。
6089-was-controller-apis	scheduling	Important	implementable / alpha	v1.37 / v1.38 / v1.39	WorkloadWithJob	给 controller 提供 workload-aware scheduling building blocks, 而不是强制所有 workload 使用同一顶层 API。
6132-prequeueing-hints	scheduling	Important	implementable / beta	- / v1.37 / v1.39	SchedulerPreQueueingHints	在 Pod 入 activeQ 前判断事件是否值得唤醒, 降低 DRA/gang/resize 等等待型场景的无效重试。
5598-opportunistic-batching	scheduling	Important	implementable / beta	- / v1.35 / v1.38	Opportunistic Batching	scheduler 批量处理可调度机会, 提高大规模集群吞吐。
4381-dra-structured-parameters	node + scheduling +	Landmark	implemented /	v1.30 / v1.32	DynamicResourceAllocation, DRASchedulerFilterTimeout	DRA 主线: ResourceSlice/ResourceClaim 结构化设

KEP	SIG	著名度	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
	autoscaling		stable	/ v1.34		备参数，让 scheduler/ autoscaler 可推理设备。
3063-dynamic-resource-allocation	node	Historical	withdrawn / alpha	v1.26 / - / -	DynamicResourceAllocation, DRAControlPlaneController	早期 opaque DRA controller allocation 路线，已撤回；主要价值是解释为什么转向 structured parameters。
5075-dra-consumable-capacity	scheduling	Landmark	implementable / beta	v1.34 / v1.36 / v1.38	DRAConsumableCapacity	设备不再只是离散实例，也可以有可消费 capacity，例如 GPU memory、带宽、license。
5517-dra-node-allocatable-resources	scheduling	Important	implementable / alpha	v1.36 / v1.38 / v1.39	DRANodeAllocatableResources	暴露 DRA node allocatable，给调度和容量推理提供节点级库存信号。
5729-resourceclaim-support-for-workloads	scheduling	Important	implementable / beta	v1.36 / v1.37 / -	DRAWorkloadResourceClaims	ResourceClaim 支持 workload 级使用，连接 DRA 和 PodGroup/gang scheduling。
5941-dra-shared-consumable-capacity	scheduling	Important	implementable / alpha	v1.37 / v1.38 / v1.39	DRASharedConsumableCapacity	多个 logical device 共享同一个 capacity pool，处理共享 buffer、memory 或 fabric bandwidth。
5963-device-compatibility-groups	scheduling	Important	implementable / alpha	v1.37 / v1.38 / v1.39	DRADeviceCompatibilityGroups	表达多个设备必须兼容或来自同一组合集合，适合多卡/多 NIC 配对。
	scheduling	Watch	provisional	v1.37 /	DRADerivedAttributes	

KEP	SIG	著名度	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
6080-dra-derived-attributes			1 / alpha	v1.38 / v1.40		从 driver 属性派生标准化匹配属性；还不是稳定实现主线。
4817-resource-claim-device-status	node	Important	implementable / stable	v1.32 / v1.33 / v1.37	DRAResourceClaimDeviceStatus	将分配到的设备信息写入 ResourceClaim status，给用户和控制器可观测结果。
5304-dra-attributes-downward-api	node	Important	implementable / beta	v1.36 / v1.37 / v1.38	NA	把 DRA device attributes 暴露给 Pod，方便 workload 发现自己拿到的设备属性。
5677-dra-resource-availability-visibility	node	Important	implementable / alpha	v1.36 / - / -	DRAResourcePoolStatus	暴露资源池可用性，减少用户只看到 claim pending 却不知道设备库存原因。
5945-dra-optional-node-preparation	node	Important	implementable / alpha	v1.37 / v1.38 / v1.39	DRAOptionalNodePreparation	允许某些 DRA 场景跳过强制 NodePrepare，降低轻量设备或无需节点准备的路径成本。
6072-dra-standard-numanode	node	Important	implementable / stable	- / - / v1.37	-	标准化 DRA numaNode 属性，连接 DRA 和 Topology Manager / NUMA-aware scheduling。
4951-configurable-hpa-tolerance	autoscaling	Important	implementable / stable	v1.33 / v1.35 / v1.37	HPAConfigurableTolerance	把 HPA tolerance 从全局参数下放到 HPA scaleUp/scaleDown 规则。

KEP	SIG	著名度	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
1610-container-resource-autoscaling	autoscaling	Important	implemented / stable	v1.20 / v1.27 / v1.30	HPAContainerMetrics	HPA 可按指定 container 的 CPU/memory 扩缩，避免 sidecar 稀释主容器指标。
5325-hpa-pod-selection-accuracy	autoscaling	Important	implementable / alpha	v1.35 / v1.36 / v1.37	HPASelectionStrategy	HPA 可按 owner reference 过滤 Pod，避免 label selector 误选其他 workload。
2021-scale-from-zero	autoscaling	Landmark	implementable / beta	v1.16 / v1.37 / x.y	HPAScaleToZero	HPA 基于 object/external metrics 支持 0 副本恢复，用 condition 区分自动缩零与用户暂停。
5679-external-metric-fallback	autoscaling	Important	implementable / alpha	v1.36 / v1.37 / v1.38	HPAExternalMetricFallback	external metric 连续失败时使用固定 fallback replicas，并进入 HPA 多指标 max 合并。
5030-attach-limit-autoscaler	autoscaling	Important	implementable / beta	v1.35 / v1.37 / v1.38	VolumeLimitScaling	Cluster Autoscaler 模拟 CSI volume attach limit，避免加了节点仍不可调度。
1287-in-place-update-pod-resources	node	Landmark	implemented / stable	v1.27 / v1.33 / v1.35	InPlacePodVerticalScaling, InPlacePodVerticalScalingAlllocatedStatus	Pod /resize subresource 与 desired/allocated/actual 状态模型，支持 CPU/memory 原地调整。
5419-pod-level-	node	Important	implementable	v1.35 /	InPlacePodLevelResourcesVe	将 in-place resize 扩展到 Pod-level resources。

KEP	SIG	著名度	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
resources-in-place-resize		important	beta	v1.36 / -	VerticalScaling, PodLevelResources	
5526-pod-level-resource-managers	node	Important	implementable / beta	v1.36 / v1.37 / v1.39	PodLevelResources, PodLevelResourceManagers	CPU/Memory/Topology Manager 从 container scope 扩展到 Pod scope。
5554-in-place-update-pod-resources-alongside-static-cpu-manager-policy	node	Important	implementable / alpha	v1.37 / v1.38 / v1.39	InPlacePodVerticalScalingExclusiveCPUs	处理 static CPU Manager exclusive CPUs 下的原地 resize。
6122-configurable-scaling-delay-with-pod-resource-exposure	node	Important	implementable / alpha	v1.37 / v1.39 / v1.40	DownwardAPIAsSignedResources 等	在收回 exclusive CPUs 前暴露资源变化并给应用准备窗口。
5836-scheduler-preemption-for-ippr	scheduling	Important	implementable / alpha	v1.37 / v1.38 / v1.39	SchedulerPreemptionForPodResize	scheduler 为 deferred in-place resize 在已绑定节点上触发抢占。
2371-cri-pod-container-stats	node	Important	implementable / beta	v1.29 / v1.37 / -	PodAndContainerStatsFromCRI	kubelet stats 从 cAdvisor 迁往 CRI，影响 metrics-server/HPA 的输入可靠性。
	node					

KEP	SIG	著名度	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
5394-psi-node-conditions		Important	implementable / alpha	v1.36 / - / -	PSINodeCondition	将 PSI pressure 升级为 Node Conditions，便于调度、排障和自动化响应。
4680-add-resource-health-to-pod-status	node	Important	implementable / beta	v1.31 / v1.36 / v1.37	ResourceHealthStatus	Device Plugin / DRA 资源健康进入 Pod status。
2033-kubelet-in-usersns-aka-rootless	node	Important	implementable / beta	v1.22 / v1.37 / -	KubeletInUserNamespace	kubelet 以 user namespace / rootless 模式运行，降低节点控制面特权。
5607-hostnetwork-usersns	node	Important	implementable / alpha	v1.35 / - / -	UserNamespaceHostNetworkSupport	HostNetwork Pod 也可使用 user namespaces，补上高权限网络场景隔离。
4438-container-restart-termination	node	Important	implementable / alpha	v1.37 / v1.38 / v1.39	SidecarsRestartableDuringPodTermination	明确 Pod 终止期间 sidecar 是否可 restart，修补 lifecycle 边界。
4563-eviction-request-api	node	Important	implementable / alpha	v1.37 / - / -	EvictionRequestAPI	给 eviction 请求更清晰的 API 表达。

P1 Foundation KEPs

这些 KEP 是底座。它们不一定都需要单独长文，但阅读 P0 时经常要回到这里确认前提。

KEP	SIG	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
624-scheduling-framework	scheduling	implemented / stable	v1.16 / - / v1.19	-	kube-scheduler extension points, 后续 DRA、gang、queuing、preemption 都依赖它。
785-scheduler-component-config-api	scheduling	implemented / stable	- / v1.19 / v1.25	-	scheduler config 版本化。
1451-multi-scheduling-profiles	scheduling	implementable / beta	v1.18 / v1.19 / v1.22	-	一个 scheduler 实例支持多个 profile。
3521-pod-scheduling-readiness	scheduling	implemented / stable	v1.26 / v1.27 / v1.30	PodSchedulingReadiness	scheduling gates 控制 Pod 何时进入调度。
4247-queueinghint	scheduling	implemented / stable	v1.26 / v1.32 / v1.34	SchedulerQueueingHints	plugin-specific requeue hint。
895-pod-topology-spread	scheduling	implemented / stable	v1.16 / v1.18 / v1.19	EvenPodsSpread	Pod 副本按 topology domain 分散。
3633-matchlabelkeys-to-podaffinity	scheduling	implemented / stable	v1.29 / v1.31 / v1.33	MatchLabelKeysInPodAffinity	affinity/anti-affinity 支持 matchLabelKeys / mismatchLabelKeys。
4832-async-preemption	scheduling	implementable / beta	v1.32 / v1.33 / -	SchedulerAsyncPreemption	抢占异步化, 减少 scheduling cycle 阻塞。
5007-device-attach-before-pod-scheduled	scheduling	implementable / beta	v1.34 / v1.36 / v1.37	DRADeviceBindingConditions	设备 attach / readiness 条件进入调度等待路径。
5055-dra-device-taints-and-tolerations	scheduling	implementable / stable	v1.33 / v1.36 / v1.37	DRADeviceTaints,	设备级 taints/tolerations。

KEP	SIG	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
				DRADeviceTaintRules	
4815-dra-partitionable-devices	scheduling	implementable / beta	v1.33 / v1.36 / -	DRAPartitionableDevices	动态或逻辑分区设备。
4816-dra-prioritized-list	scheduling	implementable / stable	v1.33 / v1.34 / v1.36	DRAPrioritizedList	用户可表达设备请求的优先级备选列表。
5732-topology-aware-workload-scheduling	scheduling	implementable / beta	v1.36 / v1.37 / v1.39	TopologyAwareWorkloadScheduling	PodGroup / workload 级 topology placement。
5229-asynchronous-api-calls-during-scheduling	scheduling	implementable / beta	- / v1.34 / -	SchedulerAsyncAPICalls	调度期间 API 写操作异步化。
853-configurable-hpa-scale-velocity	autoscaling	implemented / stable	- / - / -	-	HPA behavior policies、stabilization window、scale velocity。
2702-graduate-hpa-api-to-GA	autoscaling	implemented / stable	- / - / v1.23	-	HPA v2 API GA。
117-hpa-metrics-specificity	autoscaling	implemented / stable	- / - / -	-	custom/external metrics label selector。
3570-cpumanager	node	implemented / stable	v1.8 / v1.10 / v1.26	CPUManager	Guaranteed workload 的 cpuset 分配。
1769-memory-manager	node			MemoryManager	NUMA-aware memory allocation。

KEP	SIG	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
		implemented / stable	v1.21 / v1.22 / v1.32		
693-topology-manager	node	implemented / stable	v1.16 / v1.18 / v1.27	TopologyManager	汇总 CPU/memory/device hints。
2837-pod-level-resource-spec	node	implementable / stable	v1.33 / v1.34 / v1.37	PodLevelResources	Pod-level resource spec, 是 pod-scope resource managers 的 API 前提。
3573-device-plugin	node	implemented / stable	v1.8 / v1.10 / v1.26	DevicePlugins	传统设备插件 API。
4009-add-cdi-devices-to-device-plugin-api	node	implemented / stable	v1.28 / v1.29 / v1.31	DevicePluginCDIDevices	device plugin 返回 CDI devices。
3695-pod-resources-for-dra	node	implemented / stable	v1.27 / v1.34 / v1.36	KubeletPodResourcesDynamicResource	PodResources API 包含 DRA allocation。
2040-kubelet-cri	node	implementable / beta	v1.5 / v1.23 / -	-	kubelet/runtime 通过 CRI 解耦。
2221-remove-dockershim	node	implemented / stable	- / - / -	-	kubelet 移除内置 dockershim。
585-runtime-class	node	implemented / stable	v1.12 / v1.14 / v1.20	RuntimeClass	Pod 选择 runtime handler。
753-sidecar-containers	node	implemented / stable	v1.28 / v1.29 / v1.33	SidecarContainers	用 restartable init container 表达 sidecar lifecycle。

KEP	SIG	状态	Alpha / Beta / GA	Feature gate	关键实现 / 设计
127-user-namespaces	node	implemented / stable	v1.25 / v1.35 / v1.36	UserNamespacesSupport	Pod user namespace 隔离。
2413-seccomp-by-default	node	implementable / stable	v1.22 / v1.25 / v1.27	SeccompDefault	默认启用 seccomp runtime default。
2862-fine-grained-kubelet-authz	node	implementable / stable	v1.32 / v1.33 / v1.36	KubeletFineGrainedAuthz	kubelet API 细粒度授权。
2254-cgroup-v2	node	implementable / stable	v1.18 / v1.22 / v1.25	-	cgroup v2 支撑现代资源控制。
727-resource-metrics-endpoint	node	implemented / stable	v1.14 / - / v1.29	-	kubelet /metrics/resource 稳定入口。
4205-psi-metric	node	implemented / stable	v1.33 / v1.34 / v1.36	KubeletPSI	kubelet 暴露 PSI metrics。
5328-node-declared-features	node	implementable / stable	v1.35 / v1.36 / v1.37	NodeDeclaredFeatures	节点声明能力，供调度/控制器使用。

设计阅读索引

想理解的问题	先读
为什么 gang scheduling 不是普通 Pod 队列插件	kubernetes-workload-gang-scheduling-design
为什么 DRA 需要 structured parameters	kubernetes-dra-design-deep-dive
HPA 为什么需要 tolerance、owner filtering、fallback	kubernetes-hpa-autoscaling-design
/resize 为什么要拆 desired/allocated/actual	kubernetes-in-place-pod-resize-design

想理解的问题	先读
kubelet/CRI/resource managers/安全/可观测如何连接	kubernetes-node-runtime-observability-security-design
scheduler framework、queue、preemption 的底座	kubernetes-scheduler-core-design

后续追踪规则

1. 新 KEP 进入本页时，必须记录 status、stage、latest-milestone、Alpha/Beta/GA milestones 和 feature gate。
2. implementable + stable 不自动等同于 metadata 已标 implemented；需要继续看 KEP implementation history 和 release note。
3. provisional 和 withdrawn 不放入生产能力清单，只用于设计脉络解释。
4. 若一个 KEP 同时影响多个 SIG，例如 DRA 和 autoscaler，要在对应设计页互链并说明边界。

Kubernetes KEPs Design Tracking

这页是 KEP 设计方案追踪表，服务于后续长期更新。源摘要和设计脉络见 [src-kubernetes-keps-design-tracking](#)；按 feature 合并后的覆盖矩阵见 [kubernetes-keps-feature-coverage](#)；逐个 KEP 的 Alpha/Beta/GA、是否实现、feature gates 和关键实现路径见 [kubernetes-keps-implementation-matrix](#)。

使用方式

- 先按 SIG 找到 KEP。
- 再看 分类 判断它属于哪条设计线。
- 优先级 用于决定后续是否需要单独深挖：
- P0：近期活跃、跨 SIG 影响大、或与 AI/HPC/GPU/平台工程强相关。
- P1：稳定基础设计，适合作为背景知识或项目对照。
- P2：历史、边缘或低频补充。

重要设计文档详解

设计文档	覆盖 KEP	重点
kubernetes-keps-feature-coverage	all P0/P1 groups	回答“整理了哪些 feature”，按合并设计组列出 scheduling / autoscaling / node 的覆盖状态。
kubernetes-keps-implementation-matrix	all P0/P1 KEPs	逐个 KEP 追踪 status、stage、Alpha/Beta/GA milestones、feature gate、是否落地和关键实现路径。
kubernetes-scheduler-core-design	624, 785, 1451, 3521, 4247, 6132, 5598, 895, 4832	Scheduler framework、profiles、queue/requeue、topology placement、async preemption 和调度性能底座。
kubernetes-workload-gang-scheduling-design	4671, 5710, 6012, 6089, 5732	Workload / PodGroup API、gang scheduling、workload-aware preemption、topology-aware workload scheduling 和 controller API building blocks。
kubernetes-dra-design-deep-dive	4381, 3063, 5007, 5075, 4815, 4816, 5055	DRA structured parameters、ResourceSlice/ ResourceClaim、scheduler plugin、kubelet plugin、CDI、autoscaler 可推理性和设备扩展 KEP。
kubernetes-hpa-autoscaling-design	4951, 2021, 5679, 1610, 5325, 5030	HPA tolerance、container resource metrics、pod selection accuracy、scale from zero、external metric fallback 和 attach-limit autoscaler。

设计文档	覆盖 KEP	重点
kubernetes-in-place-pod-resize-design	1287, 5419, 5526, 5554, 6122, 5836	Pod /resize subresource、desired/allocated/actuated/actual 状态、Pod-level resources、static CPU manager、scheduler preemption for deferred resize。
kubernetes-node-runtime-observability-security-design	2040, 2221, 585, 3570, 1769, 693, 753, 127, 2033, 2371, 5394, 4680	kubelet/CRI、resource managers、sidecar lifecycle、user namespace/rootless kubelet、CRI stats、PSI 和 resource health。

SIG Scheduling

P0 活跃设计线

分类	KEP	状态 / 阶段	设计方案	优先级
workload/gang	4671-gang-scheduling	implementable / beta / v1.37	用 Workload/PodGroup 表达整组 Pod 调度，解决 minCount、组装、整体可行性、preemption、binding 和失败回退。	P0
workload/gang	5710-workload-aware-preemption	implementable / beta / v1.37	抢占从单 Pod 扩展到 workload 级，避免高优先级 gang 抢占后仍无法整体运行。	P0
workload/gang	6012-composite-podgroup-api	implementable / alpha / v1.37	表达组合 PodGroup，适合多组件 workload。	P0
workload/gang	6089-was-controller-apis	implementable / alpha / v1.37	Workload Aware Scheduling controller API，给 controller 暴露 workload-aware 调度状态。	P0
queue	6132-prequeueing-hints	implementable / beta / v1.37	在 Pod 入 activeQ 前通过 hint 判断事件是否足以唤醒调度。	P0
queue	5598-opportunistic-batching	implementable / beta / v1.37	批量化调度机会，降低重复计算，提高吞吐。	P0
				P0

分类	KEP	状态 / 阶段	设计方案	优先级
DRA scheduling	5075-dra-consumable-capacity	implementable / beta / v1.37	DRA 支持可消费容量，不再只处理离散设备实例。	
DRA scheduling	5517-dra-node-allocatable-resources	implementable / alpha / v1.37	暴露 DRA node allocatable，支撑调度和容量推理。	P0
DRA scheduling	5729-resourceclaim-support-for-workloads	implementable / beta / v1.37	ResourceClaim 支持 workload 级使用，和 PodGroup/gang 交叉。	P0
DRA scheduling	5941-dra-shared-consumable-capacity	implementable / alpha / v1.37	多设备共享同一 consumable capacity。	P0
DRA scheduling	5963-device-compatibility-groups	implementable / alpha / v1.37	设备兼容组，表达多设备组合约束。	P0
DRA scheduling	6080-dra-derived-attributes	provisional / alpha / v1.37	从设备属性派生拓扑/匹配属性，解决 driver 属性表达不一致。	P0
resize/preemption	5836-scheduler-preemption-for-ipp	implementable / alpha / v1.37	scheduler 处理 in-place pod resize 引发的资源冲突和抢占。	P0

P1 基础设计线

分类	KEP	状态 / 阶段	设计方案	优先级
framework	624-scheduling-framework	implemented / stable / v1.19	scheduler plugin 扩展点体系，是后续所有调度 KEP 的底座。	P1
config	785-scheduler-component-config-api	implemented / stable / v1.25	versioned scheduler ComponentConfig。	P1

分类	KEP	状态 / 阶段	设计方案	优先级
config	1451-multi-scheduling-profiles	implementable / beta / v1.19	一个 scheduler 实例支持多个 profile。	P1
queue	3521-pod-scheduling-readiness	implemented / stable / v1.30	scheduling gates 控制 Pod 是否进入调度。	P1
queue	4247-queueinghint	implemented / stable / v1.34	plugin-specific QueueingHint, 减少无效 requeue。	P1
queue	5142-pop-backoffq-when-activeq-empty	implementable / beta / v1.33	activeQ 空时提前处理 backoffQ。	P1
queue	5501-reflect-preenqueue-rejections-in-pod-status	implementable / beta / v1.35	PreEnqueue 拒绝原因进入 Pod status。	P1
topology	895-pod-topology-spread	implemented / stable / v1.19	Pod 拓扑分布基础设计。	P1
topology	1258-default-pod-topology-spread	implementable / stable / v1.24	默认拓扑分布策略。	P1
topology	3022-min-domains-in-pod-topology-spread	implemented / stable / v1.30	PodTopologySpread 最小 domain。	P1
topology	3094-pod-topology-spread-considering-taints	implemented / stable / v1.33	skew 计算考虑 taints/ tolerations。	P1
topology	3243-respect-pod-topology-spread-after-rolling-upgrades	implementable / beta / v1.34	rolling upgrade 后继续尊重 spread。	P1
affinity	2249-pod-affinity-namespace-selector	implementable / stable / v1.24	Pod affinity 支持 namespace selector。	P1
affinity	3633-matchlabelkeys-to-podaffinity	implemented / stable / v1.33	affinity/anti-affinity 支持 matchLabelKeys。	P1
scoring	2458-node-resource-score-strategy	implementable / beta / v1.22	节点资源打分策略可配置。	P1
preemption	4832-async-preemption	implementable / beta / v1.33	异步化 preemption。	P1
				P1

分类	KEP	状态 / 阶段	设计方案	优先级
preemption	5278-nominated-node-name-for-expectation	implementable / beta / v1.35	改善 nominated node 期望表达。	
DRA scheduling	5007-device-attach-before-pod-scheduled	implementable / beta / v1.36	设备绑定条件进入调度路径。	P1
DRA scheduling	5004-dra-extended-resource	implementable / stable / v1.37	Extended resources 通过 DRA driver 处理。	P1
DRA scheduling	5055-dra-device-taints-and-tolerations	implementable / stable / v1.37	设备级 taints/tolerations。	P1
DRA scheduling	4815-dra-partitionable-devices	implementable / beta / v1.36	可分区设备。	P1
DRA scheduling	4816-dra-prioritized-list	implementable / stable / v1.36	设备请求优先级备选列表。	P1
DRA scheduling	5234-dra-resourceslice-mixins	implementable / alpha / v1.34	ResourceSlice mixins。	P1
DRA scheduling	5491-dra-list-types-for-attributes	implementable / alpha / v1.37	DRA 属性支持 list 类型。	P1
placement	5732-topology-aware-workload-scheduling	implementable / beta / v1.37	workload 级拓扑感知调度。	P1
scheduler/API	5229-asynchronous-api-calls-during-scheduling	implementable / beta / v1.34	调度期间 API 调用异步化。	P1

P2 历史和补充

分类	KEP	状态 / 阶段	设计方案	优先级
extender	1819-scheduler-extender	implemented / stable / v1.19	HTTP extender 历史扩展路径。	P2
priority	268-priority-preemption	implementable / alpha	Priority/preemption 基础线。	P2
priority	902-non-preempting-priorityclass	implemented / stable / v1.24	高优先级但不抢占。	P2
preemption	3280-guarantee-pdb-when-preemption-happens	implementable / alpha / v1.27	抢占时保护 PDB。	P2
job	2926-job-mutable-scheduling-directives	implemented / stable / v1.27	Job scheduling directives 可变。	P2
pod	3838-pod-mutable-scheduling-directives	implementable / stable / v1.30	Pod scheduling directives 可变。	P2
taint	382-taint-node-by-condition	implemented / stable	Node condition taint。	P2
taint	3902-decoupled-taint-manager	implemented / stable / v1.34	TaintManager 从 NodeLifecycleController 解耦。	P2
daemons et	548-schedule-daemonset-pods	implemented / stable	DaemonSet pods 交给 scheduler。	P2
quota	986-resource-quota-scope-selectors	implemented / stable	ResourceQuota scope selectors。	P2
quota	2372-node-labels-quota	provisional	基于 node labels 的 ResourceQuota。	P2
score	964-binpacking-priority	implementable / alpha	RequestedToCapacityRatio 支持 extended resource binpacking。	P2
historical	583-coscheduling	provisional	gang scheduling 前身。	P2
historical	5832-decouple-podgroup-api	implementable / alpha / v1.36	PodGroup API 解耦。	P2
config	2891-simplified-config		scheduler config 简化。	P2

分类	KEP	状态 / 阶段	设计方案	优先级
		implementable / beta / v1.23		
placement	5471-enable-sla-based-scheduling	implementable / alpha / v1.35	threshold-based placement。	P2
DRA scheduling	5027-dra-admin-controlled-device-attributes	provisional / alpha / v1.33	管理员控制设备属性。	P2

SIG Autoscaling

分类	KEP	状态 / 阶段	设计方案	优先级
HPA behavior	4951-configurable-hpa-tolerance	implementable / stable / v1.37	per-HPA / per-direction tolerance，避免全局 10% 过粗。	P0
HPA behavior	853-configurable-hpa-scale-velocity	implemented / stable	HPA behavior policies、stabilization window、scale up/down velocity。	P1
HPA API	2702-graduate-hpa-api-to-GA	implemented / stable / v1.23	HPA v2 API GA。	P1
metrics	1610-container-resource-autoscaling	implemented / stable / v1.30	container-level resource metrics，解决 sidecar 稀释主容器指标。	P0
metrics	117-hpa-metrics-specificity	implemented / stable	custom/external metrics label selector。	P1
metrics	5325-hpa-pod-selection-accuracy	implementable / v1.35	更准确的 HPA Pod selection，避免读到非 target Pod 指标。	P0
scale from zero	2021-scale-from-zero	implementable / beta / v1.37	HPA 基于 object/external metrics 从 0 副本启动。	P0
failure policy	5679-external-metric-fallback	implementable / alpha / v1.36	外部指标失败时 fallback，降低 metrics adapter 故障影响。	P0
cluster autoscaler	5030-attach-limit-autoscaler	implementable / beta / v1.37	CSI volume attach limit 进入 Cluster Autoscaler 推理。	P0

SIG Node

P0 活跃设计线

分类	KEP	状态 / 阶段	设计方案	优先级
DRA/ device	4381-dra-structured-parameters	implemented / stable / v1.35	DRA structured parameters, 取代 opaque allocation 路线, 是 DRA 主线。	P0
DRA/ device	3063-dynamic-resource-allocation	withdrawn / alpha / v1.32	早期 opaque DRA controller 协商方案, 因 autoscaler 不可推理和 apiserver 往返复杂而撤回。	P0
DRA/ device	4817-resource-claim-device-status	implementable / stable / v1.37	ResourceClaim status 标准化设备信息。	P0
DRA/ device	5304-dra-attributes-downward-api	implementable / beta / v1.37	DRA device attributes 暴露到 Pod。	P0
DRA/ device	5677-dra-resource-availability-visibility	implementable / alpha / v1.37	DRA resource availability 可见性。	P0
DRA/ device	5945-dra-optional-node-preparation	implementable / alpha / v1.37	DRA NodePrepare 可选化。	P0
DRA/ device	6072-dra-standard-numanode	implementable / stable / v1.37	DRA 标准 numaNode 属性。	P0
resize/ resources	1287-in-place-update-pod-resources	implemented / stable / v1.35	Pod resource requests/limits 原地更新。	P0
				P0

分类	KEP	状态 / 阶段	设计方案	优先级
resize/ resources	5419-pod-level-resources-in-place-resize	implementable / beta / v1.36	Pod-level resources 的 in-place resize。	
resize/ resources	5526-pod-level-resource-managers	implementable / beta / v1.37	Pod-level resource managers。	P0
resize/ resources	5554-in-place-update-pod-resources-alongside-static-cpu-manager-policy	implementable / alpha / v1.37	in-place resize 与 static CPU Manager policy 协作。	P0
resize/ resources	6122-configurable-scaling-delay-with-pod-resource-exposure	implementable / alpha / v1.37	Pod resource exposure 后的 scaling delay。	P0
observability	2371-cri-pod-container-stats	implementable / beta / v1.37	CRI-full container/pod stats，减少 cAdvisor 依赖。	P0
observability	5394-psi-node-conditions	implementable / alpha / v1.36	PSI 信号进入 Node Conditions。	P0
observability	4680-add-resource-health-to-pod-status	implementable / beta / v1.36	Device Plugin/DRA resource health 进入 Pod status。	P0
security	2033-kubelet-in-usersns-aka-rootless	implementable / beta / v1.37	rootless kubelet。	P0
security	5607-hostnetwork-usersns	implementable / alpha / v1.36	HostNetwork Pod 使用 user namespaces。	P0
lifecycle	4438-container-restart-termination	implementable /	Pod 终止期间 sidecar restart 行为。	P0

分类	KEP	状态 / 阶段	设计方案	优先级
		alpha / v1.37		
lifecycle	4563-eviction-request-api	implementable / alpha / v1.37	EvictionRequest API。	P0

P1 基础设计线

分类	KEP	状态 / 阶段	设计方案	优先级
resource manager	3570-cpumanager	implemented / stable / v1.26	CPU Manager cpuset 分配。	P1
resource manager	1769-memory-manager	implemented / stable / v1.32	Memory Manager 和 Topology Manager hint 协作。	P1
resource manager	693-topology-manager	implemented / stable / v1.27	CPU/Memory/Device NUMA hint 聚合。	P1
resource manager	3545-improved-multi-numa-alignment	implemented / stable / v1.32	多 NUMA 对齐改进。	P1
resource manager	2837-pod-level-resource-spec	implementable / stable / v1.37	Pod-level resource spec。	P1
resource manager	2570-memory-qos	implementable / beta / v1.37	Memory QoS with cgroup v2。	P1
resource manager	2902-cpumanager-distribute-cpus-policy-option	implementable / beta / v1.33	CPUManager 分散 CPU policy。	P1
resource manager	4540-strict-cpu-reservation	implemented / stable / v1.35	strict CPU reservation。	P1
device	3573-device-plugin	implemented / stable / v1.26	Device Plugin API。	P1

分类	KEP	状态 / 阶段	设计方案	优先级
device	4009-add-cdi-devices-to-device-plugin-api	implemented / stable / v1.31	Device Plugin 返回 CDI devices。	P1
device	3695-pod-resources-for-dra	implemented / stable / v1.36	PodResources API 包含 DRA resources。	P1
runtime	2040-kubelet-cri	implementable / beta / v1.23	kubelet CRI 支撑。	P1
runtime	2221-remove-dockershim	implemented / stable / v1.24	移除 dockershim。	P1
runtime	585-runtime-class	implemented / stable / v1.20	RuntimeClass。	P1
runtime	4033-group-driver-detection-over-cri	implemented / stable / v1.37	CRI 发现 cgroup driver。	P1
runtime	5825-cri-pagination	implementable / alpha / v1.36	CRI list streaming/pagination。	P1
lifecycle	753-sidecar-containers	implemented / stable / v1.33	Sidecar containers。	P1
lifecycle	277-ephemeral-containers	implemented / stable / v1.25	Ephemeral containers。	P1
lifecycle	2000-graceful-node-shutdown	implementable / beta / v1.21	Graceful node shutdown。	P1
lifecycle	2712-pod-priority-based-graceful-node-shutdown	implementable / beta / v1.24	Priority-based graceful shutdown。	P1
lifecycle	5307-container-restart-policy	implementable / beta / v1.35	Container restart policy。	P1
security	127-user-namespaces	implemented / stable / v1.36	User namespaces。	P1
security	2413-seccomp-by-default	implementable / stable / v1.27	Seccomp by default。	P1
security	2862-fine-grained-kubelet-authz	implementable / stable / v1.36	Kubelet API 细粒度授权。	P1

分类	KEP	状态 / 阶段	设计方案	优先级
security	2254-cgroup-v2	implementable / stable / v1.25	cgroup v2。	P1
security	5573-remove-cgroup-v1	implementable / beta / v1.35	移除 cgroup v1。	P1
observability	727-resource-metrics-endpoint	implemented / stable / v1.29	Kubelet resource metrics endpoint。	P1
observability	4205-psi-metric	implemented / stable / v1.36	PSI metrics。	P1
observability	5067-pod-generation	implemented / stable / v1.35	Pod generation。	P1
observability	5328-node-declared-features	implementable / stable / v1.37	Node declared features。	P1

P2 补充清单

分类	KEP	状态 / 阶段	设计方案	优先级
storage/resource	1029-ephemeral-storage-quotas	implementable / beta	Ephemeral storage quota。	P2
storage/resource	1539-hugepages	implemented / stable	HugePages。	P2
storage/resource	1967-size-memory-backed-volumes	implemented / stable	memory-backed volume size。	P2
storage/resource	6030-dynamic-resize-of-memory-backed-volumes	implementable / alpha	memory-backed volume 动态 resize。	P2
runtime	1547-building-kubelet-without-docker	implemented / stable	Dockerless kubelet。	P2
runtime	4216-image-pull-per-runtime-class	implementable / alpha	RuntimeClass 维度镜像拉取。	P2
runtime	4191-split-image-filesystem	implementable / beta	split image filesystem。	P2

分类	KEP	状态 / 阶段	设计方案	优先级
runtime	4210-max-image-gc-age	implemented / stable	image GC max age。	P2
runtime	5365-ImageVolume-with-image-digest	implementable / alpha	ImageVolume with digest。	P2
probe/ log	2727-grpc-probe	implemented / stable	gRPC probe。	P2
probe/ log	4939-grpc-probe-with-tls	implementable / alpha	gRPC probe with TLS。	P2
probe/ log	5999-h2c-container-probes	implementable / alpha	h2c probes。	P2
probe/ log	2411-cri-container-log-rotation	implemented / stable	CRI log rotation。	P2
probe/ log	3288-separate-stdout-from-stderr	implementable / alpha	stdout/stderr split stream。	P2
lifecycle	3960-pod-lifecycle-sleep-action	implemented / stable	PreStop sleep action。	P2
lifecycle	4818-allow-zero-value-for-sleep-action-of-prestop-hook	implemented / stable	PreStop sleep zero value。	P2
lifecycle	4960-container-stop-signals	implementable / beta	Container stop signals。	P2
lifecycle	5532-restart-all-containers-on-container-exits	implementable / beta	Restart all containers on exits。	P2
lifecycle	5593-configure-the-max-crashloopbackoff-delay	implementable / beta	CrashLoopBackOff max delay。	P2
security	135-seccomp	implemented / stable	Seccomp GA。	P2
security	24-apparmor	implementable / stable	AppArmor。	P2
security	1898-hardened-exec	implementable / alpha	Harden exec against SSRF。	P2

分类	KEP	状态 / 阶段	设计方案	优先级
security	213-run-as-group	implemented / stable	RunAsGroup。	P2
security	3619-supplemental-groups-policy	implemented / stable	SupplementalGroups policy。	P2
security	4265-proc-mount	implementable / stable	ProcMount option。	P2
security	5474-enable-writable-cgroups	implementable / alpha	Writable cgroups。	P2
security	5855-noexec-emptyDir	implementable / alpha	emptyDir mount options。	P2
node status	589-efficient-node-heartbeats	implemented / stable	Efficient node heartbeat。	P2
node status	3085-pod-conditions-for-starting-completion-of-sandbox-creation	implementable / stable	Pod networking ready condition。	P2
node status	5683-lifecycle-conditions	implementable / alpha	Node lifecycle conditions。	P2
node feature	793-node-os-arch-labels	implementable / alpha	OS/arch labels GA。	P2
node feature	4742-node-topology-downward-api	implementable / beta	Node topology labels via Downward API。	P2
node config	281-dynamic-kubelet-configuration	removed	Dynamic kubelet config, 已移除。	P2
node config	3983-drop-in-configuration	implemented / stable	kubelet drop-in config dir。	P2
node config	4580-deprecate-kubelet-runonce	provisional / alpha	Deprecate kubelet runonce。	P2
misc	2400-node-swap	implemented / stable	Node swap。	P2
misc	2535-ensure-secret-pulled-images	implementable / beta	Ensure secret pulled images。	P2

分类	KEP	状态 / 阶段	设计方案	优先级
misc	4639-oci-volume-source	implemented / stable	OCI images as VolumeSource。	P2
misc	5758-per-container-ulimits-configuration	implementable / alpha	Per-container ulimits。	P2
misc	5823-pod-level-checkpoint-restore	implementable / alpha	Pod-level checkpoint/restore。	P2
misc	6035-exec-session-identity	implementable / alpha	Exec session identity propagation。	P2
misc	6063-pod-pid-limit	implementable / alpha	Per-Pod PID limit。	P2

下一轮要补的数据

1. 从 prod-readiness/sig-scheduling/*.yaml、prod-readiness/sig-autoscaling/*.yaml、prod-readiness/sig-node/*.yaml 补 feature gates、metrics、rollback、scalability、upgrade/downgrade。
2. 增加“版本视图”：按 latest-milestone 输出 v1.35、v1.36、v1.37 的活跃 KEP 列表。
3. 增加“跨项目视图”：把 KEP 和 [kueue](#)、[karpenter](#)、[scheduler-plugins](#)、[metrics-server](#)、[prometheus-adapter](#)、[node-feature-discovery](#)、[k8s-gpu-device-stack](#) 对齐。
4. 从 P2 中挑选是否需要新增专题：quota/capacity governance、probe/logging、storage/resource local capacity。

Kubernetes

容器编排平台，CNCF 毕业项目，云原生基础设施的核心。

最新动态

v1.36 (2026-04 即将发布)

- 弃用 `Service.spec.externalIPs` (安全风险)
- 移除 `gitRepo Volume Driver`
- SELinux 卷标签加速 GA
- ingress-nginx 退役
- 详见 [src-k8s-v1.36-sneak-peek](#)

生态工具

- GitOps 交付: [argocd](#)、Flux
- 轻量发行版: K3s (适合 on-prem 和边缘场景, 见 [src-k3s-gitops-k0rdent](#))
- 可观测性: opentelemetry、Prometheus、Grafana
- AI 运维: HolmesGPT 自动告警诊断 (见 [src-holmesgpt-k8s-alerts](#))
- 安全: AI 漏洞发现带来新挑战 (见 [src-ai-vulnerability-discovery](#))

KEP 设计追踪

[src-kubernetes-keps-design-tracking](#) 把本地 Kubernetes enhancements 中 sig-scheduling、sig-autoscaling、sig-node 三个 SIG 的 KEP 整理成 [kubernetes-keps-design-tracking](#), 用于持续追踪不同 SIG 的设计方案, 而不是只按单一“资源编排”概念归档。

2026-07-15 对 upstream kubernetes/enhancements 的刷新见 [k8s-v1.37-scheduling-node-dra-progress](#)。当前 v1.37 进展主线集中在三处: Gang Scheduling / Topology-aware workload scheduling 进入 beta, DRA 的 ResourceClaim for workloads / Consumable Capacity / numaNode / device status 持续成熟, Node 侧 Pod-level resource managers、Memory QoS、rootless kubelet 和 Pod checkpoint/restore 继续推进。

当前追踪模型按 SIG、设计分类、status、stage、latest milestone、优先级和跨 SIG 依赖组织:

- **SIG Scheduling** — scheduler framework/config、queue/requeue、topology/affinity、preemption、workload/gang、DRA/device-aware scheduling。
- **SIG Autoscaling** — HPA API/behavior、tolerance、metrics specificity、container metrics、scale from zero、external metrics failure handling 和 cluster autoscaler integration。

- **SIG Node** — kubelet/CRI、CPU/Memory/Topology Manager、in-place resource resize、device plugin/DRA/CDI、pod lifecycle、security/isolation 和 node health。

第一批 P0 追踪线包括 DRA device taints 与 partitionable devices、queueing hint / scheduler async preemption、HPA configurable tolerance / container resource metrics / scale-to-zero、in-place pod resize 与 sidecar containers。 [kubernetes-keps-implementation-matrix](#) 已把 P0/P1 的 status、Alpha/Beta/GA、feature gates 和关键实现路径拉成 KEP 级矩阵；后续再补 prod-readiness.yaml 中的 metrics、rollback、scalability 和 upgrade/downgrade 细节。

已按合并设计组拉出 feature 覆盖矩阵和六篇重点设计文档详解：

- [kubernetes-keps-feature-coverage](#) — 汇总 scheduling / autoscaling / node 的重要 feature 覆盖状态。
- [kubernetes-keps-implementation-matrix](#) — 逐项追踪重要 KEP 是否实现、处于 alpha/beta/GA 哪个阶段、feature gate 和关键实现路径。
- [kubernetes-scheduler-core-design](#) — Scheduler framework、profiles、queue/requeue、topology placement 和 async preemption。
- [kubernetes-workload-gang-scheduling-design](#) — Workload / PodGroup、gang scheduling、workload-aware preemption 和 controller API building blocks。
- [kubernetes-dra-design-deep-dive](#) — DRA structured parameters、ResourceSlice/ResourceClaim、scheduler/kubelet plugin 和 autoscaler 可推理性。
- [kubernetes-hpa-autoscaling-design](#) — HPA tolerance、container metrics、pod selection、scale from zero 和 external metric fallback。
- [kubernetes-in-place-pod-resize-design](#) — Pod / resize、资源状态机、Pod-level resources、static CPU manager 和 resize-induced preemption。
- [kubernetes-node-runtime-observability-security-design](#) — kubelet/CRI、resource managers、sidecar lifecycle、security 和 node observability。

相关概念

- [gitops](#) — 声明式交付模式
- ebpf — 内核级可观测和网络
- [cloud-native-security](#) — 云原生安全
- [kubernetes-keps-design-tracking](#) — 按 SIG 分类追踪 Kubernetes KEP 设计方案

Agent Runtime / Sandbox 细分选型地图

已有 [agent-runtime-sandbox-project-map](#) 做了完整分层。这页面向选型：你需要的是 sandbox primitive、E2B-like sandbox 平台、session API、安全 runtime、多 Agent 平台，还是 AI gateway。

GitHub 当前核验

截至 2026-06-15 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	当前定位
agent-sandbox	https://github.com/kubernetes-sigs/agent-sandbox	2026-06-10	2.8k	Go	K8s-native isolated stateful singleton workload
openkruise-agents	https://github.com/openkruise/agents	2026-06-12	202	Go	K8s-native agent sandbox lifecycle platform
agentcube	https://github.com/volcano-sh/agentcube	2026-06-08	147	Go	Sandbox session orchestration
OpenShell	https://github.com/NVIDIA/OpenShell	2026-06-11	7k	Rust	Safe private runtime for autonomous agents
NemoClaw	https://github.com/NVIDIA/NemoClaw	2026-06-11	21k	TypeScript	OpenShell 内运行 Hermes/OpenClaw
HiClaw	https://github.com/agentscope-ai/HiClaw	2026-06-10	4.8k	Go	Matrix + K8s 的 collaborative multi-agent OS
AgentScope	https://github.com/agentscope-ai/agentscope	2026-06-09	26k	Python	可观测、可理解、可信任的 Agent 构建框架
agentgateway	https://github.com/agentgateway/agentgateway	2026-06-10	3.2k	Rust	AI agents / MCP servers 的 agentic proxy

分层选择

你要解决的问题	首选	不适合
K8s 上安全运行一个有状态 Agent 容器	agent-sandbox	需要完整产品 API、IM 协作或工具治理

你要解决的问题	首选	不适合
自托管 E2B-like sandbox service	openkruise-agents	只想要最小 CRD 或不能接受快速演进的 manager/gateway/runtime 组合
把 sandbox 包成 session/invocation API	agentcube	不使用 Kubernetes 或不想引入 Redis/session registry
本地/私有强安全 Agent runtime	OpenShell	只需要普通容器隔离
给 OpenShell 配套 agent onboarding	NemoClaw	只想写一个 Agent 应用框架
多 Agent 协作 + 人在回路	HiClaw	不想引入 Matrix / K8s operator
Python Agent 应用服务化	AgentScope	需要强 sandbox 原语
LLM/MCP/A2A 流量治理	agentgateway	需要运行 Agent 进程本身

架构边界



agent-sandbox 处在 sandbox lifecycle primitive；[openkruise-agents](#) 处在 sandbox platform layer，把 CRD、E2B API、Envoy route、runtime/CSI/identity 扩展组合起来；OpenShell 处在 runtime enforcement；[agentgateway](#) 处在 network/tool gateway；HiClaw 和 [agentcube](#) 是上层产品化编排；AgentScope 是应用框架层。

按问题定位项目

我要解决什么问题？	
— 写 Agent 应用 / event loop	→ AgentScope
— 提供 session / invocation API	→ AgentCube
— 管理 K8s sandbox 生命周期	→ agent-sandbox
— 做 E2B-like 平台	→ OpenKruise Agents

└─ 做进程/网络/凭据强制隔离	→ OpenShell
└─ 做多人协作与人在回路	→ HiClaw
└─ 管理 LLM/MCP/A2A 出口	→ agentgateway

关键对比

维度	agent-sandbox	openkruise-agents	OpenShell	agentcube	HiClaw	agentgateway
控制面	K8s controller	sandbox-manager + K8s controllers	Gateway desired state	Router + WorkloadManager	K8s operator + Matrix/Higress	Gateway API/xDS
数据面	Pod/PVC/Service	Envoy ext_proc / Go filter + Sandbox Pod	supervisor + policy proxy	Sandbox pod + PicoD	Worker containers + Matrix rooms	Rust L7 proxy
状态	K8s API + PVC	K8s API + route registry + key storage	object store + sandbox state	Redis/ValKey session registry	etcd/kine + Matrix + MinIO	route/backend/policy stores
凭据	交给上层	E2B API key storage + security token propagation	gateway 托管 provider key	Router JWT / workload auth	Higress 托管真实凭据	backend secret / policy
用户入口	CRD / SDK	E2B API + CRD	CLI / SDK	HTTP / Python SDK / Dify	Matrix / CRD / CLI	CRD / Gateway / UI

采用建议

- 做“Agent 运行在 K8s 里的基础设施”：先看 agent-sandbox；需要 E2B-compatible API、warm pool、路由和 runtime 扩展时看 [openkruise-agents](#)；需要 invocation/session 产品 API 时再看 [agentcube](#)。
- 做“安全私有 Agent 运行时”：先看 OpenShell，再看 NemoClaw 如何编排。
- 做“团队协作 Agent 平台”：HiClaw 的 Matrix-first 协作面比普通 Agent framework 更贴近人类介入。
- 做“工具/模型访问治理”：[agentgateway](#) 应作为 sandbox 外侧的流量控制点。

Agent Runtime / Sandbox 项目地图

这页把当前知识库里和 Agent runtime、sandbox、会话编排、安全控制面相关的项目放到同一张工程地图里。核心结论：这些项目都在解决“Agent 怎么安全、可恢复、可观测地长期运行”，但抽象层不一样。

```
Developer / user API
  ↓
Agent app framework / CLI / SDK
  ↓
session orchestration / agent control plane
  ↓
sandbox lifecycle primitive
  ↓
runtime enforcement: container / supervisor / proxy / policy
  ↓
network, credentials, storage, inference, tool access
```

通用 Agent Session 生命周期

```
Create / Claim
  ↓
Provision or WarmPool adoption
  ↓
Ready + route + identity
  ↓
Running session
  ├── tool/model/network request → policy → external service
  ├── idle timeout → suspend / hibernate
  ├── checkpoint → snapshot / PVC / workspace
  ├── resume → restore route + identity
  └── expiry / delete → revoke → cleanup / retain data
```

这个方向不能只看“能不能启动一个容器”。Agent runtime 的特殊性在于：它是长寿命、有状态、会执行不可信输入、会持有工具能力、会访问模型和外部网络，并且经常需要人类中途介入。真正的系统边界包括生命周期、身份、凭据、网络、文件、推理路由、工具权限和恢复。

一句话分层

项目	一句话定位	抽象层
agent-sandbox	K8s SIG Apps 的 Sandbox CRD，把单个有状态 Agent 容器建模成 K8s 一等资源	Sandbox 基础设施原语
openkruise-agents	OpenKruise 的 Agent sandbox lifecycle platform，把 SandboxSet/Claim、E2B API、Envoy 路由和 runtime/CSI/identity 扩展组合起来	K8s sandbox 平台层
agentcube	基于 agent-sandbox 的 serverless session orchestration layer，给 Code Interpreter / AgentRuntime 提供 HTTP/SDK 调用入口	会话编排层
OpenShell	NVIDIA 的安全私有 Agent runtime，Gateway 控制面 + sandbox Supervisor + OPA/Z3 policy proxy	安全 runtime / enforcement
NemoClaw	OpenShell sandbox 内 always-on Agent 的 host-side CLI 控制面，编排 onboarding、provider、policy 和 agent setup	Host-side 编排层
HiClaw	K8s operator + Matrix IM + Higress 网关的多 Agent 协作平台	多 Agent 平台 / 协作层
AgentScope	Python 多 Agent 应用框架，事件流 ReAct loop + toolkit/MCP/skill + workspace/offload + FastAPI service	应用框架 / 服务层
agentgateway	面向 LLM / MCP / A2A 的 Rust L7 数据面 + Gateway API 控制面	Agent 网络 / 工具网关

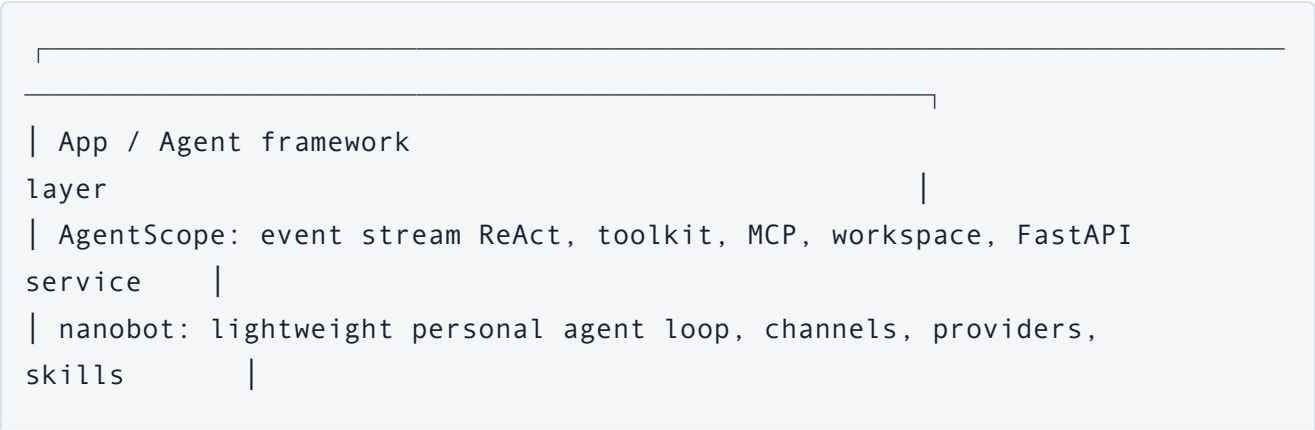
横向对比

维度	agent-sandbox	openkruise-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
主问题	安全运行一个有状态 Agent 容器	K8s 上运营 E2B-like sandbox 服务	把 Sandbox 变成可调用 session	sandbox 内 runtime enforcement	OpenShell onboarding + agent setup	多 Agent 运维与协作	Agent app 构建与服务化	AI 协议流量治理
用户入口	K8s CRD / Go/Python SDK	E2B API + K8s CRD	HTTP invocation / Python SDK / Dify /	CLI / SDK / TUI	nemoclave / nemohermes CLI	Matrix / CLI / CRD	Python SDK / FastAPI	Gateway API / xDS / UI

维度	agent-sandbox	openkruise-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
			LangChain					
控制面	K8s controller-runtime	sandbox-manager + agent-sandbox-controller	Router + WorkloadManager + Redis	Gateway object store + compute runtime	Host-side FSM + OpenShell adapters	K8s operator + embedded/in-cluster mode	FastAPI service + storage / session manager	Go controller + ADS server
数据面	Pod + headless Service + PVC	Envoy ext_proc / Go filter + Sandbox Pod	Sandbox Pod / PicoD / reverse proxy	Supervisor + restricted child + policy proxy	OpenShell sandbox + OpenClaw/ Hermes	Worker/ Manager containers + Matrix rooms	Agent event loop + workspace	Rust proxy for LLM/ MCP/ A2A
生命周期	Sandbox 0/1 replicas, shutdownTime, WarmPool	SandboxSet pool、Claim、pause/resume、clone/checkpoint	session create/route/GC, WarmPool claim	create/delete/watch/reconcile/resume	onboard resume/fresh, registry lock	CR reconcile worker/team/human/manager	session run serialization, continuation events	listener / route/ backend config hot update
隔离机制	委托 K8s runtimeClass/ NetworkPolicy/ securityContext	K8s runtime + agent-runtime/ CSI/ traffic-proxy 扩展	继承 agent-sandbox + Router JWT	Supervisor/ proxy/seccomp/netns/TLS/OPA	继承 OpenShell policy/shields	容器隔离 + Higress 凭据隔离	Workspace Local/ Docker /E2B	CEL RBAC, mTLS/ HBONE, guardrails
凭据	不直接托管凭据	E2B API key	Router → PicoD	Gateway 托管		Higress 托管真	Credential/	backend

维度	agent-sandbox	openkruis e-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
模型		storage + identity token propagation	JWT, WorkloadManager auth	provider credentials	Gateway 是凭据 system-of-record	实凭据, Worker 只持 consumer key	session / workspace service	credentials + policy/filter
状态	K8s API + PVC	K8s API + route registry + key storage + memberlist	Redis/ValKey session registry + K8s	SQLite / PostgreSQL object store	~/ .nemoclaw JSON + Gateway state	etcd/kine + Matrix + MinIO + gateway	storage / session / workspace	xDS stores + control plane
典型强项	K8s-native、简单、WarmPool	CRD + E2B API + Envoy route + runtime 扩展一体化	产品化 invocation API	安全 enforcement 边界最细	Onboarding 和 provider/policy 编排	IM-first 协作和凭据零暴露	Agent app 编程体验	LLM/MCP/A2A 统一网关
典型代价	只提供底层原语	系统面宽, API/roadmap 仍在快速演进	依赖 agent-sandbox API 演进	系统复杂度高	host-side 状态机复杂	学习曲线和容器开销	不是 sandbox 原语	不直接运行 Agent

分层图谱



↓

| Multi-agent / session product
layer |
| HiClaw: Worker/Team/Human/Manager CRD + Matrix +
Higress |
| OpenKruise Agents: SandboxSet/Claim + E2B API + Envoy
route |
| AgentCube: AgentRuntime/CodeInterpreter + Router +
WorkloadManager |
| NemoClaw: OpenShell onboarding + OpenClaw/Hermes
setup |

↓

| Sandbox / runtime primitive
layer |
| agent-sandbox: Sandbox/Claim/Template/WarmPool → Pod/PVC/
Service |
| OpenShell: Gateway desired state → driver → Supervisor/proxy
enforcement |

↓

| Network / credential / tool gateway
layer |
| agentgateway: LLM/MCP/A2A proxy, Gateway API, CEL, HBONE,
xDS |
| Higress/OpenShell gateway: provider credentials and inference
routing |

关键关系：

- [agentcube](#) 明确站在 agent-sandbox 之上：前者做 session API，后者做 Sandbox 生命周期。
- [openkruise-agents](#) 站在 agent-sandbox 相邻但更平台化的位置：它把 SandboxSet/Claim、E2B API、Envoy 路由、agent-runtime、CSI 和 identity token 放到同一套 K8s 发行版里。
- NemoClaw 明确站在 OpenShell 之上：前者做 onboarding 和 agent setup，后者做 sandbox runtime 和 enforcement。
- HiClaw 和 agent-sandbox 是互补关系：HiClaw 管多 Agent 协作，agent-sandbox 可以成为底层隔离原语。
- AgentScope 和 agent-sandbox 处在相邻问题域：AgentScope 的 Workspace 能接 Local/Docker/E2B，agent-sandbox 是更云原生的执行环境原语。
- [agentgateway](#) 不运行 Agent，但控制 Agent 访问模型、MCP、A2A 的网络边界。

工程剖面

agent-sandbox：Sandbox 作为 K8s 原语

agent-sandbox 的核心贡献是把 Agent runtime 从普通 Deployment/StatefulSet 里抽出来。Agent sandbox 的语义是：

- 单实例，replicas 只能 0/1。
- 有稳定身份，自动创建 headless Service。
- 有持久存储，PVC 生命周期不随 Pod 自动删除。
- 可以暂停/恢复，scale to 0/1。
- 可以过期，shutdownTime 是声明式生命周期。
- 可以 WarmPool 预热，然后通过 Claim 领养。

它避免把安全机制做成私有实现，而是把隔离委托给 K8s 原语：runtimeClassName、gVisor/Kata、NetworkPolicy、securityContext、serviceAccount。这个设计的工程价值是可组合：任何上层平台都可以继承 K8s 生态能力。

风险也清楚：它只提供底层原语，不负责多 Agent 协作、工具协议、凭据网关、HTTP invocation 或 Agent app framework。这些必须由上层补齐。

[openkruise-agents](#)：把 Sandbox 原语扩展成 E2B-like 平台

[openkruise-agents](#) 的关键不是再造一个最小 Sandbox CRD，而是把 Agent sandbox 生命周期的产品面补齐：

```
E2B API / K8s CRD
  ↓
sandbox-manager
  ↓
SandboxClaim / SandboxSet / Sandbox
```

```
↓
agent-sandbox-controller
↓
Envoy route registry → Sandbox Pod IP:port
```

它把 warm pool 明确建模成 SandboxSet，把用户请求建模成 SandboxClaim，把 API 入口做成 E2B-compatible /sandboxes / /snapshots，再通过 Envoy ext_proc 或 Go filter 把请求路由到具体 sandbox。这个组合适合做自托管 E2B-like workspace 服务。

风险在于系统面比 agent-sandbox 大很多：key storage、route registry、memberlist peer sync、dynamic CSI mount、identity token propagation、checkpoint/hibernation 和 runtime sidecar 都需要分别验证。选型时应把它看成平台工程项目，而不是单一 controller。

[agentcube](#)：把 Sandbox 包成可调用会话

[agentcube](#) 的定位比 agent-sandbox 高一层。它关心开发者如何通过 HTTP/SDK 调用一个 Code Interpreter 或 AgentRuntime，而不是让用户直接写 Sandbox CR。

关键路径：

```
Client invocation
↓
Router checks x-agentcube-session-id
↓
WorkloadManager creates Sandbox or SandboxClaim
↓
agent-sandbox starts/adopts Pod
↓
Router reverse-proxies to PodIP/entrypoint
↓
Redis/ValKey tracks session TTL and activity
```

它最重要的架构判断是不重写 Sandbox controller。WorkloadManager 只把 AgentCube 的 AgentRuntime / CodeInterpreter 翻译成 agent-sandbox CRD，底层 lifecycle、WarmPool adoption、Pod/PVC/Service 都交给 agent-sandbox。

安全上，PicoD 用 Router-signed JWT 替代 SSH；这简化 SDK，但 Router 变成强信任边界，后续需要 auth、mTLS、RBAC、审计一起兜住。

OpenShell：Gateway desired state, Supervisor runtime enforcement

OpenShell 的分层边界最安全工程化：

```
Gateway owns desired state
↓
compute driver provisions sandbox
↓
Supervisor owns runtime enforcement
↓
policy proxy controls process/network/inference egress
```

Gateway 保存 sandbox、provider、policy、settings、inference、sessions 等状态；sandbox 内 Supervisor 才真正执行网络、进程、文件、TLS、provider credential 注入和 proxy enforcement。这个边界成立，因为只有 sandbox 内能看见进程身份、socket、binary hash、TLS 流和运行时凭据。

OpenShell 的关键不是“有 sandbox”，而是把 Agent egress 变成可审计、可裁决、可提案的策略流。普通网络访问走 OPA；inference.local 是特殊本地能力，代理到 provider，但 Agent 不直接持有真实 key。

NemoClaw: OpenShell 上的 host-side Agent 编排

NemoClaw 是 OpenShell 的产品化编排层，不是另一个推理或 sandbox 引擎。它把用户真正要完成的步骤串起来：

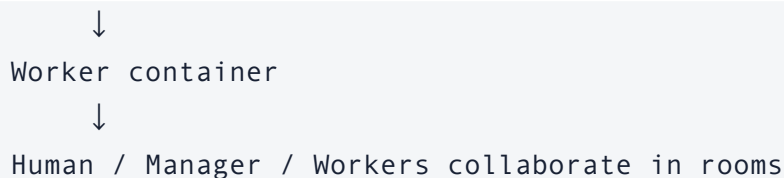
- preflight 检查 Docker/OpenShell/GPU/DNS。
- 启动或复用 named OpenShell gateway。
- 选择 provider/model/credential route。
- 创建或复用 sandbox。
- 安装/同步 OpenClaw 或 Hermes runtime config。
- 应用 baseline policy 和 presets。
- 编译 messaging channel manifest。

它的核心是可恢复 onboard FSM：host-side JSON registry、session state、全局 lock、resume/fresh/non-interactive。这个模式适合复杂本地开发工具，但也带来状态恢复、部分写入和版本迁移成本。

HiClaw: Agent 是 CR，也是 IM 用户

HiClaw 的设计很鲜明：把 Agent 运维做成 K8s operator，同时把协作面放到 Matrix IM。

```
Worker CR
↓
controller reconcile
↓
Matrix user + room
↓
Higress consumer key
```



这解决两个常被 Agent framework 忽略的问题：

- 人类如何自然介入：直接进 Matrix 房间。
- Agent 凭据如何隔离：Worker 只拿 Higress consumer key，真实 provider/API key 在网关侧。

它的代价是平台复杂度高。每个 Worker 是容器、Matrix 用户、gateway consumer 三位一体；Provisioner 同时编排 Matrix/Higress/Credentials/Manager bootstrap，横向拆分压力已经出现。

AgentScope：应用框架里的 runtime 边界

AgentScope 不提供强 sandbox 原语，但它定义了生产 Agent 应用的运行时边界：

- AgentEvent / Msg 是 SDK、SSE、存储和 UI 的统一协议。
- ReAct loop 把人类确认和外部执行建模成 continuation event。
- Tool execution 被拆成权限/上下文写入和 raw I/O。
- Workspace 承接 tools、MCP、skills 和 context/tool-result offload。
- FastAPI service 每轮从 storage/session/workspace 重新组装 Agent。

这类框架更适合应用服务，而不是底层隔离。它可以把 Docker/E2B/未来 agent-sandbox 作为 Workspace backend，把自身保持在 Agent 编排与事件流层。

[agentgateway](#)：Agent 访问外界的网络控制点

[agentgateway](#) 解决的是 Agent 流量，而不是 Agent 生命周期。它把 LLM API、MCP tools、A2A 通信统一到一个 Rust L7 数据面，控制面复用 Gateway API/xDS。

对 runtime/sandbox 体系来说，它的价值是：

- LLM provider 统一协议和 guardrails。
- MCP tools federation + CEL RBAC。
- A2A Agent Card URL rewrite 和代理。
- HBONE/mTLS/telemetry/access log。
- 用声明式 CRD 管路由、策略和后端。

如果 agent-sandbox 是“Agent 跑在哪里”，[agentgateway](#) 就是“Agent 能访问什么、怎么访问、怎么审计”。

核心难点

1. 生命周期不是 Pod 生命周期

Agent sandbox 的生命周期包含：创建、恢复、暂停、续期、过期、热领养、GC、startup resume、长任务 offload。这比普通 Pod 的 Running/Terminated 更复杂。

- agent-sandbox 用 shutdownTime 和 0/1 replicas 表达暂停/过期。
- [openkruise-agents](#) 用 SandboxSet / SandboxClaim 表达 pool 与领取，用 E2B API 管 pause/resume/clone/checkpoint。
- [agentcube](#) 用 Redis sorted set 管 session TTL 和 last activity。
- OpenShell 用 Gateway store + driver watch/reconcile/resume。
- NemoClaw 用 host-side FSM 和 registry 做 onboarding resume。

结论：Agent runtime 必须显式建模“会话”和“工作负载”的差异。

2. 凭据不能进 Agent 进程

Agent 会执行模型生成的工具调用，也会读取用户输入。真实 API key、GitHub PAT、云 AK 如果进入 Agent 进程，就会被 prompt injection 放大。

成熟方向都是网关托管：

- HiClaw：真实 key 在 Higress，Worker 只持 consumer key。
- OpenShell / NemoClaw：Gateway 是 provider credential system-of-record，sandbox 通过 `inference.local` 使用模型。
- [agentgateway](#)：后端 credentials 与 policy/filter 在网关侧。

这和 agent-credential-isolation 是同一条设计线。

3. 网络策略需要进程级语义

K8s NetworkPolicy 只能管 Pod 维度，Agent sandbox 内可能有多个进程：agent、tool、shell、browser、package manager。OpenShell 通过 proxy + `/proc/net/tcp` + binary identity 把网络裁决细化到进程/二进制身份，这是更细粒度的 runtime enforcement。

对于大多数平台，第一阶段可以用 NetworkPolicy + egress proxy；高风险场景才需要 OpenShell 这种 Supervisor/proxy 模型。

4. WarmPool 是性能优化，也是所有权协议

预热不是“提前启动 Pod”这么简单。关键难点是 ready sandbox 被 claim 领养时，ownerReference、labels、status、Pod exclusivity 必须一致。

agent-sandbox 的 SandboxClaim adoption 协议是底层实现；[agentcube](#) 只在 CodeInterpreter warmPoolSize 上暴露能力。这是合理分工：底层保证所有权不变量，上层只表达“我需要低冷启动”。[openkruise-agents](#) 则把 warm pool 升级成平台级 SandboxSet + SandboxClaim 协议，并在 claim 时叠加 inplace update、dynamic CSI mount、runtime init 和 identity token。

5. 人在回路不是 UI 附加项

多 Agent 系统需要人类在任务中途介入、批准、观察和接管。

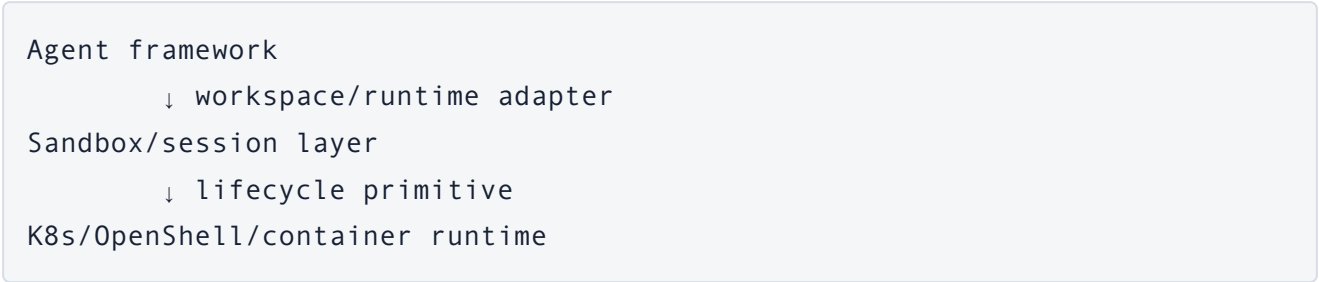
- HiClaw 用 Matrix 房间把人类和 Agent 放在同一个协作平面。
- AgentScope 用 RequireUserConfirmEvent 和 continuation event 把人类确认放进事件流。
- OpenShell 的 policy proposal 默认人工审批。

人在回路应该是协议和状态机能力，不应该只是一个前端按钮。

6. App framework 和 sandbox runtime 要分层

AgentScope、nanobot 这类应用框架解决“如何思考、调用工具、持久化上下文”；agent-sandbox/OpenShell 解决“如何安全运行”。把两层混在一起会导致框架既要懂 ReAct，又要懂容器、网络、凭据、恢复，复杂度失控。

比较稳的组合是：



设计分型

分型	代表项目	设计重心	适合场景
Sandbox 原语型	agent-sandbox	CRD、Pod/PVC/Service、WarmPool、K8s 原语	云原生 Agent 执行环境底座
Session 编排型	agentcube	HTTP invocation、session registry、Router/WorkloadManager	Code interpreter / serverless agent runtime
安全 runtime 型	OpenShell	Gateway desired state、Supervisor、policy proxy	高风险 autonomous agent、凭据隔离、网络审计
Host-side 编排型	NemoClaw	onboarding FSM、provider/policy/agent setup	把安全 runtime 包成可用产品

分型	代表项目	设计重心	适合场景
多 Agent 平台型	HiClaw	CRD + IM + 网关凭据 + Worker/Team	企业多 Agent 协作和运维
应用框架型	AgentScope, nanobot	ReAct loop、event stream、tools、workspace	构建 Agent 应用或个人 Agent
网络网关型	agentgateway	LLM/MCP/A2A 流量、policy、telemetry	Agent 外部访问治理

选型建议

目标	优先看	工程关注点
想把 Agent 容器建模成 K8s 原语	agent-sandbox	Sandbox/Claim/WarmPool、PVC 生命周期、NetworkPolicy
想做 Code Interpreter / AgentRuntime 会话服务	agentcube	Router/WorkloadManager、session registry、WarmPool adoption
想研究最细的安全 sandbox runtime	OpenShell	Gateway/Supervisor 分界、policy proxy、inference.local
想把 OpenShell 包成用户可用 CLI 工作流	NemoClaw	onboard FSM、provider/policy/messaging manifest
想做多 Agent 协作平台	HiClaw	CRD、Matrix、Higress、credential isolation
想做 Agent 应用服务	AgentScope	AgentEvent、tool permission、workspace/offload、FastAPI
想治理 Agent 的 LLM/MCP/A2A 流量	agentgateway	Gateway API、xDS、CEL、MCP federation、guardrails

相关页面

- [agent-sandbox](#)
- [agentcube](#)
- [HiClaw](#)
- [agentgateway](#)
- [declarative-agent-management](#)
- [agent-credential-isolation](#)
- [cloud-native-security](#)
- [ai-agent-plugin-patterns](#)

- mcp

Kubernetes Core / Controller 项目地图

当前上游核验（2026-09-22）

本节是对官方仓库默认分支的当前观察；[src-controller-runtime-architecture](#)、[src-kubebuilder-architecture](#) 与 [src-controller-tools-architecture](#) 则是 2026-06-14 摄入的 raw-backed Source 快照，保留当时的代码结构与设计语境。

项目	当前证据	稳定职责边界	M5-A 位置
controller-runtime	6ab2188a1fb1	组合 Manager、Cache、Client、Controller/Reconciler、Scheme、Webhook 与 envtest；不负责项目脚手架	controller runtime composition
kubebuilder	5f31d1f3c075	面向作者的 Kubernetes API/CRD/controller/webhook 脚手架、插件与开发工作流，建立在 controller-runtime 和 controller-tools 之上	author workflow / scaffolding
controller-tools	030a93937cbb	用 marker 驱动 controller-gen 生成 CRD、RBAC、webhook、deepcopy/object 与 apply-configuration 等资产	marker / type generation
client-go	Kubernetes staged 基座（本次不单独锚定 HEAD）	提供 clientset、discovery/dynamic client、transport 与 controller 所需 cache/informer/workqueue 机制；必须按 Kubernetes minor 版本选择匹配的 client-go / k8s.io/* 依赖	client foundation

上表的三个 commit 是本次执行时默认分支锚点，不代表 release；生产项目应按 controller-runtime / controller-tools / Kubebuilder 与 client-go / Kubernetes 的兼容矩阵锁定版本。官方入口：[controller-runtime repo](#)、[Kubebuilder repo](#)、[Kubebuilder Book](#)、[controller-tools repo](#)、[client-go docs](#)。

这页把 [src-k8s-core-controllers-stars](#) 从 359 个 star 项目整理成 Kubernetes controller/operator 学习与选型地图。核心结论：K8s 平台工程的主线不是“会写一个 reconcile”，而是理解 API machinery、client/cache/workqueue、CRD/webhook、controller-runtime/kubebuilder、调度/多集群/安全/可观测这些层如何组合。

D1 · Controller 工具链职责图

BUILD-TIME（作者与生成链）
Kubebuilder CLI — scaffold —> Go API types / markers / project layout

```

controller / webhook / test 骨架
Go API types / markers — parse —> controller-tools / controller-gen
controller-gen — generate —> CRD / RBAC / webhook manifests
      — generate —> deepcopy (object generator) /
applyconfiguration
作者补齐业务逻辑，构建 controller 镜像并安装生成的 manifests

RUNTIME (控制器执行链)
controller 二进制使用 controller-runtime 的组合层
  Manager：管理 Cache / Client / Controller-Reconciler / Webhook 生命周期
  Scheme：Go types 与 GVK 映射；供 Client 等组件使用
  client-go 基座：REST client / watch / informer / cache / workqueue
  Client — write / uncached read —> Kubernetes API server
  Kubernetes API server — admission request —> Webhook (同步响应)

测试支持 (不属于生产 reconcile 路径)：controller-runtime / envtest

```

图例：实线 — label —> 表示顺序生成或同步调用/API 写入；虚线 - - label - -> 专指下图的异步 watch、入队和重试反馈，标签说明触发原因。缩进表示组件组合，不表示一次调用。[kubebuilder](#) 组织作者 workflow，[controller-tools](#) 解析 marker/type 并生成资产，[controller-runtime](#) 组合 client-go 的基础机制；这些项目分层组合，不能相互替代。Webhook 处理同步 admission 请求；envtest 为测试启动 API server 等控制面进程，不是生产集群的控制器。

D3 · Reconcile 控制循环

```

Manager 管理 Cache 与 Controller, Controller 消费自己的 workqueue
Kubernetes API server - - watch / observed-state feedback - -> Cache /
informer
Cache / informer - - event handler: enqueue root object key - -> workqueue
workqueue — worker 取 key 并调用 —> Reconciler
Reconciler
  |— Client.Get/List (默认缓存读) —> Cache：期望状态 + 已观测状态
  |— Client.Create/Update/Patch/Delete —> API server
  |                                     metadata/finalizer / owned
resources
  |— Client.Status().Update/Patch —> API server
  |                                     status / conditions /
observedGeneration
  |— return Result / error —> Controller

```

Controller - - 可重试 error: rate-limited retry - -> workqueue
 Controller - - RequeueAfter: delayed requeue - -> workqueue
 API 写入后的变化 - - 经上述 watch/cache/enqueue 路径 - -> 后续 reconcile

Reconcile 是 level-based、幂等、异步收敛的控制循环，不是一次 event 对应一次必须完成的 request handler。事件可能合并；handler 把本对象或关联对象的变化映射为待处理 key，Reconciler 重新读取期望状态与可观测状态并计算差异。API 调用返回成功只确认该次写入成功，不代表 cache 已更新或依赖资源已经就绪；后续 watch 或显式 requeue 推进收敛。返回空 Result 且无 error 时，不主动安排重试。参见 [controller-runtime FAQ](#) 与 [src-controller-runtime-architecture](#)。

D4 · 状态与一致性

状态/机制	Owner / operation	陈旧或失败风险	恢复/收敛方式
Cached read / direct write	Manager 默认 Client 对已缓存类型读 cache，写请求直达 API server；可配置直读	写成功后立即读 cache 仍可能是旧值；重复创建或错误判断缺失	确定性命名、幂等操作与后续 watch/requeue 再评估；必要时 APIReader 直读，但仍需处理读写间的并发
resourceVersion conflict	API server 对 Update 或带版本前提的 Patch 做乐观并发检查	旧版本写入被拒绝；无版本前提的 Patch 不自动提供同等保护	重读、重算再提交；按更新策略显式选择版本前提，避免盲目覆盖竞争字段
spec / status / conditions / observedGeneration	用户或上游 controller 写 spec；本 controller 按 API 约定写 status/condition	status 对应旧 generation，或把“已观察”误解成“已成功”	observedGeneration 记录实际处理的 generation；condition 表达处理结果，仅在内容变化时写入；lastTransitionTime 随 condition 状态转换更新
ownerReference / GC	controller 设置 owner UID/作用域；garbage collector 按引用与删除策略处理依赖资源	错 UID、跨作用域引用或错误接管导致泄漏/误删；ownerReference 本身不配置 watch	校验 owner 与 controller 归属，显式设置 owned-resource watch；缺失资源且仍被期望时重建，错误归属需修正代码/对象
Finalizer / deletionTimestamp	controller 在删除前完成外部清理，成功后移除 finalizer	外部系统不可用、权限变化或旧 controller 消失导致删除卡住	可重试的清理 + backoff；无法恢复时由人工核实外部状态后审慎移除 finalizer

状态/机制	Owner / operation	陈旧或失败风险	恢复/收敛方式
Requeue / rate limit / idempotency	Controller 根据 Result/error 安排队列；业务代码负责副作用幂等	高频事件、自触发 status write 或非幂等副作用形成 hot loop；队列不提供 exactly-once	差异写入、幂等键和可恢复清理；错误重试使用 backoff，定时检查用 RequeueAfter，不假设它等同错误退避

API 一致性与删除机制分别参见 [Kubernetes API Concepts](#) 和 [Kubebuilder Finalizers](#)。

D5 · 失败边界

起因	直接影响 / 因果链	自动恢复边界	需要人工修复的情况
Stale cache	旧观测值 → 旧决策或写冲突	watch 恢复与缓存追上后可收敛；显式定时 requeue 可补充检查，不能替代失效的 watch	cache 长期不同步、watch/RBAC 错误或业务逻辑把旧值当不变量
Write conflict	并发变更 → API server 拒绝旧 resourceVersion 写入	重读、重算、requeue；幂等 reconcile 通常可自恢复	持续字段所有权争用或错误的 patch 策略
Hot reconcile	自触发写入/永久错误 → queue、API server 与日志压力	backoff 限制错误重试；普通 watch 入队不会统一受错误退避保护，不能靠重试机制消除热循环	修正非幂等逻辑、无差异 status patch 或永久错误分类；谨慎配置 predicate，避免过滤必要事件
Webhook / certificate unavailable	admission 调用失败 → failurePolicy: Fail 时写入失败，Ignore 时可能跳过失败调用继续	已配置的 Pod 重启/证书轮换恢复服务后，调用方重试才可继续；框架不自动修复证书配置	错误 CA bundle、Service/DNS、证书轮换配置或 webhook 逻辑；显式拒绝请求不因 Ignore 而放行
Leader switch	启用 leader election 时旧 leader 失租 → 可用副本获得租约并接管	依赖正确的选主配置与健康副本；缓存同步和初始/后续事件重建待处理工作，队列并非持久化交接	非幂等外部副作用已部分执行，或新 leader 因配置/权限无法启动；选主不保证外部操作 exactly-once
Stuck finalizer	deletionTimestamp 已设 → 清理失败 → 对象长期 Terminating	外部依赖恢复且清理幂等时可重试收敛	controller 已移除、外部资源不可查或永久失败；需核实后再人工处理 finalizer

起因	直接影响 / 因果链	自动恢复边界	需要人工修复的情况
Wrong ownership / status	错 ownerReference → 泄漏/误 GC; 错 status/condition → 上游误判	只有当 reconcile 逻辑能识别并重建/更正时才可自恢复	修正 controller 逻辑和现有对象, 评估误删/外部副作用后重新 reconcile

学习路径

阶段	关键项目	应该掌握什么
API 基础	kubernetes, kubectl, apimachinery	object meta、GVK/GVR、watch、resourceVersion、server-side apply
Client 基座	client-go, sample-controller	informer、lister、workqueue、rate limit、reconcile 幂等
Operator 工具链	controller-runtime, kubebuilder, controller-tools	区分脚手架、marker/type 生成与运行时; 掌握 manager、cache、client、scheme、webhook、envtest
生产控制器	KEDA, autoscaler, kueue, gateway-api	状态机、finalizer、条件、扩缩、队列、跨 namespace 引用
平台组合	Argo CD, Karmada, vcluster, kcp	GitOps、多集群、虚拟集群、控制面复用
诊断与观测	prometheus-operator, kube-state-metrics, k8sgpt	metrics、events、health、AI Ops 解释层

核心工具链边界

client-go

client-go 是 Kubernetes API client 与 informer/workqueue 基座。学习它能理解控制器的底层机制: watch 如何变成本地 cache, 事件如何进 queue, reconcile 为什么必须幂等, rate limiter 如何避免失败风暴。

controller-runtime

controller-runtime 把 client-go 常用模式抽成 Manager、Controller、Reconciler、Cache、Client、Scheme、Webhook、Predicate 和 envtest。大多数现代 Operator 项目不直接手写底层 informer, 而是通过 controller-runtime 组织 reconcile。

kubebuilder

kubebuilder 是 CRD/controller 项目脚手架和代码生成路径：API type、marker、CRD YAML、webhook、RBAC、manager main、测试环境都从这里组织。它解决“项目怎样标准化”，不是替代 controller-runtime。

controller-tools

controller-tools 提供 marker 解析与 controller-gen 生成器，把 Go API types/markers 转换成 CRD、RBAC、webhook、deepcopy/object 和 apply-configuration 等资产。它既不负责项目脚手架，也不运行 reconcile loop。

和当前 AI Infra 页面的关系

当前 wiki 中很多项目本质上都是 controller/operator：

- agent-sandbox 和 [agentcube](#) 用 CRD 管理 sandbox/session。
- [openkruise-kruise](#)、[openkruise-rollouts](#)、[kruise-game](#) 说明 workload controller 可以扩展到 [kubernetes-workload-automation](#) 下的 workload enhancement、release governance 和 specialized workload。
- [gateway-api](#)、[agentgateway](#)、[kgateway](#)、[envoy-ai-gateway](#) 都依赖 Gateway API / CRD / controller 语义。
- [llm-d](#)、[kserve](#)、[ome](#)、[kubeai](#)、[aibrix](#) 把 model serving 变成 Kubernetes control plane。
- [gpu-operator](#)、[dra-driver-nvidia-gpu](#)、[hami](#) 把 GPU 生命周期、DRA 和 sharing 策略放进 K8s。

因此理解 controller-runtime / kubebuilder / client-go，是理解这些 AI Infra 项目的共同底座。

Kubernetes SIGs 维度拆分

更完整的候选清单见 [llm-d-kubernetes-sigs-candidate-map](#)。如果按工程维度而不是 SIG 名称拆，下一批最有价值的是：

维度	P0 项目	和当前 wiki 的关系
网络	external-dns	补 gateway-api 之外的 DNS/LB/Ingress 控制器实战。
存储 / Secret	secrets-store-csi-driver	补 cloud-native-security 、凭据注入和 CSI 侧的工程边界。
调度 / 资源	kueue , karpenter , scheduler-plugins	补 AI/HPC/batch workload queueing、节点弹性和 scheduler 扩展。
可观测 / 性能	metrics-server , prometheus-adapter , inference-perf	补 HPA/custom metrics 和 GenAI benchmark。
	kind , kubespray , cri-tools	

维度	P0 项目	和当前 wiki 的关系
计算 / Runtime		补本地测试集群、生产集群部署和 kubelet/CRI 边界。
API / Operator	controller-runtime , kubebuilder , controller-tools , cluster-api	把当前 controller 学习路径升级为正式架构页候选。
AI Infra 交叉	mcp-lifecycle-operator , kube-agentic-networking , lws , jobset	连接 mcp、Agent runtime、LLM serving 和分布式 workload API。

OpenKruise 补充视角

[openkruise-project-candidate-map](#) 把 OpenKruise 生态拆成 workload enhancement、release governance、specialized workload、observability 和 controller operation boundary。概念层统一放入 [kubernetes-workload-automation](#)，避免按项目维度散出太多概念页。这条线补的是“业务 workload API 如何比原生 Deployment/StatefulSet 更贴近生产语义”，和 controller-runtime/kubebuilder 的“如何写 controller”是互补关系。

选型提示

- 想理解控制器底层：读 client-go 和 sample-controller。
- 想写生产 Operator：用 kubebuilder 组织项目、controller-tools 生成资产、controller-runtime 实现控制循环；已有工程可直接组合后两者，无需重建脚手架。
- 想理解平台控制面：对比 KEDA、Kueue、autoscaler、gateway-api、prometheus-operator。
- 想理解 AI Infra on K8s：把 model serving operator、GPU operator、agent sandbox operator 放到同一个 reconcile 模型下看。

Kubernetes DRA Design Deep Dive

当前上游核验（2026-09-27）

本节区分当前官方证据与下文 2026-07 的 KEP 设计/状态笔记；[src-dra-driver-nvidia-gpu-architecture](#) 是 2026-06 的 raw-backed Source 快照，保留其历史仓库/HEAD 和 ASCII 图。当前默认分支 commit 不代表 release，基础 DRA 稳定也不等于所有 Kubernetes 扩展或 NVIDIA driver 功能都已稳定。完整设备栈边界见 [k8s-gpu-device-stack](#)。

项目	当前 commit	当前证据边界	
kubernetes	6c1c7702cf20	Kubernetes DRA API、scheduler allocation 与 kubelet Prepare/Unprepare；各扩展阶段按官方文档及该 commit 的 feature 定义单独核验	
dra-driver-nvidia-gpu	495bf4c59b94	NVIDIA GPU/ComputeDomain 驱动实现；通过 NVIDIA 仓库入口查询时返回此规范链接，不以 Kubernetes 的 feature stage 替代厂商支持矩阵	
能力 / feature gate		本次核验的阶段与版本	本页相关边界 / 官方依据
基础 DRA / DynamicResourceAllocation		Stable，自 Kubernetes v1.35	DRA 概览 ；基础 Claim/设备分配稳定不覆盖下列所有扩展
Device binding conditions / DRADeviceBindingConditions		Beta，自 v1.36，默认启用	延迟 Pod binding 以等待外部设备准备； 当前 feature 定义 ；仍需对应 driver/status 支持
Consumable capacity / DRAConsumableCapacity		Beta，自 v1.36，默认启用	多 Claim 消耗同一设备容量； DRA Features
Partitionable devices / DRAPartitionableDevices		Beta，自 v1.36，默认启用	通过共享计数器表达逻辑设备资源重叠； DRA Features
Prioritized list / DRAPrioritizedList		GA，自 v1.36；v1.37 锁定启用	按优先级尝试设备请求候选； 当前 feature 定义
Device taints / DRADeviceTaints		当前源码 v1.37 条目标记 GA、默认启用；v1.38 条目锁定启用	设备 taint/toleration 的阶段不同于相关附加规则； 当前 feature 定义 ，版本化源码条目不是未来 release 已发布的证明

能力 / feature gate	本次核验的阶段与版本	本页相关边界 / 官方依据
Optional node operations / DRAOptionalNodeOperations	Alpha, 自 v1.37, 默认关闭	driver 可声明跳过特定 node-local 操作, 不能推广为所有 GPU 都无需 Prepare; DRA Features

该表只列本页相关且已核验的能力, 不是完整 feature 清单。API 对象语义见 [DRA API Objects](#)。NVIDIA 侧须分别核对 [锚定 driver README](#) 和 [GPU Operator 26.7 管理路径](#): 独立仓库 GPU plugin 的支持说明与 Operator 管理版本的默认启用方式不能混用。下文 KEP 状态表属于历史演进记录, 不覆盖本节的当前阶段核验。

M5-C 中的职责边界

[k8s-gpu-device-stack](#) 是设备栈入口, 本页聚焦 [kubernetes-dra](#) 的 Claim、调度分配、节点准备与回收。

Kubernetes 提供 DRA API 和 scheduler allocation; driver 发布 ResourceSlice 库存及设备约束, 解释厂商配置并完成相应准备。DeviceClass 可由管理员或 driver 安装流程提供; [node-feature-discovery](#) 的节点标签不能替代它或 ResourceSlice。

kubelet 驱动节点侧 Prepare/Unprepare, [dra-driver-nvidia-gpu](#) 等插件操作具体设备, [cdi](#) 把已准备设备交给兼容 runtime 注入。[gpu-operator](#) 管理受管组件部署和升级, [karpenter](#) 管理节点容量; 两者都不是单次 DRA Claim 的通用分配器。是否支持特定扩展仍以页首 feature-stage 表和目标 driver/release 为准, 这里不重复或提升其阶段。

DRA Allocation / Binding Lifecycle

```

Admin / driver installation — create selectors + config —> DeviceClass
DRA driver - - publish inventory / attributes / capacities - ->
ResourceSlice
User / workload controller — create directly —> ResourceClaim
User — create ResourceClaimTemplate + referencing Pod —> API
Pod/template - - watch / reconcile - -> resourceclaim controller
resourceclaim controller — create owned Claim / record Pod claim reference
—> API

Pod + Claim + DeviceClass + ResourceSlice - - watch / scheduling input - ->
scheduler
PreFilter / Filter — evaluate feasible Node + devices —> select
compatible Node
Reserve — retain candidate allocation in memory —> PreBind
PreBind — API write —> Claim.status.allocation + reservedFor
| — no binding conditions —> continue binding

```

```

└─ binding conditions enabled + present → wait in PreBind
Claim allocation - - observed by external driver controller - -> device
preparation
driver controller ─ write device conditions → Claim.status.devices
device conditions - - observed ready / failed / timeout - -> PreBind result
PreBind success → Bind Pod to compatible Node
PreBind failure → Unreserve / cleanup this attempt - - retry feedback -
-> queue

```

图例：实线 — label → 表示有序调用、API 写入或节点动作，虚线 - - label - -> 表示发布、watch、反馈与重试。图按引用 Claim 的普通 Pod 路径展开：已经分配的 Claim 会被校验和复用，不能把每一轮都理解成覆盖 allocation；模板需要由 resourceclaim controller 实例化，不是 scheduler 直接把模板当作已存在的 Claim。API 写入者也分开：scheduler 负责本次分配与消费者 reservation，外部 driver/controller 按协议更新设备 conditions，resourceclaim controller 管理派生 Claim 与消费者生命周期。

锚定版本的 [DynamicResources 插件](#) 在 Reserve 记录候选，在 PreBind 的 bindClaim 写回 allocation/reservation，然后按启用的扩展与设备声明等待 binding conditions。没有该条件时直接继续 binding；出现 failure condition 或超时则本次绑定失败。后续 Filter/PostFilter 可按 Claim 的消费者限制决定是否清除失败 allocation，不能把 Unreserve 等同于无条件清空所有共享 Claim。

binding conditions 允许外部准备发生在 Pod binding 前；它不是节点侧 NodePrepareResources 的另一种名字。后者通常在 Pod 已绑定、kubelet 准备容器时发生。Claim 分配成功、外部条件就绪、Pod 绑定成功、节点准备成功和容器运行是不同检查点。对象语义与模板控制器分别见 [DRA API Objects](#) 和 [resourceclaim controller](#)。

kubelet Prepare / CDI / Release

```

Bound Pod + allocated/reserved Claim - - observed state - -> kubelet DRA
manager
kubelet ─ track Pod/Claim reference + checkpoint → node-local state
kubelet ─ NodePrepareResources → vendor DRA plugin
plugin ─ validate allocation / configure device / recover own state →
device + checkpoint
plugin ─ prepare CDI spec + return CDI device IDs → kubelet
kubelet ─ checkpoint prepared result / CRI device configuration →
runtime
runtime ─ resolve CDI spec / inject devices / start container → running
workload

Pod termination - - node reconciliation - -> kubelet drops this Pod
reference

```

```

└─ other local Pods still use Claim → retain shared prepared state
└─ no local users → NodeUnprepareResources → plugin cleanup
    └─ successful result → kubelet updates cache / checkpoint
Pod completion / deletion - - API watch - -> resourceclaim controller
controller ─ remove completed consumer reservation → Claim status
last consumer released ─ eligible deallocation / finalizer cleanup →
reusable Claim
owned Claim deletion ─ owner / GC lifecycle → Claim removed when cleanup
permits

```

两条终止分支分别处理节点状态和 API 状态，不是跨组件事务，也不保证总按图中文字顺序完成。当前 [kubelet DRA manager](#) 在 RPC 前后记录 Claim/Pod 与准备结果；其他本地 Pod 仍引用同一 Claim 时延迟 Unprepare。kubelet checkpoint 与 vendor plugin 自己的 checkpoint/config 是不同责任：重启恢复需同时对照 API、节点记录及真实硬件，不能把删除任一记录当作已完成设备释放。

当前 resourceclaim controller 在最后消费者从“仍使用”转为“不再使用”时，可清除相应 allocation 和不再需要的 finalizer，使保留的 Claim 以后重新分配。模板派生 Claim 随 owner 生命周期回收；用户直接创建的持久 Claim 对象可以保留，但保留对象不意味着永久保留同一设备。多消费者场景不能因其中一个 Pod 退出就清空全部 reservation 或拆掉共享准备状态。

图中 Prepare/Unprepare 是常规 node-local 路径。页首 DRAOptionalNodeOperations 仍为独立 Alpha 扩展：只有启用该能力且 allocation 携带合法 skip 声明时，kubelet 才跳过指定节点操作；这不是所有 GPU 默认行为。CDI 负责容器注入，不保证硬件准备、健康或隔离策略本身正确。正式 RPC 名称为复数 NodePrepareResources / NodeUnprepareResources，下方历史笔记的单数简写不应作为接口名使用。

DRA 失败与恢复路径

失败点 / 可见影响	自动恢复 / API 与节点边界	Operator 与人工处理边界
ResourceSlice 陈旧或 pool 不完整：设备不可选或节点准备发现不符	driver 重新发布完整库存，scheduler 根据相关更新重试；节点 plugin 仍须校验设备现状	Operator 可恢复受管 driver，但错误设备 identity/generation、权限或硬件丢失需修复
Allocation / reservation 并发冲突：PreBind status 写入失败	API 并发检查拒绝冲突写入，scheduler 重读重算并清理本次保留；不覆盖其他消费者	持续争用、错误 UID/字段所有权或自定义控制器乱写 status 需修正
Binding conditions 超时或失败：有 allocation 但 Pod 未绑定	PreBind 失败触发 Unreserve/重试；后续 deallocation 受 Claim reservation 条件限制，不保证立即换设备	修复外部准备 controller、设备条件/RBAC 协议或不可恢复资源故障；延长超时不能制造 Ready

失败点 / 可见影响	自动恢复 / API 与节点边界	Operator 与人工处理边界
Driver / kubelet 重启及 checkpoint 不一致：准备状态难以恢复	kubelet 重连插件并依据引用/准备记录重试；vendor plugin 按自己的幂等恢复规则核对硬件	Operator 重建进程不等于硬件回滚；损坏/丢失 checkpoint、升级不兼容需受控恢复
Prepare 或 Unprepare 失败：容器无法启动或节点清理未完成	kubelet 对可重试错误再次调用，插件需容忍重复 RPC 和部分完成；共享 Claim 按本地引用计数处理	检查配置、权限、设备占用和插件错误；永久错误需人工处置，不能仅删除 finalizer
CDI spec / runtime 不匹配：Claim 与 Prepare 成功但容器启动失败	修复配置后启动路径可重试；CDI 注入不负责重新选择设备	核对 CDI IDs、spec 路径、runtime 支持与 driver/toolkit 版本；Operator 只修复其管理范围
MIG / VFIO / ComputeDomain 部分变更：API、checkpoint 与硬件/IMEX 不一致	driver/controller 按具体模式重放或清理；API 重试没有通用硬件补偿保证	确认其他消费者后处理 MIG 实例、驱动绑定、域成员或排空节点；不能假设一次重启全部恢复
节点失联或非优雅关机：NodeUnprepare 可能未执行	控制面继续观察消费者和节点状态；节点返回后由 kubelet/plugin 重新协调，外部设备可能需独立清理	对无法返回的节点须核实设备/网络连接已隔离及外部状态已释放，再处理遗留 Claim；API 删除不证明旧节点已停止使用

故障应分别核对 Claim allocation/reservedFor/device conditions、Pod binding、kubelet checkpoint、vendor plugin 状态及 runtime 配置。上表是当前职责归纳；下方保留的 KEP 风险条目不能直接当成所有版本/driver 的通用恢复保证。

KEP 设计与演进（历史笔记）

以下保留 Kubernetes Dynamic Resource Allocation 的历史设计脉络。核心是 [sig-node/4381-dra-structured-parameters](#)，它把早期 [3063-dynamic-resource-allocation](#) 的 opaque driver 协商路线反转为主线：设备参数必须结构化地暴露给 scheduler 和 autoscaler，Kubernetes 才能可靠做调度和容量推理。逐个 KEP 的 Alpha/Beta/GA、是否实现和 feature gate 见 [kubernetes-keps-implementation-matrix](#)；当前阶段以页首核验表为准。

一句话定位

DRA 是 Kubernetes 对 GPU、NIC、FPGA、DPU、network-attached accelerator、可分区设备和共享容量的下一代设备资源模型。它不是 device plugin 的简单替代，而是把“设备发现、设备选择、claim 状态、节点准备、CDI 注入、scheduler 推理、autoscaler 模拟”放到同一个 API 体系里。

为什么 device plugin 不够

Device Plugin API 适合“节点本地、离散、可计数”的资源，例如 `nvidia.com/gpu: 1`。但新设备场景更复杂：

- 设备可能不在节点本地，而是通过 fabric 动态连接。
- 一个设备可能切成多个 partition，例如 MIG-like GPU。
- 多个容器或多个 Pod 可能共享同一已初始化设备。
- 用户需要选择设备属性，例如型号、内存、PCIe root、driver version、NUMA locality。
- scheduler 和 autoscaler 必须能知道“新增一个节点后是否能满足 claim”。

如果设备选择完全由 vendor driver 在调度后 opaque 决定，scheduler 只能猜，Cluster Autoscaler / [karpenter](#) 也无法模拟。

4381 的核心模型

历史模型按四个角色拆分：DRA driver 发布带 devices/attributes/capacities 的 ResourceSlice；用户/controller 创建带 requests/selectors/config 的 ResourceClaim 或模板；kube-scheduler 对照库存评估请求并写 allocation status；kubelet 调用 DRA plugin Prepare/Unprepare，再将 CDI 设备交给 runtime。当前完整顺序与模板实例化、条件等待的细分见上方两张图。

ResourceSlice 是 driver 发布的资源库存。每个 device 有名字、属性和 capacity。属性可以被 CEL selector 匹配，capacity 用 quantity 表达。

DeviceClass 是集群管理员定义的设备类别，承载通用 selector 和配置。用户侧 ResourceClaim 引用它，并可继续补充 request selector 和 config。

ResourceClaim 是用户或 controller 要的资源。它的 spec 不变，status 由系统写入 allocation result、reservation、设备结果等状态。

结构化参数的关键价值

早期 3063 让 DRA driver 通过 control-plane controller 参与 allocation。问题是 scheduler / autoscaler 看不懂 driver 的自定义逻辑。4381 的结构化参数让 Kubernetes 至少能理解：

- 有哪些设备。
- 设备在哪些节点或资源池。
- 每个设备有哪些标准化或 vendor-specific 属性。
- request 如何用 selector 表达过滤条件。
- claim 是否已经被 allocated / reserved。
- allocation 结果能否被 kubelet 和 driver 重放。

这不是为了让 Kubernetes 理解所有 vendor 细节，而是把“调度必须知道的部分”变成 API 结构，剩下的配置参数仍可 opaque 地传给 driver。

Scheduler 插件路径

DRA scheduler plugin 不是只在 Filter 阶段做一次检查。它覆盖多个 extension point:

阶段	作用
EventsToRegister	注册 ResourceClaim、ResourceSlice、DeviceClass 等事件，并通过 queueing hints 精准唤醒相关 Pod。
PreEnqueue	快速检查 Pod 引用的 claim 是否存在，不存在就先不要进入正常调度。
PreFilter	收集 claim、class、slice、已分配资源和 in-flight allocation，准备高效 filter。
Filter	判断候选节点是否能满足 claim，执行 selector/capacity/match-attribute 等检查。
Reserve	节点已选定后，在内存里计算 allocation result。
PreBind	在独立 goroutine 中把 allocation/reservation 写回 ResourceClaim status。
Unreserve	绑定失败或调度失败时释放 reservation，避免 deadlock。

这个设计把昂贵或阻塞 API 写操作放到 PreBind 旁路，减少主 scheduling cycle 的阻塞。

ResourceClaim 状态机

DRA 的状态核心在 ResourceClaim.status:

历史状态模型依次描述: unallocated; scheduler 选择 node/device 后 allocated; reservedFor 纳入消费者后被 Pod 或其他 consumer 使用; 消费者退出后进入 deallocated/reusable。它概括的是控制面生命周期，不应把其中一条状态转换当成所有节点准备状态已经同步释放，具体回收边界见上方 kubelet Prepare / CDI / Release。

几个设计点很关键:

- allocation 是否非空决定 claim 是否已分配。
- reservedFor 决定 claim 是否正被某些 consumer 使用。
- 多 scheduler 并发时，status update conflict 是正常同步机制。
- Unreserve 必须释放 reservation，否则两个 Pod 各占一个 claim 等另一个 claim，会形成永久 deadlock。
- kube-controller-manager 清理完成 Pod 的 reservation，并释放不再使用的 allocation。

kubelet 和 DRA plugin

kubelet 通过 plugin registration 发现 DRA kubelet plugin。Pod 绑定到节点并且 claim 已 allocated/reserved 后，kubelet 调用：

- NodePrepareResource：让 driver 在本节点准备设备，返回 CDI device IDs 或等价注入信息。
- NodeUnprepareResource：Pod 不再使用资源时释放节点准备状态。

这条路径的关键是：scheduler 决定“用哪个设备”，kubelet/driver 负责“把设备准备好并注入容器”。如果设备在 scheduler 决定后消失，kubelet plugin 必须二次确认，Pod 可能保持无法启动直到资源恢复或被重新处理。

DRA 与 Autoscaler

DRA 最重要的教训来自 3063 到 4381 的路线变化：Cluster Autoscaler 需要模拟未来节点上的资源可用性。对于 node-local 设备，如果 claim 参数 opaque，autoscaler 无法知道加哪类节点能满足 Pod。

结构化参数让 autoscaler 至少可以读取 ResourceSlice 模型和 claim selector，判断“创建某个 node group 的节点是否可能让 pending Pod 调度成功”。这也是 DRA 为什么不能只停留在 driver 自定义 allocation 的原因。

关键扩展 KEP

KEP	解决的问题	设计意义
5007-device-attach-before-pod-scheduled	设备 attach/初始化可能是异步的，Pod 不应先绑定再失败。	把 device binding conditions 放入 scheduler PreBind 等待路径。
5075-dra-consumable-capacity	设备不一定是离散实例，也可能有可消费容量。	支撑 GPU memory、带宽、license、共享 buffer 等容量型设备。
5941-dra-shared-consumable-capacity	多个设备共享一组 capacity。	处理多个 logical device 背后共享同一个物理资源池。
4815-dra-partitionable-devices	设备可以动态切分。	让 MIG-like 或 TPU-like partition 不再只能预先静态发布。
4816-dra-prioritized-list	用户可接受多种设备配置。	用优先级备选列表表达“首选 A，不行则 B”。
5055-dra-device-taints-and-tolerations	设备健康或策略需要暂时排除。	将 node taint 思路下沉到 device 层。

KEP	解决的问题	设计意义
6080-dra-derived-attributes	不同 driver 属性表达不一致。	用派生属性对齐调度可用的 topology/compatibility 语义。

失败模式和风险

风险	解释	设计处理
stale ResourceSlice	driver 发布的设备状态过期。	kubelet plugin 在 NodePrepare 时二次确认。
scheduler filter 成本高	CEL、设备数量、claim 数量、match attributes 会放大计算成本。	Filter timeout、queueing hints、scheduler metrics。
claim 并发 reservation	多 scheduler 或多个 Pod 争同一 claim。	ResourceVersion/UID/status conflict 作为同步点。
node reboot / kubelet restart	旧 ResourceSlice 可能残留。	kubelet 启动时删除本节点 ResourceSlices，等待 driver 重建。
opaque 参数过多	autoscaler 和 scheduler 无法推理。	structured parameters 作为 GA 主线，opaque config 仅用于 driver 准备。
non-graceful node shutdown	NodeUnprepare 不一定被调用。	driver 需要在 Deallocate 或外部状态中恢复。

和 GPU 生态的关系

- [device-plugin](#) 仍适合简单离散资源；DRA 适合需要属性、claim、共享、拓扑、partition、跨 Pod 生命周期的资源。
- [cdi](#) 是设备注入 runtime 的标准表达，DRA kubelet plugin 可以返回 CDI devices。
- [dra-driver-nvidia-gpu](#) 是观察 DRA 在真实 GPU/MIG/VFIO 场景落地的重要项目。
- [node-feature-discovery](#) 仍然有价值，但它更多发布节点能力标签；DRA 发布的是可分配设备库存。
- [k8s-gpu-device-stack](#) 应该把 DRA 作为下一代 GPU 资源抽象主线，而不是 device plugin 的旁支。

阅读顺序

1. 4381-dra-structured-parameters: DRA 主线设计。
2. 3063-dynamic-resource-allocation: 理解为什么 opaque control-plane allocation 被降级。
3. 4815 / 5075 / 5941: 理解 partition 和 capacity。
4. 5007: 理解 device attach/readiness 如何进入调度。
5. 5055 / 4816 / 6080: 理解生产化调度表达。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
4381-dra-structured-parameters	implemented / stable, 已实现/GA	v1.30 / v1.32 / v1.34	DynamicResourceAllocation, DRASchedulerFilterTimeout	ResourceSlice/ResourceClaim/DeviceClass 结构化参数, scheduler 和 autoscaler 可推理。
3063-dynamic-resource-allocation	withdrawn / alpha, 已撤回	v1.26 / - / -	DynamicResourceAllocation, DRAControlPlaneController	opaque control-plane allocation 路线, 因调度和 autoscaler 不可推理被 4381 取代。
5007-device-attach-before-pod-scheduled	implementable / beta, 仍在 beta	v1.34 / v1.36 / v1.37	DRADeviceBindingConditions	device binding/readiness 条件进入 scheduler PreBind 等待路径。
5075-dra-consumable-capacity	implementable / beta, 仍在 beta	v1.34 / v1.36 / v1.38	DRAConsumableCapacity	设备可表达可消费 capacity, 而不是只表达离散实例。
5729-resourceclaim-support-for-workloads	implementable / beta, 仍在 beta	v1.36 / v1.37 / -	DRAWorkloadResourceClaims	ResourceClaim 支持 workload 级消费者, 连接 DRA 和 PodGroup。
5941-dra-shared-consumable-capacity	implementable / alpha, 仍在 alpha	v1.37 / v1.38 / v1.39	DRASharedConsumableCapacity	多个设备共享同一 capacity pool。
5963-device-		v1.37 /	DRADeviceCompatibilityGroups	表达多设备组合兼容性。

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
compatibility-groups	implementable / alpha, 仍在 alpha	v1.38 / v1.39		
4815-dra-partitionable-devices	implementable / beta, 仍在 beta	v1.33 / v1.36 / -	DRAPartitionableDevices	动态或逻辑分区设备进入 ResourceSlice 模型。
4816-dra-prioritized-list	implementable / stable, GA 目标已达	v1.33 / v1.34 / v1.36	DRAPrioritizedList	用户可声明设备请求的优先级备选列表。
5055-dra-device-taints-and-tolerations	implementable / stable, GA 目标已达	v1.33 / v1.36 / v1.37	DRADeviceTaints, DRADeviceTaintRules	设备级 taints/tolerations, 支持健康和策略隔离。
4817-resource-claim-device-status	implementable / stable, GA 目标已达	v1.32 / v1.33 / v1.37	DRAResourceClaimDeviceStatus	分配结果和设备信息进入 ResourceClaim status。
5304-dra-attributes-downward-api	implementable / beta, 仍在 beta	v1.36 / v1.37 / v1.38	NA	将 DRA device attributes 暴露给 Pod 内 workload。
4680-add-resource-health-to-pod-status	implementable / beta, 仍在 beta	v1.31 / v1.36 / v1.37	ResourceHealthStatus	Device Plugin/DRA resource health 进入 Pod status。

追踪重点

- DRA API 版本从 beta 到 GA 的字段稳定性。
- scheduler DRA plugin 的性能数据，尤其是 Filter latency 和 queueing hint 命中率。
- Cluster Autoscaler / [karpenter](#) 对 DRA structured parameters 的真实支持。
- NVIDIA、其他 GPU/NIC/DPU driver 是否采用 ResourceSlice/ResourceClaim 标准路径。
- DRA 与 Workload/PodGroup、Topology Manager、NUMA、CDI 的联动是否形成闭环。

Kubernetes GPU / Device Stack 项目地图

当前上游核验 (2026-09-27)

以下 commit 是执行时官方默认分支快照，不代表 release 或整套 GPU 能力的成熟度。[src-node-feature-discovery-architecture](#)、[src-gpu-operator-architecture](#)、[src-k8s-device-plugin-architecture](#)、[src-dra-driver-nvidia-gpu-architecture](#) 与 [src-hami-architecture](#) 保留 2026-06 的 raw-backed Source 及原始 ASCII 图；相关 KEP 笔记保留 2026-07 的设计/演进语境。当前 DRA 基础能力与扩展阶段分别见 [kubernetes-dra-design-deep-dive](#)。

项目	当前 commit	当前职责 / allocation 边界	M5-C 层
node-feature-discovery	386fd a4332 ba	worker/master 发布节点能力与 labels, NodeFeature/NodeFeatureRule 表达发现和规则, topology-updater/GC 管拓扑数据及清理；不执行每次 Pod 设备分配	Discovery / capability
gpu-operator	60526 e35ef ee	管理 GPU 软件组件生命周期；官方 26.7 文档区分 Device Plugin 的 ClusterPolicy 路线与 DRA 的 GPUCluster 路线，后者不直接管理 GPU driver	Node software lifecycle
k8s-device-plugin	86142 cf1a9 3f	kubelet 注册、ListAndWatch/health、Allocate 与 extended resources；按配置返回 env/mount/CDI 设备交接信息	Device Plugin allocation / injection handoff
dra-driver-nvidia-gpu	495bf 4c59b 94	DRA GPU 与 ComputeDomain 插件/控制面，发布设备库存并执行节点 Prepare/Unprepare；GPU 配置能力和 ComputeDomain 支持范围需按安装版本分别判断	DRA driver / node preparation
hami	a2dd1 91b2e 7f	scheduler/extender 计算设备共享与 reservation, annotation 交接给 device-plugin Allocate, HAMi-core 在容器侧实施相应隔离	Sharing / allocation handoff / isolation

官方依据：[NFD 当前介绍](#)、[GPU Operator 26.7 DRA 管理路径](#)、[NVIDIA Device Plugin README](#)、[NVIDIA DRA README](#) 与 [HAMi FAQ](#)。通过 NVIDIA/k8s-dra-driver-gpu 查询当前 HEAD 时，GitHub 返回的规范 commit 链接仍位于 kubernetes-sigs/dra-driver-nvidia-gpu，这里采用实际返回的链接。

共存与版本边界：GPU Operator 26.7 文档规定集群使用 GPUCluster 或 ClusterPolicy 之一，不支持两者并存或直接原地迁移；HAMi FAQ 要求避免同一节点上的插件争用同一 GPU resource。上述 NVIDIA DRA 仓库 README 仍区分受支持的 ComputeDomain 与未正式支持、默认关闭的独立 GPU plugin，不能用它推定

Operator 26.7 所管理 driver 的相同默认值。部署判断须绑定具体 release、Helm 配置和 feature gates，不能只看 [device-plugin](#)、[kubernetes-dra](#)、[cdi](#) 或 [gpu-sharing](#) 名称。

这页把 [src-k8s-gpu-device-plugins-stars](#) 从 star list 整理成 Kubernetes GPU / 异构设备资源层地图。核心结论：LLM serving 的 GPU 底座已经不只是“把 /dev/nvidia0 挂进 Pod”，而是 driver/operator、container runtime、device discovery、DRA/CDI、sharing、scheduler、observability、diagnostics 和 fake device 测试环境的组合。

D1 · GPU Device Stack

DISCOVERY / CAPABILITY

NFD: NodeFeature / NodeFeatureRule / labels / NodeResourceTopology

- - published capability: informs placement / operator policy - ->

NODE SOFTWARE LIFECYCLE

GPU Operator: reconcile selected policy and managed components

Driver / runtime prerequisites must match the chosen installation path

|— configure Path A —> ClusterPolicy / Device Plugin

| extended resource / kubelet Allocate

| — env / mounts / optional CDI —> runtime

|— configure Path B —> GPUCluster / DRA driver

DeviceClass + ResourceSlice / ResourceClaim

scheduler allocation / kubelet

NodePrepareResources

— CDI device handoff —> runtime

Path A OR Path B: alternative GPU Operator management modes

Telemetry / health - - observed state - -> operators / node recovery

图例：实线 — label —> 表示有序控制动作、API 调用或节点交接；虚线 - - label - -> 表示发现发布、reconcile、watch/health 反馈及重试。上层发现信号供控制器决策，不表示每次分配都同步调用 NFD。 [node-feature-discovery](#) 发现能力， [gpu-operator](#) 管理组件生命周期； [device-plugin](#) / [kubernetes-dra](#) 承担各自的 allocation 路径， [cdi](#) 位于设备注入和 runtime 交接处，不是第三种 scheduler 或 allocator。

图中的分叉表示选择，不表示把两条 GPU 管理路径同时安装。按 [GPU Operator 26.7 文档](#)，ClusterPolicy 与 GPUCluster 不能在同一集群并存，也不支持从 ClusterPolicy/独立 DRA Helm 安装直接原地迁移到 GPUCluster。GPUCluster 不管理 GPU driver，需独立 NVIDIA Driver 或预装 driver，并事先准备 CDI-compatible runtime；不能把 ClusterPolicy 的 driver/toolkit 管理默认套到另一条路径。即使脱离 Operator 手动部署，也须确保同一物理设备及资源名有明确分配所有权，避免被不同 allocator 重复分配。

D2 · Device Plugin Allocation

```
Device plugin — register endpoint / resource name —> kubelet
Device plugin - - ListAndWatch: devices / health - -> kubelet device
manager
kubelet - - Node status publication - -> API: capacity / allocatable

Pod extended-resource request + observed Node allocatable
— scheduler Node fit / bind —> Pod assigned to Node
- - Pod watch - -> kubelet
kubelet — select device IDs / Allocate RPC —> device plugin
device plugin — Allocate response: env / mounts / device specs / CDI IDs
—> kubelet
kubelet — CRI container configuration —> container runtime
runtime — apply configured device injection / start —> workload

Device health change - - ListAndWatch update - -> allocatable update /
future scheduling
```

这是传统 extended-resource 路径：scheduler 先按节点资源数量等条件选 Node，节点侧 kubelet/device plugin 完成具体设备分配与容器配置。NVIDIA 的 device-list strategy 决定 envvar、volume-mounts、CDI annotation 或 CDI CRI 交接方式；不是所有安装都走 CDI。健康设备变为 unhealthy 时，kubelet 减少相应 allocatable，capacity 保持原值；这不等于自动迁移或修复已使用故障设备的容器。参见 [Kubernetes Device Plugins](#) 和 [NVIDIA plugin 配置](#)。

D3 · Discovery / Operator Control Loops

```
NFD worker — scan hardware / OS / local feature sources —> feature set
feature set - - publish NodeFeature - -> Kubernetes API
NodeFeature / NodeFeatureRule - - watch / evaluate rules - -> NFD master
NFD master — update labels / configured annotations or resources —> Node
NFD topology-updater - - publish topology - -> NodeResourceTopology
NFD GC - - observe stale / removed nodes - -> cleanup obsolete NFD objects
Node capability / topology - - consumed state - -> scheduler / operator /
policy

Admin — apply ClusterPolicy OR GPUCluster —> Kubernetes API
desired policy / operand changes - - watch / enqueue - -> GPU Operator
reconcile
GPU Operator — create / update managed operands —> DaemonSets /
```

```

Deployments / config
operand readiness / errors - - observed feedback - -> GPU Operator
GPU Operator — write conditions / status —> policy object
retryable error / further change - - requeue - -> GPU Operator reconcile

```

NFD 的 labels、feature 与 topology 数据是已观测状态，可能受扫描周期、API/watch 延迟或 worker 故障影响；NFD GC 清理过期对象不等于硬件故障恢复。Operator 的 Ready 描述所管理组件状态，不承诺每个 Pod 的 GPU 请求都能满足。分配哪块设备、Prepare 是否成功以及容器内隔离是否生效，仍属于后续路径。NFD 组件与数据对象见 [当前 Introduction](#)。

D4 · DRA Resource 生命周期

阶段	对象 / 操作与负责方	必须区分的状态
类别与库存	管理员或 driver 安装流程提供 DeviceClass；DRA driver 发布 ResourceSlice 设备库存、属性和容量	DeviceClass 是选择/配置规则，ResourceSlice 是库存；NFD labels 不能替代设备库存
请求	用户或 workload controller 创建 ResourceClaim；也可通过 ResourceClaimTemplate 为 Pod 派生 Claim	请求存在不等于已分配，模板本身不是一次 allocation
分配与绑定	scheduler 根据 Claim、class、slice 与 Node 约束选择设备并写 allocation/reservation；启用相关扩展时等待 binding conditions，再绑定 Pod	已分配、已绑定、外部设备就绪是不同状态；条件等待依赖相应 feature 和 driver
节点准备	kubelet 调用 DRA plugin 的 NodePrepareResources；driver 配置设备并维护节点侧状态，返回 CDI 设备信息	Claim status 成功不证明节点硬件配置/CDI 已完成；可选 node operations 有独立阶段限制
Runtime 交接	kubelet 经 CRI 把 CDI device IDs 交给兼容 runtime，由 runtime 按 CDI spec 应用设备配置	CDI 是注入接口，不决定 Claim 分配或共享额度
终止与回收	kubelet 在节点不再需要该 Claim 时执行 NodeUnprepareResources；控制面按 Claim/消费者生命周期处理 reservation、删除及 allocation 回收	节点 unprepare 不等于所有消费者已退出或持久 Claim 已释放；不能因一个 Pod 退出就盲删共享状态

完整状态所有权、binding conditions、Prepare/checkpoint/CDI 与回收细节见 [kubernetes-dra-design-deep-dive](#)；官方流程见 [How DRA Works](#)。基础 DRA 自 v1.35 stable，各扩展和厂商 driver 的支持阶段仍须分别核

Sharing / Isolation Overlay

[hami](#) 的经典路径横跨调度、分配交接和容器隔离：scheduler/extender 根据节点设备视图计算 memory/core/count 与设备 reservation，再写 Pod allocation annotations；device-plugin 在 kubelet Allocate 时读取这些信息并返回设备、环境与挂载配置；HAMi-core 在容器内执行相应 CUDA 调用拦截和限制。[hami.io/vgpu-devices-to-allocate](#) 与 [hami.io/bind-phase](#) 是分配进度协议，不是业务 Ready 标志，也不应由用户手工伪造。参见 [HAMi Protocol](#)。

同一节点不能让 HAMi 与 NVIDIA 官方 device plugin 同时争用 [nvidia.com/gpu](#)；GPU Operator 可负责其他软件组件，但要按目标版本关闭冲突的 allocation operand。HAMi-core 的软件拦截、MIG 的硬件分区以及 time-slicing 的共享策略也不能视为等价隔离保障，需验证实际调用路径、显存限制和尾延迟。[HAMi FAQ](#) 说明了插件共存及软件隔离边界。[HAMi-DRA](#) 改变的是 Claim/设备分配路径，仍可使用同一 HAMi-core 隔离层；切换分配 API 不会自动升级隔离强度。

D5 · 失败边界

失败点 / 可见影响	API、节点或 Operator 自动恢复边界	需要人工确认 / 修复的情况
NFD 状态陈旧：labels/topology 与节点不符	worker 恢复扫描、master 更新与 GC 清理可重新收敛；消费者需等待新观测	检查 worker 权限、feature source、规则和节点实际硬件；修复标签来源，不能只反复调度
Operator operand 未就绪：driver/toolkit/plugin/validator 阻塞	reconcile 可重建或更新所管理对象并持续报告 conditions	内核/driver 不兼容、镜像/权限、安装模式冲突或外部 runtime 配置需修复；重建 Pod 不能保证硬件恢复
Device Plugin 注册、Allocate 或 health 失败：资源消失/容器无法启动	plugin 按实现重注册，kubelet 更新健康和 allocatable，并重试相应启动路径	检查 socket/API 版本、NVML/driver、资源名争用及 Allocate 错误；运行中 GPU 故障可能需要节点处置或重建 workload
ResourceSlice / Claim 状态漂移：库存过期、设备无匹配或 allocation 不一致	driver 发布新库存、scheduler/controller 重读与重试可处理暂时不同步	错 driver/device identity、pool generation、selector 或存量 Claim 与硬件变化冲突需核对；不盲改已分配 Claim status
Binding conditions 等待超时：设备已选但 Pod 未绑定	scheduler 依条件和超时路径结束本次尝试、清理相应保留状态并重试	外部准备 controller、状态权限或条件协议永久失败需要修复；单纯延长超时不产生设备就绪
NodePrepare / CDI / checkpoint 错误：Pod 已绑定但不能启动或清理	kubelet/plugin 对可重试错误重试，driver 按自身 checkpoint 恢复	核对 CDI runtime/spec、checkpoint 与真实设备状态；不直接删除 checkpoint/finalizer 来掩盖未完成清理

失败点 / 可见影响	API、节点或 Operator 自动恢复边界	需要人工确认 / 修复的情况
	节点状态；Operator 可恢复受管进程	
MIG / VFIO / ComputeDomain 部分变更：API 与节点/IMEX 状态不一致	各 driver/controller 按支持的幂等与恢复路径重算；API 重试本身不回滚硬件修改	按模式检查 MIG 实例、驱动绑定、IMEX/域成员；必要时排空节点后人工修复，评估其他消费者影响
HAMi annotation 交接失败：reservation 与 Allocate 进度不一致	scheduler/device-plugin 依协议更新 pending allocation 与完成状态，可重试的 API 错误按实现恢复	检查版本、编码、目标节点、锁/缓存和设备状态；不能伪造 success 或仅重写注解当作已分配
隔离或 overcommit 不符合预期：OOM、抢占干扰或尾延迟恶化	配额记账与进程重启不能自动修复软件拦截绕过、配置错误或硬件容量不足	验证 HAMi-core 注入、调用兼容性、共享倍率与真实负载；按隔离目标调整配置或设备模式

失败矩阵是按职责归纳的排障边界，不承诺所有实现都可自动恢复。先判断故障停在 API 记账、节点准备、组件生命周期还是容器隔离，再对照 D2 – D4 和具体 driver 的 conditions、events、日志与设备状态。

一句话分层

层	代表项目	要解决的问题
节点发现	node-feature-discovery	发布能力标签、规则结果与拓扑信号，供 placement/operator 使用
驱动与节点软件栈	gpu-operator , NVIDIA container toolkit, DCGM	节点怎样安装、升级和暴露 GPU 软件栈
设备暴露	k8s-device-plugin , Intel device plugins, host device plugin	kubelet 如何发现并分配专用设备
声明式设备分配	kubernetes-dra , dra-driver-nvidia-gpu	Pod 如何声明更复杂的设备需求和分配结果
Runtime 注入	cdi , NVIDIA container toolkit	怎样把已分配设备的 runtime 配置交给容器引擎
GPU sharing / vGPU	hami , Volcano vGPU, gpushare, vgpu-scheduler	多租户如何切分、调度和隔离 GPU
调度与队列	kueue , scheduler-plugins , autoscaler	GPU batch / AI workload 如何排队和扩缩

层	代表项目	要解决的问题
观测与诊断	DCGM exporter, GPUD, fake-gpu	怎么发现 GPU 健康、利用率和测试调度逻辑

核心项目边界

[node-feature-discovery](#)

NFD 提供 worker/master、topology-updater 与 GC，发布硬件/系统能力、规则标签及资源拓扑。它帮助 scheduler 和 Operator 识别节点，不维护每个 Pod 的设备 allocation，也不替代 Device Plugin 或 DRA driver。

[k8s-device-plugin](#)

NVIDIA 官方 GPU device plugin 是传统路径的基线：通过 NVML/CUDA discovery 向 kubelet gRPC 注册 [nvidia.com/gpu](#) 等资源，并在 Allocate 阶段用 env、volume、CDI 等方式交接设备配置。它可提供配置支持的 MIG/time-slicing/MPS 资源暴露，但不负责 driver 生命周期、完整多租户治理或模型 serving。

[gpu-operator](#)

GPU Operator 是生产集群运维入口：传统 ClusterPolicy / NVIDIA Driver 路径管理 driver、container toolkit、device plugin、DCGM、MIG manager、validator 等组件；当前 GPUCluster 提供另一条 DRA 管理路径，依赖独立 driver/runtime 准备。它解决“节点 GPU 软件栈怎样持续正确”，不决定单次 Pod 获得哪块设备。

[hami](#)

HAMi 是 GPU sharing / vGPU 方向的重要样本。它通过 webhook、scheduler extender、device plugin 和多厂商 device abstraction，把 GPU 显存、算力、切分和调度策略放到 K8s 资源模型旁边。它解决“多个 workload 如何共享 GPU”，但也带来调度、隔离、观测和兼容性的复杂度。

[dra-driver-nvidia-gpu](#)

NVIDIA DRA driver 把设备库存与节点准备接入 ResourceClaim / ResourceSlice 分配模型，GPU 与 ComputeDomain 路径具有各自生命周期。基础 DRA 已稳定，但动态设备、sharing、VFIO 等能力仍需按 Kubernetes 扩展和厂商 release 分别评估，不能无条件替代现有 device plugin 部署。

和 LLM serving 的关系

llm-inference 会把 GPU 资源层进一步拉进架构设计：

- Prefill/Decode 分离要求不同 GPU 池承担不同负载。
- KV cache offload 让显存、CPU 内存、NVMe 和网络都成为资源决策。

- LoRA / adapter serving 需要更细的内存与模型资产管理。
- GPU sharing 会影响 tail latency, 不能只看平均利用率。
- DRA/CDI 会影响 serving operator 如何把“设备分配结果”交给 runtime。

所以 GPU stack 不应只作为运维背景, 而应和 [llm-d](#)、[aibrix](#)、[kserve](#)、[gpustack](#)、[kubeflow](#)、[kueue](#)、[node-feature-discovery](#) 等项目分层比较, 区分 model serving、准入与节点发现。

选型提示

- 生产 NVIDIA GPU 集群基础栈: 先看 [gpu-operator](#) + [k8s-device-plugin](#)。
- 多租户共享和 vGPU: 看 [hami](#), 同时关注隔离和性能尾延迟。
- 下一代设备 API: 看 [kubernetes-dra](#) + [dra-driver-nvidia-gpu](#) + [cdi](#)。
- LLM serving 平台: GPU 栈只是底座, 还需要 llm-inference、inference-routing、[model-serving-operator](#) 和可观测能力。

Kubernetes HPA Autoscaling Design

这页拉出 sig-autoscaling 中最值得详细讲解的一组设计文档：HPA tolerance、container resource metrics、pod selection accuracy、scale from zero、external metric fallback，以及 Cluster Autoscaler 的 attach limit integration。逐个 KEP 的 Alpha/Beta/GA、是否实现和 feature gate 见 [kubernetes-keps-implementation-matrix](#)。

一句话定位

这组 KEP 的共同目标是让 HPA 从“基于粗粒度全局参数和 selector 的普通副本数调节器”，变成更可解释、更可靠、更适合高成本 workload 的 autoscaling 控制面。

HPA 的设计问题

HPA 的核心公式是根据当前指标与目标指标的 ratio 推导 desired replicas。问题在生产里通常不在公式本身，而在这些边界：

- 10% 全局 tolerance 对小 workload 和大 workload 都不合适。
- Pod selector 可能选中同 label 但不属于目标 workload 的 Pod。
- sidecar-heavy workload 中，Pod 总资源指标会稀释真正主容器的信号。
- 外部指标不可用时，HPA 缺少保守 fallback。
- replicas: 0 既可能表示用户手动暂停，也可能表示自动 scale-to-zero，语义混乱。
- Cluster Autoscaler 需要理解 scheduler 之外的容量约束，例如 CSI attach limit。

Configurable HPA Tolerance

4951-configurable-hpa-tolerance 给 HPAScalingRules 增加 tolerance 字段。它的设计非常小，但影响很大：

HPA

```
spec.behavior.scaleUp.tolerance
spec.behavior.scaleDown.tolerance
```

过去 tolerance 是 kube-controller-manager 的全局参数，默认 10%。这意味着：

- 100 个副本的 workload，7% 负载上涨不会扩到 107。
- GPU/LLM serving workload 的每个副本很贵，scale-down tolerance 又不能太激进。
- scale-up 和 scale-down 通常应该有不同策略。

新字段让 workload owner 可以对单个 HPA 配置 scale-up 和 scale-down 的容忍区间。字段缺省时仍使用全局默认值，因此兼容性好。

设计取舍：它没有改变 HPA 算法，只是把一个已有参数从 cluster-wide 下放到 workload-level。这种 KEP 风险小、收益明确。

Container Resource Autoscaling

1610-container-resource-autoscaling 解决的是 sidecar 干扰。传统 resource metrics 以 Pod 为单位聚合，适合“Pod 中只有一个主要业务容器”的假设。但现在常见 Pod 包含：

- service mesh sidecar
- logging/metrics sidecar
- model warmup sidecar
- proxy 或 adapter sidecar

如果 HPA 读 Pod 总 CPU/memory，主容器压力会被 sidecar 稀释，或者 sidecar 抖动会误导扩缩容。

Container resource metrics 让 HPA 指定某个 container 的 CPU/memory 作为 scaling 信号。它把指标目标从“Pod 总和”改成“业务容器”。这对 LLM inference、agent runtime、带 proxy 的 serving workload 很关键。

Pod Selection Accuracy

5325-hpa-pod-selection-accuracy 的问题更隐蔽：HPA 通常先用 target workload 的 label selector 找 Pod。但 label 不等于 ownership。不同 controller、canary、历史 Pod 或同 label workload 都可能被选中。

它提出 SelectionStrategy：

策略	含义
LabelSelector	当前行为，只按 label selector。
OwnerReference	先按 label selector，再沿 owner reference 过滤，只保留目标 workload 拥有的 Pod。

OwnerReference 需要 controller 做额外 API 查询，因此设计里包含 cache、TTL、fallback 行为。如果 ownership 检查失败，系统回退到默认 selector 行为，避免因为 RBAC 或 API 错误导致 autoscaling 完全失效。

这条 KEP 的本质是：HPA 的准确性依赖“指标分母是否正确”。如果 Pod 集合错了，再精细的 metrics 和算法都没用。

Scale From Zero

2021-scale-from-zero 让 HPA 支持基于 object/external metrics 从 0 副本启动。它明确不支持 resource metrics 从 0 启动，因为 CPU/memory 需要运行中的 Pod 才能测量。

关键设计点：

- 允许 minReplicas: 0。
- 引入 HPA status condition 表达 ScaledToZero。
- 当 HPA 自己把 workload 从 1 缩到 0 时记录条件。
- 当外部/object metric 表明需要恢复时，从 0 扩到 1 并清掉条件。

为什么需要 condition? 因为在 Kubernetes 里 replicas: 0 也可能是用户手动暂停 workload。如果 HPA 看到 0 就自动拉起，会破坏用户意图；如果永远不拉起，又无法实现 scale-from-zero。ScaledToZero 用状态区分“这是 HPA 自动缩到 0”还是“用户手动设置为 0”。

External Metric Fallback

5679-external-metric-fallback 处理外部指标不可用。外部指标经常来自 CloudWatch、Prometheus adapter、队列系统、第三方监控或 SaaS API，这些系统不在 Kubernetes 控制面直接掌控内。

设计选择：

- fallback 只用于 external metrics。
- 每个 external metric 独立配置 fallback。
- fallback 使用固定 replicas，而不是 last-known-good 或自动估算。
- failure threshold 用 duration，而不是失败次数。
- fallback desired replicas 进入 HPA 原有多指标算法，和其他指标取 max。

duration-based 设计很实际：HPA sync period 可以是 15s，也可以被不同集群改成 30s。用“连续失败 3 次”会在不同集群表现不同；用“失败持续 180 秒”更稳定。

HPA 决策链路

```
metric source
|
+-- resource / container resource / pods / object / external
|
pod selection
|
+-- label selector
+-- optional owner-reference filtering
|
replica calculation
|
+-- target ratio
+-- per-direction tolerance
+-- scale behavior policies
```



```

|
multi-metric merge
|
+-- max desired replicas
+-- fallback desired replicas can join here
|
scale subresource update

```

Cluster Autoscaler Attach Limit

5030-attach-limit-autoscaler 不在 HPA 内部，但属于 autoscaling 设计链。它解决 CSI volume attach limit 对 scale-up 模拟的影响。

如果 scheduler 因 volume attach limit 无法放置 Pod，Cluster Autoscaler 需要知道新增什么节点才有用。否则可能出现：

- autoscaler 加了节点，但 Pod 仍因为 attach limit pending。
- scheduler 和 autoscaler 对“节点是否能承载 Pod”的模拟不一致。
- scale from zero node group 时缺少真实节点 attach limit 信息。

这个 KEP 的意义是把 autoscaler 和 scheduler 的可行性判断拉近。类似问题未来也会出现在 DRA、PodGroup、topology-aware workload scheduling 上。

关键失败模式

失败模式	设计处理
tolerance 太粗导致扩容慢	per-HPA / per-direction tolerance。
sidecar 稀释业务指标	container resource metrics。
label selector 选错 Pod	OwnerReference selection strategy。
0 副本和手动暂停混淆	ScaledToZero condition。
外部指标故障导致无法扩容	per-metric external fallback。
autoscaler 加节点但仍不可调度	attach limit 纳入 autoscaler 模拟。

和指标生态的关系

- [metrics-server](#) 提供 resource/container resource metrics，是 HPA resource path 的基础。
- [prometheus-adapter](#) 提供 custom/external metrics，是 scale-from-zero、external fallback、business metric scaling 的关键路径。

- [llm-d-workload-variant-autoscaler](#) 是 deprecated 的历史设计快照，只用于迁移/设计研究，不再作为当前集成建议。llm-d 新部署使用 [EPP metrics → KEDA Prometheus scaler → HPA](#)；[metrics-server](#) / [prometheus-adapter](#) 仍是指标管道参考，不等于采用 WVA。

阅读顺序

1. 4951-configurable-hpa-tolerance: 最小但最实用的 API 下放。
2. 1610-container-resource-autoscaling: 理解 sidecar-heavy workload 的指标边界。
3. 5325-hpa-pod-selection-accuracy: 理解 HPA 为什么可能读错 Pod 集合。
4. 2021-scale-from-zero: 理解 0 副本状态语义。
5. 5679-external-metric-fallback: 理解外部指标失败策略。
6. 5030-attach-limit-autoscaler: 理解 autoscaler 和 scheduler 模拟一致性。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
4951-configurable-hpa-tolerance	implementable / stable, GA 目标 v1.37	v1.33 / v1.35 / v1.37	HPAConfigurableTolerance	HPAScalingRules.tolerance 下放到单个 HPA 的 scaleUp/scaleDown。
853-configurable-hpa-scale-velocity	implemented / stable, 已实现/GA	- / - / -	-	HPA behavior policies、stabilization window、scale velocity。
2702-graduate-hpa-api-to-GA	implemented / stable, 已实现/GA	- / - / v1.23	-	HPA v2 API GA。
1610-container-resource-autoscaling	implemented / stable, 已实现/GA	v1.20 / v1.27 / v1.30	HPAContainerMetrics	HPA 使用指定 container 的 CPU/memory 指标。
117-hpa-metrics-specificity	implemented / stable, 已实现/GA	- / - / -	-	custom/external metrics 支持 label selector。

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
5325-hpa-pod-selection-accuracy	implementable / alpha, 仍在 alpha	v1.35 / v1.36 / v1.37	HPASelectionStrategy	OwnerReference selection strategy, 过滤非目标 workload Pod。
2021-scale-from-zero	implementable / beta, 仍在 beta	v1.16 / v1.37 / x.y	HPAScaleToZero	HPA 基于 object/external metrics 从 0 恢复, 并用 condition 标记自动缩零。
5679-external-metric-fallback	implementable / alpha, 仍在 alpha	v1.36 / v1.37 / v1.38	HPAExternalMetricFallback	external metric 连续失败后用固定 fallback replicas 进入 max 合并。
5030-attach-limit-autoscaler	implementable / beta, 仍在 beta	v1.35 / v1.37 / v1.38	VolumeLimitScaling	Cluster Autoscaler scale-up 模拟纳入 CSI attach limit。

追踪重点

- HPA v2 API 新字段是否进入 stable, 旧版本 conversion 是否完整。
- per-metric fallback 状态是否足够可观测, 是否有 events/conditions/metrics。
- OwnerReference filtering 的 cache 是否会带来 API server 压力。
- scale-from-zero 与 KEDA、Karpenter、queue-based autoscaler 的边界。
- DRA、PodGroup、volume attach、topology 等复杂约束是否继续进入 autoscaler 模拟。

Kubernetes In-Place Pod Resize Design

这页拉出 sig-node 和 sig-scheduling 交叉的一组重要设计文档：1287-in-place-update-pod-resources 是主线，5419、5526、5554、6122、5836 继续扩展到 Pod-level resources、CPU/Memory manager、static CPU policy、延迟暴露和 scheduler preemption。逐个 KEP 的 Alpha/Beta/GA、是否实现和 feature gate 见 [kubernetes-keps-implementation-matrix](#)。

一句话定位

In-place Pod resize 让 Pod 的 CPU/memory requests/limits 可以在不重建 Pod、不重启容器的情况下更新。它把 Pod resource 从“创建时不可变配置”改成“spec desired state + status actual/allocated state”的控制循环。

为什么重要

传统 Kubernetes 要改 Pod 资源，必须重建 Pod。对无状态服务这通常可以接受，但对这些场景代价很高：

- Stateful workload 重建会影响可用性和数据 locality。
- Batch / training workload 重启会浪费已经运行的计算进度。
- LLM serving 或多副本少的服务，重启一个 Pod 就可能造成明显容量缺口。
- VPA 想调整资源时，必须用 eviction/recreate，行为粗糙。

In-place resize 的目标不是接管所有生命周期，而是提供底层能力：资源可以被请求变更，kubelet 尽量在本节点原地完成，失败时把状态暴露给上层 controller 决策。

核心状态模型

1287 的关键是把资源状态拆成四层：

```
Desired
  what user/controller wants
  Pod.spec.containers[*].resources
  |
  v
Allocated
  what kubelet admitted and checkpointed
  Pod.status.containerStatuses[*].allocatedResources
  |
  v
Actuated
  what kubelet passed to runtime/cgroup
```

```
local checkpoint, not API
|
v
Actual
what runtime reports as running config
Pod.status.containerStatuses[*].resources
```

这个拆分解决了一个长期问题：spec.resources 不能再被理解为“实际正在使用的资源”，它只是 desired state。真实资源要从 status 看。

API 设计

资源变更只能通过 Pod /resize subresource 进行，允许修改：

- .spec.containers[*].resources
- .spec.initContainers[*].resources，仅限 sidecar 场景
- .spec.resizePolicy

普通 Pod update 仍不能直接改 resources。这样做的原因是 resize 有独立 validation、status、权限和 rollout 语义，不应该混进普通 Pod spec update。

resizePolicy 允许用户为 CPU/memory 指定是否需要重启容器：

策略	含义
NotRequired	默认，尽量不重启容器原地调整。
RestartContainer	资源变化需要重启容器才能生效。

这不是强保证。runtime 如果知道无法无重启完成，应该返回错误，kubelet 后续重试或由上层处理。

Resize Conditions

Resize 通过 Pod conditions 暴露进度：

Condition	含义
PodResizePending	desired resources 已变化，但 kubelet 尚未分配。
PodResizeInProgress	allocated 和 actual/actuated 不一致，正在执行。
PodResizePreemptionDisabled	节点策略禁止为 resize 触发 preemption。

PodResizePending 细分两个重要 reason:

- Deferred: 理论可行, 但当前节点空闲资源不足, 后续会重试。
- Infeasible: 不可行, 不会自动重试, 例如超过节点容量、static pod、某些 policy 不支持。

这使上层 controller 能区分“等一等可能成功”和“这次 resize 设计上不可能成功”。

Kubelet 控制流

```
Pod /resize request
|
+-- API validation
|
kubelet observes updated Pod
|
+-- check node allocatable and current allocations
+-- allocate or mark Deferred/Infeasible
+-- checkpoint allocated resources
+-- call CRI UpdateContainerResources
+-- update Pod status resources/conditions
```

Kubelet 负责本节点 admission 和执行。它会按优先级处理 pending resize:

1. 不增加 request 的 resize 优先, 因为通常应当更容易成功。
2. 高 PriorityClass。
3. 高 QoS class。
4. 等待时间更长的 resize。

这只是 kubelet 内部处理顺序, 不等于 scheduler 级抢占策略。

CRI 变化

In-place resize 要求 CRI 的 UpdateContainerResources 语义更清晰:

- 调整 CPU/memory 时应尽量不故意重启容器。
- ContainerStatus 能报告当前资源配置。
- Pod-level cgroup 变化后, kubelet 可以通过 UpdatePodSandboxResources 通知 runtime/NRI plugin。

UpdatePodSandboxResources 被设计成 best effort。它失败不阻塞 kubelet 完成 resize, 因为 kubelet 仍是 Pod-level cgroup 的主要执行者。

Pod-level Resources

5419-pod-level-resources-in-place-resize 把 resize 从 container-level 扩到 Pod-level resources。它的难点是“有效 Pod 资源需求”不再只是 container 资源求和：

- API defaulting 可能影响 Pod-level 和 container-level 的最终值。
- kubelet、scheduler、cgroup、quota 都要理解 Pod-level resources。
- status 需要暴露 Pod-level allocated/actual，避免用户只看到 container 层。

这条线和 5526-pod-level-resource-managers 强相关，因为 CPU/Memory/Topology Manager 原本多以 container 为 scope 管理资源。

Static CPU Manager 和 Scale-down Delay

5554 与 6122 处理更难的高性能场景：static CPU Manager 下，Pod 可能拥有 exclusive CPUs。缩容 exclusive CPUs 不是简单改 cgroup：

- 应用可能需要先停止使用某些 CPU。
- kubelet 直接移除 cpuset 可能造成性能抖动或错误。
- 工作负载需要知道即将被收回哪些 CPU。

因此 6122 引入“资源暴露 + configurable scaling delay”方向：先通过 Downward API 暴露 desired/assigned CPU 信息，给应用一个准备窗口，再执行实际收缩。

这是 Kubernetes 从“资源由平台单向强制”走向“平台和高性能 workload 协调”的信号。

Scheduler Preemption for Resize

5836-scheduler-preemption-for-ippr 解决 Deferred resize 的下一步：如果当前节点资源不足，是否可以抢占低优先级 Pod 来让已运行 Pod 扩容？

关键点：

- resize Pod 已经绑定到节点，所以 scheduler 只能在该节点评估。
- NodeName / NodeResourcesFit 路径需要特殊处理已调度 Pod。
- 如果 resize 不 fit，DefaultPreemption 可以选择 victim。
- victim 删除后，deferred Pod 重新进入队列评估。
- scheduler 不需要额外 reservation，因为它按 max(desired, allocated, actual) 看资源，desired 已经占位。

这把 resize 从 kubelet 本地重试，扩展到 scheduler 参与的资源重排。

Node-level Preemption Policy

有些节点由外部系统支持“节点扩容”或“动态调大节点容量”。这些节点上，为了让一个 Pod resize 而抢占别的 Pod，可能不是最佳选择。

因此 5836 提出 node-level policy:

```
Node.spec.podPreemptionPolicy.disableResizePreemption
```

设计上不是让 scheduler 直接读 Node policy 后自己决定，而是让 kubelet 读本节点 policy，并在 Pod status 设置 PodResizePreemptionDisabled condition。scheduler 继续以 Pod condition 为入口，避免把 resize preemption path 直接耦合到 Node spec 细节。

这个设计保持了责任边界：

- kubelet 负责节点本地 resize 生命周期。
- scheduler 负责是否排队、是否抢占。
- Node policy 由 kubelet 转译成 Pod status 信号。

关键失败模式

失败模式	设计处理
spec 和实际资源不一致	desired / allocated / actuated / actual 四层状态。
resize 后 runtime 没真正应用	ContainerStatus.resources 暴露 actual，kubelet 重试或报错。
node 资源不足	Deferred condition，后续重试。
resize 永远不可能成功	Infeasible condition，交给上层处理。
scheduler 过度抢占	node-level preemption policy + PodResizePreemptionDisabled。
static CPU 缩容伤害 workload	scaling delay + Downward API 暴露 assigned cpuset。
与 workload-aware preemption 冲突	Alpha 先接单 Pod resize 处理，Beta 需设计 group-wide 协调。

和 autoscaling 的关系

In-place resize 是 VPA 和 HPA 之间的底层能力补位：

- HPA 通过改 replicas 横向扩缩。
- VPA 或类似 controller 可以通过 /resize 纵向改资源。

- 如果 resize 被 Deferred, scheduler preemption 或 node autoscaler 可能介入。
- 如果节点支持动态扩容, disableResizePreemption 可以偏向 node upsizing, 而不是驱逐 Pod。

未来重要问题是: VPA、Cluster Autoscaler、Karpenter、kubelet resize queue、scheduler preemption 如何形成统一优先级。

阅读顺序

1. 1287-in-place-update-pod-resources: 主线, 先理解状态模型和 /resize。
2. 5419-pod-level-resources-in-place-resize: 理解 Pod-level resources。
3. 5526-pod-level-resource-managers: 理解 CPU/Memory/Topology manager 如何支持 Pod scope。
4. 5554 / 6122: 理解 static CPU、exclusive CPUs 和 scaling delay。
5. 5836-scheduler-preemption-for-ippr: 理解 scheduler 如何为 deferred resize 抢占。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
1287-in-place-update-pod-resources	implemented / stable, 已实现/GA	v1.27 / v1.33 / v1.35	InPlacePodVerticalScaling, InPlacePodVerticalScalingAllocatedStatus	Pod /resize subresource, desired/allocated/actual 状态和 kubelet resize loop。
5419-pod-level-resources-in-place-resize	implementable / beta, 仍在 beta	v1.35 / v1.36 / -	InPlacePodLevelResourcesVerticalScaling, PodLevelResources	Pod-level resources 参与 in-place resize。
5526-pod-level-resource-managers	implementable / beta, 仍在 beta	v1.36 / v1.37 / v1.39	PodLevelResources, PodLevelResourceManagers	CPU/Memory/Topology Manager 支持 Pod scope 管理。
5554-in-place-update-pod-resources-	implementable /	v1.37 / v1.38	InPlacePodVerticalScalingExclusiveCPUs	static CPU Manager exclusive CPUs 场景下支持 resize。

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
alongside-static-cpu-manager-policy	alpha, 仍在 alpha	/ v1.39		
6122-configurable-scaling-delay-with-pod-resource-exposure	implementable / alpha, 仍在 alpha	v1.37 / v1.39 / v1.40	DownwardAPIAssignedResources 等	先向 Pod 暴露 assigned resources, 再延迟收回 exclusive CPUs。
5836-scheduler-preemption-for-ipp	implementable / alpha, 仍在 alpha	v1.37 / v1.38 / v1.39	SchedulerPreemptionForPodResize	scheduler 为 Deferred resize 在已绑定节点上选择 victim。

追踪重点

- /resize subresource 是否被 VPA、StatefulSet、Job controller 等真实使用。
- status 中 allocated/actual resources 是否被 metrics pipeline 和用户工具正确展示。
- static CPU Manager / Memory Manager / Topology Manager 的策略兼容性。
- resize-induced preemption 与 workload-aware preemption 的后续设计。
- node autoscaling 与 resize preemption 的优先级冲突。

Kubernetes Node Runtime Observability Security Design

这页合并讲 sig-node 里除 DRA 和 in-place resize 之外的核心 feature: kubelet/CRI runtime 边界、CPU/Memory/Topology Manager、Pod lifecycle、安全隔离、节点可观测和 resource health。逐个 KEP 的 Alpha/Beta/GA、是否实现和 feature gate 见 [kubernetes-keps-implementation-matrix](#)。

一句话定位

SIG Node 的核心设计方向是把 kubelet 从“绑定某个 runtime 和一堆节点本地实现细节的执行器”，演进为“通过 CRI、resource managers、status/metrics、security primitives 管理节点执行状态的标准控制点”。

Runtime Boundary: kubelet / CRI / RuntimeClass

2040-kubelet-cri、2221-remove-dockershim、585-runtime-class 是 runtime 边界的基础线：

```
kubelet
|
+-- CRI
   |
   +-- containerd
   +-- CRI-O
   +-- sandboxed runtime
   +-- Windows / other runtimes

Pod.spec.runtimeClassName
|
+-- runtime handler selection
```

设计意义：

- kubelet 不再内置 Docker 特殊路径。
- runtime 能通过 CRI 独立演进。
- RuntimeClass 把 sandboxed runtime、gVisor/Kata-like runtime、特殊 runtime handler 变成 Pod 级选择。
- 4033-group-driver-detection-over-cri 继续减少 kubelet/runtime cgroup driver 配置不一致。
- 5825-cri-pagination 面向大规模节点 runtime API 性能。

边界判断：kubelet 负责 Kubernetes Pod lifecycle 和节点资源语义；runtime 负责容器创建、运行、状态、日志和低层隔离执行。

Resource Managers

CPU Manager、Memory Manager、Topology Manager 是性能型 workload 的节点侧底座。

Manager	代表 KEP	作用
CPU Manager	3570	为 Guaranteed / latency-sensitive workload 分配 cpuset。
Memory Manager	1769	管理 memory locality，与 Topology Manager 交换 hint。
Topology Manager	693, 3545	聚合 CPU、memory、device 的 NUMA hint，决定是否满足拓扑对齐策略。
Pod-level resource managers	5526	从 container scope 扩展到 Pod scope。

这组 feature 和 [kubernetes-in-place-pod-resize-design](#) 强相关。resize 如果只改 spec，不理解 CPU Manager static policy、Memory Manager、Topology Manager，就会在高性能节点上失败或造成不可解释状态。

Device Plugin / CDI / PodResources

传统 device plugin 仍然是 Kubernetes 设备接入的稳定路径：

- 3573-device-plugin 提供 kubelet device plugin API。
- 4009-add-cdi-devices-to-device-plugin-api 让 device plugin 返回 CDI devices，减少 runtime-specific 注入逻辑。
- 3695-pod-resources-for-dra 扩展 PodResources API，使其包含 DRA resource。

这条线和 [kubernetes-dra-design-deep-dive](#) 的关系是：device plugin 是旧但稳定的离散设备模型；DRA 是新的结构化 claim 模型；CDI 是两者都可使用的 runtime 注入表达。

Pod Lifecycle

SIG Node 的 lifecycle KEP 不是杂项，它们决定 kubelet 如何解释 Pod 内部容器关系：

Feature	KEP	设计意义
Sidecar containers	753	用特殊 initContainer 语义表达 sidecar 启动、就绪、终止顺序。
Ephemeral containers	277	支持 debug container 注入，不改变正常 workload spec。

Feature	KEP	设计意义
Graceful node shutdown	2000, 2712	节点关机时按优先级有序终止 Pod。
Container restart policy	5307	细化 container 级 restart 行为。
Container restart termination	4438	处理 Pod 终止期间 sidecar restart 行为。
EvictionRequest API	4563	给 eviction 请求更清晰的 API 表达。

Sidecar containers 对 Job、service mesh、agent runtime 都很重要：没有明确 sidecar 语义时，Job 完成、主容器退出、sidecar 终止顺序会变得混乱。

Security / Isolation

节点安全线覆盖 user namespace、rootless kubelet、seccomp、cgroup v2 和 kubelet authz：

Feature	代表 KEP	设计意义
User namespaces	127	Pod 内 root 不再等价宿主 root，降低逃逸影响面。
Rootless kubelet	2033	kubelet 自身降低特权运行需求。
HostNetwork userns	5607	让 HostNetwork Pod 也能受 user namespace 保护。
Seccomp by default	2413	默认系统调用限制。
Fine-grained kubelet authz	2862	缩小 kubelet API 权限面。
cgroup v2 / remove cgroup v1	2254, 5573	统一现代 Linux resource control 基础。

这组 KEP 的共同方向是：把“容器隔离靠 runtime 默认行为”升级为“Pod API、kubelet、runtime、kernel primitives 明确协作”。

Observability / Node Health

可观测线是 autoscaling、scheduling 和排障的输入信号：

Feature	KEP	作用
Resource metrics endpoint	727	kubelet 资源指标标准入口。
CRI pod/container stats	2371	从 cAdvisor 依赖转向 CRI stats。
PSI metrics	4205	暴露 CPU/memory/io pressure stall information。

Feature	KEP	作用
PSI node conditions	5394	将 PSI 上升为 Node Conditions，便于调度/自动化响应。
Resource health in Pod status	4680	Device Plugin / DRA 资源健康进入 Pod status。
Node declared features	5328	节点能力声明，为调度和控制器提供结构化输入。

2371 对 [metrics-server](#) 和 HPA 都重要：如果 kubelet 的 stats 来源不稳定，autoscaling 的输入就不可靠。

4680 对 GPU/DRA workload 重要：Pod pending 或 degraded 时，用户需要知道是设备健康而不是普通资源不足。

合并设计图

```

Pod API
|
+-- runtimeClass / sidecar / userns / resize / resource claims
|
kubelet
|
+-- CRI runtime boundary
+-- CPU / Memory / Topology Manager
+-- Device Plugin / DRA kubelet plugin / CDI
+-- Pod lifecycle and eviction
+-- Pod status / Node conditions / metrics
|
container runtime + kernel
|
+-- cgroups / namespaces / seccomp / devices

```

和其他设计页的关系

- [kubernetes-in-place-pod-resize-design](#) 依赖 resource managers、CRI resource update、status/metrics。
- [kubernetes-dra-design-deep-dive](#) 依赖 kubelet plugin、CDI、PodResources API、resource health。
- [kubernetes-hpa-autoscaling-design](#) 依赖 kubelet stats、CRI stats、metrics-server。
- [kubernetes-scheduler-core-design](#) 依赖 node labels、resource health、Node Conditions、declared features 作为调度输入。

关键失败模式

失败模式	设计处理
kubelet/runtime 配置不一致	CRI cgroup driver detection、RuntimeClass。
container stats 来源不统一	CRI pod/container stats。
NUMA/CPU/device 不对齐	Topology Manager 聚合 hints。
sidecar 阻塞 Job 完成或终止	Sidecar containers 明确 lifecycle。
Pod root 权限过大	user namespaces、rootless kubelet、seccomp。
设备异常不可见	resource health in Pod status。
节点 pressure 只能靠人工看指标	PSI metrics -> Node Conditions。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
2040-kubelet-cri	implementable / beta, CRI 支撑主线已长期使用但 metadata 未 GA	v1.5 / v1.23 / -	-	kubelet 通过 CRI 管理 runtime, Docker 细节退出 kubelet 核心。
2221-remove-dockershim	implemented / stable, 已实现/GA	- / - / -	-	移除 kubelet 内置 dockershim。
585-runtime-class	implemented / stable, 已实现/GA	v1.12 / v1.14 / v1.20	RuntimeClasss	Pod 通过 runtimeClassName 选择 runtime handler。
3570-cpumanager	implemented / stable, 已实现/GA	v1.8 / v1.10 / v1.26	CPUManager	kubelet static policy 为 Guaranteed Pod 分配 cpuset。
1769-memory-manager	implemented / stable, 已实现/GA	v1.21 / v1.22 / v1.32	MemoryManager	kubelet 管理 NUMA-aware memory allocation。
693-topology-manager	implemented / stable, 已实现/GA		TopologyManager	

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
		v1.16 / v1.18 / v1.27		聚合 CPU/memory/device hints，决定 NUMA 对齐。
3573-device-plugin	implemented / stable, 已实现/GA	v1.8 / v1.10 / v1.26	DevicePlugins	kubelet device plugin API。
4009-add-cdi-devices-to-device-plugin-api	implemented / stable, 已实现/GA	v1.28 / v1.29 / v1.31	DevicePluginCDIDevices	device plugin 可返回 CDI devices。
3695-pod-resources-for-dra	implemented / stable, 已实现/GA	v1.27 / v1.34 / v1.36	KubeletPodResourcesDynamicResource	PodResources API 暴露 DRA resources。
753-sidecar-containers	implemented / stable, 已实现/GA	v1.28 / v1.29 / v1.33	SidecarContainers	用 restartable init containers 表达 sidecar lifecycle。
127-user-namespaces	implemented / stable, 已实现/GA	v1.25 / v1.35 / v1.36	UserNamespacesSupport	Pod user namespace 隔离。
2033-kubelet-in-usersns-aka-rootless	implementable / beta, 仍在 beta	v1.22 / v1.37 / -	KubeletInUserNamespace	kubelet rootless/user namespace 运行。
5607-hostnetwork-usersns	implementable / alpha, 仍在 alpha	v1.35 / - / -	UserNamespacesHostNetworkSupport	HostNetwork Pod 使用 user namespaces。
2371-cri-pod-container-stats	implementable / beta, 仍在 beta	v1.29 / v1.37 / -	PodAndContainerStatsFromCRI	kubelet Pod/container stats 迁往 CRI。
			KubeletPSI	kubelet 暴露 PSI metrics。

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
4205-psi-metric	implemented / stable, 已实现/GA	v1.33 / v1.34 / v1.36		
5394-psi-node-conditions	implementable / alpha, 仍在 alpha	v1.36 / - / -	PSINodeCondition	PSI pressure 转为 Node Conditions。
4680-add-resource-health-to-pod-status	implementable / beta, 仍在 beta	v1.31 / v1.36 / v1.37	ResourceHealthStatus	Device Plugin/DRA resource health 写入 Pod status。
5328-node-declared-features	implementable / stable, GA 目标已达	v1.35 / v1.36 / v1.37	NodeDeclaredFeatures	节点能力结构化声明, 供调度/控制器消费。

追踪重点

- CRI stats 是否足够替代 cAdvisor 依赖, 并被 [metrics-server](#) 稳定消费。
- resource health status 是否覆盖 DRA 和传统 device plugin。
- rootless kubelet / HostNetwork userns 是否有清晰生产约束。
- CPU/Memory/Topology Manager 与 Pod-level resources、in-place resize 的兼容性。
- sidecar containers 与 Job、LWS、agent runtime 的实际行为是否符合用户直觉。

Kubernetes Scheduler Core Design

当前上游核验 (2026-09-25)

本节以执行时官方默认分支为当前证据，commit 不代表 release。Kubernetes KEP 页面与下文既有内容保留历史设计/演进证据；具体实现、feature stage 和默认行为须按目标 Kubernetes 版本核对，不能从提案状态或默认分支推定已经发布。[src-scheduler-plugins-architecture](#) 仍是 2026-06-14 的 raw-backed Source 快照。

项目	当前 commit	当前职责 / 证据边界
kubernetes / kube-scheduler	ab9b0dcfd320	Scheduling Framework 组织插件扩展点；scheduling cycle 选择 Node，binding cycle 将选择应用到集群。Pod placement/binding 属于 scheduler 职责
scheduler-plugins	6df8d8e4ae5f	通过 out-of-tree plugins 扩展 kube-scheduler；其 scheduling.x-k8s.io/v1alpha1 PodGroup / ElasticQuota 与 Kubernetes 原生 API 演进分开核验，部署须匹配 Kubernetes 依赖版本

官方入口：[Scheduling Framework](#)、[Scheduler Configuration](#)、[当前 kube-scheduler 源码](#)、[Scheduler Plugins 文档](#) 与 [当前兼容矩阵](#)。跨项目主线见 [kubernetes-workload-gang-scheduling-design](#)：[kueue](#) 负责 admission，[karpenter](#) 负责节点容量反馈；通用 controller 工具链见 [k8s-core-controller-map](#)。

M5-B 中的职责边界

[kubernetes-workload-gang-scheduling-design](#) 是跨项目入口，本页下钻 Pod 的 placement/binding 与失败重试。[kueue](#) 在所管理工作负载进入实际 Pod 调度前完成配额准入，再由 integration unsuspend 工作负载或移除 Pod scheduling gates；Pod 对象可能在准入前已存在。kube-scheduler 对可调度 Pod 选择 Node 并完成 binding，[scheduler-plugins](#) 以 out-of-tree 插件扩展同一 framework，不能据此推定它接管 JobSet/LWS 等业务对象生命周期。

[karpenter](#) 从未调度 Pod 的状态和约束计算容量需求，经 NodePool/NodeClass、NodeClaim 与节点生命周期补充容量；它不执行 Pod binding。新 Node 的状态变化再成为 scheduler 重试的输入，扩容成功也不保证 Pod 的所有放置约束可满足。

Scheduling / Binding Cycle

QUEUE ENTRY

Pod event - - enqueue eligibility - -> PreEnqueue

PreEnqueue — success —> activeQ (ordered by QueueSort) — pop —>

```

profile
  PreEnqueue - - gated / reject - -> unschedulableQ

SCHEDULING CYCLE (serial, single-Pod path)
  profile —> snapshot / PreFilter —> Filter
  Filter — no feasible Node —> PostFilter / optional preemption
  PostFilter - - nomination / failure feedback: retry later - -> queue
  Filter — feasible Nodes —> PreScore / Score / NormalizeScore
    — select Node —> assume in scheduler cache —> Reserve —> Permit
  Permit — approve OR wait —> dispatch binding cycle
  Reserve / Permit failure —> Unreserve / forget assumed Pod
    - - failure / retry - -> queue

BINDING CYCLE (may run concurrently for different Pods)
  WaitOnPermit (if waiting) — approved —> PreBind —> Bind —> PostBind
  WaitOnPermit reject/timeout OR PreBind/Bind failure
    —> Unreserve / forget assumed Pod - - failure / retry - -> queue

```

实线 — label —> 表示调用或有序步骤，虚线 - - label - -> 表示事件、排队和后续重试；图中失败回队列后仍需经过下一节的唤醒与退避判断。PreEnqueue 在 activeQ 之前执行，QueueSort 决定队列顺序，并非 Pod 被 pop 后才依次执行这两步。当前图聚焦单 Pod 路径；原生 PodGroup 调度另见 L1 页的 KEP 边界。

这里按锚定 commit 的 [schedule one.go](#) 区分周期：schedulingCycle 调用 prepareForBindingCycle，其中完成 assume/reserve 和首次 Permit 调用；之后异步启动 bindingCycle，在 WaitOnPermit 等待需要放行的 Pod，再执行 PreBind/Bind/PostBind。这与 [Scheduling Framework](#) 对 Permit 位于 scheduling cycle 末尾的说明一致。图省略了版本/feature gate 相关的辅助检查，不把 Reserve/Permit 整体移到 binding cycle。

nominatedNodeName 表达候选或预期，不等于已绑定；抢占选出的候选通常要在后续调度周期重验。当前轮选择 Node 后的 assume 也是 scheduler 的内存记账，不是 API binding 已成功。Reserve/Permit 与后续阶段失败时必须释放相应保留状态；Bind 成功后才运行 PostBind。

Queue / Requeue Feedback

```

Queue admission - - admitted feedback - -> workload / Pod integration
integration — unsuspend OR remove scheduling gates —> eligible Pod
Pod - - watch / enqueue - -> PreEnqueue — success —> activeQ (QueueSort)
PreEnqueue - - gated / reject - -> unschedulableQ
activeQ — pop —> scheduling / binding attempt
attempt - - unschedulable / error feedback - -> failure handler

```

```

failure handler - - wait for relevant change - -> unschedulableQ
failure handler - - retry after backoff - -> backoffQ

Node / Pod / plugin-relevant cluster events
- - QueueingHints: eligible to retry - -> unschedulableQ re-evaluation
unschedulableQ - - retry eligible + backoff pending - -> backoffQ
unschedulableQ - - retry eligible + backoff satisfied / bypass allowed - ->
activeQ
backoffQ - - backoff complete / queue activation - -> activeQ
unschedulableQ - - periodic fallback re-evaluation - -> activeQ or backoffQ

Pod Unschedulable - - observed state / event - -> Karpenter capacity
controller
Karpenter — NodePool + NodeClass / NodeClaim lifecycle —> Node
Node - - capacity / readiness event - -> cluster events / QueueingHints
above

```

unschedulableQ 在这里是概念名；该 commit 的实现使用 unschedulableEntities，可保存 Pod 或 PodGroup。activeQ 保存可尝试项，backoffQ 保存等待退避的项；失败回队列的位置还取决于失败插件、在途事件和当前退避状态，不是所有错误都固定先进入同一个队列。QueueingHint 根据插件关心的变化决定是否值得再试，未命中时继续等待；相关事件不是“无条件唤醒全部 pending Pods”。回到 activeQ 仍须满足入队门槛，周期性兜底也不会自动解除 scheduling gates。实现依据见 [当前 scheduling queue](#)。

Kueue 的等待准入和 scheduler 的不可放置是不同状态。被 gate 挡住的 Pod 不能直接当作已经执行 Filter 后的 Unschedulable；Karpenter 也必须结合 Pod 和节点约束评估可供应容量。对允许调度后仍缺少节点的 Pod，新 Node 通过 watch/队列机制触发重算，不会绕过 Filter、Permit 或 Bind。

Scheduling 失败路径

失败点 / 直接影响	自动恢复与控制器协同	人工排查 / 修复边界
Filter 无 feasible Node：当前轮无法选 Node	按配置尝试 PostFilter；后续相关事件与退避允许重新评估	检查失败插件、资源请求、taint、affinity、拓扑等硬条件；增加不匹配的节点不会解决问题
PostFilter / preemption 无可行结果：可能无候选或 victim 释放后仍不足	抢占策略允许时选择候选，等待资源变化后重试；nomination 不保证后续绑定	检查优先级、抢占策略、不可由抢占消除的约束；不要将 PDB 理解为 scheduler preemption 的绝对保护
Reserve 失败或后续阶段失败：保留状态需清理	运行 Unreserve、撤销 assumed Pod；Unreserve 应幂等且不能失败	自定义插件泄漏资源、不可逆副作用或不完整回滚需修正代码/清理外部状态，不能仅靠 requeue

失败点 / 直接影响	自动恢复与控制器协同	人工排查 / 修复边界
Permit wait / reject / timeout: 绑定暂停或被拒绝	等待相关插件批准；拒绝/超时触发清理和重新排队	检查组成员、超时、外部条件与插件配置；增加超时不会修复永久缺失的成员
PreBind 失败：绑定前准备未完成	中止 binding cycle、清理保留状态并重试；依赖控制器恢复后再尝试	检查外部依赖、权限和插件错误；确认重复执行及补偿能处理部分完成的准备操作
Bind 失败：API 层绑定未确认	scheduler 走失败处理，依据后续 API/watch 状态判断 Pod 是否仍需调度	区分权限/冲突/网络超时；超时不证明服务端未写入，避免插件盲目重复不可逆操作
插件数据陈旧：快照或外部缓存与实际状态不一致	正常 watch 更新与下一轮 snapshot 可修正暂时陈旧；插件需注册正确的重试事件	长期不同步、错误失效策略或漏注册事件需修复；调度队列不能替代外部缓存一致性设计
插件目标冲突：Filter 交集为空或 Score 权重违背预期	只在输入变化后重算；没有自动“放松策略”的通用机制	校验 profile、插件开关/顺序/权重与业务目标，用 kube-scheduler-simulator 对照具体决策

out-of-tree 插件与 in-tree 插件都通过 framework 参与相应阶段；部署 scheduler-plugins 并不建立另一套 JobSet/LWS 所有权模型。故障定位应同时看 Pod events/status、失败 extension point 和相关控制器状态，避免把所有 Pending 都归为容量不足。

KEP 设计背景

下面保留 scheduler core 的 framework、component config、profiles、queue/requeue、topology placement、async preemption 和性能演进笔记。历史状态表不作为本次对 release/feature stage 的重新核验；逐个 KEP 的追踪入口见 [kubernetes-keps-implementation-matrix](#)。

一句话定位

Scheduler core KEP 的共同目标是把 kube-scheduler 从一个内置策略集合，演进为可配置、可扩展、可解释、可高吞吐的调度框架。

Scheduler Framework

624-scheduling-framework 是最关键的基础 KEP。它把一次 Pod 调度拆成多个 extension point，当前调用顺序与分支见上方 Scheduling / Binding Cycle。PostFilter 属于无可行节点时的恢复分支，不能画成每次 Filter 后必经的一步。

设计价值：

- 新策略不再必须 fork scheduler 或靠 HTTP extender 绕路。
- plugin 可以在 filter、score、reserve、permit、bind 等不同阶段表达不同语义。
- DRA、gang scheduling、queueing hints、in-place resize preemption 都能复用这套扩展点。

1819-scheduler-extender 仍然是历史对照。HTTP extender 能扩展 Filter/Prioritize/Bind 等阶段，但有 cache 同步、性能、错误处理和扩展点表达不足的问题。framework 的方向是把关键扩展内聚到 scheduler 进程内，以 typed plugin API 表达。

ComponentConfig 和 Profiles

785-scheduler-component-config-api 把 scheduler 配置版本化，1451-multi-scheduling-profiles 允许一个 scheduler 实例提供多个 profile。

这两个 KEP 解决的是运营问题：

- 不同 workload 可能需要不同 plugin 组合。
- 多 scheduler binary 会复制 cache、部署和 HA 成本。
- profile 让用户通过 schedulerName 选择策略，而不是部署多套控制面。

多 profile 的边界是：它共享 scheduler cache 和主进程，适合策略差异，不适合完全隔离不同调度器的故障域或权限域。

Queue / Requeue 设计

调度性能不只取决于 Filter/Score 快不快，也取决于“什么时候值得重新尝试一个 pending Pod”。

Feature	KEP	设计作用
Pod scheduling readiness	3521	用 scheduling gates 阻止条件未满足的 Pod 进入正常调度。
QueueingHint	4247	plugin 判断某个事件是否可能让某个 Pod 可调度。
PreQueueing hints	6132	在 Pod 进入 activeQ 前更早过滤无效唤醒。
Pop backoffQ when activeQ empty	5142	activeQ 空时提前处理 backoffQ，提高吞吐。
Reflect PreEnqueue rejection in Pod status	5501	把被 PreEnqueue 拒绝的原因暴露给用户。
Opportunistic batching	5598	对高吞吐场景批量处理，减少重复计算。

QueueingHint 是 DRA、gang、resize 这类等待型场景的关键。没有 hint，任何 ResourceClaim、PodGroup、Node、Pod 事件都可能唤醒大量无关 pending Pod，造成调度风暴。

Placement / Topology

普通 Pod placement 仍然是 scheduler core 的重要 feature：

- 895-pod-topology-spread 将副本分散作为一等调度约束。
- 1258-default-pod-topology-spread 给未显式配置 spread 的 workload 提供默认保护。
- 3022-min-domains-in-pod-topology-spread 避免拓扑域数量不足时误判。
- 3094-pod-topology-spread-considering-taints 让 skew 计算排除实际不可用节点。
- 3633-matchlabelkeys-to-podaffinity 让 rollout hash 等动态标签参与 affinity/anti-affinity。

这组设计和 [kubernetes-workload-gang-scheduling-design](#) 的区别是：Pod topology spread 仍然以单 Pod 为基本调度循环；topology-aware workload scheduling 则要对一组 Pod 一起生成 placement。

Preemption 基础线

Preemption 从最早的 priority/preemption 发展到更异步、更可解释：

- 902-non-preempting-priorityclass 支持“高优先级排序但不抢占”。
- 4832-async-preemption 避免 scheduler 在调度周期里同步等待 victim 删除。
- 5278-nominated-node-name-for-expectation 改善 nominated node 期望表达。
- 3280-guarantee-pdb-when-preemption-happens 关注抢占和 PDB 的冲突。

这组 feature 是 Workload-aware preemption 和 resize-induced preemption 的基础。后两者不是重新发明抢占，而是把现有抢占模型扩展到 PodGroup 或已绑定 Pod resize 场景。

Data Flow

完整事件、队列、profile、framework 与失败反馈路径已合并到上方两张图。

理解 scheduler feature 时，应该先问它改的是哪一层：

- 改 API 表达：ComponentConfig、profile、Pod topology spread。
- 改队列行为：QueueingHint、PreQueueing、backoffQ。
- 改调度计算：Filter/Score plugin、DRA Filter、TopologySpread。
- 改失败处理：PostFilter、preemption、status/event。
- 改性能：batching、async API calls、async preemption。

重要边界

边界	说明
Scheduler 不是 admission queue	多租户队列、公平性、quota 仍更适合 kueue 这类 controller。
Scheduler profile 不是强隔离	多 profile 共享进程和 cache。
Queueing hints 不改变调度结果	它只减少无意义重试，不应改变最终可调度性。
TopologySpread 不是 gang placement	它仍然按 Pod 调度，只是计算 skew。
Preemption 不保证 PDB 绝不被破坏	Kubernetes 尽量减少 disruption，但不能把 PDB 当硬约束。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
624-scheduling-framework	implemented / stable, 已实现/GA	v1.16 / - / v1.19	-	kube-scheduler 内部 extension points，替代大量 extender/fork 场景。
785-scheduler-component-config-api	implemented / stable, 已实现/GA	- / v1.19 / v1.25	-	versioned KubeSchedulerConfiguration，让调度器配置可升级。
1451-multi-scheduling-profiles	implementable / beta, 设计可实现，metadata 未标 implemented	v1.18 / v1.19 / v1.22	-	一个 scheduler 进程多个 profile，Pod 通过 schedulerName 选择策略。
3521-pod-scheduling-readiness	implemented / stable, 已实现/GA	v1.26 / v1.27 / v1.30	PodSchedulingReadiness	schedulingGates 让 Pod 在外部条件满足前不进入普通调度。
4247-queueinghint	implemented / stable, 已实现/GA	v1.26 / v1.32 / v1.34	SchedulerQueueingHints	plugin 按事件判断是否 requeue pending Pod，降低调度风暴。
	implementable / beta, 仍在 beta		SchedulerPreQueue	

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
6132- prequeueing- hints		- / v1.37 / v1.39	QueueingHints	在 activeQ 之前过滤无效唤醒，是 DRA/gang/resize 的性能补强。
5598- opportunistic- batching	implementable / beta, 仍在 beta	- / v1.35 / v1.38	OpportunisticBatching	对可调度机会做批量化，减少重复 snapshot/filter/score 成本。
895-pod- topology- spread	implemented / stable, 已实现/GA	v1.16 / v1.18 / v1.19	EvenPodsSpread	scheduler filter/score 计算 topology domain skew。
4832-async- preemption	implementable / beta, 仍在 beta	v1.32 / v1.33 / -	SchedulerAsyncPreemption	抢占 victim 删除异步化，减少主调度循环阻塞。

和其他详解页的关系

- [kubernetes-workload-gang-scheduling-design](#) 依赖 framework、queue、preemption。
- [kubernetes-dra-design-deep-dive](#) 依赖 queueing hints、Filter timeout、Reserve/PreBind。
- [kubernetes-in-place-pod-resize-design](#) 依赖 queueing hints 和 preemption failure handler。
- [scheduler-plugins](#) 是 out-of-tree 实验和生产化扩展的参考实现集合。
- [kube-scheduler-simulator](#) 适合用来观察 filter/score/preemption 的具体行为。

追踪重点

- QueueingHint / PreQueueing hints 是否覆盖 DRA、PodGroup、resize 等高等待场景。
- Async preemption 与 workload-aware preemption 是否收敛到统一的失败处理模型。
- Scheduling profile 的 plugin 参数是否继续保持可升级和可观测。
- TopologySpread 与 workload-level topology 的边界是否清晰。
- scheduler perf 中 extension point latency 是否被 DRA / CEL / batch scheduling 拉高。

Kubernetes v1.37 Scheduling / Node / DRA Progress

这页记录 2026-07-15 对 `kubernetes/enhancements master` 的最新核验，重点回答：近期 Kubernetes 在调度、Node、DRA 和相关 autoscaling KEP 上推进到了哪里。基础设计脉络见 [src-kubernetes-keps-design-tracking](#)，逐项状态矩阵见 [kubernetes-keps-implementation-matrix](#)。

总体判断

v1.37 的主线不是单点调度优化，而是三条线汇合：

1. [kubernetes-workload-gang-scheduling-design](#) 把 PodGroup / Workload 作为调度单位推进到 beta/alpha 组合。
2. [kubernetes-dra-design-deep-dive](#) 把 DRA 从结构化资源模型扩展到 capacity、workload claim、NUMA、status 和 Pod 内可见性。
3. [kubernetes-in-place-pod-resize-design](#) 与 [kubernetes-node-runtime-observability-security-design](#) 把 Pod-level resources、resize、Memory QoS、rootless kubelet 和节点状态补齐到 Node 路径。

v1.37 feature direction

```
|
+-- workload-aware scheduling
|   +-- Gang Scheduling beta
|   +-- Topology-aware workload scheduling beta
|   +-- Controller APIs alpha
|   +-- CompositePodGroup alpha
|
+-- DRA device/resource model
|   +-- ResourceClaim for workloads beta
|   +-- Consumable capacity beta
|   +-- Shared capacity / compatibility groups alpha
|   +-- ResourceClaim device status stable
|   +-- DRA attributes Downward API beta
|   +-- Standard numaNode stable
|
+-- node resource/runtime maturity
    +-- Pod-level resource managers beta
    +-- Memory QoS beta
    +-- Rootless kubelet beta
```

+++ Pod checkpoint/restore alpha
+++ EvictionRequest API alpha

SIG Scheduling

KEP	当前进展	意义
4671-gang-scheduling	implementable / beta / v1.37, 目标 stable v1.38	Workload / PodGroup 成为调度单位, AI/HPC/batch 不再只靠 out-of-tree coscheduling。
5732-topology-aware-workload-scheduling	implementable / beta / v1.37, 目标 stable v1.39	一组 Pod 作为整体选择 topology placement, 连接 GPU/NIC/zone/rack 放置。
6089-was-controller-apis	implementable / alpha / v1.37	给 Job/JobSet/LWS 等 controller 提供 workload-aware scheduling building blocks。
6012-composite-podgroup-api	implementable / alpha / v1.37	表达多组件 workload 的组合调度单元。
6132-prequeueing-hints	implementable / beta / v1.37	在 Pod 进入 activeQ 前过滤无效事件, 减少 DRA/gang/resize 等等待场景的 requeue 噪声。
5598-opportunistic-batching	implementable / beta, 近期扩展 rescoring, 目标 stable v1.38	面向大规模调度吞吐, 把批量机会和重新打分纳入 scheduler 路径。
5836-scheduler-preemption-for-ipp	implementable / alpha / v1.37	已绑定 Pod 的 in-place resize 资源冲突可交给 scheduler 抢占处理。

调度侧近期提交集中在 2026-06-15 到 2026-06-17: KEP-6089 初始合入、KEP-6012 初始文档、KEP-6132 合入、KEP-5598 rescoring 扩展、KEP-4671 preemption 行为调整。这里的信号是: workload-aware scheduling 已经从单一 gang KEP 变成 API、拓扑、抢占、队列和批处理共同推进。

DRA / Device-Aware Scheduling

KEP	当前进展	意义
5075-dra-consumable-capacity	implementable / beta / v1.37, 目标 stable v1.38	

KEP	当前进展	意义
		设备资源不再只表达离散设备，也能表达 GPU memory、带宽、license 等可消费容量。
5729-resourceclaim-support-for-workloads	implementable / beta / v1.37	ResourceClaim 支持 workload 级消费者，直接连接 DRA 与 PodGroup/gang scheduling。
5517-dra-node-allocatable-resources	implementable / alpha / v1.37	暴露 DRA node allocatable，给 scheduler/autoscaler 容量推理使用。
5941-dra-shared-consumable-capacity	implementable / alpha / v1.37	多个 logical devices 共享同一 capacity pool。
5963-device-compatibility-groups	implementable / alpha / v1.37	表达多设备组合兼容性，适合多卡/多 NIC 配对。
6080-dra-derived-attributes	provisional / alpha / v1.37	从 driver 属性派生调度可用属性，仍应视为设计观察项。

这些 KEP 把 DRA 从 “ResourceSlice + ResourceClaim” 推进到复杂设备组合。对 AI 平台最关键的是 5729 和 5075：前者让一组 Pod 共享或引用 workload 级 claim，后者让 claim 不只消费一个设备编号，而是消费容量。

SIG Node

KEP	当前进展	意义
4817-resource-claim-device-status	implementable / stable / v1.37	DRA 分配结果和设备信息进入 ResourceClaim status，成为稳定可观测面。
5304-dra-attributes-downward-api	implementable / beta / v1.37，目标 stable v1.38	Pod 内 workload 可看到 DRA device attributes。
6072-dra-standard-numanode	implementable / stable / v1.37	标准化 numaNode 设备属性，连接 DRA 和 topology-aware placement。
5945-dra-optional-node-preparation	implementable / alpha / v1.37	某些设备场景可跳过强制 NodePrepare，降低轻量路径成本。

KEP	当前进展	意义
5526-pod-level-resource-managers	implementable / beta / v1.37, 目标 stable v1.39	CPU/Memory/Topology Manager 从 container scope 扩展到 Pod scope。
6122-configurable-scaling-delay-with-pod-resource-exposure	implementable / alpha / v1.37	Pod resource exposure 后的 scaling delay 可配置, 给应用处理资源收缩窗口。
2570-memory-qos	implementable / beta / v1.37	Memory QoS 重新进入 beta 路径。
2033-kubelet-in-usersns-aka-rootless	implementable / beta / v1.37	rootless kubelet 进入 beta, 降低 kubelet 特权面。
5823-pod-level-checkpoint-restore	implementable / alpha / v1.37	Pod 级 checkpoint/restore 进入 KEP 路径。
4563-eviction-request-api	implementable / alpha / v1.37	eviction 请求 API 化, 给节点驱逐更清晰的表达。
5683-lifecycle-conditions	implementable / alpha / v1.37	Node lifecycle conditions 进入 alpha。

Node 侧的重点是把调度决策需要的节点事实标准化: DRA status、DRA attributes、NUMA、Pod-level resource manager、Memory QoS 和 resource exposure 都是在为“更复杂的调度/弹性决策”提供可信输入。

Autoscaling 联动

KEP	当前进展	意义
4951-configurable-hpa-tolerance	implementable / stable / v1.37	HPA tolerance 从全局粗粒度参数变成 HPA/方向级配置。
2021-scale-from-zero	implementable / beta / v1.37	object/external metrics 支持 0 副本恢复。
5030-attach-limit-autoscaler	implementable / beta / v1.37	Cluster Autoscaler 模拟 CSI volume attach limit。
5679-external-metric-fallback	implementable / alpha / v1.36, 目标 beta v1.37	external metric 失败时可 fallback, 降低指标系统故障对 HPA 的影响。

autoscaling 不是本页主角，但它决定这些调度能力能否被容量系统理解。DRA、PodGroup、volume attach limit、topology 和 scale-from-zero 都要求 autoscaler 不只是“加节点”，而是能模拟 scheduler 约束。

选型和跟踪建议

对 AI/HPC/GPU 平台，优先跟踪这四组：

- 1. 4671 + 5732 + 6089：Kubernetes 是否能原生表达 workload 级调度。
- 2. 5729 + 5075 + 5517：DRA 是否能让 workload 和 autoscaler 理解设备容量。
- 3. 4817 + 5304 + 6072：DRA 设备状态、属性和 NUMA 是否形成稳定可观测面。
- 4. 5526 + 6122 + 5836：Pod-level resources 和 in-place resize 是否闭环到 scheduler。

采用判断：

场景	判断
普通生产 workload	继续依赖稳定 scheduler、HPA、device plugin 路径，避免直接押注 alpha DRA 扩展。
AI/HPC batch / training	重点验证 PodGroup/gang、topology-aware workload scheduling、Kueue/JobSet/LWS 集成。
GPU/NIC/DPU 平台	跟踪 DRA consumable capacity、ResourceClaim for workloads、standard numaNode 和厂商 driver 实现。
节点资源隔离	关注 Pod-level resource managers、Memory QoS、rootless kubelet，但生产采用仍要看 feature gate 默认值和 kubelet 行为。

下一步核验

kubernetes/enhancements 只表示 KEP 元数据和设计状态。后续要继续核验：

- kubernetes/kubernetes 对应实现 PR 是否已合入。
- feature gate 默认值、API group/version 和 release notes。
- conformance / e2e 覆盖，尤其是 gang scheduling、DRA workload claim、Pod-level resource managers。
- [karpenter](#) / Cluster Autoscaler 是否能理解 PodGroup、DRA structured parameters 和 volume attach limit。
- [dra-driver-nvidia-gpu](#) 等真实 driver 是否跟进 v1.37 DRA 字段。

Kubernetes Workload and Gang Scheduling Design

当前上游核验（2026-09-25）

本节记录执行时官方默认分支的当前快照，用于 M5-B 的跨项目职责比较；commit 不是 release 标识，API 在默认分支存在也不等于已在目标集群版本发布。[src-kueue-architecture](#)、[src-jobset-architecture](#)、[src-lws-architecture](#)、[src-scheduler-plugins-architecture](#) 与 [src-karpenter-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照。下文既有 KEP 内容保留历史设计与演进语境；scheduler 底座的当前证据见 [kubernetes-scheduler-core-design](#)。

项目	当前 commit	当前 API / 职责	M5-B 层
kueue	2a766717037a	当前 <code>kueue.x-k8s.io/v1beta2</code> 包含 Workload、LocalQueue、ClusterQueue、ResourceFlavor、Cohort、AdmissionCheck、Topology；管理配额、flavor 与准入，不负责 Pod 最终绑定到 Node	Admission / quota
jobset	03f9dccef945	<code>jobset.x-k8s.io/v1alpha2</code> JobSet 用 ReplicatedJobs 组合子 Job，提供依赖、coordinator、成功/失败与重启策略	Workload / Job lifecycle
lws	d4f1525f15a4	<code>leaderworkerset.x-k8s.io/v1</code> 管理 leader/worker Pod 组； <code>disaggregatedset.x-k8s.io/v1</code> 组合多个子 LWS，管理分离推理角色、slice 与协调发布	Workload / group and role lifecycle
scheduler-plugins	6df8d8e4ae5f	kube-scheduler 的 out-of-tree framework plugins；配套 <code>scheduling.x-k8s.io/v1alpha1</code> PodGroup / ElasticQuota 支撑相应插件，与 Kubernetes 原生 Workload/PodGroup KEP API 分开看	Pod placement / scheduler extensions
karpenter	06bc3b4b94dd	<code>karpenter.sh/v1</code> NodePool / NodeClaim 联合 provider-specific NodeClass 表达容量约束与节点生命周期；响应不可调度 Pod 的容量需求，不执行 Pod binding	Node capacity / lifecycle

官方依据：[Kueue Concepts](#) 与 [当前 v1beta2 API 源码](#)、[JobSet Concepts](#) 与 [v1alpha2 API](#)、[LWS Concepts](#)、[Scheduler Plugins 文档](#)、[Karpenter NodePools](#) 与 [NodeClaims](#)。安装时仍需核对所选 release 的 CRD、feature gates 和 Kubernetes 兼容矩阵。

D1 · Workload / Admission / Placement / Capacity

WORKLOAD EXPRESSION

```
JobSet / LeaderWorkerSet (LWS) / DisaggregatedSet (DS)
- - supported integration / PodSets - -> ADMISSION AND QUOTA
```

ADMISSION AND QUOTA

```
Kueue: Workload / LocalQueue / ClusterQueue
      ResourceFlavor / Cohort / AdmissionCheck / Topology
- - admitted: unsuspend / remove scheduling gates - -> POD PLACEMENT
```

POD PLACEMENT

```
kube-scheduler framework + configured scheduler-plugins
— select feasible Node / bind Pod —> existing Node
- - unschedulable Pod feedback - -> NODE CAPACITY
```

NODE CAPACITY

```
Karpenter: Pod requirements + NodePool + provider-specific NodeClass
— create capacity request —> NodeClaim
— launch / register / initialize —> Node
Node - - capacity / readiness events: requeue - -> POD PLACEMENT
```

图例：实线 — label —> 表示顺序控制动作或同步 API 操作，不承诺跨控制器原子完成；虚线 - - label - -> 表示带条件的集成、事件、准入反馈或重试。分区表示职责，既有 Node 能放下 Pod 时无需进入扩容路径。

[jobset](#)、[lws](#) 及 LWS 仓库中的 DisaggregatedSet 表达组、角色、子资源与生命周期。[kueue](#) 决定 Workload 何时取得配额和准入，选择 flavor、必要时约束拓扑域；它不负责最终 Node 选择或 Pod binding。kube-scheduler 完成 placement/binding，[scheduler-plugins](#) 扩展其 framework 插件能力。[karpenter](#) 负责节点容量及生命周期，不绑定 Pod。图中的 integration 是条件关系：只有已启用且受当前版本支持的工作负载集成才走 Kueue 路径。

D2 · Admission 与 Scheduling 路径

```
Create JobSet / LWS / DS (roles -> child LWS)
- - supported integration / Pod templates - -> Kueue Workload / PodSets
Workload — resolve queueName —> LocalQueue — resolve reference —>
ClusterQueue
ClusterQueue + optional Cohort — evaluate —> quota / ResourceFlavor
+ configured AdmissionChecks + optional Topology constraints
```

```

— record reservation / check state —> Workload admission status
- - admitted feedback - -> integration controller
integration controller — unsuspend workload OR remove Pod scheduling gates
—> Pods
Pods - - watch / enqueue - -> kube-scheduler
QueueSort —> PreFilter —> Filter —> PreScore / Score —> Reserve —>
Permit
Permit approved —> PreBind —> Bind —> PostBind

```

这是成功路径，省略 PreEnqueue、PostFilter 等分支；完整 cycle 与失败回退见 [kubernetes-scheduler-core-design](#)。LocalQueue 引用 ClusterQueue；Cohort 是配额共享关系，不是所有任务必经的另一条队列。QuotaReserved 与 Admitted 也不是同一个完成条件：已配置的 AdmissionCheck 必须满足要求，启用 topology-aware scheduling 时还要满足对应容量/拓扑条件。准入成功不保证所有 Pod 已绑定或业务已就绪，依据见 [Kueue Concepts](#)。

集成差异决定“允许开始”的具体操作：JobSet 集成通过工作负载 suspend 状态控制启动；[LWS 集成](#) 基于 Plain Pod Group，以每个 LWS group 为准入单元，Pod 可以先存在并由 scheduling gate 暂停调度。DS 先编排子 LWS；本图不据此推定存在 DS 整体原子准入，仍需验证角色标签传递、子 LWS 集成和所用版本。不能把三种 API 都画成同一个 owner controller 创建全部 Workload 后统一 unsuspend。

D3 · Capacity Feedback

```

Pod: PodScheduled=False / Unschedulable
- - observed condition / Pod event - -> Karpenter provisioning controller
Karpenter — combine constraints —> pending Pod requirements
                                + NodePool
                                + provider-specific NodeClass
— create immutable capacity request —> NodeClaim
NodeClaim lifecycle (controller + provider + kubelet)
— launch instance —> register Node —> initialize resources —> Node
ready
Node - - watched capacity / readiness changes - -> scheduler queue / retry
kube-scheduler — re-evaluate feasible Nodes / bind —> Pod placement

```

NodeClaim 是容量请求；不可变的是容量规格，不是随阶段推进的 status。检查 Launched、Registered、Initialized 等条件，可以区分云侧启动、Node 注册和资源初始化问题。Node Ready 也不替代 scheduler 的约束检查；即使扩容成功，Pod 仍可能因 affinity、taint 或拓扑约束无法放置。Karpenter 观察 Pod/NodePool/NodeClass 并重新计算需求，事件是触发重新评估的反馈，不是 scheduler 向它发送的同步“创建指定节点”请求。参见 [NodeClaims](#)。

Disruption 是独立于扩容的控制路径：drift/consolidation 受相应 NodePool disruption budgets 和驱逐约束影响，必要时先 pre-spin replacement，再 drain/terminate。Expiration 属于 forceful 路径，不受 NodePool disruption budgets 限速，也不保证先等待替代节点就绪；PDB、do-not-disrupt 与 terminationGracePeriod 对排空和强制终止有不同作用，不能统一理解成“有预算就可安全回收”。参见 [Karpenter Disruption](#)。

D4 · Workload 生命周期

API / 所有权单位	子资源 / 关键状态	成功、失败与重启语义	扩缩 / 发布边界	API / 快照注意事项
jobset : 一组 ReplicatedJobs	子 Job、Pod; DependsOn、coordinator; 可配置共享 PVC 与 retention	success/failure policy 汇总子 Job 结果; 按规则与 maxRestarts 执行 restart strategy, 不等于无限重试	ReplicatedJob 的副本、依赖与 JobSet 重启由 JobSet controller 管理; 不是 serving group 的滚动发布模型	<code>jobset.x-k8s.io/v1alpha2</code> ; 字段可用性以表首 commit 和部署 release 为准
lws / LeaderWorkerSet: 一个 replica 是 leader/worker Pod 组	controller 管理 StatefulSet/Pod 组与服务发现, 观察副本 readiness	按配置的 group restart policy 恢复受影响组; 持续 serving 的 Ready 不等于 batch Completed	replicas 按组扩缩, LWS controller 负责组发布、placement/subgroup 约束	<code>leaderworkerset.x-k8s.io/v1</code> ; Kueue group admission 不提供跨所有 replicas 的原子启动
DisaggregatedSet: 多角色 serving topology / slices	角色映射到子 LWS; 跨角色 revision、readiness、drain 状态	组内失败由于子 LWS 策略处理, DS 协调角色发布与 drain; 不套用 JobSet 完成策略	支持角色维度与 slice 维度扩缩, 由 DS 协调跨角色 rollout	<code>disaggregatedset.x-k8s.io/v1</code> ; 当前能力不能回写成 2026-06-14 Source 已覆盖的结论
kueue Workload: 一次准入所需 PodSets	queue 引用、QuotaReserved/Admitted、AdmissionCheck 状态、eviction/	准入、驱逐与重新排队按策略推进, 业务成功/失败由集成反映; 释放	PodSet 变化、扩缩和重新准入取决于具体 integration;	当前 <code>kueue.x-k8s.io/v1beta2</code> ; 不要与 Kubernetes KEP 中的 Workload/

API / 所有权单位	子资源 / 关键状态	成功、失败与重启语义	扩缩 / 发布边界	API / 快照注意事项
	requeue; 业务子资源仍由原 controller 管理	配额不等于重启业务	Kueue 不接管 rollout	PodGroup 混为一种 API

生命周期依据: [JobSet v1alpha2 API](#)、[JobSet 当前 API 定义](#)、[LWS / DisaggregatedSet Concepts](#)。

D5 · 失败边界

失败点 / 直接影响	自动重试 / 控制器恢复边界	需要人工处理的情况
LocalQueue / ClusterQueue 缺失或 inactive: Workload 无法正常准入	队列恢复、相关依赖就绪后, Kueue 可重新评估; 重试不会自动创建租户所需配置	修正 queue-name、引用、stop policy、权限或缺失依赖, 检查 queue conditions
quota / flavor 不足: 排队或不能完成 reservation	配额释放、共享配额可用后重试; borrowing/preemption 只按已配置策略执行	调整配额、flavor 约束、共享/抢占策略或业务规模; 加 Node 不会自动增加配额
AdmissionCheck / 外部 provisioning 卡住: 有配额仍未 Admitted	对应 check controller 恢复后更新状态; Retry/Rejected 的处理依集成和策略	修复 check controller、外部凭据/服务、provisioning 配置; Kueue 不能保证外部供应成功
topology 无解: Kueue TAS 无可用地或后续 placement 仍不满足	域内容量或标签变化后重算; 不自动放宽 required topology	校验 Topology/ResourceFlavor、PodSet 分组、节点标签与硬约束, 避免假设任意新增 Node 都有效
scheduler Filter 无 feasible Node: Pod 未绑定	相关事件触发 requeue; PostFilter/preemption 仅在策略允许且能产生可行结果时有帮助	修复互斥 affinity/taint/资源条件, 或增加满足约束的容量; 抢占不能解决所有不可行条件
NodeClaim launch / register / initialize 失败: 容量请求未变成可用 Node	Karpenter/provider 按错误和超时路径重试或清理, 仍需观察阶段 conditions; 不承诺每种错误无限自愈	分别检查云配额/可用性/权限、启动与注册配置、网络与初始化资源; 修复 NodeClass / NodePool
JobSet child Job failure: JobSet 可能重启或终止失败	按 failure rules、restart strategy、maxRestarts 恢复; 达到终止条件后不会靠排队重试继续运行	修复镜像/应用/数据或依赖问题, 评估共享 PVC retention 后重新提交/恢复任务
LWS group / DS role-slice 失败: 副本不可用或 rollout/drain 停滞	LWS 按组策略恢复, DS 按角色 readiness 和发布策略继续协调; 不保证跨角色业务状态一致	检查 restart policy、readiness、角色依赖和 drain 协议, 修复持续故障或不兼容版本

失败点 / 直接影响	自动重试 / 控制器恢复边界	需要人工处理的情况
PDB / do-not-disrupt / termination 约束冲突：节点排空停滞或被强制终止	自愿 disruption 等待条件允许；配置的 NodeClaim terminationGracePeriod 到期可强制删除剩余 Pod	核对预算、保护注解、checkpoint/drain 所需时间；排空完成与保住业务进度是不同结果

排障时先定位停在“准入、放置、节点容量、业务生命周期”的哪一层，再看该层的 status/conditions 和 events。上述自动路径依赖有效配置及可恢复的外部条件；细化证据见 [Kueue 队列排障](#)、[kubernetes-scheduler-core-design](#) 与 D3/D4 的官方文档。

KEP 背景与演进（历史设计）

下面保留 sig-scheduling 的设计线：从单 Pod 调度，走向 Workload / PodGroup 作为调度单位。核心 KEP 是 4671-gang-scheduling，后续由 5710-workload-aware-preemption、6012-composite-podgroup-api、6089-was-controller-apis 和 5732-topology-aware-workload-scheduling 继续展开。逐个 KEP 的 Alpha/Beta/GA、是否实现和 feature gate 见 [kubernetes-keps-implementation-matrix](#)；这里的设计与状态表沿用历史笔记，不作为当前发布状态的重新核验。

一句话定位

这组 KEP 的目标是让 Kubernetes 原理解“一个 workload 由一组 Pod 构成，必须以组为单位做准入、放置、抢占和状态反馈”，而不是继续让每个 AI/HPC/batch controller 各自实现一套 gang scheduling 语义。

为什么重要

传统 kube-scheduler 的基本单位是 Pod。对 Web 服务这通常够用，但对分布式训练、MPI、Ray、JobSet、LeaderWorkerSet、multi-host inference 这类 workload 不够：

- 一组 Pod 需要同时启动，缺一个成员就无法训练或推理。
- 部分 Pod 被调度成功会占住昂贵 GPU/NIC/CPU 资源，但 workload 仍然不能运行。
- 抢占不能只看单个 Pod，否则可能驱逐了一批低优先级 Pod，却仍然无法让高优先级 gang 整体可运行。
- controller、scheduler、autoscaler 都需要同一个标准对象来理解“这一组 Pod 是一个调度单元”。

因此这条设计线的关键不是“增加一个插件”，而是建立新的调度 API 层。

核心对象

在这组 KEP 的对象模型中，业务 workload controller 创建或映射三类对象：Workload 保存静态调度策略/模板；PodGroup 承载运行时调度单元及 minCount、priority、status/conditions；Pod 通过 spec.schedulingGroup 引用所属 PodGroup。这里的对象层级不同于 D1 中 Kueue 的准入 API。

Workload 是策略模板，表达这个 workload 的调度层级和规则。它应该相对稳定，适合由 Job、JobSet、LWS、TrainJob、MPIJob 等 controller 创建或映射。

PodGroup 是运行时调度实例，表达某一次实际要一起调度的 Pod 集合。它有自己的生命周期、状态和垃圾回收关系。KEP 明确把 PodGroup 从 Workload 中解耦，是为了避免把大量运行时状态塞进一个长期对象，导致 etcd 对象过大、状态更新冲突和生命周期混乱。

Pod 只引用自己所属的 PodGroup。scheduler 看到 Pod 后，通过这个引用找到 group 语义。

Alpha 到 Beta 的设计演进

第一版 gang scheduling 更像“正确性屏障”：

- 1. PreEnqueue 检查 PodGroup 是否存在、scheduler 是否已经观察到至少 minCount 个 Pod。
- 2. Permit 阶段等待同组 Pod 都到达同一阶段。
- 3. 如果超时或无法满足 minCount，已占用的预约状态释放，整组回退。

这能保证“不要绑定半个 gang”，但它仍然是以 Pod 为单位推进，性能和全局决策能力有限。

Beta 方向引入 Workload Scheduling Cycle：

- 1. 从 activeQ 取出 PodGroup，获取一次 cluster snapshot。
- 2. 收集该组 pending Pods，执行 group-level placement。
- 3. 满足 minCount 时进入 binding 路径。
- 4. 需要抢占时执行 group-aware preemption 并重试；仍无法放置时标记 PodGroup unschedulable/backoff。

这个变化很关键：scheduler 不再把 group 成员当作独立 Pod 分散处理，而是在一次调度循环里看完整组的可行性。

Workload-aware Preemption

5710-workload-aware-preemption 解决的是 gang scheduling 的下一个问题：如果一组 Pod 需要抢占，victim 也可能是一组 Pod。

它把 preemption 从这四种情况统一起来：

Preemptor	Victim	说明
单 Pod	单 Pod	传统模式。
单 Pod	PodGroup	单 Pod 可能要驱逐一个低优先级 workload 的整体或部分。
PodGroup	单 Pod	高优先级 gang 抢占普通 Pod。
PodGroup	PodGroup	高优先级 workload 替换低优先级 workload。

设计重点是“先判断整个 preemptor 是否可运行，再决定是否真的驱逐”。这避免单个 Pod 的 PostFilter 过早触发 preemption，最后发现整组仍然放不下，造成无意义 disruption。

因此它引入或推动 PodGroupPostFilter 这类 group-level extension point：只有当完整 PodGroup scheduling cycle 失败后，才给插件一次完整上下文来决定是否抢占。

Priority 和 Preemption Unit

PodGroup 引入 priority 语义后，scheduler 会以 PodGroup 的 priority 作为权威值。Pod 自己的 priority 不能和 PodGroup 冲突，否则会造成用户误解：到底是 Pod 级别还是 group 级别在决定抢占？

设计上的取舍：

- Alpha 允许一定程度的文档化差异。
- Beta/GA 倾向于要求同组 Pod 和 PodGroup priority 一致。
- preemptionPolicy 也跟随同样原则，避免一个 PodGroup 内部出现“有的成员可以抢占、有的不能抢占”的不可解释状态。

Topology-aware Workload Scheduling

Gang scheduling 只解决“是否整组一起调度”，还没有解决“这一组 Pod 应该怎样靠近放置”。5732-topology-aware-workload-scheduling 把 PodGroup 扩展到 group-level placement：

- AI training 可能希望所有 workers 在同一 rack、zone、NUMA 或高带宽网络域。
- 多 Pod 不能只逐个用 pod affinity，因为逐个放置会错过组级最优解。
- scheduler 需要先生成候选 placement，再逐个 Pod 做可行性验证和评分。

这条线会和 DRA、NUMA、Node Feature Discovery、GPU topology 强交叉。未来真正有价值的是“workload 级 topology + workload 级 preemption + DRA 设备拓扑”同时成立。

Controller API 设计

6089-was-controller-apis 的核心判断是：不要强迫所有 workload controller 暴露完全相同的用户 API。Job、JobSet、LWS、TrainJob 的用户心智不同，如果硬塞一个统一 schema，反而会阻塞集成。

它选择的方向是：

- 在 scheduling.k8s.io 提供可复用 building blocks。
- 各 controller 把这些 building blocks 嵌入自己的 API。
- 用共享 workloadbuilder library 把 controller-native API 转换成 scheduler-facing Workload / PodGroup / CompositePodGroup。

这是一种“局部 API 一致性优先于全局强统一”的设计。代价是不同 workload API 的用户体验可能不完全一致；收益是 JobSet、LWS、Kueue 这类生态组件可以更快接入，而不必等待一个完美统一的顶层 workload API。

关键失败模式

失败模式	设计处理
PodGroup 不存在	Pod 在 PreEnqueue 或 filter 路径中等待，不进入普通调度。
只调度出部分成员	不绑定，释放资源，整组回退。
抢占后 group 仍不可运行	通过 group-level post-filter 避免过早驱逐。
PodGroup 状态膨胀	独立 PodGroup 对象承载 runtime status，避免压垮 Workload 对象。
controller API 集成慢	使用 reusable building blocks + workloadbuilder，而不是强制统一 API。
autoscaler 不知道加节点是否有效	当前仍是后续工作，需要和 karpenter / Cluster Autoscaler 继续对齐。

和现有项目的关系

- [kueue](#) 更偏 admission control / quota / queueing。Workload API 让底层 scheduler 有机会原理解 gang 语义。
- [scheduler-plugins](#) 里的 coscheduling 是历史上 out-of-tree 的对照实现；KEP 线是在把相关能力标准化。
- [jobset](#) 管理一组 Job；[lws](#) 管理 leader/worker 组，其当前 DisaggregatedSet 进一步组合多角色子 LWS。业务 API 的生命周期编排与 KEP 的 scheduler-facing 对象需要通过具体集成衔接。
- [karpenter](#) / Cluster Autoscaler 的现有容量反馈不能直接等同于理解原生 PodGroup 全部约束；历史 KEP 所指的 group-aware 扩容协同仍须按当前版本和集成另行核验。

阅读顺序

1. 4671-gang-scheduling：先理解 Workload / PodGroup 的对象边界。
2. 5710-workload-aware-preemption：再看抢占如何从 Pod 级扩展到 workload 级。
3. 5732-topology-aware-workload-scheduling：看 group-level placement。
4. 6089-was-controller-apis：看 Job/JobSet/LWS 等 controller 如何对接。
5. 6012-composite-podgroup-api：最后看多层 workload 如何表达。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
4671-gang-scheduling	implementable / beta, 仍在 beta	v1.35 / v1.37 / v1.38	GenericWorkload	Workload / PodGroup 成为调度单位, scheduler 做 group-level scheduling cycle。
5710-workload-aware-preemption	implementable / beta, 仍在 beta	v1.36 / v1.37 / v1.39	GenericWorkload	抢占逻辑从单 Pod 扩展到 PodGroup, 避免先驱逐后发现整组仍不可运行。
6012-composite-podgroup-api	implementable / alpha, 仍在 alpha	v1.37 / v1.38 / v1.40	CompositePodGroup	表达多组件 workload 的组合 PodGroup, 适合 JobSet/LWS/训练任务。
6089-was-controller-apis	implementable / alpha, 仍在 alpha	v1.37 / v1.38 / v1.39	WorkloadWithJob	给 workload controller 提供可嵌入 building blocks 和转换库。
5732-topology-aware-workload-scheduling	implementable / beta, 仍在 beta	v1.36 / v1.37 / v1.39	TopologyAwareWorkloadScheduling	group-level placement, 把一组 Pod 放到满足拓扑目标的节点集合。

追踪重点

后续要继续跟:

- Workload / PodGroup API 版本是否进入 beta/stable。
- Job、JobSet、LWS、Kueue 等 controller 的真实集成方式。
- PodGroupPostFilter 是否稳定, out-of-tree PostFilter 如何迁移。
- PodGroup 与 DRA ResourceClaim、topology-aware scheduling、Cluster Autoscaler/Karpenter 的联动。
- 生产指标: PodGroup scheduling latency、失败原因、binding 成功率、preemption 造成的 disruption。

llm-d / Kubernetes SIGs 候选项目地图

这页把 llm-d 组织和 kubernetes-sigs 组织里值得继续加入 wiki 的项目，按工程维度拆成候选清单。它不是 star 排行榜，而是服务于当前知识库的补全路线：哪些项目能帮助理解 llm-inference、[model-serving-operator](#)、[kubernetes-dra](#)、inference-routing、[cloud-native-security](#) 和 Kubernetes 控制器生态。

[!warning] 2026-09-22 当前状态
[llm-d-workload-variant-autoscaler](#) 的摄入/架构研究已完成，但它现为 deprecated 的历史设计，不应用于当前采用。llm-d 新部署的当前路径是 [EPP metrics](#) → [KEDA Prometheus scaler](#) → [HPA](#)。

本次核验基于 GitHub API 当前公开仓库元数据（2026-06-13）。当前已收录的同域项目包括 [llm-d](#)、llm-d-batch-gateway、llm-d-benchmark、[llm-d-workload-variant-autoscaler](#)（已完成摄入的 deprecated/历史研究）、llm-d-inference-sim、[gateway-api](#)、[gateway-api-inference-extension](#)、agent-sandbox、[dra-driver-nvidia-gpu](#)。

总体优先级

- P0：直接补当前 AI Infra / LLM serving / K8s control plane 选型缺口
- P1：强相关，但更偏评测、诊断、迁移、配套或特定场景
- P2：后续补充，先进入 backlog，不急于做完整源码架构页

优先级	项目	维度	加入价值
P0 已完成	llm-d-batch-gateway	LLM batch serving	已补正式源码架构页，把 llm-d 从在线 inference 扩展到 OpenAI Batch API / 离线 batch workload。
P0 已完成	llm-d-benchmark	LLM serving benchmark	已补正式源码架构页，作为 llm-inference-serving-project-map 的性能评测入口。
P0 已完成（历史研究）	llm-d-workload-variant-autoscaler	deprecated serving autoscaling	已补正式源码架构页，保留 variant / workload autoscaling 和资源经济的历史设计；不用于当前采用。
P0 已完成	llm-d-inference-sim	inference simulator	已补正式源码架构页，无 GPU 模拟 vLLM 行为，适合研究调度、benchmark 和路由策略。
	kucue		

优先级	项目	维度	加入价值
P0 已完成		调度 / 队列	K8s 原生 Job queueing, 是 AI/HPC/batch workload 控制面的核心项目。
P0 已完成	karpenter	节点弹性 / 成本	Node autoscaler, 连接 serving SLO、GPU 成本和容量弹性。
P0 已完成	controller-runtime	Operator SDK	现代 K8s controller 的通用抽象层, 当前大量实体页都依赖它。
P0 已完成	kubebuilder	CRD / controller 脚手架	和 controller-runtime 一起构成 Operator 工程主线。
P0 已完成	metrics-server	可观测 / autoscaling	HPA/VPA 基础指标源, 解释 autoscaling 体系必需。
P0 已完成	external-dns	网络 / DNS	K8s service/ingress/gateway 到 DNS record 的控制器代表。
P0 已完成	secrets-store-csi-driver	存储 / 凭据	CSI 方式注入外部 secret, 连接凭据治理和 runtime security。
P0 已完成	kind	计算 / 测试集群	本地 K8s 测试底座, 很多 controller/operator 项目的开发环境基线。
排除	aws-load-balancer-controller / aws-ebs-csi-driver / aws-efs-csi-driver	AWS 专项	用户明确本轮不需要。

llm-d 组织候选

项目	优先级	当前定位	应放入的 wiki 主题
llm-d-batch-gateway	P0 已完成	OpenAI-compatible /v1/batches 和 /v1/files, 把 batch job 分成 API server、processor、queue、storage、GC 等组件。	llm-inference-serving-project-map、batch-inference、 model-serving-operator
llm-d-benchmark	P0 已完成	llm-d benchmark lifecycle/workspace/harness 编排。	LLM serving benchmark、llm-inference

项目	优先级	当前定位	应放入的 wiki 主题
llm-d-workload-variant-autoscaler	P0 已完成（历史研究）	Deprecated 的 distributed inference workload variant autoscaler 架构快照，仅供迁移/设计研究。	历史 autoscaling、K8s resource economics、 model-serving-operator
llm-d-inference-sim	P0 已完成	轻量模拟 vLLM 行为，不需要 GPU 或真实大模型。	simulator、scheduler、benchmark
llm-d-latency-predictor	P1 已完成	给 inference scheduler 的 ML-based latency scoring service。	latency predictor、inference-routing
llm-d-prism	P1 已完成	分布式推理性能分析 dashboard，把 benchmark 数据做交互式分析。	observability、performance analysis
llm-d-pd-utils	P1 已完成	Agentic Skills + scripts，用于 P/D 部署 preflight、GPU topology、RDMA/NCCL/network tests。	P/D diagnostics、GPU/network validation
llm-d-batch-gateway 组件拆分	P2	API server / batch processor / GC / storage backend 可后续拆成细页。	batch serving internals
llm-d-inference-payload-processor	P2	inference payload processor，目前公开说明较薄。	router/batch 辅助组件
llm-d-infra / templates / .github	暂缓	组织 CI、模板和治理基础设施。	不建议单独建架构页
archived deployer / model-service / routing-sidecar	暂缓	旧组件或已归档项目。	仅在追溯历史设计时引用

llm-d 的知识结构缺口

当前 wiki 已经覆盖 [llm-d](#) 总入口、llm-d-router、llm-d-kv-cache 和 P0 四类外围能力：

- **Batch inference**：llm-d-batch-gateway 让在线 serving 与离线 batch workload 可以共享下游 router/model server，但 SLO、队列、存储和计费边界不同。
- **Performance evaluation**：llm-d-benchmark / llm-d-prism / llm-d-inference-sim 可以把“架构上可行”推进到“如何测、如何复现、如何调参”。
- **Autoscaling/resource economics（历史）**：[llm-d-workload-variant-autoscaler](#) 是 deprecated 的历史设计研究，曾探索与 Kueue、Karpenter、GPU 资源层的连接；当前新部署采用 EPP metrics → KEDA/HPA。
- **P/D deployment diagnostics**：llm-d-pd-utils 把 GPU topology、RDMA、NCCL、NIXL 等实际部署问题显性化。

Kubernetes SIGs 候选：按工程维度

网络

项目	优先级	为什么值得加入
external-dns	P0 已完成	从 Service/Ingress/Gateway 等 K8s 对象动态维护外部 DNS，是典型“声明式网络控制器”。
aws-load-balancer-controller	排除	AWS LB / Ingress / Gateway API 的生产代表，适合和 gateway-api 放在同一条网络入口路线。
gateway-api	已有	已收录，作为新一代流量入口 API 基础。
gateway-api-inference-extension	已有	已收录，作为 LLM inference endpoint picking 标准层。
ingress2gateway	P1 已完成	从 Ingress 迁移到 Gateway API 的工具，适合补迁移路线。
apiserver-network-proxy	P1 已完成	control plane 到节点网络代理，适合理解托管 K8s / 私有网络控制面连接。
kube-agentic-networking	P1 已完成	Agent/tool networking policy governance，和 agent-sandbox、 agentgateway 方向高度相关。
dragnet / cni-dra-driver	P2	DRA 和高性能网络结合，适合后续连接 GPU/RDMA/AI workload。

存储 / Secret / 数据面挂载

项目	优先级	为什么值得加入
secrets-store-csi-driver	P0 已完成	把外部 secret store 通过 CSI volume 注入 Pod，连接 agent-credential-isolation 和 cloud-native-security 。
aws-ebs-csi-driver	排除	块存储 CSI driver 的云厂商代表。
aws-efs-csi-driver	排除	文件存储 CSI driver 的云厂商代表，适合与 EBS 对比。
nfs-subdir-external-provisioner	P1 已完成	轻量 NFS 动态 PV，适合本地/实验集群。
sig-storage-local-static-provisioner		

项目	优先级	为什么值得加入
	P1 已完成	local PV 静态 provisioner，适合理解本地盘和调度绑定。
sig-storage-lib-external-provisioner	P1 已完成	CSI/external provisioner 的库和控制器抽象。
vsphere-csi-driver / gcp-* / azure* / alibaba-cloud-csi-driver	P2	云厂商矩阵补充，按需要选择代表，不必全量展开。

调度 / 资源 / 队列

项目	优先级	为什么值得加入
kueue	P0 已完成	K8s-native Job queueing，是 AI/HPC/batch workload 多租户队列核心。
karpenter	P0 已完成	节点级弹性和容量经济，适合接 LLM serving 的成本与 SLO。
scheduler-plugins	P0 已完成	kube-scheduler framework 的 out-of-tree 插件集合，理解调度扩展必读。
descheduler	P1 已完成	通过策略触发重调度，补“调度后优化”维度。
kwok	P1 已完成	无 kubelet 模拟大量节点/集群，适合调度和控制面扩展测试。
node-feature-discovery	P1 已完成	发现节点硬件能力，连接 GPU、NUMA、加速器调度。
kube-scheduler-simulator	P1 已完成	可视化/模拟 scheduler 行为。
dra-driver-nvidia-gpu	已有	已收录，作为 DRA + GPU 设备分配代表。

可观测 / 性能 / 诊断

项目	优先级	为什么值得加入
metrics-server		

项目	优先级	为什么值得加入
	P0 已完成	HPA/VPA 的基础资源指标源，K8s autoscaling 入口。
prometheus-adapter	P0 已完成	把 Prometheus 指标暴露为 custom/external metrics API，连接高级 autoscaling。
inference-perf	P1 已完成	GenAI inference performance benchmarking tool，可与 llm-d-benchmark 对比。
headlamp	P1 已完成	K8s UI / debugging / monitoring，可和 kubewall 、k8m 对比。
usage-metrics-collector	P2	容量和使用率指标收集，适合补平台容量管理。
resource-state-metrics / logtools / instrumentation-tools	P2	作为 SIG Instrumentation 工具链补充。

计算 / Runtime / 节点

项目	优先级	为什么值得加入
kind	P0 已完成	本地 Kubernetes in Docker，是 controller/operator 开发和 CI 的事实基线。
kubespray	P0 已完成	生产集群部署自动化，适合理解集群生命周期和裸金属/on-prem。
cri-tools	P0 已完成	CRI CLI + validation，理解 kubelet/runtime 边界。
security-profiles-operator	P1 已完成	seccomp/AppArmor/SELinux profile operator，连接 runtime security。
agent-sandbox	已有	已收录，AI Agent runtime 的 sandbox CRD。
lws	P1 已完成	LeaderWorkerSet，用一组 Pod 表达 leader/worker 分布式 workload。
jobset	P1 已完成	分布式 ML/HPC workload API，和 Kueue、batch serving 强相关。

API / Operator / 控制器开发

项目	优先级	为什么值得加入
controller-runtime	P0 已完成	Manager、cache、client、reconcile、webhook、envtest 的现代 controller 抽象层。
kubebuilder	P0 已完成	CRD/controller 项目脚手架和代码生成路径。
controller-tools	P0 已完成	CRD、RBAC、webhook、object deepcopy 等生成工具链。
cluster-api	P0 已完成	声明式集群生命周期管理，适合接多集群和平台工程。
kustomize	P1 已完成	Kubernetes YAML 定制工具链，GitOps/配置管理基础。
kro	P1 已完成	Kube Resource Orchestrator，适合与 Crossplane Composition / higher-level API 对比。
apiserver-builder-alpha	P2	aggregated apiserver/controller 旧路线，可作为历史参考。

AI Infra / Agent 交叉

项目	优先级	为什么值得加入
mcp-lifecycle-operator	P1 已完成	声明式管理 MCP Servers，和 mcp、 agentgateway 、declarative-agent-management 直接相关。
kube-agentic-networking	P1 已完成	Agent/tool 网络策略治理，适合补 Agent runtime 安全边界。
ai-conformance	P2	AI conformance definition/proposals/tests，待生态更稳定后再摄入。
inference-perf	P1 已完成	GenAI inference benchmark，与 llm-d 、 aibrix 、 kserve 有直接交集。

推荐实施顺序

第一批：直接补 AI Infra 选型缺口

- 1. llm-d-batch-gateway（已完成）

2. [llm-d-benchmark](#) (已完成)
3. [llm-d-workload-variant-autoscaler](#) (历史研究已完成; deprecated, 不用于当前采用)
4. [llm-d-inference-sim](#) (已完成)
5. [kueue](#) (已完成)
6. [karpenter](#) (已完成)
7. [metrics-server](#) (已完成)
8. [prometheus-adapter](#) (已完成)
9. [inference-perf](#) (已完成)
10. [lws](#) (已完成)
11. [jobset](#) (已完成)

这批可以让 [llm-inference-serving-project-map](#) 从“引擎/serving stack”扩展到 batch、benchmark、queueing、当前 EPP metrics → KEDA/HPA autoscaling 与 distributed workload API; WVA 仅保留为历史研究。

第二批：补 Kubernetes 控制器和平台工程底座

1. [controller-runtime](#) (已完成)
2. [kubebuilder](#) (已完成)
3. [controller-tools](#) (已完成)
4. [cluster-api](#) (已完成)
5. [kind](#) (已完成)
6. [cri-tools](#) (已完成)
7. [external-dns](#) (已完成)
8. [secrets-store-csi-driver](#) (已完成)
9. [scheduler-plugins](#) (已完成)
10. [node-feature-discovery](#) (已完成)

这批可以让 [k8s-core-controller-map](#) 从学习路径升级成更完整的平台工程架构图。

第三批：按专项补齐

- 网络专项: [ingress2gateway](#)、[apiserver-network-proxy](#)、[kube-agentic-networking](#) 已完成; [aws-load-balancer-controller](#) 按用户要求排除; [dragnet](#) 留作 P2。
- 存储专项: [nfs-subdir-external-provisioner](#)、[sig-storage-local-static-provisioner](#)、[sig-storage-lib-external-provisioner](#) 已完成; [aws-ebs-csi-driver](#)、[aws-efs-csi-driver](#) 按用户要求排除。
- 安全专项: [security-profiles-operator](#)、[secrets-store-csi-driver](#)、[kube-agentic-networking](#) 已完成。
- 工具/测试专项: [kustomize](#)、[kwok](#)、[kube-scheduler-simulator](#)、[headlamp](#)、[kro](#)、[mcp-lifecycle-operator](#) 已完成; [krew](#)、[e2e-framework](#)、[kubetest2](#) 留作 P2。

和现有页面的关系

- 更新 [llm-inference-serving-project-map](#): 加入 batch gateway、benchmark、simulator、Kueue/Karpenter/LWS/JobSet 与当前 EPP metrics → KEDA/HPA 扩缩路径; deprecated WVA 只作为历史 autoscaler 研究。
- 更新 [k8s-core-controller-map](#): 从 controller-runtime/kubebuilder 学习路径, 扩展成网络、存储、调度、可观测、计算、API/operator 的维度地图。
- 更新 [k8s-gpu-device-stack](#): 把 node-feature-discovery、Kueue、Karpenter、JobSet、LWS 放入 GPU/AI workload 的调度前置层。
- 更新 [cloud-native-security](#): 把 secrets-store-csi-driver、security-profiles-operator、kube-agentic-networking 作为 runtime/credential/network policy 三条线。
- 更新 [model-serving-operator](#): 把 [llm-d](#) 外围组件与 K8s SIG 的 queueing/autoscaling/distributed workload API 连起来。

OpenKruise 项目候选地图

这页把 openkruise GitHub 组织里值得纳入 wiki 的项目按工程价值排序。目标不是收集全部仓库，而是补齐 Kubernetes workload automation、progressive delivery、specialized workload、observability 和 controller/operator isolation 这些架构维度。

本次核验基于 GitHub API 当前公开仓库元数据（2026-06-15）。当前已收录的正式源码架构页是 [openkruise-agents](#)；本页中的 P0/P1 候选已经补成实体页，并统一归入 [kubernetes-workload-automation](#) 这个整体概念，而不是按每个项目维度继续拆概念页。

总体分层



P0：应优先摄入

项目	当前状态	加入价值	应进入的 wiki 主题
openkruise-agent	5267 stars，最近 push 2026-06-08，release v1.8.3，	OpenKruise 主仓，是理解 Kubernetes workload 增强的核心入口。重点是 CloneSet、Advanced StatefulSet、Advanced DaemonSet、SidecarSet、UnitedDeployment、WorkloadSpread、ImagePullJob	k8s-core-controller-map 、 kubernetes-kubernetes-workload-automation

项目	当前状态	加入价值	应进入的 wiki 主题
	CNCF incubating	等比原生 Deployment/StatefulSet 更细的自动化控制。	
openkruise-controllers	254 stars, 最近 push 2026-06-13, release v0.6.2	渐进式发布 / enhanced rollout 控制面。适合和 Argo Rollouts、原生 Deployment rollout、Gateway/Ingress 流量切分对比。	kubernetes-workload-automation 、 gitops 、流量治理、发布控制面
kruise-cgame	1059 stars, 最近 push 2026-05-22, release v1.0.0	Kubernetes game server management, 场景独特且成熟度相对高。适合对比 Agones、StatefulSet、GameServerSet、专用 workload operator。	kubernetes-workload-automation 、specialized workload operator、游戏服务器

已收录

项目	当前状态	wiki 位置	后续动作
openkruise-e/agents	202 stars, 最近 push 2026-06-12, release v0.3.0	openkruise-agents / src-openkruise-agents-architecture	后续可补 agents-api 和 charts 作为部署/API 支撑材料, 不需要重新排进 P0。

P1: 专题补强

项目	当前状态	加入价值	建议处理
kruise-state-metrics	12 stars, 最近 push 2025-04-28	OpenKruise CRD metrics addon, 适合补 kube-state-metrics / Prometheus 体系下的自定义 workload 可观测。	已建实体页; 后续若摄入主仓, 可补更细的 metrics schema。
kruise-tools	55 stars, 最近 push 2025-07-22, release v1.2.2	Kruise libraries/tools, 偏工具链与开发/运维辅助。	已建实体页; 作为主仓工具链支撑引用。
kruise-dashbord	0 stars, 最近 push 2025-11-19	CloneSet / Advanced StatefulSet / Advanced DaemonSet dashboard, 补 OpenKruise 运维入口。	已建实体页; 后续可放入 Kubernetes dashboard / ops UI 对比。

项目	当前状态	加入价值	建议处理
contro llerme sh	64 stars, 最近 push 2023-05-30	controller/operator isolation 管理方案，概念有价值但活跃度低。	已建实体页并接入 kubernetes-workload-automation 的 controller operation boundary, 作为 controller 高可用、隔离和多 operator 管理设计参考。

P2：支撑材料

项目	用法
openkruise/charts	Helm 部署材料。适合作为 kruise、rollouts、 openkruise-agents 摄入时的部署来源，不建议单独做实体页。
openkruise/kruise-api	kruise API definition repo，摄入主仓时引用。
openkruise/kruise-rollout-api	rollouts API definition repo，摄入 Rollouts 时引用。
openkruise/agents-api	openkruise-agents API definition repo，后续补充 API 演进时引用。
openkruise/ openkruise.io	官方文档站。每个正式 \$ingest-codebase 都应同时看对应文档。

暂不建议单独加入

项目	原因
openkruise/website	已归档，迁移到 openkruise.io。
openkruise/controller-tools	fork，不适合作为 OpenKruise 生态主项目。
openkruise/client-java / openkruise/client-rust	SDK，除非后续做多语言 client 对比。
openkruise/samples / community / summer_of_code	示例和治理材料，可作为背景，不做架构页。
openkruise/federation / controllermesh-api / game.openkruise.io / kruise-helm / kruise-argo / utils	当前更像旧实验、API/文档/插件材料，优先级低。

推荐实施顺序

1. \$ingest-codebase <https://github.com/openkruise/kruise>, 扩写 [openkruise-kruise](#)。
2. \$ingest-codebase <https://github.com/openkruise/rollouts>, 扩写 [openkruise-rollouts](#), 并同步更新 [kubernetes-workload-automation](#) 中的 release governance 部分。
3. \$ingest-codebase <https://github.com/openkruise/kruise-game>, 扩写 [kruise-game](#)。
4. 视主仓内容决定是否单独补 kruise-state-metrics
5. 如果要研究 controller/operator isolation, 再补 controllermesh

和现有知识库的关系

- [k8s-core-controller-map](#) / [kubernetes-workload-automation](#): [openkruise-kruise](#) 可以补齐“原生 workload 之外的增强 workload controller”路线。
- [cloud-native-security](#): SidecarSet、ImagePullJob、runtime hooks 和 workload spread 会影响节点/镜像/sidecar/变更安全边界。
- [agent-runtime-sandbox-project-map](#): [openkruise-agents](#) 已经接入 Agent sandbox 路线, 后续可补 agents-api 作为 API source。
- [model-serving-operator](#) / AI infra: kruise 的 workload spread、pre-download image、advanced StatefulSet 可能为大模型服务滚动、镜像预热和多可用区部署提供对照。

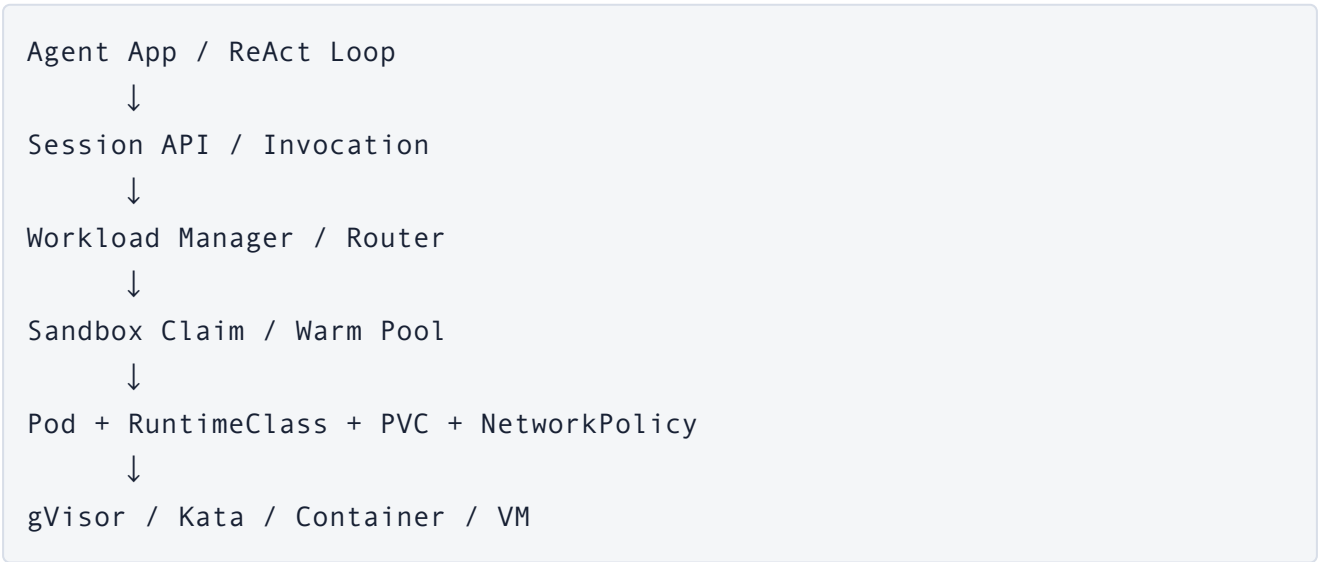
Agent Runtime Substrate

Agent runtime substrate 指承载大量 agent-like workload 的底层运行基座：worker/actor 生命周期、状态保存、唤醒路由、沙箱隔离、快照恢复、并发密度和调度策略。

为什么单独成概念

传统 agent-sandbox 解决“一个 Agent 会话如何安全运行”，而 substrate 解决“一批 Agent 会话如何高密度、可恢复、可路由地运行”。[openkruise-agents](#) 则把这条线推向 K8s 平台化：用 SandboxSet/Claim 管 warm pool 和领取协议，用 E2B API 暴露产品入口，用 Envoy route registry 把请求导向具体 sandbox。[src-substrate-architecture](#) 把这个方向推进到 WorkerPool/ActorTemplate、Redis/ValKey actor state、Envoy 唤醒路由、gVisor snapshot；agentscope-runtime 则代表 framework app 到 Agent-as-a-Service 的服务化路线。

从应用到执行环境



项目对比

项目	层级	重点
agent-sandbox	sandbox primitive	单会话隔离和有状态容器
openkruise-agents	sandbox lifecycle platform	SandboxSet/Claim warm pool + E2B API + Envoy route + runtime/CSI/identity 扩展
agentcube	session orchestration	Router + WorkloadManager + WarmPool
Substrate	runtime substrate	actor/worker multiplexing + snapshot/wakeup

项目	层级	重点
agentscope-runtime	Agent-as-a-Service	Agent app 服务化和部署器

选型提示

如果问题是安全隔离，看 sandbox；如果问题是 Kubernetes 上的 E2B-like sandbox 平台，看 [openkruise-agents](#)；如果问题是高密度运行、唤醒和恢复，看 substrate；如果问题是把 framework app 暴露成服务，看 Agent-as-a-Service runtime。

Agent Substrate 架构与设计思路分析

原文: raw/substrate-architecture-analysis.md · 仓库: <https://github.com/agent-substrate/substrate> · 分析版本 HEAD a3f4474

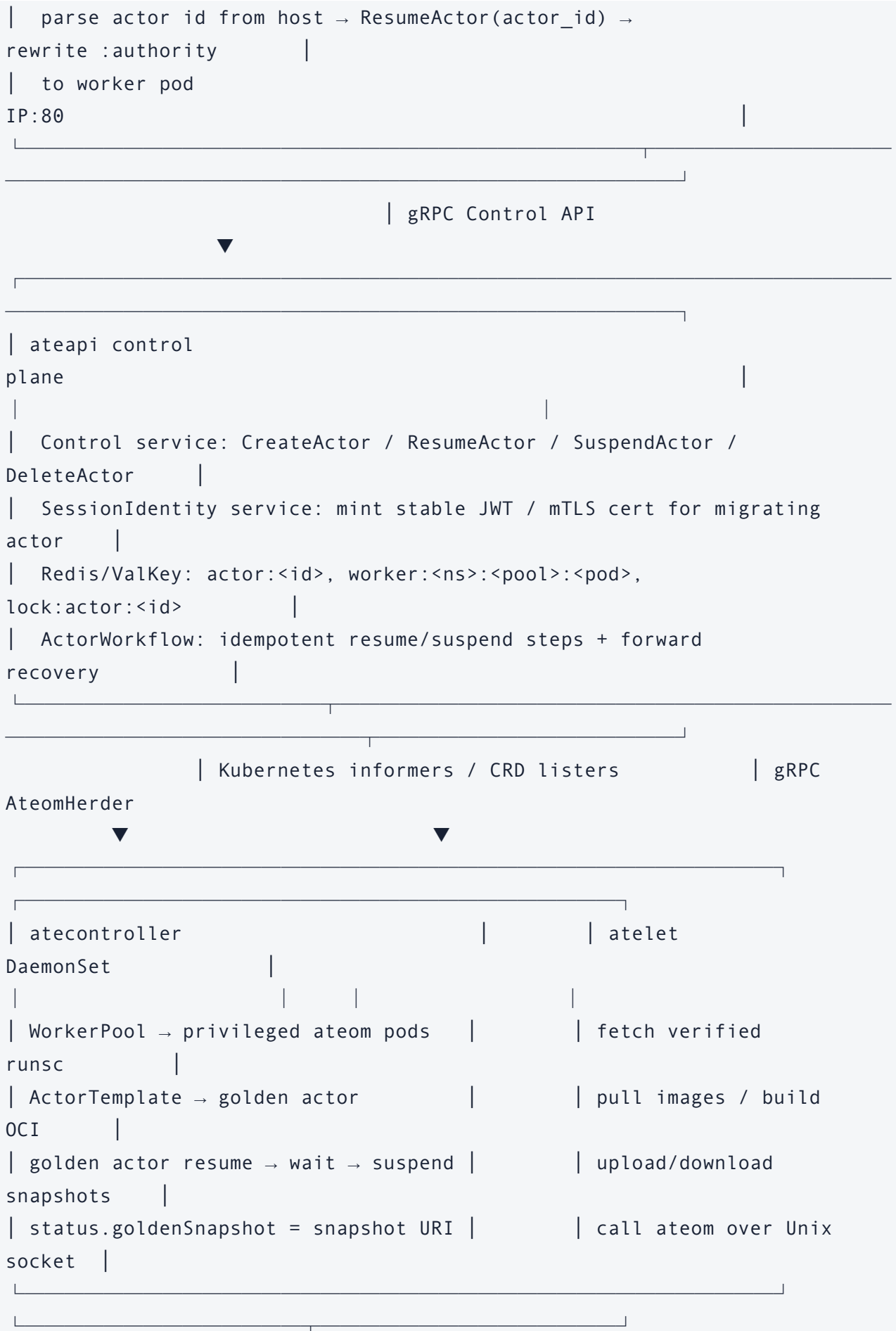
一句话定位

Agent Substrate 是一个建立在 [kubernetes](#) 之上的高密度 [Agent Runtime / Sandbox](#) substrate: Kubernetes 只负责声明式容量和基础设施, Actor 的高频生命周期、resume/suspend、worker 分配和路由唤醒由 Substrate 自己的 Redis/ValKey 控制面、Envoy ext_proc router、atelet node supervisor 和 gVisor atecom worker 承担。

它补的是 agent-sandbox 和 [agentcube](#) 之间更底层、更激进的一层: 不是“创建一个有状态 sandbox”, 而是把大量 idle actor multiplex 到较少 warm worker 上, 通过 golden snapshot、runtime snapshot、DNS/router 唤醒和 gVisor restore 降低激活延迟。官方 architecture 文档也明确标注“Much of this architecture is aspirational”, 所以要把愿景和当前代码实现分开看。

核心架构图







```

| ateom-gvisor worker
pod
|
| privileged pod, one active actor at a
time
| link-local veth: pod netns ateom0 ↔ actor netns
eth0
| nftables DNAT/MASQ compatibility
bridge
| runc create/start/checkpoint/restore pause + app
containers

```



```

| Object storage
snapshots
|
| gs:// or s3://
prefix
| checkpoint.img.zstd / pages.img.zstd /
pages_meta.img.zstd

```

模块分层

层 / 模块	职责
CRD / public API	定义 WorkerPool、ActorTemplate 和 actor lifecycle gRPC API; K8s CRD 承载低频配置, gRPC 承载高频 actor 操作。
K8s controller	把 WorkerPool 物化为 privileged ateom Deployment; 把 ActorTemplate 驱动成 golden actor → golden snapshot → Ready。

层 / 模块	职责
Control plane / DB	对外提供 Control / SessionIdentity gRPC; 用 Redis/ValKey 存 actor/worker 状态和 lock; 用 workflow step 实现 resume/suspend forward recovery。
Node supervisor	每节点/每 worker 管理 runsc、镜像拉取、OCI bundle、snapshot 上传下载, 并转调 ateam。
Sandbox herder	privileged worker pod 内的 gVisor 控制进程, 串行调用 runsc create/start/checkpoint/restore, 管理 actor netns/veth/nftables。
Router / DNS	用 CoreDNS stub domain + Envoy xDS + ext_proc, 把 actor hostname 转成 ResumeActor 调用和 worker pod IP 转发。

关键边界: [kubernetes](#) API 不存每个高频 actor transition; WorkerPool/ActorTemplate 是声明式配置, Actor/Worker 的运行状态在 Redis/ValKey, 真正 runsc 操作在 worker pod 本地完成。

关键数据流

ActorTemplate 到 golden snapshot

```

ActorTemplate CR created
|
▼
atecontroller ActorTemplateReconciler
|   PhaseInitial
▼
ateapi.CreateActor(goldenActorID)
|
▼
status.phase = ResumeGoldenActor
|
▼
ateapi.ResumeActor(goldenActorID)
|   no actor snapshot + no golden snapshot
▼
atelet.Run → ateam.RunWorkload → runsc create/start
|
▼
wait 20s
|
▼

```

```

ateapi.SuspendActor(goldenActorID)
|
▼
atelet.Checkpoint → ateam.CheckpointWorkload → upload checkpoint files
|
▼
ActorTemplate.status.goldenSnapshot = snapshot URI
ActorTemplate.status.phase = Ready

```

请求唤醒和路由

```

HTTP request
Host: <actor-id>.actors.resources.substrate.ate.dev
|
▼
CoreDNS stub domain routes to atenet-router
|
▼
Envoy RequestHeaders ext_proc
|
▼
parseActorID(host)
|
▼
singleflight ResumeActor(actor-id)
|
▼
ateapi resume workflow
|
├─ LoadActorForResume
├─ AssignWorker
├─ CallAteletRestore / Run
└─ FinalizeRunning
|
▼
actor.ateom_pod_ip returned
|
▼
mutate :authority to <worker-ip>:80
|

```

▼
Envoy dynamic_forward_proxy forwards request

Resume / Suspend workflow

```
ResumeActor(actor-id)
|
▼
Acquire lock: lock:actor:<id>
|
▼
Load actor + ActorTemplate
|
▼
Find existing assigned worker or random free worker
|
▼
Update worker(actor_id=...) then actor(status=RESUMING, worker fields)
|
▼
if actor.last_snapshot: atelet.Restore(last_snapshot)
else if template.goldenSnapshot and !boot: atelet.Restore(goldenSnapshot)
else: atelet.Run(template spec)
|
▼
Update actor(status=RUNNING)

SuspendActor(actor-id)
|
▼
Acquire lock: lock:actor:<id>
|
▼
Load actor + ActorTemplate
|
▼
Mark actor SUSPENDING + in_progress_snapshot
|
▼
atelet.Checkpoint(snapshot URI)
```



```
Free worker, move in_progress_snapshot → last_snapshot,  
clear worker fields, status=SUSPENDED
```

设计决策与哲学

- K8s 管低频配置，Substrate 管高频 actor 状态：WorkerPool/ActorTemplate 是 CRD，actor/worker 运行态进入 Redis/ValKey。Redis store 注释明确说明跨 slot 不能原子更新 actor 和 worker，所以系统必须接受 forward recovery。
- Golden snapshot 是冷启动优化根：ActorTemplate 会生成一个 golden actor，启动后暂停成 snapshot；后续 actor 可从 golden snapshot 恢复，而不是每次重新跑初始化。
- Resume/suspend 采用 client-driven forward recovery：每个 workflow step 都能判断是否已完成，失败后靠客户端重试继续推进；每个 actor 还有 lock:actor:<id> 防同 actor 并发操作。
- worker 是 warm capacity，不是 actor 本体：WorkerPool 控制 privileged ateam pods 的数量；Actor 只是逻辑记录，运行时才绑定某个 worker pod。
- gVisor/runsc 是当前 snapshot 执行后端：atelet 负责准备 runsc、OCI bundle 和 snapshot 文件；ateom-gvisor 串行调用 runsc 的 create/start/checkpoint/restore。
- 网络唤醒还是兼容桥阶段：当前路由按 actor DNS 唤醒后把请求改写到 worker IP:80；ateom 用 veth + nftables 做 actor ingress/egress 兼容，后续仍需要更严格的 gateway/policy 层，和 [agentgateway](#) 一类项目会形成互补。

关键组件深入解读

ateapi ActorWorkflow

ActorWorkflow 是当前源码里最能体现 Substrate 设计哲学的部分。它不是写一个大函数完成 resume/suspend，而是把流程拆成可跳过的步骤：resume 依次加载 actor、分配 worker、调用 atelet restore/run、最终置 running；suspend 依次加载 actor、标记 suspending、调用 atelet checkpoint、释放 worker 并提交 snapshot。

这个拆法服务于两个现实约束：actor 与 worker 状态跨 Redis Cluster key，不可能用一个强事务同时修改；调用 atelet/ateom/runsc 是外部副作用，失败可能发生在任意位置。因此 workflow 采用“状态已写入就能被下一次请求识别并继续”的模型。

atelet + ateam-gvisor

atelet 和 ateam 的分工很清楚：atelet 是 node supervisor，接收 ateapi 的 Run/Checkpoint/Restore 请求；ateom 是 worker pod 内的 gVisor herder，真正执行 runsc。atelet 负责下载/校验 runsc、拉 OCI image、展开 rootfs、生成 OCI config、下载/上传 checkpoint zstd 文件。ateom 负责创建 interior network namespace、建立 veth、安装 nftables、执行 runsc create/start/checkpoint/restore。

当前实现假设一个 worker pod 同时只运行一个 active actor；ateom 的 mutex 和 nftables 表清理逻辑都体现了这一点。

相关页面

- [agent-runtime-sandbox-selection-map](#)
- [agent-runtime-sandbox-project-map](#)
- [agent-sandbox](#)
- [agentcube](#)
- [kubernetes](#)
- [agentgateway](#)

AgentCube

AgentCube 是 Volcano 社区面向 AI Agent / Code Interpreter 的 Kubernetes 原生会话编排层。它不是替代 agent-sandbox，而是在 agent-sandbox 的 Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool 之上，补上应用级会话、HTTP 入口、代码执行 API、SDK 入口和生命周期回收。

详细架构见 [src-agentcube-architecture](#)。

关键事实

- 仓库: [volcano-sh/agentcube](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-12, 已有 tag v0.1.0
- 本 wiki 架构分析版本: HEAD 208da32 (2026-06-01)
- 阶段: 已从 Proposal 进入早期 release, 但 API 仍需按上游 agent-sandbox 演进持续核验
- 主要语言: Go 1.24.4, 模块 [github.com/volcano-sh/agentcube](#)
- 核心依赖: [sigs.k8s.io/agent-sandbox v0.1.1](#) (当前 main 的 go.mod 仍如此)
- 2 个 AgentCube CRD: AgentRuntime 与 CodeInterpreter
- 主要组件: Router、WorkloadManager、PicoD、Redis / ValKey session registry

架构定位

AgentCube 把 agent-sandbox 的底层运行时原语包装成更贴近 AI 应用的会话模型:

- AgentRuntime 面向长期运行的 Agent 服务, 声明 targetPort、PodTemplate 和 session timeout。
- CodeInterpreter 面向代码执行工作负载, 声明执行镜像、端口、session timeout、最大会话时长、warm pool 规模和鉴权模式。
- Router 是数据平面, 根据 x-agentcube-session-id 查询 Redis / ValKey, 把请求反向代理到已存在的 Sandbox, 必要时调用 WorkloadManager 创建新 session。
- WorkloadManager 是控制平面, 负责创建 Sandbox 或 SandboxClaim, 等待 Ready、做 endpoint TCP probe、写入 session 映射并执行 GC。
- PicoD 是轻量 HTTP daemon, 提供 /api/execute 与 /api/files, 用 Router 签发的 JWT 做访问控制, 替代传统 SSH 进入容器。

与 agent-sandbox 的结合

AgentCube 复用了 agent-sandbox 的隔离、Pod 生命周期和 WarmPool 能力, 但把它们隐藏在更高层 API 后面:

- 冷启动路径: AgentRuntime / CodeInterpreter -> WorkloadManager -> Sandbox -> Pod。
- 预热路径: CodeInterpreter.warmPoolSize -> SandboxTemplate + SandboxWarmPool -> 首次请求创建 SandboxClaim -> 领取预热 Sandbox。

- 会话路径：Router 从 session registry 解析 sessionId -> endpoint，后续请求不再走 K8s reconcile，而是直接代理到 PicoD 或 Agent 服务。

这个分层让 agent-sandbox 继续专注"安全、有状态、可回收的单实例容器"，AgentCube 则负责"用户请求如何变成一个可复用的 AI / Code session"。

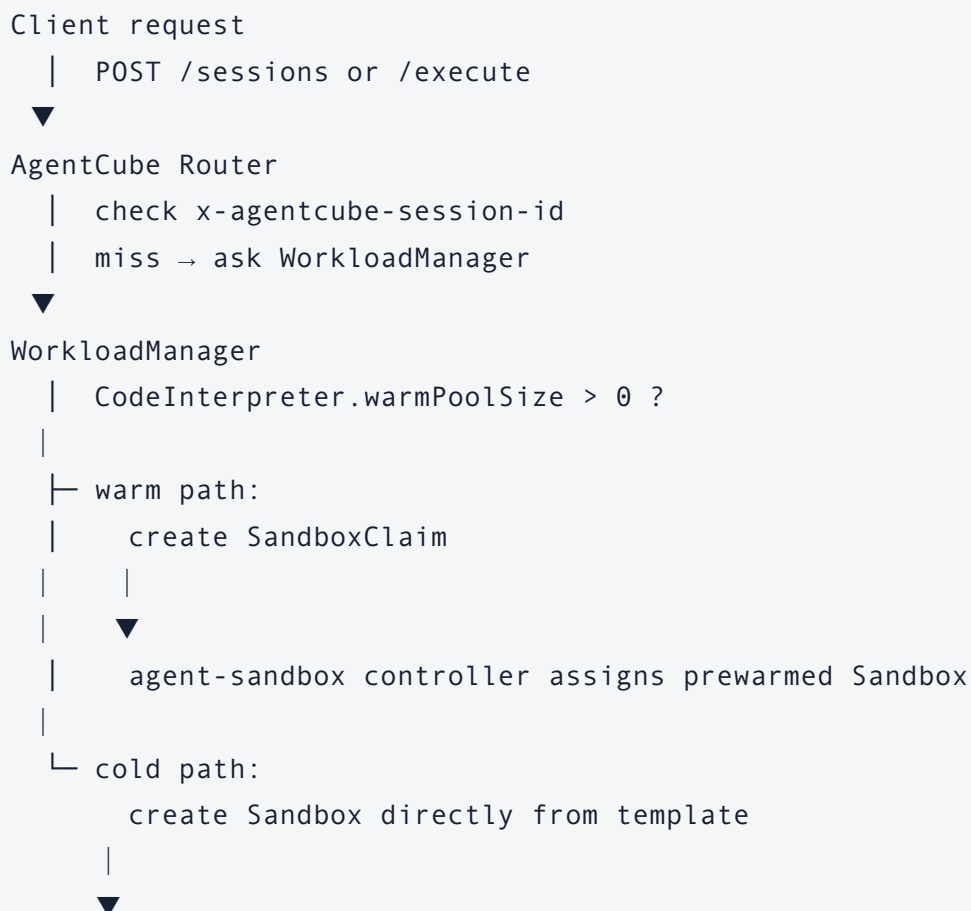
agent-sandbox API 兼容性

AgentCube 当前已发布 v0.1.0，但仍依赖 sigs.k8s.io/agent-sandbox v0.1.1。这意味着它和当前 agent-sandbox v0.4.6 / v0.5.0rc1 之间可能存在 API 追赶成本：

- SandboxTemplate / SandboxWarmPool 字段可能需要适配新版本 rolling update / backend 解耦；
- claim / warm pool 的 status 和 owner reference 语义可能变化；
- router/session registry 依赖 Sandbox Ready、endpoint、port 等状态字段，升级时要重点回归；
- AgentCube 的 CodeInterpreter.warmPoolSize 是对 agent-sandbox WarmPool 的上层封装，上游 API 变动会直接影响这一功能。

选型上，AgentCube 当前适合作为“agent-sandbox 上层应用 API”的设计参考；生产采用前应先验证它是否已跟上目标 agent-sandbox 版本。

CodeInterpreter warm pool 请求时序




```
wait Sandbox Ready + endpoint TCP probe
|
▼
write sessionId → endpoint into Redis / ValKey
|
▼
Router reverse proxy → PicoD `/api/execute` / `/api/files`
```

这条路径把高频请求和低频 K8s reconcile 分开：首次 miss 需要控制面介入，后续同一 session 直接由 Router 查 registry 并代理到 PicoD。

AgentCube 架构与设计思路分析

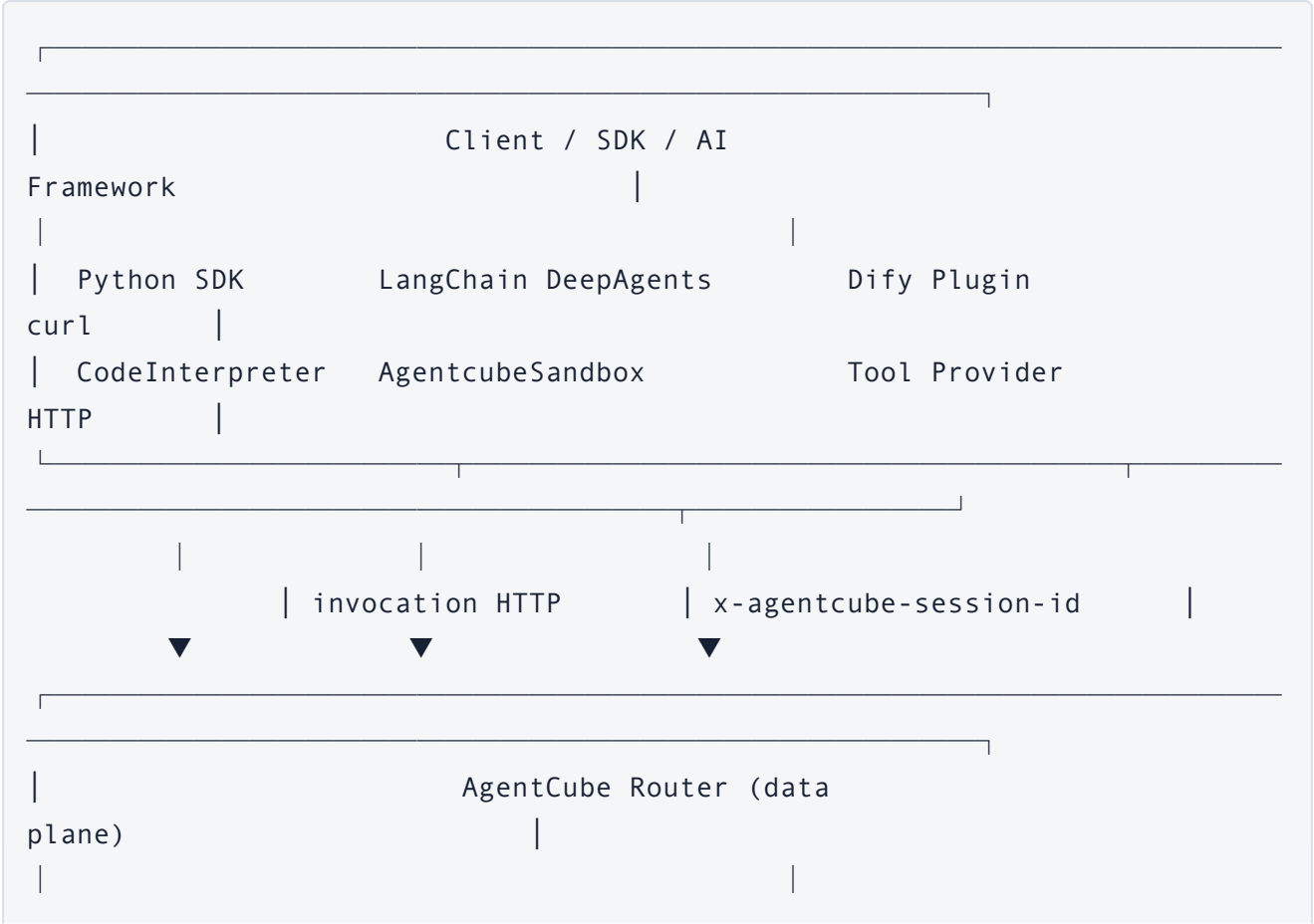
原文: raw/agentcube-architecture-analysis.md · 仓库: <https://github.com/volcano-sh/agentcube> · 分析版本 HEAD 208da32 (2026-06-01)

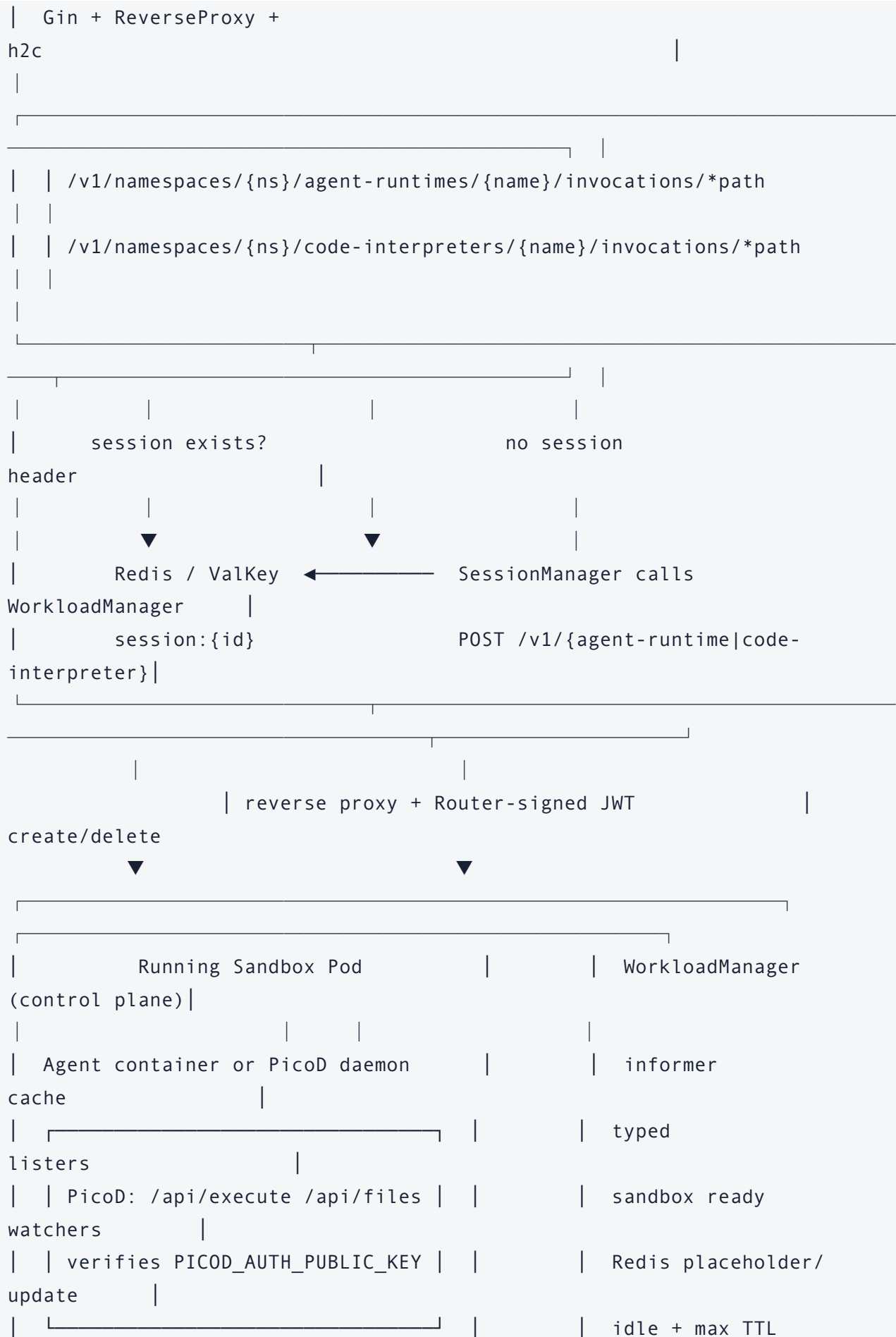
一句话定位

volcano-sh/agentcube 是 Volcano 社区面向 AI Agent / Code Interpreter 工作负载的 **serverless session orchestration layer**: 上层用 AgentRuntime / CodeInterpreter 两个 CRD 表达"可被调用的 agent/runtime 模板", 中层用 Router + WorkloadManager + Redis/ValKey 管 session、路由、GC 和预热, 下层直接复用 agent-sandbox 的 Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool 来真正创建隔离执行环境。

它和 agent-sandbox 不是竞争关系, 而是更明确的上下层关系: agent-sandbox 提供"一个稳定、有状态、可预热/可领取的 Sandbox 原语", AgentCube 把这个原语包装成"开发者能通过 HTTP/SDK 调用的 AgentRuntime / CodeInterpreter 会话服务"。

核心架构图







模块分层

层 / 模块	职责
AgentCube API 层	定义 AgentRuntime / CodeInterpreter 两个面向用户的 CRD。前者保留完整 PodSpec，后者收敛成 image/resources/runtimeClass/authMode/warmPoolSize。
数据面 Router	暴露 invocation API，按 x-agentcube-session-id 做 session affinity；没 session 时调用 WorkloadManager 创建；有 session 时查 Redis/ValKey；最后反向代理到 sandbox endpoint。

层 / 模块	职责
控制面 WorkloadManager	读 AgentCube CRD，构造 agent-sandbox CR，等待 Sandbox Ready，探测 entrypoint，写 session store，并周期性 GC。
Sandbox 内 runtime	PicoD 用轻量 HTTP API 替换 SSH：执行命令、上传/下载/列文件、workspace path jail、JWT 验证、32MB body limit。
状态存储	Redis / ValKey 抽象；session:{id} 保存 SandboxInfo，两个 sorted set 分别索引 ExpiresAt 和 LastActivityAt。
安全与身份	Router→PicoD 用 RS256 JWT；Router→WorkloadManager 可用 SA token 或 SPIFFE/mTLS；mTLS 文件来源可来自 SPIRE / cert-manager / 静态 Secret。
SDK / 集成	Python SDK、LangChain Deep Agents backend、Dify plugin、kubectl 风格 CLI，把底层 session API 包成开发者接口。
部署与 E2E	Helm 安装 Router/WorkloadManager/CRDs/SPIRE；E2E 覆盖 CodeInterpreter、WarmPool、agent-sandbox 安装和 session 生命周期。

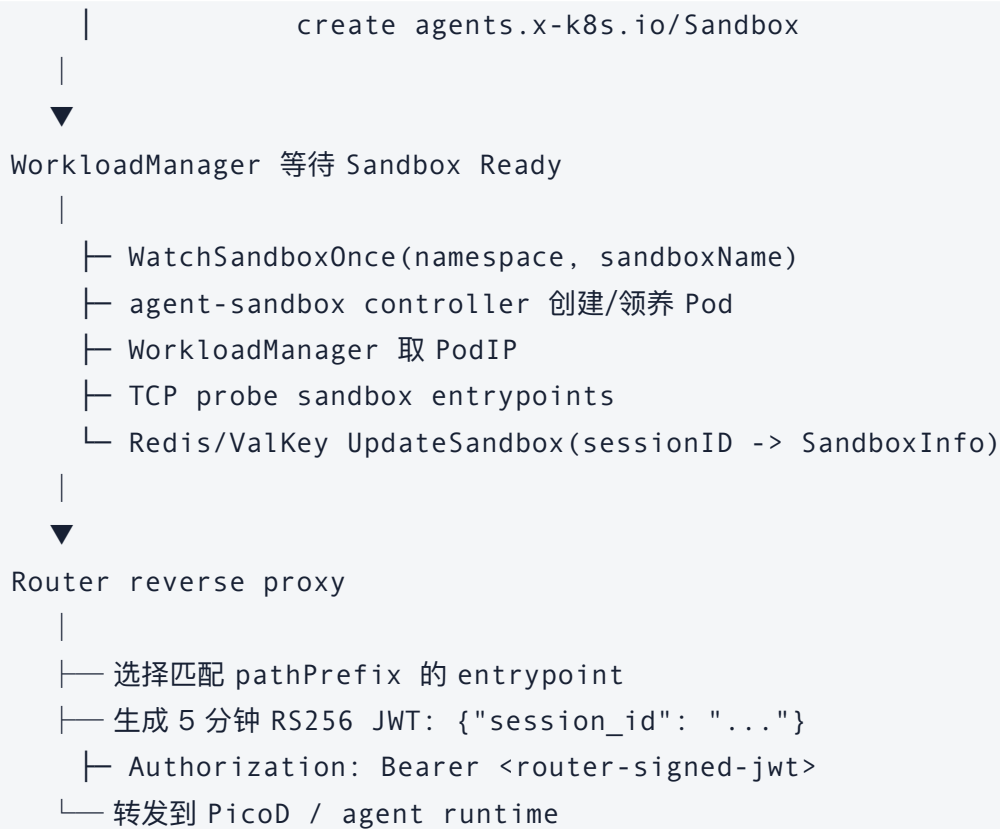
关键数据流

无 session 的首次调用

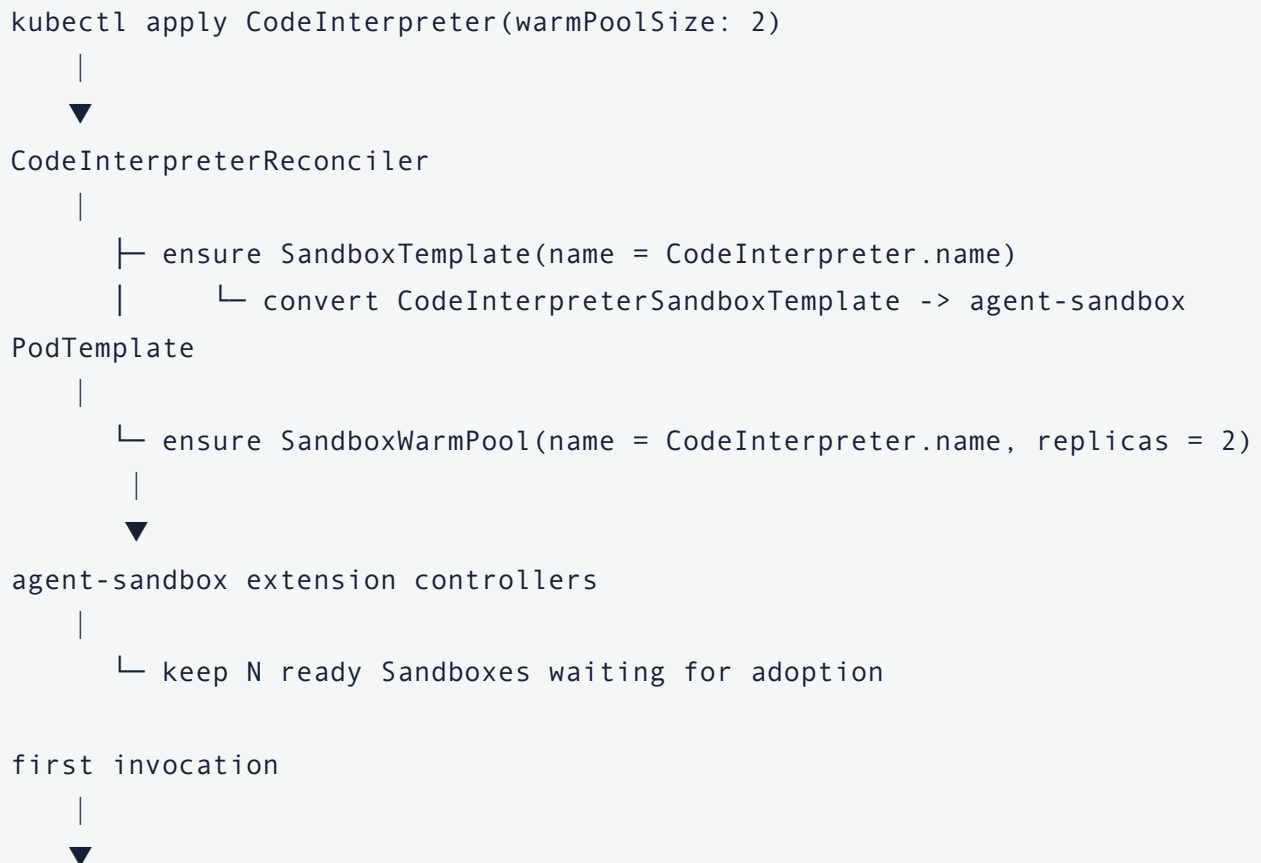
```

Client / SDK
|
|   | POST /v1/namespaces/default/code-interpreters/my-interpreter/
invocations/api/execute
|   | (no x-agentcube-session-id)
▼
AgentCube Router
|
|   | SessionManager.GetSandboxBySession(sessionID="")
|   |
|   |   ▼
|   |   | WorkloadManager POST /v1/code-interpreter
|   |   |
|   |   |   | informer lister 读取 CodeInterpreter CR
|   |   |   |   | 生成 sessionID + sandboxName
|   |   |   |   | 如果 warmPoolSize > 0:
|   |   |   |   |   | create extensions.agents.x-k8s.io/SandboxClaim
|   |   |   |   |   | 否则:

```



CodeInterpreter WarmPool 路径



```

WorkloadManager buildSandboxByCodeInterpreter
|
├─ create SandboxClaim(templateRef = CodeInterpreter.name)
├─ entry.Kind = SandboxClaim
└─ store session placeholder
|
▼
agent-sandbox SandboxClaimReconciler
|
├─ pick ready Sandbox from WarmPool
├─ transfer ownerReference to Claim
└─ annotate SandboxPodName
|
▼
WorkloadManager sees Sandbox Ready -> PodIP -> entrypoints -> session store

```

设计决策与哲学

- 把 agent-sandbox 当 substrate，不重新实现 Sandbox controller: go.mod 直接依赖 sigs.k8s.io/agent-sandbox v0.1.1, WorkloadManager 构造 agents.x-k8s.io/Sandbox 和 extensions.agents.x-k8s.io/SandboxClaim。AgentCube 自己碰 Pod reconcile 细节，专注 session API、路由、GC、SDK 和集成。
- 两个 CRD 对应两类 AI workload: AgentRuntime 允许完整 PodSpec，适合长会话 agent、volume、credential、sidecar; CodeInterpreter 收敛成 image/resources/runtimeClass/authMode/warmPoolSize，适合执行不可信代码。也因此只有 CodeInterpreter 一等支持 warmPoolSize，因为 code interpreter 更需要低冷启动延迟。
- Router / WorkloadManager 分平面: Router 是高频数据面，负责 session header、反向代理、路径匹配和并发限制; WorkloadManager 是低频控制面，负责 [kubernetes](#) API、informer、Sandbox 创建和 GC。这样 Router 可以水平扩展，状态放进 Redis/ValKey，而不是把请求路径绑死在 controller 进程内。
- 创建先写 placeholder，再等 Ready，再 update store: WorkloadManager 在真正创建 K8s 资源前先 StoreSandbox placeholder，失败时 rollback K8s 资源和 store。成功路径必须等 Ready、拿到 PodIP、并 TCP probe endpoint 后才把 store 更新为可路由信息。
- PicoD 用 HTTP + JWT 替代 SSH: PicoD 是 sandbox 内最薄的执行面，POST /api/execute 执行命令，/api/files 做文件操作。当前代码已经从早期"客户端私钥签名"演进到"Router 持有 RSA 私钥、PicoD 校验 Router 公钥"的模型: Router 创建 picod-router-identity Secret, WorkloadManager 从 Secret 缓存 public key，再注入 PICOD_AUTH_PUBLIC_KEY。这降低 SDK 复杂度，但也让 Router 成为更强的信任边界。

- mTLS 是可选增强，不阻断低延迟路径：pkg/mtls 支持文件热加载证书和 SPIFFE ID 校验，Helm 可以通过 SPIRE sidecar 给 Router / WorkloadManager 注入 SVID。Router→WorkloadManager 可 mTLS，Router→PicoD 当前仍主要走 JWT，以避免每个短会话 sandbox 的 TLS 握手成本。

与 agent-sandbox 的结合方式

维度	agent-sandbox	AgentCube
抽象层	基础设施原语：Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool	产品化会话层：AgentRuntime / CodeInterpreter + Router + SDK
用户入口	kubectl apply Sandbox / SandboxClaim 或 SDK 直接管 Sandbox	HTTP invocation API、Python SDK、LangChain/Dify/CLI
生命周期语义	1 个 Sandbox = 1 个有状态隔离 Pod，可暂停/恢复/过期	1 个 session = 1 个 Sandbox，附加 session header、activity、TTL、GC
Warm Pool	提供 WarmPool 和 Claim adoption 机制	CodeInterpreter.spec.warmPoolSize 驱动创建 Template/WarmPool，首次调用时创建 Claim
路由	headless Service / Pod 访问由上层决定	Router 根据 SandboxInfo.EntryPoints 反向代理到 PodIP:port
状态存储	K8s API / Status	Redis/ValKey session registry，跨 Router 副本共享
隔离策略	委托 K8s runtimeClassName / NetworkPolicy / securityContext	在 CRD 模板中暴露 runtimeClassName / resources，默认 image 可跑 PicoD
安全边界	运行时隔离与 K8s 原语	Router JWT、可选 WorkloadManager auth、可选 SPIFFE/mTLS

结合后的层次可以概括成：



▼
agent-sandbox CRDs own lifecycle mechanics

|

▼
Kubernetes Pod / RuntimeClass / NetworkPolicy / Service

最重要的设计收益是：AgentCube 没有把"AI Agent 平台"做成另一个私有调度器，而是把 AI Agent 的会话和工具调用语义加在 agent-sandbox 上。底层依然能继承 agent-sandbox 的 gVisor/Kata/NetworkPolicy/WarmPool 能力，上层则补齐了 HTTP/API/SDK/Redis session registry 这些 agent 应用真正需要的东西。

关键组件深入解读

WorkloadManager：从 AgentCube CRD 到 agent-sandbox CRD

核心转换在 pkg/workloadmanager/workload_builder.go。buildSandboxByAgentRuntime 读取 AgentRuntime，复制 spec.podTemplate.spec，归一化空 runtimeClassName，生成 sessionID 和随机后缀 sandboxName，然后构造 agent-sandbox 的 Sandbox。这条路径完全保留用户的 PodSpec，适合通用 agent runtime。

buildSandboxByCodeInterpreter 更专门：先检查 authMode=picod 时 public key 是否已缓存，然后为没有 ports 的 CodeInterpreter 默认补 / + 8080/HTTP。如果 warmPoolSize > 0，它不直接创建完整 Sandbox，而是创建 SandboxClaim(templateRef = codeInterpreterName)；如果没有 WarmPool，则构造带 PicoD image/env/resources 的直接 Sandbox。

风险与演进信号

- 项目仍在 alpha / proposal 到 release 的过渡期：README 还写 Early Design，release 文档写 v0.1.0。字段命名也有历史痕迹，比如 AgentRuntime.spec.targetPort vs CodeInterpreter.spec.ports，podTemplate vs template。
- Router 是强信任边界：当前 Router 持有 PicoD 私钥并能为任何 session 签执行请求。相比"客户端私钥不出本机"，这更容易用，但要靠 Router auth、mTLS、RBAC、审计来补强。
- WorkloadManager 目前承担两种职责：既是 HTTP control plane，又跑 controller-runtime manager 和 GC。规模变大后可能会拆成 API server / reconciler / GC 三个部署单元。
- agent-sandbox 版本偏早：依赖 v0.1.1，而本 wiki 之前分析的是 v0.4.5+11。AgentCube 当前锁定的 API 可能需要跟随 agent-sandbox 后续 v1alpha1 演进。

相关页面

- agent-sandbox
- [kubernetes](#)
- declarative-agent-management

- agent-credential-isolation

agentgateway

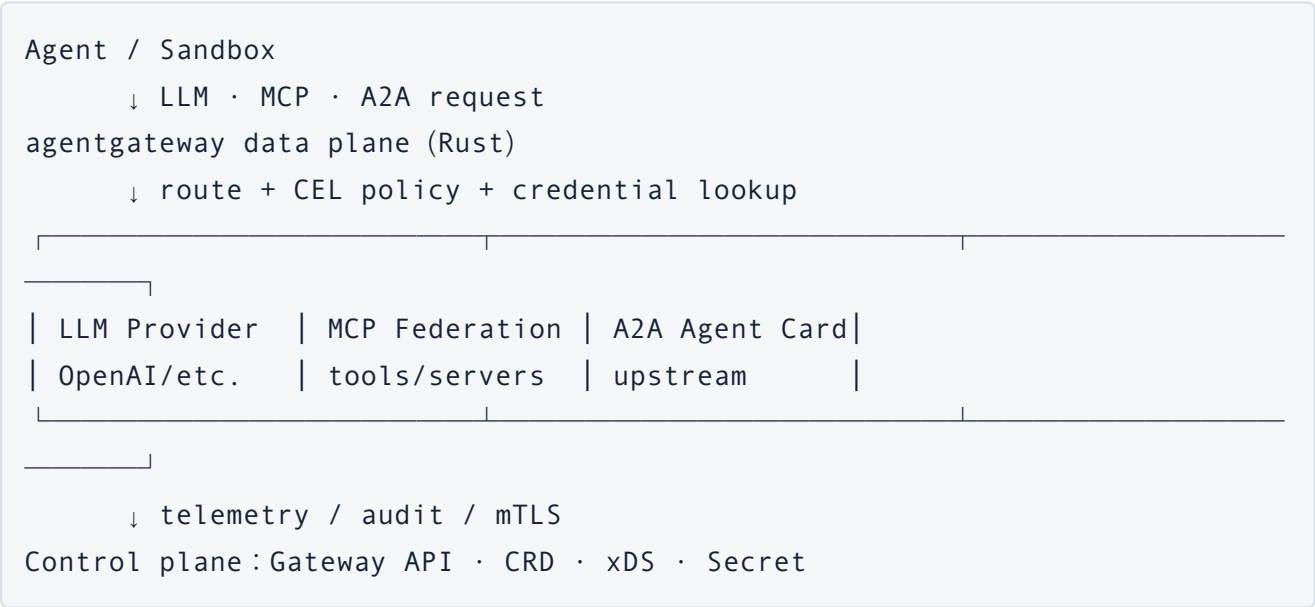
Solo.io / Istio 系出品的 AI-native L7 网关（Apache 2.0）。把三种 AI 流量统一在一个 Rust 数据面里：

- **LLM Gateway**: OpenAI / Anthropic / Gemini / Bedrock / Vertex / Azure / Copilot 七家 provider 统一为 OpenAI 兼容 API + guardrails
- **MCP Gateway**: stdio / HTTP Streamable / SSE 三种 transport + OpenAPI 自动转 MCP 工具 + 多 MCP server federation
- **A2A Gateway**: Google Agent-to-Agent 协议代理，通过重写 Agent Card 让所有调用走向 gateway

架构骨架

- 控制面 (Go) : [Gateway API](#) + 自家 CRD (AgentgatewayPolicy / AgentgatewayBackend / AgentgatewayParameters) , KRT 反应式表, xDS Delta ADS 分发
- 数据面 (Rust) : tokio + hyper + rustls, proxy/gateway → http/route → llm|mcp|a2a → upstream, CEL 做策略 IR, HBONE 跟 Istio mesh 互操作
- UI (TypeScript) : Next.js 管理界面

Agent 出口请求流程



核心设计哲学

1. 沿用 Istio 基建做 AI Gateway: Gateway API + KRT + xDS + HBONE 全套复用，只把数据面换成专为 AI 协议优化的 Rust 代理
2. 三协议共享同一条 pipeline: Route / Policy / Backend / CEL 求值统一，差别只在 Backend.kind 决定走哪个 provider 适配器

- 3. 凭据托管：LLM provider 真凭据放在 backend secret，Agent 只面对 gateway 自身认证，参见 agent-credential-isolation
- 4. 跟 agent-sandbox 互补：前者出口流量治理，后者运行时隔离，并集 = AI 工作负载完整治理面

详见 [src-agentgateway-architecture](#)。

LLM provider 适配器对照

Provider	请求侧职责	响应侧职责	典型差异
OpenAI / OpenAI-compatible	作为内部标准格式，基本透传	返回 OpenAI-compatible SSE / JSON	其他 provider 多向它收敛
Anthropic	OpenAI messages → Anthropic messages / tools	Anthropic event stream → OpenAI chunk	thinking/tool schema/ stream event 名称不同
Gemini / Vertex	OpenAI chat → generateContent / Vertex endpoint	Gemini candidates → OpenAI choices	project/location/auth 和多模态字段不同
Bedrock	OpenAI chat → Bedrock invoke，叠加 AWS SigV4	Bedrock body/stream → OpenAI-compatible	AWS auth、model id、区域化 endpoint
Azure OpenAI	OpenAI schema + Azure deployment endpoint	Azure OpenAI response → OpenAI-compatible	deployment 名称、api-version、endpoint path
Copilot	适配 GitHub Copilot provider 入口	统一成 OpenAI-compatible	认证和可用模型集合不同

agentgateway 的关键不是“支持更多 provider”本身，而是把 provider 差异压到 AIProvider / Provider trait 后面。上层 HTTPRoute、Policy、Backend、CEL、telemetry 不需要知道具体 provider 细节。

MCP federation 与 CEL RBAC

MCP Gateway 把多个 MCP server 聚合到一个入口。设计重点是命名空间和授权：

- transport 层支持 stdio / HTTP Streamable / SSE，上游协议差异由 mcp/upstream 吸收；
- federation 时要避免工具名冲突，通常需要按 server 或 route 分命名空间；
- 权限不应散落在工具实现里，而应在 gateway route / policy 层用 cel 表达式做统一判定；
- agent 只拿 gateway 侧身份，后端 MCP server 的真实凭据仍由 gateway 管。

这和 HiClaw 的 worker consumer key 模式一致：Agent 能调用哪些工具，由网关侧 consumer、route 和 policy 决定，而不是由 prompt 里的自觉性决定。

和同类 AI Gateway 的边界

项目	主要定位	agentgateway 的差异
LiteLLM	Python LLM proxy / provider adapter	agentgateway 更偏 Rust 数据面 + Gateway API/xDS/mesh 集成
OpenRouter	托管模型路由服务	agentgateway 是可自托管 infra 组件，控制面和策略在用户集群里
Portkey	托管/企业 LLM gateway	agentgateway 更云原生，强调 CRD、mesh、MCP/A2A
Envoy AI Gateway	Envoy Gateway 上的 GenAI gateway	agentgateway 不复用 Envoy 数据面，而是自写 Rust 数据面并同时覆盖 LLM/MCP/A2A

部署 walkthrough 速记

1. 安装 Gateway API CRD 与 agentgateway 自家 CRD。
2. 部署 controller，它 watch Gateway / HTTPRoute / AgentgatewayPolicy / AgentgatewayBackend。
3. 创建 GatewayClass 与 Gateway，由 AgentgatewayParameters 控制 dataplane 部署参数。
4. 创建 AgentgatewayBackend，把 AI/MCP/A2A 后端和 secret 挂进去。
5. 用 HTTPRoute.backendRef 指向 backend，并用 AgentgatewayPolicy 加鉴权、限流、转换或 CEL RBAC。
6. dataplane 通过 xDS 增量接收配置，开始处理 LLM / MCP / A2A 流量。

如果需要运行时隔离，把 Agent 容器放在 agent-sandbox / [agentcube](#) 一类 substrate 上；如果需要 LLM serving endpoint picking，把入口和 [gateway-api-inference-extension](#) / [llm-d](#) 对齐。

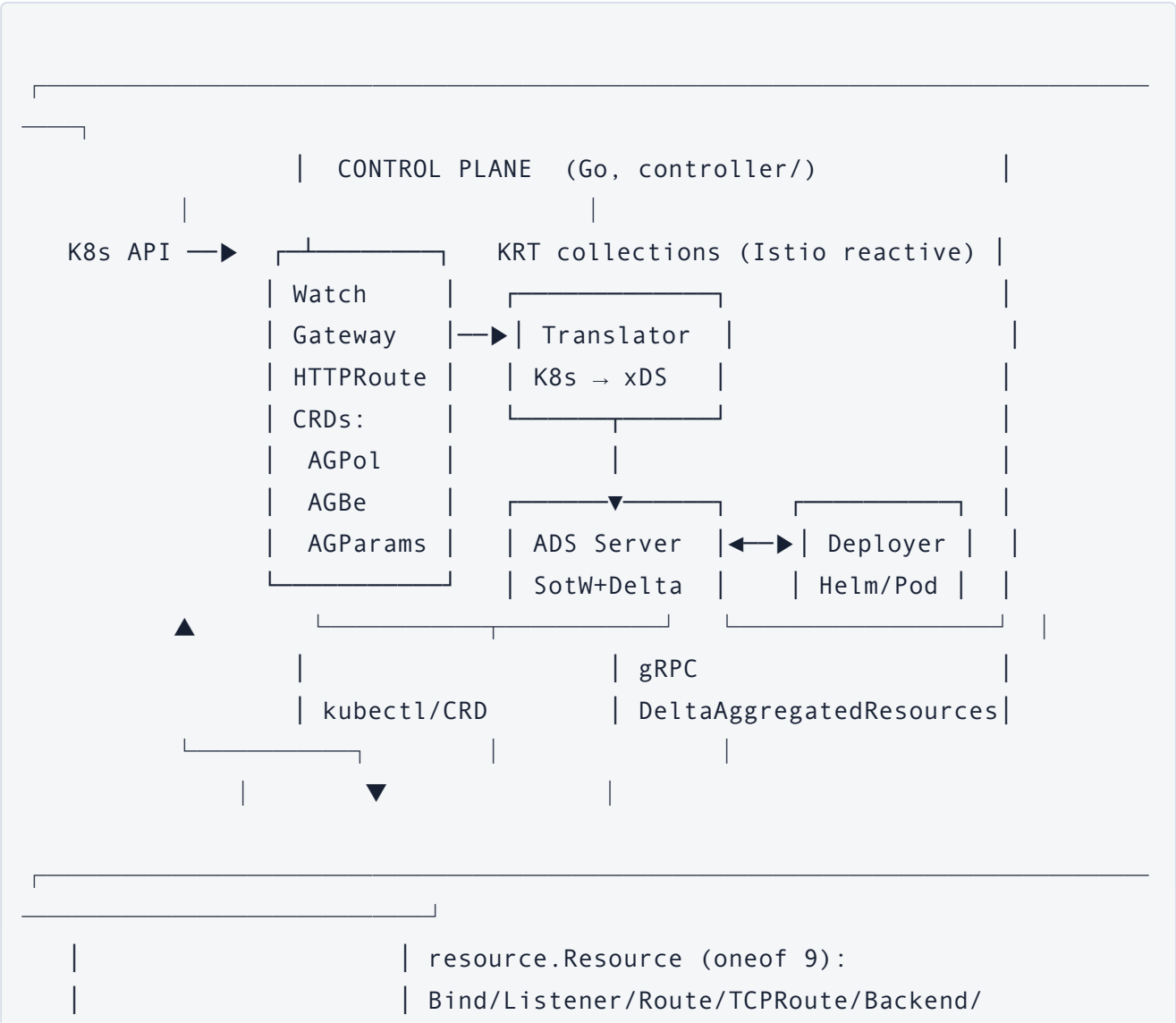
agentgateway 架构与设计思路分析

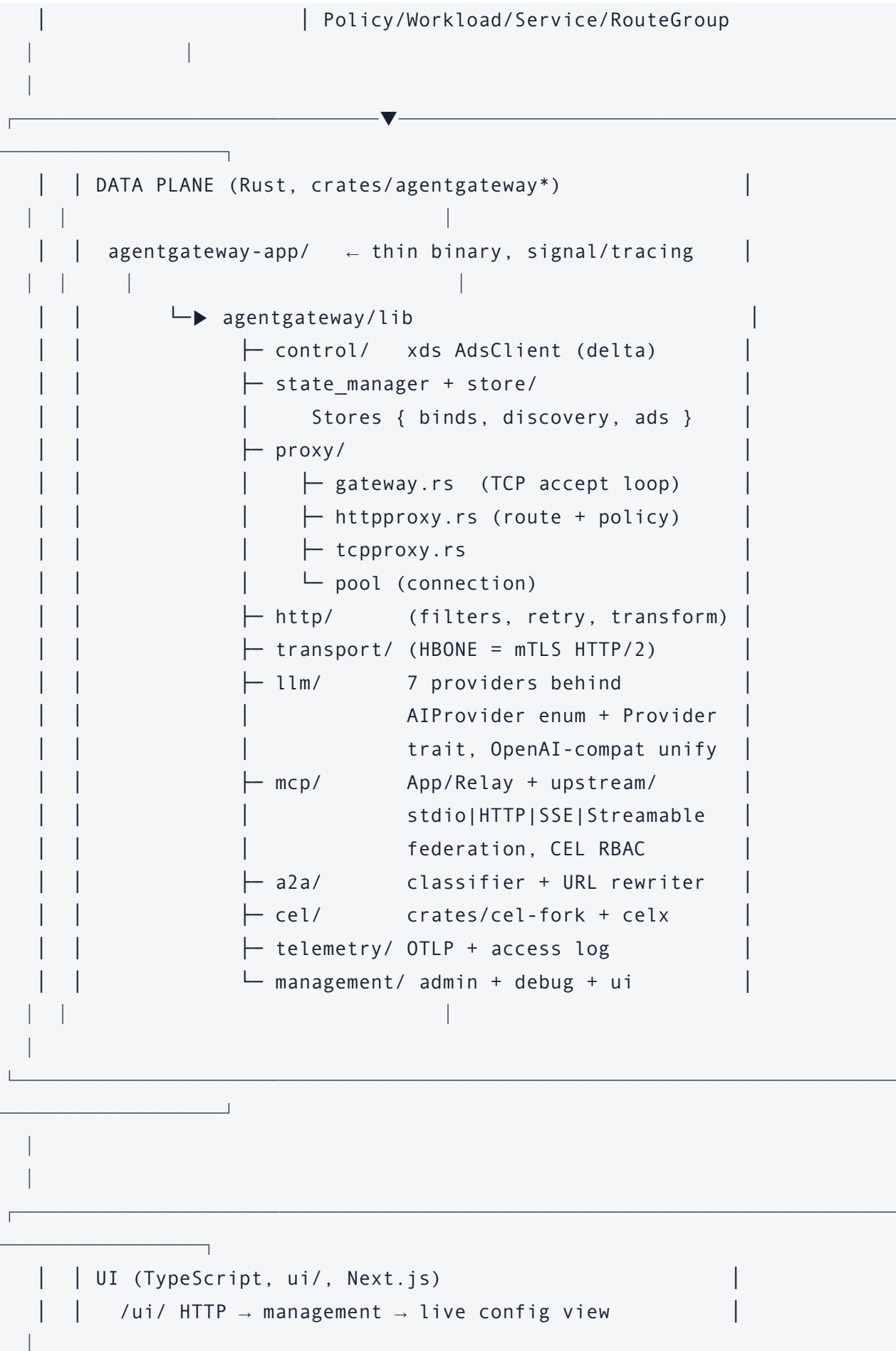
原文: raw/agentgateway-architecture-analysis.md · 仓库: [agentgateway/agentgateway](#) · 分析版本 v1.2.0-alpha.2+24 (HEAD 9ca3e04)

一句话定位

[agentgateway](#) 是面向 AI 流量的 L7 代理：把 LLM API（OpenAI / Anthropic / Gemini / Bedrock / Vertex / Azure / Copilot）、MCP 工具调用、A2A Agent 间通信 统一在一个 Rust 数据面里，用 [Gateway API](#) CRD + xDS 做声明式控制面分发。本质上是「Istio 的控制面骨架 + 为 AI 协议特化的 Rust 数据面」的组合——把 mesh 经过验证的反应式配置分发、KRT 反应式表、HBONE mTLS 隧道全套基建搬来做 AI Gateway。

核心架构图





模块分层

Workspace 顶层

模块	语言	角色
crates/agentgateway-app/	Rust	薄壳二进制（信号 / 日志 / panic hook）
crates/agentgateway/	Rust	数据面主库，三协议网关都在此
crates/xds/	Rust	xDS 客户端 + proto 绑定
crates/hbone/	Rust	Istio HBONE（mTLS over HTTP/2 CONNECT）传输
crates/celx/ + crates/cel-fork/	Rust	CEL 策略求值（fork 加 HTTP 集成）
controller/	Go	controller-runtime + ADS server + Helm deployer
api/	Go	resource.proto 的 Go 绑定
ui/	TypeScript	Next.js 管理 UI

数据面分层（crates/agentgateway/src/）

层	模块	职责
L1 配置	control/ config.rs	本地 yaml + xDS client + outbound client
L2 状态	state_manager.rs store/	反应式 Store 聚合 binds/discovery/ads
L3 代理	proxy/gateway.rs proxy/httpproxy.rs	TCP accept → TLS → route → upstream
L4 HTTP	http/	filter chain / retry / timeout / transform
L5 协议	llm/ mcp/ a2a/	三个 AI 协议网关
L6 横切	cel/ telemetry/ transport/	策略求值 / 可观测 / HBONE 隧道
L7 管理	management/ ui.rs	管理 HTTP / 健康 / UI 静态资源

自家 CRD

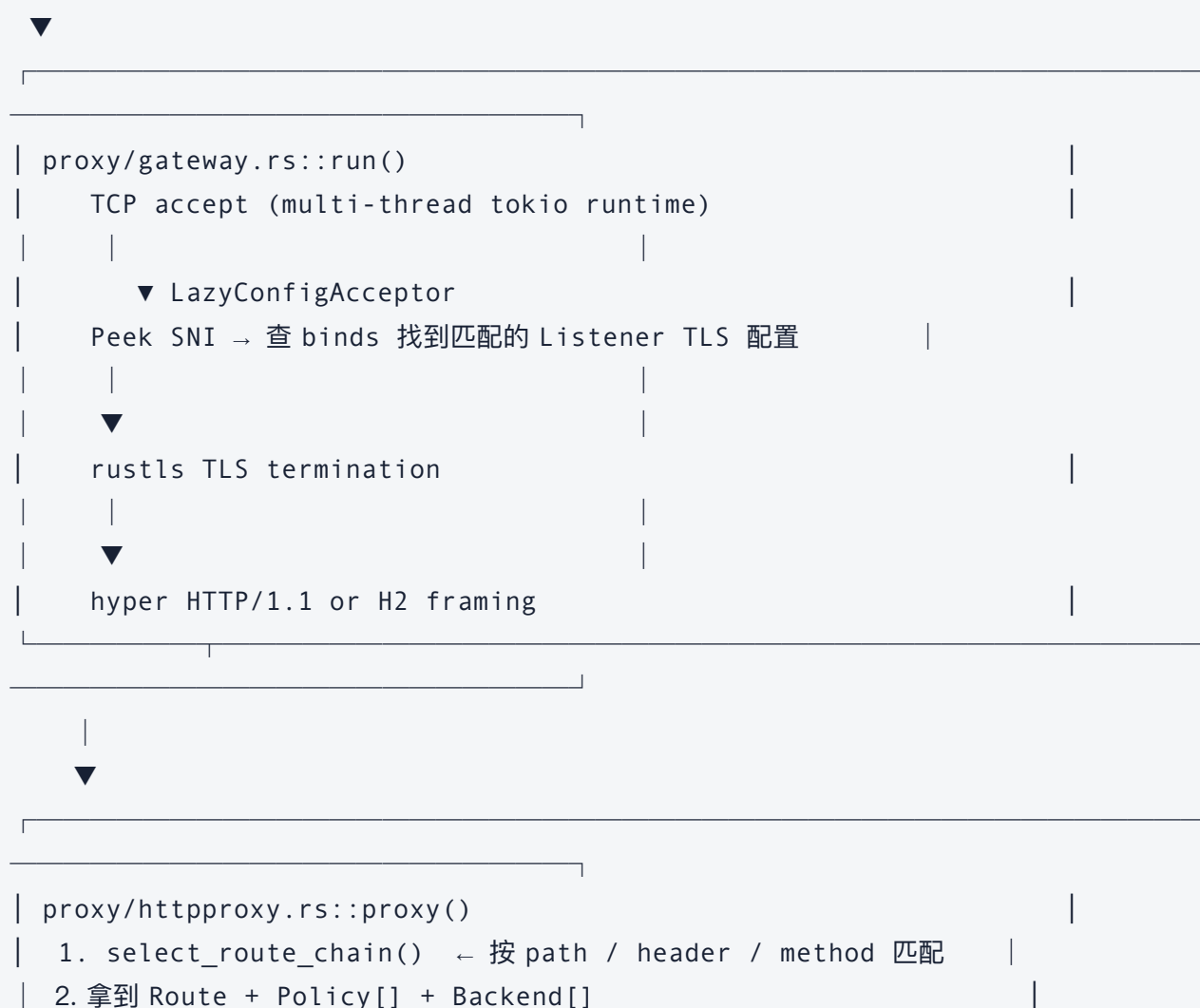
CRD	短名	用途
AgentgatewayPolicy	agpol	CEL 策略容器，挂在 Gateway / Listener / Route 上
AgentgatewayBackend	agbe	后端配置：AI / MCP / A2A / Static
AgentgatewayParameters	—	GatewayClass 级 Pod 模板参数

上游 [Gateway API](#) 直接复用：Gateway HTTPRoute GRPCRoute TCPRoute TLSRoute GatewayClass ListenerSet InferencePool。

关键数据流

一次 LLM 请求的端到端数据流

```
Client (curl / OpenAI SDK)
  | POST /v1/chat/completions (model: "gpt-4")
```





▼ 响应 (流式 SSE / chunked)

| provider.transform_response() |

| — 反向 schema 转换 → 统一 OpenAI 格式 |

| — guardrails (post-flight) |

| — telemetry: tokens_in / tokens_out / cost / latency |

| — access log + OTLP span |

▼

Client receives OpenAI-compatible SSE stream

控制面 → 数据面 配置分发

kubectl apply Gateway/HTTPRoute/agpol/agbe.yaml

▼

controller-runtime watch → KRT collection mutate

▼

Translator: K8s objs → resource.Resource oneof

├ Bind (TCP listen addr)

├ Listener (TLS / SNI / protocol)

├ Route (HTTP route + filters)

├ TCPRoute

├ Backend (AI / MCP / A2A / Service / Static)

├ Policy (CEL IR + transform + retry + ratelimit)

├ RouteGroup

└ Workload/Service (Istio Address 兼容)

▼

ADS Server (gRPC, controller/pkg/xds/server.go)

DeltaAggregatedDiscoveryService

▼ Delta xDS (增量)

agentgateway/control/AdsClient

```

    ▼ apply
state_manager + Stores { binds, discovery }
|
    ▼ 触发
proxy/gateway 重建 Listener / Route 索引

```

xDS 资源类型只有两种：

- `istio.workload.Address` —— 兼容 Istio ambient mesh 的 workload/service 发现
- `agentgateway.dev.resource.Resource` —— 主资源 oneof (9 个变体覆盖整个数据面状态)

只用 Delta ADS，单连接复用，不每种资源一个 stream。

一次 MCP 工具调用

```

MCP Client (Cursor / Claude Desktop / OpenAI Responses API)
| JSON-RPC over stdio | HTTP Streamable | SSE
▼
mcp/router.rs — 路由层根据 session id 找到对应的上游
|
▼
mcp/handler.rs — 工具列表 federation (merge_tools)
|   |
|   └─ 多个 upstream MCP server 的 tools/list 合并
|       + CEL RBAC 过滤 (mcp/rbac.rs)
|
▼ tools/call
mcp/upstream/{stdio.rs | streamablehttp.rs | sse.rs | openapi.rs}
|
▼
Upstream MCP server / OpenAPI endpoint

```

下游 transport: HTTP Streamable / SSE / stdio。上游 transport: 同三种 + `openapi.rs` 把任何 OpenAPI spec 自动桥接成 MCP tools (无需上游侧改造)。federation 通过 `handler.rs::merge_tools()` 合并多个 MCP server 的工具集，命名空间用 `_` 前缀避冲突，CEL RBAC 按 client identity 过滤可见性。

A2A (Agent-to-Agent)

```

Agent Client
| HTTP GET /.well-known/agent-card.json
▼
a2a/mod.rs — 拦截 Agent Card 发现请求

```

- | 一 透传上游，但 *重写响应里的 URL*
- | 把 upstream 真实地址换成 gateway 自身地址
- | 一 之后所有 Agent 调用都走向 gateway，自然拿到 policy/observability

A2A 是三协议里最轻的——核心就 URL 重写。Google A2A 协议把入口放在 Agent Card well-known endpoint，gateway 控制了发现，就控制了所有后续调用。

设计决策与哲学

- 沿用 Istio 的基建做 AI Gateway: Gateway API + KRT collections + xDS + HBONE 全套复用，控制面是 Istio 的影子结构。开发团队 (Solo.io / Istio 系) 选择不重新发明轮子，把 mesh 经过验证的反应式配置分发 + 反应式表建模 + mTLS 隧道搬来，只把数据面替换成「专为 AI 协议优化的 Rust 代理」。Day 1 即获得完整 mesh 兼容性 + 控制面/数据面可独立替换的解耦能力。
- 三协议共享同一条 pipeline: LLM / MCP / A2A 在数据面共享 Route / Policy / Backend / CEL 求值的同一条管道，差别只在 Backend.kind 决定走哪个 provider 适配器。带来单一策略语言 + 统一观测 + 统一 mTLS。
- CEL 作为策略 IR: 所有授权 / 转换 / 速率限制都被编译成 CEL 表达式存到 Policy.spec.expression，运行时由 Rust 数据面解释执行。controller 不预编译，避免 Go/Rust CEL 实现差异。用 cel-fork + celx 是因为原生 cel-rust 缺与 HTTP 请求对象的深度集成。
- 凭据托管 prompt injection: 与 HiClaw 的 Higress 凭据托管哲学一致——LLM provider 真凭据 (OpenAI key / AWS IAM / GCP service account) 放在 backend secret，gateway 在请求时注入，Agent / 业务端只面对 gateway 自己的认证 (JWT / API key / mTLS)。Agent 即使被注入也偷不到真凭据。详见 agent-credential-isolation。
- 控制面 / 数据面通过 proto 解耦: Go controller + Rust 数据面只用 resource.Resource proto 通信，可独立演进。未来能换掉任一端 (Crossplane 直产 xDS / envoy 实现 agentgateway proto 扩展)。
- 两个 tokio runtime 拆分热路径: main runtime 跑控制面 (admin HTTP / xDS / reload)，worker runtime 跑数据面 accept，避免控制面任务抢占 worker 影响 P99。
- OpenAI 兼容 API 做统一前门: 客户端代码零改造接 7 个 LLM provider; 非 OpenAI 特性 (Gemini 多模态、Bedrock 私有模型) 通过扩展字段暴露。

关键组件深入: LLM Provider 适配器

crates/agentgateway/src/llm/ 是数据面最厚的协议适配层。统一抽象骨架:

```
enum AIProvider {
    OpenAI, Anthropic, Gemini, Bedrock, Vertex, Azure, Copilot,
}

trait Provider {
    async fn transform_request(&self, req: OpenAIRequest) ->
    ProviderRequest;
```

```

    async fn transform_response(&self, resp: ProviderResponse) ->
OpenAIResponse;
    async fn stream_chunk(&self, chunk: Bytes) -> OpenAIChunk;
}

```

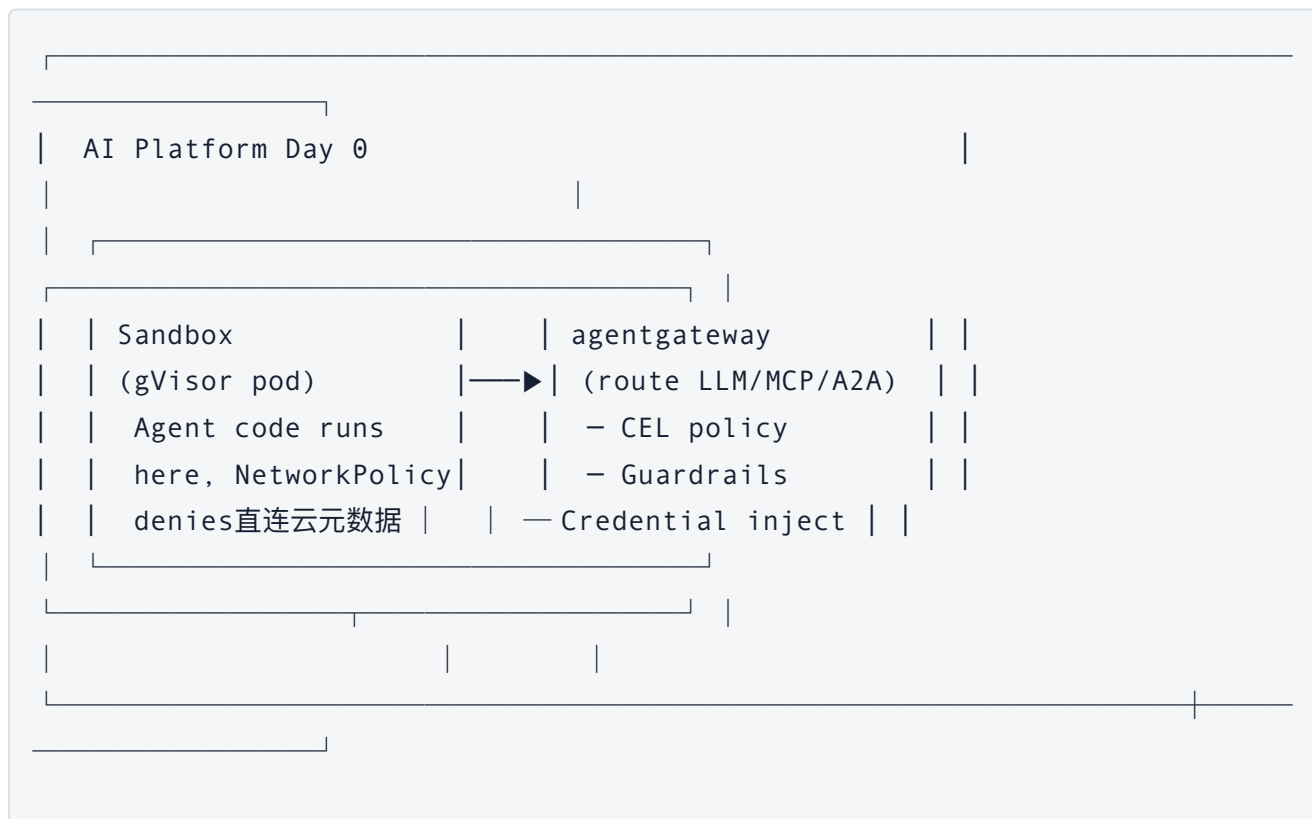
外部统一暴露 OpenAI 兼容 API (/v1/chat/completions / /v1/embeddings)，内部 dispatch 到 provider 双向翻译。

Guardrails —— LLM 流量特有的内容审查，pre-flight / post-flight 双向，支持 regex / OpenAI Moderation / AWS Bedrock Guardrails / Google Model Armor / Azure Content Safety / 自定义 webhook，每 backend 按声明顺序串成 chain。

负载均衡 —— 跨副本走 **two-random-choice** (tail-latency 比 round-robin 稳)；跨 model alias 走 weighted random。

与 HiClaw / agent-sandbox 的关系

- 跟 HiClaw 是不同层次互补：HiClaw 在应用层做 Agent 协作 (Matrix IM + Worker CRD)，agentgateway 在基础设施层做 AI 流量代理。HiClaw 的 Worker 跑用户代码访问 LLM / MCP 时，理论上可以让 agentgateway 当出口代理，由 agentgateway 接管凭据 / 审计 / guardrails。
- 跟 agent-sandbox 拼成 AI 平台底盘：agent-sandbox 做 Agent 运行时隔离 (Pod + gVisor / Kata + NetworkPolicy)，agentgateway 做 Agent 出口流量治理 (LLM / MCP / A2A)。两者并集 = AI 工作负载完整的"运行 + 通信"治理面。NetworkPolicy 限制 Sandbox Pod 出口只能走 agentgateway，强制所有 AI 流量过 policy + guardrails + 审计。



相关页面

- [agentgateway](#) —— 项目主页
- mcp —— MCP 协议（agentgateway MCP gateway 的协议基础）
- [gateway-api](#) —— K8s Gateway API（agentgateway 控制面 API 基础）
- xds —— xDS 协议（控制面 → 数据面分发）
- hbone —— Istio HBONE 隧道（agentgateway 跟 mesh 互操作）
- cel —— Common Expression Language（agentgateway 策略 IR）
- istio —— Istio（agentgateway 控制面骨架来源）
- HiClaw —— 应用层 Agent 协作平台（互补关系）
- agent-sandbox —— K8s SIG 的 AI Agent 运行时隔离（互补关系）
- ai-agent-plugin-patterns —— Agent 外挂设计原则
- agent-credential-isolation —— Agent 凭据零暴露模式

AI 运维 (AIOps)

用 AI/LLM 增强运维流程，特别是告警分诊、根因分析、自动修复。

关键发现

Runbook 比模型选择更重要（来自 STCLab 实践）：

- 有 runbook：3-4 次工具调用定位问题
- 无 runbook：20+ 步才有结论

工具

- HolmesGPT (CNCF Sandbox) — ReAct 推理模式自动诊断 K8s 告警
- Robusta — K8s 告警自动化平台
- KubeAI (CNCF) — K8s 上自托管 LLM

效果数据

每次调查成本 ~\$0.04 (~\$12/月)，排查时间从 15-20 分钟降至 <2 分钟。

详见

- [src-holmesgpt-k8s-alerts](#)

AI 驱动的漏洞发现变革

摘要

AI 模型同时加速了漏洞发现和低质量报告的泛滥，开源维护者面临严重的分诊瓶颈。

核心矛盾

- 非专家也能发现真实漏洞 → 报告量激增
- AI 也能生成大量看似合理但无效的报告 → 噪声淹没真实威胁

漏洞管道四阶段

1. AI 扫描（供过于求）→ 2. 分诊（不堪重负）→ 3. 修复开发（瓶颈中）→ 4. 消费升级（未来瓶颈）

实操建议

- 维护者：公开威胁模型、设最低报告标准、AI 辅助分诊
- Bug 发现者：必须提供完整 PoC + 修复 PR，禁止自动批量提交

AlBrix

AlBrix 是 vLLM 生态的可插拔 GenAI inference infrastructure，提供 LLM Gateway/Router、LoRA 管理、LLM app-tailored autoscaler、Unified AI Runtime、distributed inference、distributed KV cache、异构 GPU serving 和 GPU 硬件故障检测。

解决的问题

企业使用 vLLM fleet 时，真正的难点不只在 engine：还需要 adapter 管理、跨副本路由、KV 复用、动态扩缩、成本优化和 GPU 故障处理。AlBrix 把这些能力拆成可组合组件。

核心边界

AlBrix 更贴近 vLLM 生态和企业控制面，不是新的推理 engine，也不是 Kubernetes 通用模型 API。它的 KV offloading 可以作为独立组件使用，路由和 autoscaling 也可以按需组合。

适用场景

适合已有 vLLM 技术栈、希望逐步加入 LoRA、KV、Gateway、异构 GPU 和企业运维能力的团队。需要 Gateway API/InferencePool 标准化时比较 [llm-d](#)；需要完整 P/D/KV runtime 时比较 [dynamo](#)。

详见 [src-aibrix-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

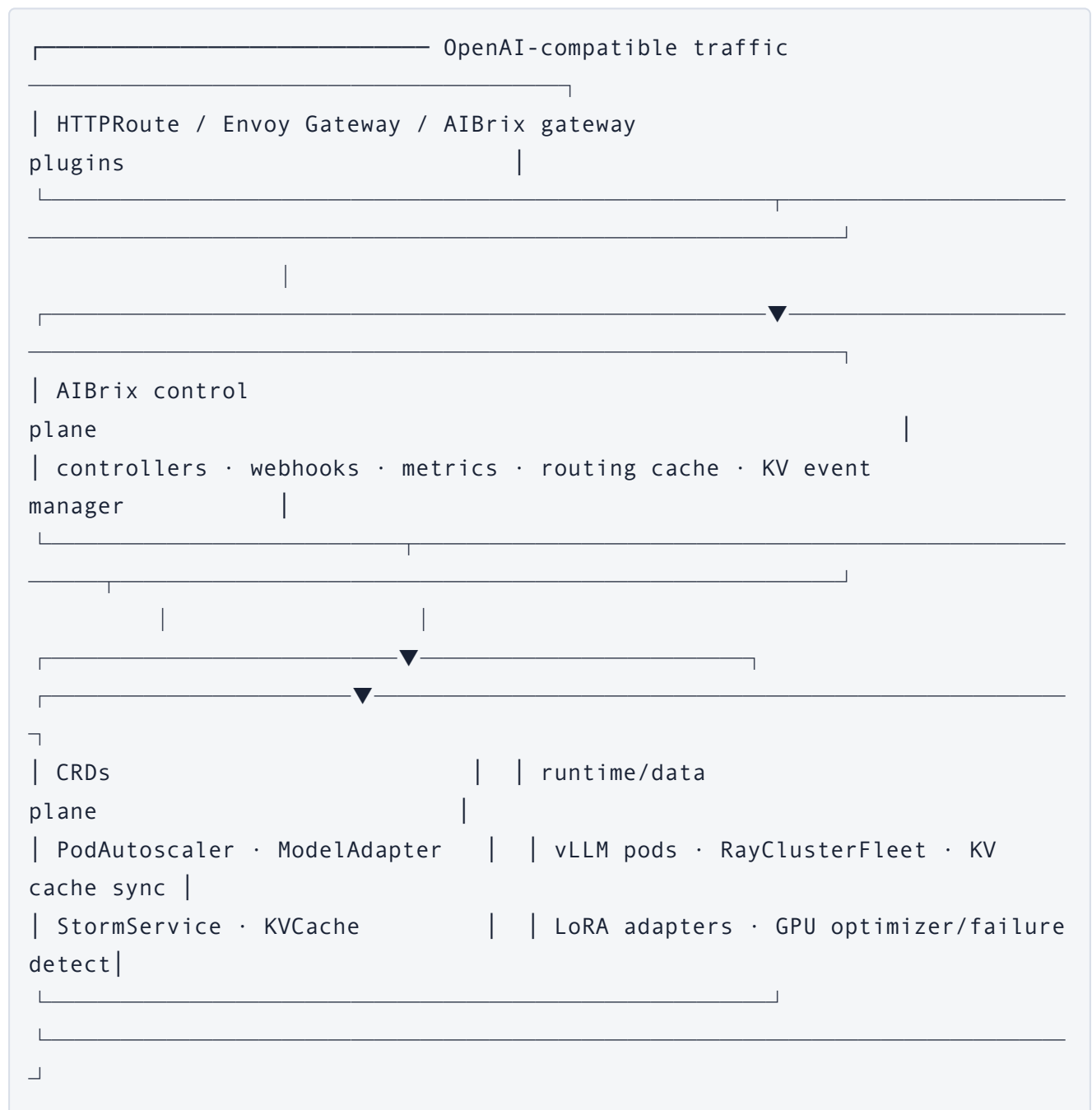
在 M4 模块地图中的位置

AlBrix 位于 Kubernetes 推理控制面层，覆盖 routing、autoscaling、模型与 adapter 生命周期、runtime，以及 KV 和多角色编排。职责边界见 llm-inference-serving-project-map，与 engine 及其他 serving 层的组合选择见 llm-serving-engine-selection-map。

AlBrix 架构与设计思路分析

vllm-project/aibrix 是 vLLM 生态的 Kubernetes GenAI inference infrastructure，而不是推理引擎。仓库包含 controller-manager、gateway plugins、KV cache watcher、CRD、webhook、metrics、cache、PodAutoscaler、ModelAdapter/RayCluster/Roleset 等模块，目标是把 vLLM 大规模部署、路由、扩缩容、LoRA、分布式推理和 KV cache 管理做成云原生控制面。

核心架构图



模块分层

层/目录	责任
cmd/controllers, pkg/controller/**	controller-runtime 控制面，PodAutoscaler 等 reconciler。
api/autoscaling, api/orchestration, api/model	CRD API: PodAutoscaler、StormService、RayClusterFleet、Roleset、KVCache、ModelAdapter。
pkg/plugins/gateway/**	gateway/routing 插件，包含 P/D disaggregation 路由逻辑。
pkg/cache, pkg/kvevent	KV event manager、cache snapshot、pod/model cache、ZMQ 事件。

关键数据流

1. 用户通过 Gateway 访问模型，gateway plugin 根据 model labels、pod metrics、KV/cache 状态选择后端。
2. controller 监听 AIBrix CRD 和 K8s workload，创建/调整 vLLM/RayCluster/RoleSet 等资源。
3. PodAutoscaler 支持 HPA/KPA/APA，不同 metric source 包括 pod/resource/custom/external/domain。

设计决策

- AIBrix 把 vLLM 从单 engine 运维提升到 K8s control plane。
- 它比 llm-d 更像一体化发行：gateway、autoscaler、LoRA、KV cache、GPU failure detection 都在一个 repo。
- CRD + controller-runtime 是主线，Gateway API/Envoy 是流量入口。

对比定位

和 [dynamo](#) 相比，AIBrix 更 Kubernetes/controller-first；Dynamo 更强调推理系统运行时和分离式 P/D/KV 数据路径。和 llm-d 相比，AIBrix 更一体化，llm-d 更生态拆分（router、kv-cache、guides）。和 [skypilot](#) 相比，AIBrix 管集群内 inference，不管多云资源采购。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

apiserver-network-proxy

apiserver-network-proxy 通过 connectivity server/agent 建立 apiserver 到节点网络的反向隧道，适合托管集群或私有节点网络。详见 [src-apiserver-network-proxy-architecture](#)。

架构边界

它解决 control plane 到 node 网络连通，不是普通 Service mesh 或 Ingress。

什么时候用

场景	判断
需要 control plane network proxy 能力	适合，apiserver-network-proxy 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , cloud-native-security 组合。

核心组件

- Server: control-plane side proxy
- Agent: node/cluster side tunnel
- gRPC streams: multiplexed connections
- Integration: apiserver egress selector

选型提示

把 apiserver-network-proxy 放在 control plane network proxy 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

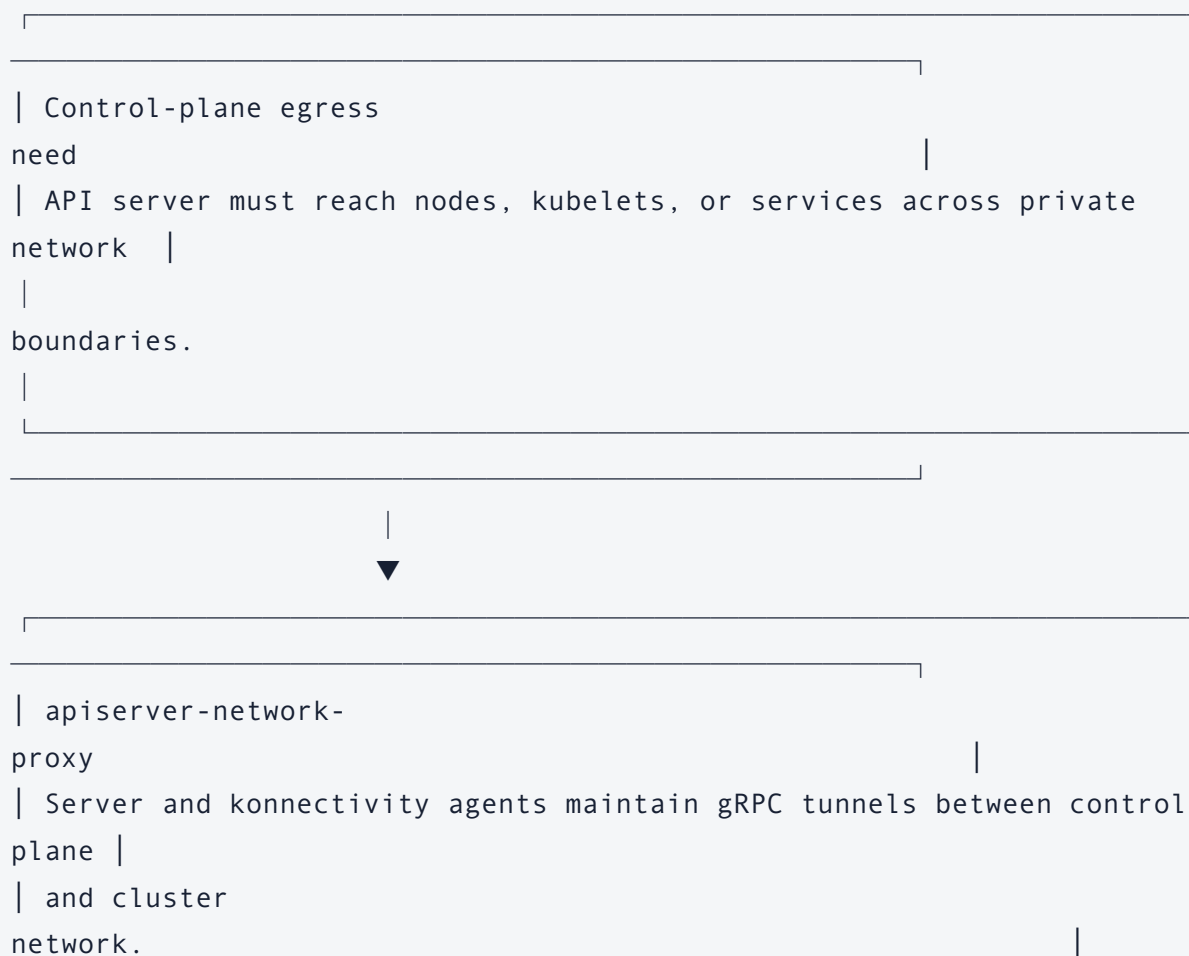
apiserver-network-proxy 架构与设计思路分析

原文: raw/apiserver-network-proxy-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/apiserver-network-proxy> · 优先级 P1

一句话定位

apiserver-network-proxy 通过 konnectivity server/agent 建立 apiserver 到节点网络的反向隧道，适合托管集群或私有节点网络。

核心架构图

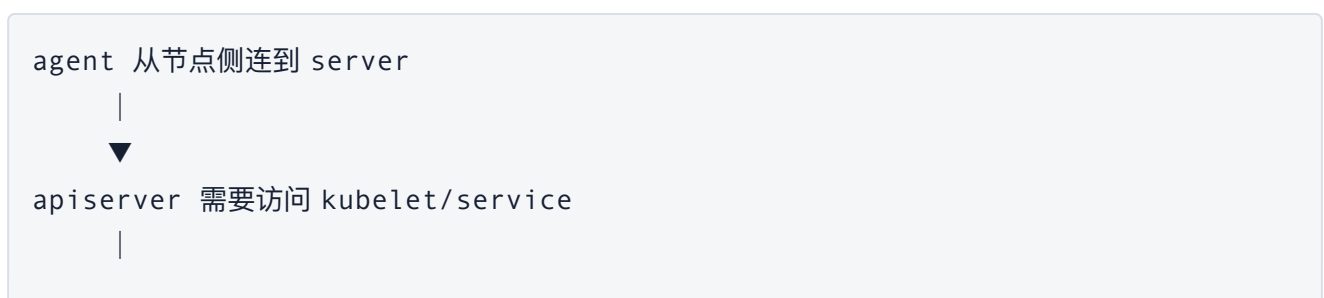


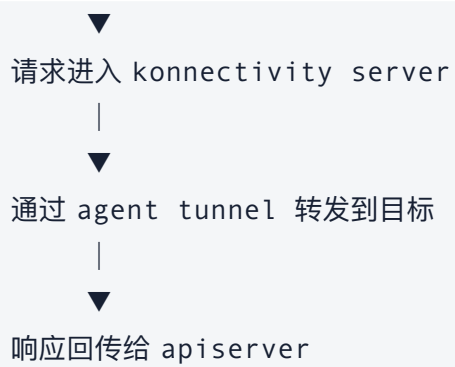


模块分层

层 / 模块	职责
Server	control-plane side proxy
Agent	node/cluster side tunnel
gRPC streams	multiplexed connections
Integration	apiserver egress selector

关键数据流





设计决策与哲学

- 补齐 control plane network proxy 维度: apiserver-network-proxy 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: 它解决 control plane 到 node 网络连通, 不是普通 Service mesh 或 Ingress。
- 选型价值: 它应和 [kubernetes](#), [cloud-native-security](#) 一起看, 而不是孤立评估。

相关页面

- [apiserver-network-proxy](#)
- [kubernetes](#)
- [cloud-native-security](#)

Argo CD

Kubernetes 声明式 GitOps 持续交付工具（CNCF 毕业项目）。

核心特性

- Git 仓库作为单一事实来源
- 多模板支持（Helm / Kustomize / Jsonnet / YAML）
- 自动漂移检测与同步
- 多集群管理
- SSO + RBAC + 多租户
- 蓝绿 / 金丝雀部署（Sync hooks）

架构

K8s controller 持续比较 live state 与 Git desired state → OutOfSync 时提供同步选项。

相关

- [gitops](#) — Argo CD 所实现的模式
- [kubernetes](#) — 运行平台
- 详见 [src-argocd-overview](#)

Argo CD 概览

摘要

Kubernetes 声明式 GitOps 持续交付工具，以 Git 仓库为应用期望状态的单一事实来源。

核心能力

- 多模板：Kustomize / Helm / Jsonnet / YAML
- 多集群部署
- SSO + RBAC + 多租户
- 自动漂移检测 + 同步
- PreSync/Sync/PostSync hooks（蓝绿、金丝雀）
- 回滚到任意 Git 提交

架构

作为 K8s controller 持续比较 live state 与 Git desired state，OutOfSync 时可可视化差异并提供同步选项。

Cluster API

Cluster API 用声明式 API 管理 Kubernetes 集群生命周期，把 Cluster/Machine/MachineDeployment 和 provider infra/bootstrap/control-plane 拆成可组合控制器。详见 [src-cluster-api-architecture](#)。

架构边界

Kubespray 偏 Ansible 部署；Cluster API 偏 Kubernetes-native 声明式集群生命周期。

什么时候用

场景	判断
需要 集群生命周期 能力	适合，Cluster API 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , cloud-native-security 组合。

核心组件

- Core API: Cluster, Machine, MachineDeployment, MachineSet
- Providers: infrastructure/bootstrap/control-plane
- Controllers: reconcile desired cluster state
- Clusterctl: provider init/move/upgrade workflow

选型提示

把 Cluster API 放在 集群生命周期 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

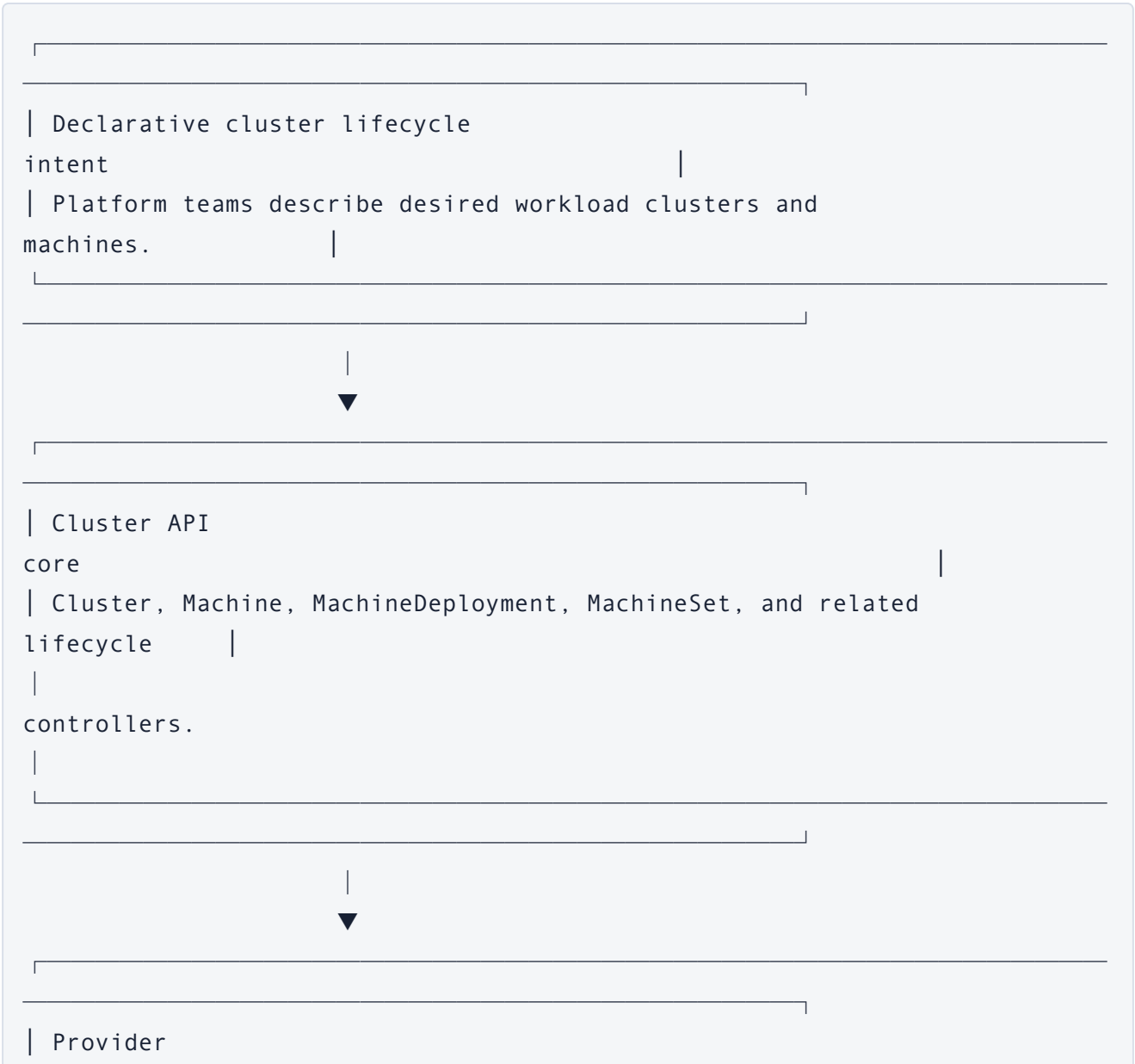
Cluster API 架构与设计思路分析

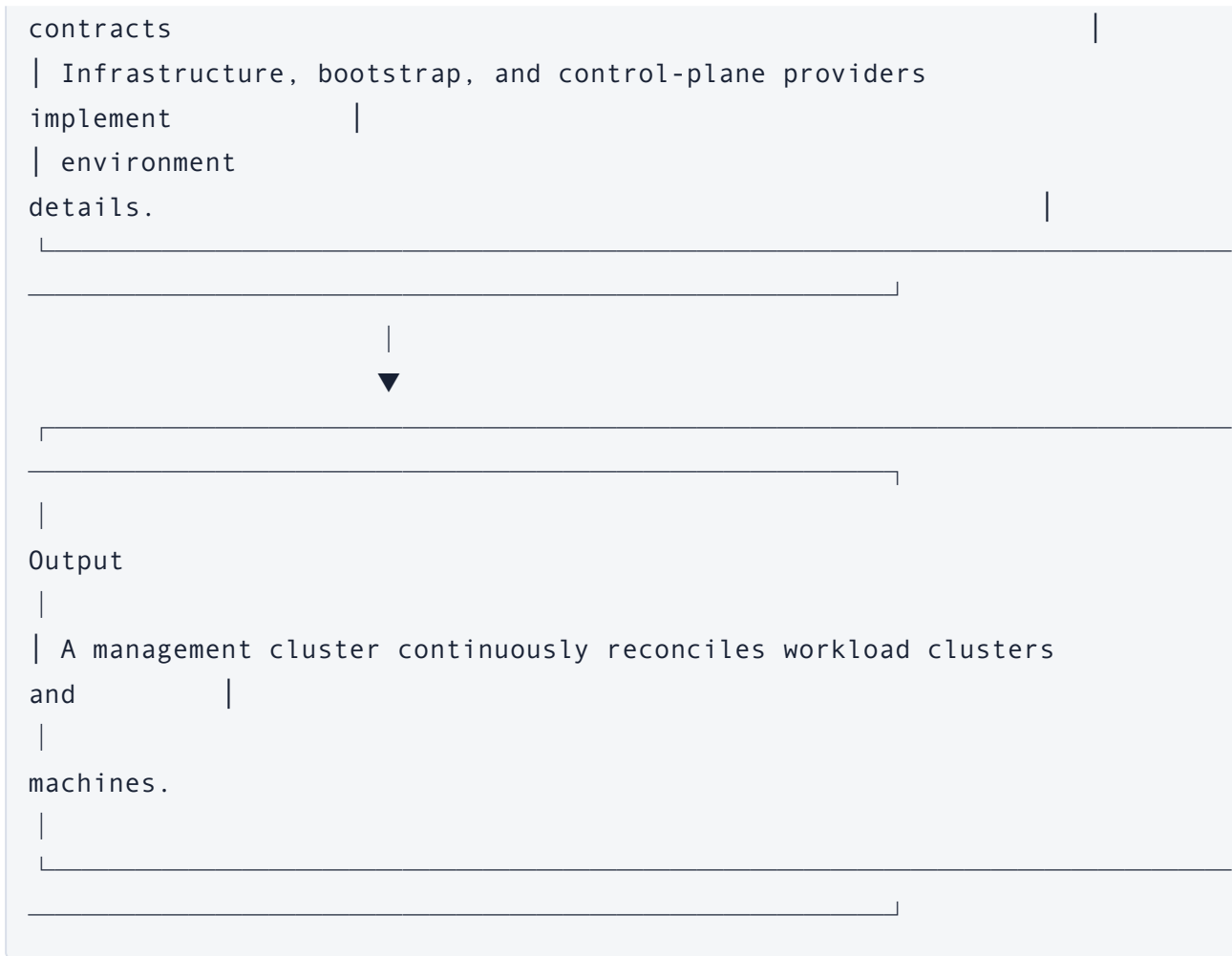
原文: raw/cluster-api-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/cluster-api> · 优先级 P0

一句话定位

Cluster API 用声明式 API 管理 Kubernetes 集群生命周期，把 Cluster/Machine/MachineDeployment 和 provider infra/bootstrap/control-plane 拆成可组合控制器。

核心架构图

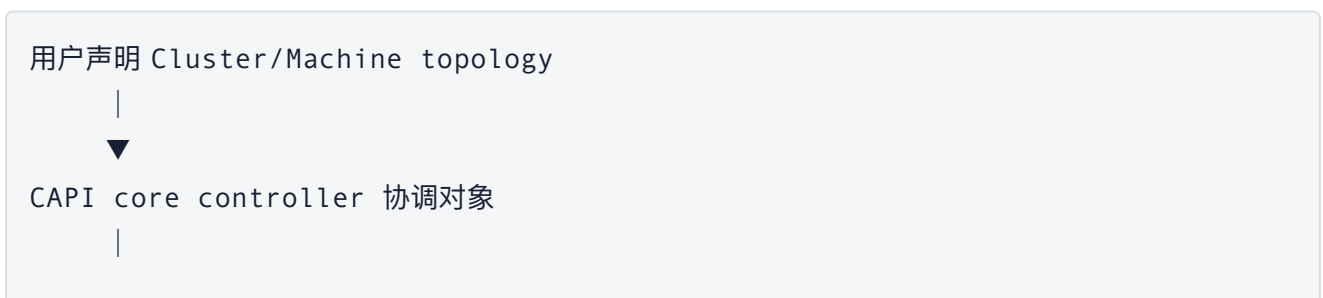




模块分层

层 / 模块	职责
Core API	Cluster, Machine, MachineDeployment, MachineSet
Providers	infrastructure/bootstrap/control-plane
Controllers	reconcile desired cluster state
Clusterctl	provider init/move/upgrade workflow

关键数据流





设计决策与哲学

- 补齐 集群生命周期 维度：Cluster API 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Kubespray 偏 Ansible 部署；Cluster API 偏 Kubernetes-native 声明式集群生命周期。
- 选型价值：它应和 [kubernetes](#), [cloud-native-security](#) 一起看，而不是孤立评估。

相关页面

- [cluster-api](#)
- [kubernetes](#)
- [cloud-native-security](#)

Container Device Interface

CDI (Container Device Interface) 用标准化 spec 描述容器需要的设备节点、mount、env、hooks 等 edits，由兼容的 container runtime 按选定的 CDI device ID 应用配置。它是设备注入接口，不负责 scheduler 选节点或 allocator 选设备。

在 M5-C Device / GPU 地图中的位置

设备工具链或 driver 生成/维护 CDI spec；[k8s-device-plugin](#) 可通过配置的 CDI strategy 返回设备标识，[dra-driver-nvidia-gpu](#) 可在 Prepare 后返回 CDI devices，kubelet 再把相应配置交给 runtime 消费。[gpu-operator](#) 管理受管 driver/toolkit 等组件的部署，[hami](#) 可在支持的分配路径交接设备与隔离配置，两者都不能把 CDI 当作独立分配决策器。

两条注入路径见 [k8s-gpu-device-stack](#)，Prepare/CDI/checkpoint 与共享 Claim 回收边界见 [kubernetes-dra-design-deep-dive](#)。CDI 注入成功也不证明设备健康或共享隔离策略已满足。

在 GPU 栈中的位置

[k8s-device-plugin](#) 支持通过 CDI annotations/device list strategy 把 NVIDIA 设备交给容器；[dra-driver-nvidia-gpu](#) 也在 Prepare 阶段生成或引用 CDI 配置。CDI 是传统 device plugin 与 DRA 都会复用的设备注入抽象。

选型提示

如果你在比较 envvar、volume-mounts、CDI annotations，优先理解 CDI：它更适合复杂设备、hooks 和 runtime edits 的标准化。

controller-runtime

controller-runtime 是现代 Kubernetes controller 的通用库，封装 Manager、cache、client、reconcile、webhook、envtest 等生产控制器骨架。详见 [src-controller-runtime-architecture](#)。

证据说明：本次官方默认分支快照为 [6ab2188a1fb1](#)，该 commit 不代表 release。[src-controller-runtime-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目证据与职责图见 [k8s-core-controller-map](#)。

在 M5-A Controller 地图中的位置

controller-runtime 在 client-go 基础上组合 Manager、Cache、Client、Controller/Reconciler 与 Webhook，并提供 envtest 测试支持，承担运行时框架职责。[kubebuilder](#) 负责作者工作流与项目脚手架，[controller-tools](#) 负责 CRD 等资产生成；controller-runtime 本身不承担这两类生成工作。三者的组合关系与控制循环见 [k8s-core-controller-map](#)。

架构边界

client-go 是底层机制；controller-runtime 是现代 operator 工程默认抽象；kubebuilder 在其上做项目脚手架。

什么时候用

场景	判断
需要 Operator SDK 能力	适合，controller-runtime 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , model-serving-operator , declarative-agent-management 组合。

核心组件

- Manager: lifecycle、leader election、scheme、metrics
- Cache/Client: informer cache + API writer
- Controller/Reconciler: workqueue and reconcile loop
- Webhook/envtest: admission and test harness

选型提示

把 controller-runtime 放在 Operator SDK 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

controller-runtime 架构与设计思路分析

原文: [raw/controller-runtime-architecture-analysis.md](#) · 仓库: <https://github.com/kubernetes-sigs/controller-runtime> · 优先级 P0

一句话定位

controller-runtime 是现代 Kubernetes controller 的通用库，封装 Manager、cache、client、reconcile、webhook、envtest 等生产控制器骨架。

核心架构图

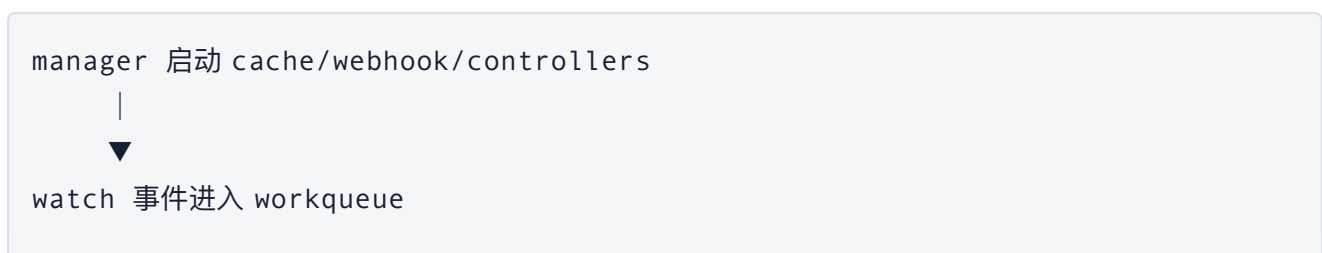




模块分层

层 / 模块	职责
Manager	lifecycle、leader election、scheme、metrics
Cache/Client	informer cache + API writer
Controller/Reconciler	workqueue and reconcile loop
Webhook/envtest	admission and test harness

关键数据流





设计决策与哲学

- 补齐 Operator SDK 维度: controller-runtime 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: client-go 是底层机制; controller-runtime 是现代 operator 工程默认抽象; kubebuilder 在其上做项目脚手架。
- 选型价值: 它应和 [kubernetes](#), [model-serving-operator](#), declarative-agent-management 一起看, 而不是孤立评估。

相关页面

- [controller-runtime](#)
- [kubernetes](#)
- [model-serving-operator](#)
- declarative-agent-management

controller-tools

controller-tools 提供 controller-gen，用 Go marker 生成 CRD、RBAC、webhook、deepcopy 等 Kubernetes API 工程资产。详见 [src-controller-tools-architecture](#)。

证据说明：本次官方默认分支快照为 [030a93937cbb](#)，该 commit 不代表 release。[src-controller-tools-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目证据与职责图见 [k8s-core-controller-map](#)。

在 M5-A Controller 地图中的位置

controller-tools 的 controller-gen 解析 Go markers/types，生成 CRD、RBAC、webhook manifests、deepcopy（object generator）与 applyconfiguration 等资产。[kubebuilder](#) 把它接入作者工作流；[controller-runtime](#) 负责运行时控制器框架，controller-tools 不运行 controller。完整生成链与运行时边界见 [k8s-core-controller-map](#)。

架构边界

kubebuilder 是脚手架；controller-tools 是实际生成 CRD/RBAC 等产物的工具链。

什么时候用

场景	判断
需要 API 生成工具 能力	适合，controller-tools 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , model-serving-operator , declarative-agent-management 组合。

核心组件

- Markers parser: 读取 Go type/comment markers
- CRD generator: OpenAPI schema and validation
- RBAC/webhook generators
- object/deepcopy generation

选型提示

把 controller-tools 放在 API 生成工具 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

controller-tools 架构与设计思路分析

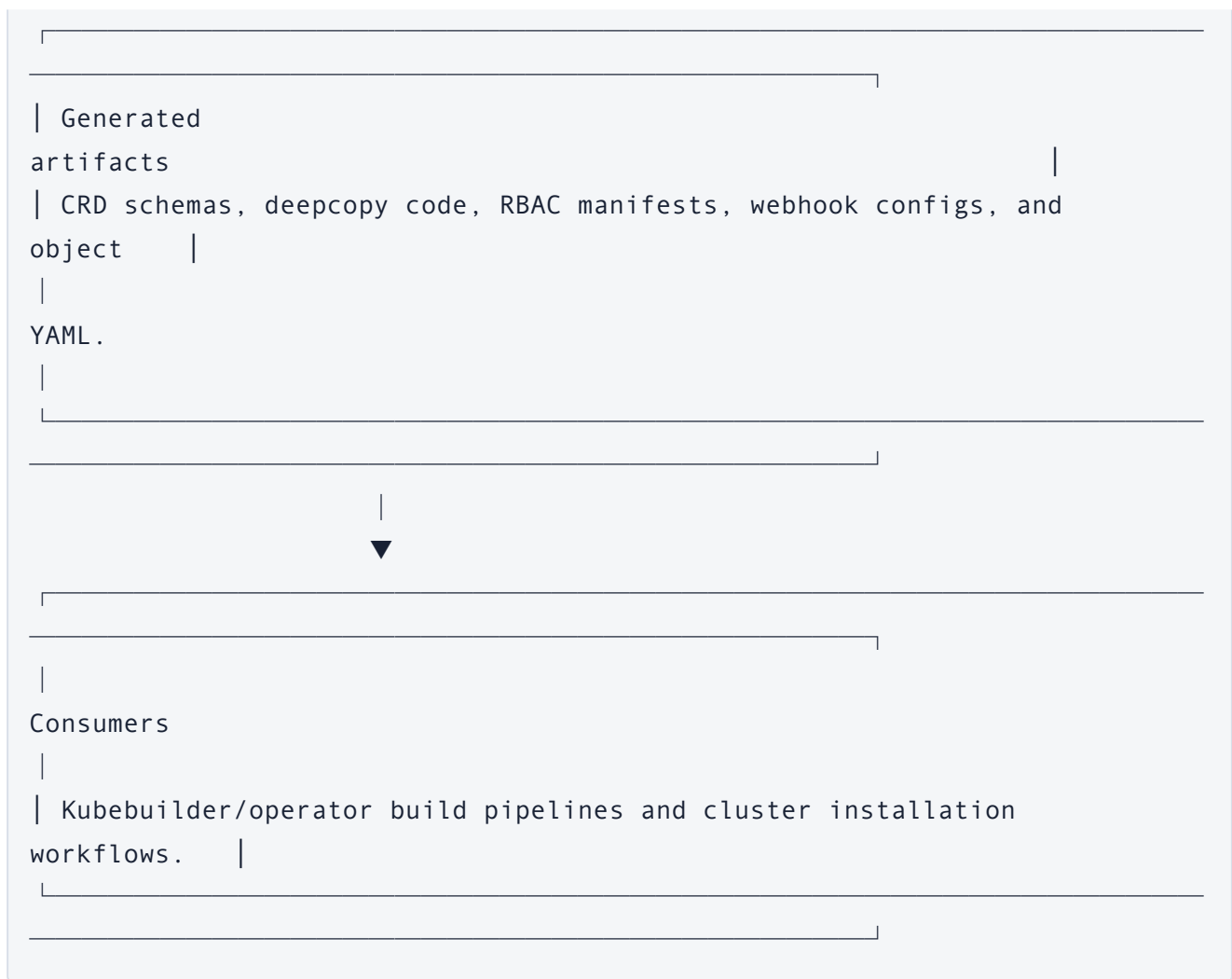
原文: raw/controller-tools-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/controller-tools> · 优先级 P0

一句话定位

controller-tools 提供 controller-gen，用 Go marker 生成 CRD、RBAC、webhook、deepcopy 等 Kubernetes API 工程资产。

核心架构图

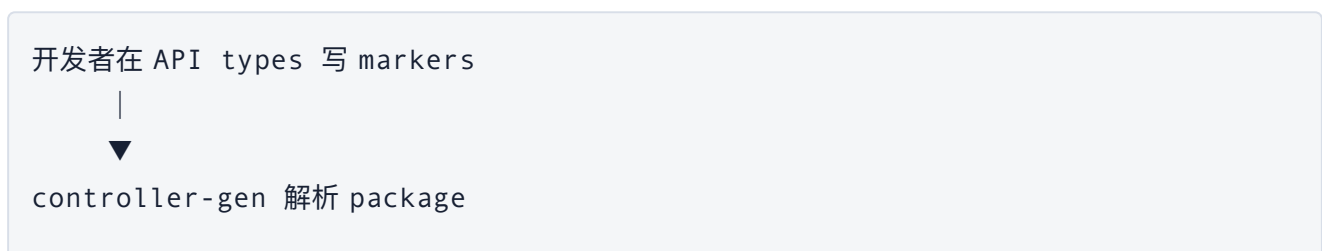




模块分层

层 / 模块	职责
Markers parser	读取 Go type/comment markers
CRD generator	OpenAPI schema and validation
RBAC/webhook generators	RBAC/webhook generators
object/deepcopy generation	object/deepcopy generation

关键数据流





设计决策与哲学

- 补齐 API 生成工具 维度：controller-tools 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：kubebuilder 是脚手架；controller-tools 是实际生成 CRD/RBAC 等产物的工具链。
- 选型价值：它应和 [kubernetes](#), [model-serving-operator](#), declarative-agent-management 一起看，而不是孤立评估。

相关页面

- [controller-tools](#)
- [kubernetes](#)
- [model-serving-operator](#)
- declarative-agent-management

ControllerMesh

ControllerMesh 是 OpenKruise 生态里的 controller/operator isolation and management 项目。根据 [src-openkruise-projects-current-state](#)，它属于 P1：活跃度不如 Kruise / Rollouts / Kruise Game，但适合作为 [kubernetes-workload-automation](#) 中控制器运行边界、故障半径和多 operator 管理的设计参考。

架构边界

它不解决单个业务 workload 的生命周期，而是关注控制器本身如何隔离、分片、管理和降低 blast radius。这个问题和 [controller-runtime](#) 常见的单 manager 多 controller 模式不同：后者解决工程骨架，ControllerMesh 更偏控制器运行形态和隔离策略。

什么时候看它

场景	判断
控制器太多、权限太大或故障影响面太广	值得研究 controllermesh ，并结合 cloud-native-security 看权限和审计边界。
想写一个普通 CRD controller	先看 controller-runtime / kubebuilder 。
想理解 OpenKruise workload	先看 openkruise-kruise 。

选型提示

ControllerMesh 的架构价值大于当前项目热度。它提醒平台工程不要只关注 workload controller 的业务逻辑，也要关注控制器自身的权限、隔离、可观测和故障边界。

CRI Tools

CRI Tools 提供 `crictl` 和 `critest`，用于操作与验证 kubelet Container Runtime Interface。详见 [src-cri-tools-architecture](#)。

架构边界

`kubectl` 面向 Kubernetes API；`crictl` 直接面向 CRI runtime，是节点级诊断工具。

什么时候用

场景	判断
需要 计算 / Runtime 能力	适合，CRI Tools 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , cloud-native-security 组合。

核心组件

- `crictl`: `inspect/run/exec/logs/images/pods`
- `critest`: CRI conformance/validation
- Runtime endpoint config
- Kubelet/runtime debugging workflow

选型提示

把 CRI Tools 放在 计算 / Runtime 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

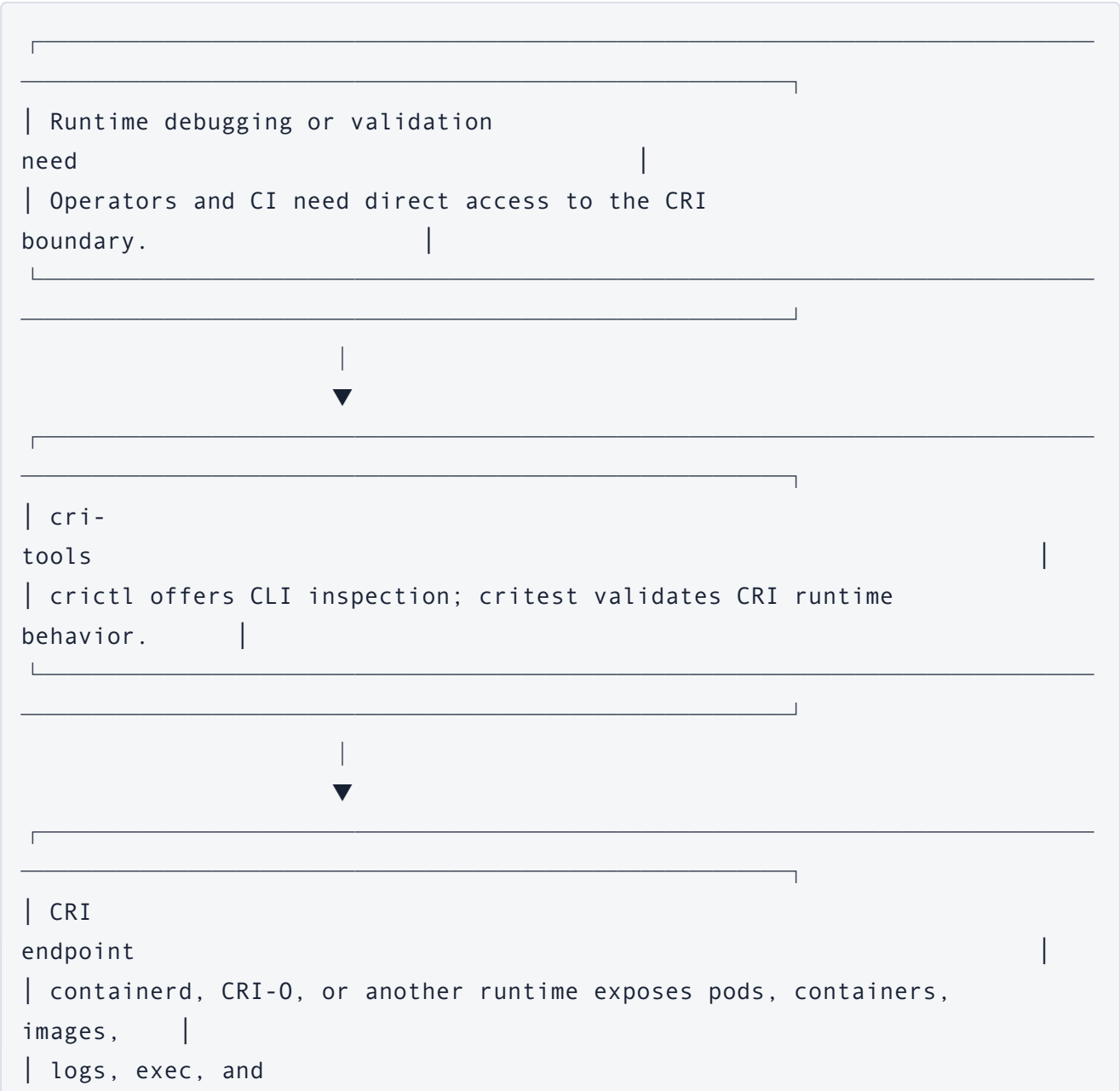
CRI Tools 架构与设计思路分析

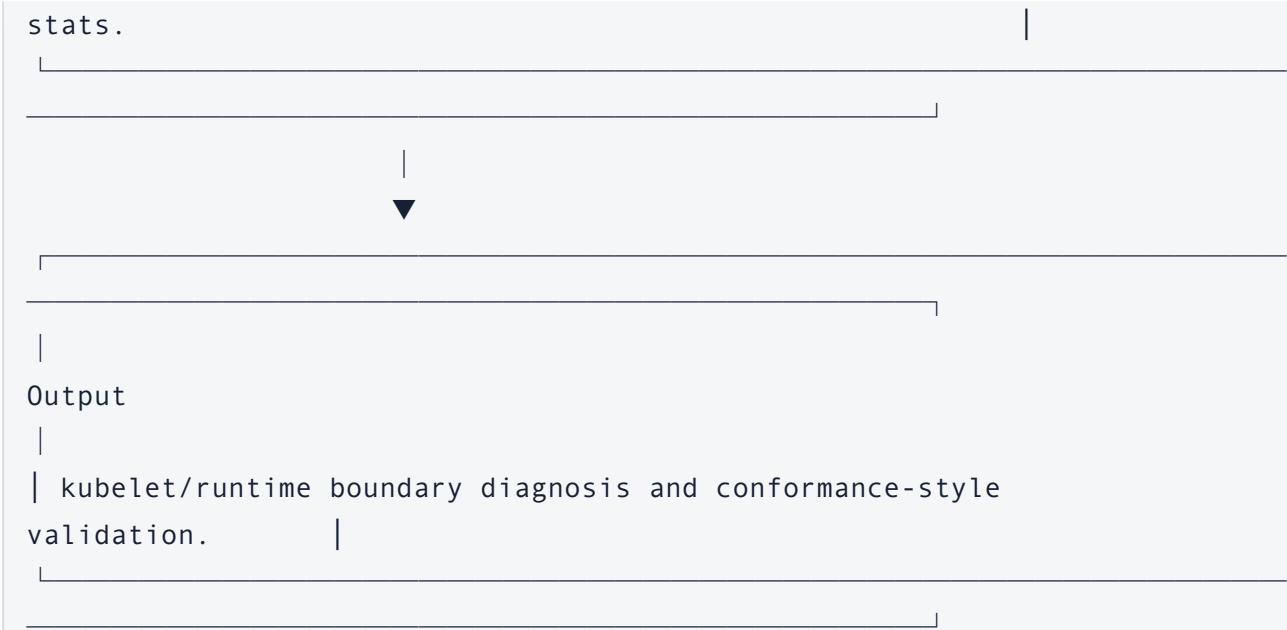
原文: raw/cri-tools-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/cri-tools> · 优先级 P0

一句话定位

CRI Tools 提供 crictl 和 critest，用于操作与验证 kubelet Container Runtime Interface。

核心架构图





模块分层

层 / 模块	职责
crictl	inspect/run/exec/logs/images/pods
critest	CRI conformance/validation
Runtime endpoint config	Runtime endpoint config
Kubelet/runtime debugging workflow	Kubelet/runtime debugging workflow

关键数据流





设计决策与哲学

- 补齐 计算 / Runtime 维度：CRI Tools 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：kubectl 面向 Kubernetes API；crictl 直接面向 CRI runtime，是节点级诊断工具。
- 选型价值：它应和 [kubernetes](#), [cloud-native-security](#) 一起看，而不是孤立评估。

相关页面

- [cri-tools](#)
- [kubernetes](#)
- [cloud-native-security](#)

Descheduler

Descheduler 根据策略驱逐已经运行的 Pods，让 kube-scheduler 有机会重新放置，修复节点漂移、拓扑不均、约束变化等问题。详见 [src-descheduler-architecture](#)。

架构边界

scheduler 决定新 Pod 放哪；descheduler 处理运行一段时间后的布局退化。

什么时候用

场景	判断
需要 调度后优化 能力	适合，Descheduler 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , llm-inference 组合。

核心组件

- Policy config: strategies/profiles
- Strategies: remove duplicates, low utilization, topology spread, affinity violations
- Evictor: safe pod eviction
- CronJob/controller deployment modes

选型提示

把 Descheduler 放在 调度后优化 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

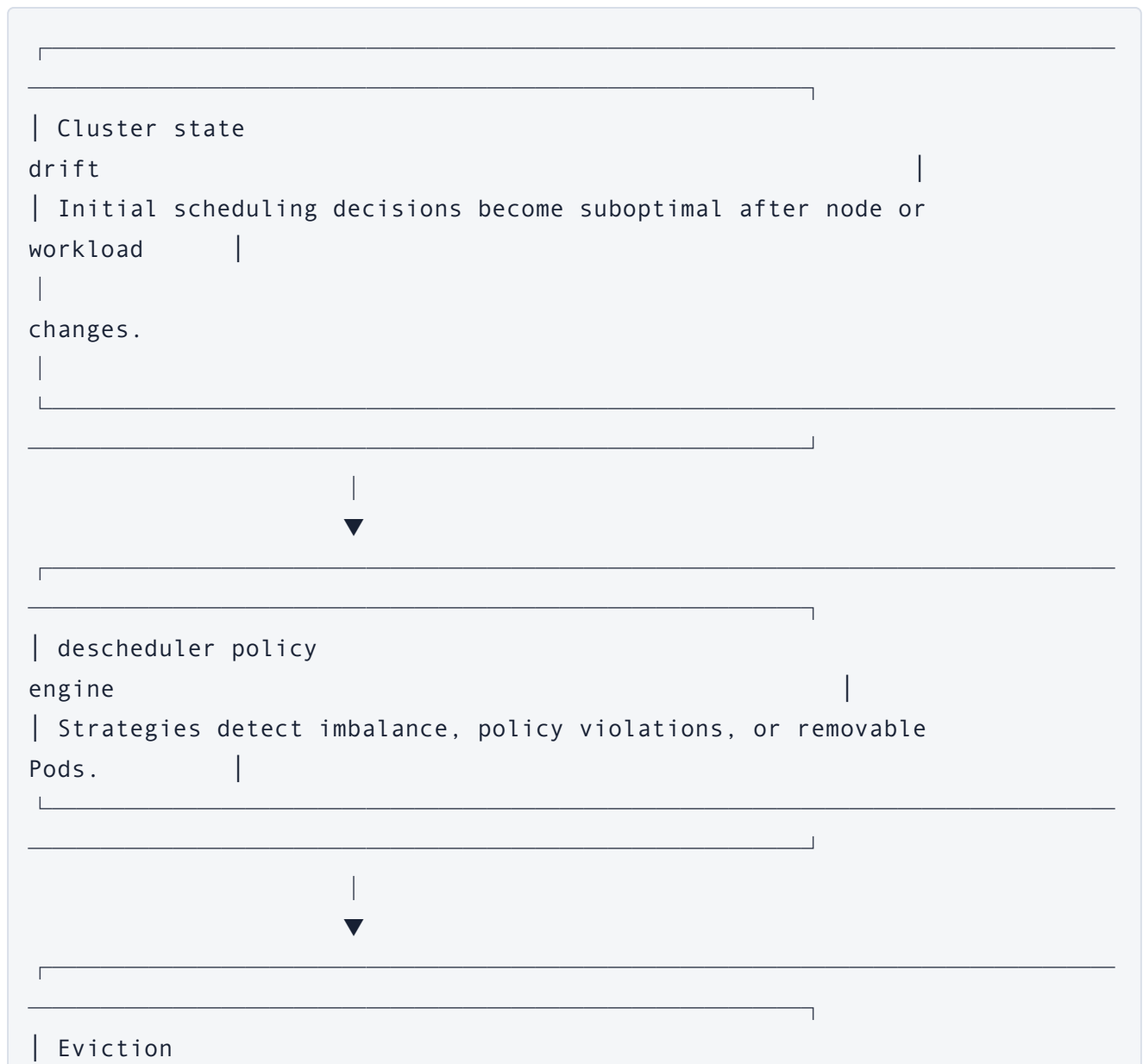
Descheduler 架构与设计思路分析

原文: raw/descheduler-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/descheduler> · 优先级 P1

一句话定位

Descheduler 根据策略驱逐已经运行的 Pods，让 kube-scheduler 有机会重新放置，修复节点漂移、拓扑不均、约束变化等问题。

核心架构图





模块分层

层 / 模块	职责
Policy config	strategies/profiles
Strategies	remove duplicates, low utilization, topology spread, affinity violations
Evictor	safe pod eviction
CronJob/controller deployment modes	CronJob/controller deployment modes

关键数据流



evict Pod

|



scheduler 重新调度

设计决策与哲学

- 补齐 调度后优化 维度：Descheduler 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：scheduler 决定新 Pod 放哪；descheduler 处理运行一段时间后的布局退化。
- 选型价值：它应和 [kubernetes](#), llm-inference 一起看，而不是孤立评估。

相关页面

- [descheduler](#)
- [kubernetes](#)
- llm-inference

DRA Driver for NVIDIA GPUs

NVIDIA GPU 的 Kubernetes Dynamic Resource Allocation driver，围绕 ResourceClaim、ResourceSlice、NodePrepareResources、ComputeDomain/Multi-Node NVLink、动态 MIG/VFIO 配置展开。详见 [src-dra-driver-nvidia-gpu-architecture](#)。

证据说明：2026-09-27 通过 NVIDIA 仓库入口查询的官方默认分支快照为 [495bf4c59b94](#)，链接采用 GitHub 返回的规范仓库，commit 不代表 release。[src-dra-driver-nvidia-gpu-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，阶段与生命周期见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

driver 提供 DeviceClass 等安装配置、发布 ResourceSlice 库存，并通过 NodePrepareResources/NodeUnprepareResources 执行设备配置与 [cdi](#) 交接；GPU 与 ComputeDomain 分别处理本机设备和 Multi-Node NVLink 相关生命周期。这不同于 [k8s-device-plugin](#) 的传统 extended-resource Allocate 路径，也不等于 [node-feature-discovery](#) 的节点能力发现。

厂商支持矩阵必须与 Kubernetes feature stage 分开：锚定仓库 README 对独立 GPU plugin 与 ComputeDomain 的支持说明，不能直接套用到 [gpu-operator](#) 26.7 所管理版本。基础 DRA stable 不保证所有动态 MIG/VFIO/sharing 功能同样稳定。

架构边界

它采用 Kubernetes DRA 设备资源 API，不是传统 device plugin API 的小补丁。与 [k8s-device-plugin](#) 相比，它把资源声明、配置和调度语义前移到 Claim 模型；[hami](#) 也有 DRA 接入，比较时应看厂商设备配置能力、分配路径与容器隔离，而非是否使用 DRA。

选型判断

适合需要 ResourceClaim/ResourceSlice、动态设备配置或 ComputeDomain 的平台团队；部署应按所选 Kubernetes/driver release 核对 API、feature gates 和硬件支持。

只需要 GPU extended resource 暴露时可评估 [k8s-device-plugin](#) 和 [gpu-operator](#)；选择 DRA 时须独立验证分配、节点准备与回收全链路。

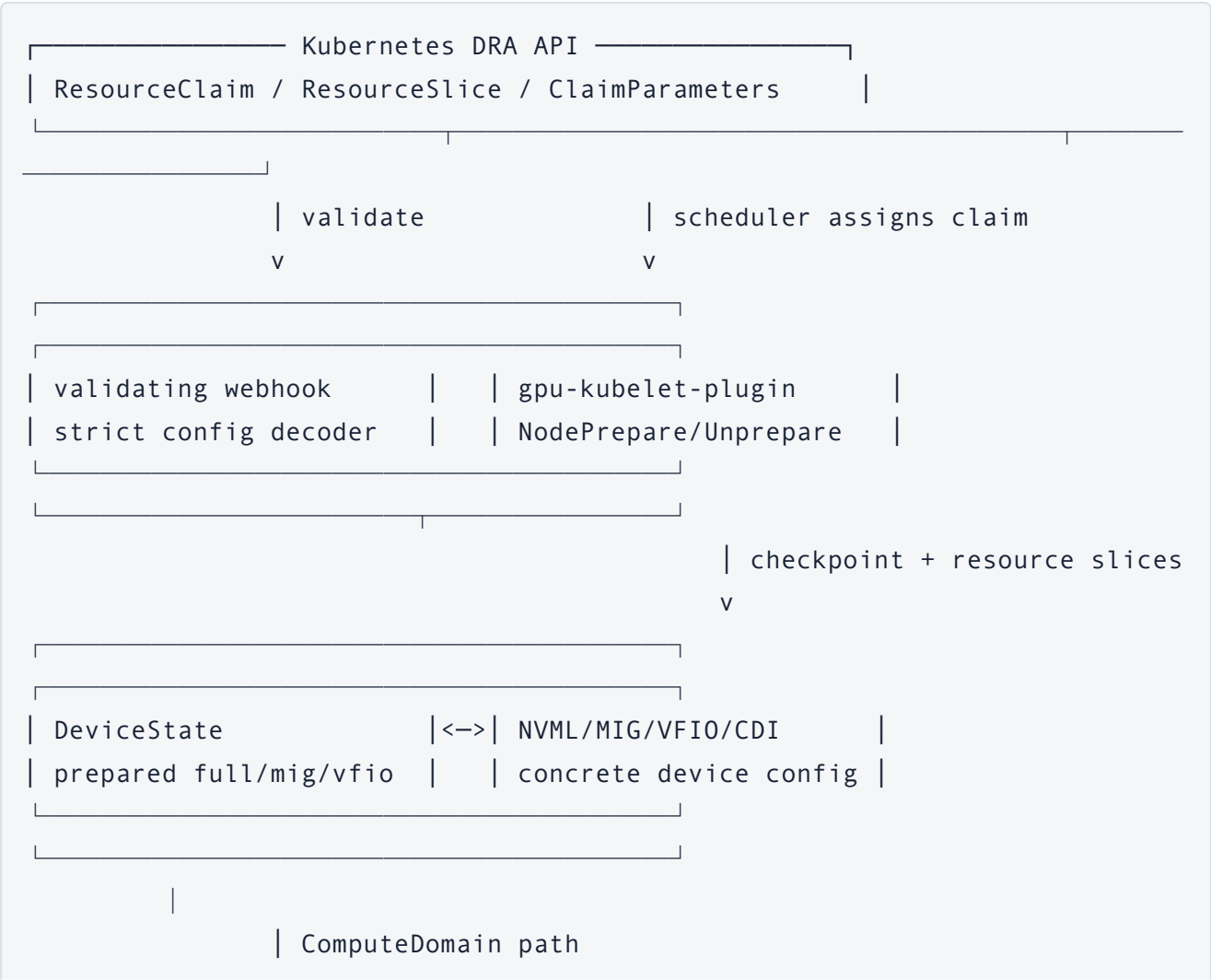
DRA Driver for NVIDIA GPUs 架构与设计思路分析

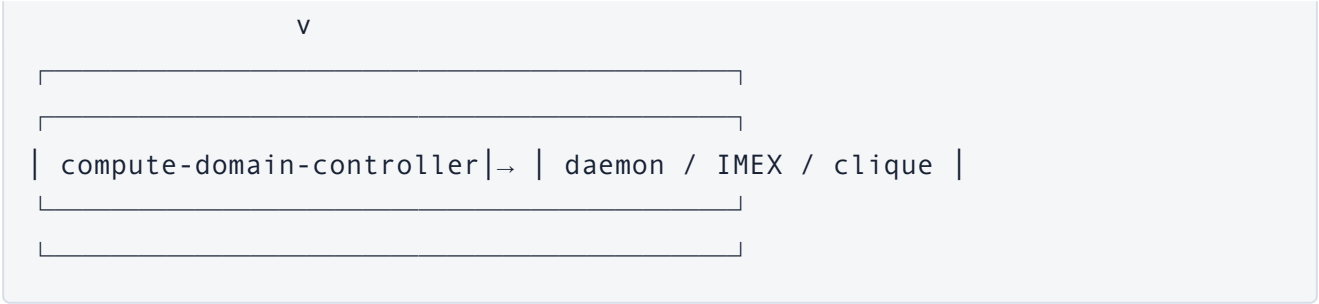
原文: raw/dra-driver-nvidia-gpu-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/dra-driver-nvidia-gpu> · 分析版本 HEAD 749a743 (2026-06-11)

一句话定位

NVIDIA GPU 的 Kubernetes Dynamic Resource Allocation driver，围绕 ResourceClaim/ResourceSlice、NodePrepareResources、ComputeDomain/Multi-Node NVLink 和动态 MIG/VFIO 配置展开。它代表 K8s 新一代设备分配模型，而不是传统 device plugin API 的增量补丁。这页和 [kubernetes](#) dra [cdi gpu-sharing](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图





模块分层

层 / 模块	职责
DRA kubelet plugins	实现 kubelet DRA plugin，发布 ResourceSlice，执行 Prepare/Unprepare 和 checkpoint 清理。
GPU state/config	抽象 full GPU、MIG、VFIO、sharing、checkpoint 与资源 slice。
ComputeDomain 控制面	管理 ComputeDomain/Clique 生命周期，动态渲染 daemonset 与 IMEX primitives。
API 与 webhook	定义 GpuConfig/MigDeviceConfig/ComputeDomain 等配置，并验证 ResourceClaim 参数。
部署与站点	Helm/demo/docs 展示 K8s 1.32+ DRA 使用方式。

关键数据流



设计决策与哲学

- 接受 DRA 的声明式 claim 模型：资源配置进入 ResourceClaim，而不是 Pod annotation；这把厂商调度逻辑接到 Kubernetes 1.32+ DRA 语义上。
- checkpoint 是动态设备生命周期的事实来源：NewDriver 在 DynamicMIG 下会清理未知 MIG devices，避免重启后静态/动态状态混乱。
- GPU 和 ComputeDomain 分成两个插件/控制面：ComputeDomain 面向 Multi-Node NVLink/IMEX，GPU 面向本机 device allocation，生命周期和风险边界不同。

与同类项目的架构差异

维度	DRA Driver for NVIDIA GPUs	同类 A	同类 B
定位	K8s DRA NVIDIA driver	k8s-device-plugin: beta device plugin API	HAMi: scheduler extender vGPU sharing
资源 API	ResourceClaim/ ResourceSlice	extended resource + kubelet plugin	Pod annotation + extender
高级能力	dynamic MIG/VFIO/ ComputeDomain	MIG/MPS/time-slicing/CDI	memory/core sharing + heterogeneous vendors

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- [dra](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)

Dynamo

NVIDIA 开源的数据中心级 LLM 推理编排层。Dynamo 位于推理 engine 之上，负责入口、路由、P/D 协调、KV 传输、容量控制、Kubernetes 部署与故障连续性；不替代 SGLang、vLLM 或 TensorRT-LLM 的 GPU 执行内核。

架构边界

Frontend / Gateway EPP → KV-aware Router → backend workers 是请求路径；DGDR / Profiler / AIConfigurator → DGD / DCD → Operator 是部署路径；Planner → connector / Operator 是运行时容量路径。三条路径通过 Runtime 的 discovery、request 和 event planes 连接。

什么时候用

- 多 GPU、多节点或长上下文在线推理。
- 需要独立扩缩 prefill/decode、KV-aware routing 或高速 KV transfer。
- 需要按 TTFT/ITL/SLA 和 TCO 做容量规划与自动扩缩。
- 需要 Gateway API 入口、请求迁移、canary health check 或 topology-aware deployment。

什么时候不用

单模型、单 GPU、没有跨节点协调和 SLA 控制需求时，直接使用 vllm 或 sglang 更简单。若只需要标准 Kubernetes 模型生命周期而不需要 Dynamo 的 P/D、KV 和 Planner，应优先比较 [kserve](#)、[ome](#) 等 Operator。

与同类比较

项目	主要边界	与 Dynamo 的差异
vllm / sglang	推理 engine	更靠近单机/多 GPU 执行与 batching
llm-d	K8s inference stack、router、pool	更强调云原生入口与组件生态，Dynamo 更强调 engine 上方的端到端编排
gateway-api-inference-extension	Gateway API inference endpoint picking	可作为 Dynamo 的入口承载，不能替代完整 P/D/KV/Planner
kserve / ome	模型服务 CRD/operator	更偏声明式生命周期，Dynamo 提供更深的 serving runtime 协调

详细架构与业务问题见 [src-dynamo-architecture](#)。

在 M4 模块地图中的位置

Dynamo 围绕可插拔 engine 提供分布式 serving runtime，协调 frontend/routing、服务发现与事件、worker 角色、KV transfer 和容量规划。职责边界见 llm-inference-serving-project-map，与 engine、Kubernetes 组件的组合选择见 llm-serving-engine-selection-map。

Envoy AI Gateway

Envoy AI Gateway 是基于 Envoy Gateway 的开源 GenAI traffic gateway，用于把应用请求接入多个 hosted provider 或 self-hosted model serving cluster。

解决的问题

企业需要统一处理 provider 认证、顶层路由、global rate limit、协议和失败边界，同时又希望把自托管集群内部的细粒度 endpoint picking 交给专用 inference router。

核心架构

README 采用 two-tier gateway: Tier One 负责 authentication、top-level routing 和 global rate limiting; Tier Two 负责 self-hosted model access，并可接 endpoint picker / LLM-aware routing。它支持 OpenAI、Azure OpenAI、Gemini、Bedrock、Anthropic 等 provider 方向。

边界与选型

它是 edge/provider governance，不是 P/D worker router，也不是模型质量路由算法。典型组合是 Envoy AI Gateway → semantic-router（可选）→ llm-d-router / [gateway-api-inference-extension](#) → model server。

详见 [src-ai-gateway-architecture](#)、[src-k8s-gateway-routing-comparison](#)。

external-dns

ExternalDNS 从 Service、Ingress、Gateway 等 Kubernetes 对象动态维护外部 DNS records，是声明式网络控制器代表。详见 [src-external-dns-architecture](#)。

架构边界

Gateway/Ingress 决定流量入口；ExternalDNS 负责把入口地址发布到 DNS。

什么时候用

场景	判断
需要 网络 / DNS 能力	适合，external-dns 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 gateway-api , cloud-native-security 组合。

核心组件

- Sources: service/ingress/gateway/istio/contour 等
- Registry: TXT ownership and conflict protection
- Provider: Route53/CloudDNS/Cloudflare 等 DNS API
- Controller loop: desired endpoints -> record changes

选型提示

把 external-dns 放在 网络 / DNS 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

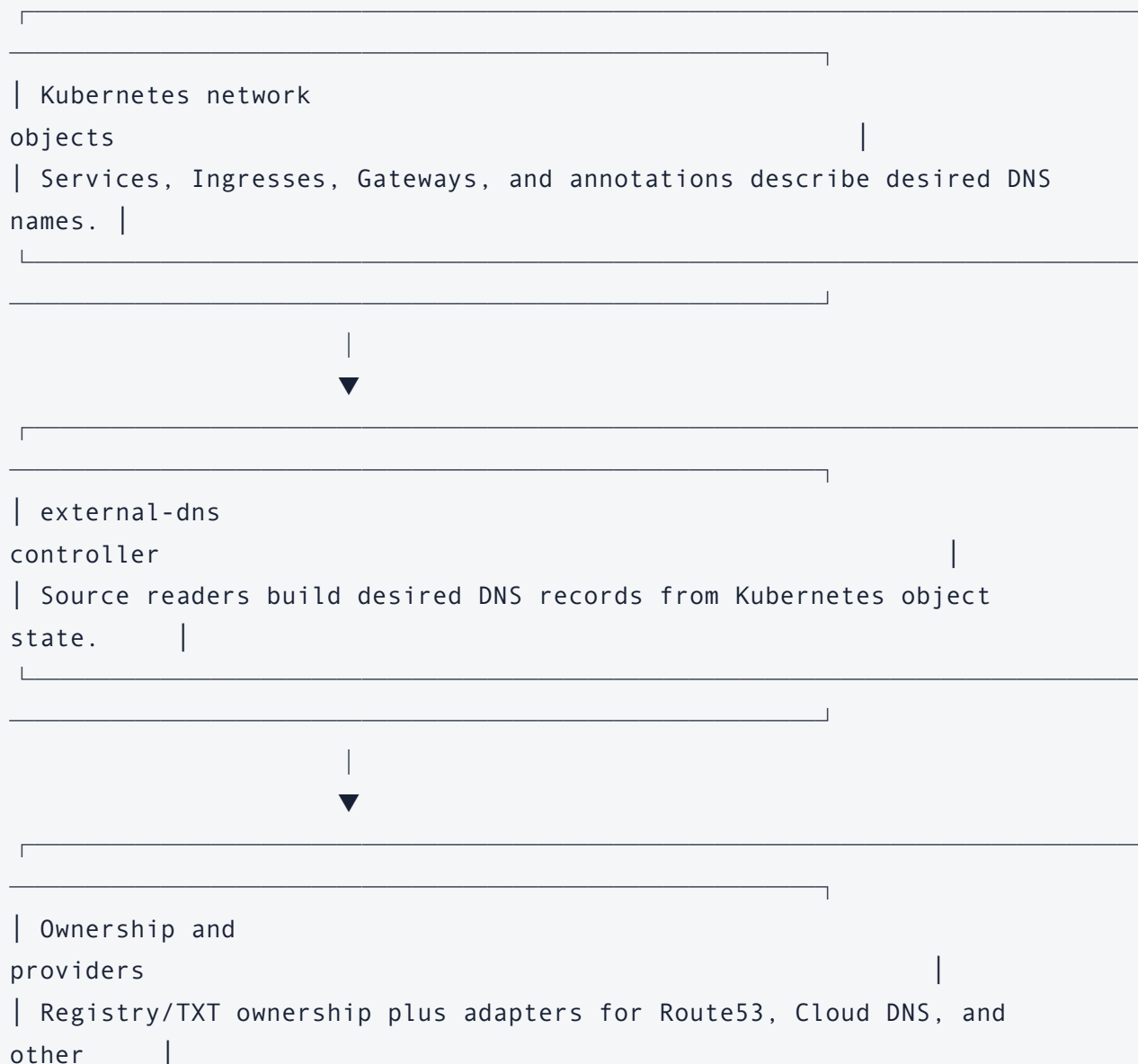
external-dns 架构与设计思路分析

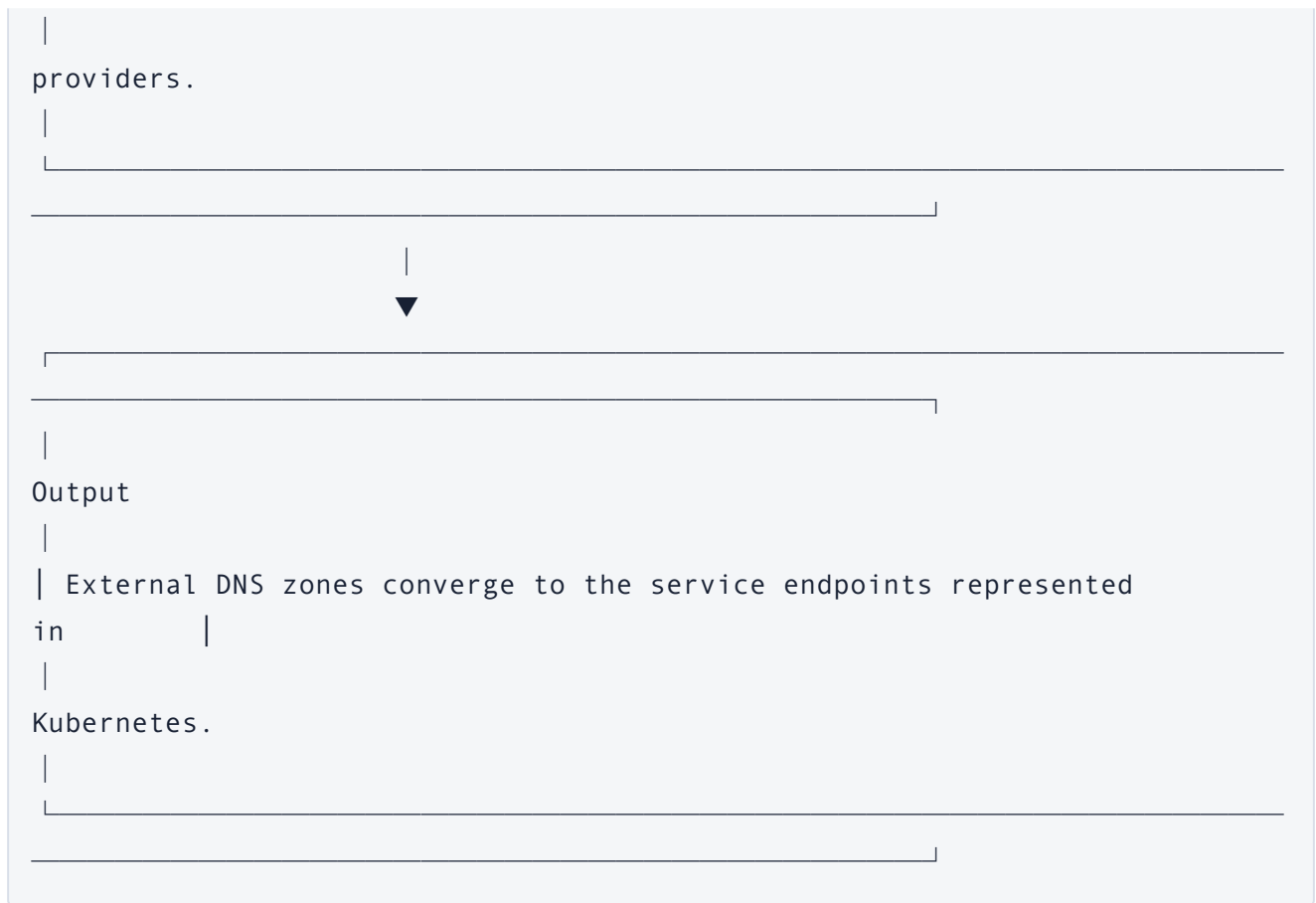
原文: [raw/external-dns-architecture-analysis.md](#) · 仓库: <https://github.com/kubernetes-sigs/external-dns> · 优先级 P0

一句话定位

ExternalDNS 从 Service、Ingress、Gateway 等 Kubernetes 对象动态维护外部 DNS records，是声明式网络控制器代表。

核心架构图

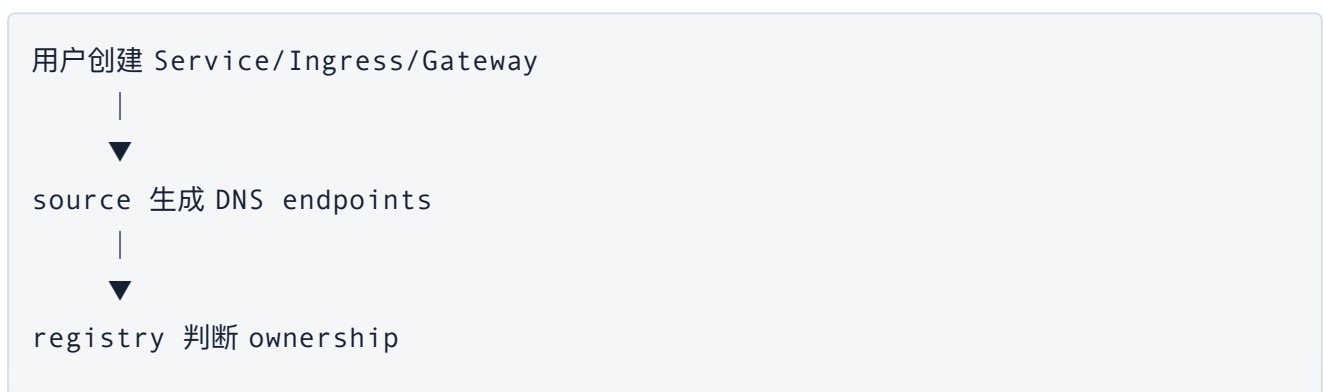




模块分层

层 / 模块	职责
Sources	service/ingress/gateway/istio/contour 等
Registry	TXT ownership and conflict protection
Provider	Route53/CloudDNS/Cloudflare 等 DNS API
Controller loop	desired endpoints -> record changes

关键数据流





设计决策与哲学

- 补齐 网络 / DNS 维度：external-dns 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Gateway/Ingress 决定流量入口；ExternalDNS 负责把入口地址发布到 DNS。
- 选型价值：它应和 [gateway-api](#), [cloud-native-security](#) 一起看，而不是孤立评估。

相关页面

- [external-dns](#)
- [kubernetes](#)
- [gateway-api](#)
- [cloud-native-security](#)

Gateway API Inference Extension

Gateway API Inference Extension 是 Kubernetes Gateway API 的推理扩展，定义 InferencePool、Endpoint Picker interaction 和 inference endpoint picking 的协议/控制面边界。

解决的问题

通用 Service/负载均衡不理解模型身份、推理协议和 endpoint 级运行状态。Inference Extension 让 Gateway 在转发请求前询问模型感知的 Endpoint Picker，从而为 LLM serving 提供标准化选择接口。

核心边界

它是协议和 API 扩展，不是完整模型服务平台、推理 engine 或具体路由算法。llm-d-router 是其重要生产化实现之一；[kserve](#)、[kthena](#) 等平台可以在这条标准边界上集成。

适用场景

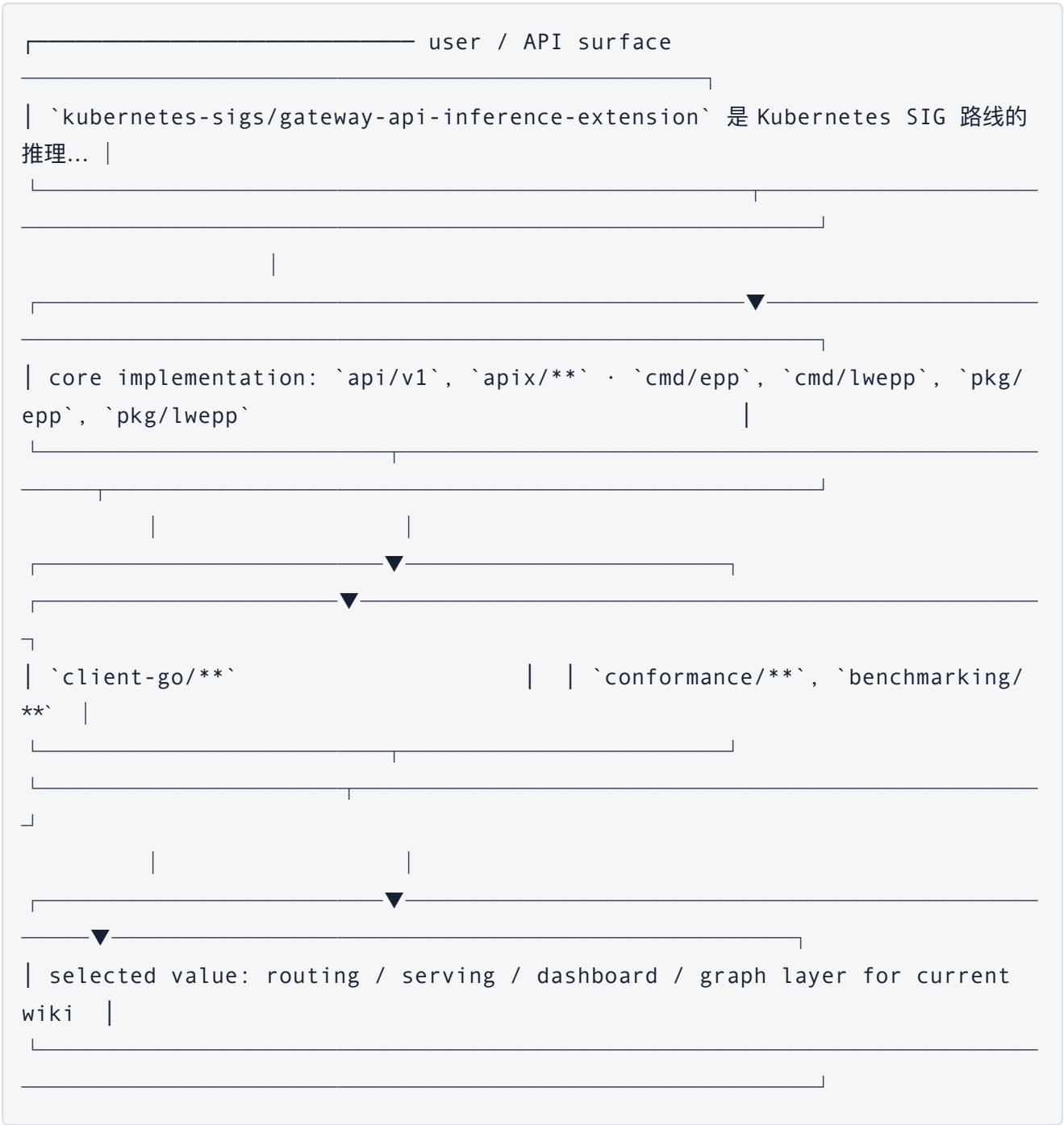
适合希望复用 Gateway API、把 endpoint picking 从具体 proxy 中解耦的 Kubernetes 平台。语义/成本/质量模型选择应放在它之前，由 semantic-router 或 routellm 完成。

详见 [src-gateway-api-inference-extension-architecture](#)、[src-k8s-gateway-routing-comparison](#)。

Gateway API Inference Extension 架构与设计思路分析

kubernetes-sigs/gateway-api-inference-extension 是 Kubernetes SIG 路线的推理流量标准化入口。它定义 InferencePool 等 API，提供 EPP/LWEPP、client-go、conformance、benchmarking 和 Gateway API 集成，是 llm-d-router 等项目对齐的标准层。

核心架构图



模块分层

层/目录	责任
api/v1, apix/**	InferencePool 和 EndpointPicker 相关 API。
cmd/epp, cmd/lwepp, pkg/epp, pkg/lwepp	Endpoint Picker 实现。
client-go/**	生成客户端。
conformance/**, benchmarking/**	一致性与性能验证。

关键数据流

1. HTTPRoute/Gateway 指向 InferencePool。
2. EPP/LWEPP 根据 endpoint 状态、端口和模型能力选择后端。
3. conformance/benchmark 确保实现符合 Gateway API 推理扩展语义。

设计决策

- 把推理路由变成 Gateway API extension，而不是每个 serving 系统自定义。
- 保留 EPP 参考实现，同时让 downstream 可替换。
- 最近支持 port-aware endpoint filtering，说明 data parallelism 已进入标准关注点。

对比定位

和 llm-d-router 相比，它是标准/API 和参考层；和 Envoy AI Gateway 相比，它更偏 endpoint picking，不管 provider auth/cost/prompt mutation。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

GitOps

以 Git 仓库为基础设施和应用配置的单一事实来源的运维方法论。

核心原则

1. 声明式 — 系统期望状态用声明式描述
2. 版本化 — 所有变更通过 Git 管理
3. 自动拉取 — Agent 持续将实际状态向期望状态调和
4. 持续调和 — 漂移自动检测和修复

工具生态

工具	定位
argocd	K8s 应用交付
Flux	K8s 应用交付 (CNCF)
k0rdent	多集群声明式管理
Tekton	CI 流水线

实践案例

- On-prem K3s 集群的 GitOps 部署 → [src-k3s-gitops-k0rdent](#)
- Argo CD 概览 → [src-argocd-overview](#)

GPU Sharing

GPU sharing 指多个 workload 共享同一物理 GPU 或 MIG/MPS/time-slicing/vGPU 资源的调度与隔离方法。目标是在吞吐、隔离、成本和利用率之间取平衡。

在 M5-C Device / GPU 地图中的位置

共享方案要分别回答四件事：scheduler 如何选择设备并记账，allocator 如何交接设备，硬件是否分区，runtime 如何限制实际使用。[node-feature-discovery](#) 提供能力信号；[k8s-device-plugin](#)、[dra-driver-nvidia-gpu](#) 与 [hami](#) 采用的资源模型和隔离机制不同。[cdi](#) 只承载注入配置，不把记账额度自动变成隔离保证。完整分层见 [k8s-gpu-device-stack](#)，Claim 与节点准备的区别见 [kubernetes-dra-design-deep-dive](#)。

部署时须为同一物理设备和资源名明确分配所有权，尤其不能让 HAMi 与 NVIDIA 官方 plugin 在同一节点争用 `nvidia.com/gpu`。[gpu-operator](#) 可以管理软件组件，但需关闭冲突的 `allocation operand`；HAMi-DRA 改变分配路径，不等于改变 HAMi-core 的软件隔离边界。

主要路线

路线	代表	特点
MIG	k8s-device-plugin , gpu-operator	硬件分区；plugin 可暴露预配置实例，分区配置由相应管理组件完成，粒度受硬件/profile 限制
Time-slicing	k8s-device-plugin	复用执行时间，不等于独立显存或硬件故障域隔离
MPS	k8s-device-plugin	通过 MPS 服务共享执行与配置资源限制；支持范围依版本，不等价于 MIG 硬件分区
HAMi-core memory/core sharing	hami	调度/记账配合容器内调用拦截，隔离效果取决于注入与兼容调用路径
DRA device configuration	dra-driver-nvidia-gpu	声明式 Claim 分配和设备配置；基础 API 稳定不意味着全部动态分区/共享扩展已稳定
Application-aware elastic KV	kvcached	只让接入的 vLLM/SGLang KV physical pages 按需占用 VRAM；不提供通用 CUDA 显存 quota、执行隔离或 K8s 调度

KVCacheD 在 GPU sharing 中的位置

kvcached 属于应用内、语义感知的内存复用：它知道哪些分配是KV block/token，因此可以在不停止模型的情况下按request生命周期map/unmap physical pages。MIG/MPS/time-slicing/HAMi则从设备或runtime层管理更广泛的GPU workload。

两者可以组合，但不能混为一谈：KVCacheD提高同卡多模型的KV利用率，外层方案仍需负责设备分配、记账、执行共享与租户隔离。详见 [elastic-kv-cache](#) 和 [kvcached-sglang-vllm-knowledge-system](#)。

和 GPU Operator 的关系

[gpu-operator](#) 管软件栈生命周期；GPU sharing 管 workload 如何共享设备。两者层级不同，但生产环境经常同时出现。

GPUStack

GPUStack 是 GPU cluster manager 与 Model-as-a-Service 平台，能够跨本地服务器、Kubernetes 和云环境管理 GPU，配置 vLLM、SGLang、TensorRT-LLM 或自定义 inference engine，并提供模型服务和 GPU 运维能力。

解决的问题

GPU 资源、模型部署、engine 参数、认证、监控和计量通常分散在不同系统。GPUStack 把 GPU 调度、模型服务、engine 选择、故障恢复、访问控制、token/API 计量和多集群可见性整合起来。

核心边界

GPUStack 更像一体化 GPU/MaaS 平台，而不是单纯 K8s CRD 库或专用 P/D router。它的价值在资源与服务整体运营；如果组织已有成熟 K8s/Gateway/Operator，只想补一个模型生命周期层，[kserve](#) 或 [ome](#) 可能更贴合。

适用场景

适合 GPU 云、私有 GPU 集群、跨环境 Model-as-a-Service、需要多种加速器和统一计量运维的团队。需要 Gateway API/InferencePool 标准化看 [llm-d](#)；需要深度 P/D/KV 编排看 [dynamo](#) 或 [kthena](#)。

详见 [src-gpustack-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

HAMi

Kubernetes 异构 GPU sharing / vGPU 项目，通过 mutating webhook、scheduler extender、device plugin 和多厂商设备后端实现 GPU memory/core/count 等细粒度共享。详见 [src-hami-architecture](#)。

证据说明：2026-09-27 官方默认分支快照为 [a2dd191b2e7f](#)，commit 不代表 release。[src-hami-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，Claim/Prepare 对照见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

经典路径由 scheduler/extender 计算设备 reservation，经 Pod annotations 交接给 device-plugin Allocate，再由 HAMi-core 实施容器侧 memory/core 隔离。[node-feature-discovery](#) 的能力标签不能替代共享设备记账；[gpu-operator](#) 可管软件组件，但须避免其 [k8s-device-plugin](#) operand 与 HAMi 在同一节点争用 `nvidia.com/gpu`。

HAMi-DRA 改变 Claim/设备分配路径，不自动改变 HAMi-core 的隔离能力；与 [dra-driver-nvidia-gpu](#) 对照时应分别核验设备支持、分配语义与 runtime 隔离。依据见 [HAMi Protocol](#)、[FAQ](#) 与 [安装路线对比](#)。

架构边界

HAMi 不是官方基础 device plugin，也不是 GPU 软件栈 operator。它把调度、quota、annotation、device plugin 和隔离机制结合起来，解决“一个物理 GPU 如何被多个 workload 细粒度共享”。

选型判断

- 暴露 NVIDIA GPU 给 Kubernetes：看 [k8s-device-plugin](#)。
- 管理 driver/runtime/device-plugin/DCGM：看 [gpu-operator](#)。
- 细粒度 vGPU / GPU sharing：看 HAMi。
- 走 ResourceClaim/ResourceSlice：对照 HAMi-DRA 与 [dra-driver-nvidia-gpu](#) 的设备支持和隔离需求。

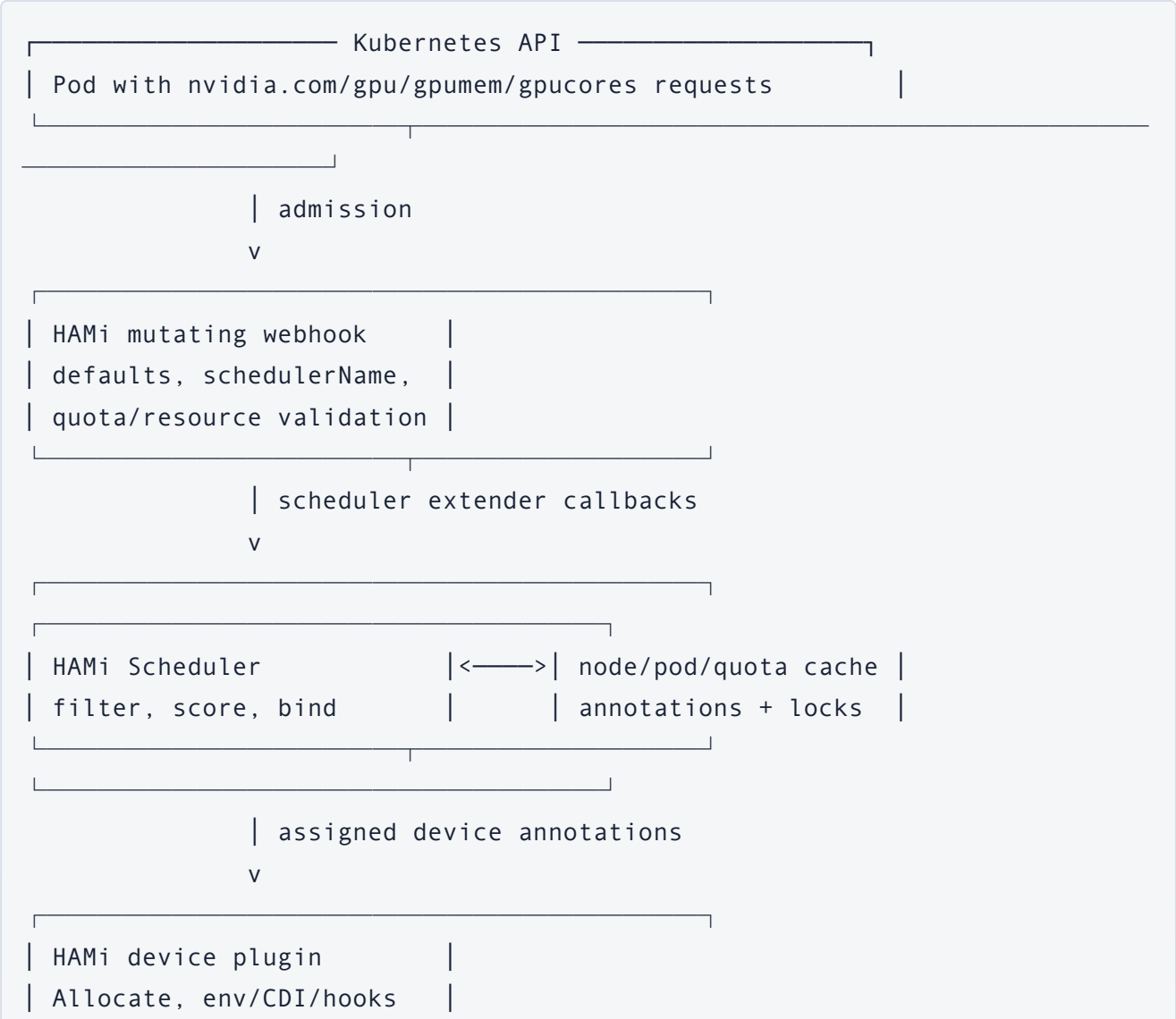
HAMi 架构与设计思路分析

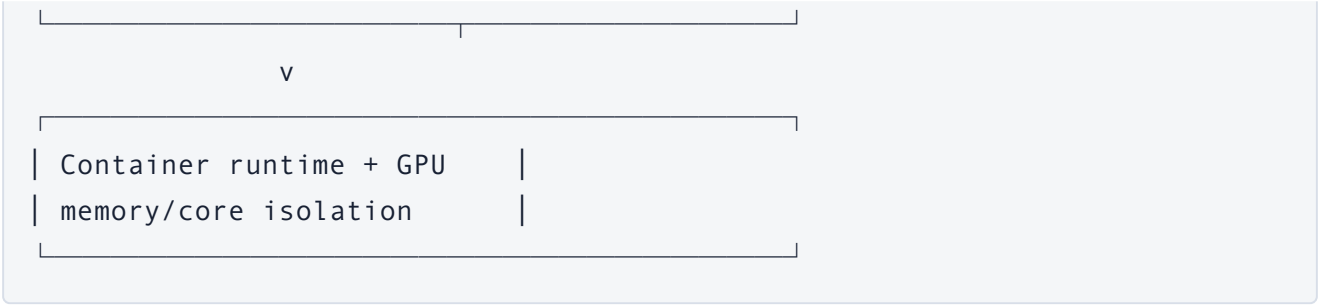
原文: raw/hami-architecture-analysis.md · 仓库: <https://github.com/Project-HAMi/HAMi>
HAMi · 分析版本 HEAD 5dca58e (2026-06-11)

一句话定位

Kubernetes 异构 GPU sharing / vGPU 项目，通过 mutating webhook、scheduler extender、device plugin 和设备厂商后端，把 GPU memory/core/count 等细粒度配额写入 Pod 注解并在 Allocate 阶段兑现。它更像“共享调度与隔离层”，不是 NVIDIA 官方基础 device plugin 的简单替代。本页和 [kubernetes](#) 的 [cdi gpu-sharing](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图

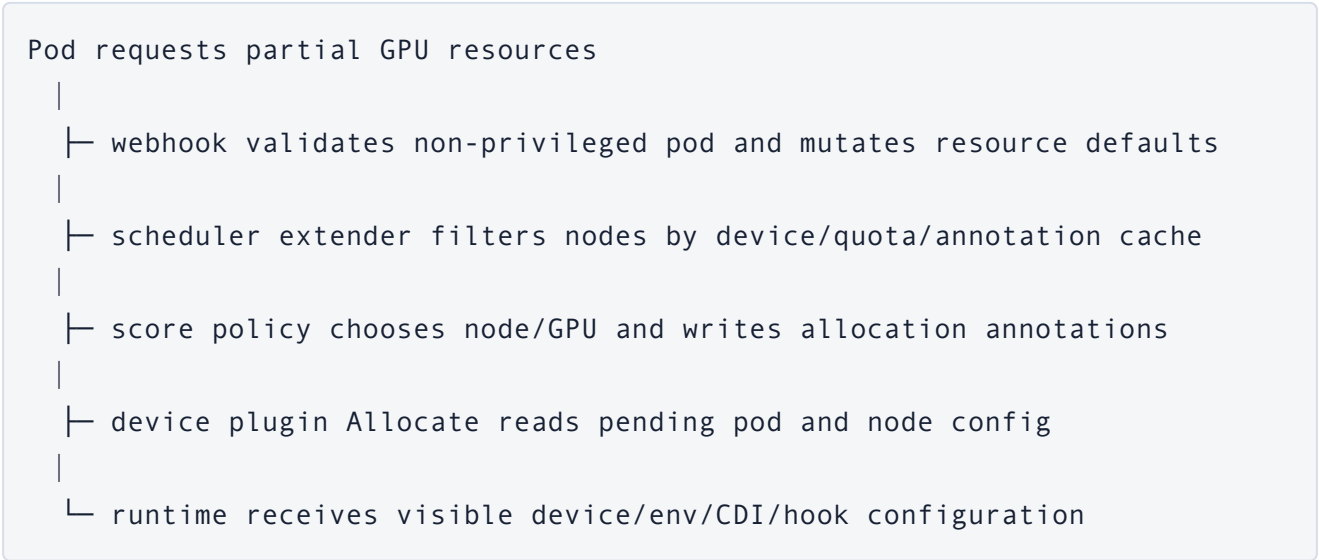




模块分层

层 / 模块	职责
Webhook	拦截 Pod admission，注入 schedulerName、默认 GPU memory/core/count、quota 检查。
Scheduler extender	维护 node/pod/quota cache，提供 /filter /bind /score 路由和 binpack/spread 策略。
设备抽象	多厂商设备注册、资源名、quota、annotation 编解码和打分逻辑。
Device plugin	对接 kubelet device plugin API，读取 node config，注册虚拟化资源，生成 env/CDI/挂载。
Monitor/metrics	反馈容器内 GPU 使用、Prometheus 指标和调度可观测性。

关键数据流



设计决策与哲学

- 使用 scheduler extender 而不是只靠 device plugin：GPU memory/core 共享需要调度时全局决策，单纯 kubelet Allocate 已经太晚。
- annotation 是调度到分配的契约：Scheduler 选择设备后通过 Pod/Node annotation 传递给 device plugin，降低对 kube-scheduler 内部扩展的侵入。
- 多厂商后端共享同一 device abstraction：pkg/device 提供 NVIDIA/AMD/Ascend/Cambricon 等扩展点，HAMi 定位异构设备共享而非单厂商工具。

与同类项目的架构差异

维度	HAMi	同类 A	同类 B
定位	vGPU/GPU sharing 调度与隔离	k8s-device-plugin: 官方 GPU 暴露	GPU Operator: 软件栈生命周期管理
调度参与	强，webhook + extender + bind	弱，主要 kubelet plugin	间接，部署组件
资源粒度	memory/core/count/厂商自定义	GPU/MIG/MPS/time-slicing	driver/runtime/plugin/DCGM 组件

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- [dra](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)

Headlamp

Headlamp 是可扩展 Kubernetes web UI，面向 dashboard、debugging、monitoring 和插件扩展。详见 [src-headlamp-architecture](#)。

架构边界

kubewall 是 single-binary dashboard；Headlamp 更强调插件化和通用 Kubernetes UI。

什么时候用

场景	判断
需要 Kubernetes UI 能力	适合，Headlamp 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , kubewall 组合。

核心组件

- Frontend: React/TypeScript UI
- Backend/proxy: Kubernetes API access
- Plugin system: UI and cluster extensions
- Auth/context: kubeconfig, in-cluster, OIDC-like deployments

选型提示

把 Headlamp 放在 Kubernetes UI 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

Headlamp 架构与设计思路分析

原文: raw/headlamp-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/headlamp> · 优先级 P1

一句话定位

Headlamp 是可扩展 Kubernetes web UI，面向 dashboard、debugging、monitoring 和插件扩展。

核心架构图

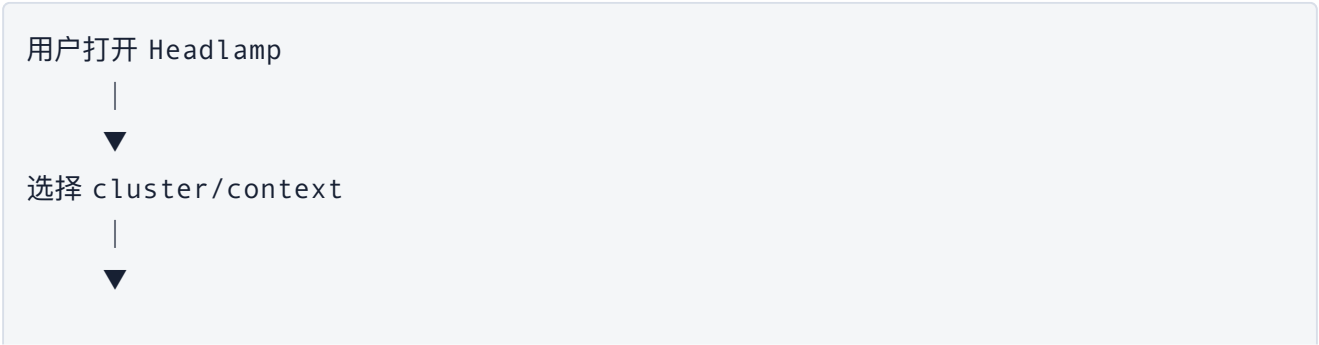




模块分层

层 / 模块	职责
Frontend	React/TypeScript UI
Backend/proxy	Kubernetes API access
Plugin system	UI and cluster extensions
Auth/context	kubeconfig, in-cluster, OIDC-like deployments

关键数据流



后端代理 Kubernetes API



前端展示 workloads/events/logs/resources



插件扩展额外视图或动作

设计决策与哲学

- 补齐 Kubernetes UI 维度：Headlamp 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：kubewall 是 single-binary dashboard；Headlamp 更强调插件化和通用 Kubernetes UI。
- 选型价值：它应和 [kubernetes](#), [kubewall](#) 一起看，而不是孤立评估。

相关页面

- [headlamp](#)
- [kubernetes](#)
- [kubewall](#)

Higress

Higress 是阿里系 AI Native API Gateway，基于 Envoy/Istio 控制面和多语言 WASM plugins。它在当前 wiki 中主要用于理解 AI gateway、模型路由、MCP/HTTP 策略和凭据治理，尤其是 HiClaw 背景中的网关能力。详见 [src-higress-architecture](#)。

架构边界

Higress 是通用 API Gateway 产品向 AI Gateway 扩展，不是只服务 LLM 的窄代理。它的优势来自成熟 API gateway 能力、服务发现、插件生态和 WASM 扩展。

关键设计

- cmd/higress、pkg/** 承载控制面和网关逻辑。
- plugins/** 提供 Go/Rust/C++/AssemblyScript 等多语言 WASM 插件。
- Envoy/Istio 基础设施负责通用 L7 流量治理。
- model-router、credential governance、MCP/LLM 插件把网关扩展到 AI 工作负载。

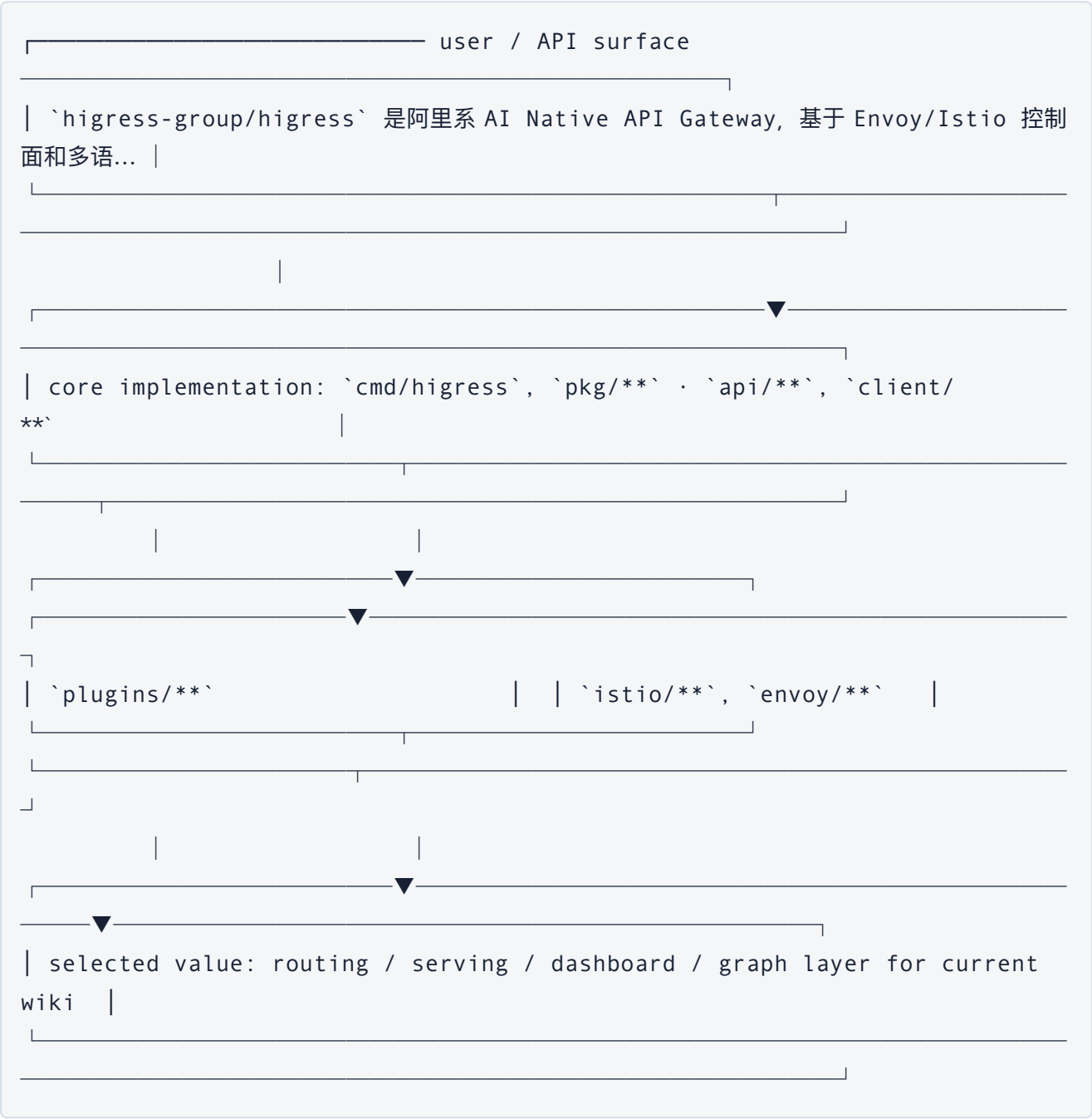
选型判断

需要传统 API gateway 和 AI gateway 合一时看 Higress；需要 GenAI 专用 Envoy Gateway 方案看 Envoy AI Gateway；需要 Kubernetes Gateway API 原生通用网关看 [kgateway](#)；需要 agent-side LLM/MCP/A2A 出口治理看 [agentgateway](#)。

Higress 架构与设计思路分析

higress-group/higress 是阿里系 AI Native API Gateway，基于 Envoy/Istio 控制面和多语言 WASM plugins。P1 中它的价值是 AI gateway/模型路由/凭据治理背景，尤其和 HiClaw 的凭据托管、MCP/LLM 网关能力相关。

核心架构图



模块分层

层/目录	责任
cmd/higress, pkg/**	Higress 控制面与网关逻辑。
api/**, client/**	API 与客户端。
plugins/**	WASM 插件：Go/Rust/C++/AssemblyScript。
istio/**, envoy/**	上游控制面/数据面依赖。

关键数据流

1. 用户配置 route/model-router/plugin。
2. 控制面生成 Envoy/Istio 配置并下发。
3. WASM 插件在数据面处理 AI auth、model route、MCP/HTTP 策略。

设计决策

- 插件化和服务发现能力强，适合传统 API gateway + AI gateway 融合。
- model-router 保留原始模型名选项说明多 provider/model 映射是活跃问题。
- 仓库较大，应聚焦 plugin/model-router/control plane，不必全读 Istio vendor。

对比定位

和 kgateway 相比，Higress 更偏 API gateway 产品和插件生态；和 Envoy AI Gateway 相比，范围更宽但 GenAI 专注度较低。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

inference-perf

GenAI inference performance benchmarking tool，用于对 OpenAI-compatible/serving endpoint 做负载、延迟和吞吐测量。详见 [src-inference-perf-architecture](#)。

架构边界

和 llm-d-benchmark 相比，inference-perf 更像单个 benchmark harness；llm-d-benchmark 负责更完整的实验生命周期。

什么时候用

场景	判断
需要 GenAI benchmark 能力	适合，inference-perf 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-d-benchmark, llm-inference, llm-d-inference-sim 组合。

核心组件

- CLI/config: benchmark 参数与 endpoint 配置
- Load generator: 并发、请求分布、payload 模板
- Metrics collector: latency/throughput/error/token stats
- Reports: 结果输出供 llm-d-benchmark / serving 选型使用

选型提示

把 inference-perf 放在 GenAI benchmark 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

inference-perf 架构与设计思路分析

原文: raw/inference-perf-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/inference-perf> · 优先级 P1

一句话定位

GenAI inference performance benchmarking tool，用于对 OpenAI-compatible/serving endpoint 做负载、延迟和吞吐测量。

核心架构图

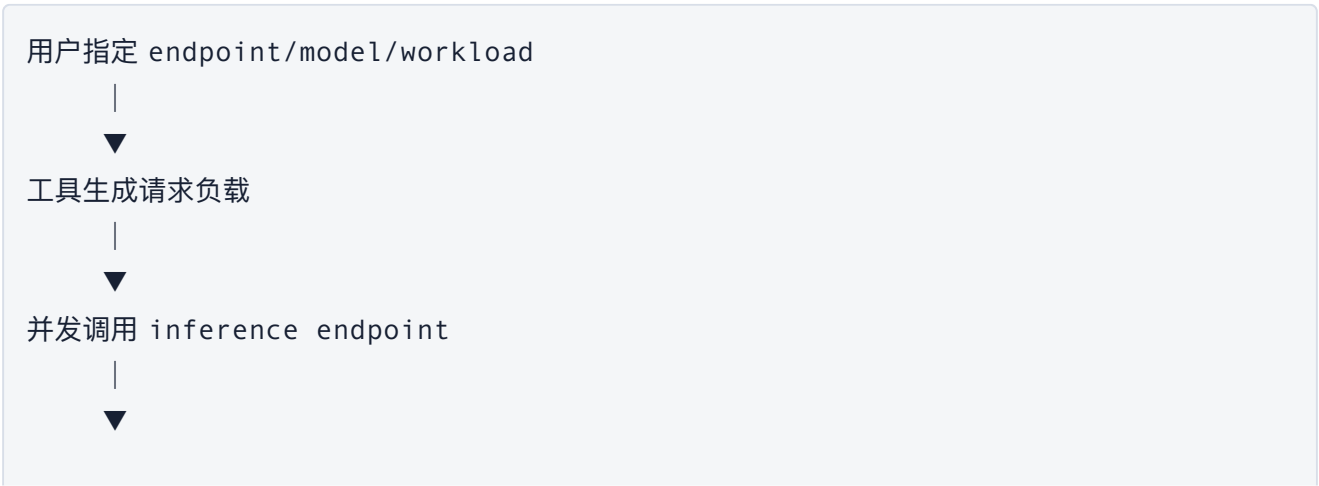




模块分层

层 / 模块	职责
CLI/config	benchmark 参数与 endpoint 配置
Load generator	并发、请求分布、payload 模板
Metrics collector	latency/throughput/error/token stats
Reports	结果输出供 llm-d-benchmark / serving 选型使用

关键数据流



收集 TTFT/ITL/latency/throughput

|



输出 benchmark report

设计决策与哲学

- 补齐 GenAI benchmark 维度: inference-perf 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: 和 llm-d-benchmark 相比，inference-perf 更像单个 benchmark harness; llm-d-benchmark 负责更完整的实验生命周期。
- 选型价值: 它应和 llm-d-benchmark, llm-inference, llm-d-inference-sim 一起看，而不是孤立评估。

相关页面

- [inference-perf](#)
- [kubernetes](#)
- llm-d-benchmark
- llm-inference
- llm-d-inference-sim

ingress2gateway

ingress2gateway 把 Kubernetes Ingress resources 转换成 Gateway API resources，帮助从 annotation-heavy Ingress 迁移到 Gateway/HTTPRoute。详见 [src-ingress2gateway-architecture](#)。

架构边界

它不负责流量转发，只负责迁移配置模型。

什么时候用

场景	判断
需要 Ingress -> Gateway API migration 能力	适合，ingress2gateway 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 gateway-api , inference-routing 组合。

核心组件

- Parser: 读取 Ingress/Service annotations
- Provider translators: nginx/contour/gce 等差异
- Gateway API renderer: Gateway/HTTPRoute/TLSRoute
- CLI/report: 迁移建议和限制

选型提示

把 ingress2gateway 放在 Ingress -> Gateway API migration 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

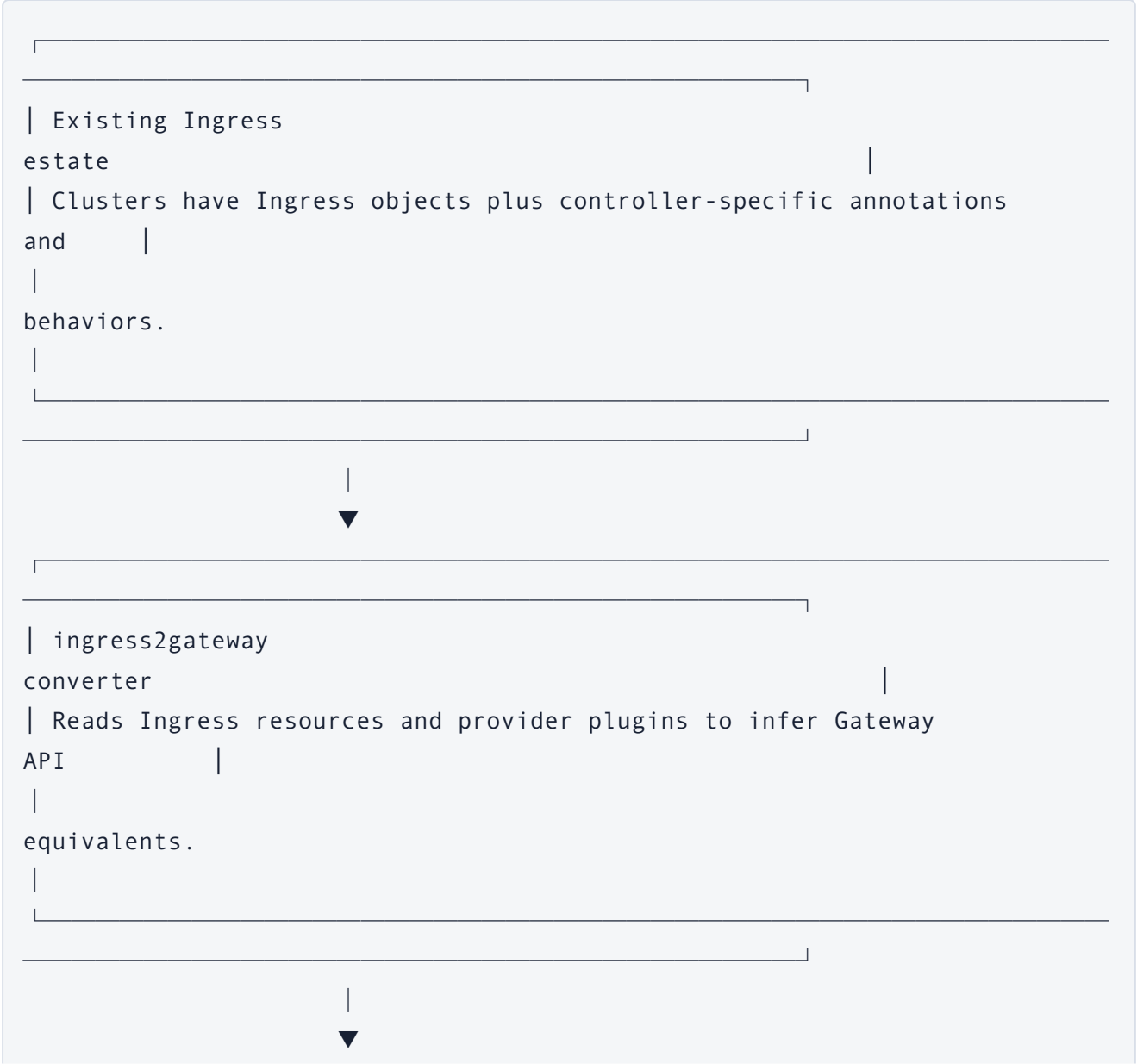
ingress2gateway 架构与设计思路分析

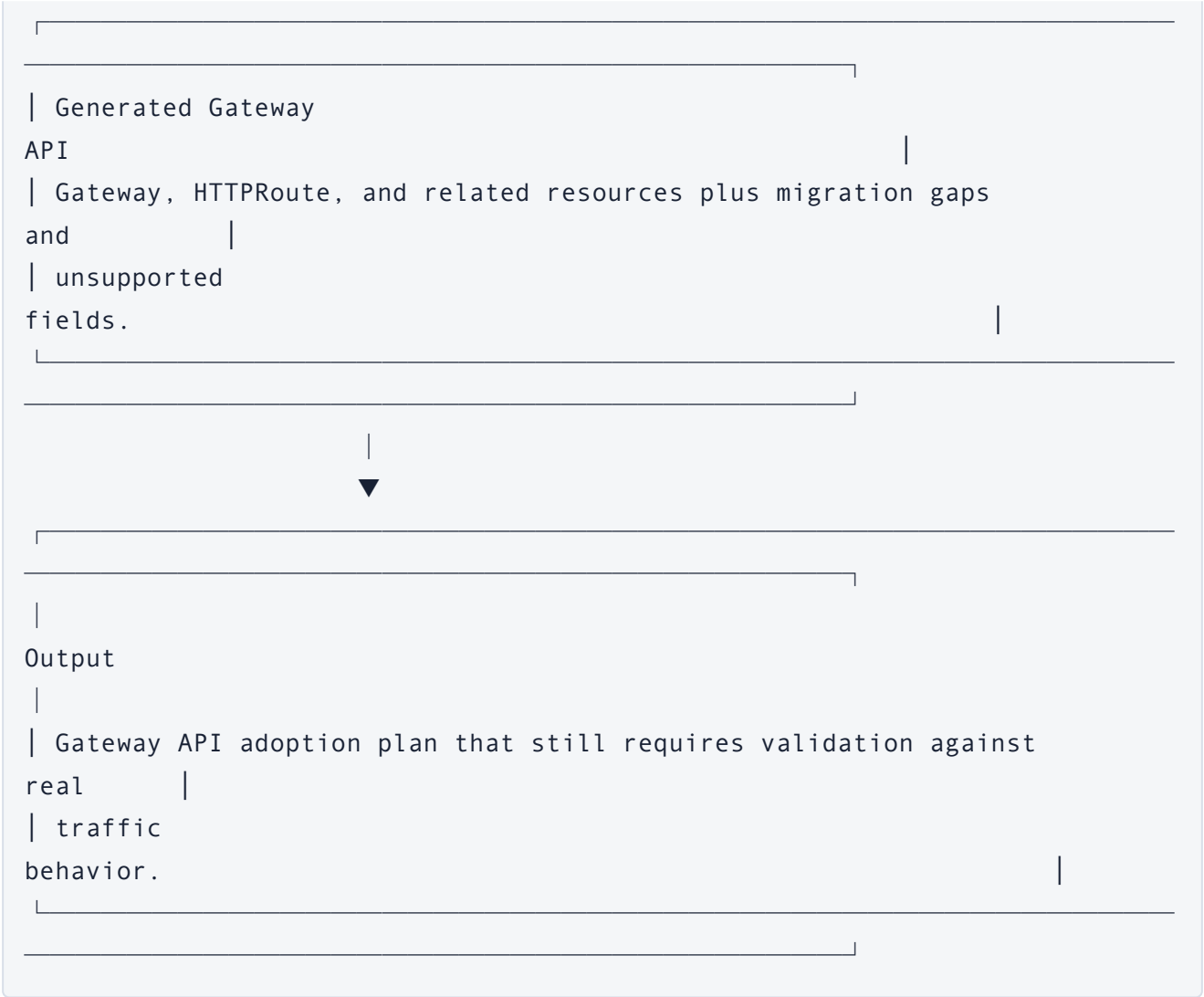
原文: raw/ingress2gateway-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/ingress2gateway> · 优先级 P1

一句话定位

ingress2gateway 把 Kubernetes Ingress resources 转换成 Gateway API resources，帮助从 annotation-heavy Ingress 迁移到 Gateway/HTTPRoute。

核心架构图

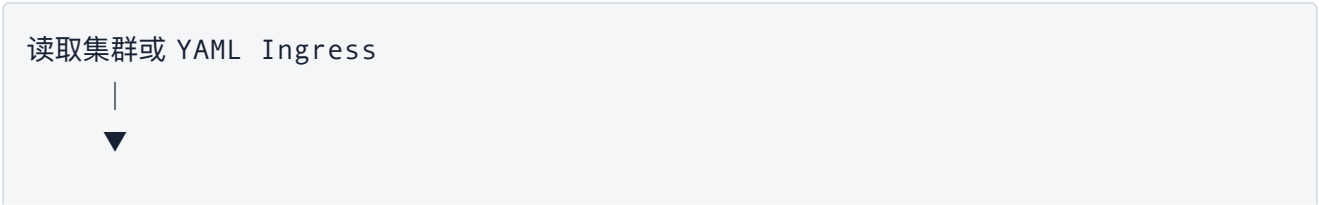




模块分层

层 / 模块	职责
Parser	读取 Ingress/Service annotations
Provider translators	nginx/contour/gce 等差异
Gateway API renderer	Gateway/HTTPRoute/TLSRoute
CLI/report	迁移建议和限制

关键数据流



识别规则和 provider annotations



转换成 Gateway API resources



输出 YAML 和 warnings



用户审查后应用

设计决策与哲学

- 补齐 Ingress -> Gateway API migration 维度: ingress2gateway 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: 它不负责流量转发, 只负责迁移配置模型。
- 选型价值: 它应和 [gateway-api](#), inference-routing 一起看, 而不是孤立评估。

相关页面

- [ingress2gateway](#)
- [kubernetes](#)
- [gateway-api](#)
- inference-routing

JobSet

JobSet 是 K8s native API for distributed ML training and HPC workloads，用多个 replicated jobs 表达一个整体作业。详见 [src-jobset-architecture](#)。

证据说明：2026-09-25 核验的官方默认分支快照为 [03f9dccef945](#)，该 commit 不代表 release。[src-jobset-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目职责见 [kubernetes-workload-gang-scheduling-design](#)，调度器流程见 [kubernetes-scheduler-core-design](#)。

在 M5-B Workload / Scheduling 地图中的位置

在上述 `jobset.x-k8s.io/v1alpha2` 快照中，JobSet 用 ReplicatedJobs 管理子 Job、依赖关系、成功/失败策略、重启和共享卷的创建/保留生命周期。它表达作业集合，与 [lws](#) 的持续服务组形成对照；quota admission 交给 [kueue](#)，最终 Pod placement 交给 kube-scheduler 及配置的 [scheduler-plugins](#)，这些职责不由 JobSet 接管。

架构边界

Kueue 负责 queue/admission；JobSet 负责表达分布式作业拓扑，二者常一起出现。

什么时候用

场景	判断
需要 分布式 workload API 能力	适合，JobSet 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-inference, batch-inference, kueue 组合。

核心组件

- API: JobSet / replicatedJobs
- Controller: child Job lifecycle and status aggregation
- Failure policy: restart / recreate / fail fast
- Integrations: Kueue, batch, ML training

选型提示

把 JobSet 放在 分布式 workload API 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

JobSet 架构与设计思路分析

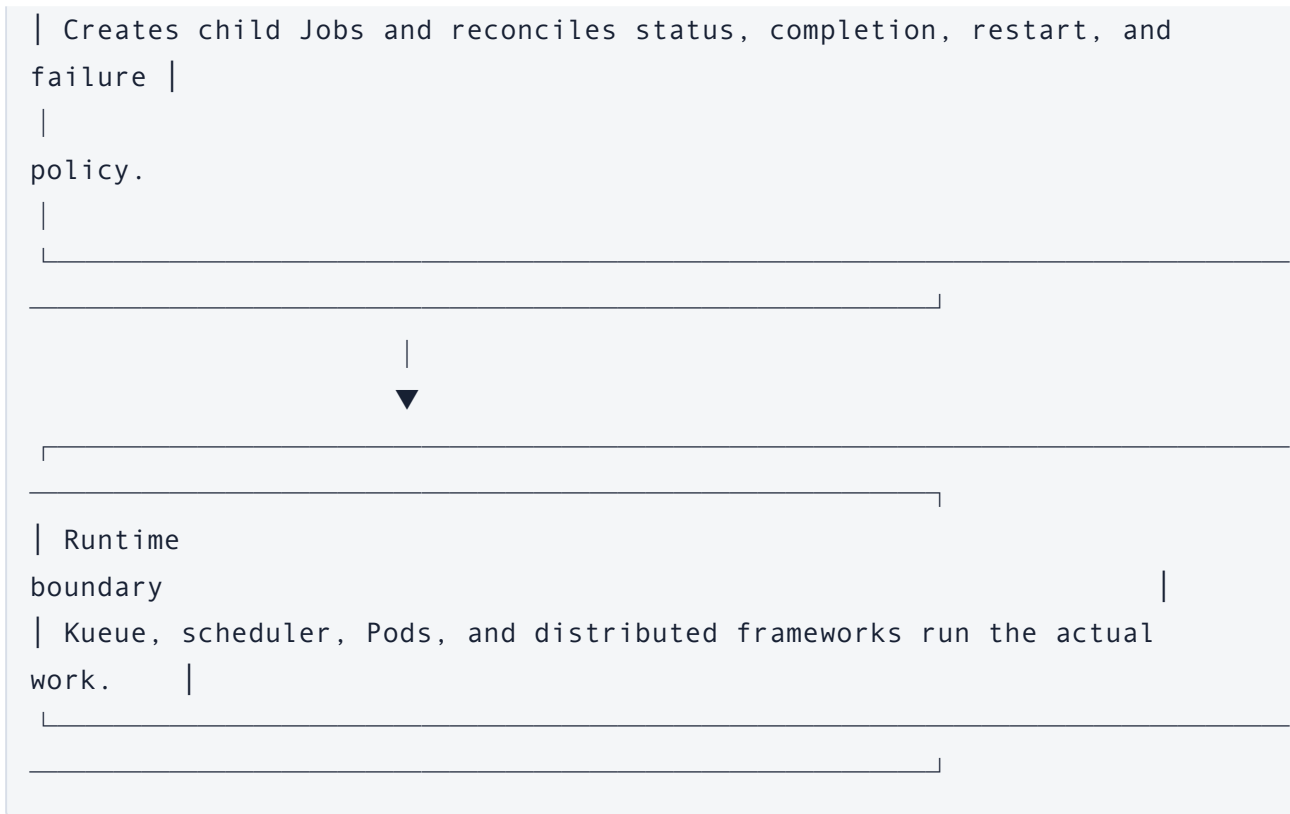
原文: raw/jobset-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/jobset> · 优先级 P1

一句话定位

JobSet 是 K8s native API for distributed ML training and HPC workloads, 用多个 replicated jobs 表达一个整体作业。

核心架构图

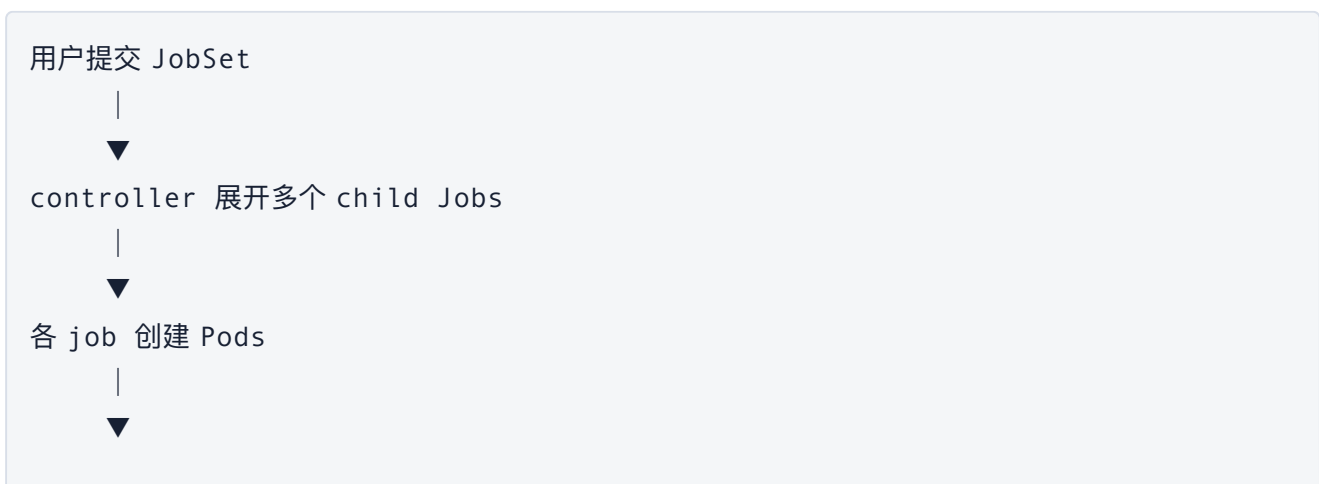




模块分层

层 / 模块	职责
API	JobSet / replicatedJobs
Controller	child Job lifecycle and status aggregation
Failure policy	restart / recreate / fail fast
Integrations	Kueue, batch, ML training

关键数据流



聚合成功/失败状态

|



按 failure policy 处理重试或终止

设计决策与哲学

- 补齐 分布式 workload API 维度: JobSet 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: Kueue 负责 queue/admission; JobSet 负责表达分布式作业拓扑, 二者常一起出现。
- 选型价值: 它应和 llm-inference, batch-inference, [kueue](#) 一起看, 而不是孤立评估。

相关页面

- [jobset](#)
- [kubernetes](#)
- llm-inference
- batch-inference
- [kueue](#)

K3s + k0rdent GitOps 部署 On-Prem 集群

摘要

使用 K3s + k0rdent 声明式模板 + Proxmox 虚拟化，将 on-prem K8s 部署变为完全声明式、GitOps 兼容的流程。

架构

k0rdent 管理层 → Proxmox BYOT 基础设施 → Control Plane Provider → K3s Bootstrap → 运行集群

关键设计

- BYOT (Bring Your Own Template) : 复用已有 VM 模板 + cloud-init, 跳过镜像构建
- 持续调和: k0rdent 持续监控并自动纠正配置漂移
- Helm chart 管理: VM 克隆、资源分配、SSH 密钥注入全部声明式

k8m

k8m 是轻量 Kubernetes AI dashboard，把集群浏览、操作权限、异常检测和 AI/MCP 能力放进一个小型控制台。详见 [src-k8m-architecture](#)。

架构边界

k8m 是 dashboard-first，不是 CLI assistant，也不是通用 Agent framework。它更适合把 AI 辅助嵌到集群管理 UI 里，而不是让 agent 直接接管复杂 workflow。

关键设计

- Go backend 的 `internal/dao`、`pkg/service`、`pkg/controller` 和 `pkg/plugins` 负责资源访问与插件能力。
- `ui/**` 提供管理控制台。
- AI/MCP 能力通过插件进入资源解释、排障和操作辅助。
- 项目价值主要在产品形态参考：小型控制台如何接入 AI，而不是 agent loop 本身。

选型判断

需要 UI-first K8s AI 管理入口时看 k8m。需要单二进制 dashboard 对照看 [kubewall](#)；需要 kubectl 工作流看 [kubectl-ai](#)；需要 agentic K8s 操作层看 [kagent](#)。

k8m 架构与设计思路分析

weibaohui/k8m 是轻量 K8s AI dashboard。目录显示 Go backend (internal, pkg/controller/service/plugins) + 前端 UI + deploy manifests + MCP/自托管 AI 文档，定位是把集群浏览、操作权限、异常检测和 AI/MCP 能力压进一个小型控制台。

核心架构图



模块分层

层/目录	责任
internal/dao, pkg/service	后端数据访问和业务服务。
pkg/controller, pkg/plugins	K8s 控制/插件扩展。
pkg/middleware, pkg/models	HTTP 中间件和模型定义。
ui/**	前端控制台。

关键数据流

1. 用户通过 UI 选择集群/资源。
2. 后端调用 K8s API 和 plugins 获取资源状态。
3. AI/MCP 功能用于解释、排障或辅助生成操作。

设计决策

- Dashboard-first，比 CLI assistant 更适合非命令行用户。
- 插件/MCP 让 AI 能力可插拔，而不是写死在 controller 中。
- 项目活跃度比 P0 项目弱，适合作为形态参考。

对比定位

和 kubewall 相比，k8m 更强调 AI/MCP；和 kagent 相比，它是管理界面，不是 agent workflow 平台。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

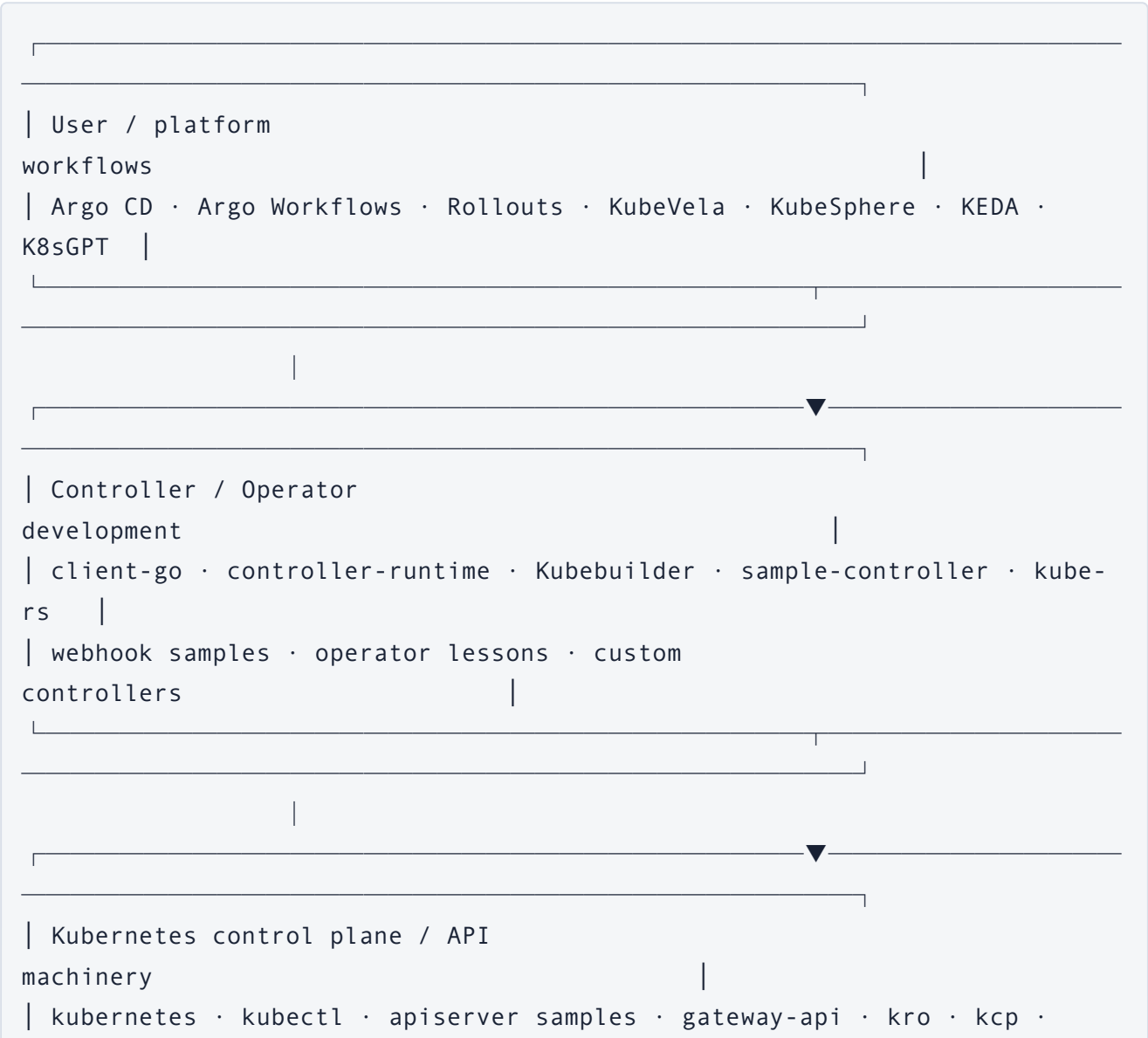
K8s Core & Controllers Star 项目清单整理

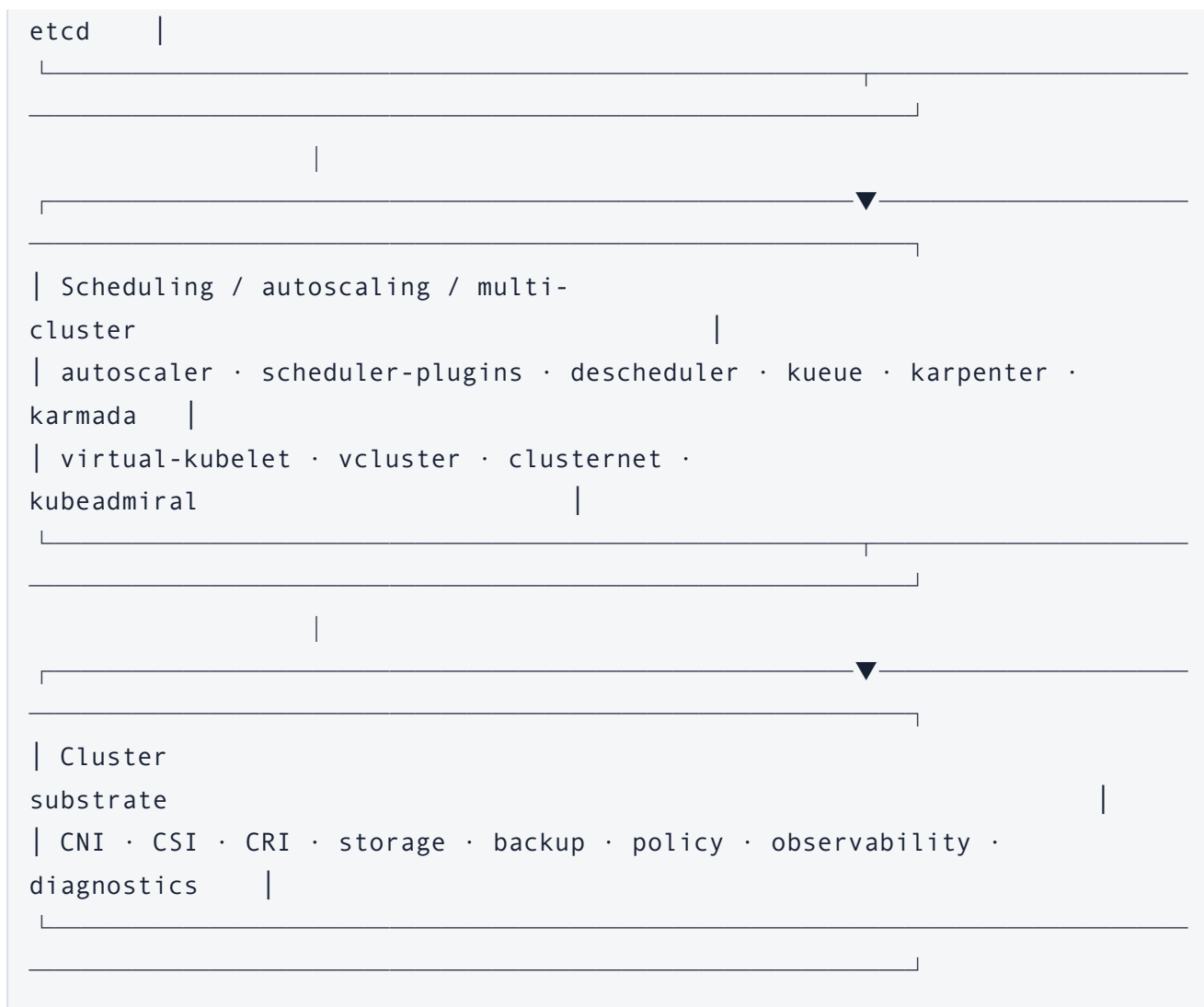
原文: raw/k8s-core-controllers-stars-list.md · 来源: [googs1025 的 K8s Core & Controllers Stars list](#) · 抓取日期: 2026-06-04 · 仓库数: 359

一句话定位

这个 Star list 是一张 [kubernetes](#) 平台工程底座地图：主线 Kubernetes、kubectl/client-go/controller-runtime/kubebuilder/operator SDK、CRD/webhook/sample-apiserver、调度/autoscaling/multicluster、网络/存储/备份、安全/策略、可观测/诊断，以及大量中文 controller/operator 学习实践项目都在同一张图里。

分层地图





观察

- 这个 list 的核心不是“控制器项目很多”，而是完整呈现了 Kubernetes 平台工程的学习路径：先看主线/kubectl/client-go，再看 controller-runtime/kubebuilder/sample-controller，之后扩展到调度、网络、存储、安全、可观测和多集群。
- Controller 开发有明显的分层：client-go 是 API client 基座，controller-runtime 抽象 informer/cache/reconcile，kubebuilder 提供脚手架和 CRD 生成，sample-controller/sample-apiserver 是理解底层机制的参照。
- 真实生产控制器已经远超 Operator CRUD：KEDA、autoscaler、descheduler、kueue、karpenter、Grove、Armada 都是“控制循环 + 调度/队列/资源经济”的组合。
- 多集群和虚拟集群是另一条主线：Karmada、vCluster、Virtual Kubelet、KubeEdge/OpenYurt、kcp/Clusternet/KubeAdmiral 代表了不同的 control plane 复用方式。
- 可观测和 AI Ops 已进入 Kubernetes controller 生态：metrics-server/kube-state-metrics/prometheus-operator 是传统指标层，k8sgpt/kubectl-ai/krr/kubewizard/kube-agent-helper 则把诊断和建议推向 AI 辅助。

优先深挖候选

优先级	项目	原因
1	kubernetes-sigs/controller-runtime	Operator/controller 开发的核心抽象层。
2	kubernetes-sigs/kubebuilder	CRD/API 生成和 controller 项目结构的标准样板。
3	kubernetes/sample-controller	理解 informer/workqueue/reconcile 的最小官方参照。
4	kubernetes-sigs/kueue	AI/Batch workload queueing 与调度控制面。
5	kubernetes/autoscaler	HPA/VPA/cluster-autoscaler 等弹性控制器集合。
6	loft-sh/vcluster	虚拟集群是平台多租户的重要路线。
7	karmada-io/karmada	多云多集群编排代表项目。
8	gateway-api	新一代 Kubernetes 网络 API，和 agentgateway/AI gateway 相关。
9	prometheus-operator/prometheus-operator	Operator 模式的经典生产级案例。
10	k8sgpt-ai/k8sgpt	AI 辅助 Kubernetes 诊断代表项目。

相关页面

- [kubernetes](#)
- [gateway-api](#)
- [gitops](#)
- [ai-ops](#)
- declarative-agent-management
- agent-sandbox
- [agentcube](#)

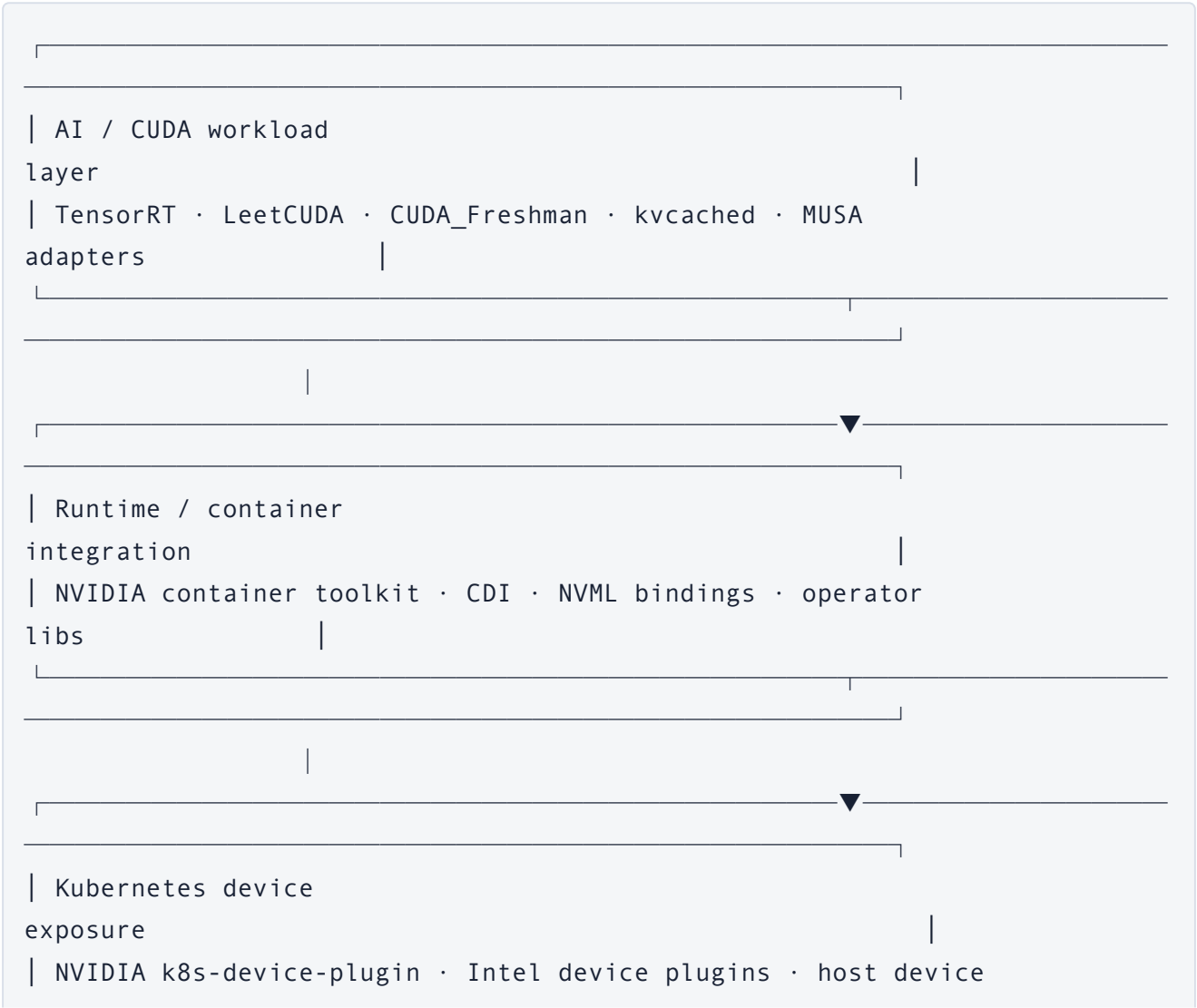
K8s GPU & Device Plugins Star 项目清单整理

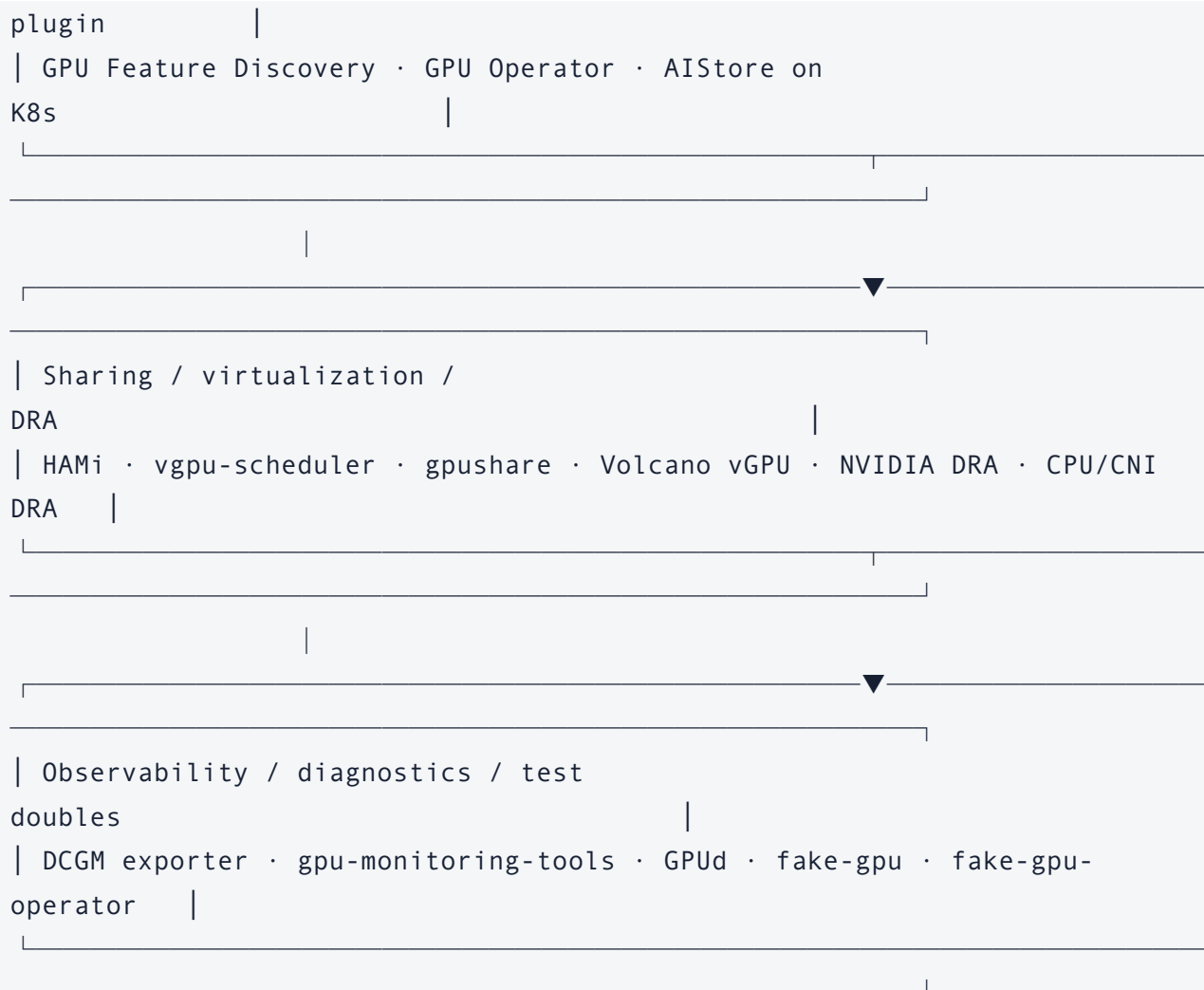
原文: raw/k8s-gpu-device-plugins-stars-list.md · 来源: [googs1025 的 K8s GPU & Device Plugins Stars list](#) · 抓取日期: 2026-06-04 · 仓库数: 36

一句话定位

这个 Star list 聚焦 [kubernetes](#) 异构设备资源层: 从最传统的 device plugin、GPU Operator、container toolkit、DCGM/NVML 监控, 到 vGPU / GPU sharing、Dynamic Resource Allocation、CDI、fake GPU 测试环境, 再延伸到 CUDA 学习、TensorRT、KV cache 虚拟化这类 GPU workload 能力。

分层地图





观察

- Kubernetes GPU 资源层已经从“device plugin 把卡暴露出来”扩展成完整栈：driver/runtime/operator、feature discovery、metrics、diagnostics、admission、scheduler、sharing 和 DRA 都是独立问题。
- vGPU/GPU sharing 仍有多条历史路线并存：gpushare、TKE gpu-manager、4paradigm vgpu-scheduler、HAMi、Volcano vGPU；其中 HAMi 是当前更值得深挖的社区化代表。
- DRA 是下一代设备资源抽象的主线：NVIDIA GPU DRA、CPU DRA、CNI DRA、DRA example driver 说明 Kubernetes 正在把“设备分配”从 device plugin 扩展成更声明式、更可组合的资源 API。
- GPU 可观测不只是 metrics：DCGM exporter/NVML 是数据源，GPUd/fake-gpu/fake-gpu-operator/knavigator 面向诊断、测试和调度系统开发。
- llm-inference 把 GPU 资源层进一步复杂化：kvcached 把 KV cache 也推向可虚拟化资源，和 GPU sharing、kv-cache-offload、DRA 之间有明显交叉。

优先深挖候选

优先级	项目	原因
1	NVIDIA/k8s-device-plugin	Kubernetes GPU device plugin 标准参照。
2	NVIDIA/gpu-operator	生产 GPU 集群运维主入口。
3	Project-HAMi/HAMi	当前 GPU sharing / vGPU 方向最值得系统分析的项目。
4	kubernetes-sigs/dra-driver-nvidia-gpu	Kubernetes DRA + GPU 的下一代资源 API 样板。
5	cncf-tags/container-device-interface	CDI 是 container runtime 侧设备声明标准，和 DRA/device plugin 相邻。
6	NVIDIA/dcgm-exporter	GPU metrics 和 Prometheus 集成标配。
7	leptonai/gpud	GPU 诊断从 metrics 走向自动化 health/issue detection。
8	ovg-project/kvcached	KV cache 资源化和 GPU sharing/LLM serving 的交叉点。

相关页面

- [kubernetes](#)
- llm-inference
- kv-cache-offload
- vllm
- sglang
- [dynamo](#)
- disaggregated-serving

kagent

Cloud Native agentic AI 项目，把 Kubernetes / DevOps 操作、工具、Helm/UI 和 MCP 能力包装成 agentic workflow。详见 [src-kagent-architecture](#)。

架构边界

kagent 不应和通用 Agent framework 混放。它的主场是 [ai-ops](#)：围绕 K8s/DevOps 操作场景做 agentic automation，而不是提供通用个人 coding agent 执行 loop。

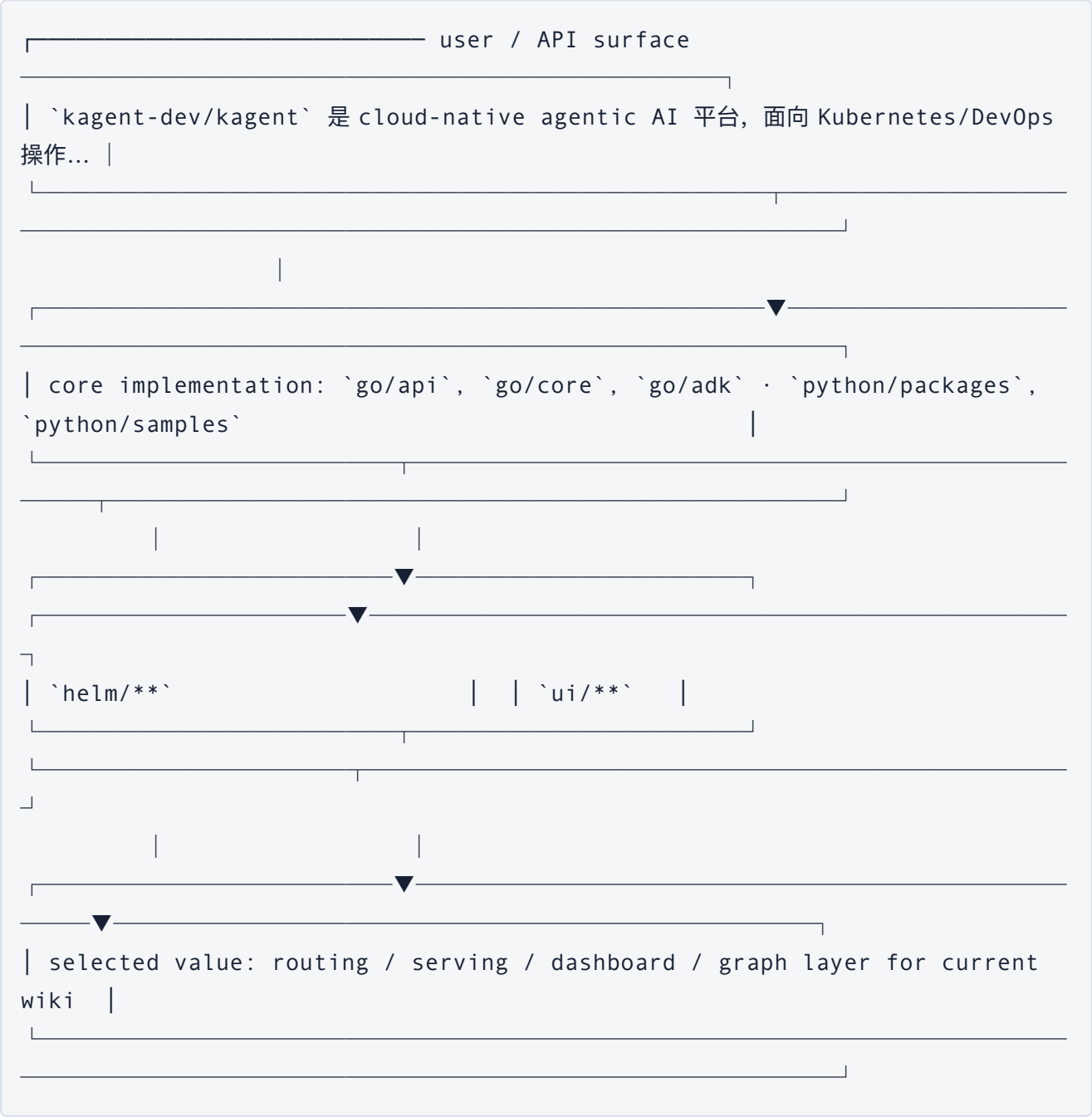
选型判断

- 想在 kubectl 里问答/执行：看 kubectl-ai。
- 想在 dashboard 管理 K8s：看 k8m / kubewall。
- 想把 K8s/DevOps 操作组织成 agentic workflow：看 kagent。

kagent 架构与设计思路分析

kagent-dev/kagent 是 cloud-native agentic AI 平台，面向 Kubernetes/DevOps 操作而不是通用 chat agent。仓库由 Go control plane、Python package/samples、Helm charts、UI、内置 tools/skills 组成，适合补 [ai-ops](#) 中“agentic workflow + MCP + K8s runtime”的路线。

核心架构图



模块分层

层/目录	责任
go/api, go/core, go/adk	Go 侧 API、核心对象和 ADK 集成。
python/packages, python/samples	Python agent/tool package 和示例。
helm/**	kagent、agents、tools 的安装分发。
ui/**	管理界面。

关键数据流

1. 用户从 UI/CLI/API 创建或调用 agent。
2. agent 通过工具/MCP/K8s API 执行 DevOps 操作。
3. Go control plane 负责状态、配置、流式事件和部署对象，Helm 安装到集群。

设计决策

- 按 cloud-native control plane 组织，而不是单机 CLI。
- 把 tools/skills 作为可扩展资产，适合和 MCP 网关路线结合。
- 最近 Bedrock streaming 修复说明其模型 provider 事件流是关键路径。

对比定位

和 kubectl-ai 相比，kagent 更像平台；和 k8m/kubewall 相比，它不是 dashboard，而是 agentic workflow 运行层。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Karpenter

Kubernetes node autoscaler，用 NodePool/NodeClaim/CloudProvider 把 pending pods 转换成最合适的节点容量，并做 consolidation 降本。详见 [src-karpenter-architecture](#)。

证据说明：2026-09-25 核验的官方默认分支快照为 [06bc3b4b94dd](#)，该 commit 不代表 release。[src-karpenter-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目职责见 [kubernetes-workload-gang-scheduling-design](#)，调度器流程见 [kubernetes-scheduler-core-design](#)。

在 M5-B Workload / Scheduling 地图中的位置

在上述快照中，Karpenter 将不可调度 Pod 的需求与 `karpenter.sh/v1` NodePool、provider-specific NodeClass 约束组合为 NodeClaim，推进节点启动、注册、初始化及后续 disruption/termination。它提供容量，不决定 [kueue](#) 的 quota admission；最终 Pod placement/binding 仍由 kube-scheduler 及配置的 [scheduler-plugins](#) 完成。节点回收的 drift、consolidation、expiration 具有不同约束，不能统一理解为自动无损替换。

架构边界

和 HPA/KEDA 不同，Karpenter 扩的是节点容量；和 Cluster Autoscaler 相比，它更强调按 pending pods 即时求解容量。

什么时候用

场景	判断
需要 节点弹性 / 成本 能力	适合，Karpenter 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-inference, kubernetes , model-serving-operator 组合。

核心组件

- API: NodePool / NodeClaim / EC2NodeClass-like provider API
- Controller: provisioning、disruption、consolidation、termination
- Scheduler simulation: 从 pending pods 推导 instance requirements
- CloudProvider boundary: 云厂商容量、价格、可用区和实例类型

选型提示

把 Karpenter 放在 节点弹性 / 成本 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

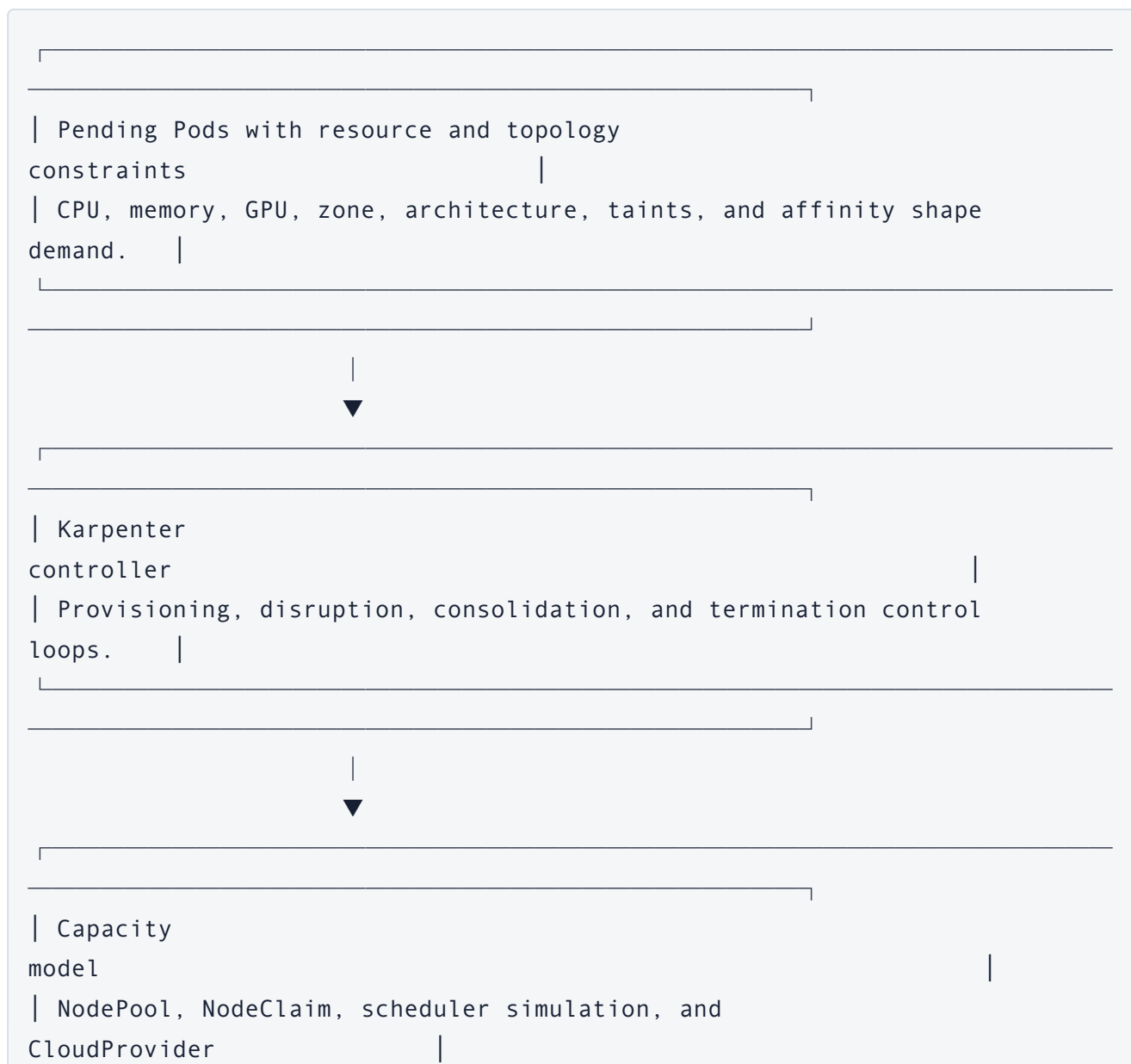
Karpenter 架构与设计思路分析

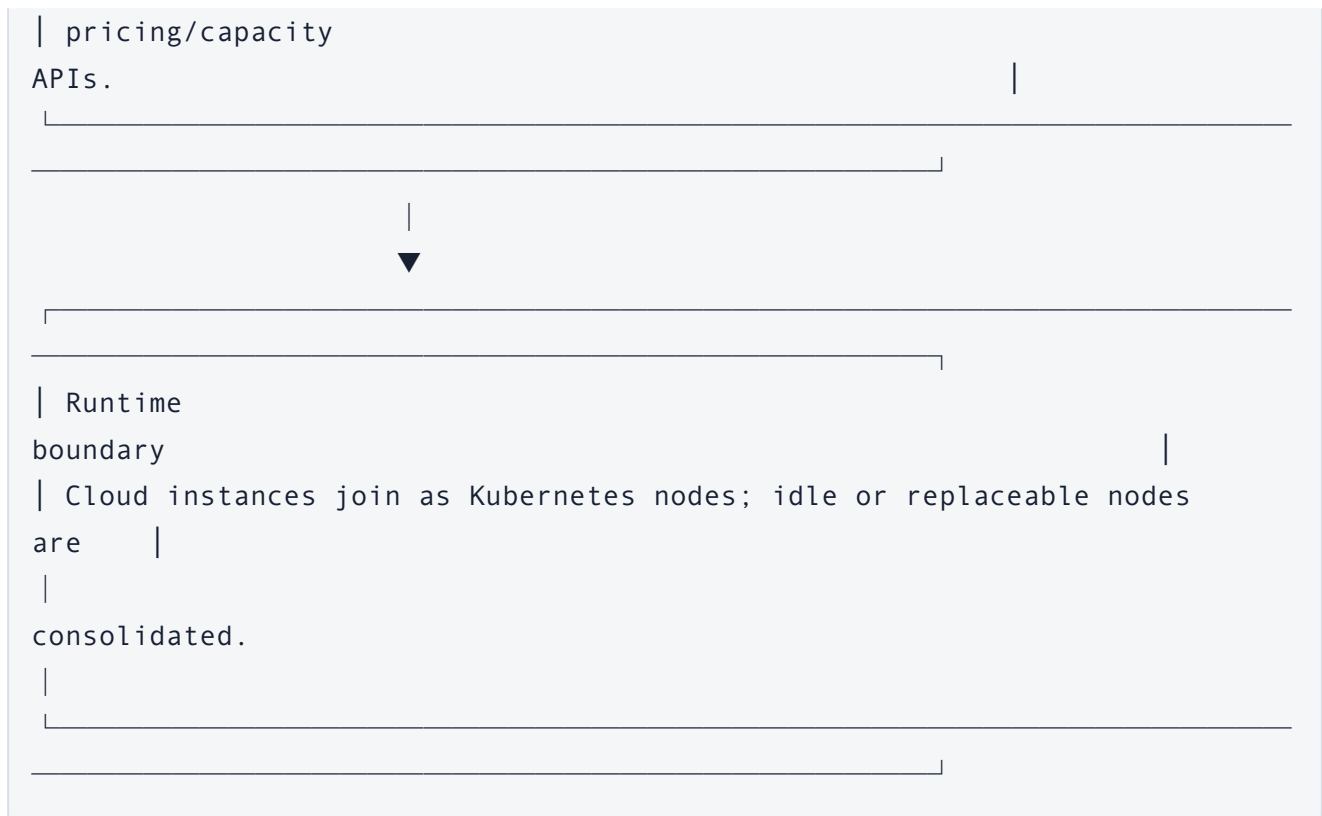
原文: raw/karpenter-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/karpenter> · 优先级 P0

一句话定位

Kubernetes node autoscaler, 用 NodePool/NodeClaim/CloudProvider 把 pending pods 转换成最合适的节点容量, 并做 consolidation 降本。

核心架构图

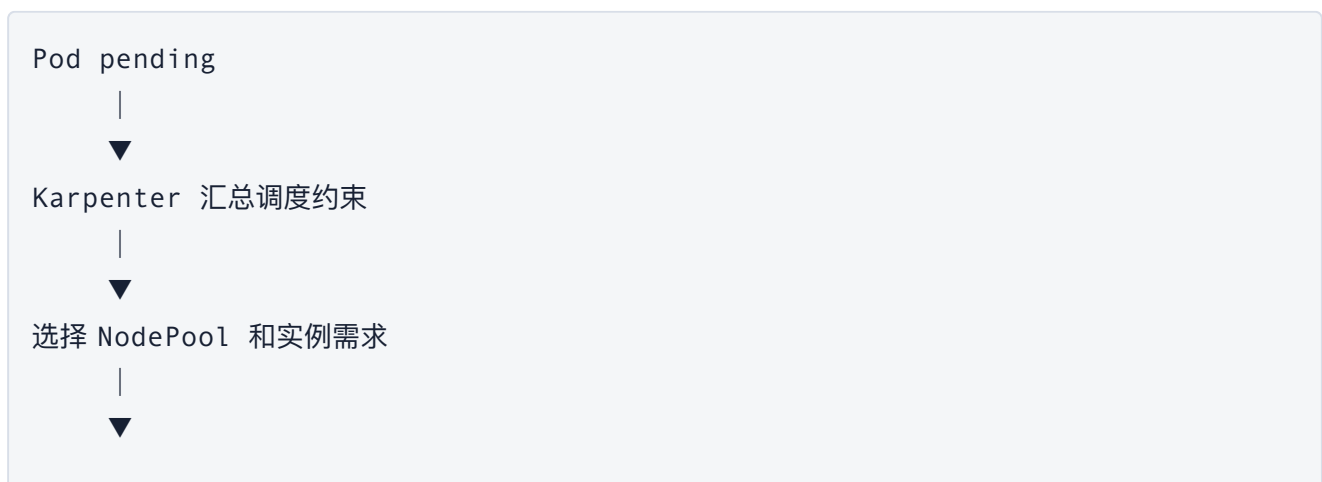




模块分层

层 / 模块	职责
API	NodePool / NodeClaim / EC2NodeClass-like provider API
Controller	provisioning、disruption、consolidation、termination
Scheduler simulation	从 pending pods 推导 instance requirements
CloudProvider boundary	云厂商容量、价格、可用区和实例类型

关键数据流



创建 NodeClaim/云主机



节点加入集群并承载 Pod



空闲或可合并时 disruption/consolidation

设计决策与哲学

- 补齐 节点弹性 / 成本 维度：Karpenter 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：和 HPA/KEDA 不同，Karpenter 扩的是节点容量；和 Cluster Autoscaler 相比，它更强调按 pending pods 即时求解容量。
- 选型价值：它应和 llm-inference, [kubernetes](#), [model-serving-operator](#) 一起看，而不是孤立评估。

相关页面

- [karpenter](#)
- [kubernetes](#)
- llm-inference
- [model-serving-operator](#)

kgateway

kgateway 是通用 cloud-native API Gateway，并带 AI Gateway 能力。仓库核心是 Gateway API 资源、controller/deployer、Envoy xDS、plugins、policies、SDS、安全和 conformance/e2e。详见 [src/kgateway-architecture](#)。

架构边界

kgateway 的主线是 Kubernetes / [Gateway API](#) 原生通用网关，AI Gateway 是其 policy/plugin 能力分支。它不是只面向模型 API 的窄代理。

关键设计

- api/v1alpha1、pkg/kgateway、pkg/xds 负责 Gateway API 扩展、controller 和 xDS 生成。
- pkg/plugins、pkg/pluginsdk 提供扩展机制。
- pkg/deployer 和 pkg/sds 管 Envoy 部署和 Secret Discovery。
- ReferenceGrant / cross-namespace 引用治理是安全边界重点。

选型判断

已有 Gateway API / Envoy 体系并想统一通用 API 与 AI gateway 时看 kgateway。GenAI 专用治理看 Envoy AI Gateway；产品化插件网关看 [higress](#)；agent protocol unified gateway 看 [agentgateway](#)。

kgateway 架构与设计思路分析

kgateway-dev/kgateway 是通用 cloud-native API Gateway，并带 AI Gateway 能力。仓库核心是 Gateway API 资源、controller/deployer、Envoy xDS、plugins、policies、SDS、安全和 conformance/e2e；最近 reference grant mode 也说明跨 namespace 引用治理是主路径。

核心架构图



模块分层

层/目录	责任
api/v1alpha1, pkg/kgateway, pkg/xds	Gateway API 扩展、controller 和 xDS 生成。
pkg/plugins, pkg/pluginsdk	插件系统。
pkg/deployer, pkg/sds	Envoy 部署和 Secret Discovery。
install/helm, examples/**	安装和示例。

关键数据流

1. Gateway API/自定义 policy 被 controller watch。
2. kgateway 生成 Envoy 配置/xDS 并部署/更新代理。
3. plugins/policies 处理 auth、traffic、AI 等增强能力。

设计决策

- 先做通用 Gateway，再把 AI 场景做成 policy/plugin。
- reference grant 强化跨 namespace 安全边界。
- 适合已有 Gateway API/Envoy 体系的团队。

对比定位

和 Envoy AI Gateway 相比，kgateway 范围更宽；和 Higress 相比，它更 Kubernetes/Gateway API 原生。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

kind

kind 是 Kubernetes IN Docker，用 Docker/Podman 容器模拟节点并用 kubeadm 拉起本地测试集群。详见 [src-kind-architecture](#)。

架构边界

kind 适合 controller/operator CI 和本地测试；kubespray 适合真实机器的生产/准生产集群部署。

什么时候用

场景	判断
需要 计算 / 测试集群 能力	适合，kind 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , model-serving-operator 组合。

核心组件

- CLI: kind create/delete/load/export
- Node image: systemd/kubelet/containerd/kubeadm
- Cluster config: control-plane/worker/networking
- Provider: docker/podman node lifecycle

选型提示

把 kind 放在 计算 / 测试集群 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

kind 架构与设计思路分析

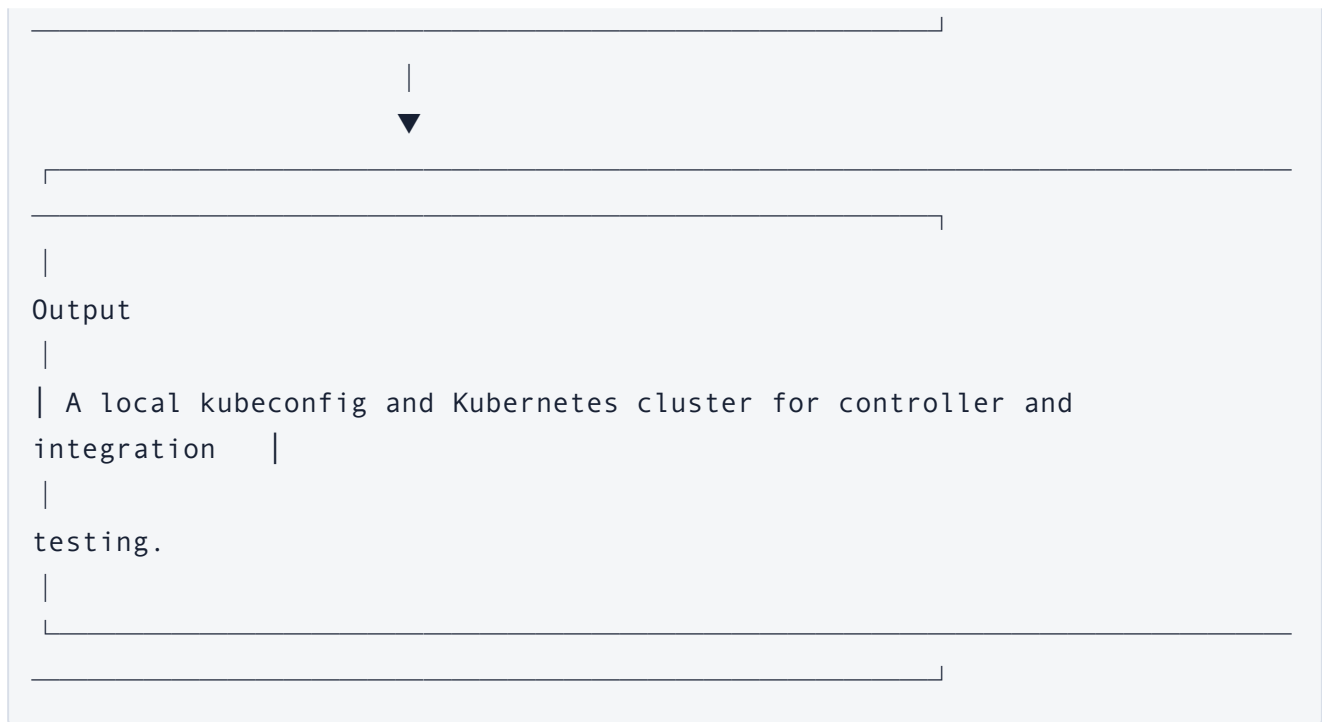
原文: raw/kind-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kind>
kind · 优先级 P0

一句话定位

kind 是 Kubernetes IN Docker，用 Docker/Podman 容器模拟节点并用 kubeadm 拉起本地测试集群。

核心架构图

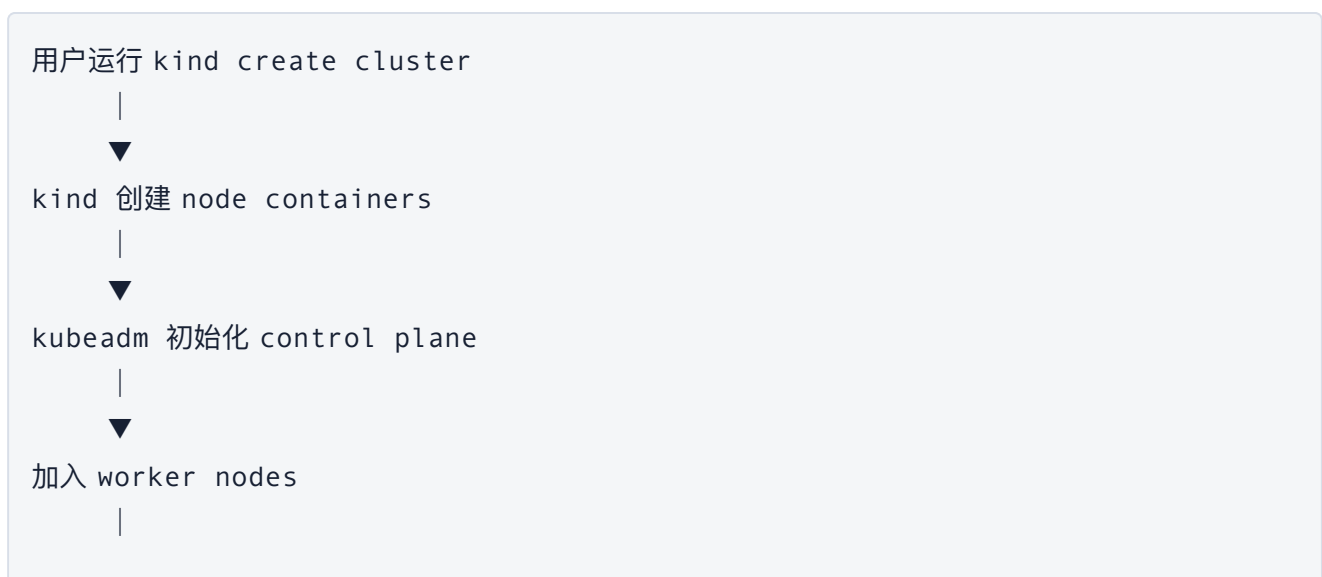




模块分层

层 / 模块	职责
CLI	kind create/delete/load/export
Node image	systemd/kubelet/containerd/kubeadm
Cluster config	control-plane/worker/networking
Provider	docker/podman node lifecycle

关键数据流



设计决策与哲学

- 补齐 计算 / 测试集群 维度：kind 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：kind 适合 controller/operator CI 和本地测试；kubespray 适合真实机器的生产/准生产集群部署。
- 选型价值：它应和 [kubernetes](#), [model-serving-operator](#) 一起看，而不是孤立评估。

相关页面

- [kind](#)
- [kubernetes](#)
- [model-serving-operator](#)

KRO

KRO (Kube Resource Orchestrator) 用 ResourceGraphDefinition 把多个 Kubernetes resources 组合成更高层 API。详见 [src-kro-architecture](#)。

架构边界

Crossplane composition 偏跨云资源；KRO 偏 Kubernetes resource graph 和平台 API 组合。

什么时候用

场景	判断
需要 higher-level API orchestration 能力	适合，KRO 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , model-serving-operator 组合。

核心组件

- ResourceGraphDefinition API
- Controller: graph reconciliation
- Generated instances/resources
- Status/value propagation

选型提示

把 KRO 放在 higher-level API orchestration 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

KRO 架构与设计思路分析

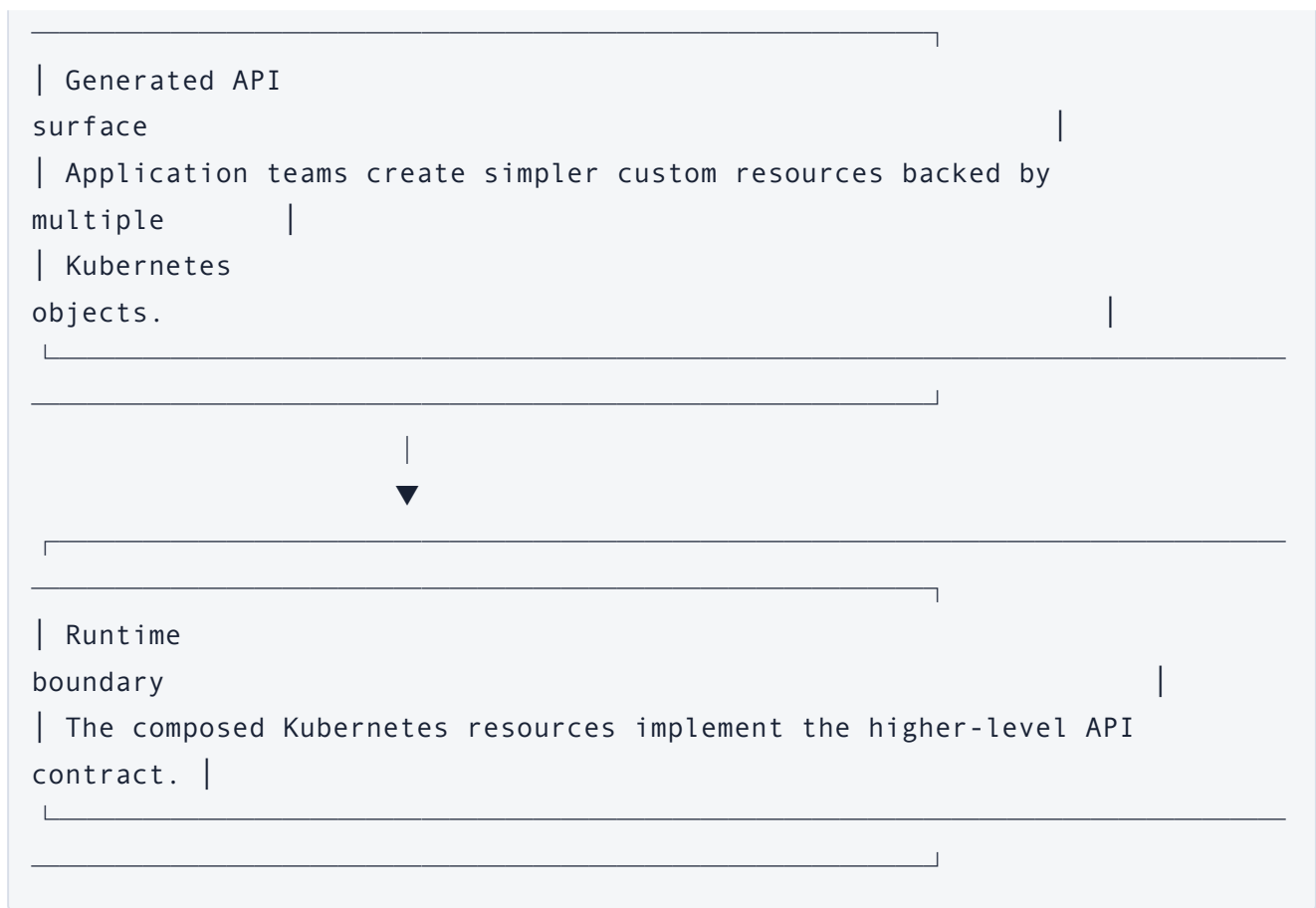
原文: raw/kro-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kro> · 优先级 P1

一句话定位

KRO (Kube Resource Orchestrator) 用 ResourceGraphDefinition 把多个 Kubernetes resources 组合成更高层 API。

核心架构图



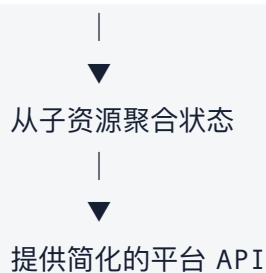


模块分层

层 / 模块	职责
ResourceGraphDefinition API	ResourceGraphDefinition API
Controller	graph reconciliation
Generated instances/resources	Generated instances/resources
Status/value propagation	Status/value propagation

关键数据流





设计决策与哲学

- 补齐 higher-level API orchestration 维度：KRO 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Crossplane composition 偏跨云资源；KRO 偏 Kubernetes resource graph 和平台 API 组合。
- 选型价值：它应和 [kubernetes](#), [model-serving-operator](#) 一起看，而不是孤立评估。

相关页面

- [kro](#)
- [kubernetes](#)
- [model-serving-operator](#)

Kruise Dashboard

Kruise Dashboard 是 OpenKruise 的运维 UI 方向项目，面向 CloneSet、Advanced StatefulSet、Advanced DaemonSet 等 OpenKruise workload 的可视化管理。根据 [src-openkruise-projects-current-state](#)，它属于 OpenKruise P1/P2 边界项目，适合放到 Kubernetes dashboard / ops UI 维度做对比。

架构边界

它不是 OpenKruise workload 控制器本身。它的价值在于把 OpenKruise CRD 的状态、操作和排障入口暴露给用户，和 [headlamp](#)、[kubewall](#)、[k8m](#) 这类 Kubernetes UI 形成对照。

什么时候看它

场景	判断
团队已经使用 OpenKruise workload	可作为运维入口候选。
想研究 Kubernetes dashboard 扩展	可和 headlamp / kubewall / k8m 对比。
想理解 OpenKruise 核心控制器	先看 openkruise-kruise 。

选型提示

Dashboard 的优先级取决于 OpenKruise workload 是否已经成为生产日常操作对象；如果只是研究控制器架构，源码优先级低于主仓和 metrics。

Kruise Game

Kruise Game 是 OpenKruise 面向 game server management 的 specialized workload operator。根据 [src-openkruise-projects-current-state](#)，它是 OpenKruise P0 项目，适合研究长连接、有状态、生命周期敏感的游戏服务器如何用 Kubernetes workload API 表达和运维。

架构边界

它不是通用 Deployment 替代品，而是把游戏服务器的实例状态、运维操作、网络和生命周期管理做成专用控制器。它和 [openkruise-kruise](#) 的区别在于：Kruise 强调通用 workload enhancement；Kruise Game 强调游戏服务器这个垂直 workload。

什么时候看它

场景	判断
需要研究 specialized workload operator	适合，GameServer 类 workload 是典型案例。
需要对比 Agones / StatefulSet	适合，后续可单独摄入源码做横向对比。
只需要通用 workload 增强	先看 openkruise-kruise 。
只需要 AI model serving	看 model-serving-operator / llm-inference。

选型提示

Kruise Game 的价值在于把业务域状态放进 Kubernetes API，而不是把游戏服务器硬塞进通用 Deployment。这和 AI inference 里把 model、batch、worker group 做成 CRD 是同一类设计思想。

Kruise State Metrics

Kruise State Metrics 是 OpenKruise CRD metrics addon。根据 [src-openkruise-projects-current-state](#)，它属于 OpenKruise P1，价值在于把 CloneSet、Advanced StatefulSet、SidecarSet 等 OpenKruise workload 状态转成可观测指标。

架构边界

它不是资源指标源，和 [metrics-server](#) 不同；它更接近 kube-state-metrics 的模式，把 Kubernetes API 对象状态转成 Prometheus 可采集指标。和 [prometheus-adapter](#) 的关系是：前者产出状态指标，后者可以把 Prometheus 指标暴露给 custom/external metrics API。

什么时候看它

场景	判断
已经使用 OpenKruise workload	适合，用它补 CRD 状态可观测。
想把 OpenKruise 指标接入 autoscaling	需要和 Prometheus / prometheus-adapter 组合验证。
只需要 Pod CPU/memory 指标	看 metrics-server 。

选型提示

OpenKruise workload 一旦进入生产，就需要配套状态指标；否则 CloneSet、Advanced StatefulSet 这类增强 workload 会比原生 Deployment 更难解释和排障。

Kruise Tools

Kruise Tools 是 OpenKruise 生态的 libraries/tools 项目。根据 [src-openkruise-projects-current-state](#)，它属于 OpenKruise P1，更适合作为 [openkruise-kruise](#) 摄入时的工具链支撑，而不是独立架构主线。

架构边界

它不是 OpenKruise 主控制面，也不是用户必须优先学习的入口。它的价值主要在开发、运维、库复用或辅助工具上。

什么时候看它

场景	判断
已经在研究 OpenKruise 主仓实现	可以看，作为工具链和复用库补充。
想快速理解 OpenKruise workload 模型	先看 openkruise-kruise 。
想研究 OpenKruise Agent sandbox	看 openkruise-agents 。

选型提示

把 Kruise Tools 放在支撑层，不要和 Kruise、Rollouts、Kruise Game 这类主 workload/control-plane 项目同权重比较。

KServe

KServe 是 Kubernetes-native 的标准化 distributed generative and predictive AI inference platform。它用 InferenceService、LLMInferenceService、InferenceGraph、controller 和 webhook 把传统 ML 与 GenAI 放到同一个服务平台。

解决的问题

平台团队往往同时承载 TensorFlow/PyTorch/XGBoost 等 predictive workload 和 vLLM/llm-d 等 LLM workload。KServe 提供统一 API、部署模式、模型缓存、OpenAI-compatible GenAI 接口、KV offload、request-based autoscaling、canary 和 scale-to-zero 能力。

核心边界

KServe 是模型服务标准化与生命周期 control plane，不是推理 kernel，也不是专门的 KV router。它可以选 serverless/Knative、RawDeployment、ModelMesh 等路径；不同模式在冷启动、scale-to-zero、canary 和运维复杂度上有差异。

适用场景

适合 KubeFlow/Kubernetes 生态、需要 predictive + generative 统一 API 和成熟 operator 治理的组织。若只做专用 LLM P/D serving，比较 [llm-d](#)、[dynamo](#)、[kthena](#)；若只想快速自托管模型，比较 [kubeflow](#)。

详见 [src-kservice-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

KServe 架构与设计思路分析

kserve/kserve 是 Kubernetes 标准化 model serving 平台。仓库超过 200MB，本次缩小到 cmd/manager/llmisvc/router/localmodel、pkg/apis/controller/webhook、charts/config/docs。新近 commit 修复 LLMISvc HTTPRoute parent status 过滤，说明 Gateway API/LLM service 路线正在快速演进。

核心架构图



模块分层

层/目录	责任
cmd/manager, pkg/controller	核心 controller manager。
cmd/llmsvc, config/llmsvc	LLM InferenceService 相关控制面。
cmd/localmodel, config/localmodels	本地模型缓存/分发。
pkg/apis, pkg/webhook	API 和 webhook。

关键数据流

1. 用户创建 InferenceService/LLMSvc/LocalModel 资源。
2. controllers 创建 predictor/router/storage runtime 和 Gateway/HTTPRoute。
3. status/webhook 维护可用性和校验。

设计决策

- KServe 是标准化/成熟路线，兼容传统 ML 与 GenAI。
- LLMSvc 是向 LLM serving 专门化的新增重点。
- 仓库历史包袱大，读源码要聚焦当前 LLMSvc/localmodel/control plane。

对比定位

和 OME/KubeAI 相比，KServe 更成熟和标准；和 llm-d/AIBrix 相比，它是 serving platform 基座，不专门做 SOTA routing/KV 优化。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Kthena

Kthena 是 Volcano 社区的 Kubernetes-native AI serving platform，覆盖模型生命周期、请求路由、P/D 分离、扩缩、限流、canary、拓扑感知和 gang scheduling。

核心抽象

- **ModelBooster**：一站式部署 API，级联创建下游 serving 资源。
- **ModelServing**：以 ServingGroup × Role 表达推理 workload，支持 Prefill/Decode、恢复、滚动更新、拓扑和 gang 调度。
- **ModelServer**：通过 workload selector 发现并暴露推理 Pod，承载重试、超时和连接策略。
- **ModelRoute**：按模型、LoRA、路径或 header 匹配，支持权重流量、canary、token rate limit。
- **AutoScalingPolicy**：指标驱动、panic/stabilization window、异构资源和 P/D role-level 扩缩。

架构边界

Kthena 的显著设计是 control plane 与 data plane 解耦：workload controller 和 router 可分别安装，router 也能接入 Gateway API。它比纯 model operator 更靠近完整 LLM serving platform；比 RBG 更直接提供模型/路由 API；与 [llm-d](#) 的差异在于更强调 Volcano 拓扑/gang 和自身的 ModelRoute/ModelServing 资源模型。

适用与限制

适合希望在 Kubernetes 内获得一体化 LLM serving、P/D role autoscaling、KV/LoRA-aware routing 和 Volcano 调度的团队。若组织已经标准化 Gateway API/InferencePool 并希望最大化复用该生态，应同时比较 [llm-d](#)；若主要需求是跨多云 GPU 管理和 MaaS，应比较 [gpustack](#)。

kube-agentic-networking

kube-agentic-networking 为 Kubernetes 中 agents/tools 提供 agentic networking policies and governance，面向 AI Agent 出口、工具调用和网络权限边界。详见 [src-kube-agentic-networking-architecture](#)。

架构边界

它和 agentgateway/agent-sandbox 互补：sandbox 管运行时隔离，gateway/networking 管出入口策略。

什么时候用

场景	判断
需要 Agent networking governance 能力	适合，kube-agentic-networking 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 agentgateway , agent-sandbox, cloud-native-security 组合。

核心组件

- CRD/policies: agent/tool/network intent
- Controller: policy reconciliation
- Gateway/network integration
- Audit/governance signals

选型提示

把 kube-agentic-networking 放在 Agent networking governance 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

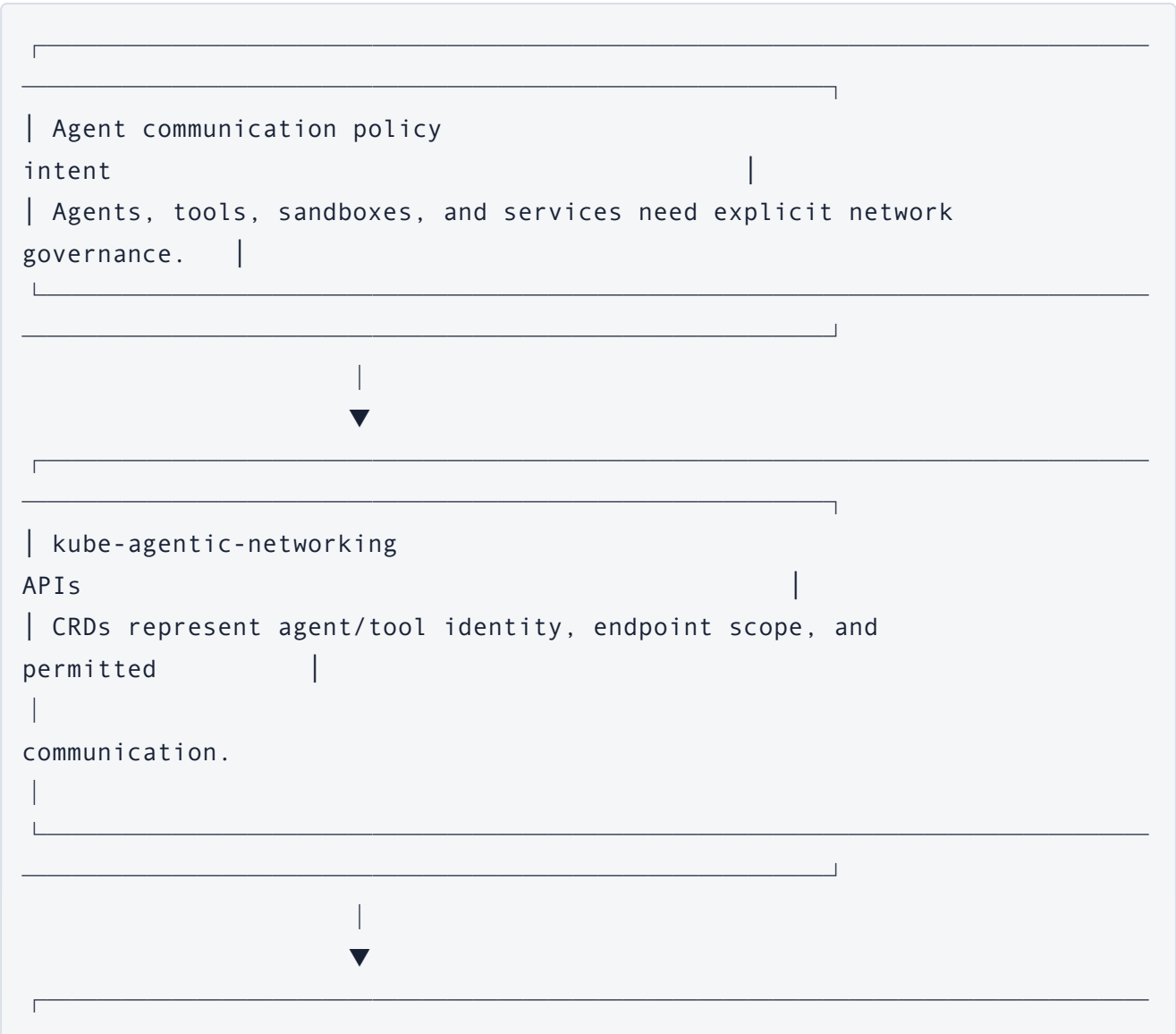
kube-agentic-networking 架构与设计思路分析

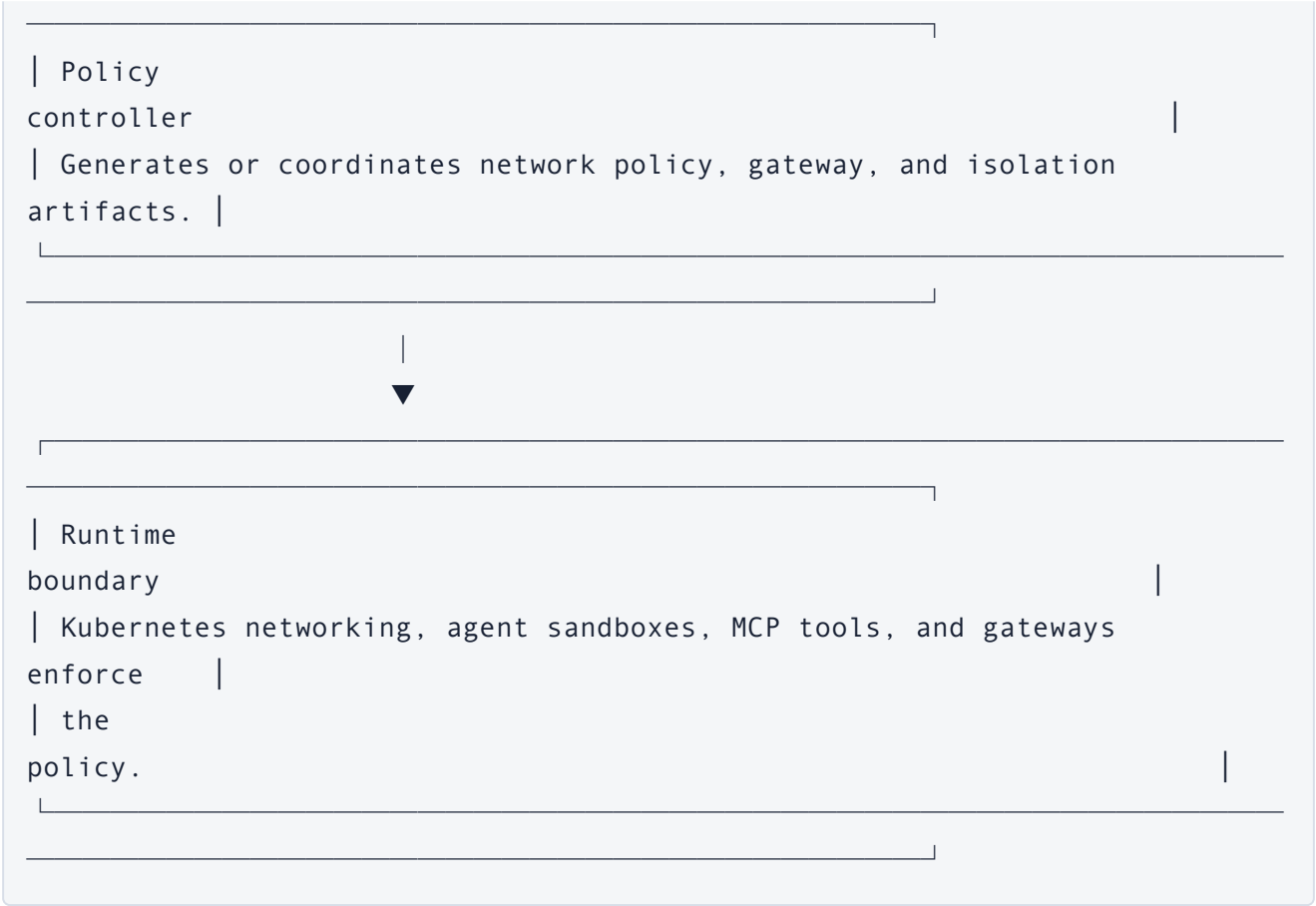
原文: raw/kube-agentic-networking-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kube-agentic-networking> · 优先级 P1

一句话定位

kube-agentic-networking 为 Kubernetes 中 agents/tools 提供 agentic networking policies and governance，面向 AI Agent 出口、工具调用和网络权限边界。

核心架构图



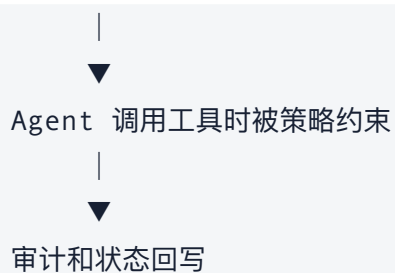


模块分层

层 / 模块	职责
CRD/policies	agent/tool/network intent
Controller	policy reconciliation
Gateway/network integration	Gateway/network integration
Audit/governance signals	Audit/governance signals

关键数据流





设计决策与哲学

- 补齐 Agent networking governance 维度: kube-agentic-networking 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: 它和 agentgateway/agent-sandbox 互补: sandbox 管运行时隔离, gateway/networking 管出入口策略。
- 选型价值: 它应和 [agentgateway](#), agent-sandbox, [cloud-native-security](#) 一起看, 而不是孤立评估。

相关页面

- [kube-agentic-networking](#)
- [kubernetes](#)
- [agentgateway](#)
- agent-sandbox
- [cloud-native-security](#)

kube-scheduler-simulator

kube-scheduler-simulator 提供 Kubernetes scheduler 行为模拟和可视化，用于理解 filter/score、调度失败原因和策略效果。 详见 [src-kube-scheduler-simulator-architecture](#)。

架构边界

KWOK 模拟大规模集群对象；scheduler simulator 解释单次或少量调度决策过程。

什么时候用

场景	判断
需要 scheduler 可视化模拟 能力	适合，kube-scheduler-simulator 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes 组合。

核心组件

- Simulator API/server
- Scheduler integration
- Frontend visualization
- Scenario objects: nodes/pods/policies

选型提示

把 kube-scheduler-simulator 放在 scheduler 可视化模拟 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

kube-scheduler-simulator 架构与设计思路分析

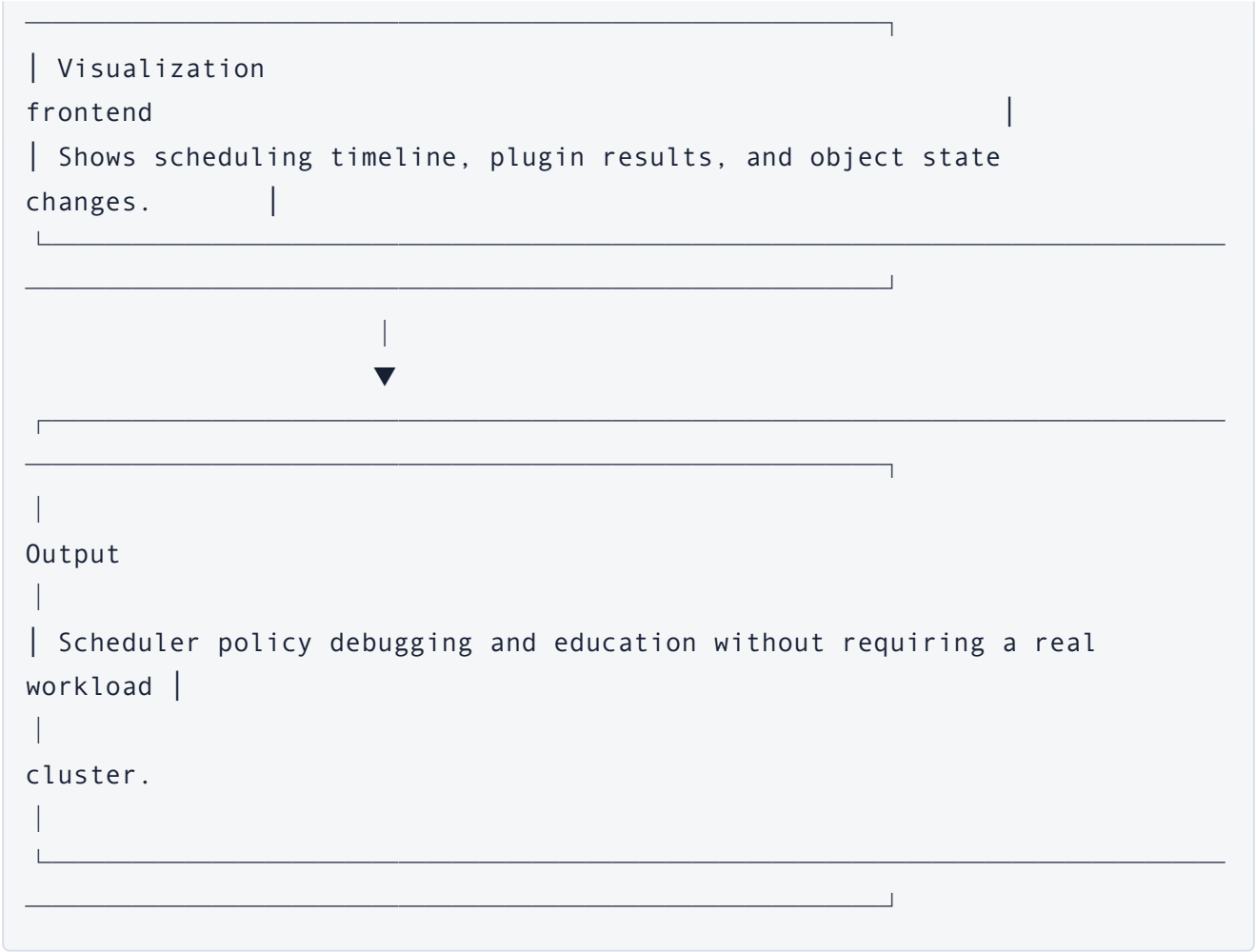
原文: raw/kube-scheduler-simulator-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kube-scheduler-simulator> · 优先级 P1

一句话定位

kube-scheduler-simulator 提供 Kubernetes scheduler 行为模拟和可视化，用于理解 filter/score、调度失败原因和策略效果。

核心架构图

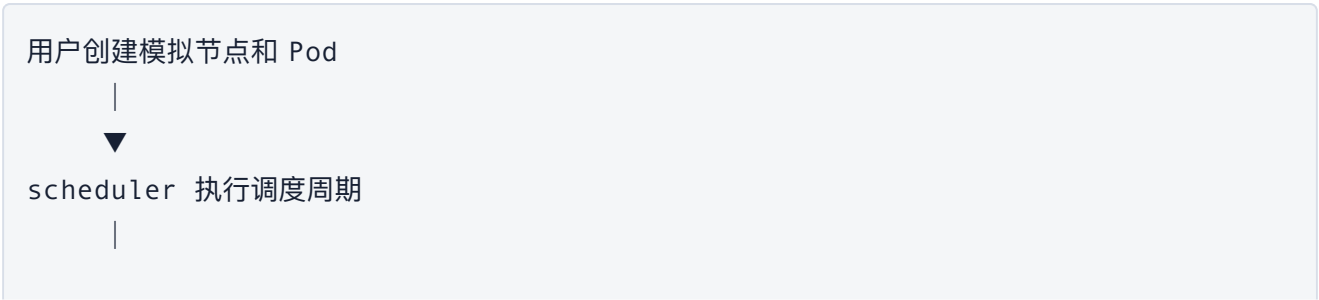


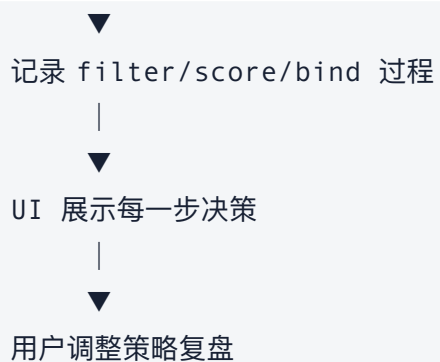


模块分层

层 / 模块	职责
Simulator API/server	Simulator API/server
Scheduler integration	Scheduler integration
Frontend visualization	Frontend visualization
Scenario objects	nodes/pods/policies

关键数据流





设计决策与哲学

- 补齐 scheduler 可视化模拟 维度: kube-scheduler-simulator 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: KWOK 模拟大规模集群对象; scheduler simulator 解释单次或少量调度决策过程。
- 选型价值: 它应和 [kubernetes](#) 一起看, 而不是孤立评估。

相关页面

- [kube-scheduler-simulator](#)
- [kubernetes](#)

KubeAI

KubeAI 是轻量 Kubernetes AI inference operator，使用 Model CRD、OpenAI-compatible model proxy、model loader 和 autoscaler 管理 LLM、VLM、embedding、speech-to-text 等服务。

解决的问题

把“模型文件 + 推理 engine”快速转换成 Kubernetes 内可调用的 OpenAI-compatible endpoint，并在模型闲置或流量变化时执行加载、卸载和扩缩，降低自托管 AI 服务的上手成本。

核心边界

KubeAI 偏应用友好和快速部署，不负责重新实现 vLLM/SGLang kernel，也不是以多角色 P/D、复杂 KV locality 或 GPU topology 为中心的编排层。复杂场景通常需要额外的 Gateway、调度或 serving stack。

适用场景

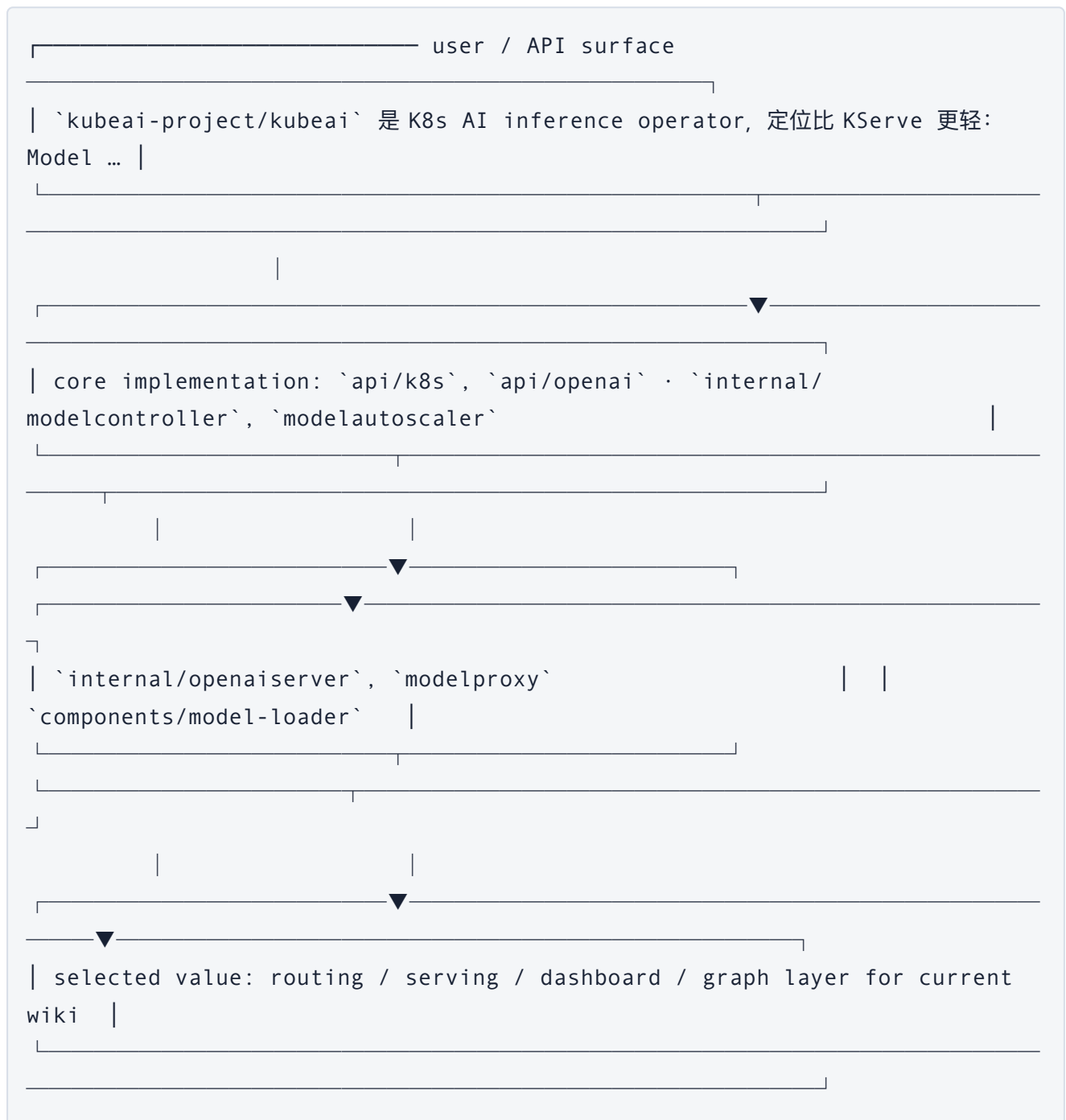
适合中小规模自托管、模型种类较多、希望 scale-from-zero 和统一 API 的团队。需要 predictive + GenAI 统一平台看 [kserve](#)；需要 enterprise vLLM 组件看 [aibrix](#)；需要深度 P/D/KV 看 [dynamo](#) 或 [kthena](#)。

详见 [src-kubeai-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

KubeAI 架构与设计思路分析

kubeai-project/kubeai 是 K8s AI inference operator，定位比 KServe 更轻：Model CRD、OpenAI-compatible server、model proxy、autoscaler、model loader、vLLM client、Helm charts 和内置模型 manifests。

核心架构图



模块分层

层/目录	责任
api/k8s, api/openai	K8s API 和 OpenAI API 类型。
internal/modelcontroller, modelautoscaler	Model controller 和扩缩容。
internal/openaiserver, modelproxy	OpenAI-compatible endpoint 和代理。
components/model-loader	模型加载组件。

关键数据流

1. 用户创建 Model CRD 或安装 charts/models。
2. controller 创建 pod/deployment 并注入 model labels。
3. OpenAI server/model proxy 统一暴露 endpoint。

设计决策

- 以 Model 为核心资源，简化用户心智。
- 内置模型 charts/manifests 适合快速上手。
- 能力覆盖广但高级 routing/KV 优化弱于 llm-d/AIBrix。

对比定位

和 KServe 相比更轻；和 GPUStack 相比更 Kubernetes operator；和 OME 相比路线更直接。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Kubebuilder

Kubebuilder 是构建 Kubernetes APIs using CRDs 的 SDK，把 API type、marker、controller-runtime manager、webhook、RBAC 和 manifests 生成流程标准化。详见 [src-kubebuilder-architecture](#)。

证据说明：本次官方默认分支快照为 [5f31d1f3c075](#)，该 commit 不代表 release。[src-kubebuilder-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目证据与职责图见 [k8s-core-controller-map](#)。

在 M5-A Controller 地图中的位置

Kubebuilder 负责作者工作流、脚手架与项目布局，通过生成的工程组织 [controller-runtime](#) 组件，并调用 [controller-tools](#) 生成 CRD、RBAC 等资产。运行时 reconcile 语义由 controller-runtime 与项目自己的业务逻辑承担；Kubebuilder 不负责执行控制循环。三者的构建期与运行期边界见 [k8s-core-controller-map](#)。

架构边界

kubebuilder 解决项目结构和生成路径；controller-runtime 解决运行时 controller 抽象。

什么时候用

场景	判断
需要 CRD / controller 脚手架能力	适合，Kubebuilder 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , model-serving-operator , declarative-agent-management 组合。

核心组件

- CLI scaffolding: init/create api/webhook
- API markers: kubebuilder validation/printcolumn/rbac
- Project layout: api/ controllers/ config/
- Generation: CRD/RBAC/webhook/deepcopy manifests

选型提示

把 Kubebuilder 放在 CRD / controller 脚手架 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

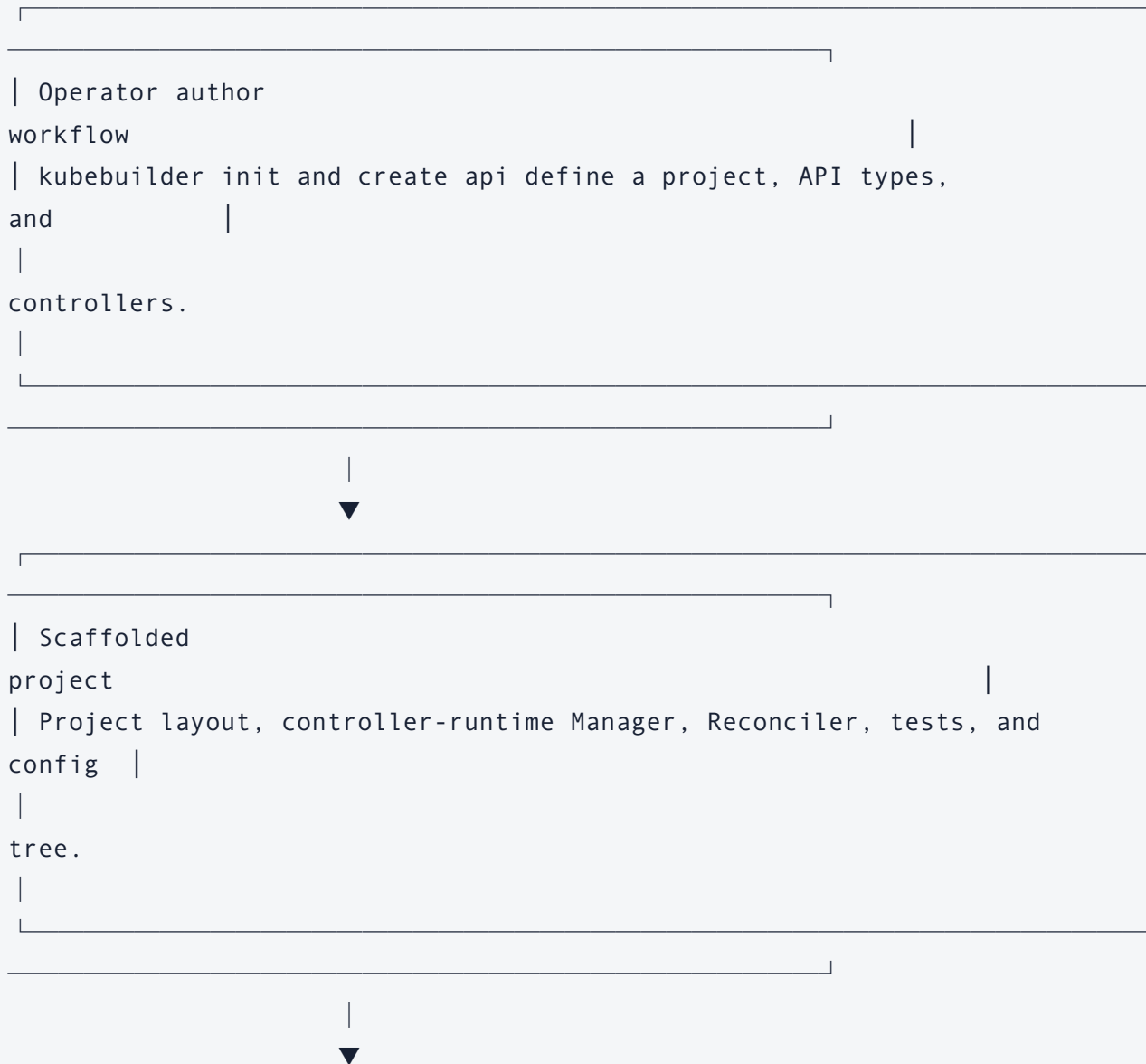
Kubebuilder 架构与设计思路分析

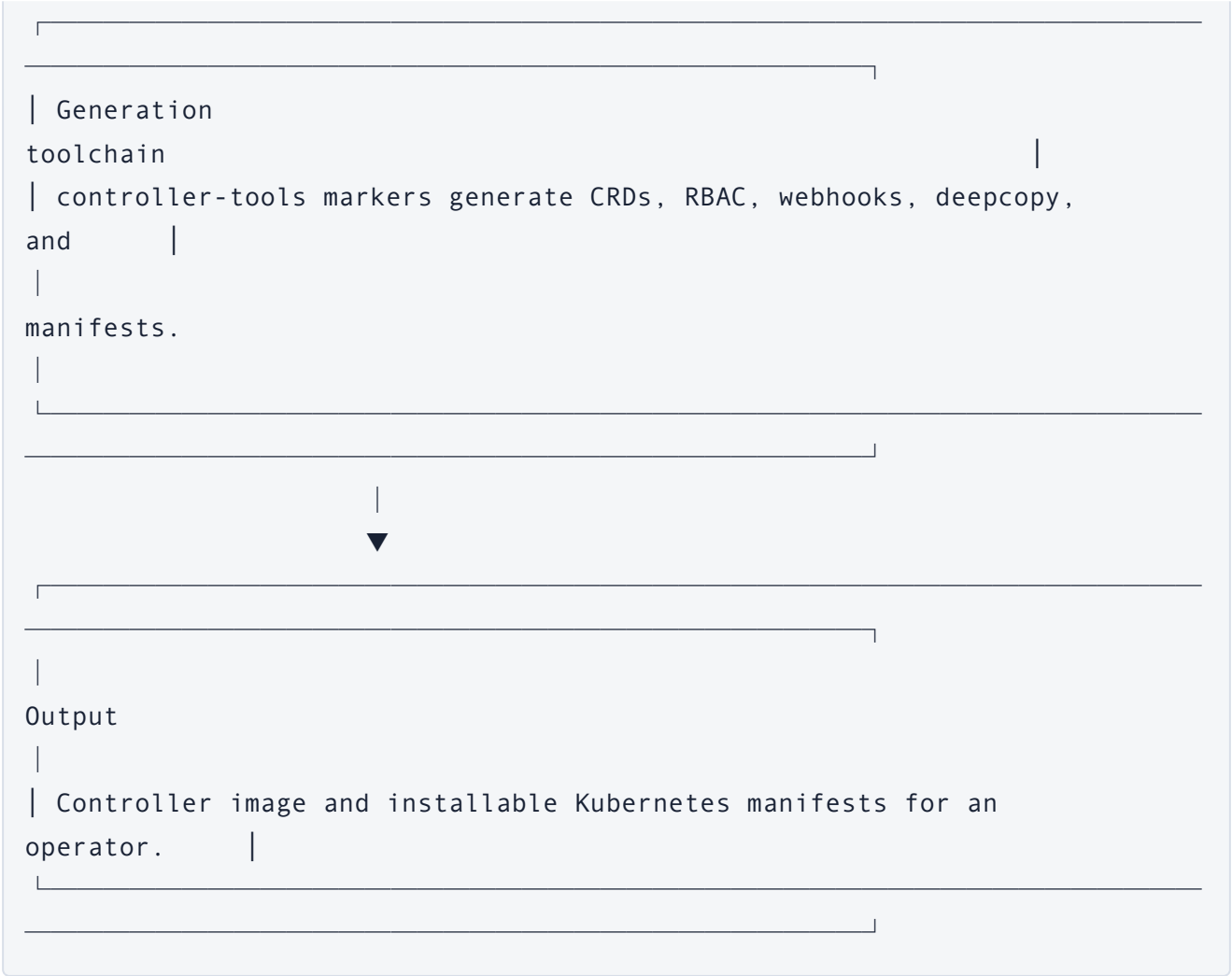
原文: raw/kubebuilder-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kubebuilder> · 优先级 P0

一句话定位

Kubebuilder 是构建 Kubernetes APIs using CRDs 的 SDK，把 API type、marker、controller-runtime manager、webhook、RBAC 和 manifests 生成流程标准化。

核心架构图

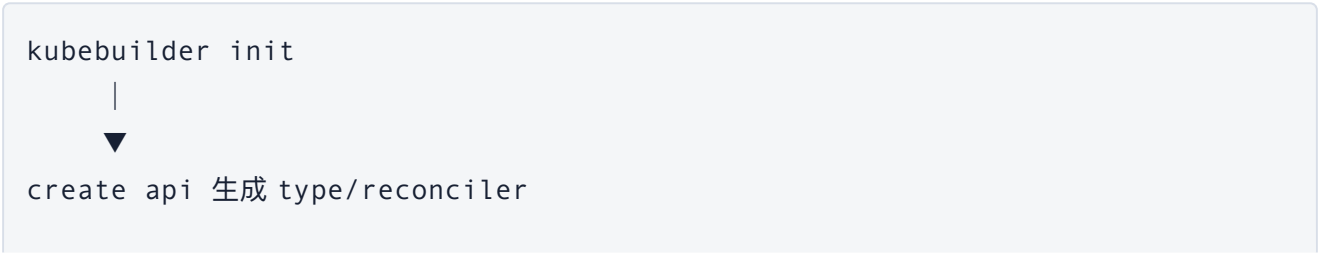


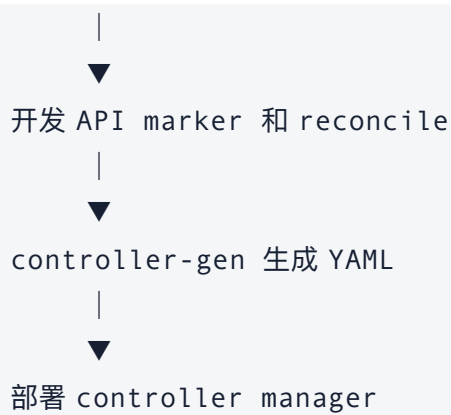


模块分层

层 / 模块	职责
CLI scaffolding	init/create api/webhook
API markers	kubebuilder validation/printcolumn/rbac
Project layout	api/ controllers/ config/
Generation	CRD/RBAC/webhook/deepcopy manifests

关键数据流





设计决策与哲学

- 补齐 CRD / controller 脚手架 维度: Kubebuilder 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: kubebuilder 解决项目结构和生成路径; controller-runtime 解决运行时 controller 抽象。
- 选型价值: 它应和 [kubernetes](#), [model-serving-operator](#), declarative-agent-management 一起看, 而不是孤立评估。

相关页面

- [kubebuilder](#)
- [kubernetes](#)
- [model-serving-operator](#)
- declarative-agent-management

kubectl-ai

kubectl-ai 是 GoogleCloudPlatform 出品的 kubectl 入口 Kubernetes assistant。它以 Go CLI / agent loop 为核心，内置 bash、kubectl、journal、sessions、sandbox，并支持 MCP server mode。详见 [src-kubectl-ai-architecture](#)。

架构边界

kubectl-ai 是 CLI-first 的 [ai-ops](#) 工具，不是 dashboard，也不是完整 agent workflow 平台。它把自然语言问题转成 kubectl/bash 查询和解释，并能作为 MCP server 被其他 agent 调用。

关键设计

- cmd/** 提供 CLI 命令入口。
- pkg/agent 和 pkg/tools 管理 agent loop 与内置工具。
- pkg/mcp 让它从 standalone CLI 变成 K8s tool server。
- pkg/sessions / pkg/journal 保留会话与操作记录。

选型判断

想在终端里自然语言操作 Kubernetes，看 kubectl-ai；想把 K8s/DevOps 操作组织成 agentic workflow，看 [kagent](#)；想用 UI 管集群并接入 AI，看 [k8m](#) / [kubewall](#)。

kubectl-ai 架构与设计思路分析

GoogleCloudPlatform/kubectl-ai 是 kubectl 入口的 K8s assistant。核心是 Go CLI/agent loop，内置 bash、kubectl、journal、sessions、sandbox 和 MCP server mode；最近 commit 特别修复 MCP server mode 下 built-in tools 注册，说明它既是交互 CLI，也是可被其他 agent 调用的 K8s tool server。

核心架构图



模块分层

层/目录	责任
cmd/**	CLI 命令入口。
pkg/agent, pkg/tools	agent loop 和 bash/kubectl 等工具。
pkg/mcp	MCP server mode。
pkg/sessions, pkg/journal	会话与操作记录。

关键数据流

1. 用户在终端输入自然语言问题。
2. agent 选择 bash/kubectl 内置工具查询集群。
3. 结果写入 journal/session，并生成解释或下一步命令。

设计决策

- 把 kubectl 作为主要 UX，降低 K8s 用户迁移成本。
- MCP server mode 让它从 standalone CLI 变成其他 agent 的 K8s backend。
- 工具面相对克制，适合安全审计。

对比定位

和 kagent 相比更轻、更 CLI-first；和 K8s dashboard 类项目相比，它不提供完整 UI，而是把 kubectl workflow 自然语言化。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Kubernetes Device Plugin

Kubernetes Device Plugin 是 kubelet 扩展机制，让厂商把 GPU、FPGA、RDMA 等特殊硬件注册为节点资源，并在 kubelet 调用 Allocate 时返回设备与容器配置。

在 M5-C Device / GPU 地图中的位置

Device Plugin 向 kubelet 注册资源名与 endpoint，通过 ListAndWatch 发布设备/health，再由 kubelet 更新节点 extended-resource capacity/allocatable。scheduler 按节点资源选择位置，节点侧 Allocate 返回 env、mount 或 cdi 交接信息；完整流程见 [k8s-gpu-device-stack](#) 的 D2。[node-feature-discovery](#) 提供能力标签，[gpu-operator](#) 管受管组件，均不替代本次设备分配。

[dra-driver-nvidia-gpu](#) 的 Claim/库存/Prepare 路径见 [kubernetes-dra-design-deep-dive](#)。两种 API 不应让同一设备被重复分配；例如 [hami](#) 与 NVIDIA 官方 plugin 不能在同一节点争用 `nvidia.com/gpu`，共存须明确资源所有权与安装模式。

核心流程

设备发现后建立 plugin Unix socket，向 kubelet 注册并通过 ListAndWatch 报告库存/健康；Pod 请求资源并绑定节点后，kubelet 选择设备 ID、调用 Allocate，再经容器配置将设备交给 runtime。

代表项目

[k8s-device-plugin](#) 是 NVIDIA 官方实现，覆盖 NVML/CUDA/Tegra/VFIO discovery、MIG/MPS/time-slicing、health check、env/volume/CDI device list strategy。

与 DRA 的关系

DRA 不是简单替代所有 device plugin，而是为更复杂的声明式资源配置提供新 API。理解 device plugin 仍是理解 GPU on Kubernetes 的基础。

Kubernetes Dynamic Resource Allocation

Kubernetes DRA 是新一代特殊设备资源分配模型，用 ResourceClaim、ResourceSlice 和 driver plugin 让 GPU、DPU、RDMA 等设备的配置与调度更声明式、更可扩展。

在 M5-C Device / GPU 地图中的位置

基础 DRA 自 Kubernetes v1.35 stable；binding conditions、consumable capacity、partitionable devices、optional node operations 等扩展独立分阶段，见 [kubernetes-dra-design-deep-dive](#) 的「当前上游核验（2026-09-27）」表及其官方证据。ResourceClaim allocation 是控制面分配，NodePrepareResources 是节点准备，[cdi](#) 是 runtime 注入交接，不能将任一步成功当作全链路已就绪。

[k8s-gpu-device-stack](#) 对比 [k8s-device-plugin](#) 与 [dra-driver-nvidia-gpu](#) 的路径：[node-feature-discovery](#) 的节点信号和 [gpu-operator](#) 的组件管理不取代 Claim 分配，[hami](#) 的 DRA 接入也不自动改变容器隔离强度。

和传统 device plugin 的区别

传统 [device-plugin](#) 主要把节点设备暴露成 extended resources，Pod 请求后由 kubelet Allocate。DRA 把资源 claim、参数、slice 和第三方 driver 放到调度语义里，让设备厂商能表达更复杂的配置，例如 dynamic MIG、VFIO、ComputeDomain。

代表项目

[dra-driver-nvidia-gpu](#) 是 NVIDIA GPU 的 DRA driver 实现，覆盖 ResourceClaim validation、NodePrepare/Unprepare、ResourceSlice、dynamic MIG/VFIO 和 ComputeDomain。

Upstream 进展

2026-07-15 复核 kubernetes/enhancements 后，DRA 的近期重点已经从 ResourceSlice / ResourceClaim 基础模型，扩展到 workload 级 claim、可消费容量、共享容量、设备兼容组、ResourceClaim device status、DRA attributes Downward API 和标准 numaNode 属性。详见 [k8s-v1.37-scheduling-node-dra-progress](#) 和 [kubernetes-dra-design-deep-dive](#)。

对 GPU/AI 平台来说，最值得跟踪的是 5729-resourceclaim-support-for-workloads、5075-dra-consumable-capacity、4817-resource-claim-device-status、5304-dra-attributes-downward-api 和 6072-dra-standard-numanode，因为它们把设备容量、状态和拓扑从 driver 私有逻辑推进到 Kubernetes 可推理 API。

选型提示

选型应比较目标 release 的设备需求、driver 支持和运维路径：简单 extended-resource 场景可用 [k8s-device-plugin](#)；需要 Claim、属性选择或跨 Pod 生命周期时评估 DRA。基础 API 已稳定，但动态 MIG、共享和其他扩展仍须逐项核对 Kubernetes 与厂商支持范围，不能从基础 DRA 的阶段推导整套部署成熟度。

Kubernetes Gateway API

K8s SIG-Network 的新一代入口 API，取代 Ingress。设计为更具表达力、可扩展、面向角色（infrastructure provider / cluster operator / application developer 三层职责分离）。核心资源族：

资源	角色
GatewayClass	infrastructure provider 注册的 Gateway 实现（类比 IngressClass）
Gateway	cluster operator 部署的 gateway 实例（绑定 IP / 证书 / 监听端口）
HTTPRoute / GRPCRoute / TCPRoute / TLSRoute	application developer 挂在 Gateway 上的具体路由
ListenerSet (experimental)	跨 Gateway 共享 listener 配置
InferencePool (experimental)	AI 推理负载均衡（项目 wg-inference 中）
ReferenceGrant	跨 namespace 引用授权

在 [agentgateway](#) 中的使用

agentgateway 直接复用上游 Gateway API 资源做路由声明，不 fork、不改 Gateway API 本体。扩展能力放在引用型自家 CRD：

- AgentgatewayPolicy 挂在 Gateway / Listener / Route 上做 CEL 策略
- AgentgatewayBackend 在 HTTPRoute 的 backendRef 里被引用，定义 AI / MCP / A2A 后端
- AgentgatewayParameters 跟 GatewayClass 的 parametersRef 关联，控制 deployer 行为

这种"主资源用上游 API，扩展用自家引用型 CRD"的模式跟 Istio Gateway API 实现是同款。

与 Ingress 的对照

维度	Ingress	Gateway API
角色模型	一个资源里混合平台和应用意图	GatewayClass / Gateway / Route 分离职责
协议覆盖	主要 HTTP/HTTPS	HTTP、gRPC、TCP、TLS 等资源族
扩展方式	annotation 堆叠，controller-specific	标准字段 + policy attachment + extension points
多团队边界	namespace / host 冲突处理弱	ReferenceGrant、Route attachment 更明确

维度	Ingress	Gateway API
适合场景	简单 Web ingress	多租户入口、mesh interop、AI inference routing

主流实现位置

实现	特点	适合关注点
Istio	Gateway API + service mesh / HBONE / xDS 生态	mesh 与 ingress 统一
Envoy Gateway	以 Envoy 为数据面的 Gateway API 实现	API gateway / GenAI gateway 扩展
Cilium Gateway API	eBPF datapath + Cilium 网络策略生态	网络、安全、L7 policy 结合
Kong / NGINX Gateway Fabric	传统 API gateway / ingress controller 演进	已有网关生态迁移
agentgateway	复用 Gateway API 控制面，扩展 AI/MCP/ A2A backend	AI-native L7 gateway

GAMMA 与 Service Mesh

GAMMA（Gateway API for Mesh Management and Administration）把 Gateway API 的 Route / policy attachment 思路扩展到 service mesh 内部流量。对 AI infra 的价值是：入口流量、服务间流量、Agent 到工具/模型的出口流量可以逐渐使用同一套资源模型表达，而不是 Ingress、VirtualService、网关插件各写一套。

[agentgateway](#) 借用 istio / xds / hbone 的思路，说明 Gateway API 已经不只是“北南向入口”，也可以成为 AI gateway 控制面的公共语言。

InferencePool 与 AI 推理调度

[gateway-api-inference-extension](#) 把推理 endpoint picking 推进 Gateway API extension 层：

- InferencePool 表示一组可被 gateway 选择的推理后端；
- EPP / LWEPP 作为 Endpoint Picker 参考实现；
- 下游如 [llm-d](#) 可以把 KV cache、P/D 分离、负载、端口、模型能力等信息接进 picking；
- 它解决的是“Gateway 该把这个推理请求发给哪个 endpoint”，不是 provider auth、prompt mutation 或模型本身推理。

因此 Gateway API 在 AI 场景里的位置正在上移：从普通 HTTP ingress，变成 LLM serving routing、AI gateway policy 和 mesh interop 的共同 API 层。

Kubernetes KEP Design Tracking

这页是 /Users/zhenyu.jiang/enhancements/keps 中 sig-scheduling、sig-autoscaling、sig-node 三个 SIG 的 KEP 设计方案源摘要。它的目标不是抽象一个新概念，而是建立一个可持续更新的 KEP 设计追踪入口：按 SIG、设计方向、状态、成熟度、交叉依赖和阅读优先级组织 KEP。

更偏清单和后续追踪的表格见 [kubernetes-keps-design-tracking](#)；逐个 KEP 的实现状态、Alpha/Beta/GA、feature gates 和关键实现路径见 [kubernetes-keps-implementation-matrix](#)。

本轮已把最重要的合并设计组单独拉出详解：

- [kubernetes-keps-feature-coverage](#)
- [kubernetes-keps-implementation-matrix](#)
- [kubernetes-scheduler-core-design](#)
- [kubernetes-workload-gang-scheduling-design](#)
- [kubernetes-dra-design-deep-dive](#)
- [kubernetes-hpa-autoscaling-design](#)
- [kubernetes-in-place-pod-resize-design](#)
- [kubernetes-node-runtime-observability-security-design](#)

2026-07-15 新增 upstream refresh: [k8s-v1.37-scheduling-node-dra-progress](#) 记录 kubernetes/enhancements 当前 master 中调度、Node、DRA 和 autoscaling 的 v1.37 进展。它不是新的源码摄入，而是对本源摘要和 [kubernetes-keps-implementation-matrix](#) 的状态刷新入口。

范围和方法

本轮只读取三个 SIG 的 KEP 目录，不纳入 prod-readiness/* YAML:

SIG	README 数量	追踪重点
sig-scheduling	58	scheduler framework、队列、抢占、workload/gang、PodGroup、DRA 调度语义。
sig-autoscaling	9	HPA API、metrics、behavior、tolerance、scale from zero、fallback、Cluster Autoscaler 集成。
sig-node	125	kubelet、CRI、resource managers、device plugin、DRA/CDI、Pod lifecycle、安全、节点健康。

追踪维度：

维度	说明
SIG	KEP 所属目录和 owning-sig。
分类	设计方向，不按 KEP 文件夹机械排序。
状态	从 keps.yaml 读取 status/stage/latest-milestone，但保留人工判断。
方案要点	该 KEP 解决的设计问题、API/控制面边界和与其他 KEP 的关系。
追踪优先级	P0 表示正在形成设计主线或近期 v1.35-v1.37 活跃；P1 表示稳定但仍是背景知识；P2 表示历史/补充。

总体设计线索

```

KEP design tracking
|
+-- sig-scheduling
|   +-- scheduler framework / config
|   +-- queue / requeue / batching
|   +-- placement / topology / affinity
|   +-- preemption / disruption
|   +-- workload / PodGroup / gang
|   +-- DRA / device-aware scheduling
|
+-- sig-autoscaling
|   +-- HPA API and behavior
|   +-- metrics specificity and pod selection
|   +-- scale from zero and external metrics fallback
|   +-- Cluster Autoscaler integration
|
+-- sig-node
    +-- kubelet / CRI / runtime boundary
    +-- CPU / memory / topology managers
    +-- DRA / device plugin / CDI
    +-- Pod lifecycle and resize
    +-- security / isolation
    +-- observability / node health

```

SIG Scheduling 设计方案

1. Scheduler framework / config

代表 KEP:

KEP	状态	方案要点
624-scheduling-framework	implemented / stable	把调度过程拆成 QueueSort、PreFilter、Filter、PostFilter、PreScore、Score、Reserve、Permit、PreBind、Bind、PostBind 等扩展点。核心设计是让复杂策略变成 plugin，而不是继续膨胀 scheduler core。
785-scheduler-component-config-api	implemented / stable	给 scheduler 配置提供 versioned ComponentConfig，支撑 profile、plugin enable/disable/order/args。
1451-multi-scheduling-profiles	implementable / beta	多个 scheduling profile 共享同一个 scheduler 实例，减少多 scheduler binary 的部署和 cache 成本。
1819-scheduler-extender	implemented / stable	历史扩展路径：HTTP extender 在 Filter/Prioritize/Preempt/Bind 等阶段介入，但性能、cache、错误回滚和扩展点数量都有局限。

设计判断：后续 KEP 基本都建立在 scheduler framework 之上。追踪 scheduling KEP 时，先判断它是在新增 framework extension point、改变 queue 行为，还是只是在某个 plugin 里新增策略。

2. Queue / requeue / batching

代表 KEP:

KEP	状态	方案要点
3521-pod-scheduling-readiness	implemented / stable	通过 scheduling gates 让 Pod 在外部条件满足前不进入正常调度，避免 scheduler 反复尝试明显不可调度的 Pod。
4247-queueinghint	implemented / stable	每个 plugin 给事件注册 QueueingHint，告诉调度队列某类事件是否可能让某个 Pod 可调度。重点是减少无效 requeue，并允许 DRA 等等待型场景跳过不必要 backoff。
6132-prequeueing-hints	implementable / beta	在 PreQueueing 阶段过滤事件，让调度队列在 Pod 进入 activeQ 之前更早判断是否值得唤醒。

KEP	状态	方案要点
5142-pop-backoffq-when-activeq-empty	implementable / beta	activeQ 空时从 backoffQ 提前取 Pod，提高队列吞吐和资源利用。
5501-reflect-preenqueue-rejections-in-pod-status	implementable / beta	把 PreEnqueue 拒绝原因反映到 Pod status，降低“为什么 pending”的排障成本。
5598-opportunistic-batching	implementable / beta	利用批处理机会减少重复计算，面向高吞吐调度场景。

设计判断：queue 线是近期调度性能和可解释性的核心。它把 scheduler 从“失败后等待 backoff”推进到“插件能解释哪些事件真正有用”。

3. Placement / topology / affinity

代表 KEP:

KEP	状态	方案要点
895-pod-topology-spread	implemented / stable	将 spread 约束作为一等调度策略，解决跨 zone/node/domain 的副本分布问题。
1258-default-pod-topology-spread	implementable / stable	默认拓扑分布策略，减少用户未配置 spread 时的集中风险。
3022-min-domains-in-pod-topology-spread	implemented / stable	为 PodTopologySpread 增加最小 domain 语义，避免 domain 数不足时产生误判。
3094-pod-topology-spread-considering-taints	implemented / stable	skew 计算纳入 taints/tolerations，避免把实际不可用节点当成可用拓扑域。
3243-respect-pod-topology-spread-after-rolling-upgrades	implementable / beta	滚动升级后继续尊重 PodTopologySpread，关注长期分布漂移。
2249-pod-affinity-namespace-selector	implementable / stable	Pod affinity 引入 namespace selector，提高跨 namespace 选择能力。
3633-matchlabelkeys-topodaffinity	implemented / stable	affinity/anti-affinity 支持 matchLabelKeys/mismatchLabelKeys，让同一 rollout/hash 这类动态标签参与匹配。

KEP	状态	方案要点
2458-node-resource-score-strategy	implementable / beta	节点资源打分策略可配置，支撑 MostAllocated / RequestedToCapacityRatio 等资源拟合策略。
5471-enable-sla-based-scheduling	implementable / alpha	扩展 toleration operator 支撑 threshold-based placement，适合 SLA/容量阈值表达。

设计判断：placement 线要区分“节点可行性 filter”和“偏好 score”。拓扑 KEP 还会和 DRA、NUMA、PodGroup 交叉，后续应重点看 topology-aware workload scheduling。

4. Preemption / disruption

代表 KEP:

KEP	状态	方案要点
268-priority-preemption	implementable / alpha	Priority 和 preemption 的基础线，定义高优先级 Pod 通过驱逐低优先级 Pod 获得容量。
902-non-preempting-priorityclass	implemented / stable	PriorityClass 支持 non-preempting，用高优先级排序但不驱逐已有 Pod。
3280-guarantee-pdb-when-preemption-happens	implementable / alpha	抢占时更严格地保护 PDB，降低控制面为了调度破坏可用性的风险。
4832-async-preemption	implementable / beta	将抢占执行异步化，避免调度周期被删除 victim Pod 等动作阻塞。
5278-nominated-node-name-for-expectation	implementable / beta	改善 nominated node 的期望表达，减少抢占和绑定过程中的不一致。
5710-workload-aware-preemption	implementable / beta	针对 PodGroup/workload 做整体抢占决策，避免单 Pod 抢占导致 gang 仍无法整体运行。
5836-scheduler-preemption-for-ipp	implementable / alpha	面向 In-Place Pod Resize 的 scheduler preemption，把 resize 引发的资源竞争纳入调度处理。

设计判断：preemption 正在从单 Pod 局部优化转向 workload/disruption-aware 决策。跟踪时要同时看 victim 选择、PDB/可用性、异步执行和 PodGroup 语义。

5. Workload / PodGroup / gang

代表 KEP:

KEP	状态	方案要点
4671-gang-scheduling	implementable / beta / v1.37	用 Workload/PodGroup 表达一组 Pod 的整体调度。核心问题是 minCount、组装等待、整体可行性、抢占、binding 顺序和失败回退。
5710-workload-aware-preemption	implementable / beta / v1.37	与 gang scheduling 强耦合，避免先抢占再发现整个 workload 仍不可运行。
5832-decouple-podgroup-api	implementable / alpha	将 PodGroup API 从特定 workload API 中解耦，为通用 workload scheduling 做 API 基础。
6012-composite-podgroup-api	implementable / alpha	支撑组合 PodGroup，处理多组件 workload 的整体调度表达。
6089-was-controller-apis	implementable / alpha	Workload Aware Scheduling controller API，把 workload-aware 状态从 scheduler 内部扩展到控制器接口。
583-coscheduling	provisional	历史 coscheduling 方案，适合作为 gang scheduling 的前身对照。

设计判断：AI/HPC/batch 平台要重点跟踪这条线。它决定 Kubernetes 是否能原生表达“要么一组 Pod 一起运行，要么先不要占资源”的语义。

6. DRA / device-aware scheduling

代表 KEP：

KEP	状态	方案要点
5007-device-attach-before-pod-scheduled	implementable / beta	设备绑定条件进入调度路径，避免 Pod 绑定后才发现设备准备失败。
5075-dra-consumable-capacity	implementable / beta / v1.37	DRA 设备不仅是离散实例，也可能有可消费容量；调度需要理解容量消耗和剩余量。
5941-dra-shared-consumable-capacity	implementable / alpha	多个相关设备共享同一 consumable capacity，适合复杂加速器或拓扑共享资源。
5517-dra-node-allocatable-resources	implementable / alpha	把 DRA 相关 node allocatable 资源暴露给调度和容量推理。

KEP	状态	方案要点
5729-resourceclaim-support-for-workloads	implementable / beta	ResourceClaim 从单 Pod 扩展到 workload 级别，和 PodGroup/gang 产生交叉。
6080-dra-derived-attributes	provisional / alpha	用表达式从设备属性派生拓扑/匹配属性，解决不同 driver 发布属性不完全一致的问题。
5963-device-compatibility-groups	implementable / alpha	表达设备兼容组，让多设备组合调度不只依赖简单属性相等。
5055-dra-device-taints-and-tolerations	implementable / stable	将 taints/tolerations 语义扩展到设备层。
4815-dra-partitionable-devices	implementable / beta	支撑可分区设备，例如可切分的 GPU/MIG-like 资源。
4816-dra-prioritized-list	implementable / stable	设备请求支持优先级备选列表，改善多种设备配置可接受的场景。

设计判断：DRA 是 scheduling/node/autoscaling 三方交叉最密集的线。调度侧要关注 filter/score/reserve；node 侧要关注 NodePrepare/ResourceClaim status；autoscaling 侧要关注是否能推理新增节点后的设备可用性。

SIG Autoscaling 设计方案

1. HPA API / behavior

KEP	状态	方案要点
2702-graduate-hpa-api-to-GA	implemented / stable	HPA v2 API 稳定化，是多指标和 behavior 的承载面。
853-configurable-hpa-scale-velocity	implemented / stable	用 <code>behavior.scaleUp/scaleDown</code> 、 <code>policies</code> 、 <code>selectPolicy</code> 、 <code>stabilizationWindowSeconds</code> 表达扩缩速度。核心是从硬编码全局行为转向 workload 级策略。
4951-configurable-hpa-tolerance	implementable / stable / v1.37	per-HPA 或 per-direction tolerance，解决统一 10% 容忍度对大规模和敏感 workload 都太粗的问题。

设计判断：HPA 设计不只是算法，更是 API 表达能力。后续追踪应看新字段是否真正能被用户以 workload 粒度配置，并且是否有清晰 rollback 语义。

2. Metrics / target selection

KEP	状态	方案要点
117-hpa-metrics-specificity	implemented / stable	custom/external metrics 增加 label selector，避免为了 HPA 改造整个指标管道。
1610-container-resource-autoscaling	implemented / stable	HPA 可针对单个 container 的 CPU/memory，而不是 Pod 总和。解决主容器和 sidecar 资源变化不一致时的错误扩缩。
5325-hpa-pod-selection-accuracy	implementable / v1.35	通过更准确的 Pod selection strategy 减少 label selector 匹配到非目标 Pod 的风险。

设计判断：metrics 线要看“指标是否代表真正 target”。对 sidecar-heavy workload、operator 生成 Pod、跨 owner selector 的场景，Pod 选择和 container-level metrics 比算法本身更关键。

3. Scale from zero / metrics failure

KEP	状态	方案要点
2021-scale-from-zero	implementable / beta / v1.37	让 object/external metrics 支持从 0 副本启动。关键前提是无 Pod 时仍可读到外部业务信号。
5679-external-metric-fallback	implementable / alpha / v1.36	外部指标获取失败时允许 fallback，避免 metrics adapter 或外部系统短暂失败导致错误扩缩。

设计判断：这条线直接影响事件驱动和队列驱动 workload。追踪时要记录 fallback 是保守维持、默认值、还是业务指定值，以及失败信号如何暴露给用户。

4. Cluster Autoscaler integration

KEP	状态	方案要点
5030-attach-limit-autoscaler	implementable / beta / v1.37	将 CSI Volume attach limit 纳入 Cluster Autoscaler 推理，避免新增节点后仍因挂载上限无法调度。

设计判断：autoscaling 不只有 HPA。Cluster Autoscaler/Karpenter 需要模拟 scheduling 结果；DRA、volume attach、topology 和 PodGroup 都会让“加节点是否有用”变复杂。

SIG Node 设计方案

1. kubelet / CRI / runtime boundary

KEP	状态	方案要点
2040-kubelet-cri	implementable / beta	kubelet 运行时接口收敛到 CRI，是 dockershim 移除和多 runtime 支撑的前提。
2221-remove-dockershim	implemented / stable	移除 dockershim，把 runtime 支撑交给 CRI 实现，明确 kubelet/runtime 责任边界。
585-runtime-class	implemented / stable	用 RuntimeClass 把 runtime handler 选择暴露成 Pod 级 API。
4216-image-pull-per-runtime-class	implementable / alpha	镜像拉取按 runtime class 区分，适合 sandboxed runtime 或特殊 runtime 镜像链路。
2371-cri-pod-container-stats	implementable / beta / v1.37	从 cAdvisor 依赖转向 CRI 完整 Pod/Container stats，是 metrics 和 autoscaling 的基础。
5825-cri-pagination	implementable / alpha	CRI list streaming/pagination，面向大规模节点 runtime API 性能。
4033-group-driver-detection-over-cri	implemented / stable	通过 CRI 发现 cgroup driver，减少 kubelet/runtime 配置不一致。

设计判断：node 侧 runtime 线的核心是 kubelet 不再绑定 Docker 或 cAdvisor 具体实现，而是通过 CRI/RuntimeClass/stats API 形成可替换边界。

2. CPU / memory / topology managers

KEP	状态	方案要点
3570-cpumanager	implemented / stable	kubelet CPU Manager 管理 cpuset 分配，面向 Guaranteed QoS 和性能敏感 workload。
1769-memory-manager	implemented / stable	Memory Manager 通过 hint protocol 与 Topology Manager 协作，管理 NUMA/hugepages/Guaranteed memory。
693-topology-manager	implemented / stable	汇总 CPU/Memory/Device 等 hint provider 的 NUMA affinity，为本地资源对齐提供统一决策。
3545-improved-multi-numa-alignment	implemented / stable	改善多 NUMA 对齐，支撑更复杂机器拓扑。

KEP	状态	方案要点
2902-cpumanager-distribute-cpus-policy-option	implementable / beta	CPUManager 支持跨 NUMA 分散 CPU，而不是总是 pack。
4540-strict-cpu-reservation	implemented / stable	严格限制 reservedSystemCPUs 给系统 daemon/interrupt，保护 workload CPU 隔离。
4800-cpumanager-split-uncorecache	implementable / stable	CPU Manager 感知 uncore cache 拓扑，进一步细化性能隔离。
5526-pod-level-resource-managers	implementable / beta / v1.37	资源管理从 container 级向 Pod 级演进，适配 Pod-level resources。

设计判断：这条线是性能型 Kubernetes 的底座。追踪时要记录它与 scheduler 拓扑、DRA numaNode、Node Feature Discovery 和 GPU/NIC locality 的关系。

3. Pod resources / resize

KEP	状态	方案要点
1287-in-place-update-pod-resources	implemented / stable	Pod resource requests/limits 可原地更新，用 desired/actual resource status 和 CRI UpdateContainerResources 替代“改资源必须重建 Pod”。
5419-pod-level-resources-in-place-resize	implementable / beta	将 in-place resize 扩展到 Pod-level resources。
2837-pod-level-resource-spec	implementable / stable	Pod 级资源规格，改变资源请求从 container sum 到 Pod envelope 的表达方式。
5554-in-place-update-pod-resources-alongside-static-cpu-manager-policy	implementable / alpha	in-place resize 和 static CPU Manager policy 的交叉问题。
6122-configurable-scaling-delay-with-pod-resource-exposure	implementable / alpha	Pod resource exposure 后的 scaling delay 可配置，连接 node 资源变化和 autoscaling 反馈延迟。

设计判断：resize 线是 scheduler/autoscaling/node 的交叉点。它不仅是 kubelet 能不能改 cgroup，还涉及 quota、QoS class、scheduler 可行性、preemption 和 HPA/VPA 反馈。

4. DRA / device plugin / CDI

KEP	状态	方案要点
3573-device-plugin	implemented / stable	传统设备插件模型：device plugin 通过 gRPC 注册设备，kubelet Allocate 后注入 container。
4009-add-cdi-devices-to-device-plugin-api	implemented / stable	device plugin API 支持 CDI devices，把设备注入从 runtime-specific flags 转为标准 CDI spec。
4381-dra-structured-parameters	implemented / stable	DRA structured parameters 成为 DRA 主线，替代 3063 opaque control-plane negotiation 路线，使 scheduler/autoscaler 能理解设备可用性。
3063-dynamic-resource-allocation	withdrawn	早期 DRA with control-plane controller，因 Cluster Autoscaler 无法推理 opaque driver allocation、scheduler/driver 通过 apiserver 往返复杂而撤回。
4817-resource-claim-device-status	implementable / stable	ResourceClaim status 暴露设备状态，可能包含网络接口等标准化数据。
5304-dra-attributes-downward-api	implementable / beta	DRA device attributes 通过 Downward API 暴露给 Pod。
5677-dra-resource-availability-visibility	implementable / alpha	暴露 DRA resource availability，让用户和控制器能理解资源为什么不可用。
5945-dra-optional-node-preparation	implementable / alpha	DRA NodePrepare 可选化，适合不需要节点准备的设备或轻量资源。
6072-dra-standard-numanode	implementable / stable	DRA 标准化 numaNode 设备属性，连接 DRA 和 Topology Manager。

设计判断：DRA 追踪必须同时看 sig-node 和 sig-scheduling。3063 到 4381 的路线变化是关键设计教训：如果 Kubernetes 不能理解资源参数，调度和 autoscaling 就无法可靠推理。

5. Pod lifecycle / probes / sidecars

KEP	状态	方案要点
753-sidecar-containers	implemented / stable	Sidecar 作为特殊 initContainer 语义，解决启动顺序、就绪、终止顺序、Job 完成和资源计算问题。
277-ephemeral-containers	implemented / stable	debug container 进入 Pod 生命周期，用于线上排障而不改变常规 container spec。
2000-graceful-node-shutdown	implementable / beta	节点关机时 kubelet 按优雅终止流程处理 Pod。
2712-pod-priority-based-graceful-node-shutdown	implementable / beta	节点关机时按 Pod priority 分配关机顺序和时间。
5307-container-restart-policy	implementable / beta	Container 级 restart rules，细化 Pod 内不同容器的重启语义。
4438-container-restart-termination	implementable / alpha	Pod 终止期间 sidecar restart 行为，解决 sidecar termination 的边界条件。
4603-tune-crashloopbackoff	implementable / alpha	CrashLoopBackOff 参数可调，改善不同 workload 的失败重试体验。
4939-grpc-probe-with-tls	implementable / alpha	gRPC probe 支持 TLS，提升 probe 安全性。
5999-h2c-container-probes	implementable / alpha	HTTP/2 cleartext probe 支持，适配更多应用协议。

设计判断：Pod lifecycle 线的设计难点是“API 语义看似小，实际影响 scheduler、Job controller、endpoint readiness、resource accounting 和 rollback”。

6. Security / isolation

KEP	状态	方案要点
135-seccomp	implemented / stable	seccomp 支撑进入 Kubernetes。
2413-seccomp-by-default	implementable / stable	默认启用 seccomp profile，把安全默认值下沉到节点执行层。
127-user-namespaces	implemented / stable	Pod 支持 user namespace，降低容器 root 与宿主 root 绑定风险。

KEP	状态	方案要点
2033-kubelet-in-usersns-aka-rootless	implementable / beta / v1.37	kubelet rootless mode，进一步降低节点组件权限。
5607-hostnetwork-usersns	implementable / alpha	HostNetwork Pod 使用 user namespaces，处理网络特权和用户隔离交叉。
1898-hardened-exec	implementable / alpha	加固 kubelet exec endpoint，防 SSRF/代理滥用。
2862-fine-grained-kubelet-authz	implementable / stable	kubelet API 细粒度授权，减少节点 API 面的权限过宽。
2254-cgroup-v2	implementable / stable	cgroup v2 成为资源隔离基础。
5573-remove-cgroup-v1	implementable / beta	移除 cgroup v1，清理旧资源隔离路径。

设计判断：node security 不是单一 admission policy，而是 kubelet API、runtime、namespace、cgroup、exec/attach/probe 多条路径的组合。

7. Observability / node health

KEP	状态	方案要点
589-efficient-node-heartbeats	implemented / stable	降低 node heartbeat 成本，提高大规模集群节点状态同步效率。
727-resource-metrics-endpoint	implemented / stable	kubelet resource metrics endpoint，是 metrics-server 和资源观测基础。
4205-psi-metric	implemented / stable	暴露 Pressure Stall Information，补足 CPU/memory/io pressure 信号。
5394-psi-node-conditions	implementable / alpha	将 PSI 上升为 Node Conditions，便于控制面自动化响应。
4680-add-resource-health-to-pod-status	implementable / beta	Device Plugin / DRA 资源健康进入 Pod status，帮助用户理解设备异常。
5067-pod-generation	implemented / stable	Pod generation 帮助区分 spec/status 同步代际。
5328-node-declared-features	implementable / stable	Node 声明 features，和 node-feature-discovery 类项目形成对照。

设计判断：node observability 线要服务上层 scheduler/autoscaler/controller，而不只是给人看 metrics。PSI、resource health、Pod generation 都是控制面反馈闭环的一部分。

跨 SIG 设计链

设计链	关联 KEP	为什么要联动追踪
DRA structured parameters	sig-node/4381, sig-node/3063, sig-scheduling/5007, 5075, 5517, 6080, 5963, sig-autoscaling/5030	DRA 只有在 scheduler/autoscaler 能理解参数时才可生产化；opaque driver 协商路线已经被撤回。
Workload/gang scheduling	sig-scheduling/4671, 5710, 5832, 6012, 6089, sig-node/1287	AI/HPC/batch 要求整组 Pod 语义，且 resize/preemption 会改变整组可行性。
HPA metrics correctness	sig-autoscaling/117, 1610, 5325, sig-node/727, 2371, 4205	HPA 正确性依赖 node metrics、CRI stats、Pod selection 和指标 adapter。
Topology-aware placement	sig-scheduling/895, 5732, sig-node/693, 1769, 6072, 4742	拓扑从 zone/node spread 下沉到 NUMA/device，再通过 Downward API 暴露给 workload。
In-place resource changes	sig-node/1287, 5419, 5554, sig-scheduling/5836, sig-node/6122	资源从 immutable spec 变成 desired/actual 状态后，scheduler、kubelet、autoscaler 都要处理延迟和冲突。

当前 PO 跟踪清单

方向	KEP	当前状态	下一步追踪
Gang / workload scheduling	4671, 5710, 6012, 6089	v1.37 beta/alpha 活跃	跟 GenericWorkload feature gate、PodGroup API 是否稳定、与 Kueue/JobSet/LWS 的边界。
DRA scheduling	5075, 5517, 5941, 5963, 6080	v1.37 beta/alpha/provisional	跟 ResourceSlice/ResourceClaim API、scheduler perf、Cluster Autoscaler/Karpenter 推理能力。
HPA behavior	4951, 2021, 5679, 5325	v1.35-v1.37 活跃	跟 HPA API 字段是否进入 stable、metrics adapter failure 行为、scale from zero 约束。
Node DRA/device	4381, 4817, 5304, 5677, 5945, 6072	stable + v1.37 alpha	跟 driver 实现、CDI、device health status、NUMA 属性标准化。

方向	KEP	当前状态	下一步追踪
Pod resources / resize	1287, 5419, 5526, 5554, 6122, 5836	stable + v1.37 alpha/beta	跟 in-place resize 的 scheduler preemption、Pod-level resources、static CPU manager。
Node observability	2371, 4205, 5394, 4680, 5328	v1.36-v1.37 活跃	跟 CRI stats 替代 cAdvisor、PSI condition、resource health status。

后续维护规则

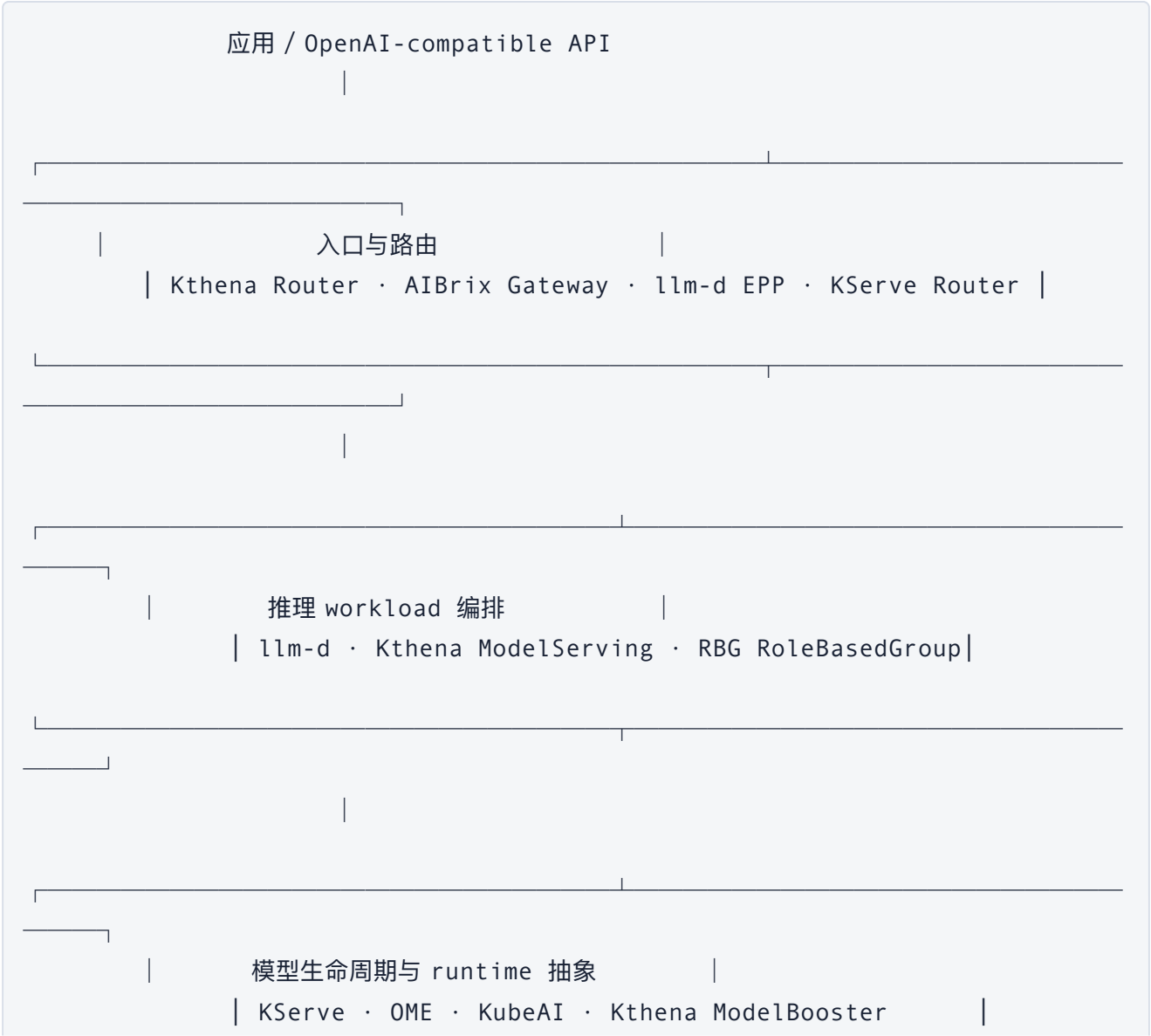
1. 新 KEP 先进入 [kubernetes-keps-design-tracking](https://kubernetes.io/docs/reference/kubernetes-api/design-tracking/) 的 SIG 表，再决定是否需要单独源摘要。
2. 每个 KEP 至少记录：KEP ID、目录、分类、status/stage/milestone、设计方案一句话、追踪优先级。
3. 如果某个 KEP 改变跨 SIG 设计链，例如 DRA、PodGroup、in-place resize、metrics pipeline，要同步更新本页的“跨 SIG 设计链”。
4. 如果某个 KEP 从 alpha/beta 到 stable，补充 graduation 条件、metrics、feature gates 和 rollback 结论。
5. 第二轮应读取 prod-readiness/sig-*/*.yaml，把每个 P0/P1 KEP 的 production readiness 信息补到 tracking 表。

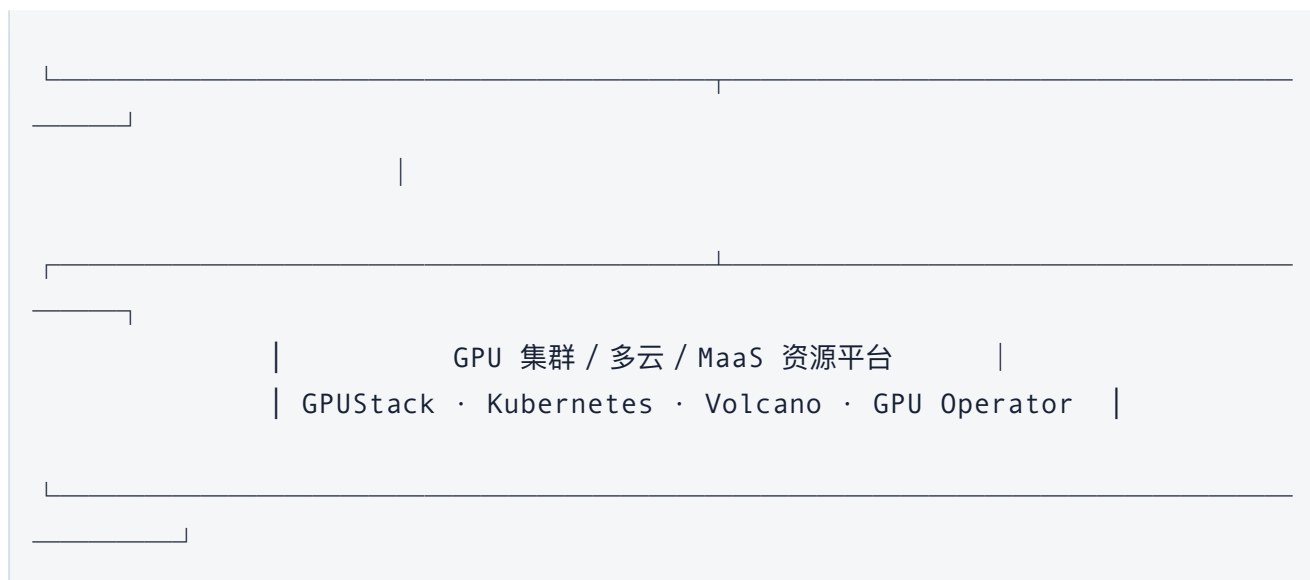
Kubernetes Serving Stack 对比：llm-d、AIBrix、KServe、KubeAI、OME、GPUStack、RBG、Kthena

原文: raw/k8s-serving-stack-comparison-2026-09-13.md · 资料范围: 各项目当前 README 与官方文档

核心判断

这八个项目不在同一个抽象层：llm-d、AIBrix、Kthena 偏分布式推理 serving；KServe、OME、KubeAI 偏模型服务 control plane；GPUStack 偏 GPU/MaaS 平台；RBG 偏多角色、有状态 workload 原语。





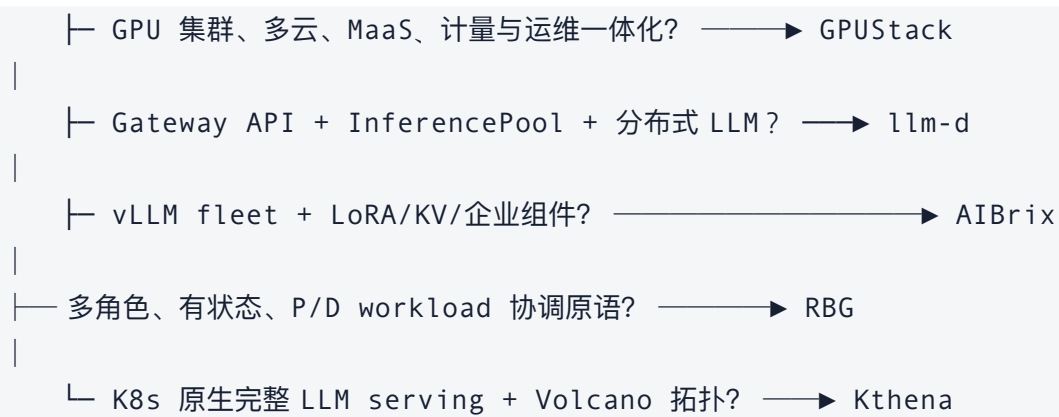
对比结论

项目	最适合	不要误解成
llm-d	Gateway API + InferencePool + 分布式 LLM	一个新的推理 engine
aibrix	vLLM fleet、LoRA、KV、企业 serving 组件	通用模型 API 标准
kserve	predictive + generative AI 的统一 K8s 平台	只针对 LLM 的专用 router
kubebai	快速部署 OpenAI-compatible 模型服务	复杂 P/D topology controller
ome	model/runtime/accelerator 生命周期抽象	KV-aware traffic plane
gpustack	GPU 集群、多云、MaaS、计量运维	轻量 K8s CRD 库
rbg	多角色、有状态、协调式 inference workload	完整 API gateway
kthenea	K8s 原生 LLM serving、路由、P/D、Volcano	只负责模型生命周期的 operator

选型路径

你首先需要什么？

- 传统 ML + GenAI 统一 API / Kubeflow 生态? —> KServe
- 轻量快速把模型变成 OpenAI API? —> KubeAI
- runtime / accelerator / model lifecycle 抽象? —> OME



这组比较补充了 llm-inference 的 K8s serving stack 层，也把 [model-serving-operator](#) 与 inference-routing 的边界拆开：有的项目管模型生命周期，有的管在线请求选择，有的管 GPU 资源，有的提供多角色 workload 原语。

Kubernetes v1.36 Sneak Peek

摘要

Kubernetes v1.36 预计 2026 年 4 月底发布，包含重要的安全强化和弃用清理。

关键变更

弃用

- `Service.spec.externalIPs` 因中间人攻击风险（CVE-2020-8554）被弃用，v1.43 移除。替代方案：[gateway-api](#)、`LoadBalancer`、`NodePort`

移除

- `gitRepo Volume Driver` 永久禁用（自 v1.11 弃用），存在 `root` 提权风险。替代：`init containers` + `git-sync`

GA 升级

- `SELinux` 卷标签加速：用 `mount -o context` 替代递归重标签，显著减少 Pod 启动时间

退役

- `ingress-nginx` 于 2026-03-24 正式退役，不再维护

Kubernetes Workload Automation

Kubernetes workload automation 指在原生 Pod/Deployment/StatefulSet 之上，用 CRD/controller、队列、弹性、分布式 workload API、发布治理、可观测和控制器运行边界，把应用生命周期从“创建 Pod”提升到“表达业务 workload 意图”。

这个概念页是整体框架，不按单个项目或单个 OpenKruise 维度拆概念。[openkruise-kruise](#)、[openkruise-rollouts](#)、[kruise-game](#)、[kueue](#)、[karpenter](#)、[lws](#)、[jobset](#) 和 [model-serving-operator](#) 都只是它在不同 workload 场景里的实现样本。

主要层次

层次	代表	关注点
Workload enhancement	openkruise-kruise	CloneSet、Advanced StatefulSet、SidecarSet、WorkloadSpread、ImagePullJob。
Release governance	openkruise-rollouts , gitops , gateway-api	分批、暂停、推进、回滚、流量切分和发布状态机。
Specialized workload	kruise-game , lws , jobset	游戏服务器、leader/worker、分布式 ML/HPC job 等业务语义。
Queueing / admission	kueue	多租户资源配额、入队、抢占和 workload admission。
Capacity automation	karpenter	pending pods 到节点容量的弹性补充和 consolidation。
Model serving	model-serving-operator	Model/InferenceService/LLMISvc 等 AI workload 生命周期。
Observability	kruise-state-metrics , metrics-server , prometheus-adapter	workload/API 对象状态、资源指标和 custom metrics。
Controller operation boundary	controllermesh , controller-runtime , cloud-native-security	控制器权限、运行实例、对象范围、故障影响面和审计。

为什么重要

AI Infra、游戏服务器、大规模在线服务和批处理平台都不只是“跑一个 Deployment”。它们需要表达批次、队列、角色、容量、镜像预热、拓扑、回滚、指标、成本和控制器边界。把这些能力放进 Kubernetes API 后，平台才能复用 RBAC、watch、status、events、GitOps 和 controller 模型。

和项目实体的关系

项目类别	实体	在整体概念中的位置
OpenKruise workload enhancement	openkruise-kruise	把原生 workload 扩展成更贴近生产运维的 API。
OpenKruise release governance	openkruise-rollouts	把上线过程拆成可推进、可暂停、可回滚的控制状态。
Specialized workload	kruise-game , lws , jobset	把游戏服务器、leader/worker、分布式 job 这类业务结构变成 Kubernetes API。
Resource admission / capacity	kueue , karpenter	一个管 workload 能否入场，一个管节点容量如何补足。
Model workload	model-serving-operator , llm-d	把模型、runtime、endpoint、batch、autoscaling 放进集群控制面。
Operations / reliability	kruise-state-metrics , controllermesh	一个补状态可观测，一个补控制器运行边界。

选型提示

先判断你要自动化的是哪一层：

- workload 对象本身不够表达业务：看 [openkruise-kruise](#)、[kruise-game](#)、[lws](#)、[jobset](#)。
- 发布过程需要受控推进：看 [openkruise-rollouts](#)，并放到 [gitops](#) 和 [gateway-api](#) 的发布/流量治理链路中评估。
- 多租户 batch/AI/HPC 需要排队入场：看 [kueue](#)。
- 节点容量和成本是瓶颈：看 [karpenter](#)。
- 模型服务生命周期是主问题：看 [model-serving-operator](#)。
- 控制器权限、故障半径或多 operator 管理是主问题：看 [controllermesh](#)，并结合 [cloud-native-security](#) 与 [k8s-core-controller-map](#) 判断运行边界。

Kubespray

Kubespray 用 Ansible inventory/roles 部署生产可用 Kubernetes 集群，覆盖 kubeadm、network plugin、etcd、HA 和云/裸金属差异。详见 [src-kubespray-architecture](#)。

架构边界

Cluster API 是 K8s-native 声明式生命周期；Kubespray 是 Ansible-based 集群安装/升级自动化。

什么时候用

场景	判断
需要 计算 / 集群部署 能力	适合，Kubespray 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , cloud-native-security 组合。

核心组件

- Inventory: hosts/group_vars/cluster config
- Ansible roles: kubeadm/etcd/network/storage/addons
- Playbooks: cluster.yml/upgrade/reset
- Provider support: bare metal/cloud/on-prem

选型提示

把 Kubespray 放在 计算 / 集群部署 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

Kubespray 架构与设计思路分析

原文: raw/kubespray-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kubespray> · 优先级 P0

一句话定位

Kubespray 用 Ansible inventory/roles 部署生产可用 Kubernetes 集群，覆盖 kubeadm、network plugin、etcd、HA 和云/裸金属差异。

核心架构图





模块分层

层 / 模块	职责
Inventory	hosts/group_vars/cluster config
Ansible roles	kubeadm/etcd/network/storage/addons
Playbooks	cluster.yml/upgrade/reset
Provider support	bare metal/cloud/on-prem

关键数据流



设计决策与哲学

- 补齐 计算 / 集群部署 维度：Kubespray 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Cluster API 是 K8s-native 声明式生命周期；Kubespray 是 Ansible-based 集群安装/升级自动化。
- 选型价值：它应和 [kubernetes](#), [cloud-native-security](#) 一起看，而不是孤立评估。

相关页面

- [kubespray](#)
- [kubernetes](#)
- [cloud-native-security](#)

kubewall

kubewall 是 single-binary Kubernetes dashboard，仓库分为 Go backend、client、charts 和 media。它在当前 wiki 中的价值是“AI integration 进入 K8s dashboard 管理体验”的对照样本。详见 [src-kubewall-architecture](#)。

架构边界

kubewall 的主体仍是 dashboard 基础设施：资源浏览、事件、port-forward、Helm 安装和前端 UI。AI 是增强层，不是完整 agent framework。

关键设计

- backend/cmd、backend/routes、backend/handlers 组成 Go dashboard server。
- backend/event 与 backend/portforward 承接集群事件和端口转发。
- client/src 是前端控制台。
- charts/kubewall 降低部署门槛。

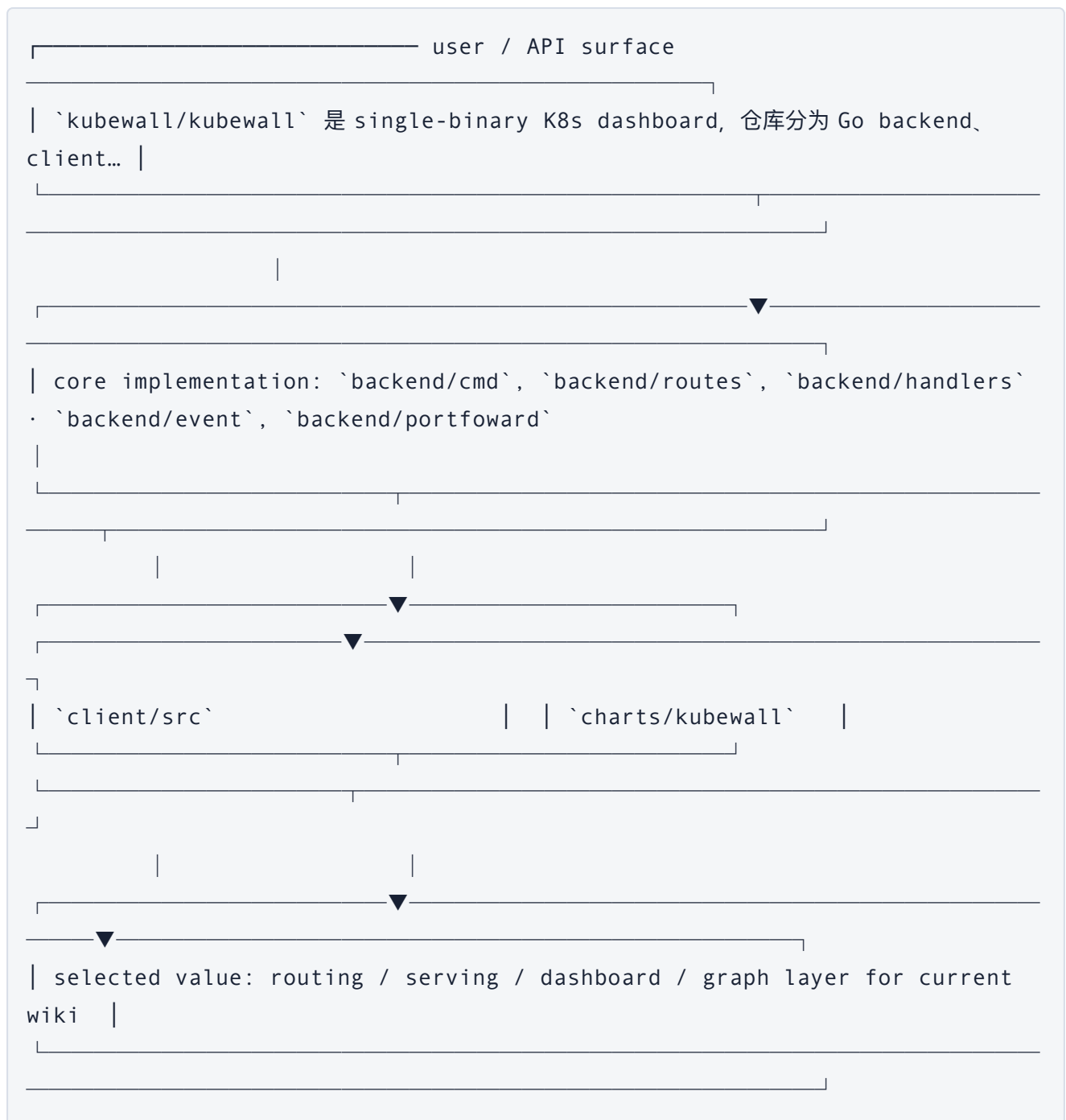
选型判断

需要轻量 dashboard 对照时看 kubewall；需要更强 AI/MCP 控制台形态看 [k8m](#)；需要命令行自然语言入口看 [kubectl-ai](#)。

kubewall 架构与设计思路分析

kubewall/kubewall 是 single-binary K8s dashboard，仓库分为 Go backend、client、charts 和 media。它在 P1 中的价值是“AI integration 进入 K8s dashboard 管理体验”的对照样本，而不是完整 agent framework。

核心架构图



模块分层

层/目录	责任
backend/cmd, backend/routes, backend/handlers	Go dashboard server、路由和 handler。
backend/event, backend/portfoward	集群事件和端口转发。
client/src	前端 UI。
charts/kubewall	Helm 安装。

关键数据流

1. 用户通过 dashboard 浏览资源。
2. backend 代理 K8s API、事件、port-forward 等能力。
3. AI 功能作为控制台辅助层接入。

设计决策

- single-binary/Helm 部署优先，降低运维门槛。
- 核心仍是 dashboard，AI 是增强层。
- 适合作为 k8m 的更轻对照。

对比定位

和 k8m 相比，kubewall 更 dashboard 基础设施；和 kubectl-ai 相比，它牺牲 CLI 灵活性换 UI 可视化。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Kueue

Kubernetes-native Job Queueing，用 ClusterQueue/LocalQueue/Workload/ResourceFlavor 把 batch、AI/HPC 和多租户资源配额做成 admission control。详见 [src-kueue-architecture](#)。

证据说明：2026-09-25 核验的官方默认分支快照为 [2a766717037a](#)，该 commit 不代表 release。[src-kueue-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目职责见 [kubernetes-workload-gang-scheduling-design](#)，调度器流程见 [kubernetes-scheduler-core-design](#)。

在 M5-B Workload / Scheduling 地图中的位置

在上述快照中，Kueue 通过 Workload、队列、Cohort 和 ResourceFlavor 管理 admission、quota 与 flavor assignment，并按配置处理 Topology 和 AdmissionCheck。[jobset](#)、[lws](#) 通过各自集成进入准入路径；Kueue 不选择最终 Node，Pod placement/binding 由 kube-scheduler 及配置的 [scheduler-plugins](#) 完成，节点容量不足则由 [karpenter](#) 等容量控制器处理。

架构边界

和 kube-scheduler 不同，Kueue 先决定 workload 是否能入场；真正的 Pod placement 仍由 scheduler 做。

什么时候用

场景	判断
需要 调度 / 队列 能力	适合，Kueue 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-inference, batch-inference, model-serving-operator 组合。

核心组件

- API: ClusterQueue / LocalQueue / Workload / ResourceFlavor
- Controller: workload admission, quota accounting, preemption
- Integrations: Job, JobSet, RayJob, MPIJob, PyTorchJob 等 batch workload
- Scheduler-like logic: cohort borrowing、fair sharing、flavor assignment

选型提示

把 Kueue 放在 调度 / 队列 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

Kueue 架构与设计思路分析

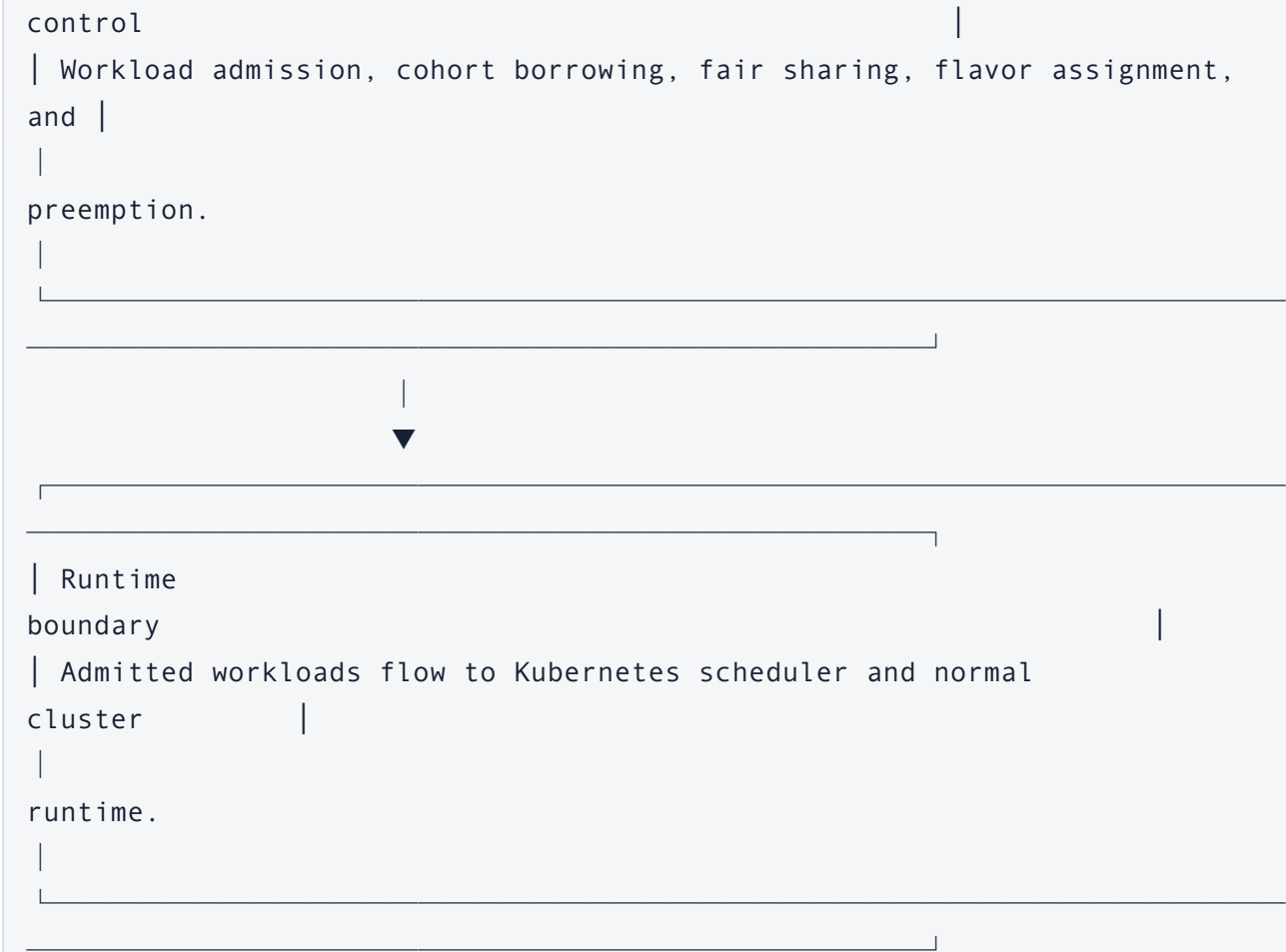
原文: raw/kueue-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kueue> · 优先级 P0

一句话定位

Kubernetes-native Job Queueing, 用 ClusterQueue/LocalQueue/Workload/ResourceFlavor 把 batch、AI/HPC 和多租户资源配额做成 admission control。

核心架构图

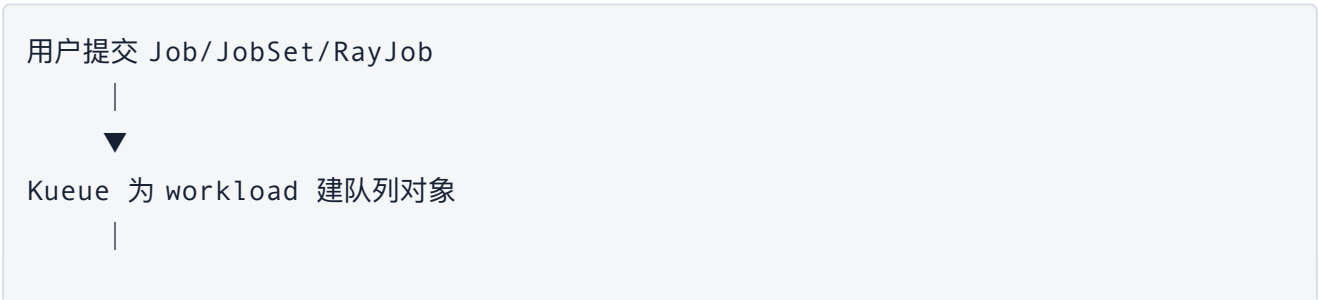


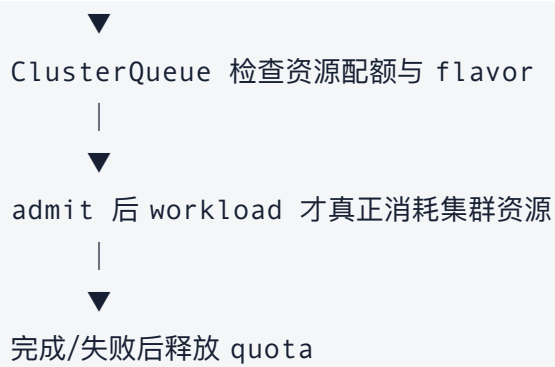


模块分层

层 / 模块	职责
API	ClusterQueue / LocalQueue / Workload / ResourceFlavor
Controller	workload admission, quota accounting, preemption
Integrations	Job, JobSet, RayJob, MPIJob, PyTorchJob 等 batch workload
Scheduler-like logic	cohort borrowing、fair sharing、flavor assignment

关键数据流





设计决策与哲学

- 补齐 调度 / 队列 维度: Kueue 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: 和 kube-scheduler 不同, Kueue 先决定 workload 是否能入场; 真正的 Pod placement 仍由 scheduler 做。
- 选型价值: 它应和 llm-inference, batch-inference, [model-serving-operator](#) 一起看, 而不是孤立评估。

相关页面

- [kueue](#)
- [kubernetes](#)
- llm-inference
- batch-inference
- [model-serving-operator](#)

Kustomize

Kustomize 用 overlay/patch/transformer 管理 Kubernetes YAML 差异，是 kubectl 原生支持的配置定制工具链。详见 [src-kustomize-architecture](#)。

架构边界

Helm 是模板和 package；Kustomize 是 YAML transformer，适合 GitOps 中的环境 overlay。

什么时候用

场景	判断
需要 配置管理 能力	适合，Kustomize 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 gitops , kubernetes 组合。

核心组件

- Resources/bases/overlays
- Transformers: namePrefix/labels/images/namespace
- Patches: strategic merge/json6902/replacements
- Generators: ConfigMap/Secret

选型提示

把 Kustomize 放在 配置管理 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

Kustomize 架构与设计思路分析

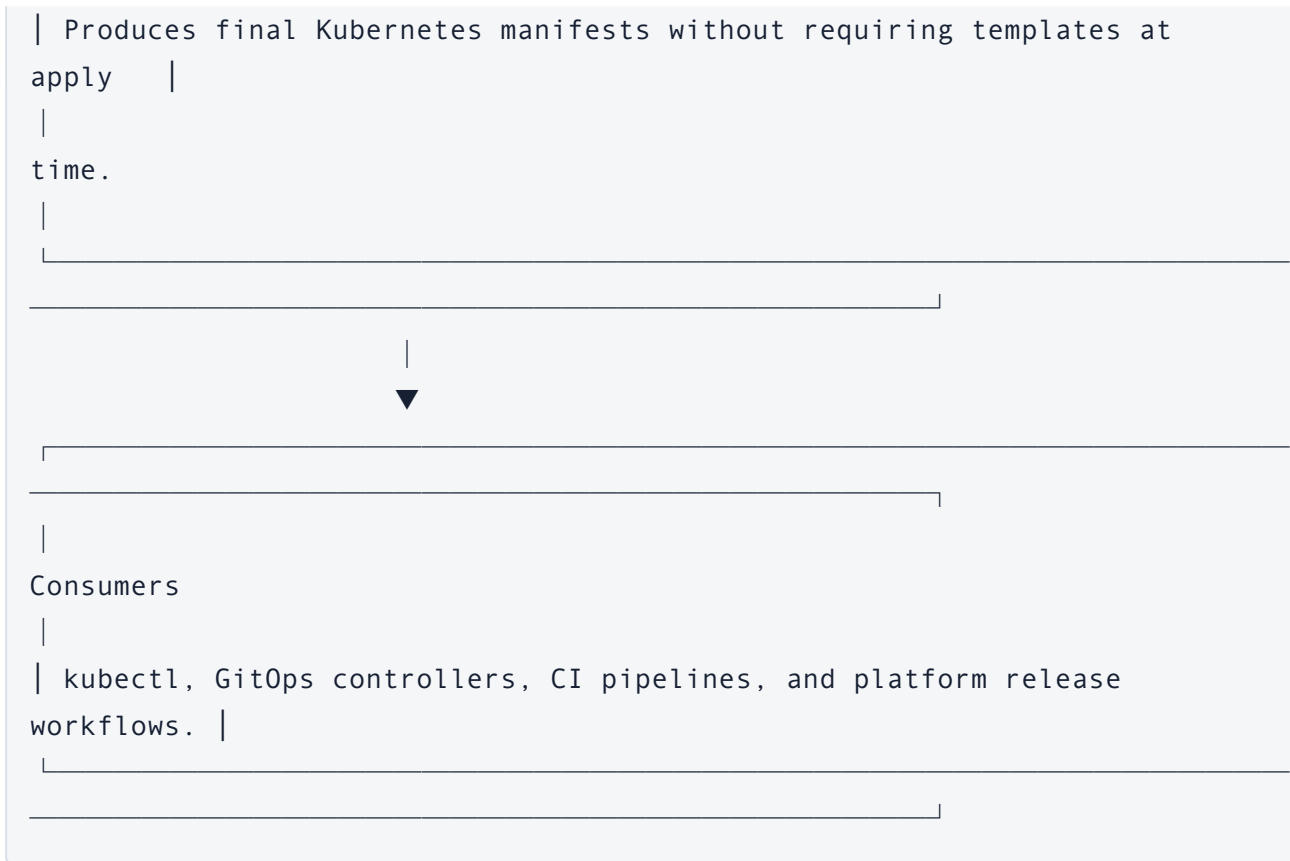
原文: raw/kustomize-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kustomize> · 优先级 P1

一句话定位

Kustomize 用 overlay/patch/transformer 管理 Kubernetes YAML 差异，是 kubectl 原生支持的配置定制工具链。

核心架构图





模块分层

层 / 模块	职责
Resources/bases/overlays	Resources/bases/overlays
Transformers	namePrefix/labels/images/namespace
Patches	strategic merge/json6902/replacements
Generators	ConfigMap/Secret

关键数据流





设计决策与哲学

- 补齐 配置管理 维度：Kustomize 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Helm 是模板和 package；Kustomize 是 YAML transformer，适合 GitOps 中的环境 overlay。
- 选型价值：它应和 [gitops](#), [kubernetes](#) 一起看，而不是孤立评估。

相关页面

- [kustomize](#)
- [kubernetes](#)
- [gitops](#)

KWOK

KWOK 是 Kubernetes WithOut Kubelet，用 fake nodes/pods 模拟大规模集群，适合调度、控制器和 scalability 测试。详见 [src-kwok-architecture](#)。

架构边界

kind 提供真实小集群；KWOK 提供便宜的大规模对象模拟。

什么时候用

场景	判断
需要 大规模集群模拟 能力	适合，KWOK 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , model-serving-operator 组合。

核心组件

- kwok controller: fake kubelet behavior
- kwokctl: cluster lifecycle
- Stage/Configuration: pod/node condition transitions
- Integrations: kind/kube-apiserver tests

选型提示

把 KWOK 放在 大规模集群模拟 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

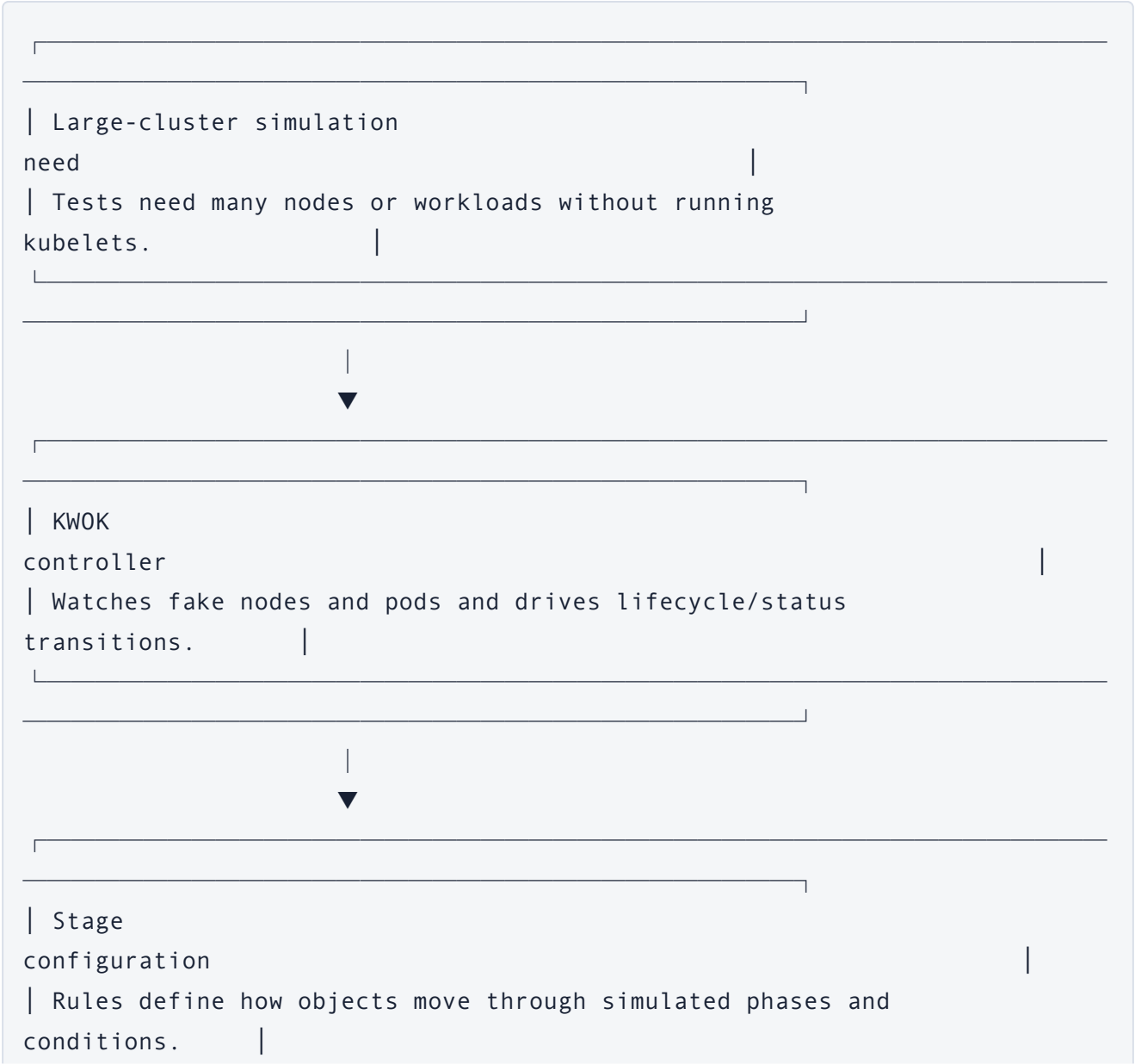
KWOK 架构与设计思路分析

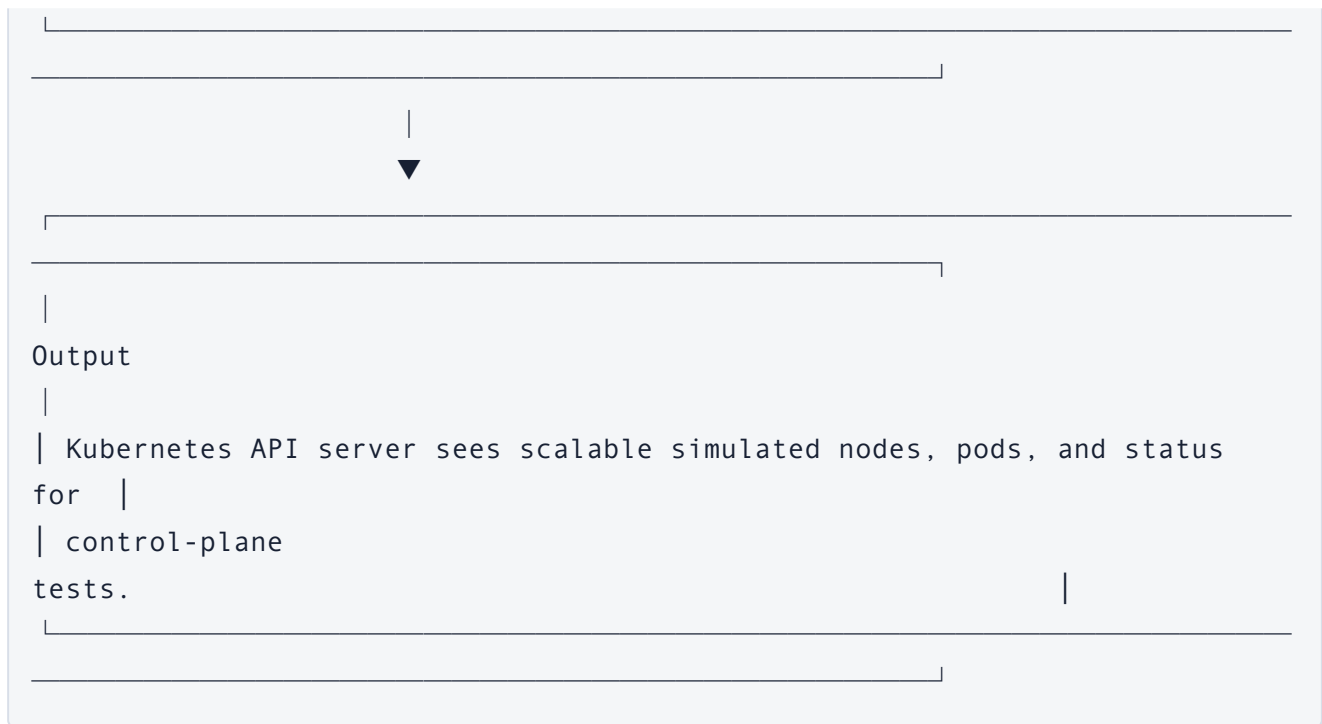
原文: raw/kwok-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/kwok> · 优先级 P1

一句话定位

KWOK 是 Kubernetes WithOut Kubelet, 用 fake nodes/pods 模拟大规模集群, 适合调度、控制器和 scalability 测试。

核心架构图

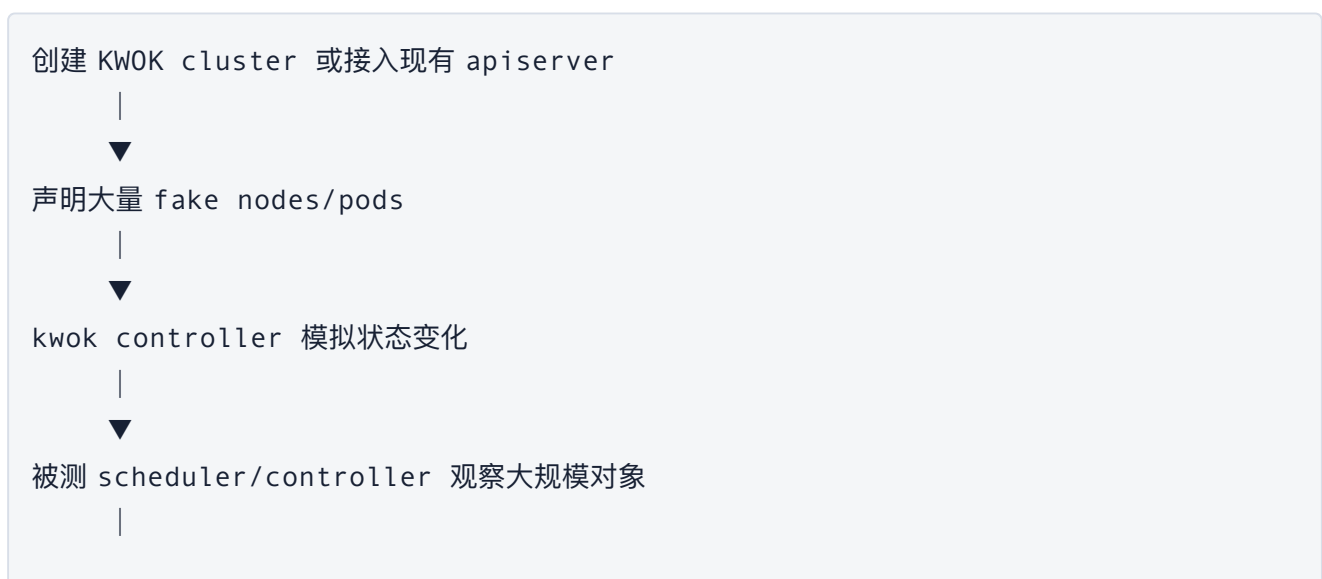




模块分层

层 / 模块	职责
kwok controller	fake kubelet behavior
kwokctl	cluster lifecycle
Stage/Configuration	pod/node condition transitions
Integrations	kind/kube-apiserver tests

关键数据流





收集性能和行为结果

设计决策与哲学

- 补齐 大规模集群模拟 维度：KWOK 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：kind 提供真实小集群；KWOK 提供便宜的大规模对象模拟。
- 选型价值：它应和 [kubernetes](#), [model-serving-operator](#) 一起看，而不是孤立评估。

相关页面

- [kwok](#)
- [kubernetes](#)
- [model-serving-operator](#)

LeaderWorkerSet

LeaderWorkerSet 用一组 leader/worker Pods 表达一个复制单元，适合 LLM inference、分布式 serving 和需要稳定 group 语义的 workload。详见 [src-lws-architecture](#)。

证据说明：2026-09-25 核验的官方默认分支快照为 [d4f1525f15a4](#)，该 commit 不代表 release。[src-lws-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目职责见 [kubernetes-workload-gang-scheduling-design](#)，调度器流程见 [kubernetes-scheduler-core-design](#)。

在 M5-B Workload / Scheduling 地图中的位置

在上述快照中，`leaderworkerset.x-k8s.io/v1` 管理同类模型服务的 leader/worker 组及其恢复、扩缩与发布；`disaggregatedset.x-k8s.io/v1` 进一步以角色和 slices 组合子 LWS，协调分离推理的多角色生命周期。它们与 [jobset](#) 的批作业集合职责不同，也不管理 quota admission：准入通过 [kueue](#) 集成，最终 placement 由 kube-scheduler 及配置的 [scheduler-plugins](#) 完成。DS 的跨角色编排不等于整组原子准入。

架构边界

JobSet 面向作业集合；LWS 面向长期运行的 leader/worker 服务复制单元。

什么时候用

场景	判断
需要 分布式 workload API 能力	适合，LeaderWorkerSet 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-inference, batch-inference, kueue 组合。

核心组件

- API: LeaderWorkerSet CRD
- Controller: replica group rollout/status
- Pod template: leader/worker roles
- Integrations: serving/HPC/AI workload

选型提示

把 LeaderWorkerSet 放在 分布式 workload API 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

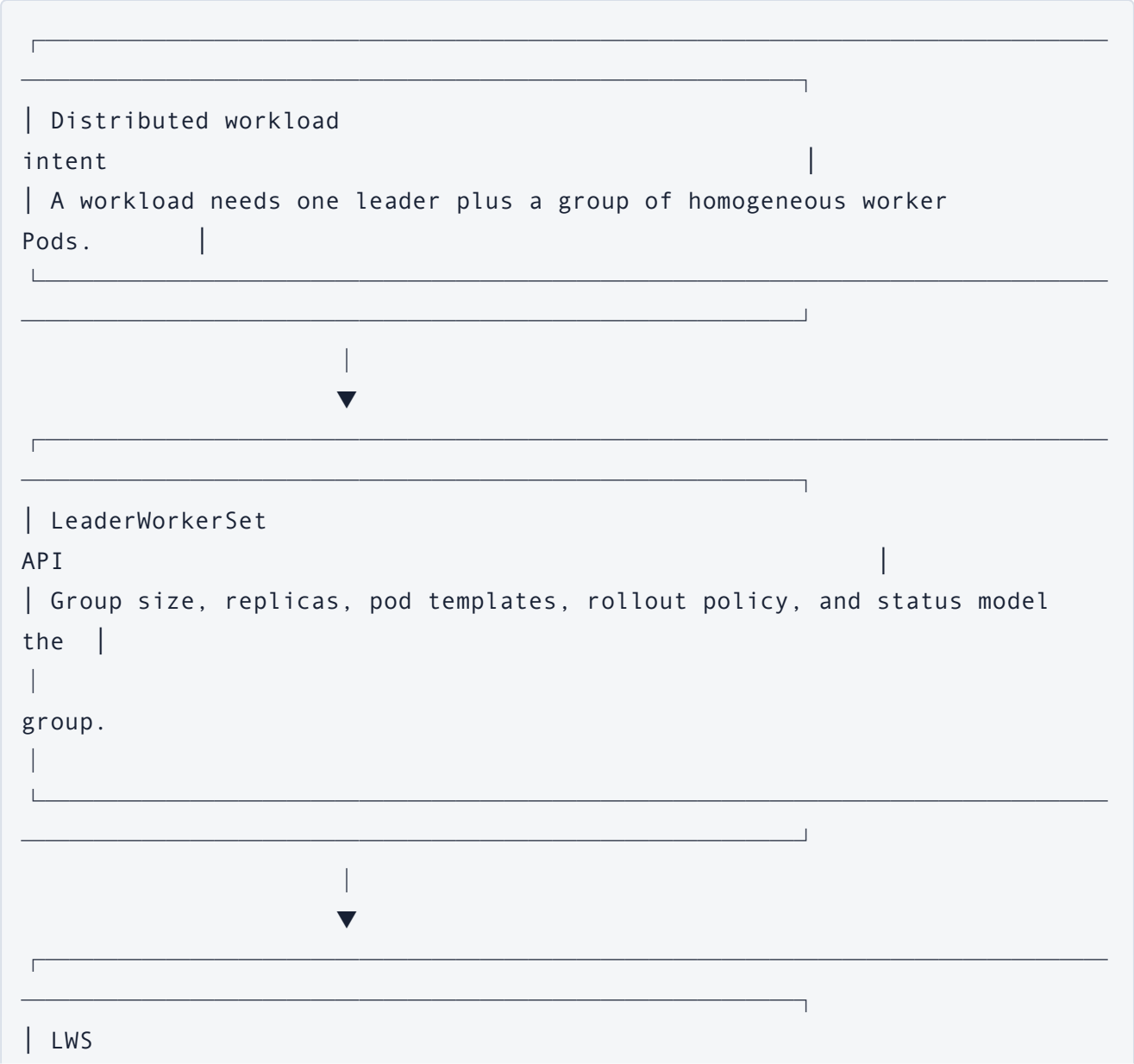
LeaderWorkerSet 架构与设计思路分析

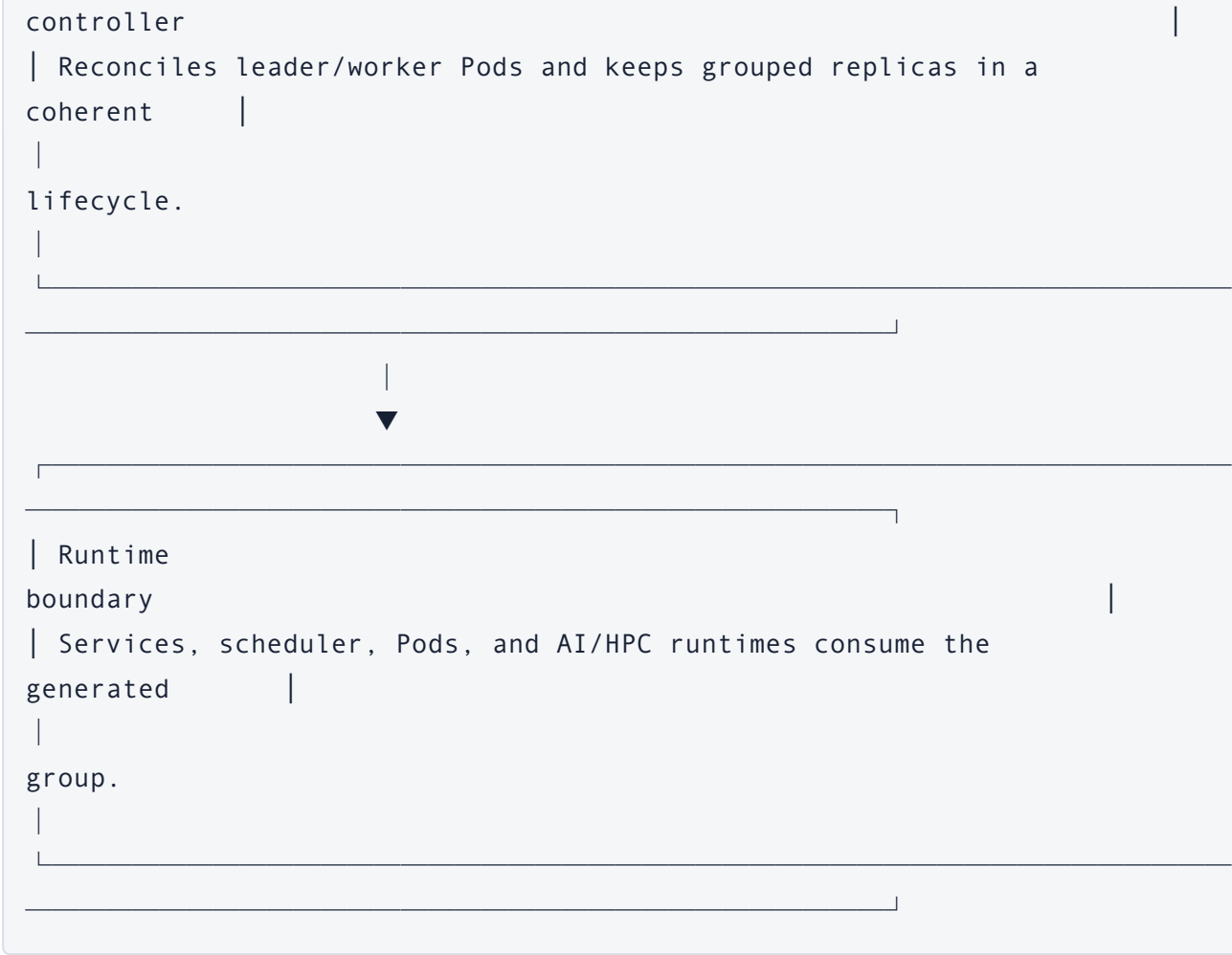
原文: raw/lws-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/lws>
lws · 优先级 P1

一句话定位

LeaderWorkerSet 用一组 leader/worker Pods 表达一个复制单元，适合 LLM inference、分布式 serving 和需要稳定 group 语义的 workload。

核心架构图

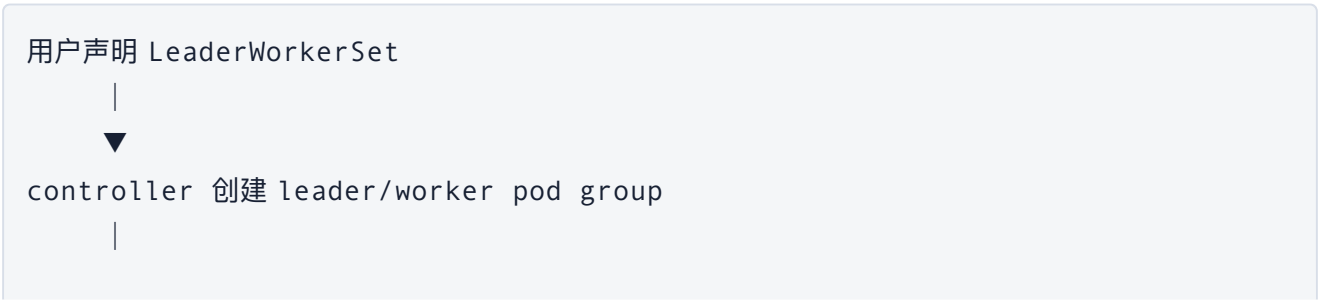


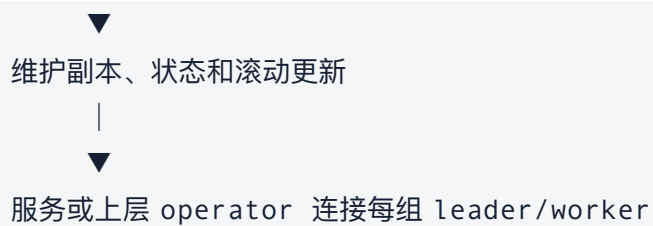


模块分层

层 / 模块	职责
API	LeaderWorkerSet CRD
Controller	replica group rollout/status
Pod template	leader/worker roles
Integrations	serving/HPC/AI workload

关键数据流





设计决策与哲学

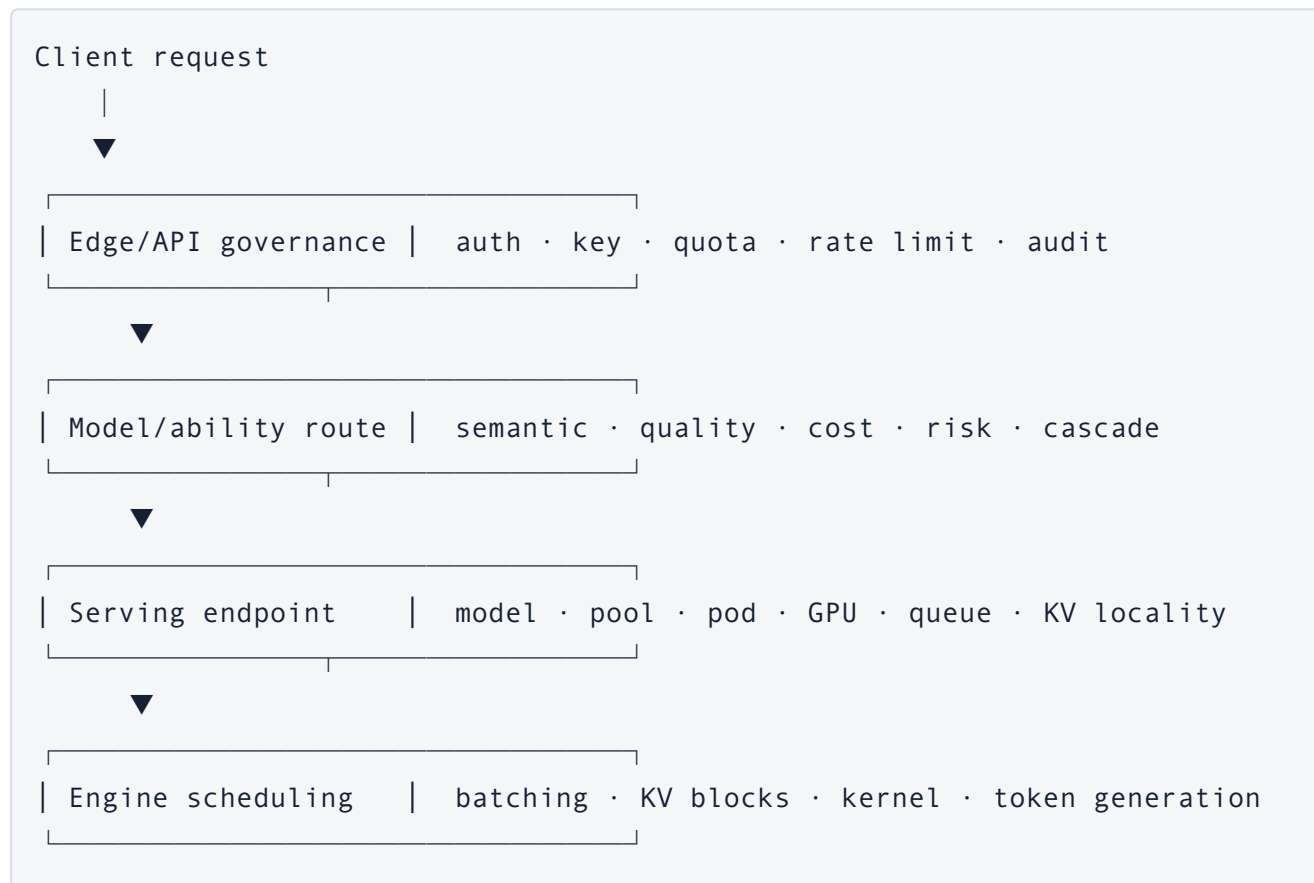
- 补齐 分布式 workload API 维度：LeaderWorkerSet 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：JobSet 面向作业集合；LWS 面向长期运行的 leader/worker 服务复制单元。
- 选型价值：它应和 llm-inference, batch-inference, [kueue](#) 一起看，而不是孤立评估。

相关页面

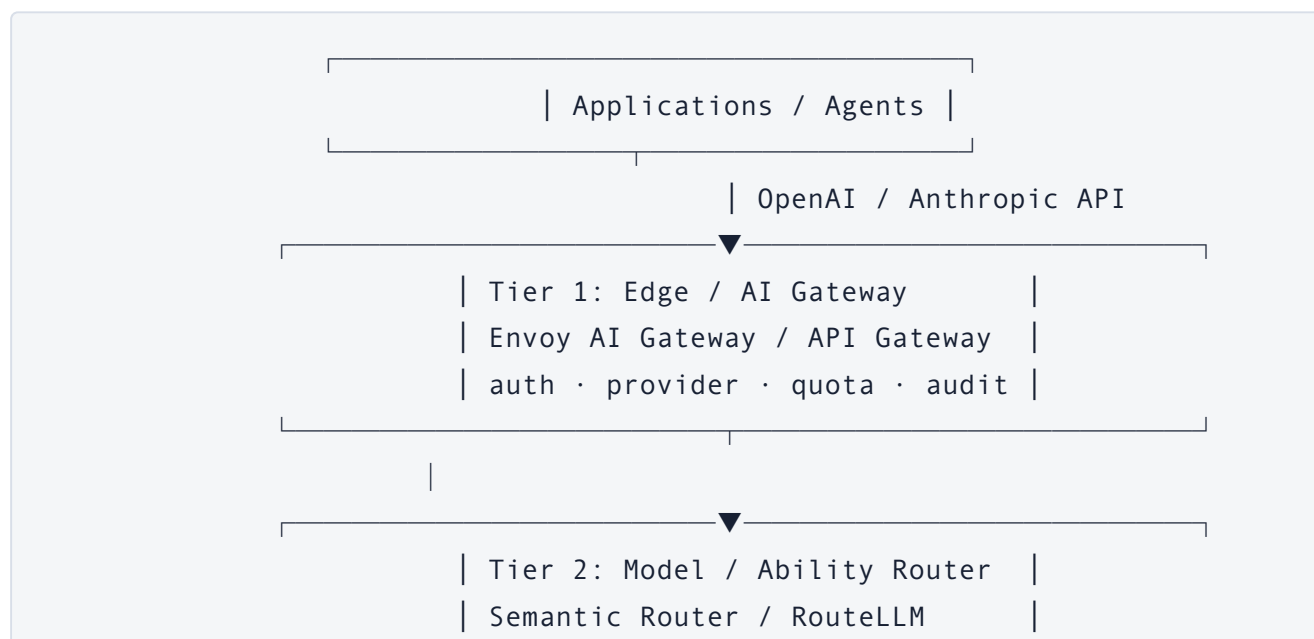
- [lws](#)
- [kubernetes](#)
- llm-inference
- batch-inference
- [kueue](#)

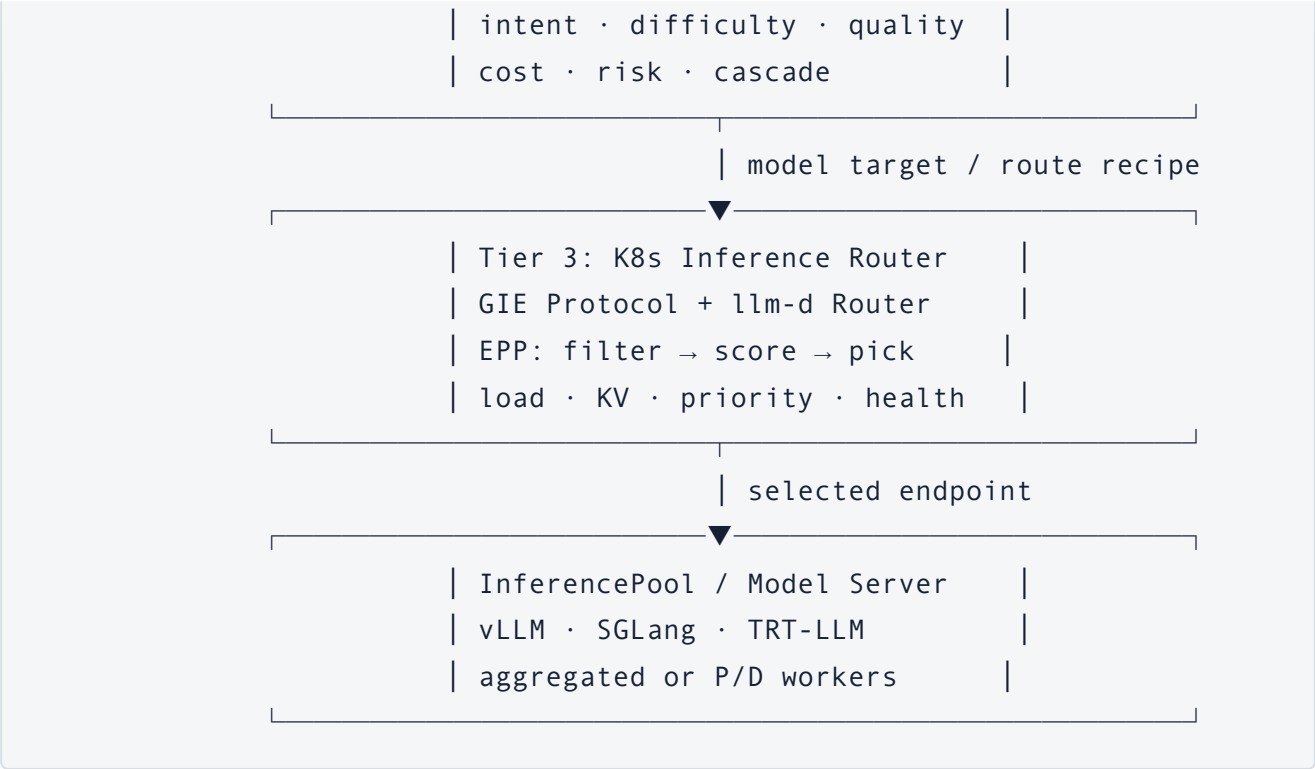
LLM 网关与路由层对比

先分清四类决策



统一架构图



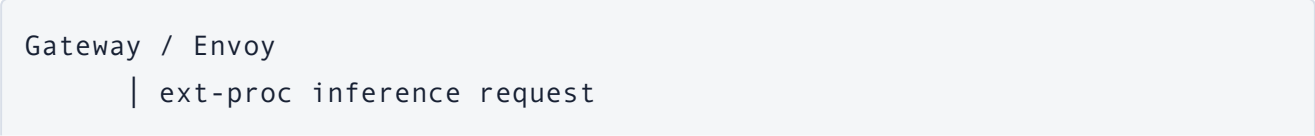


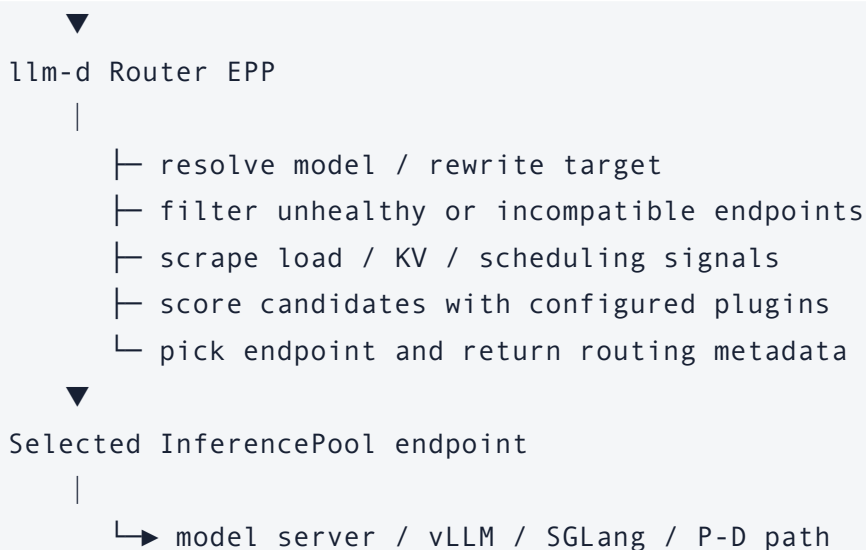
项目定位

项目	决策层	核心输出	Kubernetes 边界
llm-d-router	endpoint/pod/ worker	selected endpoint + routing metadata	EPP/ext-proc、Gateway API、 InferencePool
semantic-router	model/ability/ recipe	model、cascade、 capability path	可部署在 K8s，但不是 GIE 协 议本身
routellm	strong/weak model	cheaper 或 stronger model	SDK/OpenAI-compatible server，不是 operator
gateway-api-inference- extension	inference endpoint protocol	inference endpoint pick	Gateway API 的推理扩展
envoy-ai-gateway	edge/provider governance	provider/model upstream	Envoy Gateway + two-tier gateway

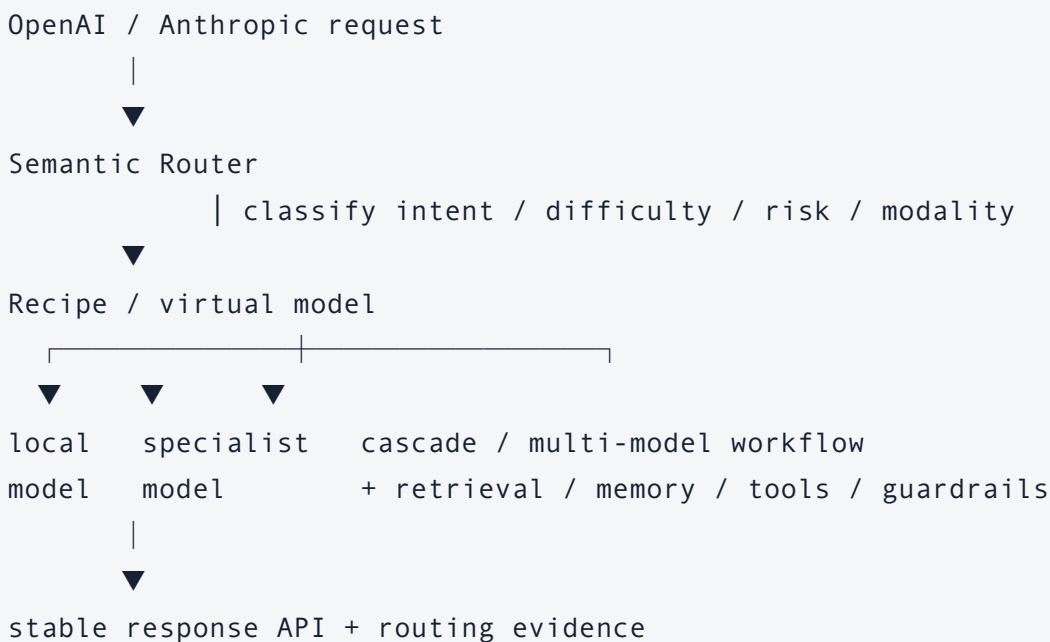
五条请求流程图

llm-d-router

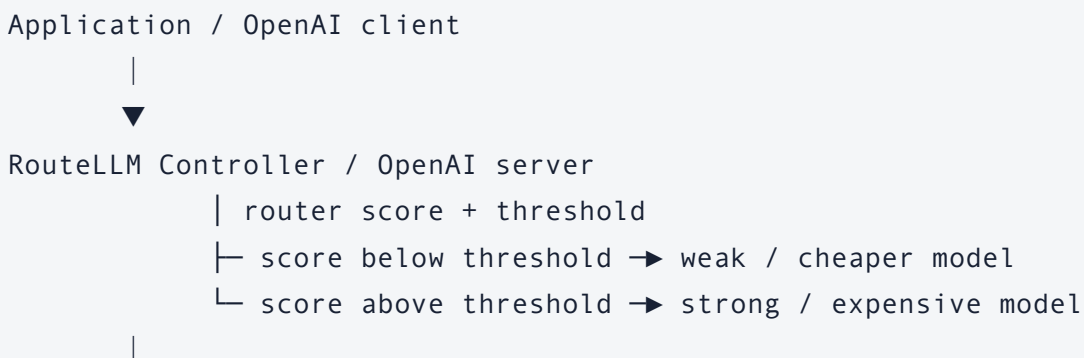


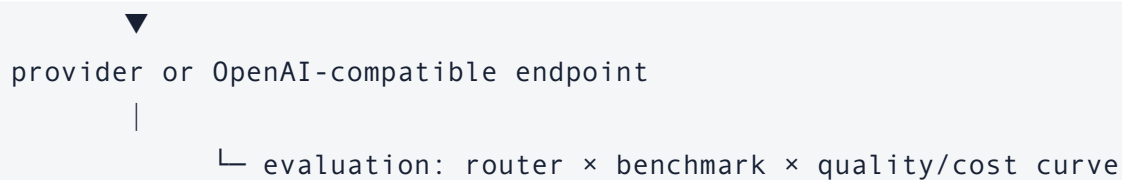


Semantic Router

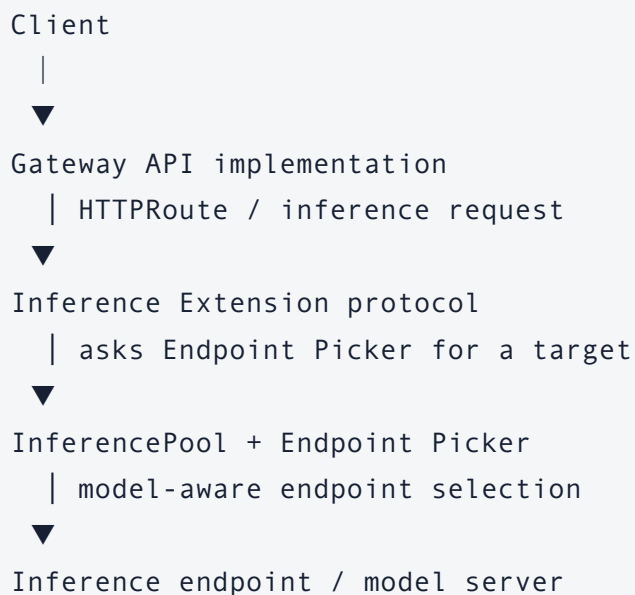


RouteLLM





Gateway API Inference Extension



Envoy AI Gateway



选型结论

- provider、key、quota、限流和审计: [envoy-ai-gateway](#)。
- 语义、风险、能力和质量-成本模型选择: semantic-router / routellm。

- KV、队列、GPU、健康和 InferencePool endpoint 选择: [gateway-api-inference-extension](#) / llm-d-router。
- 多层组合时, 必须记录每一层的 route decision、模型名、endpoint、策略版本和失败原因。

相关页面

- [inference-routing](#)
- [ai-gateway](#)
- [llm-inference](#)
- [src-k8s-gateway-routing-comparison](#)

llm-d

llm-d 是 Kubernetes-native distributed inference stack，围绕 Gateway API/Inference Extension、EPP、InferencePool 和 model server 组织，并扩展 KV cache、P/D、autoscaling、benchmark、batch 与 simulator。

解决的问题

它把单机 vLLM/SGLang engine 提升为可在 Kubernetes 上运行的分布式 LLM 服务：请求要按 endpoint 状态、KV locality 和推理负载选择，P/D worker 要能协同，容量还需依据可观测指标扩缩。

核心边界

llm-d 是 serving ecosystem，不是新的推理 engine，也不是单一 operator。Gateway/EPP 负责入口和 endpoint picking，InferencePool 表达后端集合，KV/P/D 组件补充性能关键路径。[llm-d-workload-variant-autoscaler](#) 已是 deprecated 的历史设计；新部署应按当前官方指南采用 [EPP metrics → KEDA Prometheus scaler → HPA](#)。

适用场景

适合已经采用 Gateway API、希望标准化多模型/多 endpoint 入口，并需要分布式推理性能的 Kubernetes 平台。若只需单模型快速部署，[kubecai](#) 更轻；若需要更深的 engine runtime/P-D/KV 一体化，可比较 [dynamo](#)；若需要 GPU 集群 MaaS，可比较 [gpustack](#)。

详见 [src-llm-d-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

在 M4 模块地图中的位置

llm-d 位于 Kubernetes routing/serving stack 层，通过 Proxy/EPP 和 InferencePool 将 Gateway 流量、路由信号与 Model Server Pod 连接起来。职责边界见 llm-inference-serving-project-map，与 engine 及外围组件的组合选择见 llm-serving-engine-selection-map。

llm-d Planner

[llm-d-planner](#) 是 llm-d-incubation 中的部署规划与推荐平台，把业务需求转换成模型/GPU/SLO/成本决策和 Kubernetes YAML。它处于 serving runtime 上游，不在在线 Router 请求路径中。

解决的问题

应用团队通常知道用户数、场景和预算，却不知道应该选什么模型、GPU、上下文长度、replica 数和 TTFT/ITL 目标。Planner 将这些决策从人工 trial-and-error 变成可解释、可编辑、可部署的流程。

架构图



方法与集成

Planner 使用可替换 LLM provider 做 intent extraction，把 use case 映射到 traffic profile 和 TTFT/ITL/E2E SLO；Capacity Planner 估算 weights、KV、activation、GPU 数和并发；GPU Recommender 估算不同硬件的性能；Recommendation Service 按 quality、cost、latency 生成 Best Quality、Lowest Cost、Lowest Latency、Balanced 视图；Configuration Service 生成 KServe/vLLM/HPA/ServiceMonitor YAML。它还集成 vLLM simulator 支持无 GPU 开发。

当前定位偏 POC/平台入口，生产化仍需关注配置安全、认证、多租户、数据版本和实际部署 telemetry 回流。

llm-d Router 架构与设计思路分析

llm-d/llm-d-router 是 llm-d 的智能入口。它通过 Envoy/ext-proc 或 Gateway API Inference Extension 接入流量，核心 EPP 用 filters、scorers、scrapers 和 scheduling profiles 对 InferencePool pods 做过滤/打分/选择；同时包含 disaggregation sidecar，协调 encode/prefill/decode 多阶段推理。

核心架构图



模块分层

层/目录	责任
cmd/epp, pkg/epp/server/**	EPP server 和 controller manager。
pkg/epp/config, pkg/epp/framework/ plugins/**	EndpointPickerConfig、plugins、scheduling profiles。
pkg/epp/datalayer/**	K8s binding、InferencePool endpoint graph、runtime state。
pkg/common/envoy/**	Envoy request/response/header/metadata/chunking 辅助。

关键数据流

1. proxy 收到请求后把 headers/body stream 给 EPP ext-proc。
2. EPP 解析请求模型/priority/profile，加载 EndpointPickerConfig，按 filters 排除不合适 endpoints。
3. scorers 访问 scrapers/datastore 中的 load、KV、metrics 等信号，按权重打分；picker 选择最高分 pod。

设计决策

- plugin 化 filters/scorers/scrapers 是 Router 的核心：新增路由策略不改主循环。
- EPP 通过 Gateway API/GIE 与 K8s 标准对齐，避免自定义所有流量原语。
- InferenceObjective 和 ModelRewrite 让“请求目标”进入控制面，而不是硬编码在路由器配置里。

对比定位

和 AIBrix gateway plugins 相比，llm-d-router 更聚焦标准 Router/EPP；和 [agentgateway](#) 相比，它面向 model server pod placement，而不是 MCP/A2A/LLM access governance；和 RouteLLM/semantic-router 相比，它更 K8s/runtime metrics 驱动。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

llm-d Workload Variant Autoscaler

llm-d Workload Variant Autoscaler (WVA) 是面向分布式 llm-inference 的 Kubernetes variant autoscaler，用 `VariantAutoscaling` CRD 把同一模型/`InferencePool` 下不同硬件、角色、成本或配置的 serving variant 纳入一个全局扩缩决策。详见 [src-llm-d-workload-variant-autoscaler-architecture](#)。

[!warning] Conflict
截至 2026-09-22，llm-d 当前 [dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。本 Entity 及 [src-llm-d-workload-variant-autoscaler-architecture](#) 保留其所记录版本的历史架构，不构成当前采用建议；当前采用前需重新核验。参见 llm-inference-serving-project-map。

架构边界

WVA 不直接替代 HPA/KEDA。它通过 Prometheus、GPU inventory、[gateway-api-inference-extension](#) `InferencePool`、scale target 和 capacity model 计算 desired allocation，然后把 desired/current replica 指标暴露给 HPA/KEDA 这类 autoscaler 执行。

什么时候用

场景	判断
同一模型有多个 serving variant	适合，例如不同 GPU、prefill/decode、batch/interactive 或成本档位。
希望 autoscaling 理解 InferencePool/modelID	适合，比通用 HPA 更有 serving 语义。
只需要单 Deployment CPU/GPU utilization 扩缩	可能 HPA/KEDA 直接够用。
没有 Prometheus/custom metrics 基础设施	需要先补可观测和 metrics API。

同类对比

维度	llm-d-workload-variant-autoscaler	HPA / KEDA	kserve / Knative autoscaling
决策对象	多 variant allocation	单 workload 或 event source	service/revision/model service

维度	llm-d-workload-variant-autoscaler	HPA / KEDA	kserve / Knative autoscaling
推理语义	modelID / InferencePool / variant cost	通用指标	model serving 生命周期
执行方式	发指标给 HPA/KEDA	scale subresource	controller/autoscaler

选型提示

当 serving fleet 开始出现 prefill/decode 分离、不同 GPU 型号、不同成本档位或不同 SLO 池时，WVA 才有明显价值。单模型单部署阶段先把 [model-serving-operator](#) 和普通 autoscaling 跑稳更重要。

llm-d Workload Variant Autoscaler 架构与设计思路分析

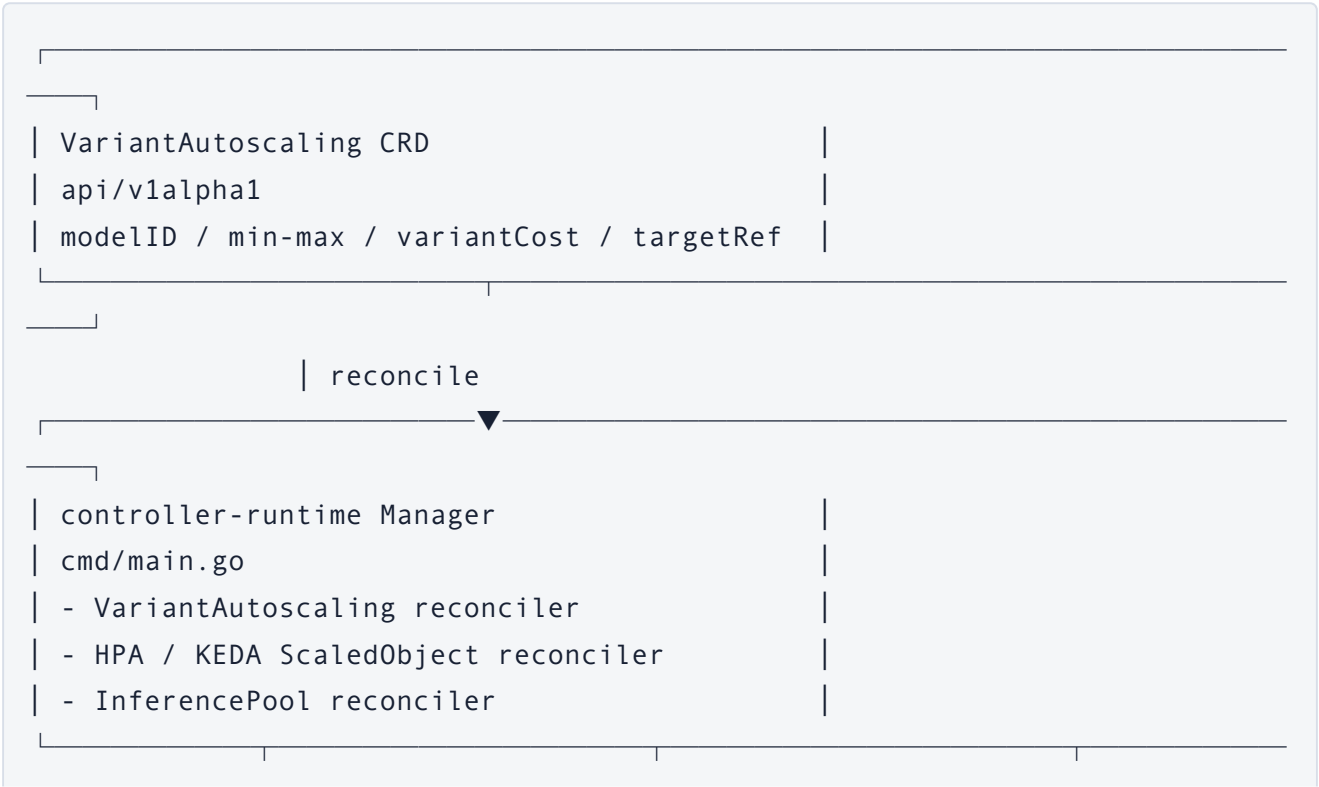
原文: raw/llm-d-workload-variant-autoscaler-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-workload-variant-autoscaler> · 分析版本 HEAD 526ce85

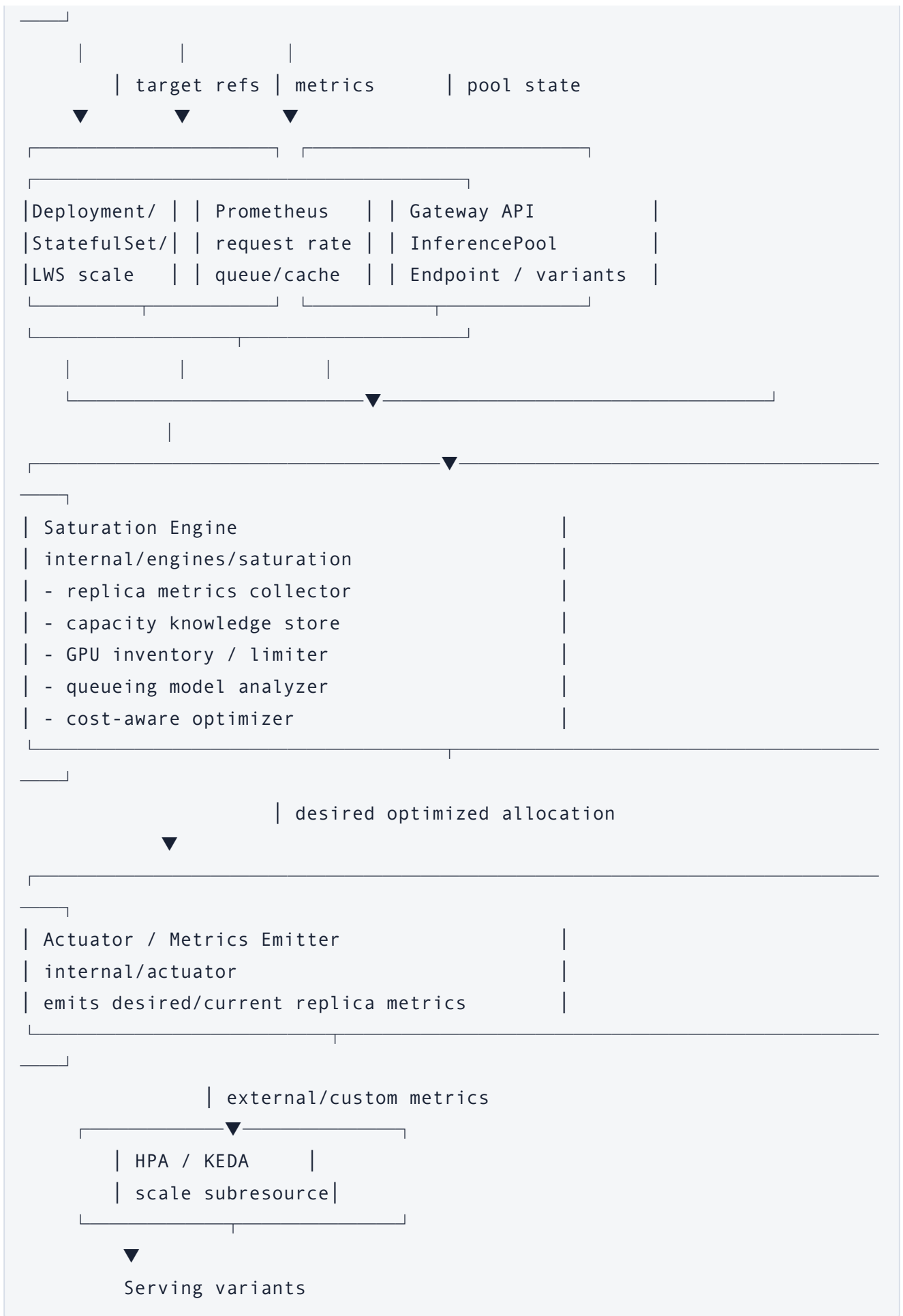
一句话定位

[llm-d-workload-variant-autoscaler](#) 是面向分布式 llm-inference 的 Kubernetes 全局 autoscaler，核心对象是同一模型/InferencePool 下的多个 serving variant。它通过 Prometheus、[gateway-api-inference-extension](#)、KEDA/HPA、GPU inventory 和 scale target 状态，把“不同硬件/角色/成本的变体应该各扩多少副本”转成外部指标和 status。

[!warning] Conflict
截至 2026-09-22，llm-d 当前 [dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。本页保留其所记录版本的历史设计与实现；当前采用前需重新核验。参见 llm-inference-serving-project-map。

核心架构图





模块分层

层 / 模块	职责
API 层	定义 VariantAutoscaling spec/status/conditions。
Manager 启动层	注册 K8s、Prometheus Operator、Gateway API Inference Extension、KEDA、LWS 等 scheme，组装 reconciler 和 engines。
Reconcile 层	解析 scale target、更新 conditions、读取 decision cache、patch status、watch workload 和 InferencePool。
Metrics 收集	采集 request rate、replica、queue/cache/capacity 相关指标。
优化引擎	维护 capacity store、GPU inventory、queueing/saturation analyzer、cost-aware optimizer。
执行动作	发出 desired/current replica 等指标供 HPA/KEDA 消费。

关键数据流





设计决策与哲学

- Variant 是 autoscaling 的一等对象：它不只看单个 Deployment 的 CPU/GPU 指标，而是把同一模型下不同硬件、角色或配置变体纳入一个优化问题。
- 不直接抢 HPA/KEDA 的职责：WVA 主要发指标，实际 scale 仍由 HPA/KEDA 完成。
- Gateway API Inference Extension 是 serving 语义入口：它需要理解 InferencePool 和 endpoint pool，而不是只做通用 workload autoscaling。
- 优化引擎需要容量知识而不只是当前利用率：capacity store、GPU inventory、queueing analyzer 和 optimizer 共同估计 serving capacity curve。

相关页面

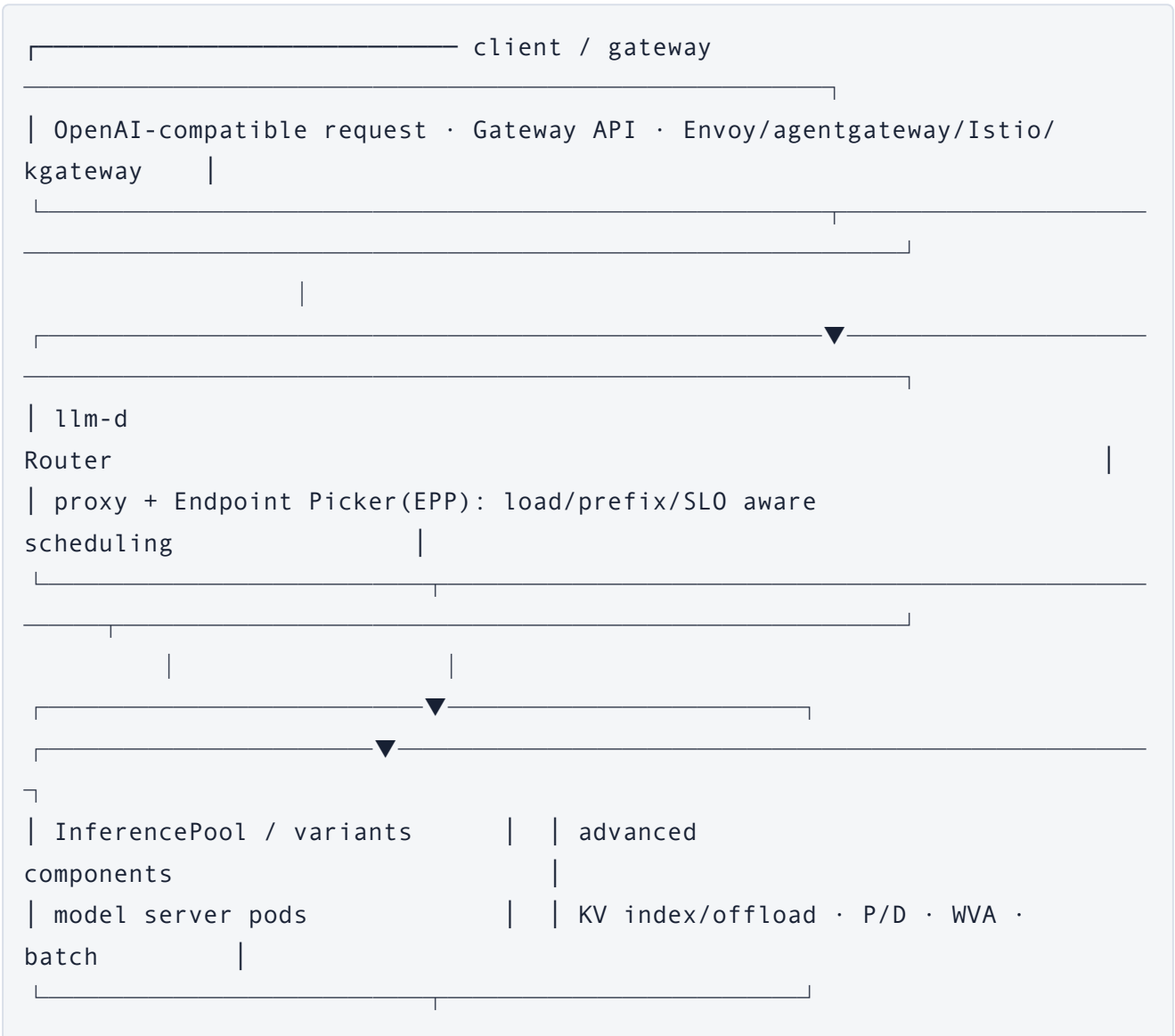
- [llm-d-workload-variant-autoscaler](#)
- [llm-d](#)
- [model-serving-operator](#)
- [gateway-api-inference-extension](#)
- [llm-inference](#)

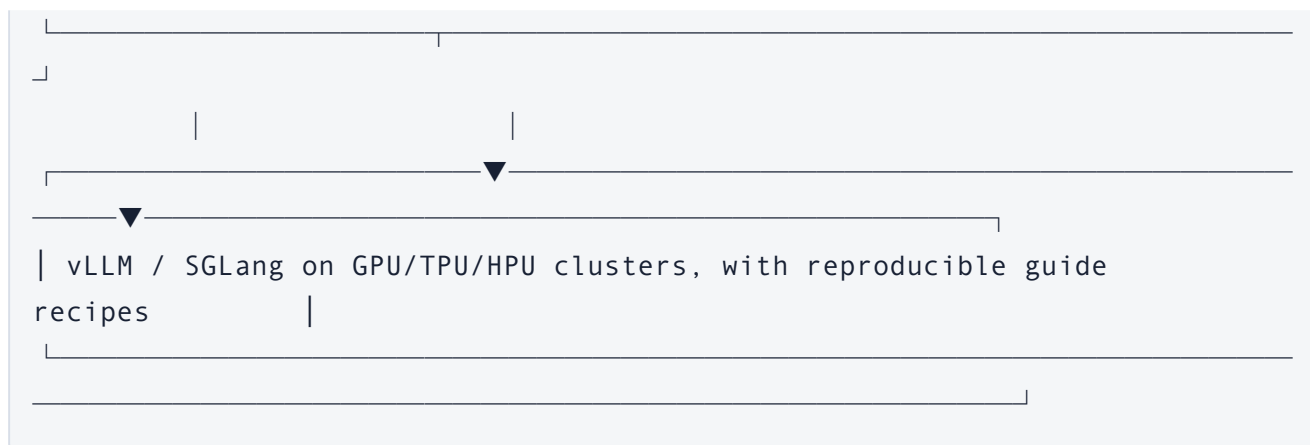
llm-d 架构与设计思路分析

llm-d/llm-d 是 Kubernetes 上的分布式 LLM serving stack 总入口。它本身更像 docs/guides/recipes/architecture hub，把 llm-d-router、Gateway API Inference Extension 的 InferencePool、vLLM/SGLang model server、KV cache management、P/D disaggregation、latency predictor、batch gateway、autoscaling 等组件组织成 well-lit paths。

[!warning] Conflict
截至 2026-09-22，llm-d 当前 [dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。本页保留其所记录版本的历史设计与实现；当前采用前需重新核验。参见 llm-inference-serving-project-map。

核心架构图





模块分层

层/目录	责任
docs/architecture/**	核心架构文档: Router、InferencePool、Model Server、KV cache、disaggregation、autoscaling、batch。
guides/**	可复现实验/部署路径: optimized baseline、P/D、wide EP、tiered prefix cache、flow control、batch、observability。
docs/proposals/**	仍在演进的设计: autoscaler、batch gateway、planner、resilience operator 等。
release/**, guides/recipes/**	发行和 Kustomize/Helm recipe 入口。

关键数据流

1. 请求经 Gateway API 进入 Router; proxy 用 ext-proc 咨询 EPP。
2. EPP 按 InferencePool 发现可用 model server pods, 并结合 load、KV affinity、priority/SLO profile 打分。
3. 高级路径中, P/D disaggregation 先选 prefill 再选 decode、KV cache 通过 index/offload 提高 reuse。

设计决策

- llm-d 把“架构标准化 + 组件拆分 + well-lit path”作为主要交付，而不是一个大二进制。
- 它承认 model server 是 vLLM/SGLang 等外部 engine，llm-d 只做其上的 orchestration 和优化。
- InferencePool 是核心抽象：像 LLM-optimized Service，用 labels/variants 表达模型、角色和性能差异。

对比定位

和 AIBrix 相比，llm-d 更标准化/生态化，组件分仓；和 [dynamo](#) 相比，它更 Kubernetes/Gateway API first；和 vllm/sglang 相比，它不是 engine，而是 engine 之上的 serving system。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

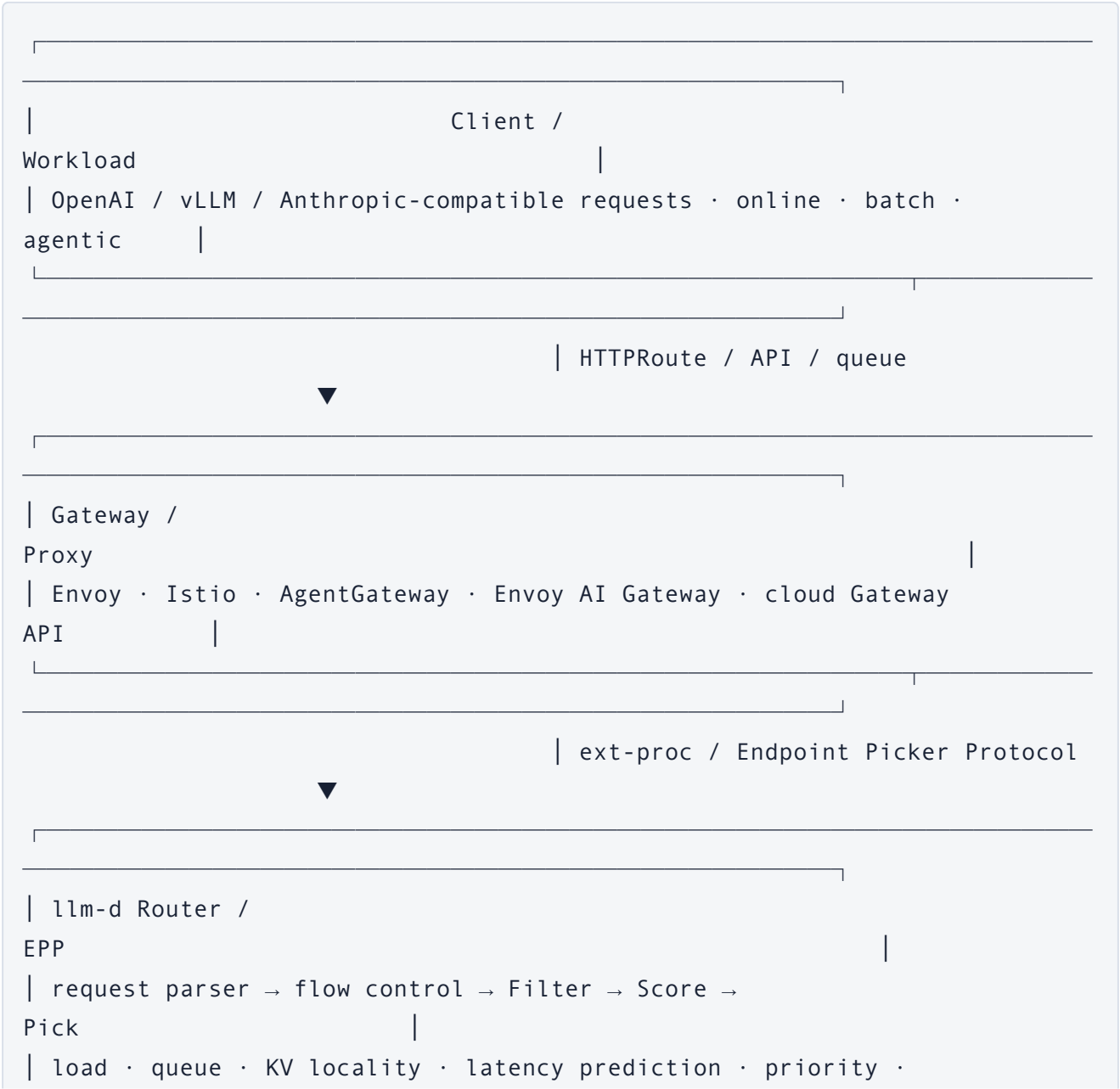
llm-d 核心项目群架构再调研

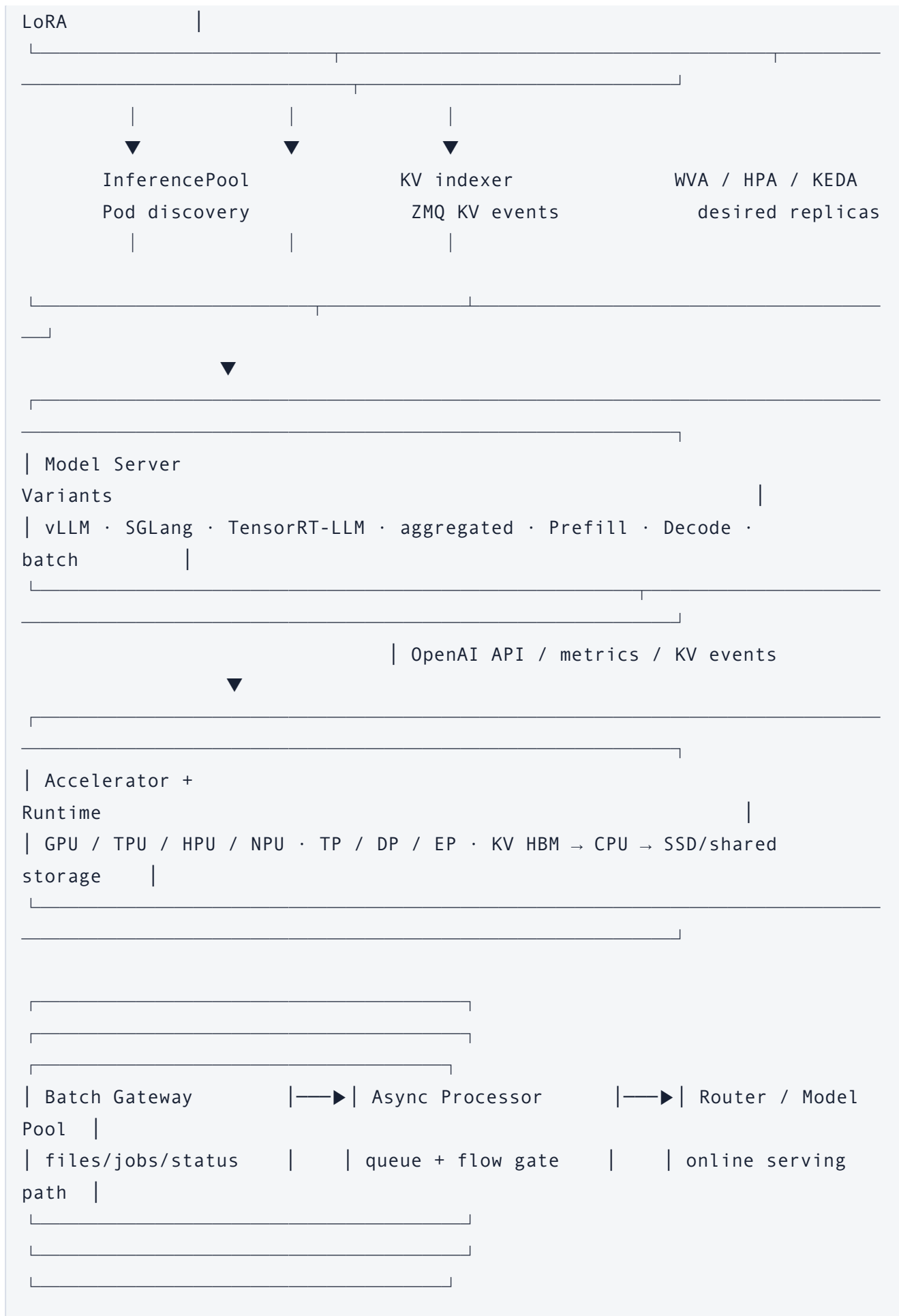
原文: raw/llm-d-core-projects-research-2026-09-13.md · 调研日期: 2026-09-13

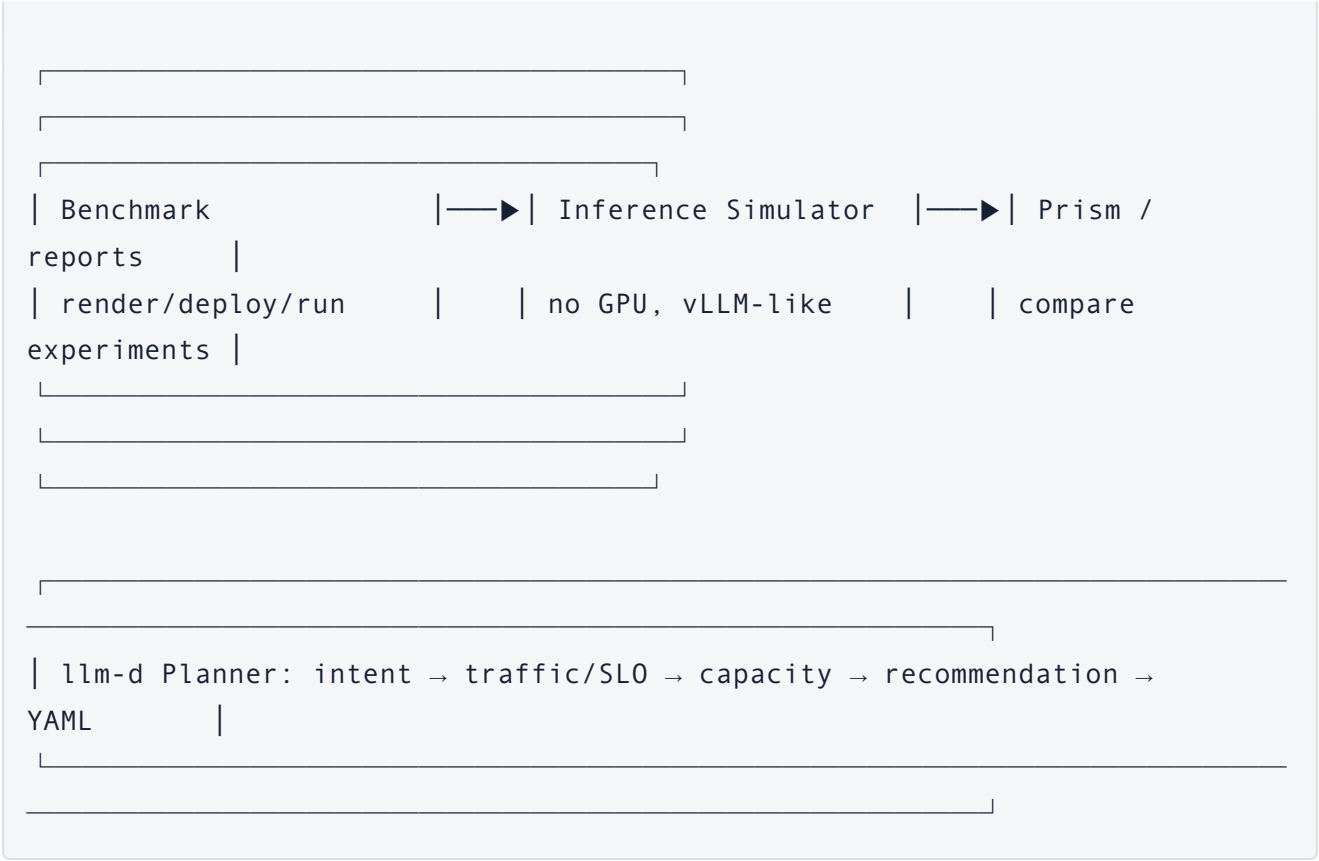
一句话定位

[llm-d](#) 是位于 vllm / sglang 之上的 Kubernetes-native 分布式推理控制与优化栈；核心项目围绕路由、KV cache、P/D、扩缩容、Batch、实验验证和部署规划形成完整链路。

总体架构图







核心项目职责

项目	作用	解决的业务问题	主要集成
llm-d	主架构、部署 recipe、API 和 well-lit paths	把模型 server 组合成生产 serving stack	Kubernetes、Gateway API、vLLM、SGLang、Prometheus
llm-d-router	Proxy + EPP 智能入口	endpoint picking、排队、公平性、P/D 编排	Envoy ext-proc、InferencePool、InferenceObjective
llm-d-kv-cache	KV event index/scorer library	重复 prefix 重算、缓存局部性丢失	vLLM/SGLang KV events、ZMQ、Router EPP
llm-d-workload-variant-autoscaler	variant 全局扩缩	异构 GPU、P/D、成本与 SLO 的联合分配	Prometheus、HPA/KEDA、InferencePool
llm-d-batch-gateway	OpenAI-compatible Batch API	大量离线请求的持久化、重试和限流	PostgreSQL、Redis/Valkey、object store、Router
llm-d-benchmark	实验生命周期编排	多环境、多配置实验不可复现	Jinja2、Kustomize/Helm、GuideLLM、inference-perf
llm-d-inference-sim	vLLM 行为模拟器		

项目	作用	解决的业务问题	主要集成
		无 GPU 验证 routing/ autoscaling/CI	OpenAI API、vLLM gRPC、 ZMQ、Prometheus
llm-d-planner	需求到部署规划	不懂 GPU/模型/SLO 的 团队难以落地	FastAPI、SQLite、LLM provider、KServe/vLLM YAML

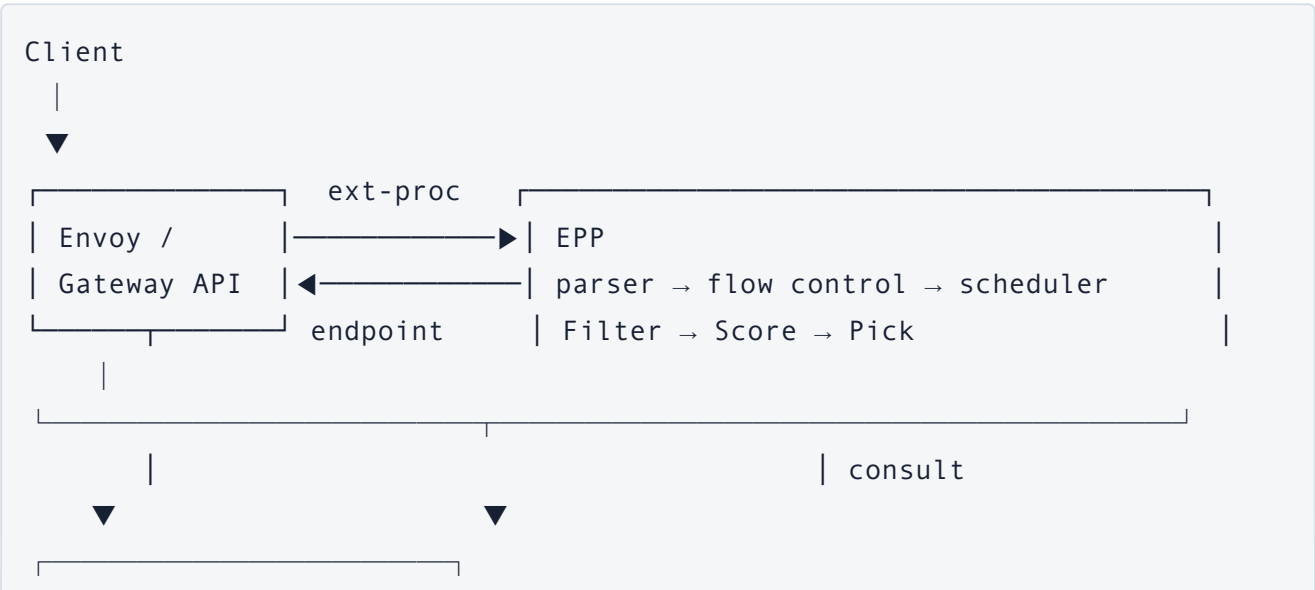
Router / EPP 请求路径

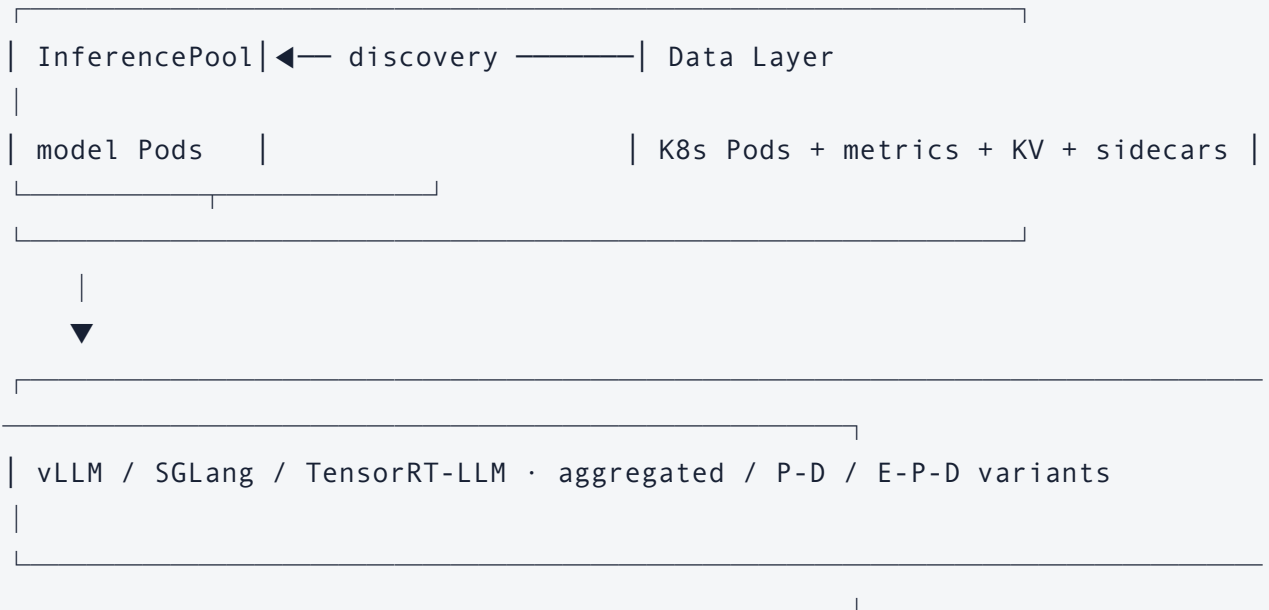
```
HTTPRoute → Gateway/Proxy
    → ext-proc 请求
    → EPP 解析 request
    → Flow Control admission / priority / fairness
    → Filter 候选端点
    → Score load / KV / latency / LoRA
    → Pick endpoint
    → Proxy 转发到 Model Server
    → vLLM/SGLang 执行 prefill + decode
    → metrics / KV events 反哺 EPP
```

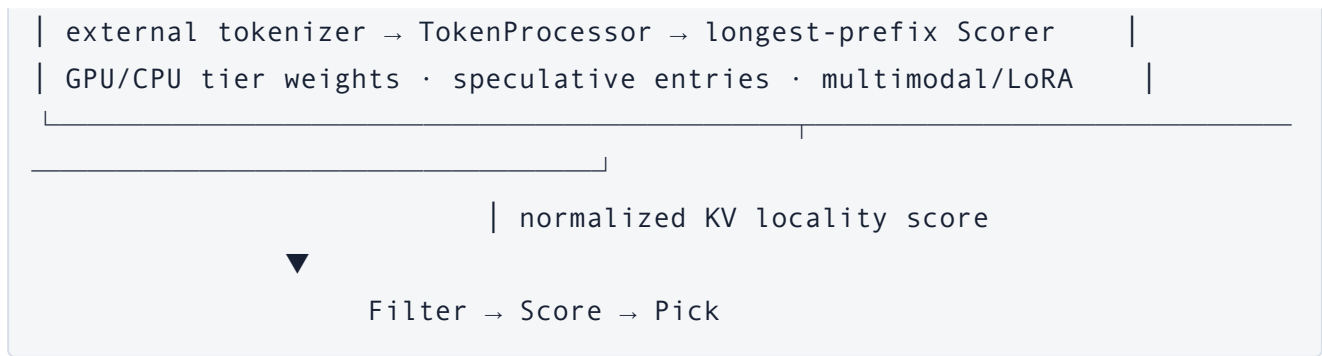
llm-d-router 的方法是插件化 Filter → Score → Pick，而不是把某个 routing heuristic 固化在 Proxy 中。EPP 还能通过 Flow Control 处理 priority band、saturation、fairness 和 ordering；Proxy 继续承担连接、TLS 和转发。

KV Cache 路径

Router 架构图







llm-d-kv-cache 的关键不是“把 KV 存到哪里”，而是把 engine 的 KV state 转成 Router 可查询的 locality signal。它支持 vLLM/SGLang event adapter、最长连续 prefix、tier 权重、multimodal/LoRA extra keys 和 speculative indexing；tokenization 正在从库内 pool 迁移到外部 host/sidecar。

Autoscaling、Batch 与实验闭环

Autoscaling

WVA 读取 queue、running、KV、latency、pending Pod 和 accelerator inventory，计算 desired replicas，再让 HPA/KEDA 执行。单一同构 Deployment 适合 KEDA + EPP metrics；多 variant、P/D、异构硬件和强 SLO 才需要 WVA。

Batch

Batch Gateway 负责 job/file/output 的 durable lifecycle；Async Processor 负责从 Redis 或 Pub/Sub 取单请求，结合系统信号做 flow control，再调用 Router。它和在线 Router 的区别是：前者优化 job 可恢复性和吞吐，后者优化单请求延迟与端点选择。

Benchmark 与 Simulator

Benchmark 用 declarative scenario、Jinja render、Kubernetes lifecycle、harness 和 workspace 实现可复现实验；Inference Sim 用 OpenAI/vLLM API、延迟公式、KV blocks、metrics 和 failure injection 让这些实验可以在无 GPU 环境做控制面 smoke test。Simulator 不是生产性能基线。

Planner 的上游入口

[llm-d-planner](#) 把自然语言或结构化业务意图转换成 traffic profile、TTFT/ITL/E2E SLO、quality/cost/latency 权重，再用 benchmark、capacity planner 和 GPU recommender 生成部署建议，最后渲染 KServe/vLLM/HPA/ServiceMonitor YAML。它是 adoption/planning 层，不在 Router 的在线请求路径内。

设计判断与边界

- [llm-d](#) 负责组合和标准化，不替代推理 engine。
- llm-d-router 负责 endpoint-level intelligence，不是通用 provider gateway。

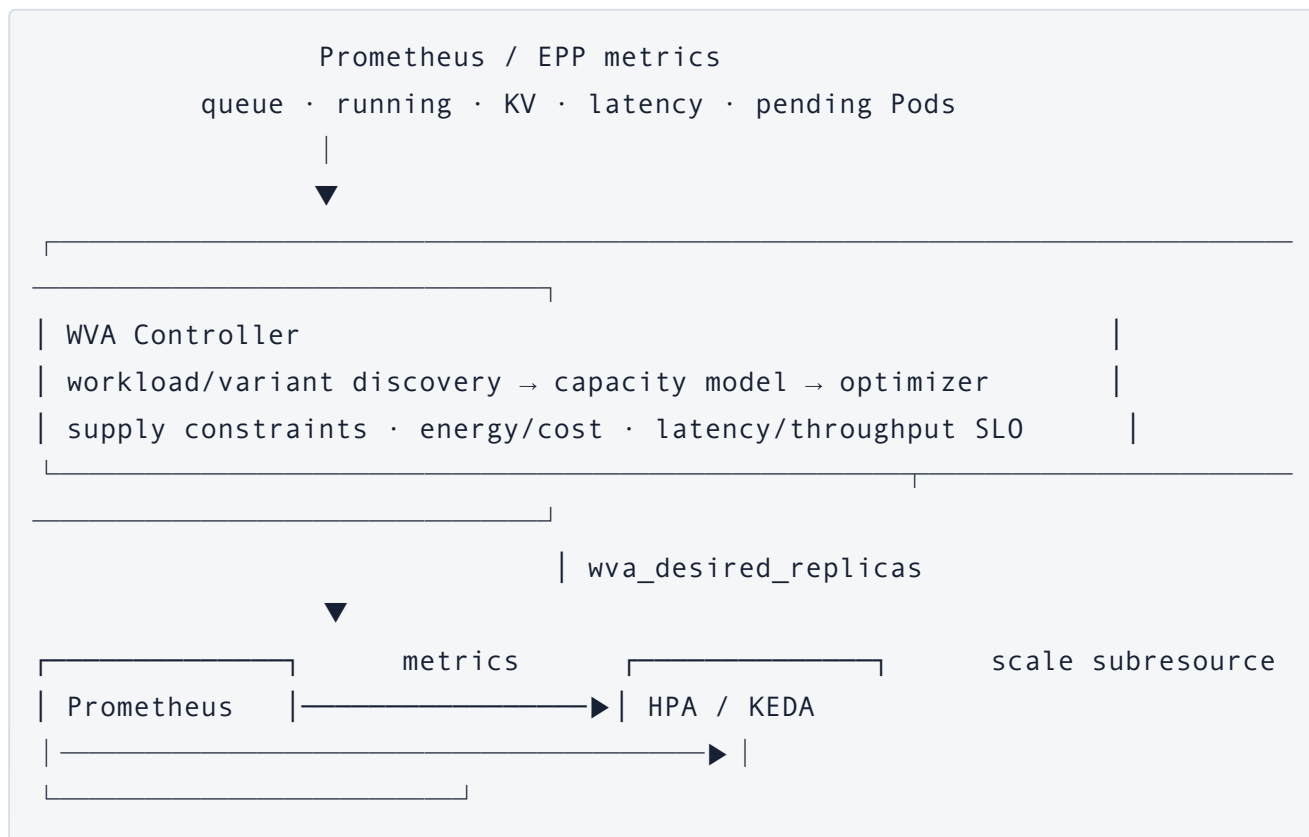
- llm-d-kv-cache 负责 locality index/scoring，不等同于 KV offload 或 KV transfer。
- WVA 通过 HPA/KEDA 执行扩缩，不直接取代 Kubernetes autoscaler。
- Batch Gateway 解决异步 job，不应把 batch 请求直接当 interactive request。
- Benchmark 与 Simulator 负责验证和实验，不能把模拟延迟当真实 GPU 指标。
- Planner 解决部署规划，当前仍要区分生产愿景和 POC 实现。

各项目架构图补充

核心请求数据流

- HTTPRoute → Gateway/Proxy
 - ext-proc 请求
 - EPP 解析 request
 - Flow Control admission / priority / fairness
 - Filter 候选端点
 - Score load / KV / latency / LoRA
 - Pick endpoint
 - Proxy 转发到 Model Server
 - vLLM/SGLang 执行 prefill + decode
 - metrics / KV events 反哺 EPP

WVA



Model-server Deployments / variants

Batch Gateway

Client

| POST /v1/files + POST /v1/batches



| Batch API Server
| validation / status |

PostgreSQL metadata

| enqueue



| Processor workers
| fetch JSONL → invoke |
| backend → write output |

| Redis/Valkey
| queue/status |



| OpenAI-compatible requests



| Router or backend

object storage / FS

▶ output JSONL



| GC / retry / reconciliation

Benchmark

Scenario YAML + cluster config + CLI overrides



| Plan phase

| Pydantic validation → config merge → Jinja render → workspace





Inference Sim



Planner



FastAPI / UI / CLI	
Intent Extraction → Specification Editor	
▼	
Knowledge Base	
model catalog · GPU catalog · SLO templates · benchmark DB	
Arena/Artificial Analysis quality · empirical/estimated data	
▼	
Recommendation	
capacity planner → config finder → quality/cost/latency scorer	
→ ranked views / what-if trade-offs	
▼	
Configuration / Deployment	
Jinja2 → KServe/vLLM/HPA/ServiceMonitor YAML → Kubernetes	

相关页面

- inference-routing
- kv-cache-offload
- disaggregated-serving
- [model-serving-operator](#)
- [gateway-api-inference-extension](#)
- vllm
- sglang

Local Static Provisioner

Local Static Provisioner 发现节点本地磁盘/目录并创建 local PersistentVolume，配合调度绑定把数据固定到节点。详见 [src-sig-storage-local-static-provisioner-architecture](#)。

架构边界

local PV 性能高但节点绑定强；动态网络存储更灵活但可能牺牲本地性能。

什么时候用

场景	判断
需要 Local PV static provisioning 能力	适合，Local Static Provisioner 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , llm-inference 组合。

核心组件

- Discovery daemon: scan mount directories
- PV controller: create/delete local PV
- Node affinity: bind PV to node
- Cleanup scripts/classes

选型提示

把 Local Static Provisioner 放在 Local PV static provisioning 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

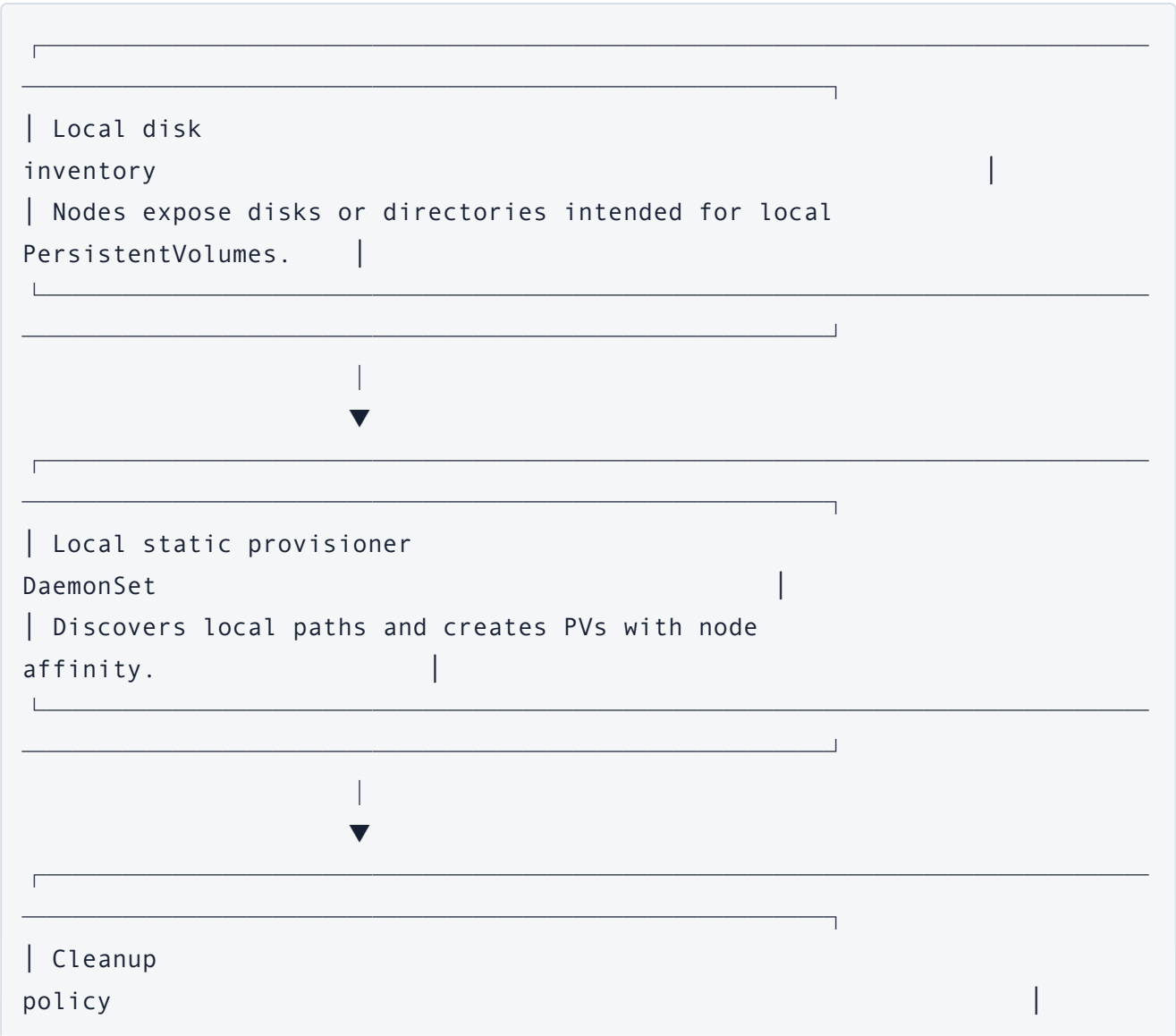
Local Static Provisioner 架构与设计思路分析

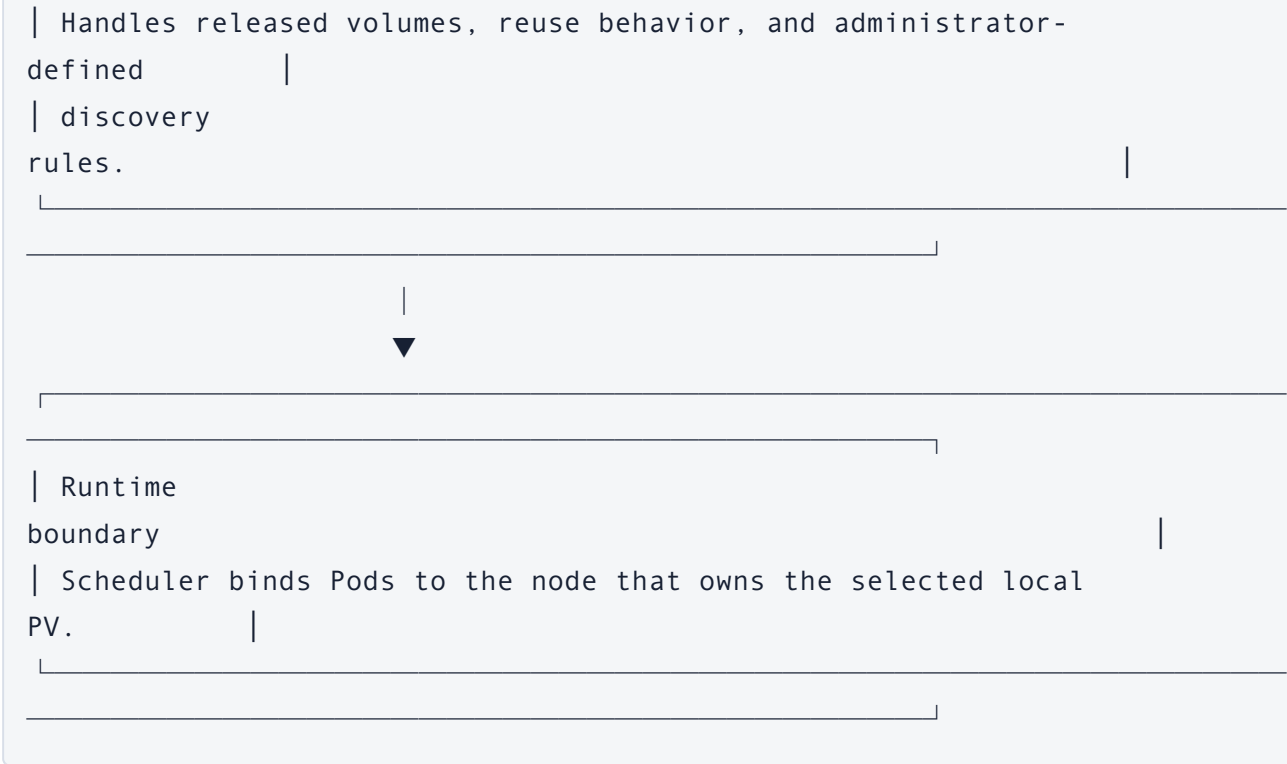
原文: raw/sig-storage-local-static-provisioner-architecture-analysis.md
· 仓库: <https://github.com/kubernetes-sigs/sig-storage-local-static-provisioner> · 优先级 P1

一句话定位

Local Static Provisioner 发现节点本地磁盘/目录并创建 local PersistentVolume，配合调度绑定把数据固定到节点。

核心架构图

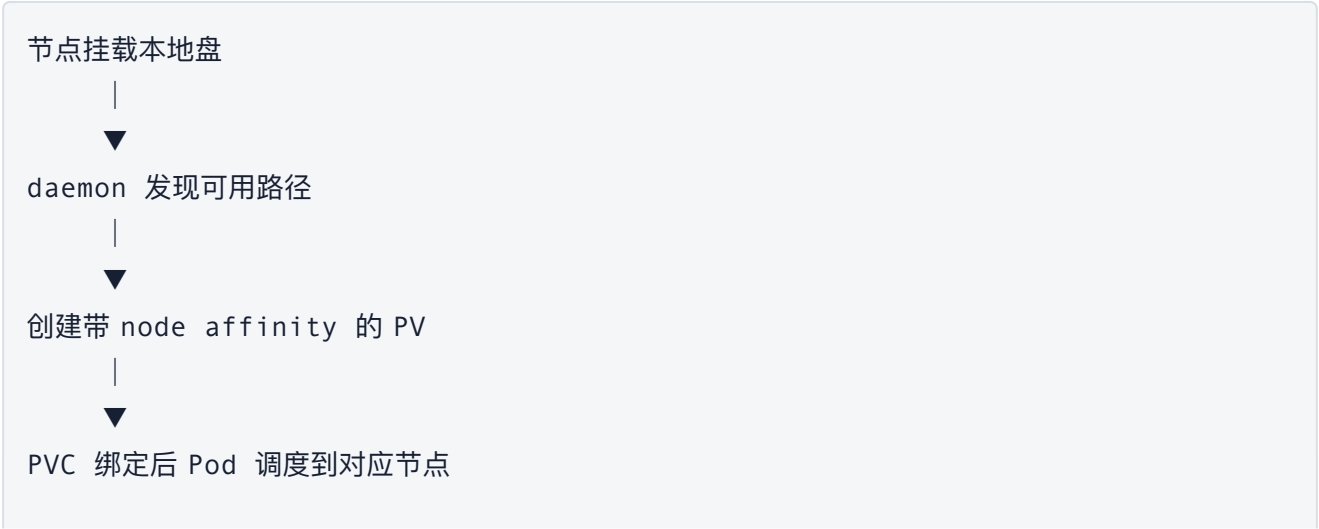




模块分层

层 / 模块	职责
Discovery daemon	scan mount directories
PV controller	create/delete local PV
Node affinity	bind PV to node
Cleanup scripts/classes	Cleanup scripts/classes

关键数据流





释放后清理或保留

设计决策与哲学

- 补齐 Local PV static provisioning 维度: Local Static Provisioner 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: local PV 性能高但节点绑定强; 动态网络存储更灵活但可能牺牲本地性能。
- 选型价值: 它应和 [kubernetes](#), llm-inference 一起看, 而不是孤立评估。

相关页面

- [sig-storage-local-static-provisioner](#)
- [kubernetes](#)
- llm-inference

MCP Lifecycle Operator

MCP Lifecycle Operator 用声明式 API 部署、管理和安全滚动 MCP Servers，把 Agent tool server 生命周期放进 Kubernetes control plane。详见 [src-mcp-lifecycle-operator-architecture](#)。

架构边界

MCP server framework 解决怎么写工具；MCP lifecycle operator 解决工具服务器如何在集群里安全运行和升级。

什么时候用

场景	判断
需要 MCP lifecycle 能力	适合，MCP Lifecycle Operator 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 mcp, declarative-agent-management, agentgateway 组合。

核心组件

- MCPServer-like APIs
- Controller: deploy/rollout/status
- Integration: gateway/auth/config/secrets
- Production automation: health, rollout, lifecycle

选型提示

把 MCP Lifecycle Operator 放在 MCP lifecycle 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

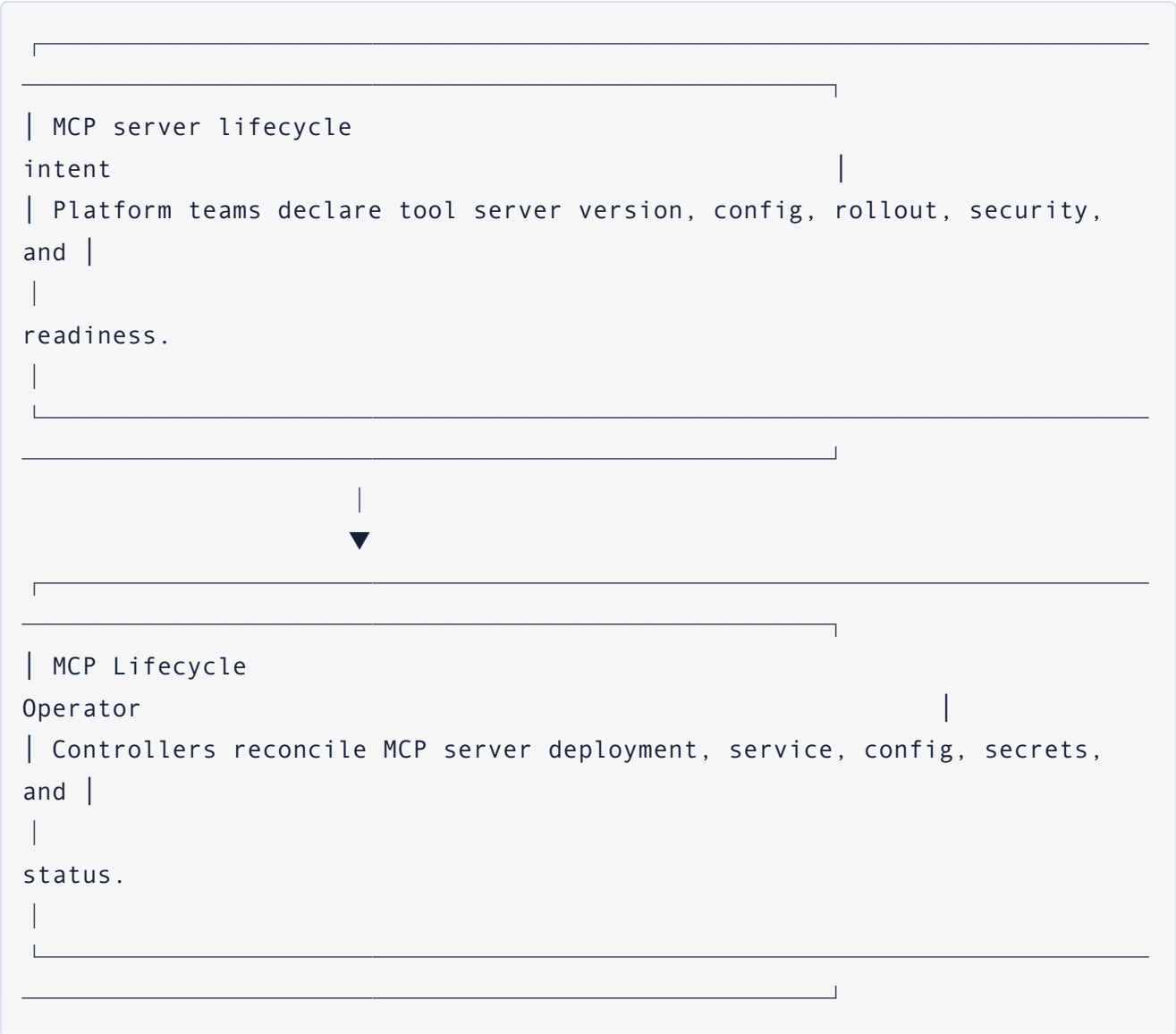
MCP Lifecycle Operator 架构与设计思路分析

原文: raw/mcp-lifecycle-operator-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/mcp-lifecycle-operator> · 优先级 P1

一句话定位

MCP Lifecycle Operator 用声明式 API 部署、管理和安全滚动 MCP Servers，把 Agent tool server 生命周期放进 Kubernetes control plane。

核心架构图





模块分层

层 / 模块	职责
MCPServer-like APIs	MCPServer-like APIs
Controller	deploy/rollout/status
Integration	gateway/auth/config/secrets
Production automation	health, rollout, lifecycle

关键数据流



设计决策与哲学

- 补齐 MCP lifecycle 维度: MCP Lifecycle Operator 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: MCP server framework 解决怎么写工具; MCP lifecycle operator 解决工具服务器如何在集群里安全运行和升级。
- 选型价值: 它应和 mcp, declarative-agent-management, [agentgateway](#) 一起看, 而不是孤立评估。

相关页面

- [mcp-lifecycle-operator](#)
- [kubernetes](#)
- mcp
- declarative-agent-management
- [agentgateway](#)

metrics-server

Kubernetes 资源指标管道，把 kubelet summary/metrics 暴露成 `metrics.k8s.io`，供 HPA/VPA/kubectl top 使用。详见 [src-metrics-server-architecture](#)。

架构边界

metrics-server 只服务资源指标，不是 Prometheus 替代品；custom/external metrics 需要 prometheus-adapter 等组件。

什么时候用

场景	判断
需要 可观测 / autoscaling 能力	适合，metrics-server 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合；metrics-server 只提供 resource metrics plumbing。当前 llm-d 新部署使用 EPP metrics → KEDA Prometheus scaler → HPA ； llm-d-workload-variant-autoscaler 是 deprecated 的历史设计，仅保留供迁移/研究参考。

核心组件

- Scraper: 周期访问 kubelet metrics
- Storage: 只保留最新 CPU/memory resource metrics
- APIService: 注册 metrics.k8s.io aggregated API
- Consumers: HPA/VPA/kubectl top

选型提示

把 metrics-server 放在 可观测 / autoscaling 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

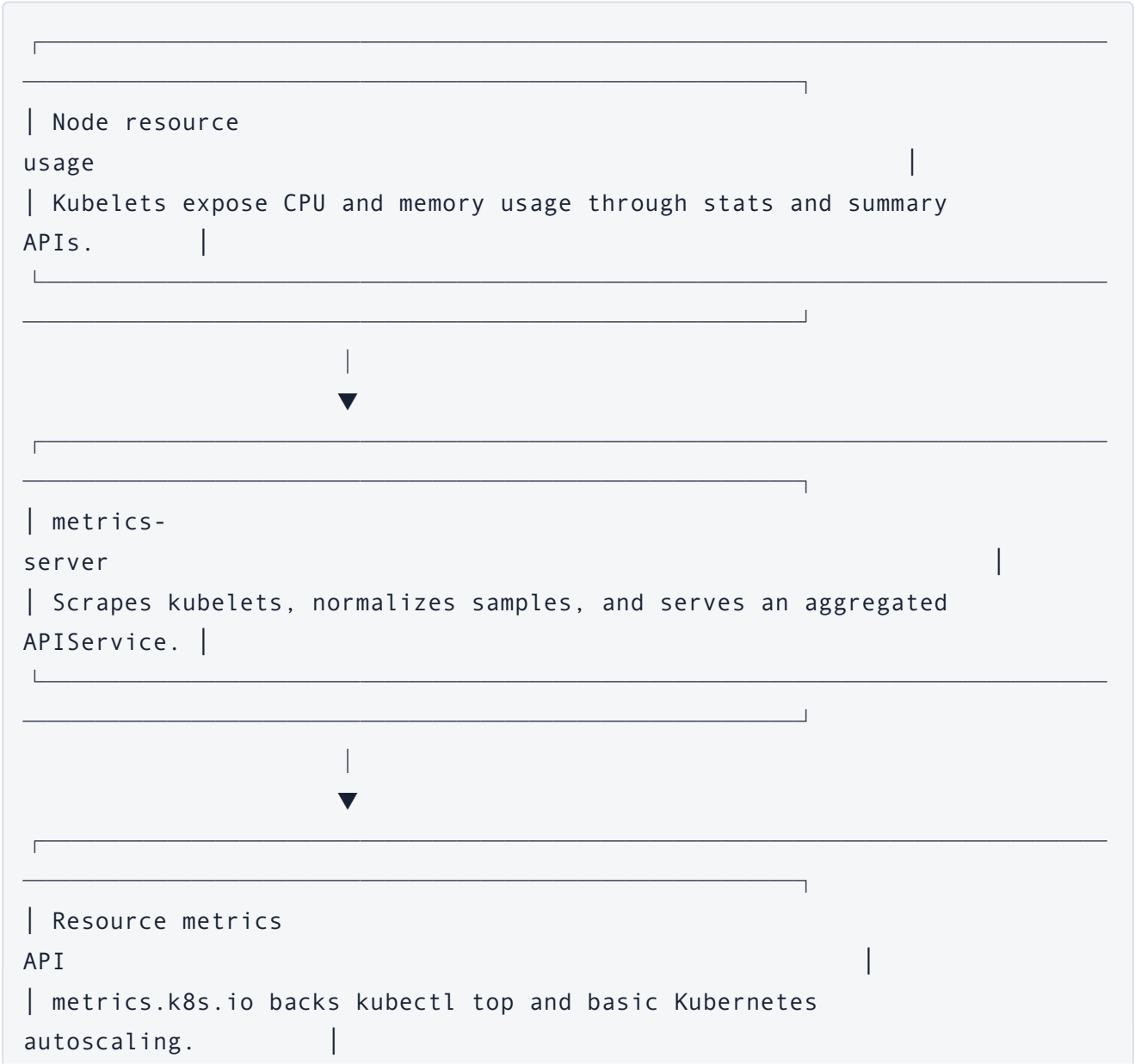
metrics-server 架构与设计思路分析

原文: raw/metrics-server-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/metrics-server> · 优先级 P0

一句话定位

Kubernetes 资源指标管道，把 kubelet summary/metrics 暴露成 metrics.k8s.io，供 HPA/VPA/kubectl top 使用。

核心架构图

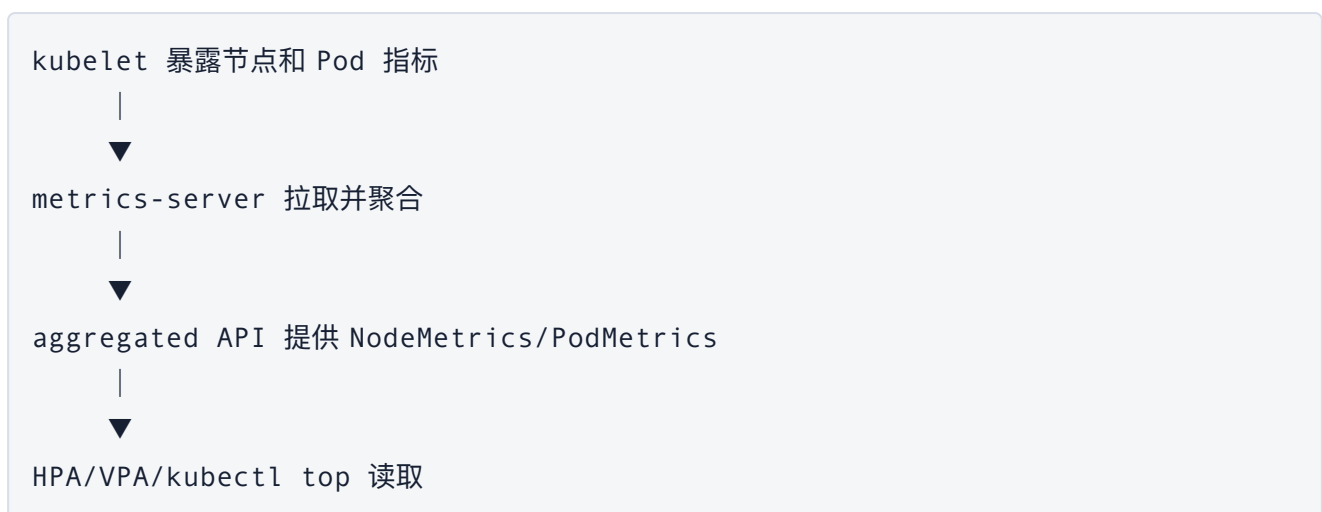




模块分层

层 / 模块	职责
Scraper	周期访问 kubelet metrics
Storage	只保留最新 CPU/memory resource metrics
APIService	注册 metrics.k8s.io aggregated API
Consumers	HPA/VPA/kubectl top

关键数据流



设计决策与哲学

- 补齐 可观测 / autoscaling 维度: metrics-server 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: metrics-server 只服务资源指标，不是 Prometheus 替代品；custom/external metrics 需要 prometheus-adapter 等组件。
- 选型价值: 它应和 llm-inference, [model-serving-operator](#), [llm-d-workload-variant-autoscaler](#) 一起看，而不是孤立评估。

相关页面

- [metrics-server](#)
- [kubernetes](#)
- llm-inference
- [model-serving-operator](#)
- [llm-d-workload-variant-autoscaler](#)

Model Serving Operator

Model serving operator 是在 Kubernetes 上声明式管理模型、runtime、endpoint、autoscaling、GPU 资源和推理服务生命周期的控制面。

代表项目

项目	重点
kserve	标准化 InferenceService / LLMISvc / LocalModel 平台
ome	Open Model Engine, CRD/controller + runtime selector + accelerator configs
kubebai	Model CRD + OpenAI-compatible proxy + autoscaler + model loader
gpustack	GPU cluster manager + vLLM/SGLang 编排 + observability
llm-d-workload-variant-autoscaler	Deprecated 的历史 variant autoscaling 设计, 仅供迁移/设计参考
llm-d-batch-gateway	不管理模型部署, 但把 batch job lifecycle 接到 model serving endpoint
rbg	用 RoleBasedGroup / RoleInstance / CoordinatedPolicy 表达多角色、有状态、跨角色协调的 inference workload
kthena	用 ModelBooster / ModelServing / ModelServer / ModelRoute 组合完整 K8s LLM serving control/data plane
openkruise-kruise	不直接管理模型服务, 但提供 workload enhancement、image preheat、workload spread 等可借鉴的 K8s workload automation 模式

和 engine 的区别

vllm / sglang 是推理 engine; model serving operator 管的是 engine 在集群里的部署、扩缩、路由、模型生命周期和 API 暴露。

和 llm-d 外围组件的关系

llm-d-batch-gateway、llm-d-benchmark、llm-d-inference-sim, 以及 deprecated/历史的 [llm-d-workload-variant-autoscaler](#), 说明 model serving operator 周围还有一圈工程能力:

能力	代表	和 operator 的边界
	llm-d-batch-gateway	

能力	代表	和 operator 的边界
Batch job control plane		不创建模型服务，复用已有 endpoint 执行异步 JSONL job。
Variant autoscaling (历史)	llm-d-workload-variant-autoscaler	Deprecated 设计曾为多个 variant 计算 desired allocation；现仅供迁移/设计参考。
Benchmark lifecycle	llm-d-benchmark	不是生产 controller，而是部署、运行、收集和分析实验。
Simulator	llm-d-inference-sim	不是真实模型服务，用于验证 controller/router/benchmark 闭环。

选型时不要把这些都归类成“serving operator”。operator 管声明式生命周期；batch、benchmark 和 simulator 分别补异步任务、评测和测试替身。对新部署，llm-d 当前路径是 [EPP metrics → KEDA Prometheus scaler → HPA](#)；[llm-d-workload-variant-autoscaler](#) 的 Entity Conflict 注记及历史设计仅用于迁移/设计参考。

和 Kubernetes 调度/指标外围的关系

模型服务 operator 只管理模型服务生命周期还不够。[dynamo](#) 展示了另一条边界：DGDR/Profiler/AIConfigurator 负责部署前配置搜索，DGD/DCD/Operator 负责资源物化，Planner 负责运行时 P/D 容量控制，而 Router 留在请求热路径。[kueue](#) / [jobset](#) / [lws](#) 决定 batch 和分布式 workload 怎么表达与入队；[karpenter](#) 决定节点容量怎样按 pending pods 扩出；[metrics-server](#) / [prometheus-adapter](#) 决定 autoscaling 能看到哪些指标。[kubernetes-workload-automation](#) 提供更上位的整体视角。

NFS Subdir External Provisioner

NFS Subdir External Provisioner 在远端 NFS server 上为 PVC 动态创建子目录，是轻量实验/中小集群常见 storage class。详见 [src-nfs-subdir-external-provisioner-architecture](#)。

架构边界

它简单易用但隔离和性能弱于云盘/CSI，适合实验或非关键共享文件场景。

什么时候用

场景	判断
需要 NFS dynamic provisioning 能力	适合，NFS Subdir External Provisioner 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , cloud-native-security 组合。

核心组件

- Provisioner controller: watch PVC/PV
- NFS backend: shared export path
- StorageClass parameters
- Cleanup/reclaim policy

选型提示

把 NFS Subdir External Provisioner 放在 NFS dynamic provisioning 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

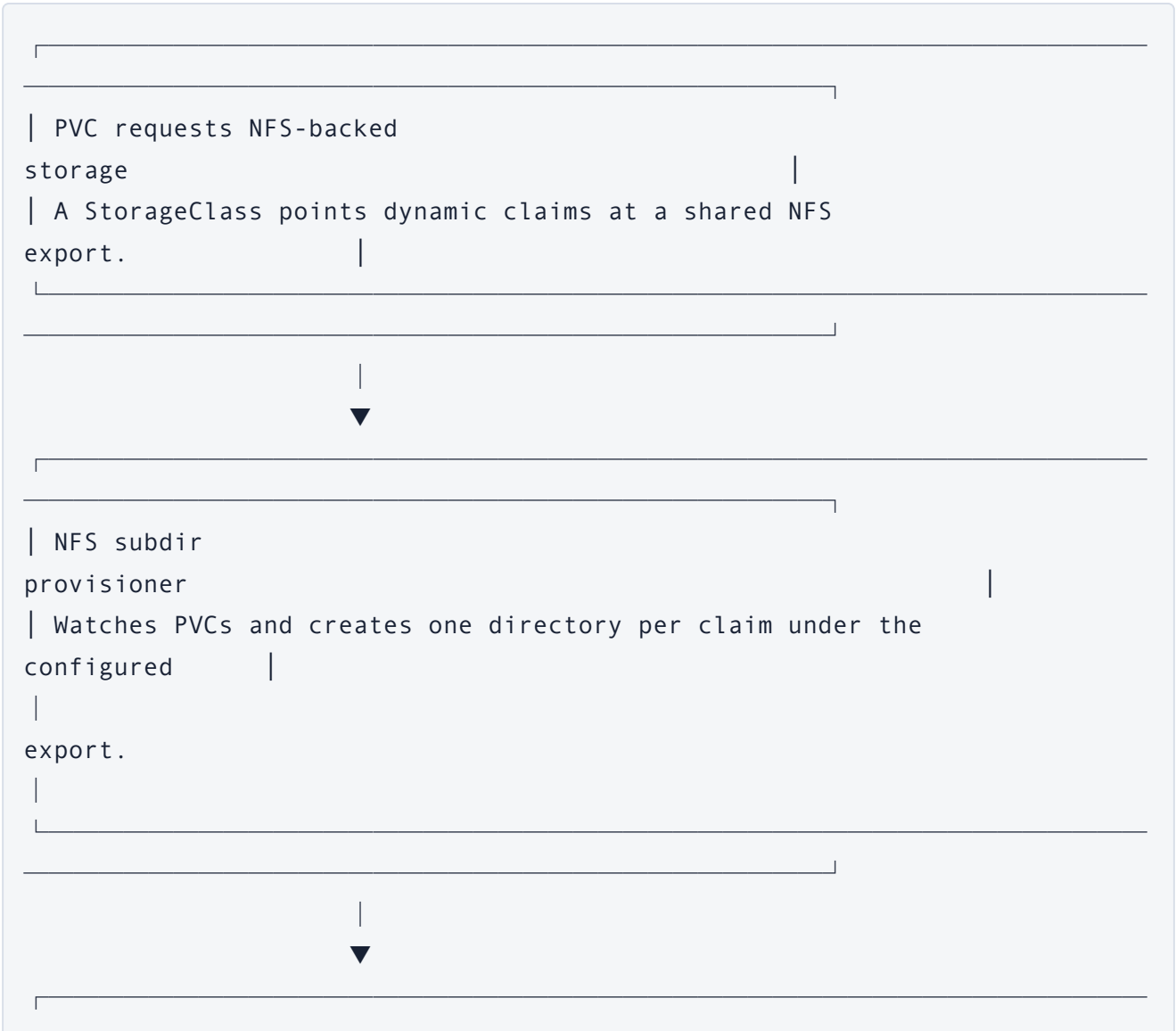
NFS Subdir External Provisioner 架构与设计思路分析

原文: raw/nfs-subdir-external-provisioner-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner> · 优先级 P1

一句话定位

NFS Subdir External Provisioner 在远端 NFS server 上为 PVC 动态创建子目录，是轻量实验/中小集群常见 storage class。

核心架构图





模块分层

层 / 模块	职责
Provisioner controller	watch PVC/PV
NFS backend	shared export path
StorageClass parameters	StorageClass parameters
Cleanup/reclaim policy	Cleanup/reclaim policy

关键数据流



生成 PV 指向该路径



Pod 挂载 PVC



删除时按 reclaim policy 清理

设计决策与哲学

- 补齐 NFS dynamic provisioning 维度：NFS Subdir External Provisioner 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：它简单易用但隔离和性能弱于云盘/CSI，适合实验或非关键共享文件场景。
- 选型价值：它应和 [kubernetes](#), [cloud-native-security](#) 一起看，而不是孤立评估。

相关页面

- [nfs-subdir-external-provisioner](#)
- [kubernetes](#)
- [cloud-native-security](#)

Node Feature Discovery

Node Feature Discovery 发现 CPU、内核、PCI、NUMA、GPU/加速器等硬件/系统能力，并写成 node labels/features 供调度使用。详见 [src-node-feature-discovery-architecture](#)。

证据说明：2026-09-27 官方默认分支快照为 [386fda4332ba](#)，commit 不代表 release。[src-node-feature-discovery-architecture](#) 保留 2026-06-14 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，DRA 分配与节点准备见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

NFD worker 检测节点特征，master 发布 labels/规则结果，topology-updater 维护 NodeResourceTopology，GC 清理过期对象。它为 [gpu-operator](#) 及 placement 提供能力信号，不分配每个 Pod 的设备；[k8s-device-plugin](#) / [dra-driver-nvidia-gpu](#) 管各自分配路径，[hami](#) 的共享记账也不能只依赖能力标签。

架构边界

device plugin 暴露可分配资源；NFD 暴露节点能力标签，常作为 GPU/NUMA/硬件调度前置信号。

什么时候用

场景	判断
需要 节点能力发现 能力	适合，Node Feature Discovery 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , llm-inference, gpu-sharing 组合。

核心组件

- nfd-worker: node local feature sources
- nfd-master/gc: label publication and cleanup
- Feature sources: cpu, kernel, pci, usb, custom hooks
- Rules: custom feature labels

选型提示

把 Node Feature Discovery 放在 节点能力发现 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

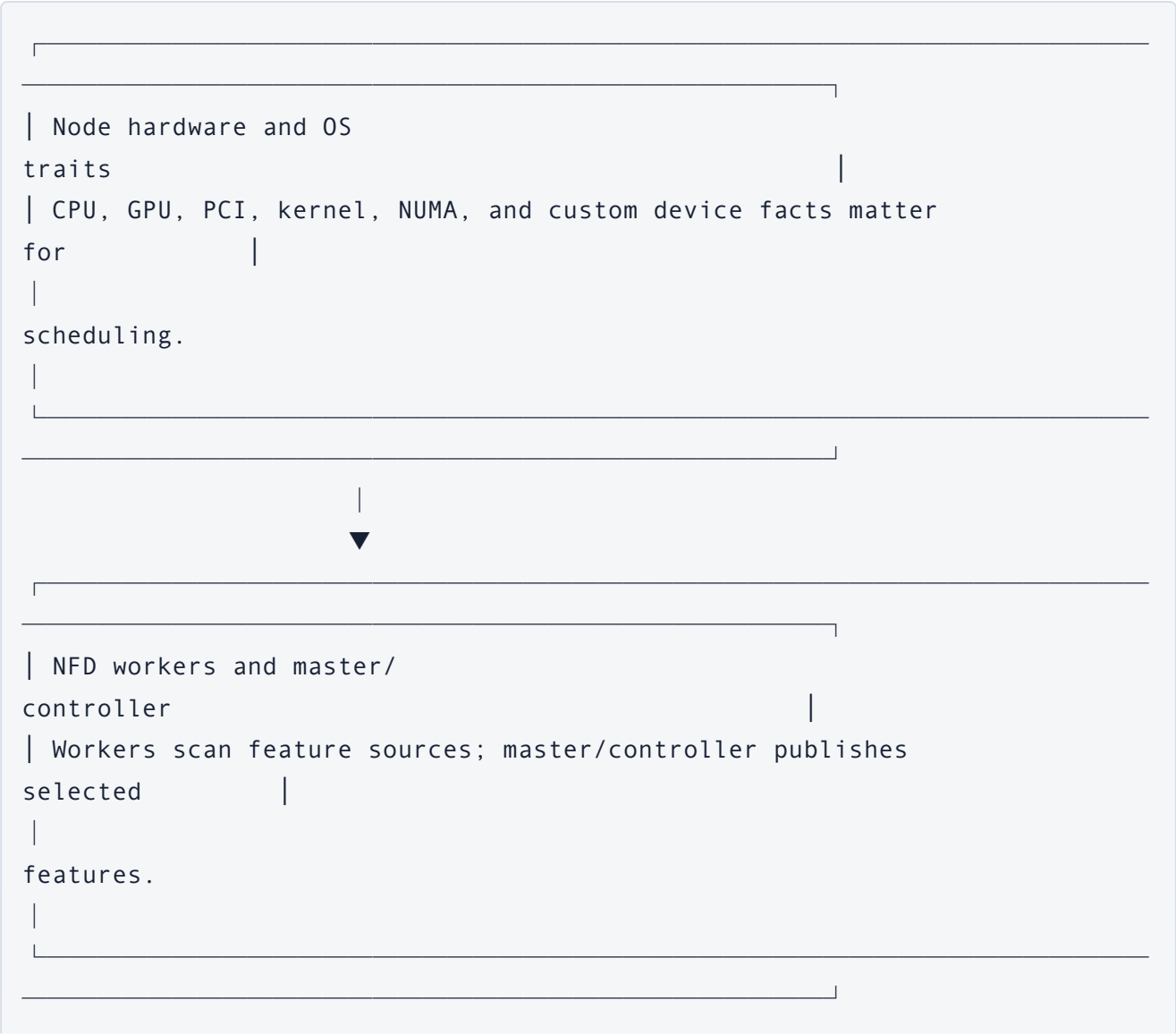
Node Feature Discovery 架构与设计思路分析

原文: [raw/node-feature-discovery-architecture-analysis.md](#) · 仓库: <https://github.com/kubernetes-sigs/node-feature-discovery> · 优先级 P1

一句话定位

Node Feature Discovery 发现 CPU、内核、PCI、NUMA、GPU/加速器等硬件/系统能力，并写成 node labels/features 供调度使用。

核心架构图





模块分层

层 / 模块	职责
nfd-worker	node local feature sources
nfd-master/gc	label publication and cleanup
Feature sources	cpu, kernel, pci, usb, custom hooks
Rules	custom feature labels

关键数据流



设计决策与哲学

- 补齐 节点能力发现 维度：Node Feature Discovery 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：device plugin 暴露可分配资源；NFD 暴露节点能力标签，常作为 GPU/NUMA/硬件调度前置信号。
- 选型价值：它应和 [kubernetes](#), llm-inference, [gpu-sharing](#) 一起看，而不是孤立评估。

相关页面

- [node-feature-discovery](#)
- [kubernetes](#)
- llm-inference
- [gpu-sharing](#)

NVIDIA Dynamo 架构与业务问题（文档重构版）

原文: raw/dynamo-architecture-analysis.md · 仓库: <https://github.com/ai-dynamo/dynamo> · 分析版本 HEAD ed64b7d
本页按当前 README 与官方文档重构；“文档明确”与“架构推导”已尽量区分。

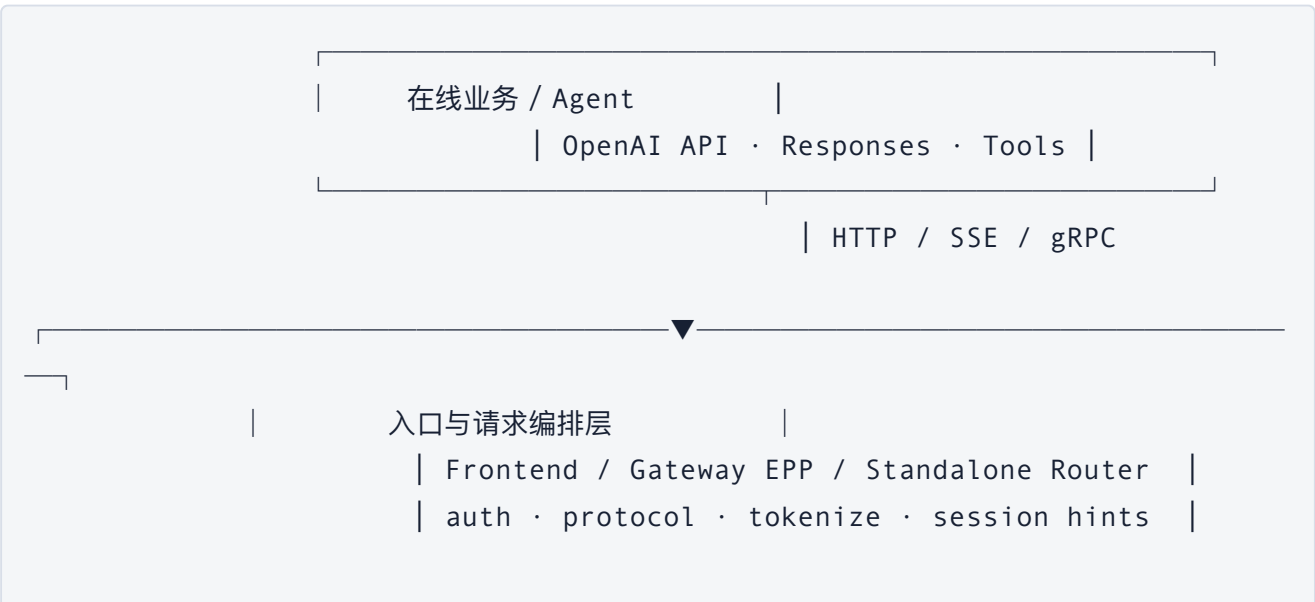
一句话定位

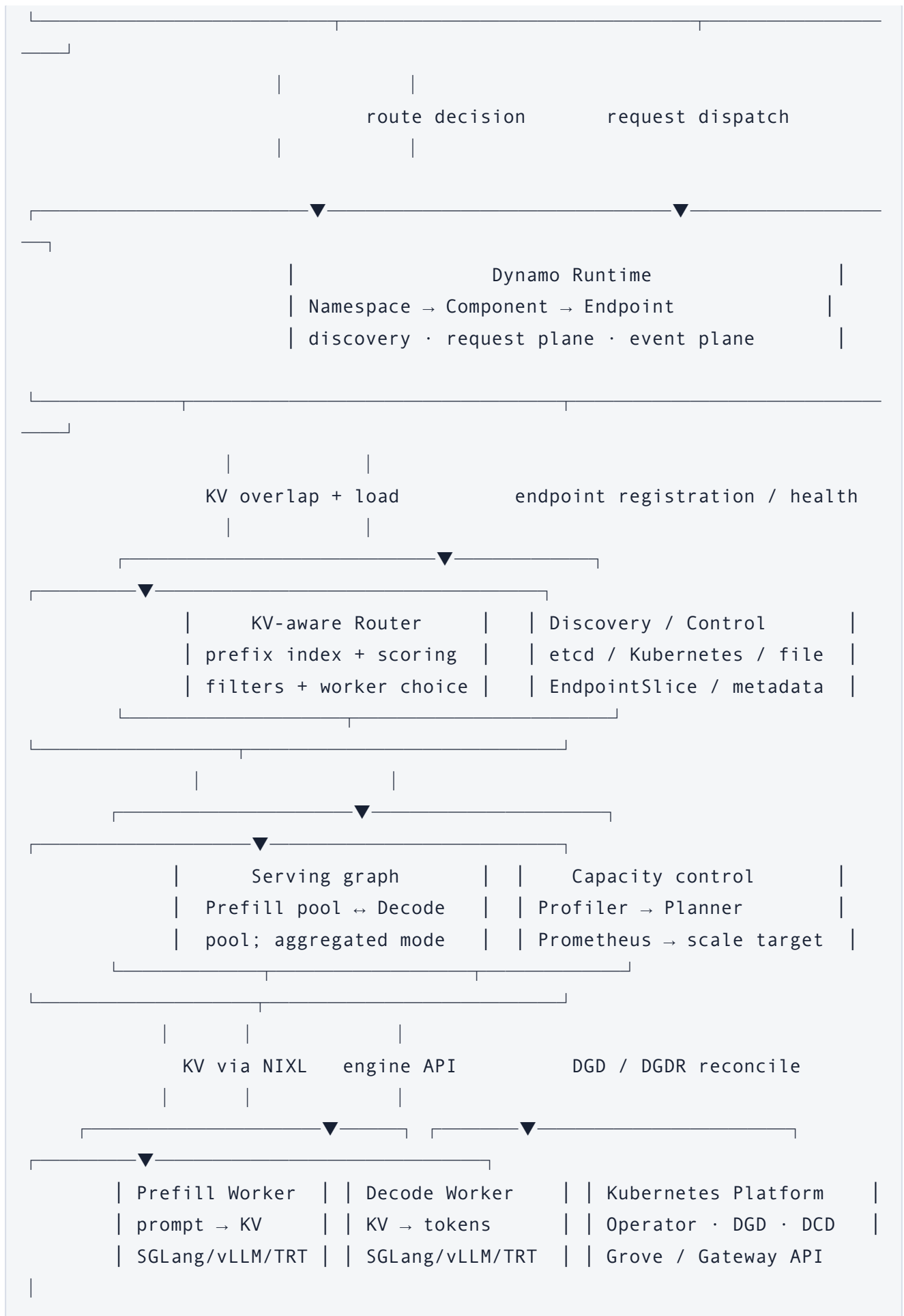
[dynamo](#) 是位于 SGLang、vLLM、TensorRT-LLM 之上的数据中心级 LLM 推理编排层。它解决跨节点、P/D 分离、KV 局部性、SLA 扩缩和故障迁移问题，而不是替代推理引擎本身。

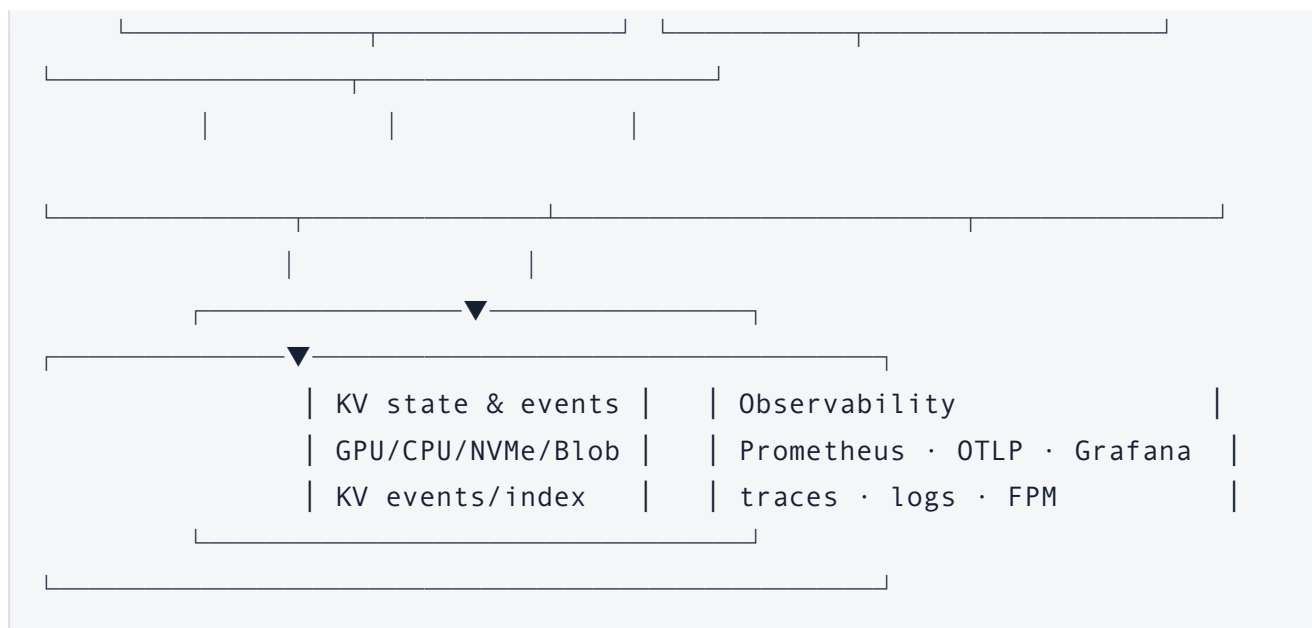
它想解决的业务问题

- 长 prompt 的 prefill 会阻塞 decode：用独立 prefill/decode 池和 NIXL KV transfer 做资源隔离。
- 传统负载均衡不知道缓存：用 KV overlap + 活动负载做成本路由，减少重复 prefill。
- GPU 容量靠人工试错：用 DGDR 描述意图，Profiler/AIConfigurator 生成 DGD，Planner 在线调整副本。
- worker 故障直接暴露给用户：用 migration、worker inhibition、canary 和 graceful shutdown 保持服务连续性。
- 平台入口各有标准：可选 Dynamo Frontend-native 路由，或 Gateway API + GAIE EPP，把入口治理和 serving selection 解耦。

核心架构图





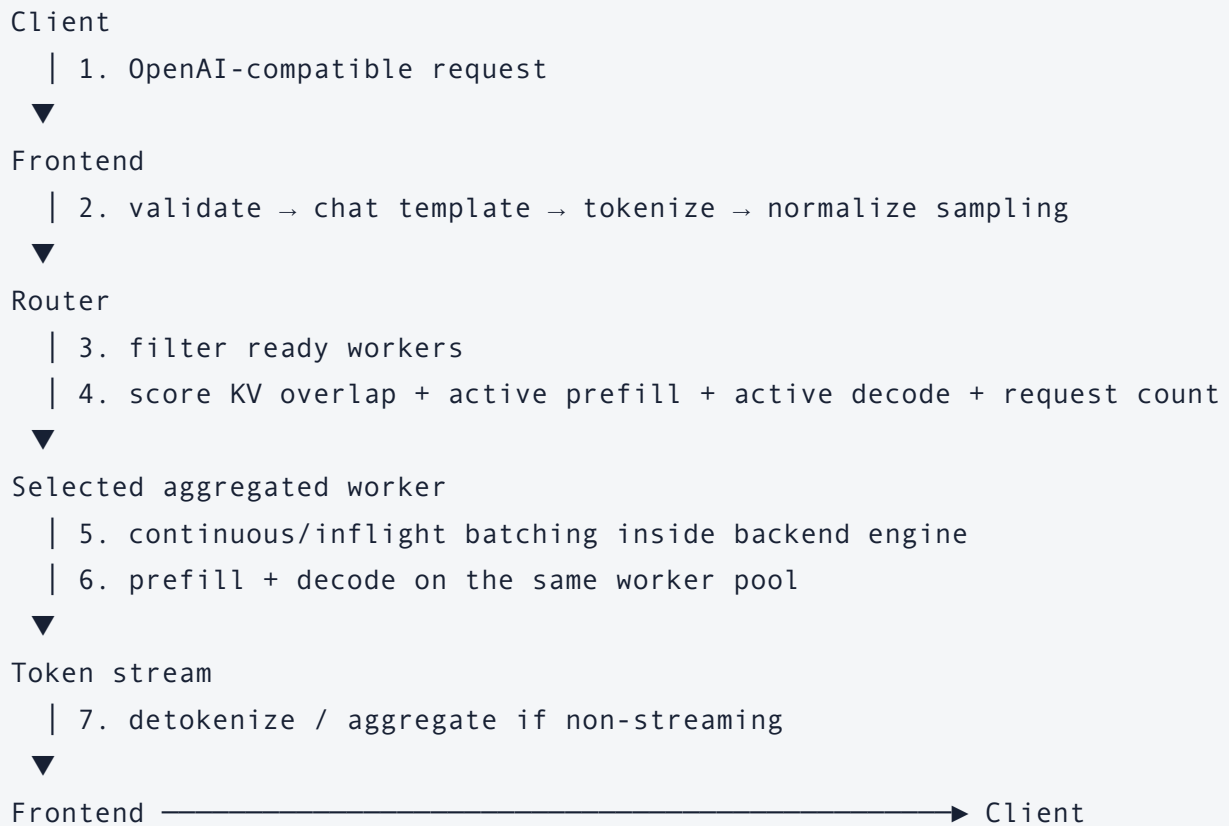


模块分层与边界

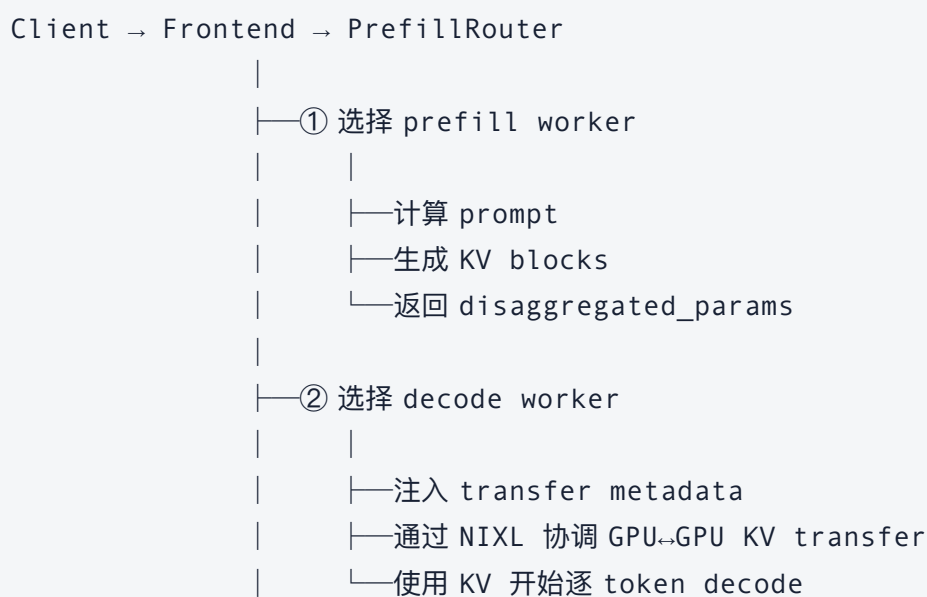
层	功能	边界
入口	Frontend、Gateway EPP、Standalone Router	协议、tokenize、入口治理；不实现 GPU kernel
决策	KV-aware Router、PrefillRouter	选择 worker；不拥有全局 Kubernetes 状态
Runtime	Namespace、Component、Endpoint、discovery/request/event planes	连接与生命周期；不替代 Operator
执行	Prefill/Decode/Aggregated Workers + backend engine	batching、sampling、KV、token generation
KV/传输	NIXL、KV index/events、KVBM/offload	KV transfer 与分层存储；不等价于强一致数据库
控制	DGDR、Profiler、AIConfigurator、Planner、DGD/DCD、Operator、Grove	配置搜索、扩缩、放置；不进入 token 热路径
横切	Prometheus、OTLP、canary、migration	观测与可靠性；不能阻塞推理热路径

请求与控制流

聚合请求

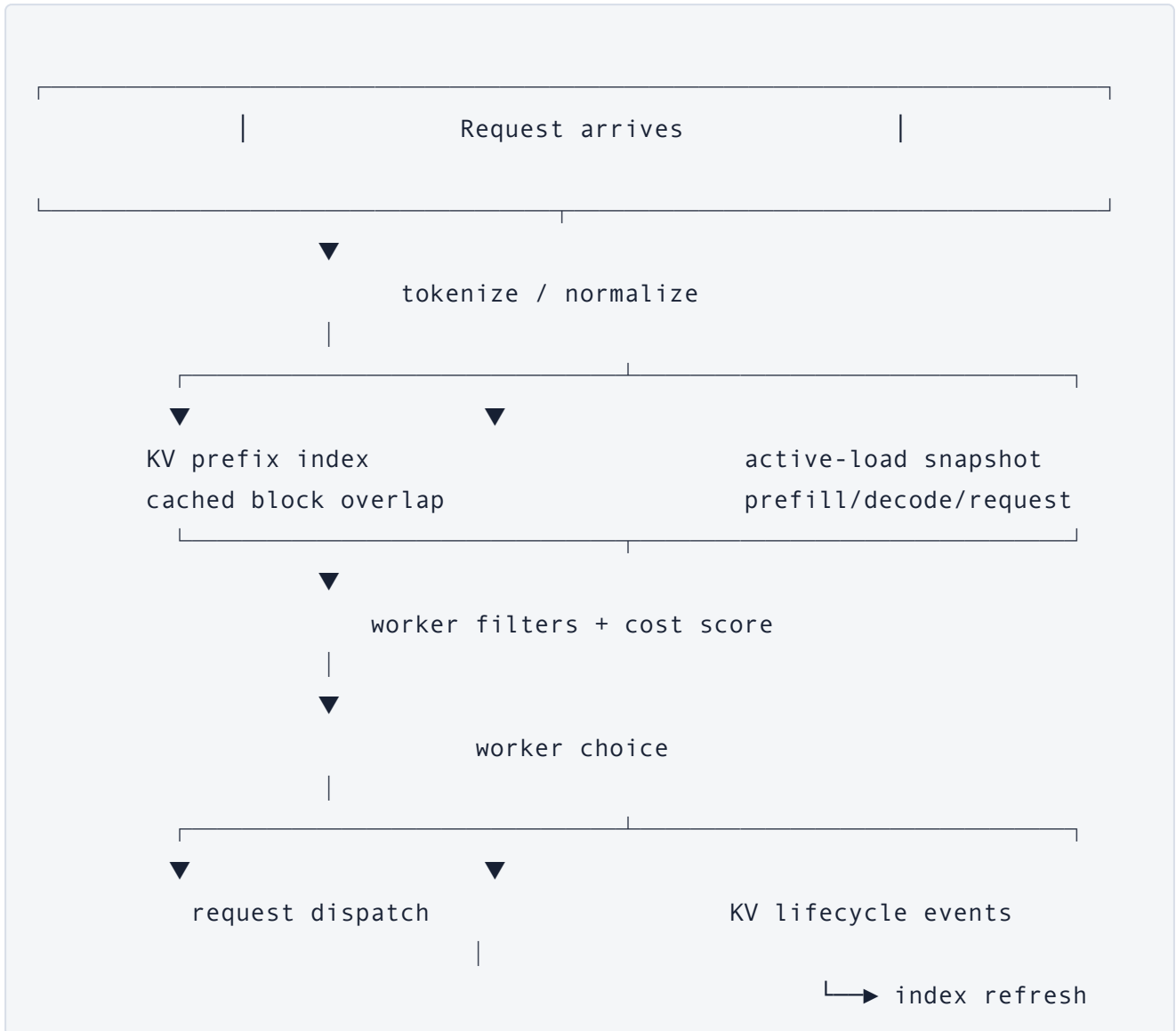


P/D 分离请求



③ stream tokens → Frontend → Client

KV-aware 路由反馈环



路由本质是把 cache locality 转成成本折扣，同时惩罚已分配的 prefill/decode 工作： $\text{adjusted_prefill} = \max(\text{raw_prefill} - \text{overlap_credit}, 0)$ ，再与 projected decode blocks、active request penalty 合并。它不是“永远选择 cache 命中最多的 worker”。详见 inference-routing 与 radix-attention。

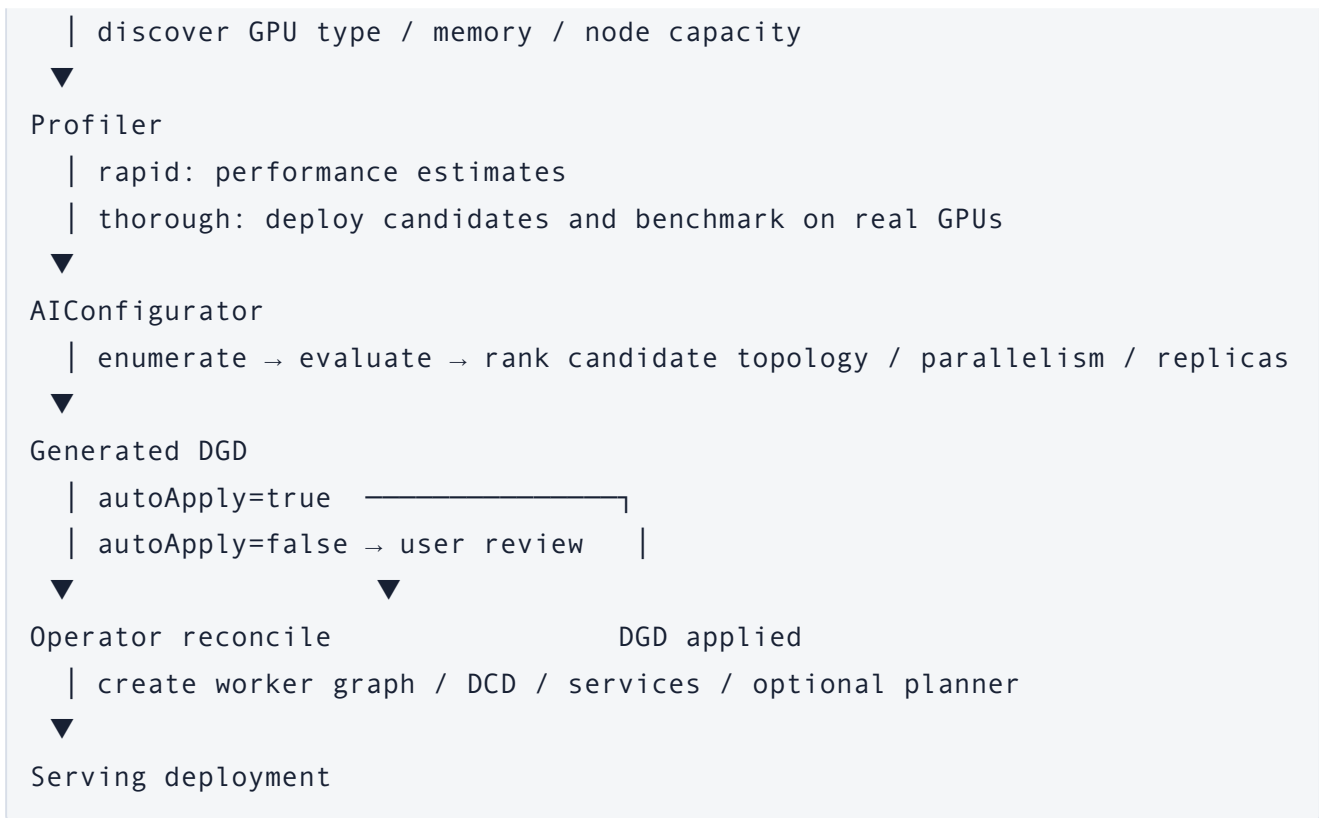
DGDR → DGD → Serving

用户意图

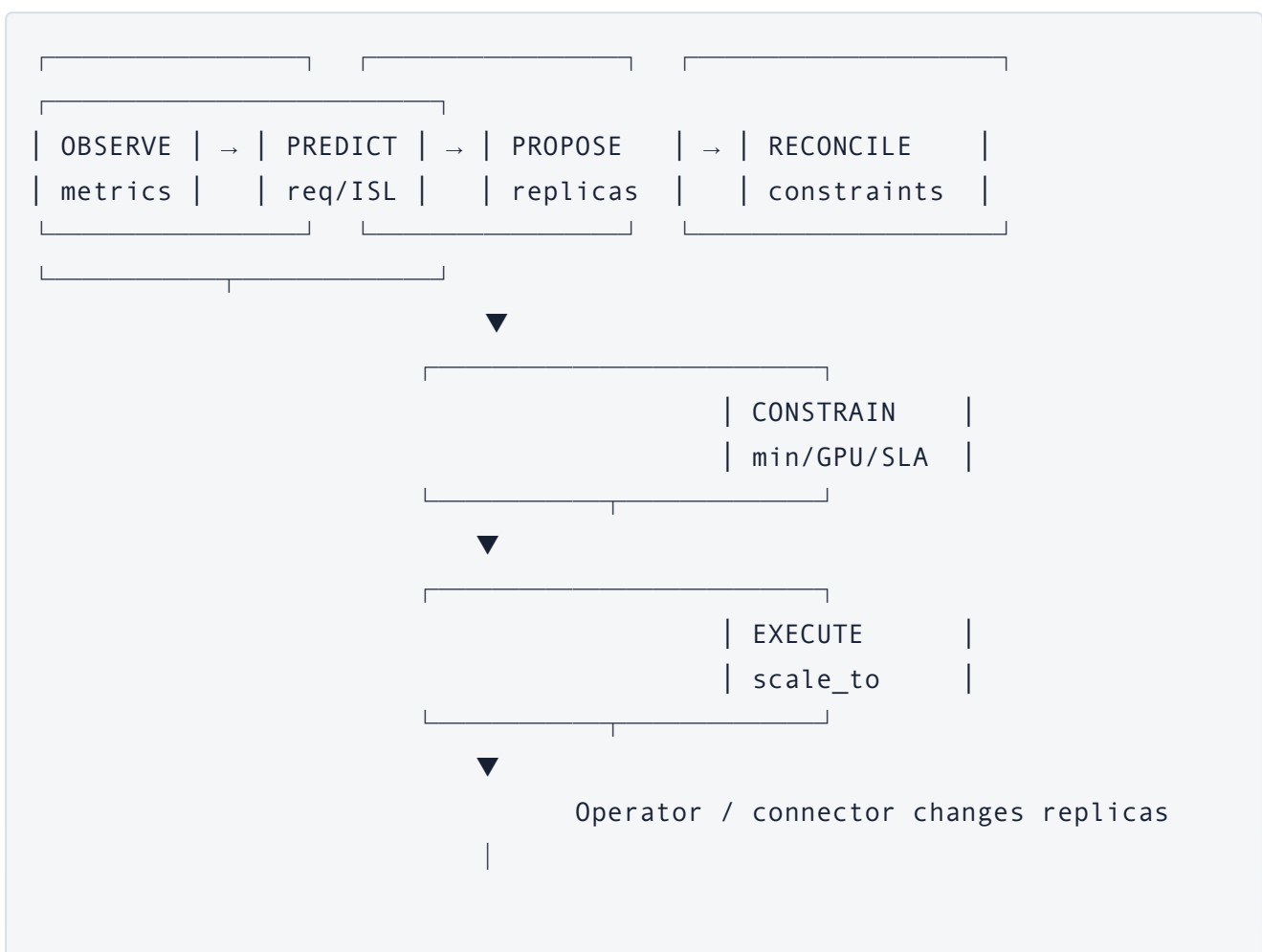
| model + backend + hardware + workload + SLA + optional planner



DGDR



Planner 扩缩闭环



OBSERVE

feedback

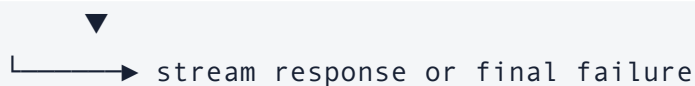
throughput loop 预测持续需求并提供下界，load loop 快速纠正 SLA 压力。它们共享 worker inventory、性能模型、KV hit rate、speculative accept length 与 GPU budget；因此 Planner 不是普通 CPU utilization HPA。

Gateway API 路由

```
Client
|
▼
Kubernetes Gateway / HTTPRoute
| 入口策略、认证、限流、边缘观测
▼
GAIE Endpoint Picker Plugin (EPP)
| 复用 Dynamo KV-aware selection
▼
Selected worker Frontend sidecar
| router-mode=direct
▼
Worker / engine
```

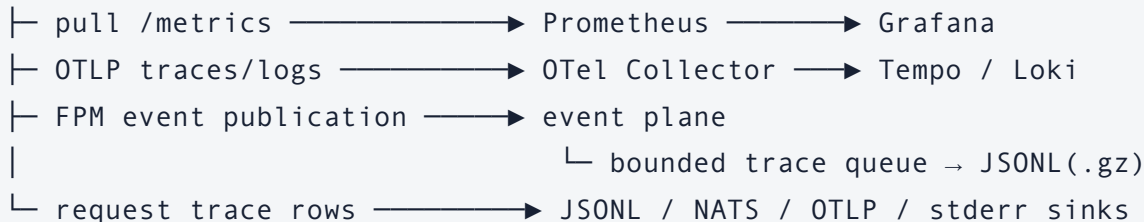
故障迁移

```
request in flight
|
▼
worker error / timeout / disconnect
|
├─不可迁移错误 → reject / return error
|
└─可迁移错误
    | cache token/request state
    | inhibit failed worker locally
    ▼
    choose healthy endpoint
    | migration_limit not exceeded
    └─┬──────────────────────────┬──
    ▼                               |
    replay request                 | limit reached
```



可观测性

Dynamo processes



关键设计判断

- Engine 与 orchestration 分离：复用 backend 的 kernel、batching、sampling 专业能力。
- KV 是集群级资源：既影响路由，也影响 P/D transfer 和分层存储。
- 三平面解耦：request、discovery、event 具有不同延迟与一致性目标。
- Profiler 与 Planner 分工：部署前找配置，部署后调容量。
- Operator 与 Router 分工：最终一致地物化资源，毫秒级做在线选择。
- 入口可替换：Frontend-native 与 Gateway API/EPP 共用 serving selection。
- 旁路可观测：观测落后时丢 trace，不阻塞推理热路径。

选型与限制

适合长上下文、多 GPU/多节点、P/D 分离、需要 KV-aware routing、SLA autoscaling 或快速扩容的生产推理。单模型、单 GPU、没有跨节点协调需求时，直接使用 vllm 或 sglang 更简单。

P/D 分离会增加 KV transfer；Planner 依赖指标与性能模型；migration 需要请求幂等；远端 KV 需要额外的数据驻留和租户隔离治理。README 中的 2x TTFT、7x 启动等数字是特定 benchmark，不应视为通用保证；本次没有运行 GPU benchmark。

相关页面

- [dynamo](#)
- llm-inference
- inference-routing
- disaggregated-serving
- kv-cache-offload
- [model-serving-operator](#)
- ai-gateway

NVIDIA GPU Operator

NVIDIA GPU 软件栈的 Kubernetes Operator，传统 ClusterPolicy/NVIDIADriver 路径管理 driver、container-toolkit、device-plugin、DCGM、MIG manager、sandbox/vGPU 等组件，当前 GPUCluster 提供独立的 DRA 管理路径。历史架构详见 [src-gpu-operator-architecture](#)。

证据说明：2026-09-27 官方默认分支快照为 [60526e35efee](#)，commit 不代表 release。[src-gpu-operator-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，DRA 分配与节点准备见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

Operator reconcile NVIDIA 节点软件及选定 operands，不选择单次 Pod 获得哪块 GPU。按 [26.7 文档](#)，ClusterPolicy 对应 [k8s-device-plugin](#) 路径，GPUCluster 对应 [dra-driver-nvidia-gpu](#) 管理路径，两者不能在同一集群并存，也不支持直接原地迁移。GPUCluster 需独立 NVIDIADriver/预装 driver 和兼容 [cdi](#) 的 runtime；这些条件不能由 [node-feature-discovery](#) 标签或 Operator Ready 替代。

架构边界

GPU Operator 管“节点软件栈怎么安装、升级、保持健康”，不是单次 Pod GPU allocation 算法。它通常部署或管理 [k8s-device-plugin](#)，并与 [hami](#)、[dra-driver-nvidia-gpu](#) 处在不同层。

选型判断

- 集群管理员管理 NVIDIA 软件栈：GPU Operator。
- kubelet 层暴露 GPU：[k8s-device-plugin](#)。
- 新 DRA API 分配：[dra-driver-nvidia-gpu](#)。
- 共享/虚拟化调度：[hami](#)。

NVIDIA GPU Operator 架构与设计思路分析

原文: raw/gpu-operator-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/gpu-operator> · 分析版本 HEAD 0219120 (2026-06-11)

一句话定位

NVIDIA GPU 软件栈的 Kubernetes Operator，用 ClusterPolicy/NVIDIADriver CRD 驱动 driver、container-toolkit、device-plugin、DCGM、MIG manager、sandbox/vGPU 等组件的声明式部署和升级。它管的是 GPU node 软件生命周期，而不是单次 Pod GPU 分配算法。这页和 [kubernetes](#) k8s-operator [cdi gpu-sharing](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图



模块分层

层 / 模块	职责
Controller manager	启动 controller-runtime manager、scheme、webhook、leader election、ClusterPolicy 和 upgrade controller。
ClusterPolicy controller	单例 ClusterPolicy reconcile，按状态机逐步部署 operands 并更新状态/metrics。
State/resources renderer	按 CR spec 和 cluster info 渲染 DaemonSet/Service/ConfigMap/CRD 等对象并 create/update。
CRD/API	定义 ClusterPolicy、NVIDIADriver 等声明式 spec/status。
升级与验证	管理 driver upgrade、operator validator、组件就绪检查。

关键数据流

```
Admin applies ClusterPolicy
|
├─ controller-runtime manager enqueues ClusterPolicy reconcile
|
├─ controller initializes singleton, cluster info, node labels
|
├─ state machine renders each operand manifest from assets/state-*
|
├─ create/update objects and check DaemonSet/Deployment readiness
|
└─ conditions/metrics expose Ready/NotReady and driver upgrade state
```

设计决策与哲学

- Operator 管组件生命周期，不直接做 GPU allocation：它部署 device plugin、driver、runtime、DCGM 等 operands，真正 Allocate 仍由 device plugin/kubelet 路径完成。
- 状态机比单次 reconcile 更贴合 GPU 软件栈：ClusterPolicyController.step() 逐状态推进，便于表达 driver/toolkit/plugin/monitoring 的依赖顺序。

- 渲染资产与 cluster info 解耦: internal/state/driver.go 根据 OpenShift、precompiled、kernel、proxy 等信息生成实际 DaemonSet。

与同类项目的架构差异

维度	NVIDIA GPU Operator	同类 A	同类 B
定位	GPU 软件栈 Operator	k8s-device-plugin: kubelet plugin	DRA driver: 新 API 分配
核心对象	ClusterPolicy/NVIDIADriver	DaemonSet + config	ResourceClaim/ResourceSlice
主要用户	集群管理员	平台/节点管理员	DRA workload/platform

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- k8s-operator
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)

NVIDIA k8s-device-plugin

NVIDIA 官方 Kubernetes device plugin，把 GPU/MIG/vGPU 发现为 kubelet extended resources，并在 Allocate 阶段通过 env、volume-mounts 或 CDI annotations 把设备传给容器。详见 [src-k8s-device-plugin-architecture](#)。

证据说明：2026-09-27 官方默认分支快照为 [86142cf1a93f](#)，commit 不代表 release。[src-k8s-device-plugin-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，DRA 对照路径见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

插件向 kubelet 注册 extended resource，通过 ListAndWatch 报告设备与健康，在 Allocate 中返回设备交接配置；它没有 [kubernetes-dra](#) 的丰富 Claim/库存/条件模型。[node-feature-discovery](#) 的发现信号和 [gpu-operator](#) 的组件部署是相邻职责；其与 [dra-driver-nvidia-gpu](#) 的路径选择应按所需 API、driver 支持与资源所有权评估。

架构边界

它是 GPU 暴露基础层，不负责安装 driver，也不负责全局调度策略。GPU Operator 常用来部署和管理它；HAMi 经典路径使用自己的 device plugin，不能与官方插件在同一节点争用 `nvidia.com/gpu`；DRA driver 则采用 ResourceClaim/ResourceSlice 分配模型。

选型判断

适合需要理解 Kubernetes GPU allocation 基础机制、MIG/MPS/time-slicing/CDI 策略的人。做 AI serving 平台时，它是 [gpu-operator](#)、[hami](#)、[kubernetes-dra](#) 前必须理解的底层节点。

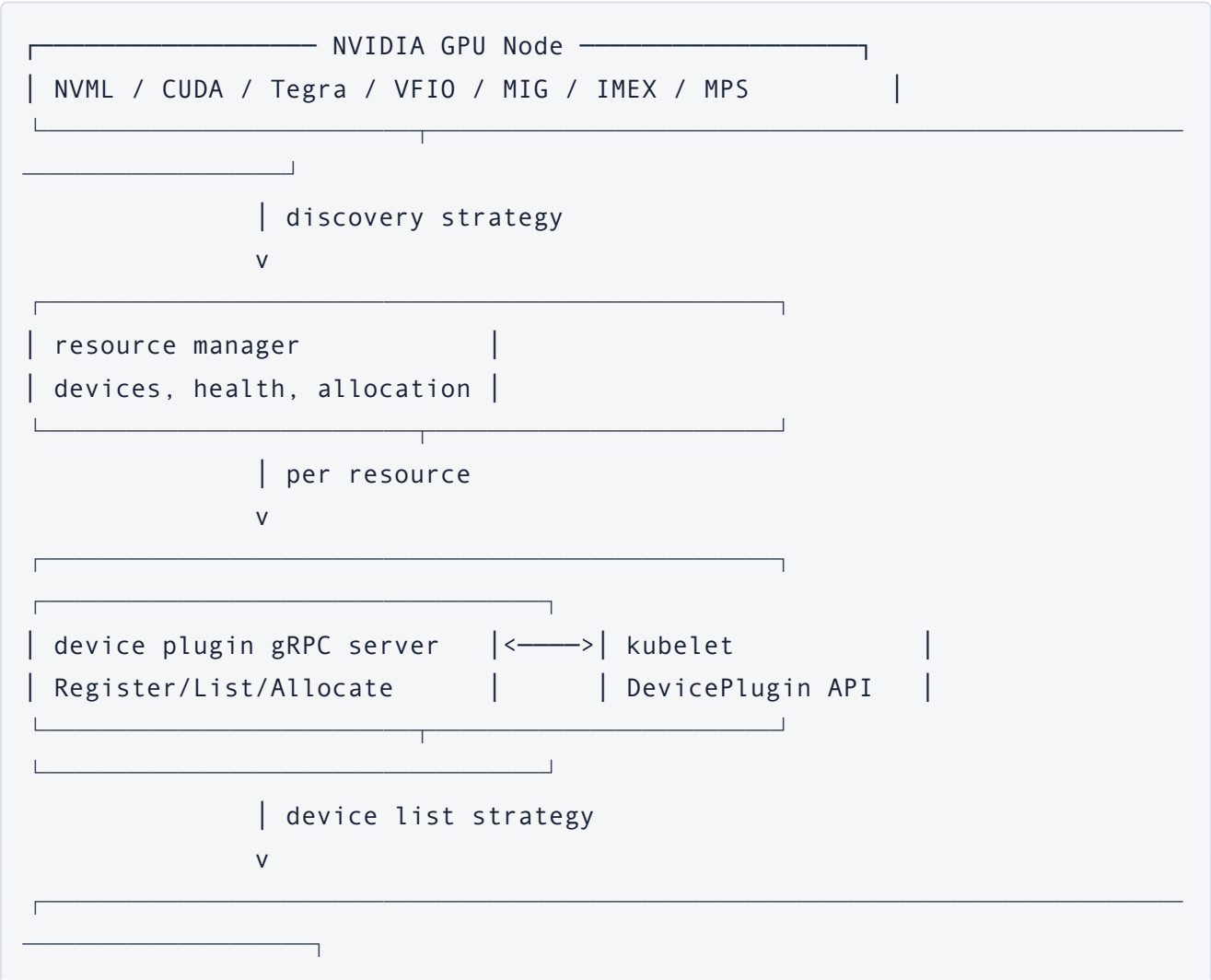
NVIDIA k8s-device-plugin 架构与设计思路分析

原文: raw/k8s-device-plugin-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/k8s-device-plugin> · 分析版本 HEAD 8688949 (2026-06-10)

一句话定位

NVIDIA 官方 Kubernetes device plugin，把节点上的 GPU/MIG/vGPU 发现为 kubelet extended resources，并在 Allocate 阶段通过 env、volume-mounts 或 CDI annotations 把设备传给容器。它是 GPU 暴露的基础层，上层 GPU Operator/HAMi/DRA 都会和它形成互补或替代关系。本页和 [kubernetes cdi gpu-sharing k8s-gpu-device-stack](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图




```
| container envvar / volume-mounts / CDI annotations |  
|  
|  
|
```

模块分层

层 / 模块	职责
CLI/config	解析 MIG strategy、device-list strategy、CDI/MPS/IMEX/driver root 等配置。
Plugin manager	解析 discovery strategy，创建 CDI handler、resource manager 和 per-resource plugin。
Resource manager	基于 NVML/CUDA/Tegra/VFIO 发现设备，处理 MIG/resource mapping、preferred allocation、health。
Device plugin server	实现 kubelet device plugin gRPC: Serve/Register/ListAndWatch/Allocate/PreStart。
Feature discovery/ MPS	提供 GPU label、MPS daemon 和 node feature discovery 集成。

关键数据流

```
Node starts nvidia-device-plugin DaemonSet  
|  
├─ main parses config/env flags and selected discovery strategy  
|  
├─ plugin-manager creates CDI spec and resource managers  
|  
├─ per-resource plugin registers Unix socket with kubelet  
|  
├─ ListAndWatch reports healthy GPU/MIG resources  
|  
└─ Allocate validates request and returns env/mount/CDI device edits
```

设计决策与哲学

- 保持 kubelet device plugin API 的基础语义：项目核心是发现、健康检查和 Allocate，不承担全局调度或多租户策略。
- 多种 device list strategy 兼容不同 runtime：envvar、volume-mounts、CDI annotations 并存，让老 runtime 和新 CDI 工作流都能接入。

- `resource manager` 隔离硬件发现差异: `internal/resource/factory.go` 按 NVML/Tegra/VFIO 选择 manager, plugin server 不直接关心硬件细节。

与同类项目的架构差异

维度	NVIDIA k8s-device-plugin	同类 A	同类 B
定位	官方 GPU device plugin 基础层	GPU Operator: 部署管理它	HAMi: 共享调度/虚拟化扩展
API	Kubelet DevicePlugin v1beta1	Operator CRD	scheduler extender + annotations
粒度	GPU/MIG/MPS/time-slicing/CDI	软件组件栈	memory/core/count 共享

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)
- dra

OME

OME (Open Model Engine) 是 Kubernetes model serving operator/control plane, 围绕 CRD/controller、model agent、runtime selector 和 accelerator configuration 组织模型部署。

解决的问题

模型服务平台需要把模型生命周期、推理 runtime、硬件加速器和部署策略解耦, 否则每接入一种 engine 或 GPU 都要复制一套 controller。OME 把 runtime 选择和 accelerator 配置提升为可复用的控制面抽象。

核心边界

OME 主要解决“模型怎么被部署和由哪个 runtime 承载”, 不以在线 KV-aware routing、P/D traffic orchestration 或 API gateway 为核心。它适合作为 serving platform 的生命周期底座, 再组合专用 router 和 engine。

适用场景

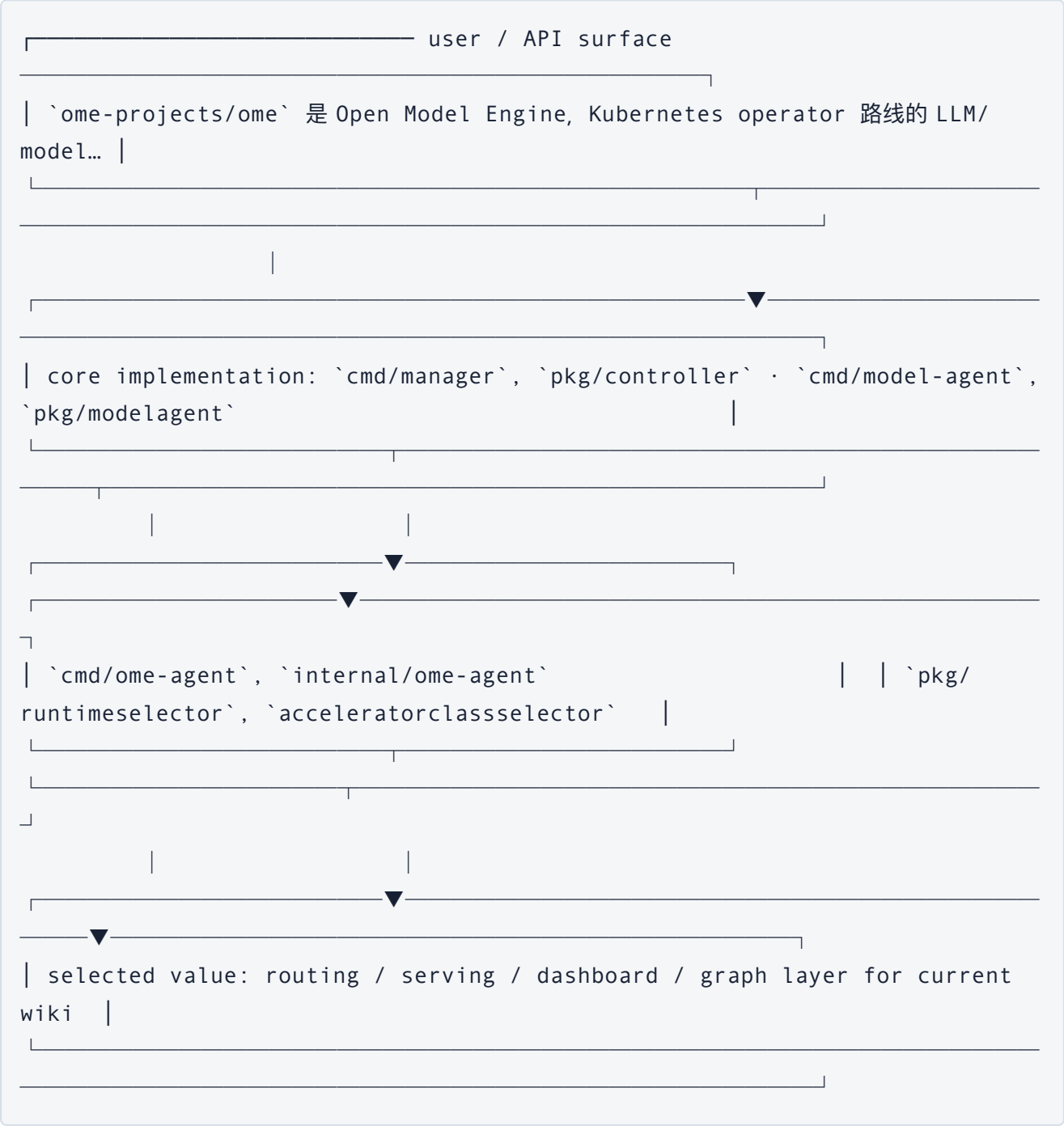
适合平台团队建设 runtime/accelerator 抽象、支持多种模型 backend, 并希望保持 Kubernetes operator 边界清晰的场景。需要统一 predictive + GenAI API 看 [kserve](#); 需要完整 LLM serving 看 [llm-d](#)、[dynamo](#) 或 [kthena](#)。

详见 [src-ome-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

OME 架构与设计思路分析

ome-projects/ome 是 Open Model Engine, Kubernetes operator 路线的 LLM/model serving 控制面。仓库包含 manager、model-agent、ome-agent、CRD/config/webhook、runtime selector、accelerator class selector、storage/model download、OEP proposals。

核心架构图



模块分层

层/目录	责任
cmd/manager, pkg/controller	operator manager 和 controllers。
cmd/model-agent, pkg/modelagent	模型拉取/管理 agent。
cmd/ome-agent, internal/ome-agent	运行时 sidecar/agent。
pkg/runtimeselector, acceleratorclassselector	runtime/accelerator 选择。

关键数据流

1. 用户提交 OME model/workload CR。
2. controller 选择 runtime/accelerator/storage 并创建 K8s workload。
3. model-agent/ome-agent 处理模型下载、探测和运行时协作。

设计决策

- operator-first，强调模型生命周期和硬件适配。
- accelerator configs 的 data parallel override 是活跃问题。
- OEP 展示路线图比当前实现更宽。

对比定位

和 KServe 相比，OME 更 LLM/accelerator-aware；和 KubeAI 相比更偏 operator 基座和扩展提案。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

OpenKruise Agents

OpenKruise Agents 是 OpenKruise 面向 AI Agent sandbox lifecycle management 的 Kubernetes 原生平台。它把 Sandbox / SandboxSet / SandboxClaim / SandboxTemplate CRD、E2B-compatible API、Envoy 路由扩展、agent-runtime sidecar、identity token 和动态 CSI mount 组合成一套“在 K8s 上运营 Agent sandbox”的控制面。

详细架构见 [src-openkruise-agents-architecture](#)。

关键事实

- 仓库: [openkruise/agents](#)
- 当前核验: GitHub API 显示默认分支 master, 最近 push 2026-06-12, 约 202 stars / 78 forks, Apache-2.0, 主语言 Go
- 本 wiki 分析版本: HEAD 0e58df8 (2026-06-12)
- 核心入口: agent-sandbox-controller、sandbox-manager、traffic-extension、sandbox-gateway
- 关键 API: Sandbox、SandboxSet、SandboxClaim、SandboxTemplate
- 产品入口: E2B-compatible / sandboxes、/ snapshots、pause/resume/connect/timeout/debug 等 API
- 典型场景: code interpreter、cloud desktop、Claude Code / OpenClaw sandbox、RL/HITL/open-world training、长寿命 workspace

架构定位

E2B 请求到 Sandbox 的流程

```
E2B API / CRD request → sandbox-manager
├─ create / claim / clone / checkpoint
└─ route + identity + key storage
    ↓
    SandboxClaim / SandboxSet / Sandbox
    ↓
    agent-sandbox controller
    ↓
    Pod + PVC + agent-runtime / CSI
    ↓
    Envoy ext_proc → Sandbox Pod IP:port
```

OpenKruise Agents 处在 agent-sandbox 和 [agentcube](#) 中间偏平台化的一层：

- 比 agent-sandbox 更完整：不仅有 sandbox lifecycle primitive，还把 warm pool、claim、E2B API、路由、identity、CSI 和 runtime sidecar 接起来。
- 比 [agentcube](#) 更底层：不主打 AgentRuntime / CodeInterpreter invocation 产品抽象，而是提供更通用的 sandbox 管理、clone/checkpoint、route 和 E2B 兼容面。
- 比 [substrate](#) 更 K8s CRD 化：Substrate 聚焦 actor/worker multiplexing 和 wake routing；OpenKruise Agents 聚焦可声明、可预热、可暂停/恢复、可路由的 sandbox。
- 可作为 Agent framework 的远端 workspace backend：AgentScope、coding agent、browser/desktop agent 可以把它当作 E2B-like 执行环境。

核心组件

组件	作用
agent-sandbox-controller	K8s controller-runtime 控制面，管理 Sandbox、SandboxSet、SandboxClaim、SandboxUpdateOps、SecurityTokenRefresh。
sandbox-manager	E2B-compatible HTTP API 控制面，负责 claim/clone/pause/resume/delete checkpoint、API key storage、team permission 和 route sync。
traffic-extension	Envoy external processing gRPC 服务，把 sandbox 请求改写为 original-dst 路由。
sandbox-gateway	Envoy Go HTTP filter 版本的数据面扩展，解析 sandboxID/port 并写 dynamic metadata。
sandboxcr infra	Infrastructure / Sandbox 抽象的 K8s CRD 实现，把 API 请求落到 SandboxClaim/Sandbox/PVC/Pod。
agent-runtime / identity / CSI	sandbox 内 runtime 初始化、短期 token 传播、动态 volume mount 的扩展点。

关键判断

- Warm pool 是控制面协议：SandboxSet 保持可用库存，SandboxClaim 显式领取；这比简单 Pod pool 更适合并发、超时、失败保留和滚动更新。
- E2B API 降低接入成本：上层 SDK 可以用 familiar API 创建、暂停、恢复、连接 sandbox；平台侧仍保留 K8s CRD/GitOps/RBAC 能力。
- 路由不走 Service per sandbox：Envoy ext_proc / Go filter 用 sandboxID 查 route registry，把流量导到 Pod IP:port，适合大量短会话/长会话混合的 sandbox。
- 安全边界要继续补齐：identity provider、API key storage、token propagation、runtimeClass、NetworkPolicy 和 audit 是生产采用前必须验证的部分。

什么时候优先看它

- 需要在 K8s 上搭建 E2B-like sandbox service;
- 希望同时拥有 CRD 管理面和 SDK/API 产品面;
- 需要 warm pool 降低 Agent / Code Interpreter 冷启动;
- 需要 pause/resume、checkpoint/clone、动态存储挂载、路由和多租户 namespace/team;
- 想把 AI Agent workspace 从本地/Docker/E2B SaaS 迁到自有 Kubernetes。

什么时候不适合

- 只需要一个本地 Agent 框架或 Python workflow runtime;
- 只想学习最小 sandbox CRD，不想同时理解 manager/gateway/identity/CSI;
- 对强 runtime enforcement 的要求高于生命周期管理，此时还要叠加 [cloud-native-security](#)、OPA/egress gateway、gVisor/Kata/seccomp 等机制;
- 当前业务不能接受快速演进 API 和 roadmap 中尚未完成的 storage/network/runtime/scheduling/observability 能力。

OpenKruise Agents 架构与设计思路分析

原文: raw/openkruise-agents-architecture-analysis.md · 仓库: <https://github.com/openkruise/agents> · 分析版本 HEAD 0e58df8 (2026-06-12)

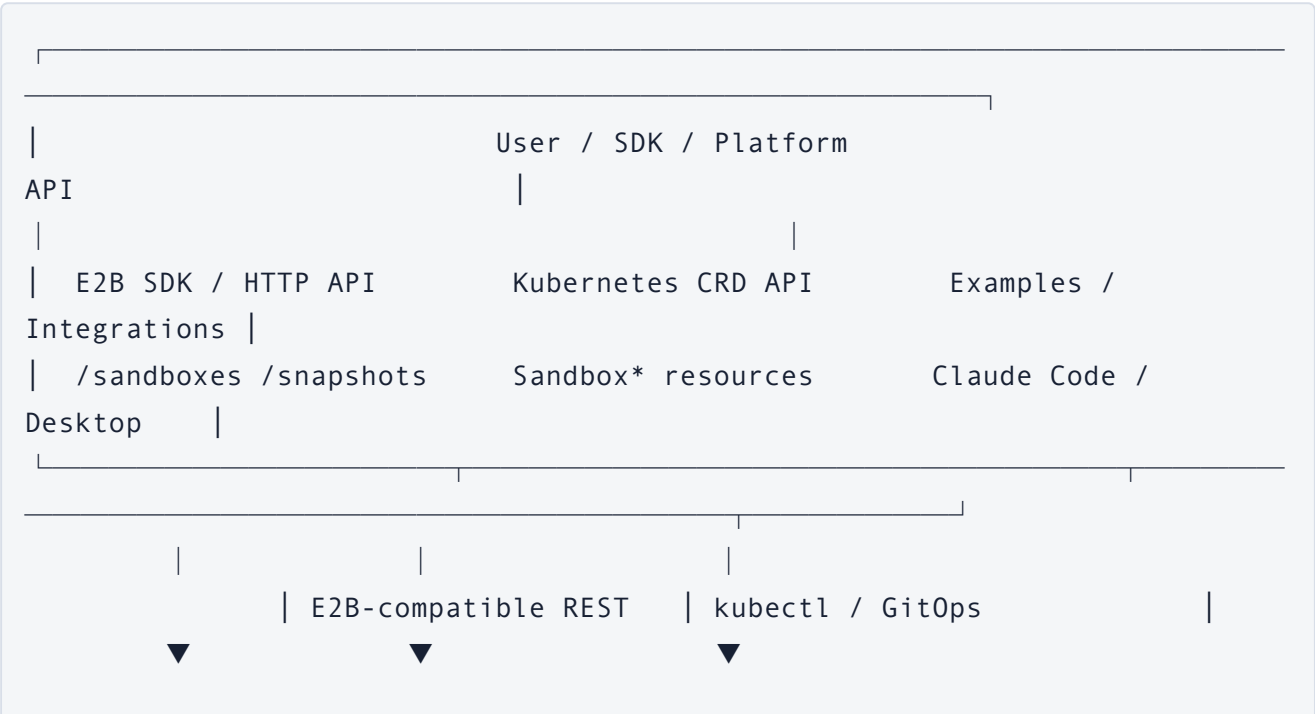
一句话定位

openkruise/agents 是 OpenKruise 面向 AI Agent sandbox lifecycle management 的 Kubernetes 原生平台层。它不只是一个 Sandbox CRD controller，而是把四类能力放在同一个工程里：

- 用 Sandbox / SandboxSet / SandboxClaim / SandboxTemplate 表达可暂停、可预热、可领取、可升级的 Agent sandbox。
- 用 agent-sandbox-controller 管 Pod/PVC/template/hash/status/rolling update/warm pool 等生命周期。
- 用 sandbox-manager 暴露 E2B-compatible API，把“创建 sandbox / clone checkpoint / pause / resume / route”变成产品 API。
- 用 traffic-extension / sandbox-gateway 接入 Envoy，把外部请求按 sandbox identity 路由到具体 Pod IP 和端口。

它和 agent-sandbox 的关系更像“上层发行版 + 兼容实现 + 产品化控制面”：README 明确说明会保持与 SIG agent-sandbox API 的兼容，同时在 SIG API 可用前内置 sandbox API 和实现。

核心架构图







模块分层

层 / 模块	职责
CRD API	定义 Sandbox、SandboxSet、SandboxClaim、SandboxTemplate，覆盖 pause、persistent contents、runtimes、claim、template、upgrade、dynamic volume mount。
K8s 控制面	controller-runtime manager；注册 Sandbox/SandboxSet/SandboxClaim/SandboxUpdateOps/SecurityTokenRefresh controller、webhook、field index、metrics cleanup。
API 控制面	暴露 E2B-compatible API；把 request adapter、auth/key storage、infra backend、memberlist peer 和 route proxy 组合成 sandbox 管理服务。
Sandbox infra	定义抽象 Infrastructure / Sandbox 接口；sandboxcr 实现用 K8s CRD 完成 claim、clone、pause/resume、checkpoint、CSI mount、identity token。
路由数据面	Envoy ext_proc 或 Go HTTP filter；从请求解析 sandboxID/port，查 route registry，写入 original-dst 元数据并做 peer 同步。
Runtime 扩展	agent-runtime sidecar、CSI volume mount provider、identity provider 和 token propagation 扩展点。
示例与集成	code interpreter、desktop、OpenClaw、Claude Code 等场景，展示如何把热池、E2B SDK、sandbox image 和 session 复用组合起来。

关键对象

Sandbox

Sandbox 是单个运行环境的期望状态。重要字段包括 paused、persistentContents、shutdownTime / pauseTime、runtimes、lifecycle、upgradePolicy、template / templateRef / volumeClaimTemplates。它把暂停、恢复、持久文件系统、runtime sidecar、动态存储和升级策略纳入同一个 K8s 对象。

SandboxSet

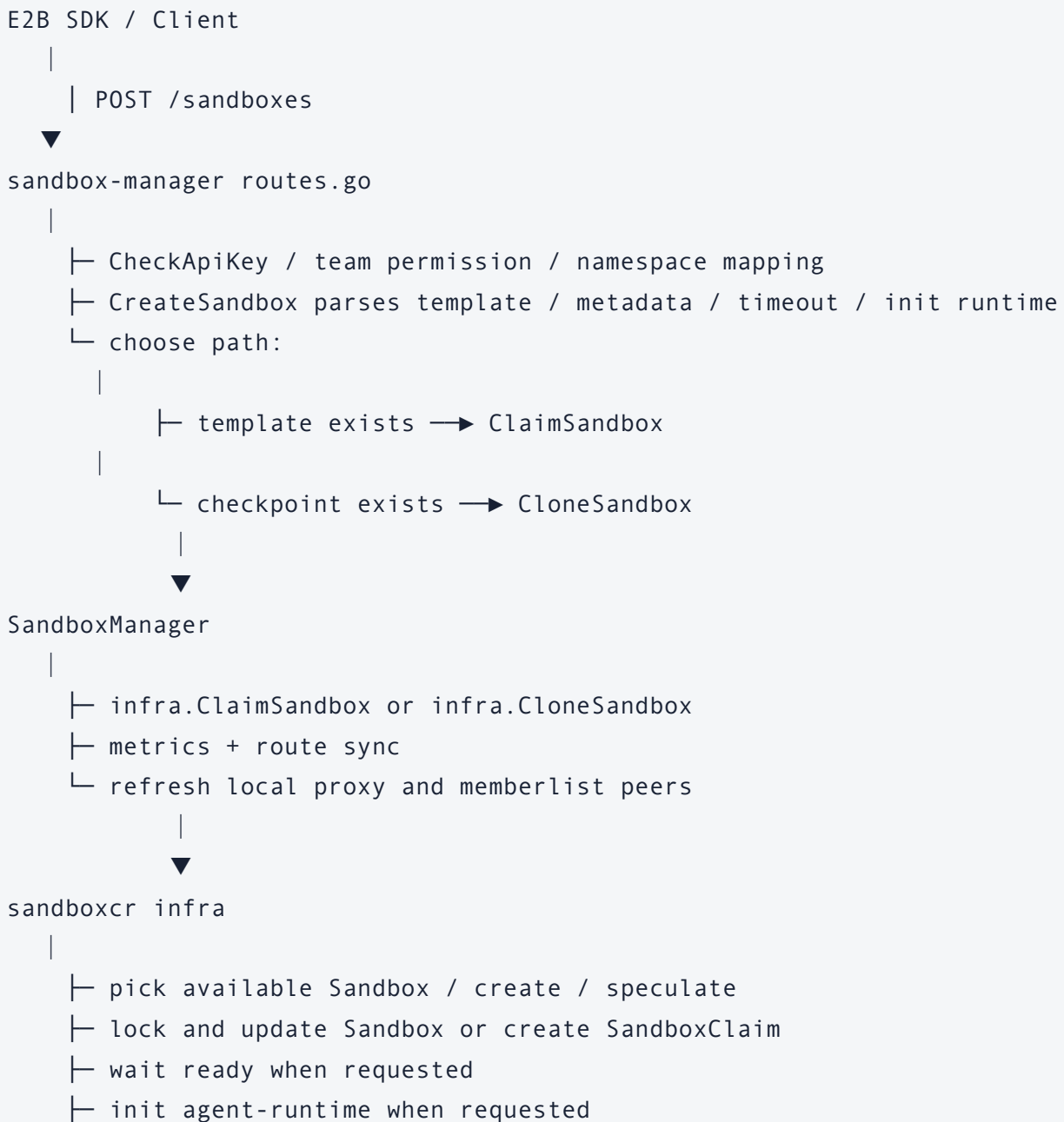
SandboxSet 维护一组“未被 claim 的可用 sandbox”，本质是 warm pool 和滚动更新控制器。replicas 表示需要保持的 unused sandboxes 数量，status 跟踪 available、updated、current/update revision，labels/ annotations 则承载 pool、template、claim、owner、hash、lock、restore-from、init-runtime request、memberlist url 等协议状态。

SandboxClaim

SandboxClaim 是把“用户请求一个 sandbox”变成控制面协议的关键对象。它通过 `templateName` 选择 `pool/template`，用 `createOnNoStock` 控制无库存时是否冷创建，用 `inplaceUpdate` 支持 `image/CPU` 更新，用 `dynamicVolumesMount` 支持请求时 CSI 挂载，并用 `reserveFailedSandbox` 支持失败现场保留。

关键数据流

E2B API 创建 sandbox



- └─ issue and propagate identity token when feature enabled
- └─ mount CSI dynamic volumes when requested

请求路由到 sandbox

```
Client request
|
▼
Envoy listener
|
└─ traffic-extension ext_proc
|   └─ parse sandboxID / port through adapter
|   └─ route registry lookup
|   └─ require route state = running
|   └─ set x-envoy-original-dst-host = route.IP:port
|
└─ or sandbox-gateway Go filter
    └─ parse sandboxID / port
    └─ route registry lookup
    └─ set dynamic metadata envoy.lb.original_dst.host
        |
        ▼
        Envoy original dst cluster
            |
            ▼
            Sandbox Pod IP:port
```

Warm pool / claim 生命周期

```
SandboxTemplate
|
▼
SandboxSet(replicas = unused capacity)
|
└─ create available Sandboxes
└─ keep template hash / revision status
└─ rolling update pool when template changes

SandboxClaim(templateName)
|
```

```

└─ select available Sandbox from matching SandboxSet
└─ lock and mark claimed / owner / timestamp
└─ apply labels, annotations, env vars and inplace update
└─ optionally initialize runtime and dynamic CSI mount
└─ wait ready -> Completed

```

设计决策

- 把 warm pool 做成声明式资源，而不是进程内队列：SandboxSet 让“保持 N 个可用 sandbox”由 K8s controller 持续收敛；SandboxClaim 则把领取动作显式化，便于处理并发、超时、失败保留和 owner/label 不变量。
- E2B API 是产品入口，CRD 是平台入口：开发者可以通过 E2B SDK 创建/暂停/恢复 sandbox，平台团队也可以通过 Sandbox* CRD 做 GitOps、RBAC、namespace、多租户和控制面调试。
- 数据面和控制面解耦：sandbox-manager 负责创建、clone、pause、resume 和 route sync；Envoy ext_proc / Go filter 只负责把请求映射到已知 sandbox route。高频请求不直接打 Kubernetes API。
- runtime 扩展是可插拔而不是硬编码：runtimes 字段和 agent-runtime、csi、traffic-proxy config 让 sidecar、动态存储、流量代理成为可声明能力；identity provider 接口也允许社区默认 token 与企业级 provider 分开演进。
- 身份安全还在工程化演进中：最近提交把 secret-backed key storage 的 ticker 刷新改成 informer-driven refresh，说明 E2B API key / token 生命周期已经成为真实控制面问题。

与同类项目的区别

项目	主要抽象	与 OpenKruise Agents 的区别
agent-sandbox	Sandbox CRD 原语	更偏 SIG 级基础 API；OpenKruise Agents 把 warm pool、E2B API、route proxy、identity/CSI/runtime 扩展也产品化进同一发行版。
agentcube	AgentRuntime / CodeInterpreter session orchestration	AgentCube 用 Router + WorkloadManager 把 agent-sandbox 包成 invocation API；OpenKruise Agents 直接提供 E2B-compatible API 和更完整的 sandbox lifecycle/runtime 扩展。
substrate	WorkerPool / ActorTemplate 高密度 actor substrate	Substrate 更强调 actor multiplexing、wake routing、snapshot；OpenKruise Agents 更贴近 K8s CRD + E2B sandbox 管理。
agentscope-runtime	AgentApp / Runner / deployer	AgentScope Runtime 是 Python app 服务化壳；OpenKruise Agents 是 Kubernetes sandbox lifecycle substrate，可作为 workspace/E2B 后端。
OpenShell	Gateway + supervisor + policy proxy	

项目	主要抽象	与 OpenKruise Agents 的区别
		OpenShell 的强项是 sandbox 内 runtime enforcement 和 policy; OpenKruise Agents 的强项是 K8s 生命周期、热池、E2B API 和路由控制面。

采用建议

- 需要“在 Kubernetes 上提供 E2B-like sandbox 服务”时，优先看 [openkruise-agents](#)。
- 只想学习最小 CRD 原语时，先看 agent-sandbox；OpenKruise Agents 需要同时理解 controller、manager、gateway、runtime sidecar 和 identity/CSI。
- 已经有 Agent framework、想接远端 workspace/sandbox backend 时，重点验证 E2B API 兼容范围、template 管理、timeout、文件/命令 API 和路由行为。
- 高风险不可信代码执行仍要额外验证 runtimeClass、NetworkPolicy、seccomp/AppArmor、egress gateway、token revoke 和 audit；OpenKruise Agents 提供扩展点，但不是自动替代强 runtime enforcement。

风险与待验证点

- API 仍处在快速演进期，README/roadmap 显示 pool、storage、network、runtime、scheduling、observability、SDK 都有大量后续事项。
- SandboxClaim 并发领取、create-on-no-stock、speculate、lock 和 expectations 是正确性关键，需要用压力测试验证。
- 路由表同步依赖 manager/gateway peer 状态，生产要验证实例重启、网络分区和 route stale 时的 fail-close 行为。
- checkpoint/hibernation 的 memory/filesystem/GPU memory 能力要按具体 runtime 和存储后端验证，不能只看字段设计。

OpenKruise Kruise

OpenKruise Kruise 是 OpenKruise 主仓，定位是 Kubernetes workload enhancement。根据 [src-openkruise-projects-current-state](#)，它是 OpenKruise 生态 P0 项目，用 CloneSet、Advanced StatefulSet、Advanced DaemonSet、SidecarSet、UnitedDeployment、WorkloadSpread、ImagePullJob 等控制器补足原生 Deployment/StatefulSet 在大规模应用管理上的细粒度自动化能力。

架构边界

它不是新的调度器，也不是 GitOps 工具。它位于 Kubernetes workload controller 层，用 CRD/controller 扩展 workload 生命周期、分批发布、sidecar 注入、镜像预热和跨域分布。

什么时候看它

场景	判断
想理解 OpenKruise 生态主线	先看 openkruise-kruise ，再看 openkruise-rollouts / kruise-game / openkruise-agents 。
原生 Deployment / StatefulSet 控制粒度不够	适合对比 CloneSet、Advanced StatefulSet、WorkloadSpread。
想研究 AI/LLM serving workload 预热和滚动	可作为 model-serving-operator 的 workload automation 对照。
只需要 Agent sandbox 生命周期	看 openkruise-agents 。

选型提示

把 Kruise 放在 [kubernetes-workload-automation](#) 里理解：它扩展的是 workload 控制器语义，而不是替代 kube-scheduler、Kueue、Karpenter 或 Gateway API。

OpenKruise Rollouts

OpenKruise Rollouts 是 OpenKruise 的 enhanced rollout 控制面，面向渐进式发布、分批发布、流量治理和应用自动化。根据 [src-openkruise-projects-current-state](#)，它是 OpenKruise P0 候选，适合和 Argo Rollouts、原生 Deployment rollout、Gateway/Ingress 流量切分一起对比；在概念层归入 [kubernetes-workload-automation](#) 的 release governance 子问题。

架构边界

它关注“新版本怎样逐步上线并受控回滚”，不是完整 GitOps 系统。GitOps 工具负责期望状态来源；Rollouts 控制实际发布节奏、批次、暂停、推进和流量策略。

什么时候看它

场景	判断
需要 Kubernetes 渐进式发布	适合，放入 kubernetes-workload-automation 的 release governance 路线评估。
想和 Gateway / Ingress 流量切分结合	适合，需要继续核验当前支持矩阵。
想管理 workload 类型本身	先看 openkruise-kruise 。
想管理 Agent sandbox	看 openkruise-agents 。

选型提示

评估 Rollouts 时不要只看 rollout 策略名，还要看它如何和 workload controller、traffic provider、metrics analysis、GitOps reconcile 和回滚状态机组合。

OpenKruise 组织项目当前状态核验

原文: raw/openkruise-projects-current-state.md · 核验日期: 2026-06-15 · 来源: GitHub API openkruise organization repositories + gh repo view 复核重点仓库。

结论

当前 wiki 已完成 [openkruise-agents](#)。OpenKruise 生态还值得补的主线不是“把所有仓库都摄入”，而是围绕 Kubernetes workload 自动化形成几条清晰路线：

- workload enhancement: [openkruise-kruise](#)
- progressive delivery: [openkruise-rollouts](#)
- specialized workload: [kruise-game](#)
- observability / operations: [kruise-state-metrics](#)、[kruise-dashboard](#)
- controller/operator isolation: [controllermesh](#)
- deployment/API support: openkruise/charts、openkruise/*-api

重点仓库元数据

仓库	stars	forks	最近 push	release	语言	定位
openkruise/kruise	5267	890	2026-06-08	v1.8.3 / 2026-02-25	Go	Automated management of large-scale applications on Kubernetes, CNCF incubating。
openkruise/rollouts	254	106	2026-06-13	v0.6.2 / 2025-12-22	Go	Enhanced Rollouts features for application automation。
openkruise/kruise-game	1059	169	2026-05-22	v1.0.0 / 2025-07-21	Go	Game Servers Management on Kubernetes。
openkruise/agents	202	78	2026-06-12	v0.3.0 / 2026-05-15	Go	Agent sandbox lifecycle management, 已摄入 wiki。
openkruise/kruise-tools	55	30	2025-07-22	v1.2.2 / 2025-06-23	Go	Kruise libraries/tools。
	64	10	2023-05-30	none	Go	controller/operator isolation and management。

仓库	stars	forks	最近 push	release	语言	定位
openkruise/ controllermesh						
openkruise/ kruise-state-metrics	12	14	2025-04-28	none	Go	OpenKruise CRD metrics addon。
openkruise/ charts	16	49	2026-06-08	agents-sandbox-manager-0.3.0 / 2026-05-22	Mustache	OpenKruise Helm charts。
openkruise/ kruise-dashboard	0	3	2025-11-19	none	TypeScript	Dashboard for CloneSet / Advanced StatefulSet / Advanced DaemonSet。
openkruise/ openkruise.io	21	98	2026-06-05	none	JavaScript	OpenKruise documentation website。

支撑 / 暂缓仓库

仓库	判断
openkruise/kruise-api	API definition repo，适合在 openkruise/kruise 摄入时引用，不建议单独建主实体。
openkruise/kruise-rollout-api	Rollouts API definition repo，作为 openkruise/rollouts 辅助材料。
openkruise/agents-api	Agents API definition repo，作为 openkruise-agents 后续补充。
openkruise/client-java / openkruise/client-rust	SDK，除非做 OpenKruise client 多语言专题，否则不单独摄入。
openkruise/samples	示例材料，随主仓引用。
openkruise/community / summer_of_code	治理材料，不做架构页。
openkruise/website	已归档，迁移到 openkruise.io。

仓库	判断
openkruise/controller-tools	fork，不纳入候选。
openkruise/federation / controllermesh-api / game.openkruise.io / kruise-helm / kruise- argo / utils	当前更像旧实验、API/文档/插件材料，暂缓。

推荐优先级

P0: kruise, rollouts, kruise-game

P1: kruise-state-metrics, kruise-tools, kruise-dashboard, controllermesh

P2: charts, API definition repos, openkruise.io docs as supporting sources

[openkruise-agents](#) 已完成正式源码架构页，后续只需要在 OpenKruise 生态图中作为已收录项目引用。

prometheus–adapter

Prometheus 到 Kubernetes custom/external metrics API 的适配层，让 HPA 能基于 QPS、队列长度、业务指标或推理指标扩缩。详见 [src-prometheus-adapter-architecture](#)。

架构边界

metrics-server 给 CPU/memory resource metrics；prometheus-adapter 给自定义/外部指标，是高级 autoscaling 的关键桥。

什么时候用

场景	判断
需要 custom/external metrics 能力	适合，prometheus-adapter 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合；prometheus-adapter 只提供 custom/external metrics plumbing。当前 llm-d 新部署使用 EPP metrics → KEDA Prometheus scaler → HPA ； llm-d-workload-variant-autoscaler 是 deprecated 的历史设计，仅保留供迁移/研究参考。

核心组件

- Discovery: 根据 rules 发现 Prometheus series
- Mapper: series -> Kubernetes resource/custom/external metric
- APIService: custom.metrics.k8s.io / external.metrics.k8s.io
- Query renderer: 把 HPA 请求转成 PromQL

选型提示

把 prometheus-adapter 放在 custom/external metrics 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

prometheus-adapter 架构与设计思路分析

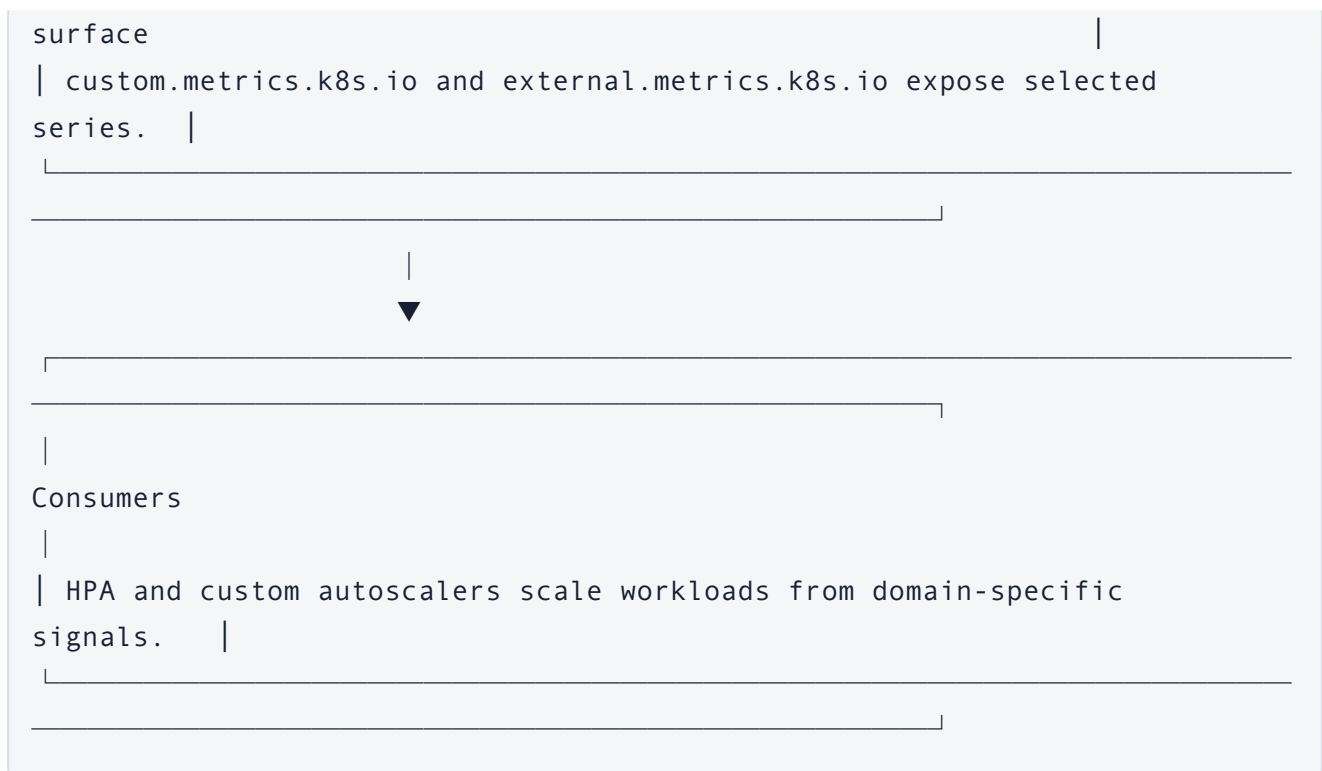
原文: raw/prometheus-adapter-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/prometheus-adapter> · 优先级 P0

一句话定位

Prometheus 到 Kubernetes custom/external metrics API 的适配层，让 HPA 能基于 QPS、队列长度、业务指标或推理指标扩缩。

核心架构图

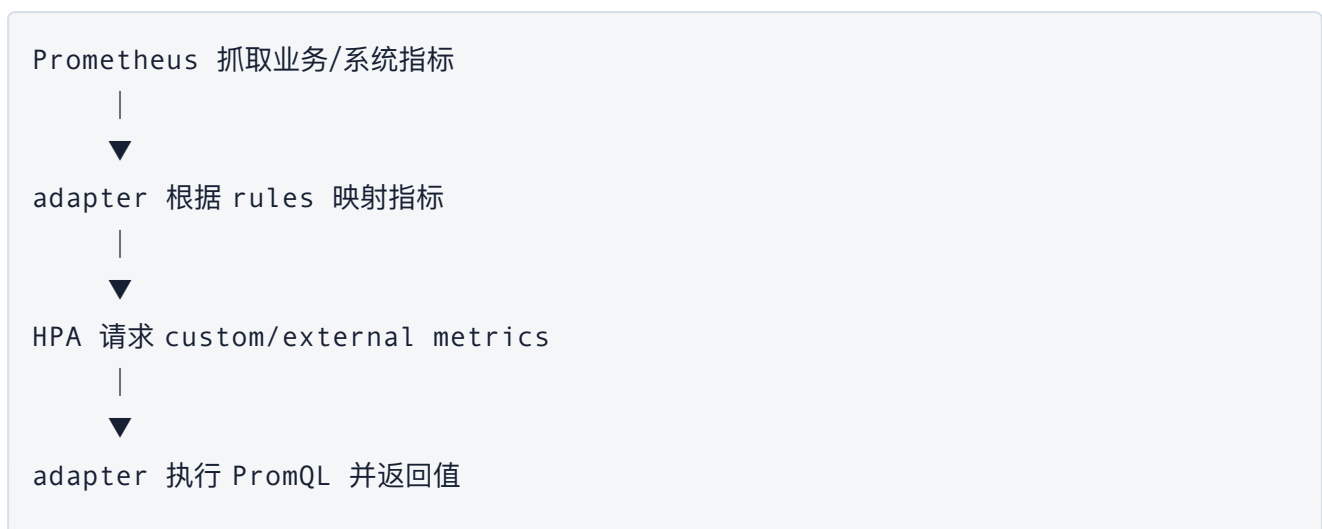




模块分层

层 / 模块	职责
Discovery	根据 rules 发现 Prometheus series
Mapper	series -> Kubernetes resource/custom/external metric
APIService	custom.metrics.k8s.io / external.metrics.k8s.io
Query renderer	把 HPA 请求转成 PromQL

关键数据流



设计决策与哲学

- 补齐 custom/external metrics 维度: prometheus-adapter 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: metrics-server 给 CPU/memory resource metrics; prometheus-adapter 给自定义/外部指标, 是高级 autoscaling 的关键桥。
- 选型价值: 它应和 llm-inference, [model-serving-operator](#), [llm-d-workload-variant-autoscaler](#) 一起看, 而不是孤立评估。

相关页面

- [prometheus-adapter](#)
- [kubernetes](#)
- llm-inference
- [model-serving-operator](#)
- [llm-d-workload-variant-autoscaler](#)

RBG (RoleBasedGroup)

RBG 是 sgl-project 的 Kubernetes workload API，用于编排多角色、有状态、分布式 AI inference workload。它把 gateway → router → prefill → decode 这类服务表达成 RoleBasedGroup，而不是把多个 Deployment/StatefulSet 当成互不相关的对象。

核心抽象

- **Role**: 每个角色拥有自己的 spec、生命周期和策略，例如 prefill/decode。
- **RoleBasedGroup**: 组成一个逻辑推理服务的多角色整体。
- **RoleInstance**: 与一组 Pod 绑定的状态与升级单元。
- **CoordinatedPolicy**: 协调跨角色升级、扩缩、maxSkew 和 progression。

架构边界

RBG 解决的是 workload topology、状态、原子操作、gang scheduling、硬件/网络亲和和跨角色协调；它不是完整 API gateway，也不是推理 engine。它适合作为 [dynamo](#)、[llm-d](#) 或自研 router/engine 上方的 Kubernetes workload substrate。

适用与限制

适合 P/D、multi-role、需要成组部署/升级/扩缩和稳定实例语义的场景。若只需要一个简单模型 Deployment + Service，RBG 的协调模型可能过重；若需要完整模型路由、限流、canary 和 API 入口，还要组合专用 serving/router 组件。

scheduler-plugins

scheduler-plugins 是基于 kube-scheduler framework 的 out-of-tree 插件集合，用于研究和生产化调度扩展。详见 [src-scheduler-plugins-architecture](#)。

证据说明：2026-09-25 核验的官方默认分支快照为 [6df8d8e4ae5f](#)，该 commit 不代表 release。[src-scheduler-plugins-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目职责见 [kubernetes-workload-gang-scheduling-design](#)，调度器流程见 [kubernetes-scheduler-core-design](#)。

在 M5-B Workload / Scheduling 地图中的位置

scheduler-plugins 通过 out-of-tree framework extensions 参与队列、过滤、评分、保留或放行等阶段，部署时需匹配 Kubernetes 依赖与插件配置。它不接管 [jobset](#)、[lws](#) 的业务生命周期，也不承担 [karpenter](#) 的节点创建；[kueue](#) 的配额准入与 scheduler 的 placement 仍是不同职责。上述快照中的 `scheduling.x-k8s.io/v1alpha1 PodGroup / ElasticQuota` 也须与原生 Kubernetes Workload/PodGroup API 区分。

架构边界

Kueue 做 workload admission；scheduler-plugins 影响 Pod 到 Node 的 placement。

什么时候用

场景	判断
需要 调度 / 资源 能力	适合，scheduler-plugins 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes , llm-inference 组合。

核心组件

- Framework plugins: queueSort/preFilter/filter/score/reserve 等扩展点
- Scheduler binary/config
- Controllers/examples for capacity/placement policies
- Integration tests and manifests

选型提示

把 scheduler-plugins 放在 调度 / 资源 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

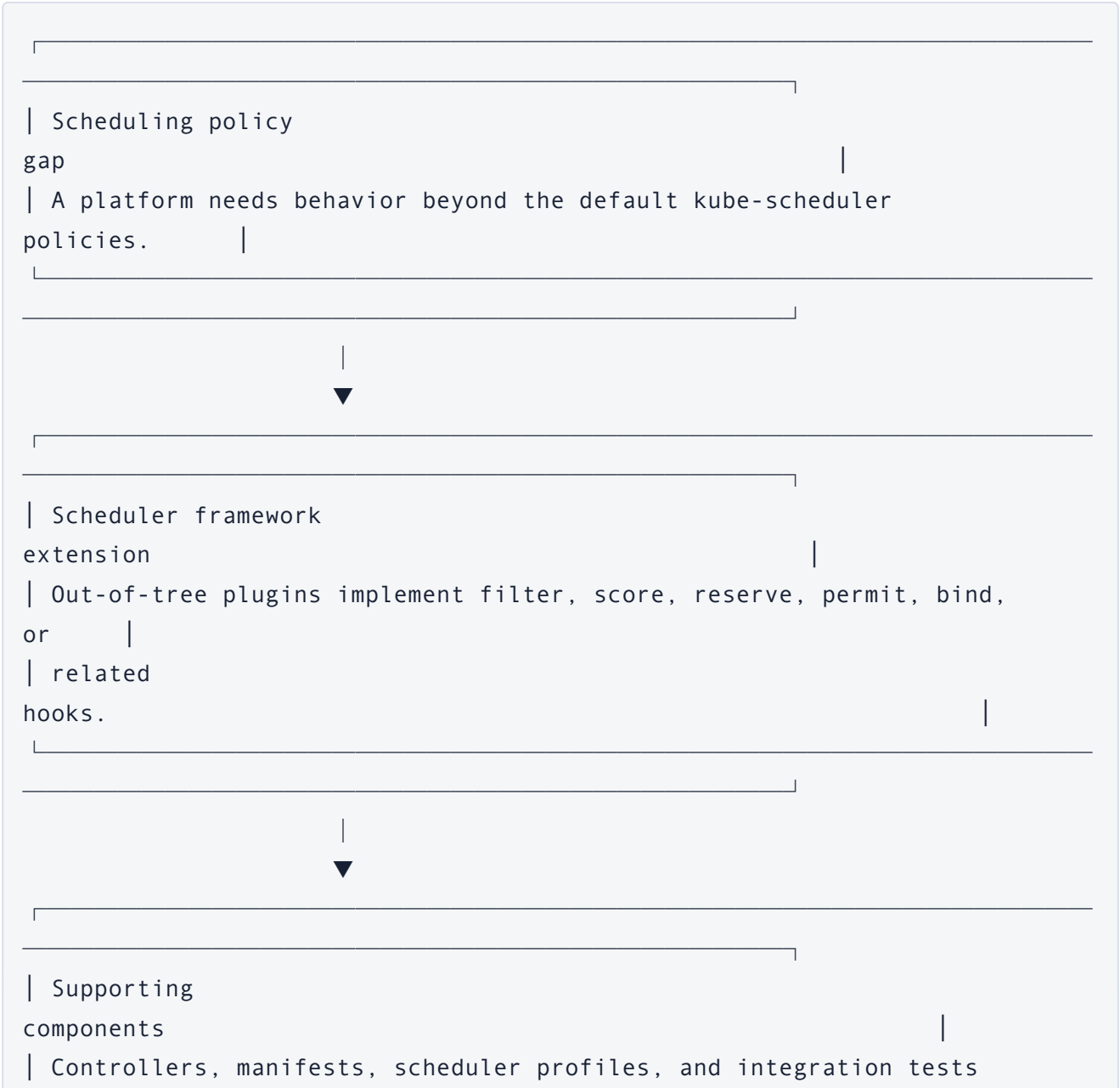
scheduler-plugins 架构与设计思路分析

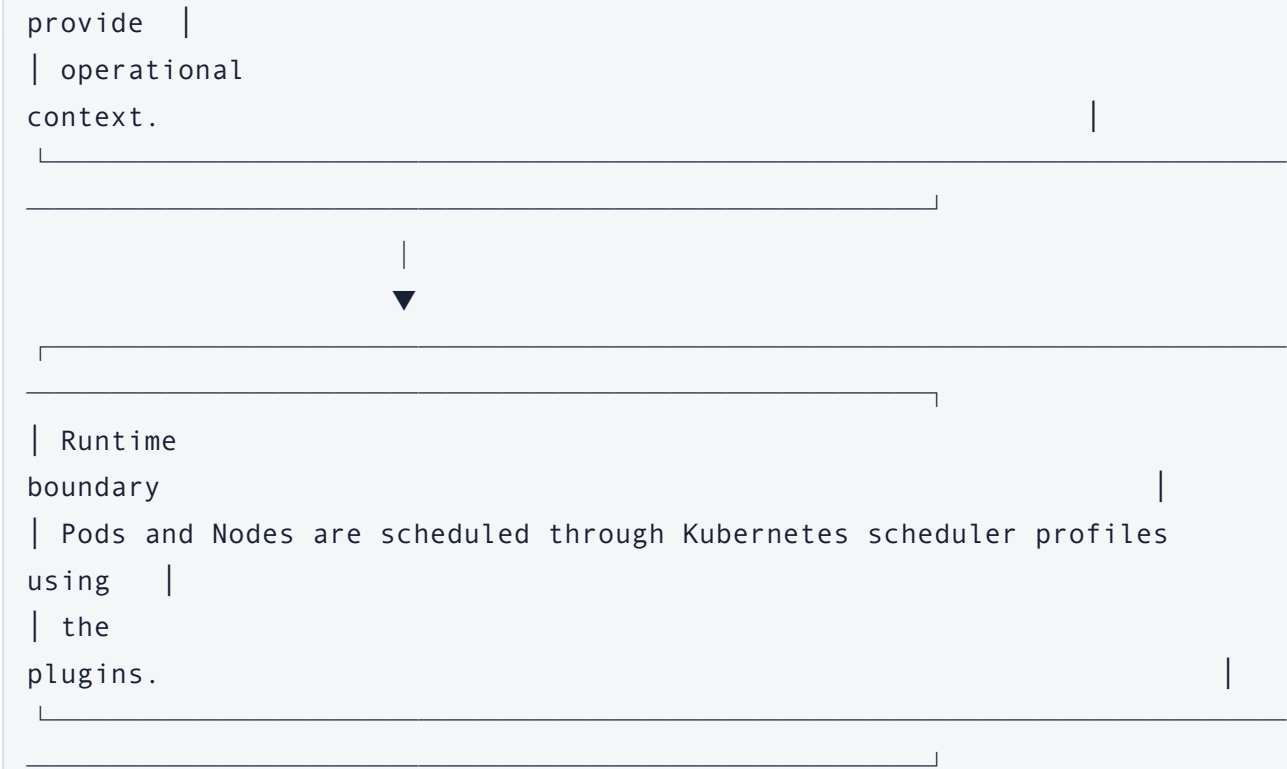
原文: raw/scheduler-plugins-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/scheduler-plugins> · 优先级 P0

一句话定位

scheduler-plugins 是基于 kube-scheduler framework 的 out-of-tree 插件集合，用于研究和生产化调度扩展。

核心架构图

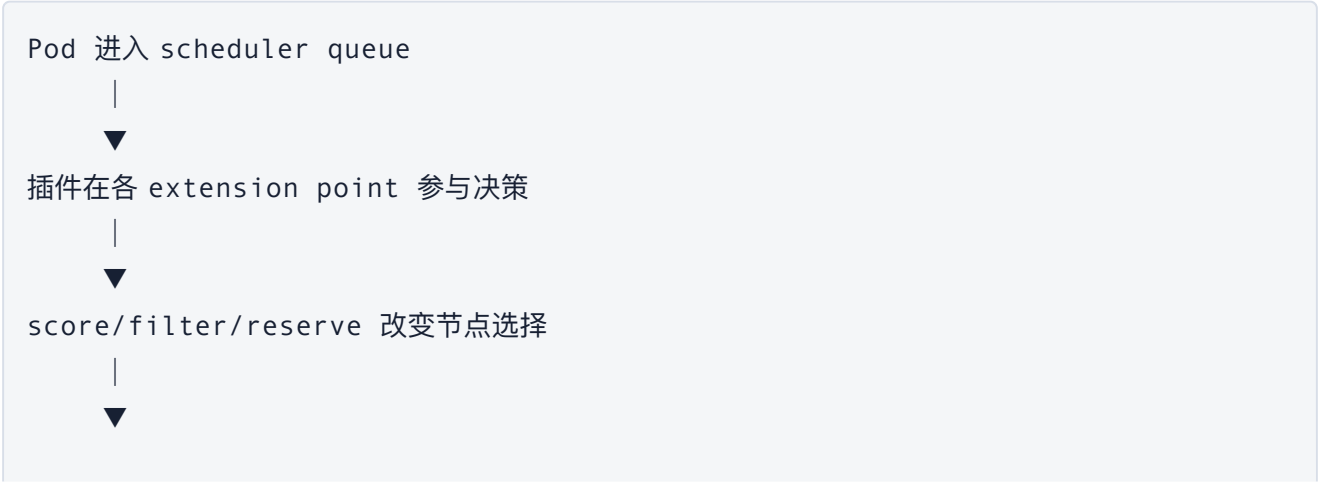




模块分层

层 / 模块	职责
Framework plugins	queueSort/preFilter/filter/score/reserve 等扩展点
Scheduler binary/config	Scheduler binary/config
Controllors/examples for capacity/placement policies	Controllors/examples for capacity/placement policies
Integration tests and manifests	Integration tests and manifests

关键数据流



scheduler bind Pod

|



控制器/指标反馈策略效果

设计决策与哲学

- 补齐 调度 / 资源 维度：scheduler-plugins 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Kueue 做 workload admission；scheduler-plugins 影响 Pod 到 Node 的 placement。
- 选型价值：它应和 [kubernetes](#), llm-inference 一起看，而不是孤立评估。

相关页面

- [scheduler-plugins](#)
- [kubernetes](#)
- llm-inference

Secrets Store CSI Driver

Secrets Store CSI Driver 通过 CSI volume 把外部 secret store 注入 Pod，并支持 provider、rotation 和可选 Kubernetes Secret 同步。详见 [src-secrets-store-csi-driver-architecture](#)。

架构边界

Kubernetes Secret 是集群内对象；Secrets Store CSI Driver 把真凭据留在外部 secret manager，运行时挂载。

什么时候用

场景	判断
需要 存储 / 凭据 能力	适合，Secrets Store CSI Driver 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 cloud-native-security , agent-credential-isolation 组合。

核心组件

- CSI node plugin: mount volume into pod
- Provider gRPC: Vault/Azure/GCP/AWS 等外部 secret
- SecretProviderClass API
- Rotation/sync controller

选型提示

把 Secrets Store CSI Driver 放在 存储 / 凭据 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

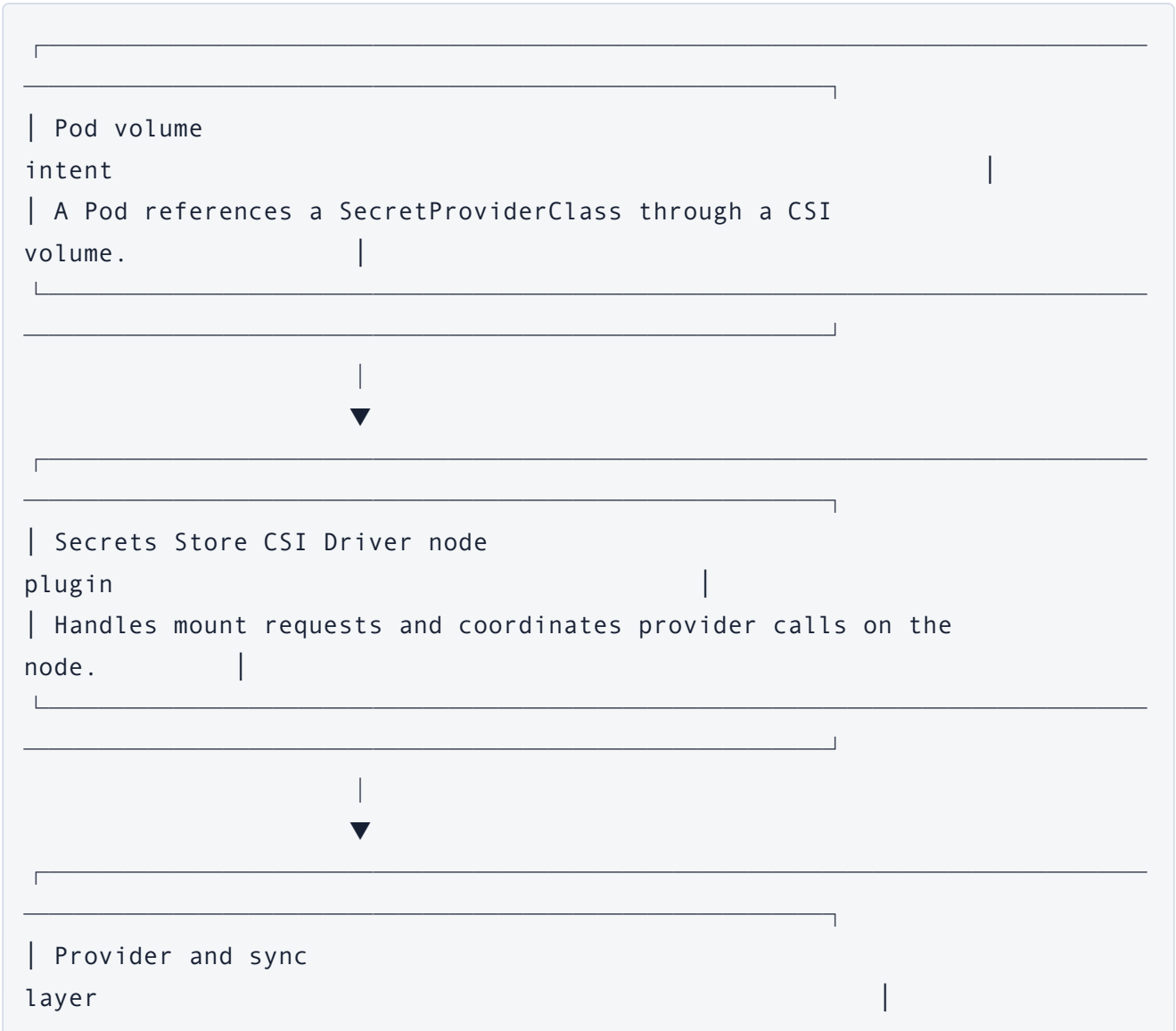
Secrets Store CSI Driver 架构与设计思路分析

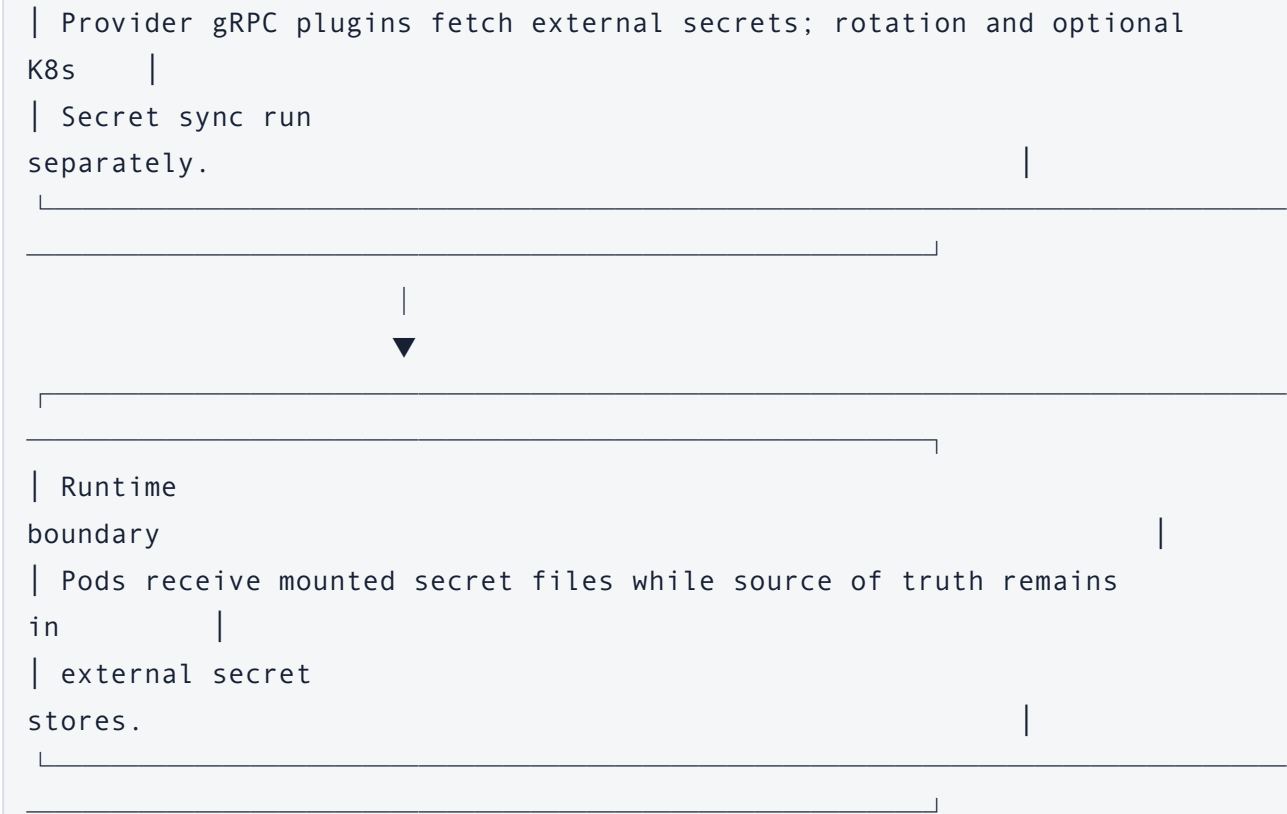
原文: raw/secrets-store-csi-driver-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/secrets-store-csi-driver> · 优先级 P0

一句话定位

Secrets Store CSI Driver 通过 CSI volume 把外部 secret store 注入 Pod，并支持 provider、rotation 和可选 Kubernetes Secret 同步。

核心架构图

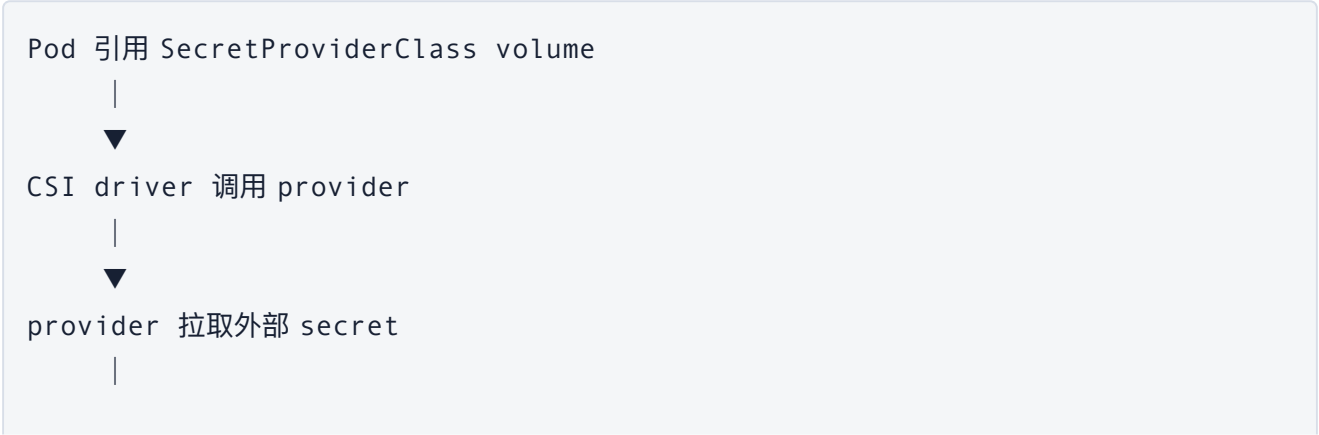




模块分层

层 / 模块	职责
CSI node plugin	mount volume into pod
Provider gRPC	Vault/Azure/GCP/AWS 等外部 secret
SecretProviderClass API	SecretProviderClass API
Rotation/sync controller	Rotation/sync controller

关键数据流





设计决策与哲学

- 补齐 存储 / 凭据 维度：Secrets Store CSI Driver 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：Kubernetes Secret 是集群内对象；Secrets Store CSI Driver 把真凭据留在外部 secret manager，运行时挂载。
- 选型价值：它应和 [cloud-native-security](#), agent-credential-isolation 一起看，而不是孤立评估。

相关页面

- [secrets-store-csi-driver](#)
- [kubernetes](#)
- [cloud-native-security](#)
- agent-credential-isolation

Security Profiles Operator

Security Profiles Operator 管理 seccomp/AppArmor/SELinux profiles，并可通过 recording 把运行时行为转成可部署 profile。详见 [src-security-profiles-operator-architecture](#)。

架构边界

NetworkPolicy 管网络；SPO 管 syscall/LSM runtime confinement，适合高风险 workload 和 Agent sandbox 边界。

什么时候用

场景	判断
需要 Runtime security 能力	适合，Security Profiles Operator 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 cloud-native-security , agent-sandbox 组合。

核心组件

- CRDs: SeccompProfile, SelinuxProfile, ProfileRecording
- Daemon/controller: install profiles on nodes
- Recorder: capture syscalls/behavior
- Admission/profile binding integrations

选型提示

把 Security Profiles Operator 放在 Runtime security 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

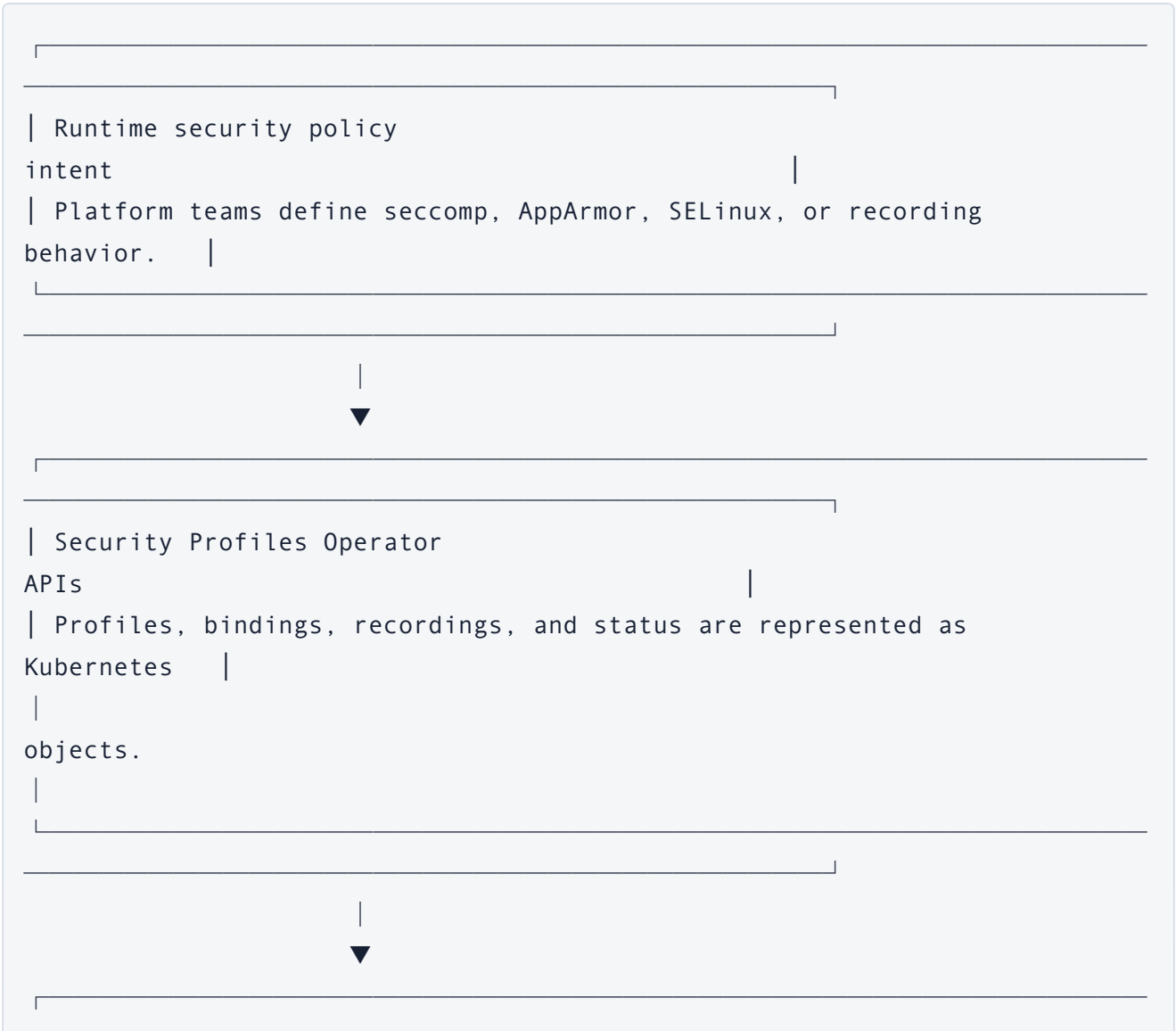
Security Profiles Operator 架构与设计思路分析

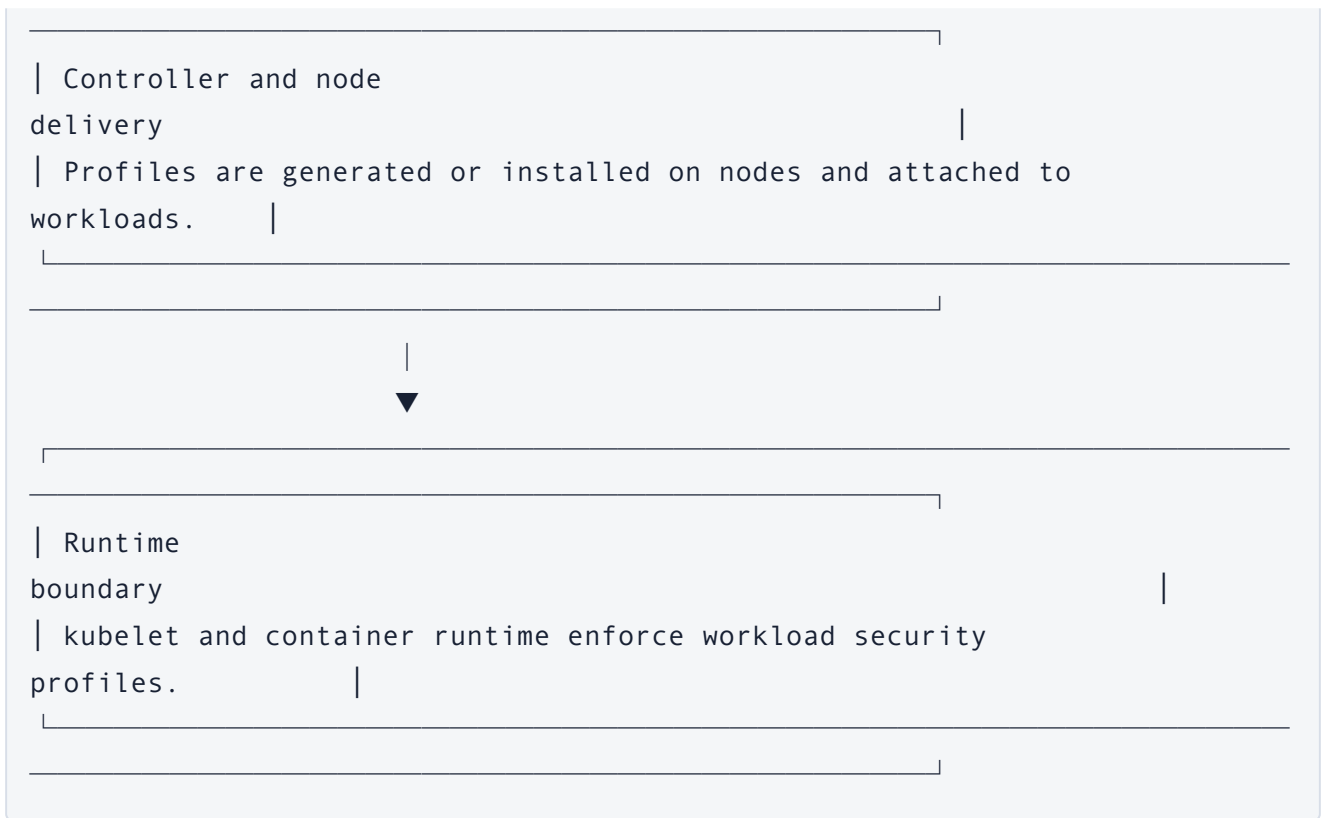
原文: raw/security-profiles-operator-architecture-analysis.md · 仓库:
<https://github.com/kubernetes-sigs/security-profiles-operator> · 优先级 P1

一句话定位

Security Profiles Operator 管理 seccomp/AppArmor/SELinux profiles，并可通过 recording 把运行时行为转成可部署 profile。

核心架构图





模块分层

层 / 模块	职责
CRDs	SeccompProfile, SelinuxProfile, ProfileRecording
Daemon/controller	install profiles on nodes
Recorder	capture syscalls/behavior
Admission/profile binding integrations	Admission/profile binding integrations

关键数据流



内核/runtime enforcement

|



状态和失败原因回写

设计决策与哲学

- 补齐 Runtime security 维度：Security Profiles Operator 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：NetworkPolicy 管网络；SPO 管 syscall/LSM runtime confinement，适合高风险 workload 和 Agent sandbox 边界。
- 选型价值：它应和 [cloud-native-security](#), agent-sandbox 一起看，而不是孤立评估。

相关页面

- [security-profiles-operator](#)
- [kubernetes](#)
- [cloud-native-security](#)
- agent-sandbox

sig-storage-lib-external-provisioner

sig-storage-lib-external-provisioner 是 Kubernetes dynamic volume provisioner 的库，抽象 PVC watch、PV 创建、reclaim 和 controller lifecycle。详见 [src-sig-storage-lib-external-provisioner-architecture](#)。

架构边界

具体 provisioner 如 NFS provisioner 处理后端细节；这个库处理 Kubernetes 控制器通用模式。

什么时候用

场景	判断
需要 external provisioner library 能力	适合，sig-storage-lib-external-provisioner 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 kubernetes 组合。

核心组件

- ProvisionController: PVC/PV watch and sync
- Provisioner interface
- Leader election/events
- Reclaim/delete handling

选型提示

把 sig-storage-lib-external-provisioner 放在 external provisioner library 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

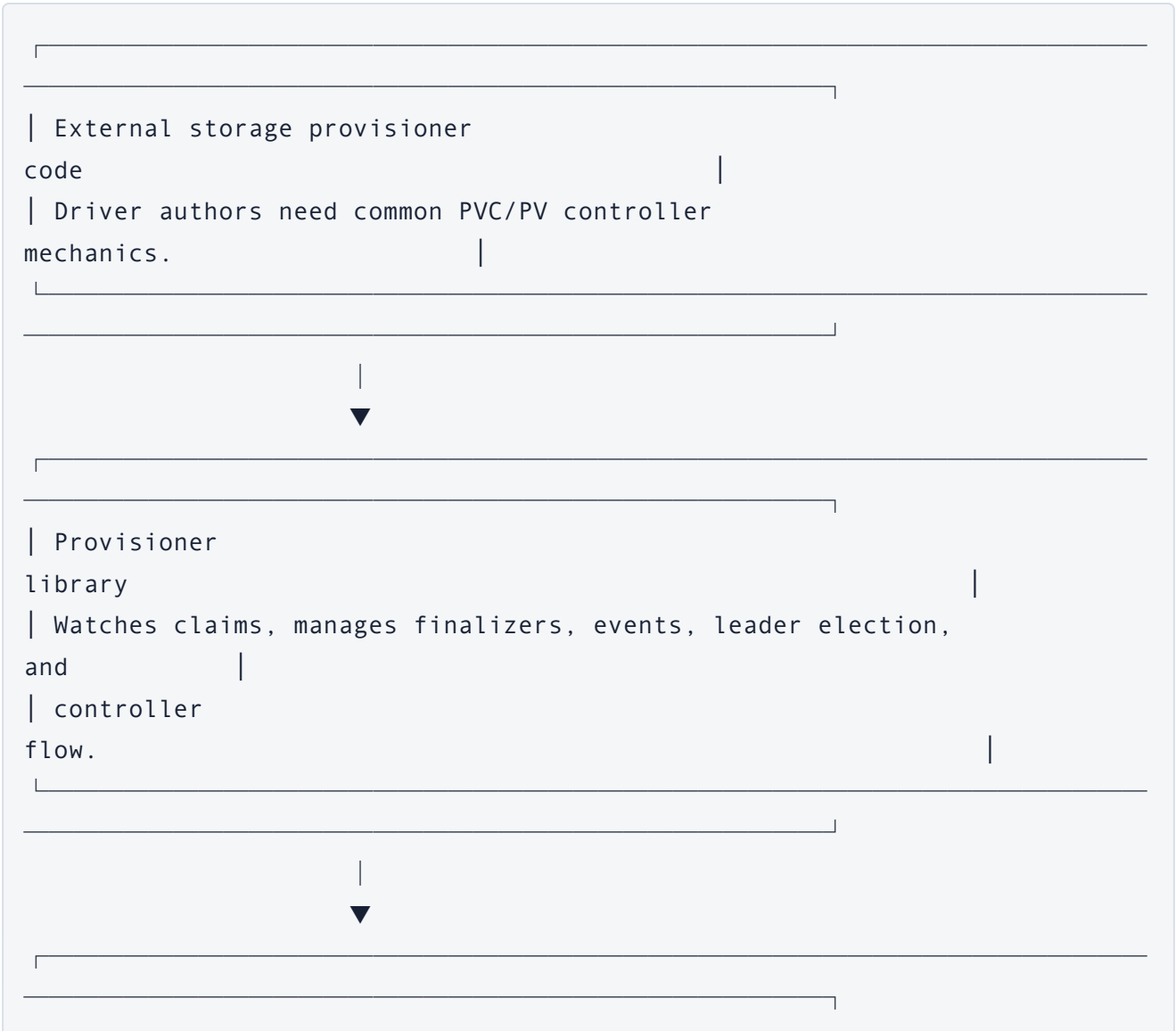
sig-storage-lib-external-provisioner 架构与设计思路分析

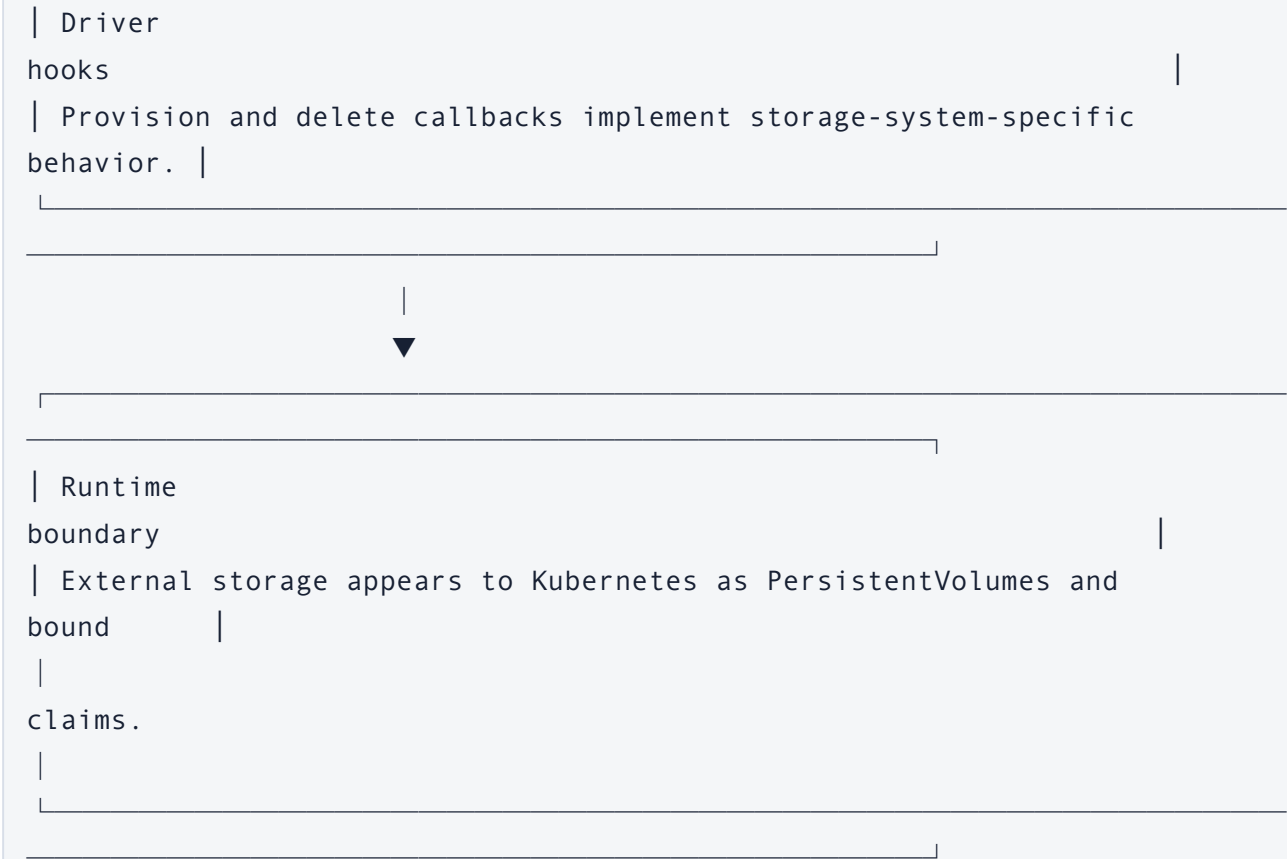
原文: raw/sig-storage-lib-external-provisioner-architecture-analysis.md
· 仓库: <https://github.com/kubernetes-sigs/sig-storage-lib-external-provisioner> · 优先级 P1

一句话定位

sig-storage-lib-external-provisioner 是 Kubernetes dynamic volume provisioner 的库，抽象 PVC watch、PV 创建、reclaim 和 controller lifecycle。

核心架构图

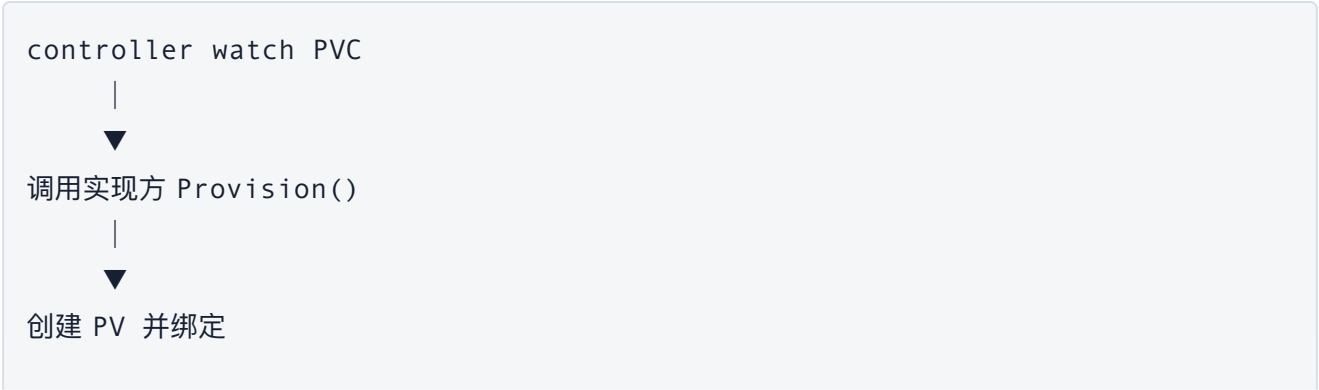




模块分层

层 / 模块	职责
ProvisionController	PVC/PV watch and sync
Provisioner interface	Provisioner interface
Leader election/events	Leader election/events
Reclaim/delete handling	Reclaim/delete handling

关键数据流





设计决策与哲学

- 补齐 external provisioner library 维度: sig-storage-lib-external-provisioner 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：具体 provisioner 如 NFS provisioner 处理后端细节；这个库处理 Kubernetes 控制器通用模式。
- 选型价值：它应和 [kubernetes](#) 一起看，而不是孤立评估。

相关页面

- [sig-storage-lib-external-provisioner](#)
- [kubernetes](#)

SkyPilot

SkyPilot 是面向 AI/ML 工作负载的多云算力控制平面。用户通过 YAML / Python API 声明 Task、Dag、Resources，SkyPilot 负责选择可用且便宜的 GPU/CPU/TPU 资源，完成 provisioning、文件同步、setup、run、日志流和自动回收。详见 [src-skypilot-architecture](#)。

架构边界

SkyPilot 不优化 LLM 推理内核，也不直接管理 KV cache。它位于 llm-inference 更上层：决定 workload 应该跑在哪个云、哪个区域、哪种实例、哪个集群，以及失败时如何 failover。

关键设计

- Client/SDK 把 YAML、CLI override、env/secrets、workdir 转成 Task/Dag。
- API server 负责鉴权、RBAC、request queue、blob storage 和日志流。
- Optimizer 按资源、价格、容量和失败 blocklist 选择 cloud/region/instance。
- CloudVmRayBackend 把资源选择落到 Ray cluster、Kubernetes、Slurm 或云 VM。
- Managed jobs / SkyServe / pools 在 launch/run 之上提供作业恢复和服务控制器。

选型判断

需要跨云 GPU 资源经济、failover 和 AI job/serve 控制时看 SkyPilot。需要单机推理引擎看 vllm / sglang；需要多节点 LLM serving 编排看 [dynamo](#) / [llm-d](#)；需要 Kubernetes GPU 设备层看 [k8s-gpu-device-stack](#)。

SkyPilot 架构与设计思路分析

原文: raw/skypilot-architecture-analysis.md · 仓库: [googs1025/skypilot](#) · 分析版
本 master HEAD 55b9185 (2026-06-01)

一句话定位

SkyPilot 是一个面向 AI/ML 工作负载的多云算力控制平面：用户用 YAML / Python API 声明 Task、Dag、Resources，SkyPilot 负责选择可用且便宜的 GPU/CPU/TPU 基础设施，完成 provisioning、文件同步、setup、run、日志流和自动回收。它的关键手段是把“用户声明”和“云厂商细节”之间拆成 API server request 队列、optimizer、CloudVmRayBackend、cloud/provision provider 接口、controller (managed jobs / serve / pools) 几层。

核心架构图



```

confirmation |
|   └─ upload workdir/file_mounts
blob |
|   └─ POST /launch, /jobs/*, /serve/* + stream request
logs |
└───┘
└───┘
      | authenticated REST request + request_id
      ▼
└───┘
└───┘
| API
server |
|   FastAPI
server.py |
|   └─ auth: loopback / oauth2-proxy / service-account JWT / basic
auth |
|   └─ RBAC + workspace
permission |
|   └─ request DB + blob storage + log
streaming |
|   └─ executor: LONG queue vs SHORT
queue |
└───┘
└───┘
      | scheduled long-running request
      ▼
└───┘
└───┘
| Execution
orchestration |
|   sky/
execution.py |
|   └─ validate + admin policy + secrets/volumes
resolve |
|   └─ OPTIMIZE: Optimizer.optimize /
optimize_job_group |
|   └─ PROVISION:
CloudVmRayBackend.provision |
|   └─ SYNC_WORKDIR / SYNC_FILE_MOUNTS /
SETUP |

```



模块分层

层 / 模块	职责
用户入口	Click CLI、Python SDK、异步 SDK、公开 API facade。把 shell/YAML/Python 调用变成 Task/Dag 和 API server request。
声明模型	用户声明的中间表示：任务、DAG、资源、volume、storage、service/pool。负责 YAML schema、env/secrets、workdir、file mounts、autostop、hooks。
API server / 控制面	FastAPI API server、request 排队/执行/日志、blob 上传、鉴权、RBAC、workspace 权限。
执行编排	统一执行 stage：validate、optimize、provision、sync、setup、pre_exec、exec、down；维护 cluster 状态、事件和 handle。
优化器	把抽象资源要求映射成具体 cloud/region/zone/instance_type，按 cost/time 选择，支持 job group 和 failover blocklist。
后端	以 Ray + skylet 作为远端运行时，创建/复用集群、同步文件、运行 setup/run、提交 job、设置 autostop/hook。
provider 实现	每家云/集群的能力声明、区域/价格/规格查询、实例生命周期 API、 kubernetes /Slurm/SSH 特化实现。
高层服务	在基础 launch/exec/down 之上实现 managed jobs 自动恢复、SkyServe replica/autoscaler、batch/pools、SSH node pools、volume。
插件/策略/观测	admin policy、server plugins、RBAC 扩展、Prometheus metrics、usage telemetry、外部故障源。

关键约束是：Task/Resources 只表达用户需求，不做云 API 调用；Cloud 类描述能力和候选资源，不直接成为长期状态对象；provision/* 做具体生命周期；CloudVmRayBackend 负责把 optimizer 选出的 launchable resources 落到远端 runtime；API server 负责请求排队和鉴权，不把所有重任务塞在 FastAPI event loop 上。

关键数据流

sky launch 主链路



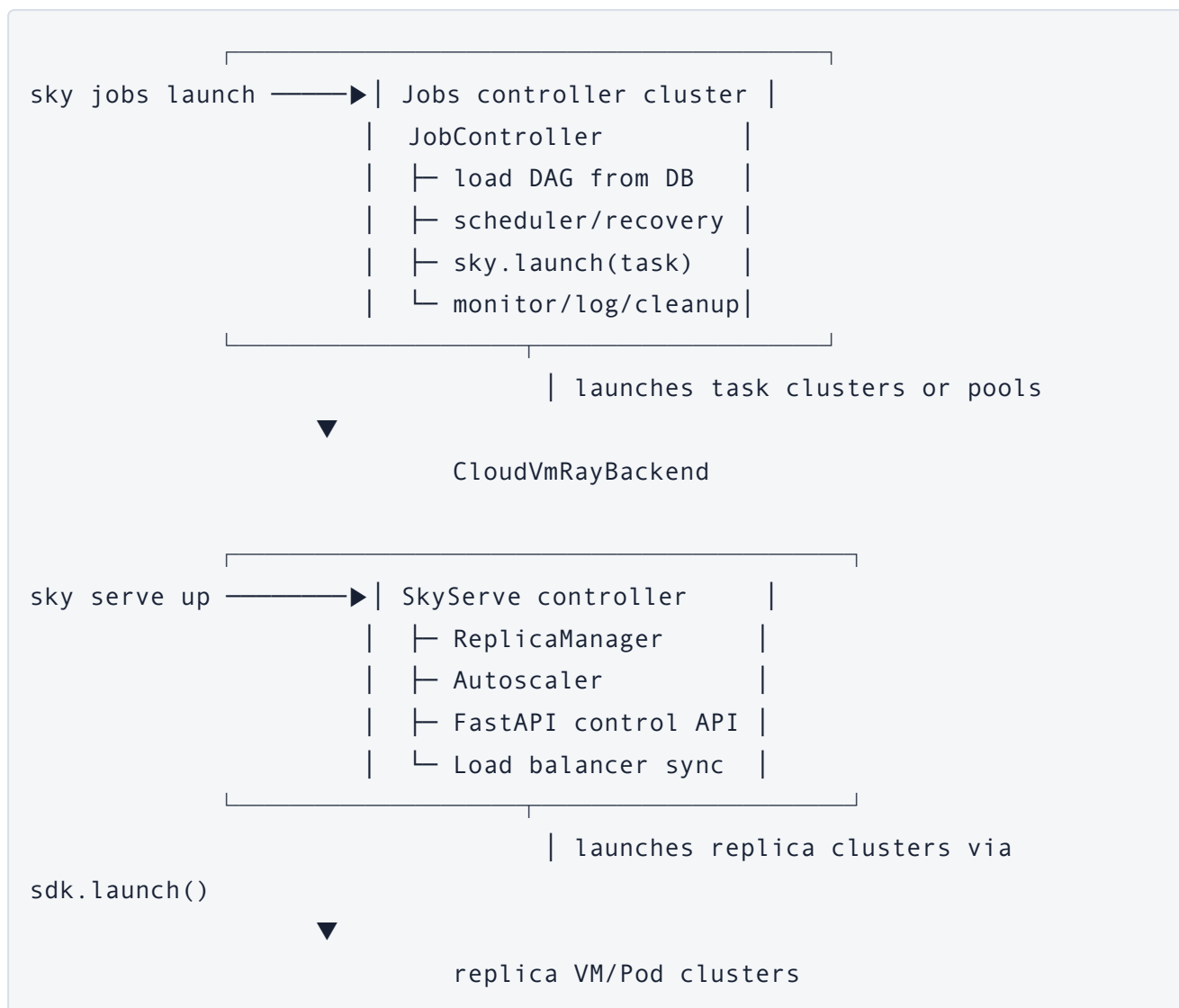
```

|   |─ load_chain_dag_from_yaml / load_job_group_from_yaml
|   └─ apply CLI resource/env/secret/workdir overrides
▼
command.py:launch()
|   └─ sdk.launch(task, cluster_name, backend=CloudVmRayBackend, ...)
▼
sdk.py:launch()
|   |─ convert_entrypoint_to_dag()
|   |─ apply client-side admin policy
|   |─ upload_mounts_to_api_server()
|   └─ POST /launch -> request_id
▼
server.py:/launch
|   └─ executor.schedule_request_async(..., func=execution.launch, LONG)
▼
execution.py:launch() / _execute_dag()
|   |─ validate, resolve volumes/secrets
|   |─ Optimizer.optimize()
|   |─ backend.register_info(dag, requested_features, planner)
|   |─ backend.provision()
|   |─ backend.sync_workdir() / sync_file_mounts()
|   |─ backend.setup()
|   |─ backend.set_autostop()
|   └─ backend.execute()
▼
CloudVmRayBackend
|   |─ distributed cluster lock
|   |─ RetryingVmProvisioner.provision_with_retries()
|   |─ cloud/provision provider APIs
|   └─ Ray job queue + skylet + logs
▼
Remote VM / Pod / Slurm allocation

```

错误和回退路径很关键：如果第一次 optimizer 无可行资源，`ResourcesUnavailableError.failover_history` 为空；如果 cloud feature check、region/zone provisioning 或 capacity 失败，`RetryingVmProvisioner` 会把失败资源加入 blocklist，调用 `Optimizer.optimize(..., blocked_resources=...)` 重新选资源，再继续尝试。

Managed Jobs / Serve 控制器路径



Managed jobs 与 serve 都不是直接改底层后端，而是复用同一套 `sdk.launch()` /

`CloudVmRayBackend`。区别是它们把自己变成“控制器集群”上的长期进程，维护 job/service 状态，并用恢复/扩缩容策略反复发起普通 launch/down。

设计决策与哲学

- API server 是所有重操作的边界：用户侧 `sky.client.sdk.launch()` 上传 mounts 后 POST / launch, server 只调度 request, 真正执行在 request worker 中。这个边界让 CLI、dashboard、远端 API server、async SDK 都围绕 request_id 和日志流工作。
- 长短请求分队列：API server 明确区分 cluster launch / job submission 这类 LONG request 和 status/logs 这类 SHORT request, 避免长 provisioning 阻塞控制面读请求。
- Task/Dag/Resources 是稳定 IR：CLI 会先把 YAML、shell command、JobGroup、CLI overrides 规整成统一声明模型；[kubernetes](#)、Slurm、AWS/GCP/Azure 等 provider 不直接影响用户接口。

- **optimizer** 在上层，**provider** 只回答能力和可行性：执行层先调用 **optimizer**，再把可重新规划的 **planner** 注入 **backend**；**provider** 层通过 **feature gates**、**catalog** 和 **provision hooks** 暴露能力。
- **跨云 failover** 是失败反馈后的重新优化：**capacity/feature/identity** 失败后，**backend** 把失败 **resources** 加入 **blocklist**，再重新调用 **optimizer**，而不是固定候选列表盲试。
- **managed jobs / serve** 是控制器模式：**jobs controller** 和 **SkyServe controller** 都通过 **sdk.launch()** / **sdk.down()** 管理实际集群，自己只维护恢复、扩缩容、状态和日志策略。
- **安全模型** 偏控制面托管 + **RBAC**：状态库记录 **clusters/users/volumes/events**；**API server** 支持 **OAuth2 proxy**、**basic auth**、**service-account JWT**、**Casbin RBAC** 和 **workspace permission**。**viewer** 是 **allowlist**，新 **endpoint** 默认不可见。

关键组件深入解读

sky launch 执行编排

execution._execute() 是 **SkyPilot** 的 **stage runner**：**validate**、**admin policy**、**volume/secrets resolve**、**storage construct** 后，进入 **_execute_dag()**。普通 **launch** 目前只支持单 **task**；**stage** 默认包含 **OPTIMIZE**、**PROVISION**、**SYNC_WORKDIR**、**SYNC_FILE_MOUNTS**、**SETUP**、**PRE_EXEC**、**EXEC**、**DOWN**。它先在锁外运行 **Optimizer.optimize()**，同时把一个 **planner** 回调注入 **backend**，供 **backend** 在拿到同名 **cluster lock** 后按最新状态重新规划，降低并发状态变化带来的竞态。

CloudVmRayBackend 与 failover

CloudVmRayBackend 是核心后端：远端执行语义来自 **Ray cluster + skylet + job queue**。**provision** 前检查 **rsync** 和 **owner identity**，再拿同名 **cluster lock**；锁内决定复用/重建/新建，随后 **build SkyPilot wheel** 并交给 **RetryingVmProvisioner**。如果 **provisioning** 失败且允许 **failover**，它会把当前资源加入 **blocklist**，再调用 **optimizer** 重新选择 **best_resources**，这就是 **SkyPilot** 能在 **capacity error** 后自动换 **region/cloud** 的关键。

与 [dynamo](#) / [vllm](#) / [sglang](#) 的关系

SkyPilot 不是推理引擎，也不直接实现 **llm-inference** 的 **batching**、**KV cache** 或模型执行。它更像 **AI compute control plane**：可以用统一 **YAML** 启动 **vllm**、**sglang**、训练任务、**RAG app**、**Jupyter**、**batch inference** 或 **disaggregated-serving** 相关组件，并在 [kubernetes](#)、**Slurm**、公有云 **GPU provider** 之间做资源选择、**failover** 和生命周期治理。

相关页面

- [kubernetes](#)
- [vllm](#)
- [sglang](#)
- [dynamo](#)
- [llm-inference](#)

- disaggregated-serving

substrate

Agent Substrate 是运行在 [kubernetes](#) 之上的高密度 agent-like workload substrate。它把 Kubernetes 当作低频容量和模板控制面，把 Actor 的高频 resume/suspend、worker 分配、状态锁和路由唤醒放到 Redis/ValKey、ateapi、atelet、ateom-gVisor 和 atenet router 里完成。详见 [src-substrate-architecture](#)。

架构边界

Substrate 不是普通的 Agent framework，也不是只创建一个 Pod 的 sandbox CRD。它的核心目标是让大量 idle actor multiplex 到较少 warm worker 上，通过 golden snapshot、runtime checkpoint/restore、actor DNS 和 Envoy ext_proc 唤醒降低激活延迟。

它和 agent-sandbox / [agentcube](#) 的区别在层级：agent-sandbox 提供有状态 Sandbox 原语，[agentcube](#) 做会话路由和 warm pool，Substrate 更底层，直接把 worker pool、actor template、snapshot 和路由恢复做成 substrate。

关键抽象

- WorkerPool：声明 worker 容量和运行模板，由 controller 物化为 privileged ateam pods。
- ActorTemplate：生成 golden actor，再 checkpoint 成 golden snapshot，供后续 actor 快速恢复。
- Actor：高频运行状态不进 K8s API，而是存 Redis/ValKey。
- atelet / ateam-gvisor：节点侧 supervisor 和 gVisor worker，负责 runsc create/start/checkpoint/restore。
- atenet：通过 actor hostname 触发 ResumeActor，再把请求转发到已分配 worker。

选型判断

适合研究或构建高密度、低空闲成本、可暂停恢复的 agent runtime。若只是给单个 agent 提供隔离，优先看 agent-sandbox 或 OpenShell；若需要团队级会话编排和 warm pool，优先看 [agentcube](#)；若要把 sandbox 出口流量纳入治理，再叠加 [agentgateway](#) / ai-gateway。

云原生安全

云原生环境下的安全实践、威胁模型和工具链。

当前趋势

AI 与漏洞发现

AI 模型同时加速漏洞发现和低质量报告泛滥。核心应对：

- 维护者：公开威胁模型 + 最低报告标准 + AI 辅助分诊
- 发现者：完整 PoC + 修复 PR，禁止批量提交
- 详见 [src-ai-vulnerability-discovery](#)

K8s v1.36 安全强化

- 弃用 externalIPs（中间人攻击风险）
- 移除 gitRepo Volume（root 提权风险）
- 详见 [src-k8s-v1.36-sneak-peek](#)

Service Mesh / Gateway 安全

Service Mesh 和 Gateway 安全的核心是把身份、流量策略和凭据边界从应用代码里抽出来：

- mTLS 让服务间调用绑定 workload identity，而不是只依赖网络位置；
- L7 policy 能按 host/path/header/JWT claims 做授权；
- [gateway-api](#) 提供更明确的入口角色分层，减少 Ingress annotation 漂移；
- [agentgateway](#) 这类 AI Gateway 把 LLM/MCP/A2A 出口流量纳入统一鉴权、限流、审计和 agent-credential-isolation。

AI Agent 场景下，mesh/gateway 的价值更高：Agent prompt 和工具调用不可完全信任，凭据、出口路由和 tool permission 应由网关策略兜住。

供应链安全

云原生供应链安全关注“镜像、依赖、构建和部署声明是否可信”：

- SBOM 记录镜像/包内依赖，方便漏洞影响面分析；
- 签名和 provenance 证明镜像来自可信构建流程；
- admission policy 在部署前检查镜像签名、基础镜像、capabilities、root 用户、hostPath 等；
- GitOps 环境下，集群实际状态应能回溯到 Git commit 和审计记录。

对 AI infra 来说，还要额外关注模型权重、adapter、prompt/skill 包、MCP server 包的来源。Agent 能动态加载技能或工具时，供应链边界不只在容器镜像，也在工具市场和技能市场。

Runtime Security

运行时安全解决“已经跑起来后发生了什么”：

- syscall / eBPF 监控用于发现异常进程、文件、网络行为；
- gVisor / Kata / microVM 提供更强隔离，适合运行不可信 Agent 代码；
- agent-sandbox 把 gVisor、Kata、NetworkPolicy 等隔离机制委托给 PodTemplate，让用户按风险选择运行时；
- [openkruise-agents](#) 在 sandbox lifecycle 上继续补 E2B API key storage、identity token propagation、dynamic CSI mount 和 Envoy route registry，因此要同时审计 API 身份、运行时 token、存储挂载和路由表 stale/fail-close 行为；
- Falco/Tetragon 这类工具适合做 runtime detection，但需要和 workload identity、namespace、owner reference 关联，才知道异常来自哪个 Agent 或任务。

Agent 特有风险

AI Agent 把云原生安全问题放大了：

- prompt injection 可能诱导 Agent 调用高危工具；
- MCP / skill 市场扩大了供应链和权限面；
- 长期运行的 Agent 有状态、有凭据、有文件系统，不能当作普通短任务；
- 观测日志可能包含 prompt、tool result、代码片段和 secret，需要 content policy 与 mask。

因此 Agent workload 推荐组合：agent-sandbox 做运行隔离，[agentgateway](#) / Gateway 做出口和凭据，loongsuite-pilot / trace 工具做审计与脱敏观测。

Secrets Store / Security Profiles / Agent Networking

[secrets-store-csi-driver](#) 把外部 secret store 通过 CSI volume 注入 Pod，适合把真凭据留在 Vault、云 secret manager 或专用 provider 中。[security-profiles-operator](#) 把 seccomp/AppArmor/SELinux profiles 变成可声明、可分发、可录制的 runtime security 对象。[kube-agentic-networking](#) 则把 Agent/tool 的网络访问策略纳入 Kubernetes governance。三者分别补凭据、syscall/LSM 和网络出口三条安全线。

用 HolmesGPT 自动诊断 Kubernetes 告警

摘要

STCLab 两人 SRE 团队用 HolmesGPT（CNCF Sandbox）+ ReAct 推理模式自动诊断 K8s 告警，核心发现：Runbook 比模型选择更重要。

关键数据

指标	改进前	改进后
日告警量	~40	~12（去重后）
单次排查时间	15-20 分钟	<2 分钟
无效工具调用	16 次/调查	2 次/调查
每次调查成本	—	~\$0.04

架构要点

- HolmesGPT ReAct 模式动态选择工具
- Markdown runbook 带元数据（可用工具、范围限制）
- Robusta 集成 + Slack 按 namespace 路由
- 未来：集成 Inspektor Gadget 的 ebpf 网络指标